C++ 程序设计

原理与实践

进阶篇

Bjarne Stroustrup 著

据原书第 2 版（Programming: Principles and Practice Using C++, Second Edition）译出

原生中文重排版

2026

版权说明

本书原名为 *Programming: Principles and Practice Using C++, Second Edition*，作者 Bjarne Stroustrup，由 Pearson Education（Addison-Wesley）出版，版权归原作者与出版者所有。本文件是基于原书内容制作的非官方中文重排版，仅供个人学习与研究使用，不得用于商业发行。

本篇为进阶篇，含原书第 12–16 章、第 20–27 章与附录 B、D、E。基础篇（第 0–11 章、第 17–19 章与附录 A、C）另成一卷。

目录

12 显示模型 7

12.1 为什么要图形？ 7
12.2 显示模型 8
12.3 第一个例子 9
12.4 使用 GUI 库 12
12.5 坐标 12
12.6 形状（Shape） 13
12.7 使用 Shape 基本要素 14
12.8 让程序运行起来 26

13 图形类 30

13.1 图形类概览 30
13.2 Point 与 Line 32
13.3 Lines 34
13.4 Color 37
13.5 Line_style 39
13.6 Open_polyline 42
13.7 Closed_polyline 43
13.8 Polygon 44
13.9 Rectangle 46
13.10 管理未命名对象 50
13.11 文本 52
13.12 圆 54
13.13 椭圆 56
13.14 带标记的折线 58
13.15 Marks 60
13.16 Mark 61
13.17 图像 63

14 图形类设计 68

14.1 设计原则 68
14.2 Shape 72
14.3 基类与派生类 80
14.4 面向对象编程的收益 87

15 函数与数据的绘制 91

15.1 引言 91
15.2 绘制简单函数 91

15.3 函数 . . . 96
15.4 坐标轴 . . . 100
15.5 近似 . . . 102
15.6 绘制数据 . . . 105

16 图形用户界面 116
16.1 用户界面的其他选择 . . . 116
16.2 “Next”按钮 . . . 117
16.3 一个简单的窗口 . . . 118
16.4 Button 及其他 Widget . . . 123
16.5 一个例子 . . . 126
16.6 控制反转 . . . 130
16.7 添加菜单 . . . 131
16.8 调试图形用户界面代码 . . . 134

20 容器与迭代器 139
20.1 存储与处理数据 . . . 140
20.2 STL 理念 . . . 143
20.3 序列与迭代器 . . . 146
20.4 链表 . . . 149
20.5 再次泛化 vector . . . 153
20.6 示例：一个简单的文本编辑器 . . . 157
20.7 vector、list 与 string . . . 162
20.8 让 vector 适配 STL . . . 165
20.9 让内建数组适配 STL . . . 167
20.10 容器概览 . . . 169

21 算法与映射 175
21.1 标准库算法 . . . 175
21.2 最简单的算法：find() . . . 176
21.3 通用搜索：find_if() . . . 179
21.4 函数对象 . . . 181
21.5 数值算法 . . . 185
21.6 关联容器 . . . 190
21.7 复制 . . . 201
21.8 排序与查找 . . . 205
21.9 容器算法 . . . 207

22 理想与历史 1
22.1 历史、理想与职业精神 . . . 1
22.2 编程语言史概述 . . . 9

23 文本处理 34
23.1 文本 . . . 34
23.2 字符串 . . . 35
23.3 I/O 流 . . . 38
23.4 映射 . . . 39
23.5 一个问题 . . . 45
23.6 正则表达式的概念 . . . 47
23.7 用正则表达式搜索 . . . 49
23.8 正则表达式语法 . . . 52
23.9 用正则表达式匹配 . . . 59
23.10 参考文献 . . . 64

24 数值计算 67
24.1 引言 . . . 67
24.2 大小、精度与溢出 . . . 68
24.3 数组 . . . 71
24.4 C 风格多维数组 . . . 72
24.5 Matrix 库 . . . 73
24.6 例子：解线性方程组 . . . 83
24.7 随机数 . . . 87
24.8 标准数学函数 . . . 90
24.9 复数 . . . 91
24.10 参考文献 . . . 93

25 嵌入式系统编程 97
25.1 嵌入式系统 . . . 97
25.2 基本概念 . . . 100
25.3 内存管理 . . . 103
25.4 地址、指针与数组 . . . 109
25.5 位、字节与字 . . . 118
25.6 编码标准 . . . 134
25.6.1 好的编码标准应该是什么样的？ . . . 135
25.6.2 规则示例 . . . 136

26 测试 144
26.1 我们想要什么 . . . 144
26.2 证明 . . . 145
26.3 测试 . . . 146
26.4 为测试而设计 . . . 161
26.5 调试 . . . 161
26.6 性能 . . . 161

26.7 参考文献 164

27 C 程序设计语言 167

27.1 C 与 C++：兄弟 168
27.2 函数 172
27.3 次要的语言差异 179
27.4 自由存储 185
27.5 C 风格字符串 186
27.6 输入输出：stdio 191
27.7 常量与宏 194
27.8 宏 195
27.9 例子：侵入式容器 198

附录 B 标准库概要 207

B.1 概述 209
B.2 错误处理 212
B.3 迭代器 214
B.4 容器 218
B.5 算法 224
B.5.1 非修改序列算法 225
B.5.2 修改序列算法 226
B.5.3 实用算法 229
B.5.4 排序与查找 229
B.5.5 集合算法 231
B.5.6 堆 232
B.5.7 排列 232
B.5.8 min 与 max 233
B.6 STL 工具 233
B.7 I/O 流 238
B.8 字符串操作 244
B.9 数值 247
B.9.6 随机数 252
B.10 时间 253
B.11 C 标准库函数 254
B.12 其他库 261

附录 D 安装 FLTK 263

D.1 引言 263
D.2 下载 FLTK 263
D.3 安装 FLTK 264
D.4 在 Visual Studio 中使用 FLTK 264

D.5 测试是否一切正常 264

附录 E 图形用户界面实现 266

E.1 回调的实现 266
E.2 `Widget` 的实现 267
E.3 `Window` 的实现 268
E.4 `Vector_ref` 269
E.5 一个例子：操纵 `Widget` 270

术语表 273

参考文献 278

12 显示模型

那时，世界还是黑白的。
直到 20 世纪 30 年代的某个时候，[它] 才变成彩色的。
——卡尔文的爸爸

本章介绍显示模型（display model，即 GUI 的输出部分），给出使用示例和若干基本概念，如屏幕坐标、直线和颜色。`Line`、`Lines`、`Polygon`、`Axis` 和 `Text` 都是 `Shape`（形状）的例子。Shape 是内存中的一种对象，我们可以在屏幕上显示它并对其进行操作。接下来两章将进一步研究这些类：第 13 章着重讨论它们的实现，第 14 章则讨论相关的设计问题。

12.1 为什么要图形？
12.2 显示模型
12.3 第一个例子
12.4 使用 GUI 库
12.5 坐标
12.6 形状（`Shape`）
12.7 使用 Shape 基本要素
12.7.1 图形头文件与 main
12.7.2 一个几乎空白的窗口
12.7.3 坐标轴（`Axis`）
12.7.4 绘制函数图形
12.7.5 多边形（`Polygon`）
12.7.6 矩形（`Rectangle`）
12.8 让程序运行起来

12.1 为什么要图形？

为什么我们要花四章讲图形，再花一章讲 GUI（图形用户界面，graphical user interface）？毕竟，这是一本关于程序设计的书，而不是一本图形学专著。还有大量有趣的主题我们并未涉及，而对图形这个话题，我们至多只能浅尝辄止。那么，“为什么要图形？”从根本上说，图形让我们能够探索软件设计、程序设计和程序设计语言设施中的若干重要领域：

- *图形是有用的（useful）*。程序设计中远不止图形这一项内容，软件也远不止通过 GUI 操纵的代码。然而在许多领域，好的图形要么必不可少，要么非常重要。例如，如果不具备绘制数据图的能力，我们简直无法想象如何研究科学计算、数据分析或任何定量学科。第 15 章将给出绘制数据图的简单（但通用）设施。
- *图形是有趣的（fun）*。很少有哪个计算领域，一段代码的效果能像图形这样立竿见影，而且在最终排除错误之后又是如此令人愉悦。即使图形没有用处，我们也会忍不住想玩玩它！
- *图形提供了大量值得阅读的有趣代码*。学习编程的一部分就是阅读大量代码，以体会好代码是什么样子。同样，要成为优秀的英文作家，也要大量阅读书籍、文章和高质量报纸。由于屏幕上所见与程序中所写直接对应，简单的图形代码比复杂度相近的大多数代码更具可读性。本章将证明，只需几分钟的介绍你就能读懂图形代码；第 13 章将展示，再过几个小时你就能编写它。
- *图形是设计范例的丰富来源*。设计并实现一个好的图形与 GUI 库实际上相当困难。图形为设计决策和设计技术提供了非常丰富的具体而实用的示例。设计类、设计函数、将软件分层（按抽象层次）以及构建库等许多最有用的技术，都可以用相对少量的图形与 GUI 代码来阐释。

- 图形是通向面向对象程序设计及其语言支持的良好入门。尽管传言并非如此，面向对象程序设计的发明并不是为了做图形（见第 22 章），但它很快就被应用到图形上，而图形也提供了面向对象设计中最易上手的一些例子。
- 一些关键的图形概念并不平凡。因此值得专门讲授，而不是任由你自己主动（并有耐心地）去查找资料。如果我们不展示图形和 GUI 是如何实现的，你可能会把它们当作“魔法”，这就违背了本书的基本宗旨之一。

12.2 显示模型

iostream 库面向的是读写字符流，就像它们出现在数值列表或书本中的样子。它对“图形位置”概念的唯一直接支持是换行符和制表符。你可以在一维字符流中嵌入颜色和二维位置等概念——这正是版面（排版、“标记”）语言如 Troff、TEX、Word、HTML 和 XML（及其相关图形包）所做的事情。例如：

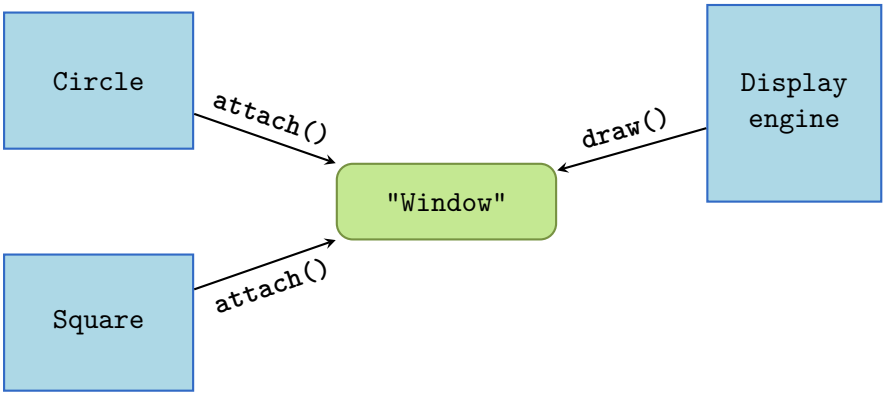
```
<hr>
<h2>
Organization
</h2>
This list is organized in three parts:
<ul>
          <li><b>Proposals</b>, numbered EPddd, . . .</li>
          <li><b>Issues</b>, numbered EIddd, . . .</li>
          <li><b>Suggestions</b>, numbered ESddd, . . .</li>
</ul>
<p>We try to . . .
<p>
```

这是一段 HTML，描述了一个标题（<h2> . . . </h2>）、一个列表（ . . . ，其中含列表项 <li> . . . ）和一个段落（<p>）。我们省略了大部分实际文本，因为它与此处无关。关键在于：版面概念可以用纯文本来表达，但所写下的字符与屏幕上显示的内容之间的联系是间接的，由一个解释这些“标记”命令的程序来掌控。这类技术从根本上说很简单，而且用途极广（你读到的东西几乎都是用它产生的），但它们也有自己的局限性。

在本章以及接下来的四章中，我们给出另一种方案：一种直接面向计算机屏幕的图形与图形用户界面概念。这些基本概念天然是图形化的（而且是二维的，以适配计算机屏幕的矩形区域），例如坐标、直线、矩形和圆。从程序设计的角度看，其目标是让内存中的对象与屏幕上的图像直接对应。

基本模型如下：我们用图形系统提供的基本对象（如直线）组装出对象，再将这些图形对象“附着（attach）”到一个表示物理屏幕的窗口对象上。一个可以将其视为显示器本身、“显示引擎”、“我们的图形库”、“GUI 库”，甚至（开玩笑地）“在屏幕背面写字的小矮人”的程序，会取出附着到窗口上的对象并把它们绘制到屏幕上：

“显示引擎”在屏幕上画线、放置文本字符串、给屏幕区域着色等等。为简单起见，我们将用“我们的 GUI 库”甚至“系统”来称呼这个显示引擎，尽管我们的 GUI 库所做的远不止绘制这些对象。正如我们的代码让 GUI 库替我们完成大部分工作一样，GUI 库也把它的大部分工作委托给操作系统。



12.3 第一个例子

我们的任务就是定义一些类，从这些类可以创建出我们想在屏幕上看到的对象。例如，我们可能想把一幅图画成一系列相连的直线。下面是一个小程序，给出了它的一个非常简单的版本：

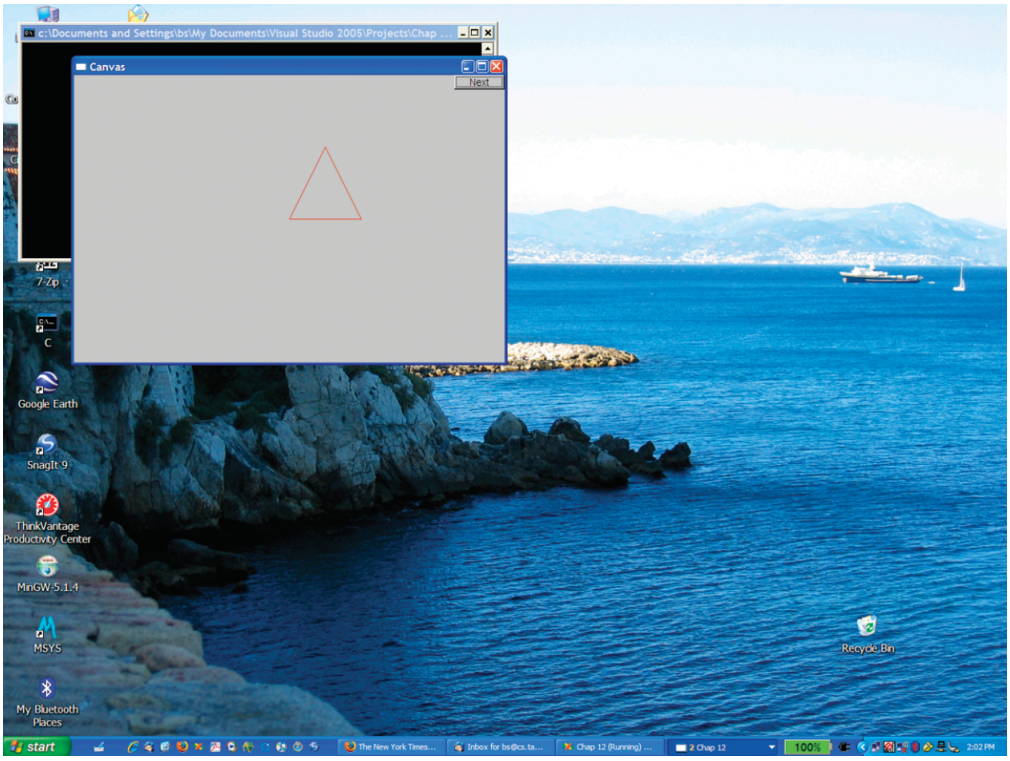
```
#include "Simple_window.h"    // 获得我们的窗口库
#include "Graph.h"            // 获得我们的图形库设施

int main()
{
    using namespace Graph_lib;  // 我们的图形设施位于 Graph_lib 中

    Point tl {100,100};         // 用作窗口左上角

    Simple_window win {tl,600,400,"Canvas"};    // 创建一个简单窗口

    Polygon poly;               // 创建一个形状（多边形）

    poly.add(Point{300,200});   // 添加一个点
    poly.add(Point{350,100});   // 再添加一个点
    poly.add(Point{400,200});   // 添加第三个点

    poly.set_color(Color::red); // 调整 poly 的属性

    win.attach (poly);          // 将 poly 连接到窗口

    win.wait_for_button();      // 把控制权交给显示引擎
}
```

运行这个程序时，屏幕看上去大致如下：

下面我们逐行分析这个程序，看看都做了些什么。首先包含图形接口库的头文件：

```
#include "Simple_window.h"    // 获得我们的窗口库
#include "Graph.h"            // 获得我们的图形库设施
```

然后，在 main() 中，我们首先告诉编译器：我们的图形设施位于 Graph_lib 中：




```
using namespace Graph_lib;     // 我们的图形设施位于 Graph_lib 中
```

接着，我们定义一个点，将其用作窗口的左上角：

```
Point tl {100,100};            // 用作窗口左上角
```

接下来，我们在屏幕上创建一个窗口：

```
Simple_window win {tl,600,400,"Canvas"};     // 创建一个简单窗口
```

我们使用的是接口库 Graph_lib 中表示窗口的类 **Simple_window**。这个 Simple_window 对象名为 **win**；也就是说，win 是 Simple_window 类的一个变量。win 的初始化列表以用作左上角的点 tl 开头，随后是 600 和 400。它们分别是窗口在屏幕上显示时的宽度和高度，以像素为单位。稍后我们会更详细地解释，但这里的要点是：我们通过给出宽度和高度来指定一个矩形。字符串 Canvas 用于标记窗口。只要留意看，你就能在窗口边框的左上角看到 Canvas 一词。

接下来，我们向窗口中放入一个对象：

```
Polygon poly;                  // 创建一个形状（多边形）

poly.add(Point{300,200});      // 添加一个点
poly.add(Point{350,100});      // 再添加一个点
poly.add(Point{400,200});      // 添加第三个点
```

我们定义了一个多边形 poly，然后向它添加点。在我们的图形库中，Polygon 初始为空，我们可以随意向它添加任意多个点。由于添加了三个点，我们得到一个三角形。点就是一对值，给出窗口内的 x 和 y（水平和垂直）坐标。

为了炫耀一下，我们随后把多边形的线染成红色：

```
poly.set_color(Color::red);    // 调整 poly 的属性
```

最后，我们把 poly 附着到窗口 win 上：

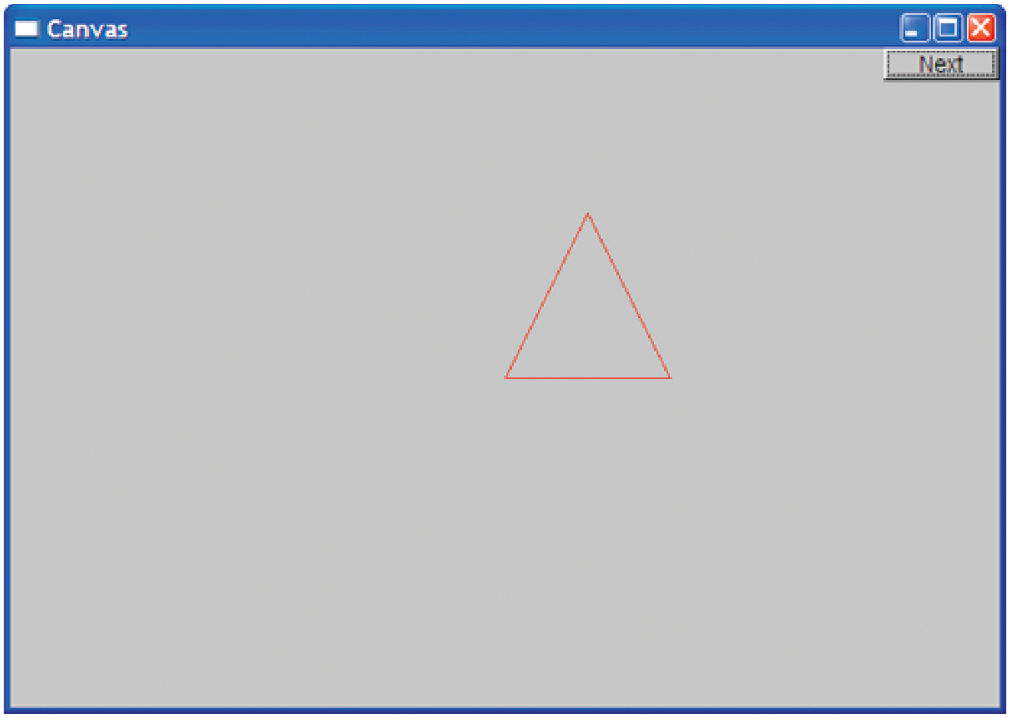
```
win.attach(poly);              // 将 poly 连接到窗口
```

如果这个程序不是运行得这么快，你就会注意到：到目前为止屏幕上什么都没有发生——什么都没有。我们创建了一个窗口（准确地说是 Simple_window 类的对象）、创建了一个多边形（名为 poly）、把它染成红色（Color::red），并把它附着到窗口（名为 win）上，但还没有请求把这个窗口显示到屏幕上。这由程序的最后一行完成：

```
win.wait_for_button();         // 把控制权交给显示引擎
```

要让 GUI 系统在屏幕上显示对象，你必须把控制权交给“系统”。我们的 wait_for_button() 正是做这件事的，而且它还会等待你“按下”（“点击”）Simple_window 的“Next”按钮之后才继续。这给了你在程序结束、窗口消失之前端详窗口的机会。按下按钮后，程序终止，窗口随之关闭。

单独看，我们的窗口是这个样子的：



你会注意到我们“作弊”了一点。那个标着“Next”的按钮是从哪儿来的？我们把它内建进了 Simple_window 类。在第 16 章中，我们将从 Simple_window 转向“纯粹的”Window，它没有这些可能带来干扰的内建设施，并展示如何编写自己的代码来控制与窗口的交互。

在接下来的三章中，当我们想分阶段（“一帧一帧地”）显示信息时，就直接用这个“Next”按钮从一个“显示”切换到下一个。

操作系统给每个窗口都加了边框，你对此太习惯了，以至于可能根本没有特别留意它。不过，本章及随后各章中的图片是在 Microsoft Windows 系统上生成的，因此你“免费”得到了右上角通常的三个按钮。这可能很有用：如果你的程序陷入真正的混乱（调试时肯定难免），你可以点击 X 按钮将其终止。在另一个系统上运行程序时，系统会按其惯例加上不同的边框。我们对边框的唯一贡献是那个标签（此处为 Canvas）。

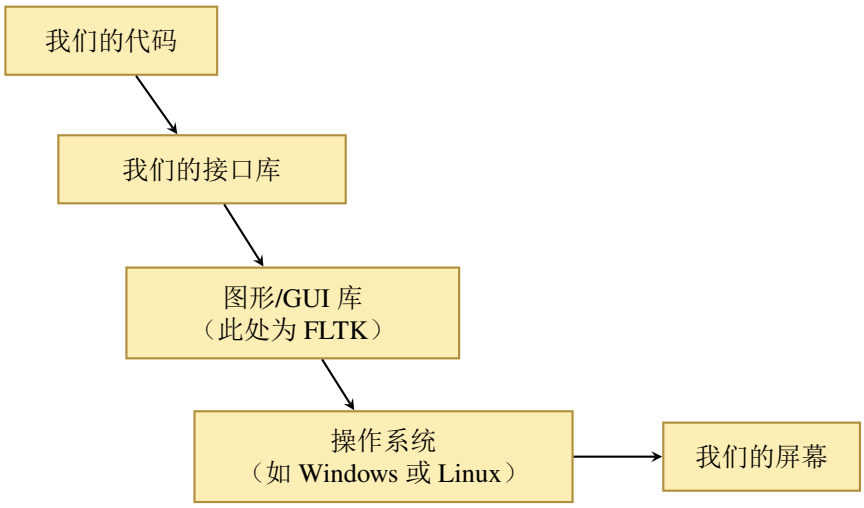
12.4 使用 GUI 库

在本书中，我们不会直接使用操作系统的图形与 GUI（图形用户界面）设施。那样做会把程序限制在单一操作系统上，还会迫使我们直接面对大量杂乱的细节。与文本 I/O 一样，我们将用一个库来抹平操作系统差异、I/O 设备差异等，并简化代码。遗憾的是，C++ 并不像提供标准流 I/O 库那样提供标准 GUI 库，所以我们只能使用众多可用的 C++ GUI 库之一。为了不把你直接绑死在某个 GUI 库上，也为了免让你一下子面对 GUI 库的全部复杂性，我们使用了一组简单的接口类，它们只需几百行代码就能在几乎任何 GUI 库上实现。

我们（目前是间接地）使用的 GUI 工具包叫做 FLTK（Fast Light Tool Kit，发音为“full tick”），来自 www.fltk.org。凡是使用 FLTK 的地方（Windows、Unix、Mac、Linux 等），我们的代码都可移植。我们的接口类也可以用其他工具包重新实现，因此使用它们的代码潜在地更具可移植性。

我们的接口类呈现的程序设计模型比常见工具包提供的要简单得多。例如，我们完整的图形与 GUI 接口库大约 600 行 C++ 代码，而极为精炼的 FLTK 文档就有 370 页。你可以从 www.fltk.org 下载它，但我们不建议你现在就这么做。一段时间内你用不着那种程度的细节。第 12–16 章介绍的一般思想可以配合任何流行的 GUI 工具包使用。当然，我们会解释接口类如何映射到 FLTK，这样你（最终）就能看到，必要时如何直接使用它（以及类似的工具包）。

我们可以像下面这样图示“图形世界”的各个部分：

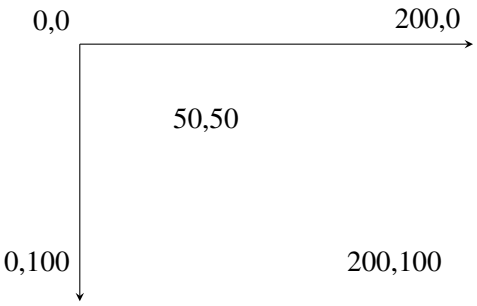


我们的接口类提供了简单且用户可扩展的二维形状基本概念，并对颜色的使用提供有限支持。为驱动这些形状，我们再呈现一个简单的 GUI 概念，它基于用户点击屏幕上自定义按钮等操作触发的“回调（callback）”函数（第 16 章）。

12.5 坐标

计算机屏幕是由像素（pixel）组成的矩形区域。像素是能被赋予某种颜色的微小点。在程序中为屏幕建模最常用的方式，是把它看作一个像素矩形。每个像素由一个 x（水平）坐标和一个 y（垂直）坐标标识。x 坐标从 0 开始，表示最左边的像素，向右递增直到最右边的像素；y 坐标从 0 开始，表示最顶部的像素，向下递增直到最底部的像素：

请注意，y 坐标是“向下增长”的。数学家尤其会觉得这很奇怪，但屏幕（和窗口）尺寸千差万别，左上角是它们唯一的共同点。



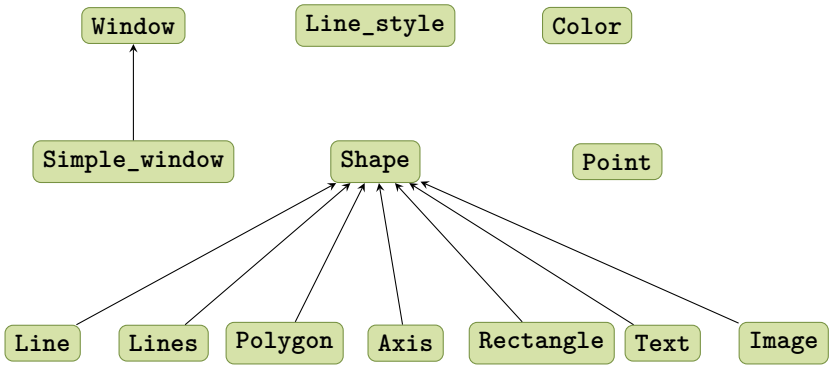
可用像素的数量取决于屏幕：1024×768、1280×1024、1400×1050 和 1600×1200 都是常见的屏幕尺寸。在通过屏幕与计算机交互的语境下，窗口是屏幕上专用于某种特定目的、由程序控制的一个矩形区域。窗口的寻址方式与屏幕完全相同。基本上，我们把窗口看作一个小屏幕。例如，当我们说

```
Simple_window win {tl,600,400,"Canvas"};
```

我们请求的是一个 600 像素宽、400 像素高的矩形区域，寻址范围为 0-599（从左到右）和 0-399（从上到下）。窗口中可用来绘图的区域通常称为画布（canvas）。600×400 指的是窗口“内部”的区域，即系统提供的边框以内的区域；它不包括系统用于标题栏、关闭按钮等的空间。

12.6 形状（Shape）

我们在屏幕上绘图的基本工具箱由大约十几个类组成：



箭头表示：指向方的类可以用在被指向方类所需之处。例如，Polygon 可以用在需要 Shape 的地方；也就是说，Polygon 是一种 Shape。

我们首先介绍并使用下列类：

- Simple_window、Window
- Shape、Text、Polygon、Line、Lines、Rectangle、Function 等
- Color、Line_style、Point
- Axis

随后（第 16 章），我们将添加 GUI（用户交互）类：

- Button、In_box、Menu 等

我们本可以轻松地添加更多类（对于“轻松”的某种定义而言），例如：

- Spline、Grid、Block_chart、Pie_chart 等

然而，定义或描述一个具备全部设施的完整 GUI 框架超出了本书的范围。

12.7 使用 Shape 基本要素

在本节中，我们将带你了解图形库的一些基本设施：Simple_window、Window、Shape、Text、Polygon、Line、Lines、Rectangle、Color、Line_style、Point、Axis。目的是让你对这些设施能做什么有一个总体的了解，但还不会详细理解其中任何一个类。在接下来的章节中，我们将逐一探究它们的设计。

现在我们将走查一个简单的程序，逐行解释代码，并展示每一行在屏幕上的效果。当你运行这个程序时，会看到随着我们向窗口添加形状、修改已有形状，图像是如何变化的。基本上，我们是在通过观察程序执行来“动画式地”展示代码的推进过程。

12.7.1 图形头文件与 main

首先，包含定义图形与 GUI 设施接口的头文件：

```
#include "Window.h"          // 普通窗口
#include "Graph.h"
```

或者

```
#include "Simple_window.h"   // 如果我们想要那个“Next”按钮
#include "Graph.h"
```

你可能已经猜到，Window.h 包含与窗口相关的设施，Graph.h 包含向窗口绘制形状（包括文本）的相关设施。这些设施定义在 Graph_lib 命名空间中。为简化书写，我们使用一条 namespace 指令，让 Graph_lib 中的名字在我们的程序中直接可用：

```
using namespace Graph_lib;
```

和往常一样，main() 包含我们想要执行的代码（直接或间接地），并处理异常：

```
int main ()
try
{
    // . . . 这里是我们的代码 . . .
}
catch(exception& e) {
    // 某种错误报告
    return 1;
}
catch(...) {
    // 更多错误报告
    return 2;
}
```

要让这个 main() 编译通过，我们需要定义了 exception。像往常一样包含 std_lib_facilities.h 即可获得它，或者我们也可以开始直接使用标准头文件，包含 <stdexcept>。

12.7.2 一个几乎空白的窗口

本节不讨论错误处理（见第 5 章，特别是 §5.6.3），而是直接进入 main() 中的图形部分：

```
Point tl {100,100};          // 我们窗口的左上角

Simple_window win {tl,600,400,"Canvas"};
    // 屏幕坐标 tl 作为左上角
    // 窗口尺寸 (600*400)
    // 标题: Canvas
win.wait_for_button();       // 显示!
```

这会创建一个 Simple_window——也就是一个带“Next”按钮的窗口——并把它显示在屏幕上。显然，要想使用 Simple_window，我们必须 #include 头文件 Simple_window.h 而不是 Window.h。这里我们明确指定了窗口应出现在屏幕上的位置：它的左上角位于 Point{100,100}。这个位置靠近屏幕左上角，但不会太贴近。

显然，Point 是一个类，它的构造函数接受一对整数，并将其解释为 (x, y) 坐标对。我们本可以写成

```
Simple_window win {Point{100,100},600,400,"Canvas"};
```

然而，我们想多次使用点 (100,100)，所以给它一个符号名更方便。600 是窗口的宽度，400 是窗口的高度，而 Canvas 则是我们希望放在窗口边框上的标签。

要真正把窗口绘制到屏幕上，必须把控制权交给 GUI 系统。我们通过调用 win.wait_for_button() 来做到这一点，结果是：

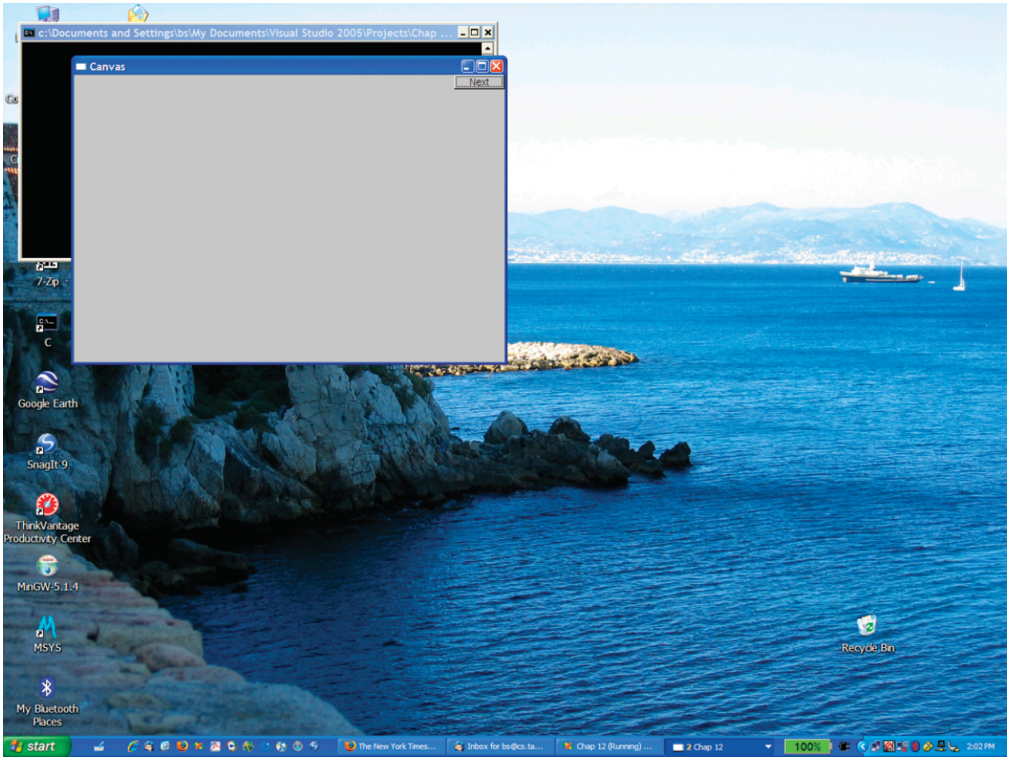
在窗口的背景中，我们看到一台笔记本电脑的屏幕（为了拍照特意清理过）。对于对无关细节感到好奇的人，我们可以告诉你：这张照片是我站在昂蒂布毕加索图书馆附近、隔着海湾眺望尼斯时拍的。部分隐藏在后面的黑色控制台窗口正是运行我们程序的那个窗口。有个控制台窗口虽然有些难看也没必要，但它有一个好处：当调试到一半的程序陷入死循环、赖着不走时，它给了我们一个杀掉窗口的有效手段。如果仔细看，你会注意到我们运行的是 Microsoft C++ 编译器，但你同样可以用其他编译器（如 Borland 或 GNU）。

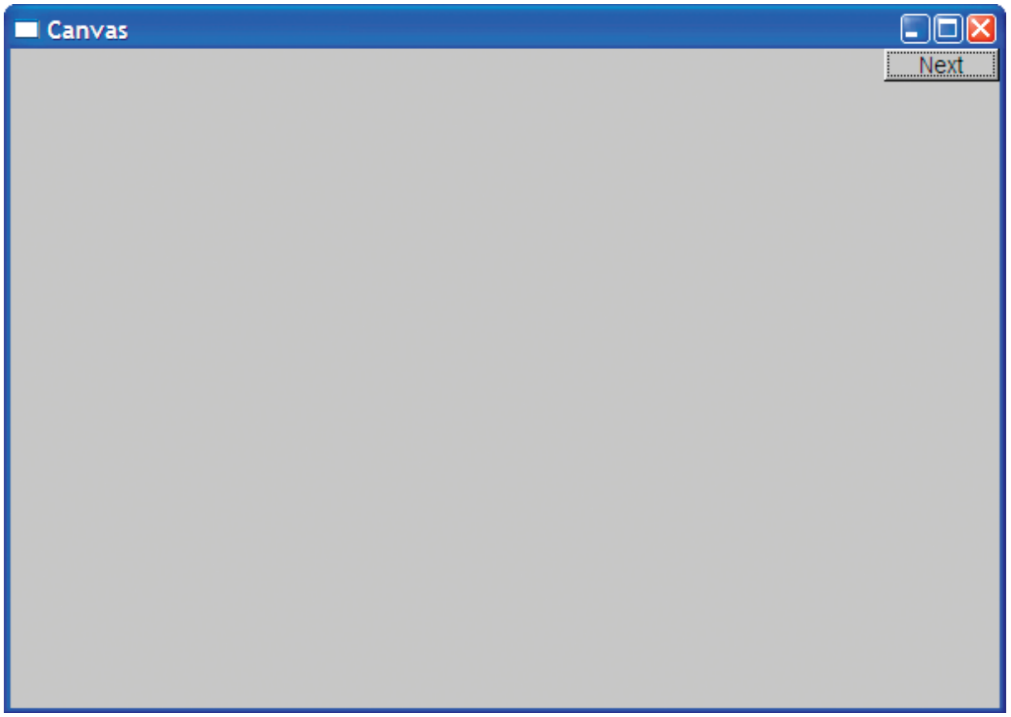
在接下来的讲解中，我们将去掉窗口周围的干扰，只显示窗口本身：

窗口的实际尺寸（以英寸计）取决于屏幕的分辨率。有些屏幕的像素比其他屏幕的大。

12.7.3 坐标轴（Axis）

一个几乎空白的窗口没什么意思，所以最好添加些信息。我们想显示什么呢？为了提醒你图形并非全是好玩的东西，我们从一个严肃且有点复杂的东西开始：坐标轴。没有坐标轴的图通常是不合格的：没有坐标轴，你就不知道数据代表什么。也许你会在随附的说明文字里解释一切，但加上坐标轴要保险得多；人们常常不读说明文字，而且一幅漂亮的图形常常会脱离它原来的上下文被单独传播。所以，图需要坐标轴：



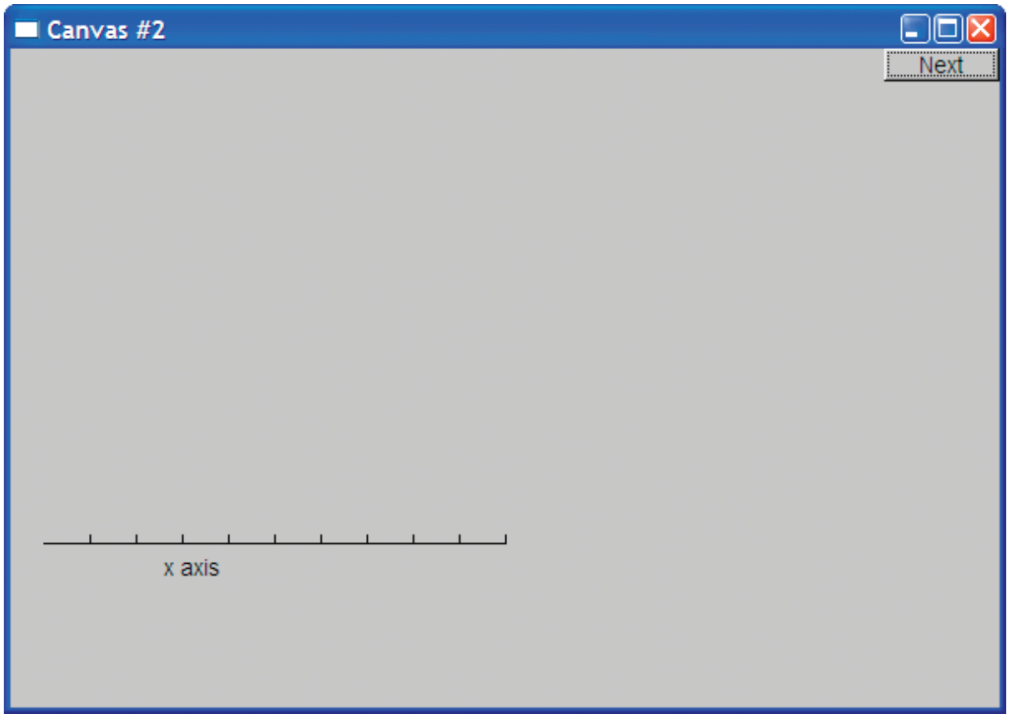



```
Axis xa {Axis::x, Point{20,300}, 280, 10, "x axis"};    // 创建一个 Axis
                                    // Axis 是一种 Shape
                                    // Axis::x 表示水平方向
                                    // 从 (20,300) 开始
                                    // 长 280 像素
                                    // 10 个“刻度”
                                    // 标注坐标轴为 "x axis"
```

```
win.attach(xa);                 // 将 xa 附着到窗口 win
win.set_label("Canvas #2");     // 重新标注窗口
win.wait_for_button();          // 显示!
```

这一串动作的顺序是：创建坐标轴对象、把它加入窗口，最后显示：

可以看到 `Axis::x` 是一条水平线。我们看到所需数量（10 个）的“刻度”以及标签“x axis”。通常，标签会解释坐标轴和刻度代表什么。自然，我们选择把 x 轴放置在窗口底部附近。在实际编程中，我们会用符号常量来表示高度和宽度，这样就可以用 `y_max-bottom_margin` 之类的写法来指称“略高于底部”，而不是用 300 这样的“魔法常数”（§4.3.1、§15.6.2）。



为了帮助识别我们的输出，我们用 `Window` 的成员函数 `set_label()` 把屏幕重新标注为 Canvas #2。

现在，添加一条 y 轴：

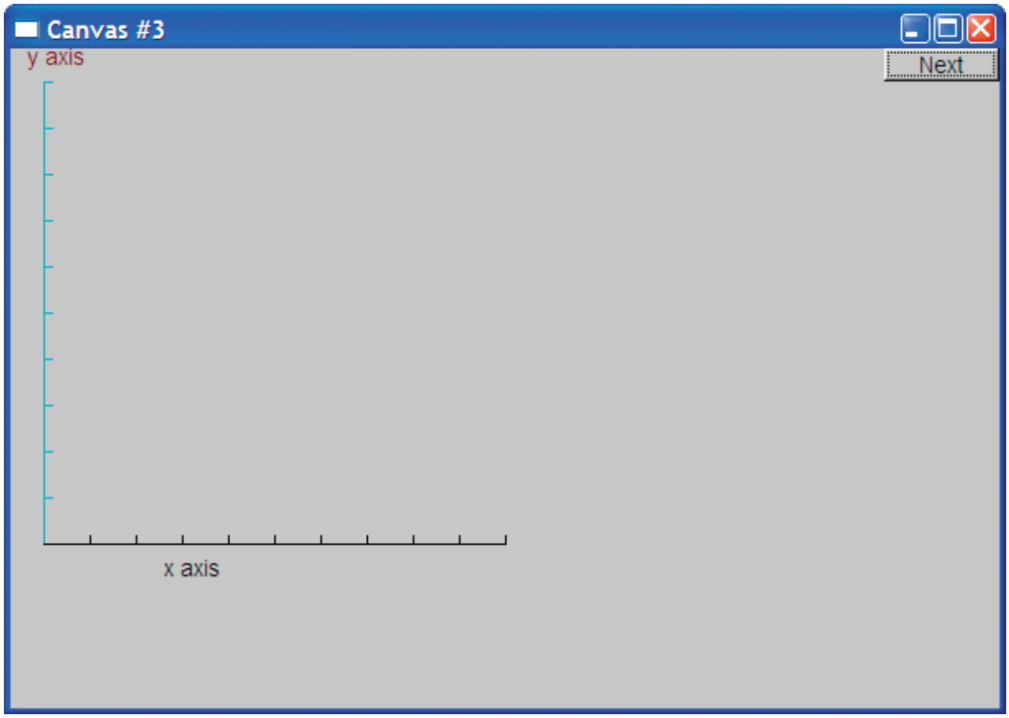
```
Axis ya {Axis::y, Point{20,300}, 280, 10, "y axis"};
ya.set_color(Color::cyan);                // 选择一种颜色
ya.label.set_color(Color::dark_red);      // 为文字选择一种颜色
win.attach(ya);
win.set_label("Canvas #3");
win.wait_for_button();                    // 显示!
```

为了展示一些设施，我们把 y 轴染成了青色，把标签染成了暗红色。

我们其实并不认为给 x 轴和 y 轴用不同颜色是个好主意。我们只是想向你展示如何设置形状的颜色以及形状中个别元素的颜色。大量用色未必是好事。特别是，新手往往用色热情有余而品味不足。

12.7.4　绘制函数图形

接下来做什么？我们现在有了一个带坐标轴的窗口，那么画一个函数的图形似乎是个好主意。我们创建一个表示正弦函数的形状并附着它：




```
Function sine {sin,0,100,Point{20,150},1000,50,50};   // 正弦曲线
        // 在区间 [0:100) 内绘制 sin(), (0,0) 位于 (20,150)
        // 使用 1000 个点; x 值比例 *50, y 值比例 *50

win.attach(sine);
win.set_label("Canvas #4");
win.wait_for_button();
```

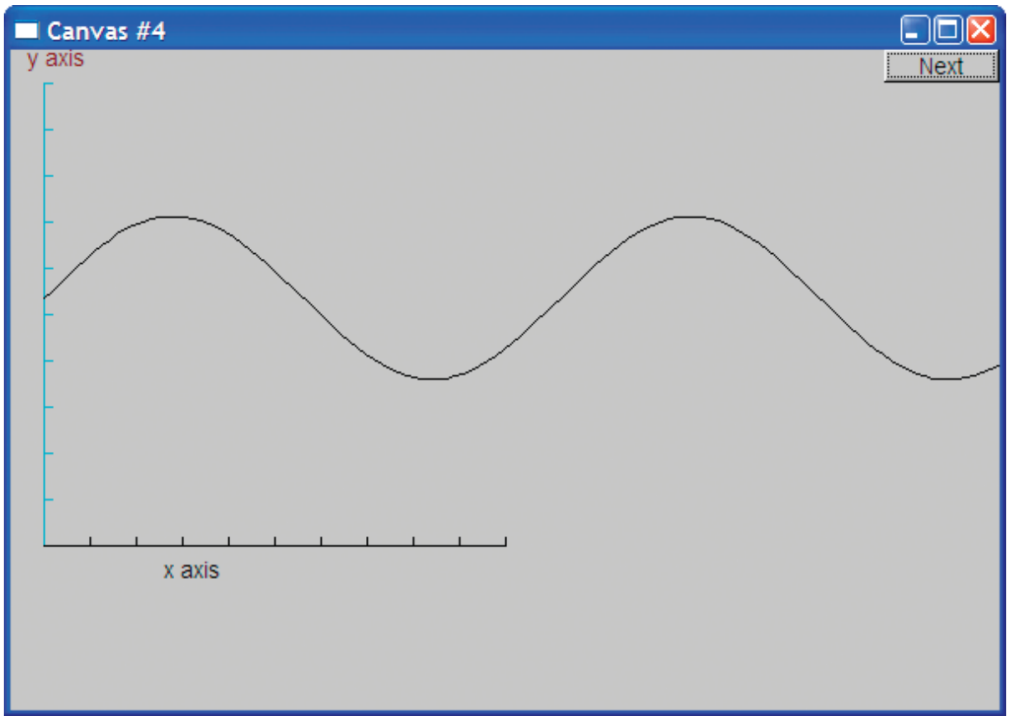
这里，名为 sine 的 Function 将使用标准库函数 sin() 生成值来绘制正弦曲线。关于如何绘制函数图形的细节见 §15.3。眼下只需注意：要绘制函数图形，我们必须说明它从哪里开始（一个 Point）、想看到它针对哪组输入值（一个范围），并且要给出如何把这些信息压缩进窗口的一些信息（比例缩放）：

注意，曲线碰到窗口边缘就简单地停住了。绘制在窗口矩形之外的点会被 GUI 系统直接忽略，永远看不到。

12.7.5 多边形（Polygon）

绘制出的函数图形是数据呈现的一个例子。第 15 章会看到更多此类内容。不过，我们还可以在窗口中绘制其他类型的对象：几何形状。我们用几何形状做图形示意、指明用户交互元素（如按钮），总之让我们的呈现更有趣。Polygon 的特征是一串点，Polygon 类用直线把它们连接起来：第一条线连接第一个点和第二个点，第二条线连接第二个点和第三个点，最后一条线连接最后一个点和第一个点：

```
sine.set_color(Color::blue);  // 我们改变了对 sine 颜色的主意
```



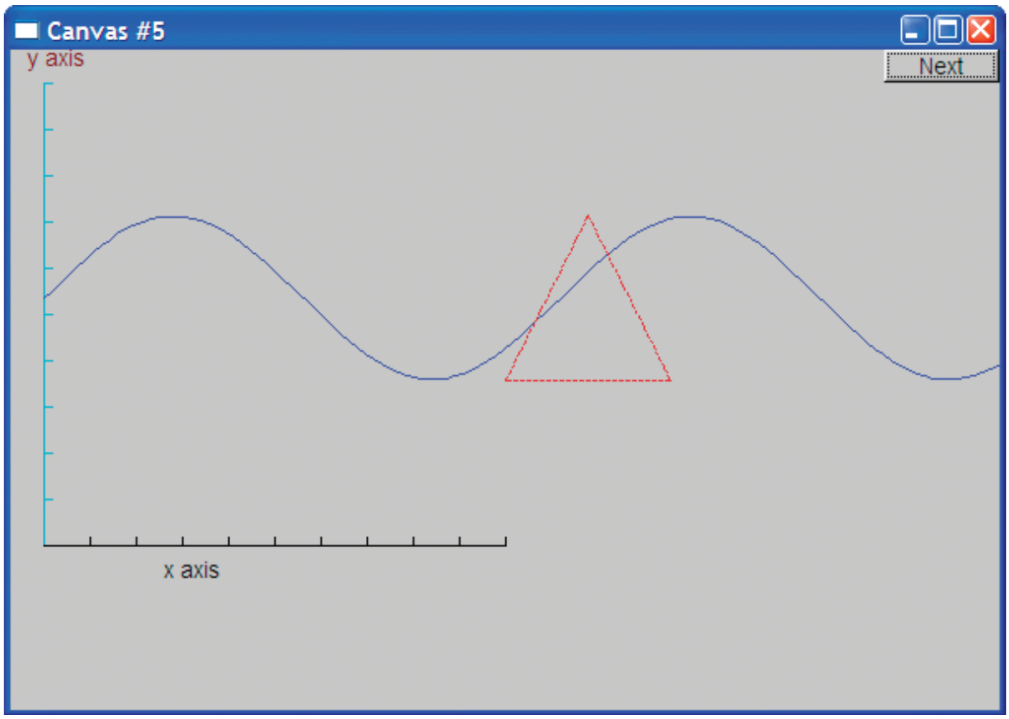

```
Polygon poly;                   // 一个多边形；Polygon 是一种 Shape
poly.add(Point{300,200});       // 三个点构成一个三角形
poly.add(Point{350,100});
poly.add(Point{400,200});


poly.set_color(Color::red);
poly.set_style(Line_style::dash);
win.attach(poly);


win.set_label("Canvas #5");
win.wait_for_button();
```

这一次我们改变了正弦曲线（sine）的颜色，只是展示如何做。然后，我们添加一个三角形——正如 §12.3 第一个例子中的那样——作为多边形的一个例子。再次设置一个颜色，最后设置一个样式。Polygon 的线有一种“样式”。默认是实线，但必要时也可以把它们改成虚线、点线等（见 §13.5）。我们得到

12.7.6 矩形（Rectangle）

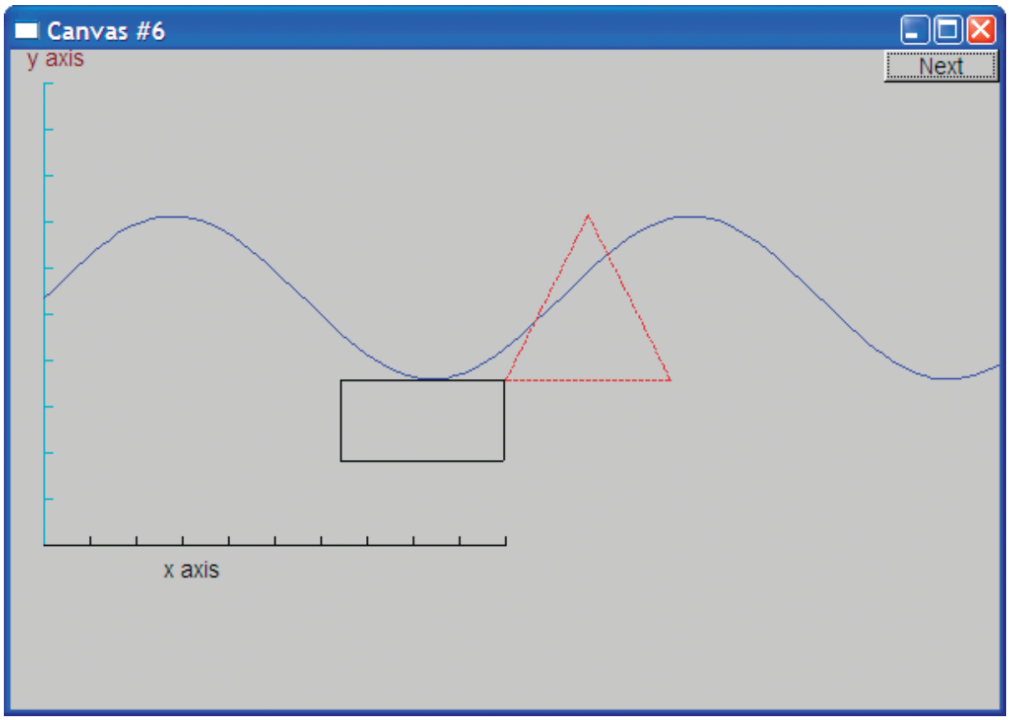
屏幕是矩形，窗口是矩形，一张纸也是矩形。事实上，现代世界中大量的形状都是矩形（至少是圆角矩形）。这是有原因的：矩形是最容易处理的形状。例如，它很容易描述（左上角加宽加高，或者左上角加右下角，等等）；很容易判断一个点是在矩形内还是矩形外；也很容易让硬件快速绘制一个像素矩形。

因此，大多数更高层的图形库对矩形的处理都比其他封闭形状更好。于是，我们提供了 Rectangle 作为独立于 Polygon 类的一个类。Rectangle 的特征是它的左上角加上宽度和高度：




```
Rectangle r {Point{200,200}, 100, 50};    // 左上角、宽度、高度
win.attach(r);
win.set_label("Canvas #6");
win.wait_for_button();
```

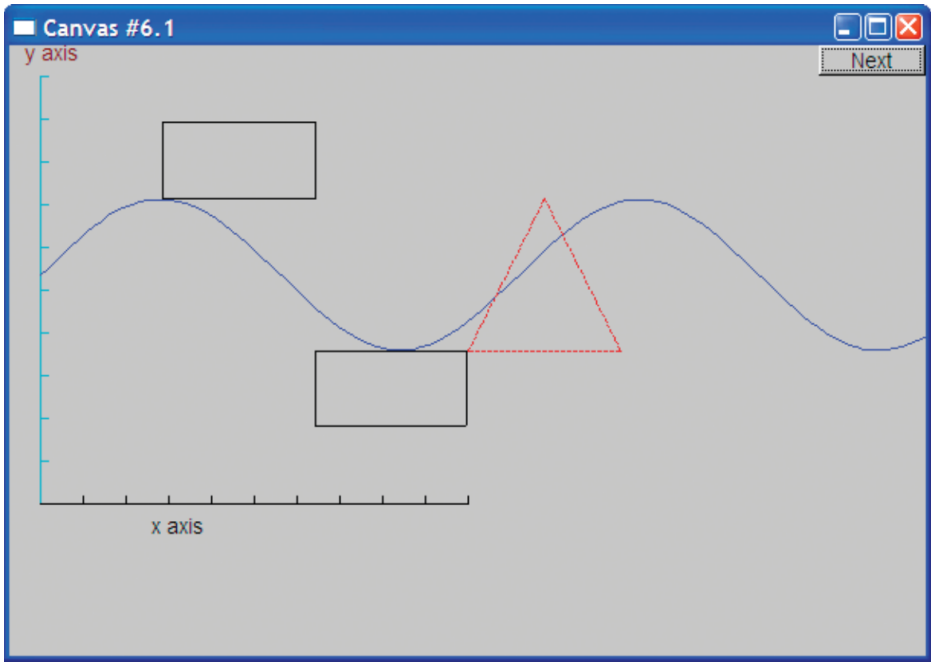
由此我们得到



请注意，在合适的位置画一个有四个点的折线并不足以构成 Rectangle。我们很容易创建一个在屏幕上看起来像 Rectangle 的 Closed_polyline（甚至可以创建一个看起来很像 Rectangle 的 Open_polyline）；例如：

```
Closed_polyline poly_rect;
poly_rect.add(Point{100,50});
poly_rect.add(Point{200,50});
poly_rect.add(Point{200,100});
poly_rect.add(Point{100,100});
win.attach(poly_rect);
```

事实上，这样的 poly_rect 在屏幕上的图像是一个矩形。然而，内存中的 poly_rect 对象并不是 Rectangle，它对矩形一无所“知”。证明这一点最简单的方法是再添加一个点：




```
poly_rect.add(Point{50,75});
```

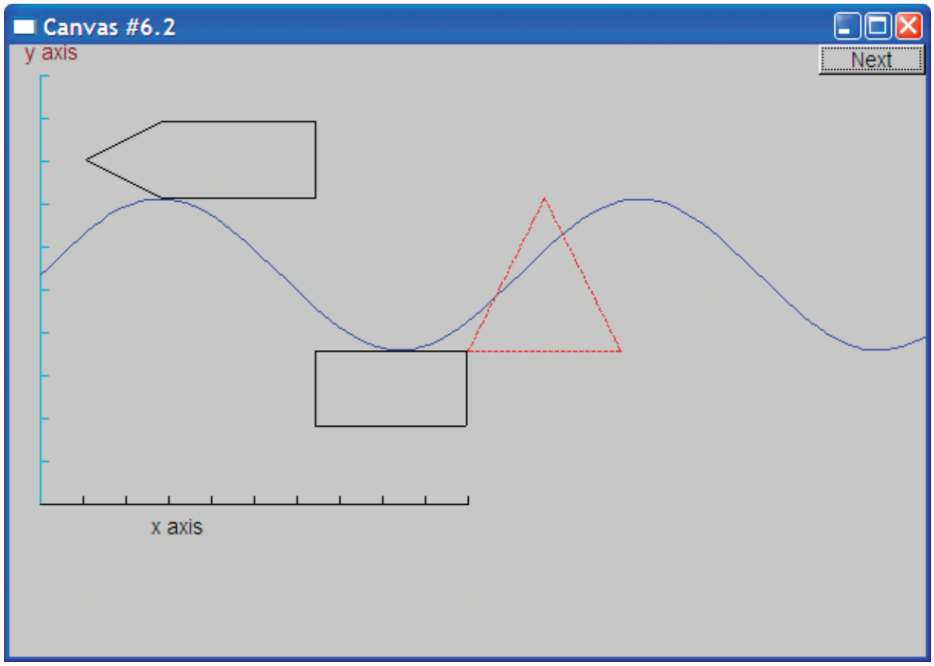
没有矩形有五个点：

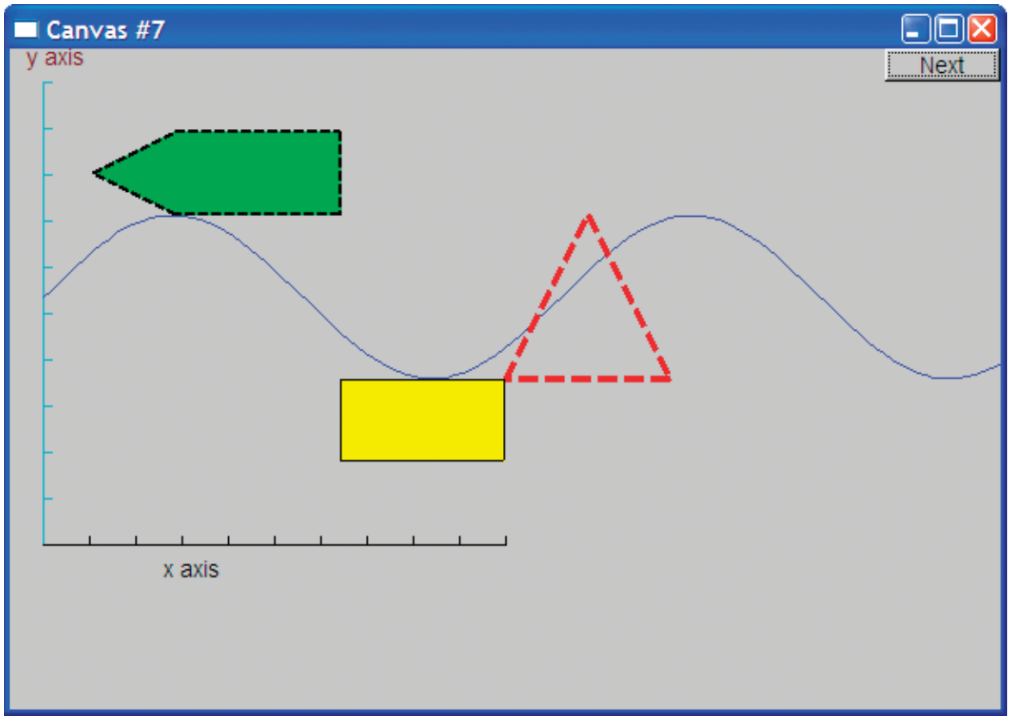
对我们关于代码的推理而言，很重要的是：Rectangle 不只是碰巧在屏幕上看起来像矩形；它维护着一些基本保证

矩形（正如我们在几何中所知的那样）也不例外。我们编写的代码依赖于这样一个事实：屏幕上的 Rectangle 确实是一个矩形，并且会一直保持如此。

12.7.7 填充

我们一直把图形画成轮廓线。我们也可以用颜色**填充**（fill）一个矩形：






```
r.set_fill_color(Color::yellow);     // 将矩形内部着色
poly.set_style(Line_style(Line_style::dash,4));
poly_rect.set_style(Line_style(Line_style::dash,2));
poly_rect.set_fill_color(Color::green);
win.set_label("Canvas #7");
win.wait_for_button();
```

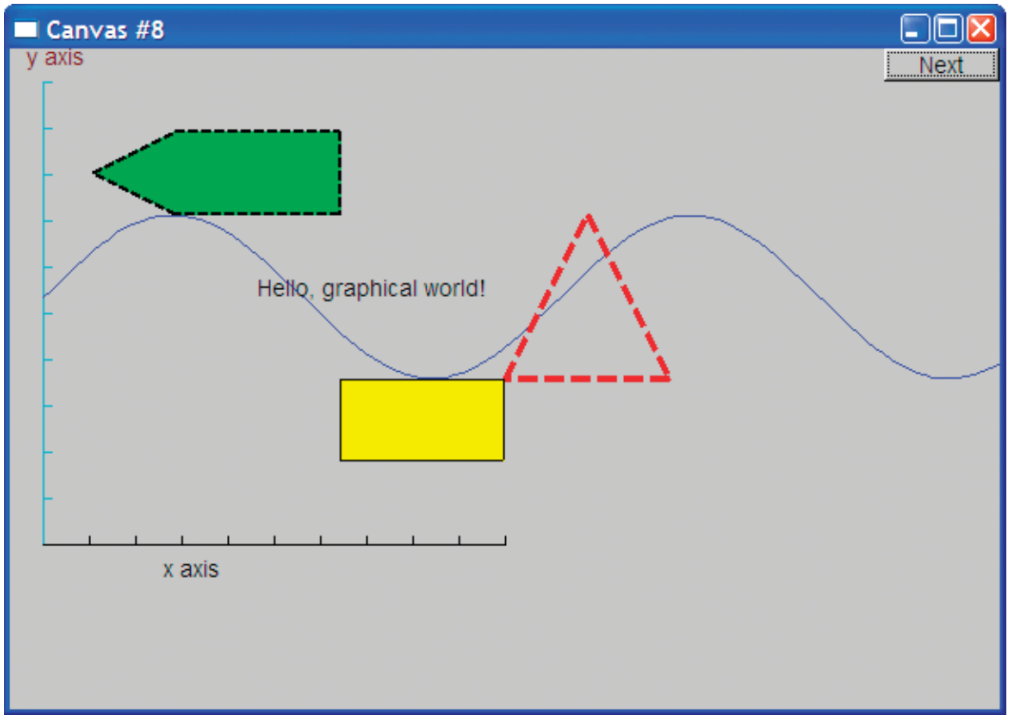
我们还决定不再喜欢三角形的（poly 的）线型，于是把它的线型设置为“粗（粗细为正常值的四倍）虚线”。类似地，我们修改了 poly_rect 的线型（现在它看起来已经不像个矩形了）：

如果仔细观察 poly_rect，你会看到轮廓线是绘制在填充之上的。

任何封闭图形都可以填充（见 §13.9）。矩形只是因为特别容易（且快速）填充而显得特殊。

12.7.8 文本

最后，如果一个绘图系统没有一种简单的书写文本的方法，它就不算完整——把每个字符都用一组线段画出来终究不是办法。我们已经在窗口自身和坐标轴上使用了标签，但我们还可以用 `Text` 对象把文本放置在任何地方：



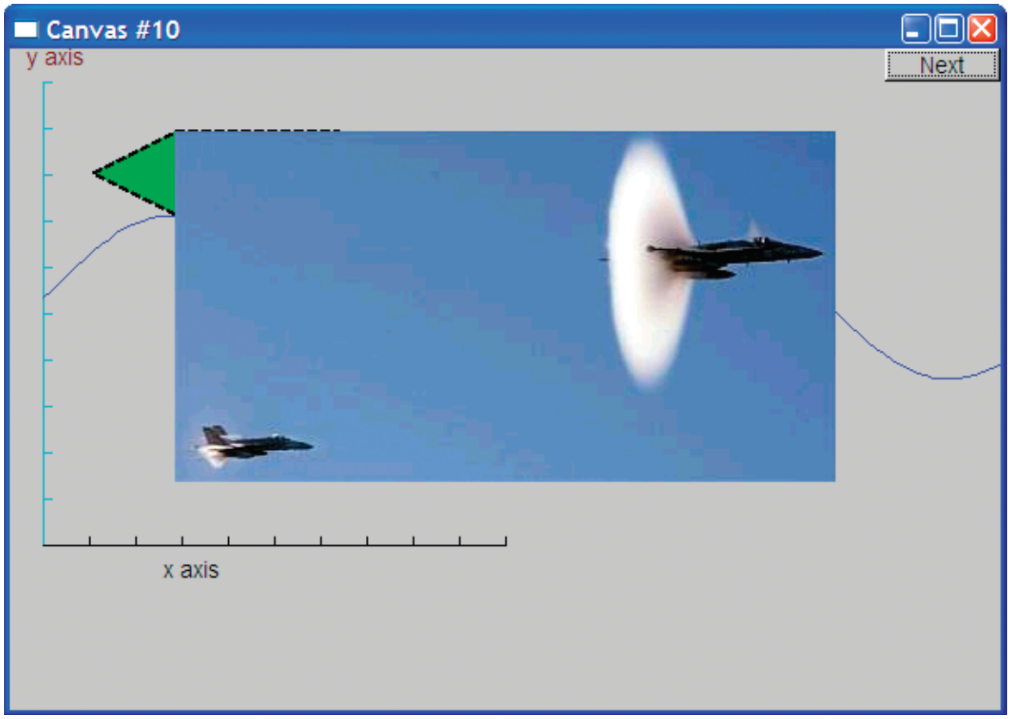

```
Text t {Point{150,150}, "Hello, graphical world!"};
win.attach(t);
win.set_label("Canvas #8");
win.wait_for_button();
```

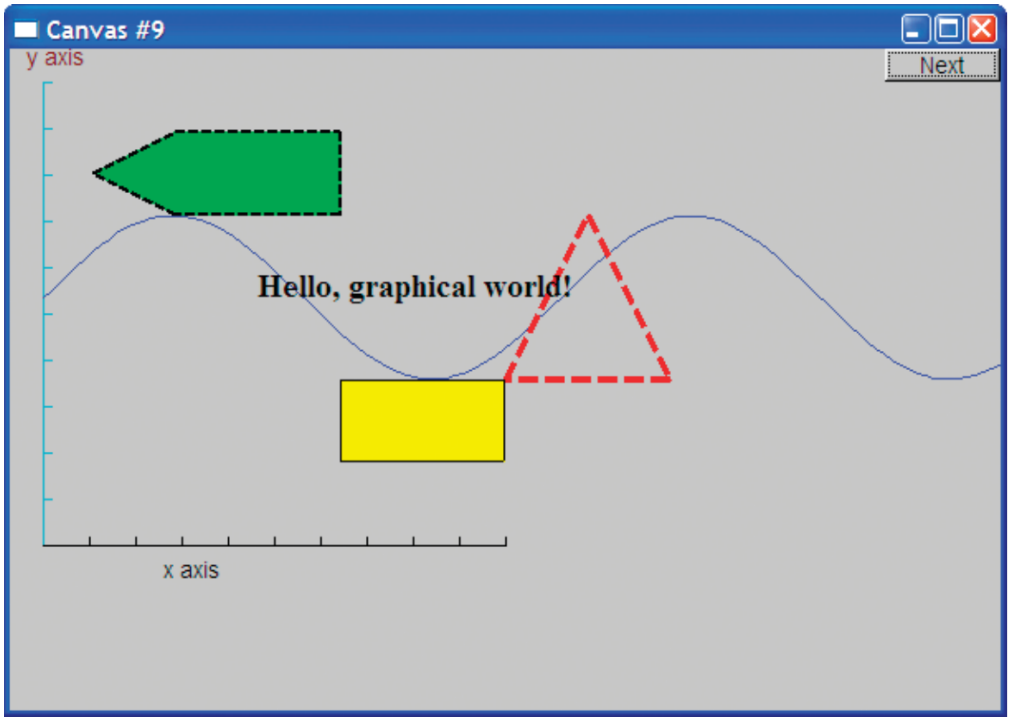
从这个窗口中你看到的基本图形元素出发，可以构建几乎任何复杂度和精巧度的显示效果。现在只需注意本章代码的一个特点：没有循环，没有选择语句，所有数据都是“硬编码”进去的。输出只是以最简单可能的方式由基本元素组合而成。一旦我们开始用数据和算法来组合这些基本元素，事情就会变得有趣起来。

我们已经见过如何控制文本的颜色：`Axis`（§12.7.3）的标签其实就是一个 `Text` 对象。此外，我们还可以选择字体并设置字符的大小：

```
t.set_font(Font::times_bold);
t.set_font_size(20);
win.set_label("Canvas #9");
win.wait_for_button();
```

我们把 `Text` 字符串 `Hello, graphical world!` 的字符放大到 20 磅，并选择了粗体的 Times 字体：





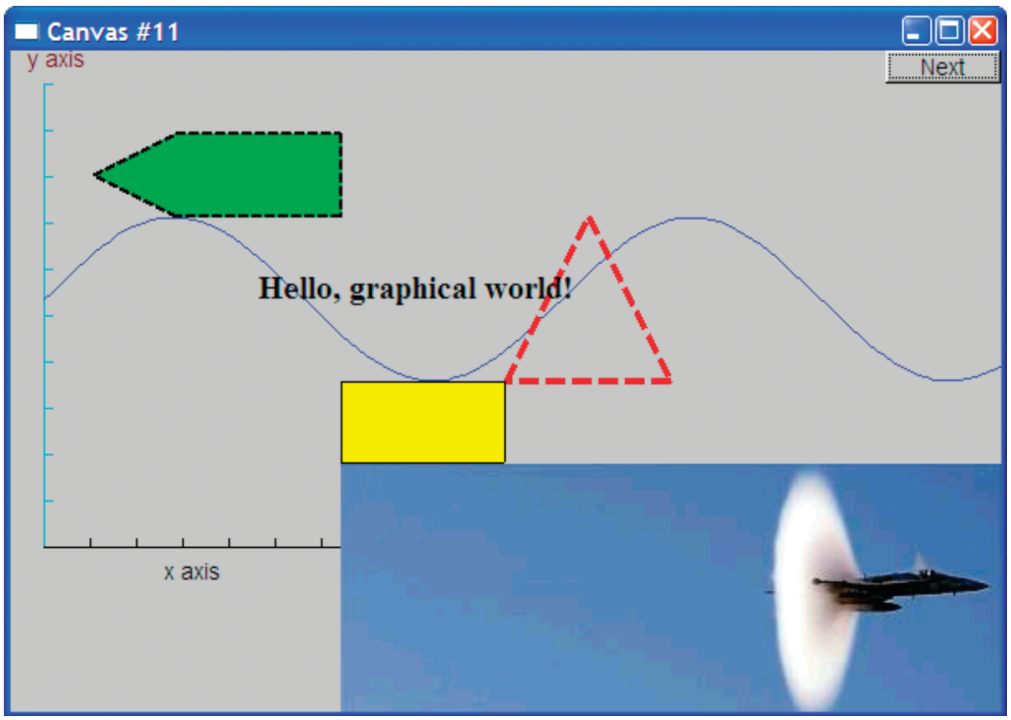
12.7.9 图像

我们还可以从文件加载图像：

```
Image ii {Point{100,50},"image.jpg"};      // 400*212 像素的 jpg
win.attach(ii);
win.set_label("Canvas #10");
win.wait_for_button();
```

碰巧，这个名为 image.jpg 的文件是一张两架飞机突破音障的照片：

这张照片相对较大，而我们把它直接放在了文本和图形的上面。因此，为了稍微整理一下窗口，我们把它挪开一点：



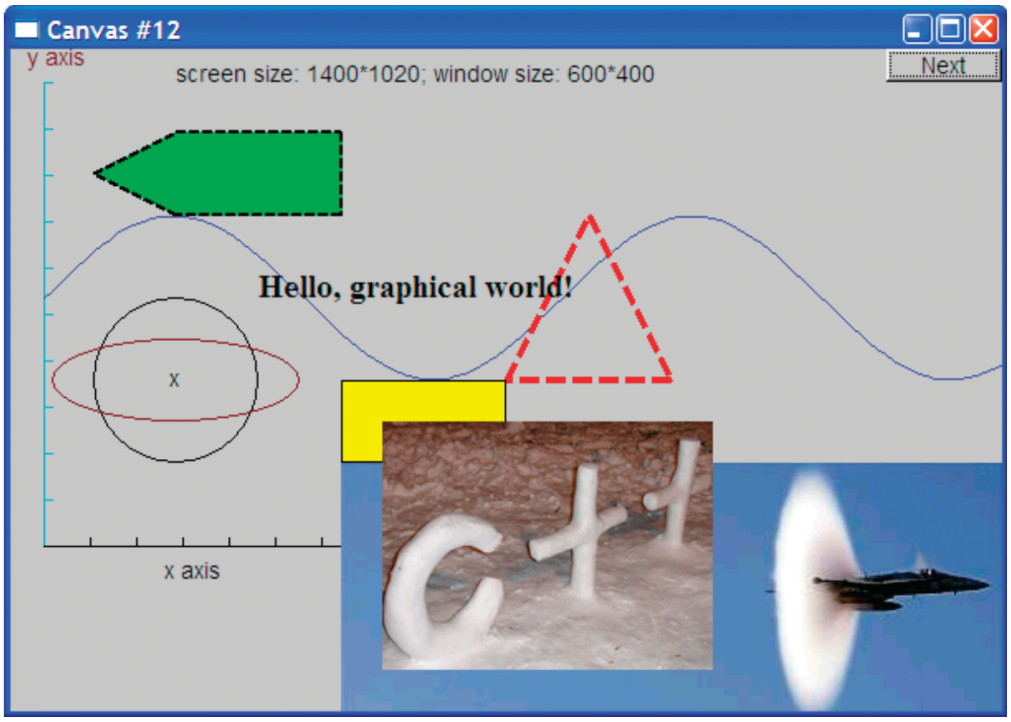

```
ii.move(100,200);
win.set_label("Canvas #11");
win.wait_for_button();
```

注意，照片中放不进窗口的部分根本没有被表示出来。本应出现在窗口之外的内容被“裁剪”（clip）掉了。

12.7.10　还有很多

这里，不加评论，再给出一些代码：

```
Circle c {Point{100,200},50};
Ellipse e {Point{100,200}, 75,25};
e.set_color(Color::dark_red);
Mark m {Point{100,200},'x'};

ostringstream oss;
oss << "screen size: " << x_max() << "*" << y_max()
    << "; window size: " << win.x_max() << "*" << win.y_max();
Text sizes {Point{100,20},oss.str()};

Image cal {Point{225,225},"snow_cpp.gif"};     // 320*240 像素的 gif
cal.set_mask(Point{40,40},200,150);            // 显示图像的中心部分
```



你能猜到这段代码是做什么的吗？它显而易见吗？

代码与屏幕上出现的内容之间的联系是直接的。如果你还没有看出这段代码是如何产生那个输出的，很快就会明白。注意我们是如何使用 `ostringstream`（§11.4）来格式化显示尺寸信息的 `Text` 对象的。

12.8　让程序运行起来

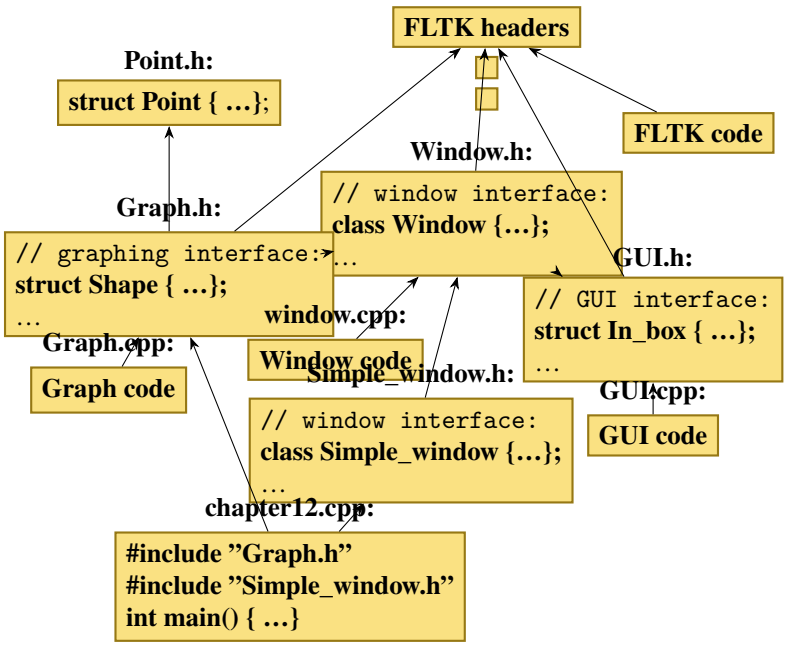
我们已经见过如何创建窗口以及如何在其中绘制各种图形。在接下来的几章中，我们将看到这些 `Shape` 类是如何定义的，并展示更多使用它们的方法。

让这个程序运行起来，比我们迄今为止给出的程序需要做更多的事情。除了 `main()` 中的代码之外，我们还需要将接口库代码编译并与我们的代码链接起来；最后，除非 FLTK 库（或我们所用的任何 GUI 系统）已安装并正确地与我们的代码链接，否则什么都运行不了。

观察这个程序的一种方式是，它由四个不同的部分组成：

- 我们的程序代码（main() 等）
- 我们的接口库（Window、Shape、Polygon 等）
- FLTK 库
- C++ 标准库

我们还间接地使用了操作系统。暂且不考虑操作系统和标准库，我们可以像这样图示图形代码的组织结构：



附录 D 说明了如何让这一切协同工作。

12.8.1 源文件

我们的图形和 GUI 接口库仅由五个头文件和三个代码文件组成：

- 头文件：
  - Point.h
  - Window.h
  - Simple_window.h
  - Graph.h
  - GUI.h
- 代码文件：
  - Window.cpp
  - Graph.cpp
  - GUI.cpp

在第 16 章之前，你可以忽略 GUI 文件。

小试身手

这个“小试身手”是图形版的“Hello, World!”程序，其目的是让你熟悉最简单的图形输出工具。

1. 编译、链接并运行一个空的 `Simple_window`，尺寸为 600
   *times*400，标签为 `My window`。注意，你必须按附录 D 的描述链接 FLTK 库；在你的代码中 `#include Graph.h` 和 `Simple_window.h`；并把 `Graph.cpp` 和 `Window.cpp` 加入你的工程。
2. 现在，将 §12.7 中的例子逐个添加进来，每添加一个小节的例子就测试一次。
3. 对每个小节的例子做一处小的修改（例如颜色、位置或点的数目）。

要点回顾

1. 我们为什么要使用图形？
2. 我们何时会尽量避免使用图形？
3. 为什么图形对程序员来说很有意思？
4. 什么是窗口（window）？
5. 我们的图形接口类（图形库）保存在哪个命名空间中？
6. 使用我们的图形库做基本绘图需要哪些头文件？
7. 最简单易用的窗口是什么？
8. 最小的窗口是什么？
9. 什么是窗口标签（window label）？
10. 如何给窗口设置标签？
11. 屏幕坐标是如何工作的？窗口坐标呢？数学坐标呢？
12. 我们可以显示的简单“图形（shape）”有哪些例子？
13. 将图形附着到窗口的命令是什么？
14. 画六边形你会用哪种基本图形？
15. 如何在窗口的某个位置写文本？
16. 如果要在一个窗口中放置你最好朋友的照片（使用你自己写的程序），你会怎么做？
17. 你创建了一个 `Window` 对象，但屏幕上什么也没出现。可能的原因有哪些？
18. 你创建了一个图形，但它没有出现在窗口中。可能的原因有哪些？

术语

颜色（color） 坐标（coordinates） 显示（display） 填充颜色（fill color） FLTK
图形（graphics） 图形用户界面（GUI） GUI 库 HTML 图像（image）
JPEG 线型（line style） 软件层（software layer） 窗口（window） XML

习题

我们建议你在这些习题中使用 `Simple_window`。

1. 分别用 `Rectangle` 和 `Polygon` 画一个矩形。将 `Polygon` 的线画成红色，`Rectangle` 的线画成蓝色。
2. 画一个 100
   *times*30 的 `Rectangle`，并在其中放置文本 `Howdy!`。
3. 画出你姓名的首字母，高 150 像素，使用粗线条，每个首字母用不同的颜色。
4. 画一个 3
   *times*3 的井字棋棋盘，格子为白色和红色相间。
5. 围绕一个矩形画一个 1/4 英寸宽的红色边框，该矩形的高度为屏幕高度的四分之三、宽度为屏幕宽度的三分之二。

6. 当你画一个超出窗口范围的 `Shape` 时会发生什么？当你画一个超出屏幕范围的 `Window` 时又会怎样？编写两个程序来演示这两种现象。
7. 画一座从正面看的二维房子，就像孩子画的那样：有一扇门、两扇窗和一个带烟囱的屋顶。可以随意添加细节，也许让烟囱冒出“烟”。
8. 画出奥运五环。如果不记得颜色，就去查一查。
9. 在屏幕上显示一幅图像，例如朋友的照片。既要在窗口标题上给图像加标题，也要在窗口内用说明文字标注图像。
10. 画出 §12.8 的文件关系图。
11. 画一系列正多边形，一个套一个。最里面的是一个等边三角形，外面套一个正方形，再外面套一个五边形，依此类推。仅限数学高手：让每个 N 边形的所有顶点都接触（$N+1$）边形的边。提示：三角函数在 `<cmath>`（§24.8、§B.9.2）中。
12. 超椭圆（superellipse）是一种由如下方程定义的二维形状

12. 在网上查一查 superellipse，以便更好地了解这类形状的样子。编写一个程序，通过连接超椭圆上的点来绘制“星形”图案。以 `a`、`b`、`m`、`n` 和 `N` 为参数。在由 `a`、`b`、`m` 和 `n` 定义的超级椭圆上选取 `N` 个点，使这些点按某种“相等”的定义等距分布。将每个点与其他一个或多个点连接（如果你愿意，可以把“连接到几个点”也作为参数，或者直接使用 `N-1`，即连接到所有其他点）。
13. 设法为上一题的线条添加颜色，让一部分线条用一种颜色，另一部分线条用另一种或几种颜色。

$$\left|\frac{x}{a}\right|^m + \left|\frac{y}{b}\right|^n = 1; \qquad m, n > 0.$$

后记

程序设计的理想是让我们的概念在程序中直接表示为实体。因此，我们常用类来表示思想，用类的对象表示现实世界中的实体，用函数表示动作和计算。图形正是这一思想有着明显应用的一个领域。我们有圆、多边形等概念，在程序中就把它们表示为 `class Circle` 和 `class Polygon`。图形领域的特别之处在于：编写图形程序时，我们还有机会在屏幕上看到这些类的对象；也就是说，程序的状态被直接呈现出来供我们观察——在大多数应用中我们没有这么幸运。这种思想、代码与输出之间的直接对应，正是图形编程如此吸引人的原因。但请务必记住，图形只是“用类在代码中直接表示概念”这一普遍思想的一个例证而已。这个思想远比图形更为通用和有用：几乎任何我们能想到的东西，都可以在代码中用类、类的对象或一组类来表示。

13 图形类

“一门不能改变你思维方式的语言不值得学习。”
——传统格言

第 12 章让我们了解了使用一组简单的接口类能做哪些图形方面的事，以及怎么做。本章将介绍其中提供的许多类，重点放在各个接口类（如 `Point`、`Color`、`Polygon` 和 `Open_polyline`）的设计、使用和实现及它们的用途上。下一章将介绍设计成组相关类的思想，并展示更多的实现技术。

13.1 图形类概览

图形库和 GUI 库提供了大量的设施。说“大量”，我们是指数以百计的类，而且每个类往往有数十个函数。阅读一份描述、手册或文档，有点像在看一本老式植物学教科书，它按晦涩的分类特征列出数千种植物的细节，令人生畏！它也可以令人兴奋——浏览一个现代图形/GUI 库的设施，会让人像置身糖果店的孩子，但很难弄清该从哪里入手，什么才是真正对你有用的。

我们这个接口库的目的之一，就是减轻一个成熟图形/GUI 库的复杂性给你带来的冲击。我们只给出了二十来个类，几乎没有多少操作，但它们已能让你产生有用的图形输出。一个密切相关的目标，是通过这些类介绍关键的图形和 GUI 概念。现在，你已经可以编写程序、把结果以简单图形的方式显示了。学完本章后，你所能编写图形程序的范围将超过大多数人的最初需求。学完第 14 章后，你将理解其中涉及的大部分设计技术和思想，从而能够按需加深理解、拓展图形表达的范围。你既可以在本章所述设施的基础上继续添加，也可以改用另一个 C++ 图形/GUI 库。

关键的接口类有：

图形接口类

Color	用于线、文本和填充形状
Line_style	用于画线
Point	用于表示屏幕上和 Window 中的位置
Line	屏幕上所看到的线段，由两个端点 Point 定义
Open_polyline	由 Point 序列定义的相连线段序列
Closed_polyline	类似 Open_polyline，只是最后有一个线段把最后一个 Point 与第一个相连
Polygon	任何两条线段都不相交的 Closed_polyline
Text	一串字符
Lines	由成对 Point 定义的一组线段
Rectangle	为快速、方便显示而优化的常见形状
Circle	由圆心和半径定义的圆
Ellipse	由圆心和两条轴定义的椭圆
Function	在一个区间中绘制的一元函数
Axis	带标签的坐标轴
Mark	由一个字符（例如 x 或 o）标记的点
Marks	由标记（例如 x 和 o）指示的点序列
Marked_polyline	点由标记指示的 Open_polyline
Image	图像文件的内容

第 15 章将考察 Function 和 Axis。第 16 章将给出主要的 GUI 接口类：

GUI 接口类

Window	屏幕上的一块区域，我们在其中显示图形对象
Simple_window	带有“Next”按钮的窗口
Button	窗口中通常带标签的矩形，按下它可运行我们的一个函数
In_box	窗口中通常带标签的框，用户可在其中输入字符串
Out_box	窗口中通常带标签的框，我们的程序可在其中输出字符串
Menu	Button 的 vector

源代码按如下方式组织到文件中：

图形接口源文件

Point.h	Point
Graph.h	其余所有图形接口类
Window.h	Window
Simple_window.h	Simple_window
GUI.h	Button 及其他 GUI 类
Graph.cpp	Graph.h 中函数的定义
Window.cpp	Window.h 中函数的定义
GUI.cpp	GUI.h 中函数的定义

除图形类之外，我们还给出一个恰好适合保存 Shape 或 Widget 集合的类：

Shape 或 Widget 的容器

Vector_ref	一种 vector，其接口便于保存未命名元素

阅读以下各节时，请不要走马观花。这里几乎没有不明显的内容，但本章的目的不只是给你看几张漂亮的图片——你在电脑屏幕或电视上天天都能看到更漂亮的图片。本章的要点是：

- 展示代码与所生成图片之间的对应关系。
- 让你习惯于阅读代码并思考它是如何工作的。
- 让你思考代码的设计——特别是如何把概念表示为代码中的类。为什么这些类长成这个样子？它们本还可以是什么样子？我们做出了许许多多的设计决策，其中大多数本可以合理地采用不同的决策，有些甚至可以完全不同。

所以请不要急于求成。否则你会错过一些重要的东西，之后可能会发现习题变得不必要的困难。

13.2 Point 与 Line

任何图形系统最基本的组成部分都是点（point）。定义点，就是定义我们组织几何空间的方式。这里，我们采用常规的、面向计算机的二维点布局，由 (x, y) 整数坐标定义。如 §12.5 所述，x 坐标从 0（表示屏幕左边缘）到 x_max()（表示屏幕右边缘）；y 坐标从 0（表示屏幕顶部）到 y_max()（表示屏幕底部）。

正如 Point.h 中所定义的，Point 就是一对 int（即坐标）：

```
struct Point {
    int x, y;
};

bool operator==(Point a, Point b) { return a.x==b.x && a.y==b.y; }
bool operator!=(Point a, Point b) { return !(a==b); }
```

在 Graph.h 中，我们能找到 Shape（我们将在第 14 章详细描述）和 Line：

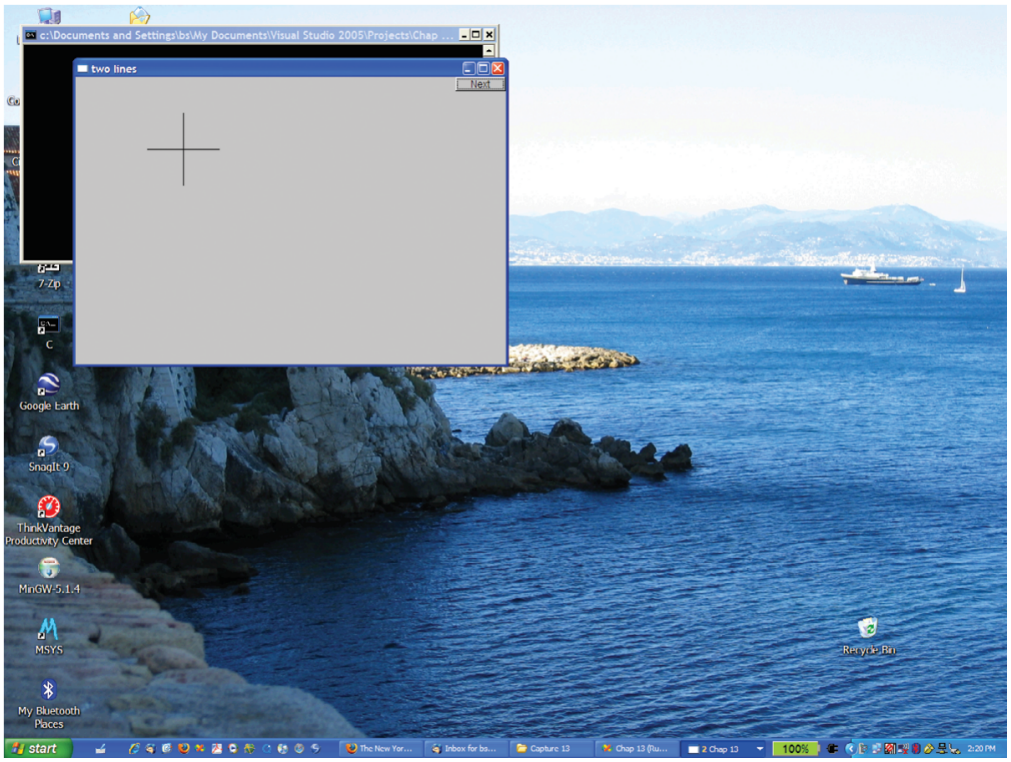
```
struct Line : Shape {     // Line 是由两个 Point 定义的 Shape
    Line(Point p1, Point p2);   // 由两个 Point 构造一条 Line
};
```

Line 是一种 Shape，这就是“: Shape”的含义。Shape 被称为 Line 的基类（base class），或简称 Line 的基（base）。基本上，Shape 提供了让 Line 的定义保持简洁所需的设施。等我们对具体图形（如 Line 和 Open_polyline）有了切身体会之后，再解释这意味着什么（第 14 章）。

一条 Line 由两个 Point 定义。省去“脚手架”（#include 等，如 §12.3 所述），我们可以像这样创建线并使其被绘制出来：

```
// 画两条线

constexpr Point x {100,100};

Simple_window win1 {x,600,400,"two lines"};

Line horizontal {x,Point{200,100}};           // 画一条水平线
Line vertical {Point{150,50},Point{150,150}}; // 画一条垂直线

win1.attach(horizontal);                      // 将线附着到窗口
win1.attach(vertical);

win1.wait_for_button();                       // 显示!
```

运行之后，我们得到



作为一个为简洁而设计的用户接口，Line 工作得相当好。不需要爱因斯坦的智商，你也能猜到

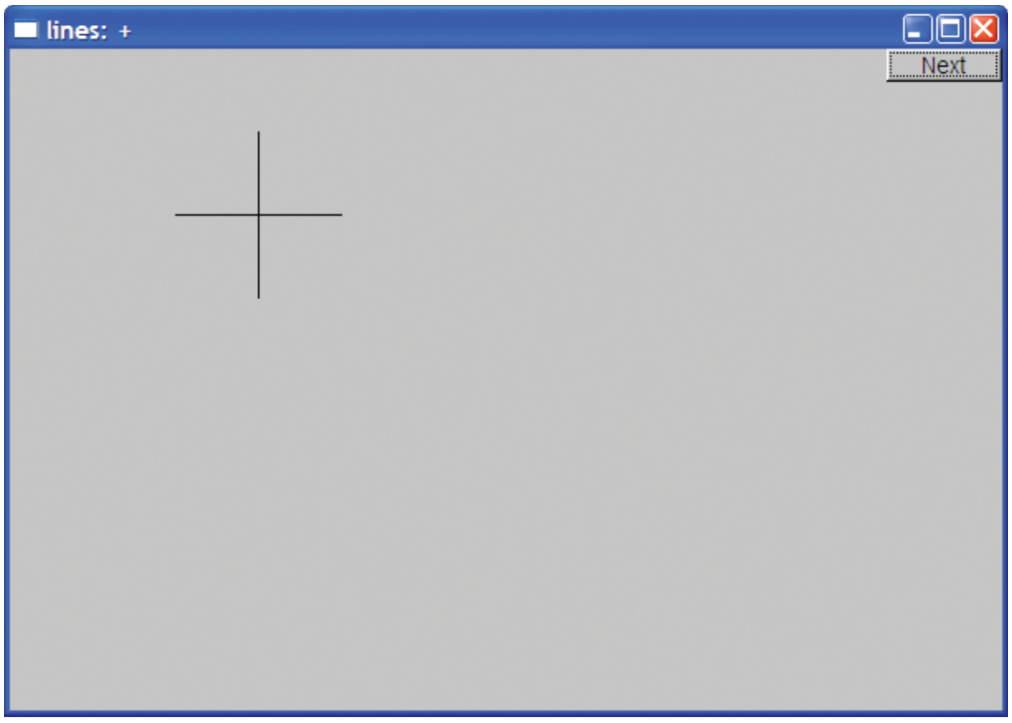
```
Line vertical {Point{150,50},Point{150,150}};
```

创建了一条从 (150,50) 到 (150,150) 的（垂直）线。当然，其中有一些实现细节，但要使用 Line，你无须了解它们。Line 的构造函数的实现同样简单：

```
Line::Line(Point p1, Point p2)    // 由两个点构造一条线
{
    add(p1);                      // 将 p1 加入此 shape
    add(p2);                      // 将 p2 加入此 shape
}
```

也就是说，它只是“添加”了两个点。添加到哪里？Line 又是如何在窗口中被画出来的？答案就在 Shape 类中。正如我们将在第 14 章描述的，Shape 能保存定义线的点，知道如何绘制由成对 Point 定义的线，并提供一个函数 add()，允许对象向其 Shape 添加一个 Point。这里的关键点（原文双关）是：定义 Line 是轻而易举的事，大部分实现工作都由“系统”完成了，我们才能专注于编写易用的简单类。

从现在开始，我们将省略 Simple_window 的定义和 attach() 的调用。它们只是一个完整程序所需的更多“脚手架”，对讨论具体的 Shape 没有多大帮助。



13.3 Lines

事实上，我们很少只画一条线。我们习惯以由许多线构成的对象来思考，例如三角形、多边形、路径、迷宫、网格、柱状图、数学函数、数据图等等。最简单的这类“复合图形对象类”之一就是 Lines：

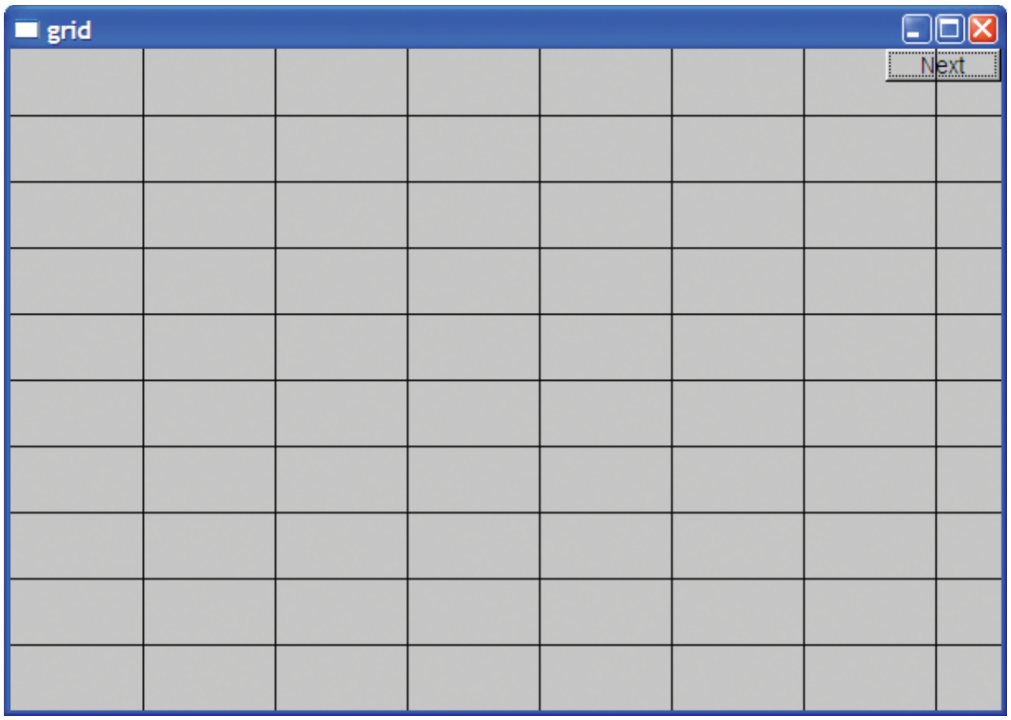
```
struct Lines : Shape {                  // 相关联的线
    Lines() {}                          // 空
    Lines(initializer_list<Point> lst); // 由 Point 列表初始化

    void draw_lines() const;
    void add(Point p1, Point p2);    // 添加一条由两个点定义的线
};
```

一个 Lines 对象就是一组线的集合，每条线由一对 Point 定义。例如，假如我们把 §13.2 中 Line 例子的两条线当作单个图形对象的组成部分，就可以像这样定义它们：

```
Lines x;
x.add(Point{100,100}, Point{200,100});     // 第一条线：水平
x.add(Point{150,50}, Point{150,150});      // 第二条线：垂直
```

这样得到的输出与 Line 版本（精确到最后一个像素）无法区分：

我们唯一能看出这是另一个窗口的方法，就是我们给它设置了不同的标签。



一组 Line 对象与一个 Lines 对象中的一组线，其差异完全在于我们如何看待所发生的事情。使用 Lines，我们表达了这样的观点：这两条线是彼此相关的，应该一起操作。例如，我们可以用一条命令改变一个 Lines 对象中所有线的颜色。反过来，我们可以给各个独立的 Line 对象设置不同的颜色。举一个更现实的例子，考虑如何定义一个网格。网格由若干均匀分布的水平线和垂直线组成。但我们把网格当作一个“东西”来考虑，所以把这些线定义为 Lines 对象的一部分，并称之为 grid：

```
int x_size = win3.x_max();    // 获取窗口的尺寸
int y_size = win3.y_max();
int x_grid = 80;
int y_grid = 40;

Lines grid;
for (int x=x_grid; x<x_size; x+=x_grid)
    grid.add(Point{x,0},Point{x,y_size});   // 垂直线
for (int y = y_grid; y<y_size; y+=y_grid)
    grid.add(Point{0,y},Point{x_size,y});   // 水平线
```

注意我们是如何用 x_max() 和 y_max() 获得窗口尺寸的。这也是第一个例子：我们编写的代码会计算出想要显示哪些对象。如果为每条网格线都定义一个命名变量来定义这个网格，那将是令人难以忍受的繁琐。由那段代码，我们得到

让我们回到 Lines 的设计。类 Lines 的成员函数是怎样实现的？Lines 只提供了两个构造函数和两个操作。

add() 函数只是把由一对点定义的一条线加入待显示的线集合中：

```
void Lines::add(Point p1, Point p2)
{
    Shape::add(p1);
    Shape::add(p2);
}
```

是的，Shape:: 限定是必需的；否则编译器会把 add(p1) 看成调用 Lines 自己的 add()（而不是 Shape 的 add()）的（非法）尝试。

draw_lines() 函数绘制用 add() 定义的线：

```
void Lines::draw_lines() const
{
    if (color().visibility())
        for (int i=1; i<number_of_points(); i+=2)
            fl_line(point(i-1).x,point(i-1).y,point(i).x,point(i).y);
}
```

也就是说，Lines::draw_lines() 一次取两个点（从第 0、1 个点开始），并用底层库的画线函数（fl_line()）在它们之间画线。可见性是 Lines 的 Color 对象（§13.4）的一个属性，因此在画线之前我们必须检查这些线是否应当可见。

正如我们在第 14 章所解释的，draw_lines() 是由“系统”调用的。我们不需要检查点的个数是否为偶数——Lines 的 add() 只能成对地加入点。函数 number_of_points() 和 point() 定义在类 Shape 中（§14.2），含义不言自明。这两个函数提供对 Shape 的点的只读访问。成员函数 draw_lines() 被定义为 const（参见 §9.7.4），因为它不修改形状。

Lines 的默认构造函数只是创建一个空对象（其中不含任何线）：从没有点开始、需要时再 add() 加点，这种模型比任何构造函数所能提供的都更灵活。不过，我们还增加了一个构造函数，它接受由 Point 对（pair）构成的 initializer_list，其中每一对点定义一条线。有了这个初始化列表构造函

数（§18.2），我们就能定义一开始就带有 0、1、2、3……条线的 Lines。例如，第一个 Lines 例子可以这样写：

```
Lines x = {
    {Point{100,100}, Point{200,100}},    // 第一条线：水平
    {Point{150,50}, Point{150,150}}      // 第二条线：垂直
};
```

甚至可以这样写：

```
Lines x = {
    {{100,100}, {200,100}},    // 第一条线：水平
    {{150,50}, {150,150}}      // 第二条线：垂直
};
```

初始化列表构造函数很容易定义：

```
void Lines::Lines(initializer_list<pair<Point,Point>> lst)
{
    for (auto p : lst) add(p.first,p.second);
}
```

这里的 auto 是类型 pair<Point,Point> 的占位符，first 和 second 则是 pair 的第一个和第二个成员的名字。类型 initializer_list 和 pair 定义在标准库中（§B.6.4、§B.6.3）。

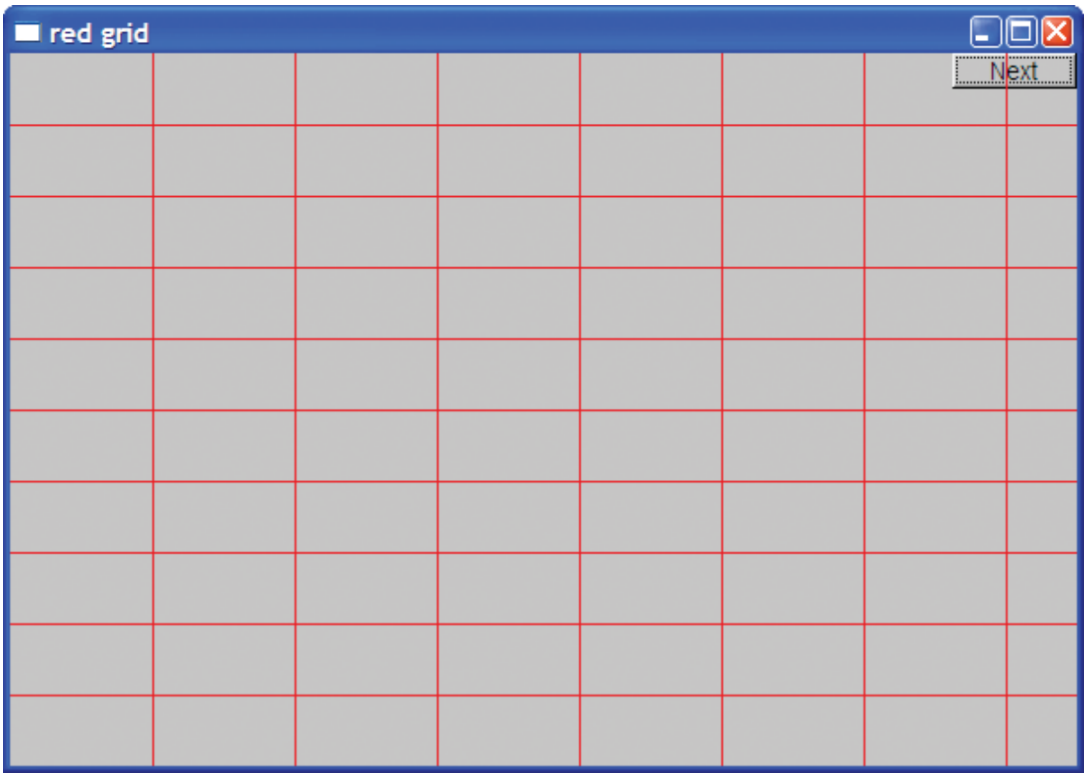
13.4 Color

Color 是我们用来表示颜色的类型。可以这样使用 Color：

```
grid.set_color(Color::red);
```

Color 定义了颜色的概念，并给一些较常见的颜色起了符号名：

```
struct Color {
    enum Color_type {
        red=FL_RED,
        blue=FL_BLUE,
        green=FL_GREEN,
        yellow=FL_YELLOW,
        white=FL_WHITE,
        black=FL_BLACK,
        magenta=FL_MAGENTA,
        cyan=FL_CYAN,
        dark_red=FL_DARK_RED,
        dark_green=FL_DARK_GREEN,
        dark_yellow=FL_DARK_YELLOW,
        dark_blue=FL_DARK_BLUE,
```



这把 grid 中定义的线染成红色。

```
        dark_magenta=FL_DARK_MAGENTA,
        dark_cyan=FL_DARK_CYAN
    };
    enum Transparency { invisible = 0, visible=255 };
    Color(Color_type cc) :c{Fl_Color(cc)}, v{visible} { }
    Color(Color_type cc, Transparency vv) :c{Fl_Color(cc)}, v{vv} { }
    Color(int cc) :c{Fl_Color(cc)}, v{visible} { }
    Color(Transparency vv) :c{Fl_Color()}, v{vv} { }    // 默认颜色
    int as_int() const { return c; }
    char visibility() const { return v; }
    void set_visibility(Transparency vv) { v=vv; }
private:
    char v;    // 目前只有不可见和可见两种状态
    Fl_Color c;
};
```

Color 的用途是

- 隐藏实现中表示颜色的方式，即 FLTK 的 Fl_Color 类型
- 在 Fl_Color 与 Color_type 的值之间做映射
- 为颜色常量提供作用域
- 提供一种简单形式的透明性（可见与不可见）

你可以这样挑选颜色：

- 从具名颜色列表中选取，例如 Color::dark_blue。
- 从大多数屏幕都能很好显示的一小组“调色板”颜色中选取：给出 0～255 范围内的一个值；例如 Color(99) 是一种深绿色。代码示例见 §13.9。

- 在 RGB（红、绿、蓝）系统中给出一个值来选取；这个系统我们在这里不解释，需要时请自行查阅。特别地，在网上搜索“RGB color”可以得到许多资料，例如 http://en.wikipedia.org/wiki/RGB_color_model 和 www.rapidtables.com/web/color/RGB_Color.htm。另见习题 13 和 14。

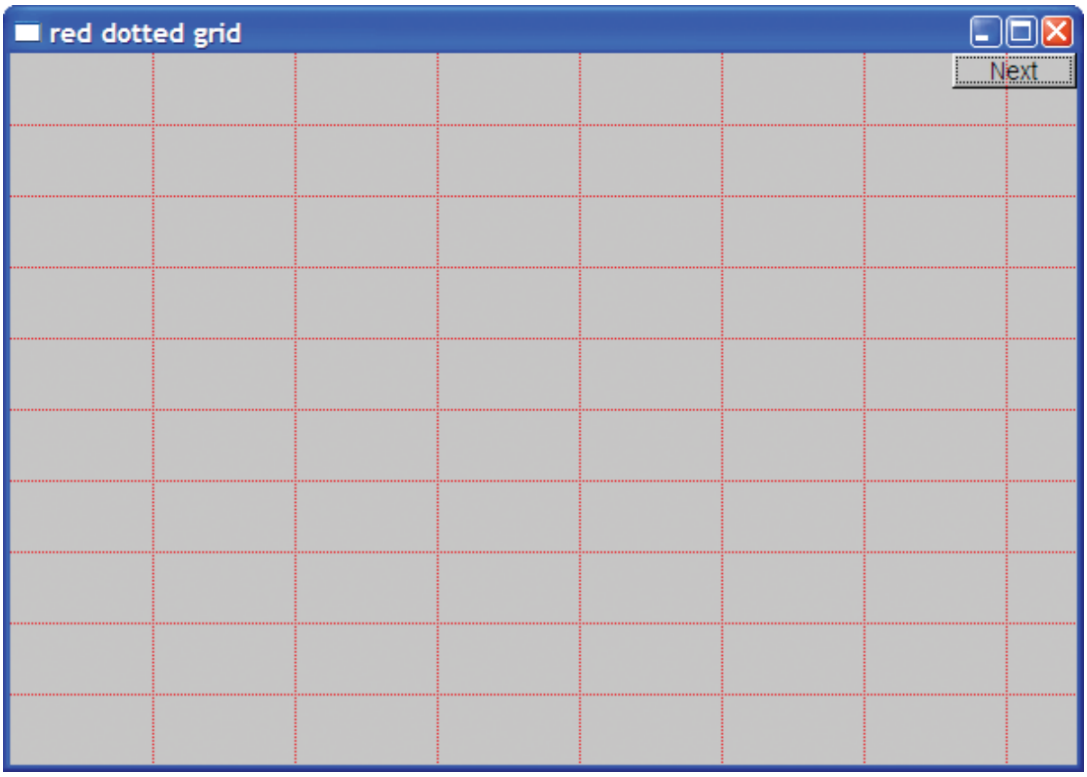
注意这里对构造函数的运用：它使 Color 既能由 Color_type 创建，也能由一个普通的 int 创建。成员 c 由每个构造函数初始化。你可能会说，c 这个名字太短、太晦涩，不适合使用；但它只在 Color 这个很小的作用域内使用，并不打算供一般使用，所以这样大概没问题。我们把成员 c 设为 private，以免用户直接使用它。表示数据成员 c 时，我们用了 FLTK 的 Fl_Color 类型，而我们并不想把它暴露给用户。不过，把一个颜色看成一个表示其 RGB 值（或其他值）的 int 是非常常见的做法，因此我们为此提供了 as_int()。注意 as_int() 是 const 成员函数，因为它实际上并不改变它所作用的那个 Color 对象。

透明性由成员 v 表示，它可以取 Color::visible 和 Color::invisible 两个值，含义不言自明。一个“不可见的颜色”竟然有用，这也许会让你感到意外；但让一个组合形状的某一部分不可见，往往非常有用。

13.5 Line_style

当我们在窗口中画好几条线时，可以用颜色、用样式，或者同时用两者把它们区分开。线样式（line style）就是用来描画线条的图案。可以这样使用 Line_style：

```
grid.set_style(Line_style::dot);
```



这把 grid 中的线显示为一系列点，而不是一条实线。

这使网格“稀疏”了一些，显得更含蓄。通过调整宽度（粗细），我们可以按自己的喜好和需要来调整网格线。

Line_style 类型看起来像这样：

```
struct Line_style {
    enum Line_style_type {
        solid=FL_SOLID,           // -------
        dash=FL_DASH,             // - - - -
        dot=FL_DOT,               // .......
        dashdot=FL_DASHDOT,       // - . - .
        dashdotdot=FL_DASHDOTDOT, // -..-..
    };
    Line_style(Line_style_type ss) :s{ss}, w{0} { }
    Line_style(Line_style_type lst, int ww) :s{lst}, w{ww} { }
    Line_style(int ss) :s{ss}, w{0} { }
    int width() const { return w; }
    int style() const { return s; }
private:
    int s;
    int w;
};
```

定义 Line_style 所用的编程技术与定义 Color 时完全相同。这里我们隐藏了一个事实：FLTK 是用普通的 int 来表示线样式的。为什么这样的东西值得隐藏？因为恰恰是这类细节可能会随着库的演进而改变。下一个 FLTK 版本很可能就有 Fl_linestyle 类型，或者我们可能把接口类改为面向另一个图形用户界面库。无论哪种情况，我们都不希望自己的代码和用户的代码里到处散落着普通的 int——只因为我们碰巧知道它们表示线样式。

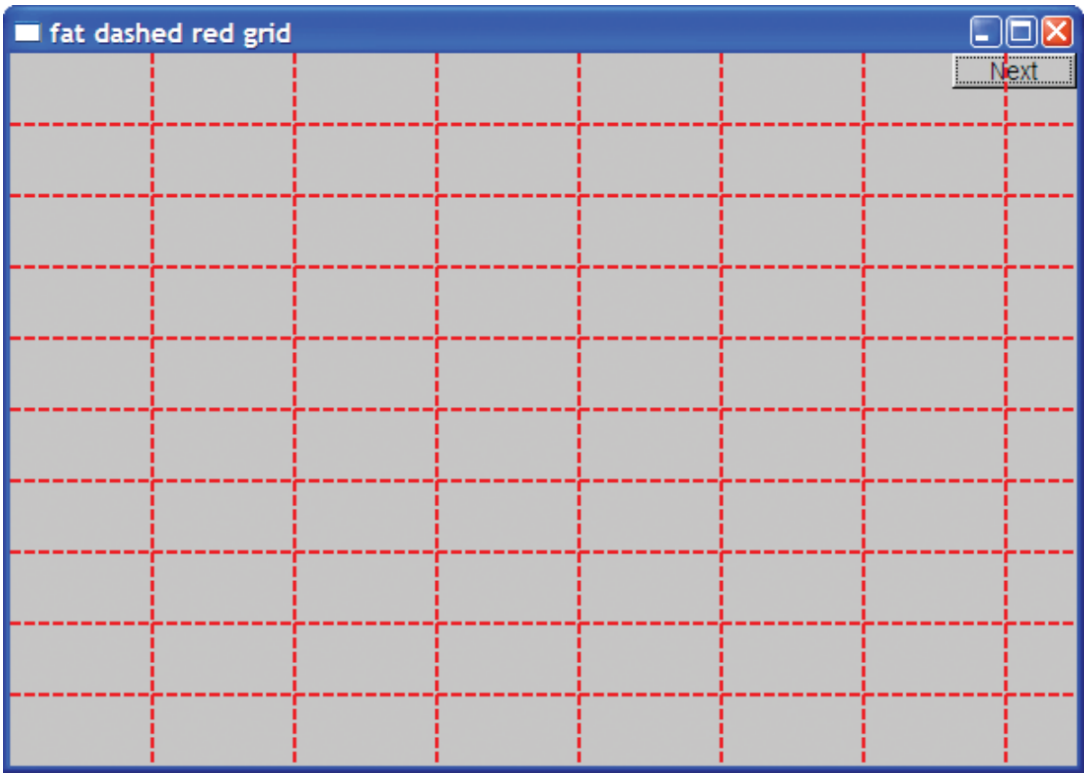
大多数时候，我们根本不关心样式，只依赖默认值（默认宽度和实线）。在我们没有显式指定的情况下，这个默认线宽由构造函数设定。设置默认值正是构造函数擅长的事情之一，而好的默认值能大大方便类的用户。

注意 Line_style 有两个“组成部分”：样式本身（例如用虚线还是实线）和宽度（所用线条的粗细）。宽度用整数计量。默认宽度是 1。我们可以这样要求一条粗虚线：

```
grid.set_style(Line_style{Line_style::dash,2});
```

注意颜色和样式作用于一个形状的所有线条。这正是把许多条线归入单个图形对象（例如 Lines、Open_polyline 或 Polygon）的优点之一。如果我们想分别控制各条线的颜色或样式，就必须把它们定义为彼此独立的 Lines。例如：

```
horizontal.set_color(Color::red);
vertical.set_color(Color::green);
```



这会得到一条粗虚线。

水平线显示为红色，垂直线显示为绿色。

13.6 Open_polyline

Open_polyline 是由一系列点所定义的一系列相连线段构成的形状。*Poly* 在希腊语中是“多”的意思，而 polyline 是相当惯用的名称，用来指由许多条线构成的形状。例如：

```
Open_polyline opl = {
    {100,100}, {150,200}, {250,250}, {300,200}
};
```

这画出的形状，就是把四个点连接起来所得到的。

基本上，Open_polyline 不过是幼儿园里玩“连点成画”时所遇到的东西的一个漂亮说法。

类 Open_polyline 是这样定义的：

```
struct Open_polyline : Shape {     // 开放的线段序列
    using Shape::Shape;     // 使用 Shape 的构造函数 (§A.16)
    void add(Point p) { Shape::add(p); }
};
```

Open_polyline 继承自 Shape。Open_polyline 的 add() 函数之所以存在，是为了让 Open_polyline 的用户能够访问 Shape 的 add()（即 Shape::add()）。我们甚至不需要定义 draw_lines()，因为 Shape 默认就会把 add() 进来的 Point 解释为一串相连的线段。

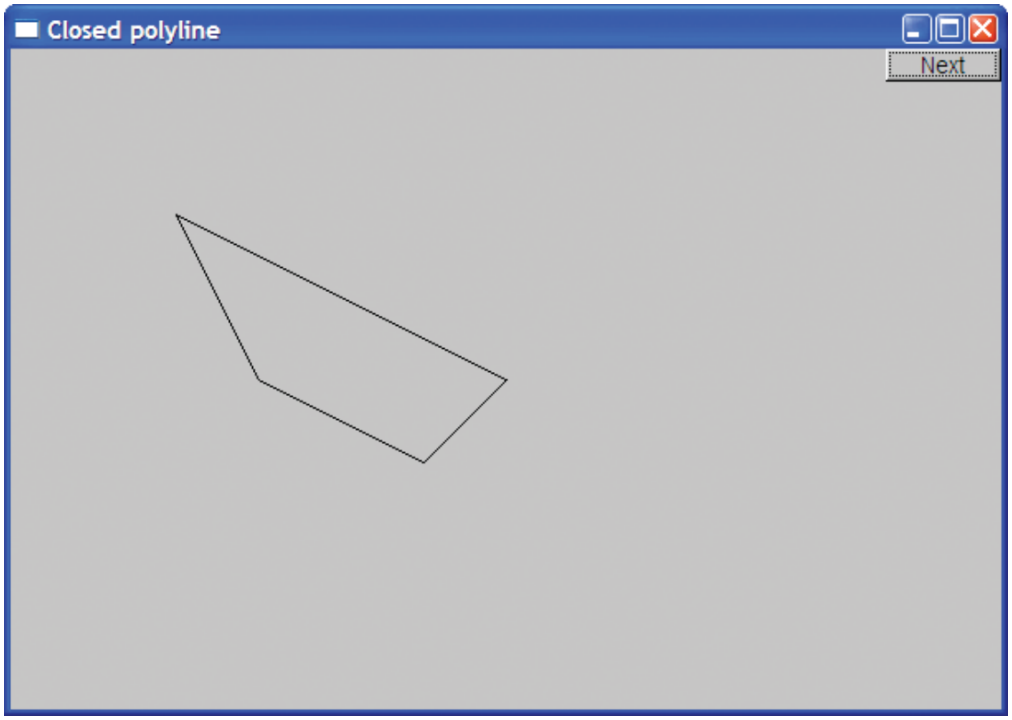
声明 using Shape::Shape 是一条 using 声明。它表示 Open_polyline 可以使用为 Shape 定义的构造函数。Shape 有一个默认构造函数（§9.7.3）和一个初始化列表构造函数（§18.2），因此这条 using 声明只是为 Open_polyline 定义这两个构造函数的简写。和 Lines 一样，初始化列表构造函数在这里是一种简写，用来代替一开始的一连串 add() 调用。

13.7 Closed_polyline

Closed_polyline 与 Open_polyline 完全一样，只是我们还会从最后一点到第一点画一条线。例如，我们可以把 §13.6 中用于 Open_polyline 的那些点用来定义一个 Closed_polyline：

```
Closed_polyline cpl = {
    {100,100}, {150,200}, {250,250}, {300,200}
};
```

结果（当然）与 §13.6 的完全相同，只多了最后那条闭合线：



Closed_polyline 的定义是

```
struct Closed_polyline : Open_polyline {     // 闭合的线段序列
    using Open_polyline::Open_polyline;      // 使用 Open_polyline 的构造函数 (§A.16)
    void draw_lines() const;
};

void Closed_polyline::draw_lines() const
{
    Open_polyline::draw_lines();          // 先画出“开放折线部分”
    // 再画闭合线:
    if (2<number_of_points() && color().visibility())
        fl_line(point(number_of_points()-1).x,
                point(number_of_points()-1).y,
                point(0).x,
                point(0).y);
}
```

using 声明（§A.16）的意思是：Closed_polyline 拥有与 Open_polyline 相同的构造函数。Closed_polyline 需要自己的 draw_lines()，以便画出那条把最后一个点连回第一个点的闭合线。

我们只需处理 Closed_polyline 与 Open_polyline 所提供的东西不同的那一点细节。这一点很重要，有时被称为“按差异编程”（programming by difference）。派生类（这里是 Closed_polyline）与基类（这里是 Open_polyline）相比有什么不同，我们就只编写这不同的部分。

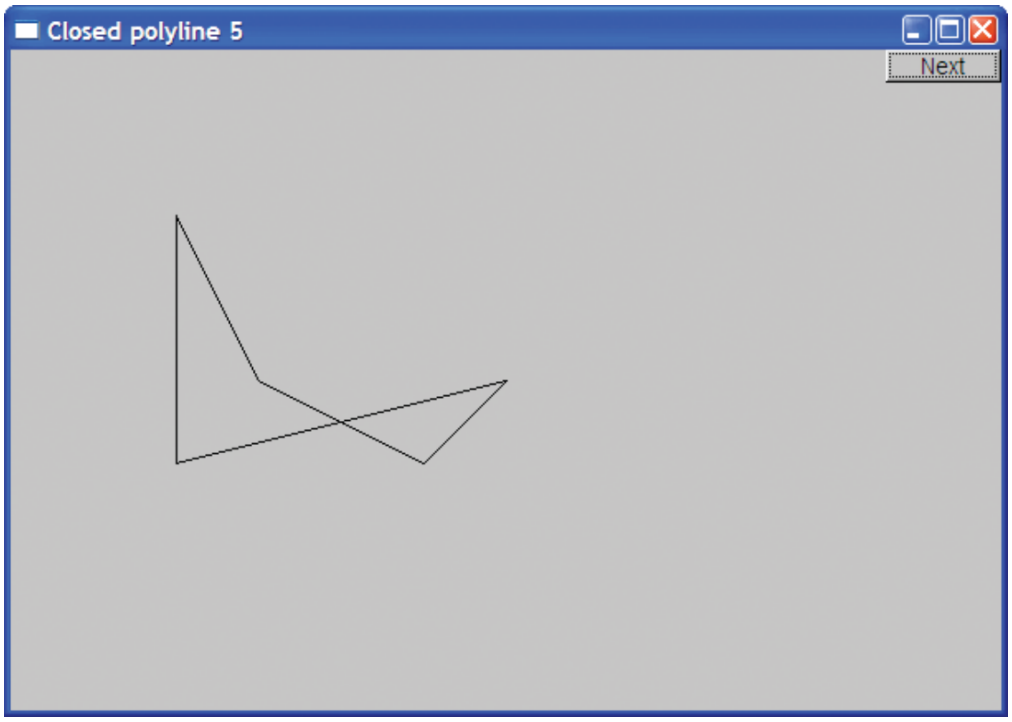
那么，这条闭合线该怎么画呢？我们用 FLTK 的画线函数 fl_line()。它接受四个 int，用来表示两个点。可见这里又一次用到了底层的图形库。不过要注意：与其他所有情形一样，对 FLTK 的提及只留在我们类的实现里，而不会暴露给用户。没有任何用户代码需要提到 fl_line()，也不需要了解那种把点隐式表示为整数对的接口。如果我们愿意，可以用另一个图形用户界面库替换 FLTK，而对我们用户的代码几乎没有影响。

13.8 Polygon

Polygon 与 Closed_polyline 非常相似。唯一的区别是：对于 Polygon，我们不允许线段相交。例如，上面那个 Closed_polyline 就是一个多边形，但我们还可以再加一个点：

```
cpl.add(Point{100,250});
```

结果就成了



按照经典定义，这个 Closed_polyline 并不是多边形。那么，怎样定义 Polygon，才能既正确刻画它与 Closed_polyline 的关系，又不违反几何学的规则呢？上面的叙述里含有一个很强的提示。Polygon 就是线段互不相交的 Closed_polyline。或者，我们也可以强调形状是由点构造出来的这一面，说：Polygon 是一个 Closed_polyline，其中不能添加某个 Point，从而定义出一条与 Polygon 已有线段相交的线段。

有了这个想法，我们这样定义 Polygon：

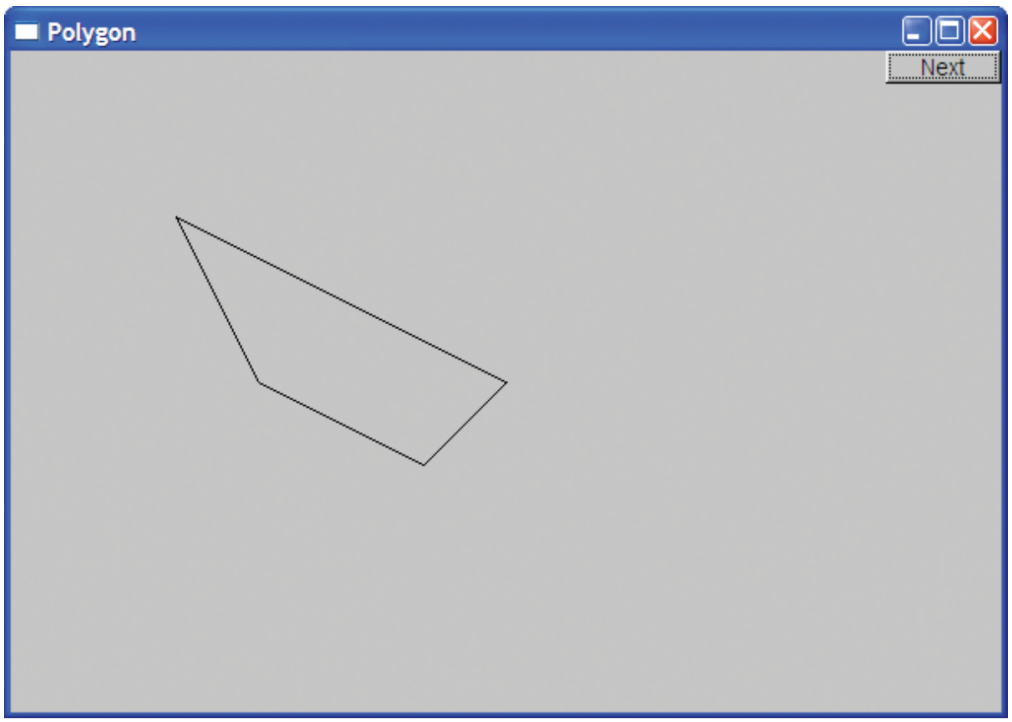
```
struct Polygon : Closed_polyline {           // 闭合的、互不相交的线段序列
    using Closed_polyline::Closed_polyline;  // 使用 Closed_polyline 的构造函数
    void add(Point p);
    void draw_lines() const;
};

void Polygon::add(Point p)
{
    // 检查新线段是否与已有线段相交（代码未给出）
    Closed_polyline::add(p);
}
```

这里我们继承了 Closed_polyline 对 draw_lines() 的定义，因而省下不少工作，也避免了代码重复。遗憾的是，我们不得不检查每一次 add()。这带来的算法效率不高（量级为 N 的平方）——定义一个含 N 个点的 Polygon 需要调用 intersect() 共 N*(N-1)/2 次。实际上，我们等于假设 Polygon 类只用于点数较少的多边形。例如，创建一个含 24 个 Point 的 Polygon 要调用 intersect() 共 24*(24-1)/2 == 276 次。这大概可以接受；但如果想要一个有 2000 个点的多边形，就要花掉约 2,000,000 次调用，那时我们可能就得寻找更好的算法，而那又可能需要修改接口。

使用初始化列表构造函数，我们可以这样创建一个多边形：

```
Polygon poly = {
    {100,100}, {150,200}, {250,250}, {300,200}
};
```

显然，这样创建出来的 Polygon 与我们最初的 Closed_polyline 完全相同（精确到最后一个像素）：



要确保 Polygon 真的表示一个多边形，结果出乎意料地麻烦。我们在 Polygon::add() 里略去的那个相交检查，可以说是整个图形库中最复杂的部分。如果你对繁琐的坐标几何运算感兴趣，不妨去看看那段代码。

麻烦在于，Polygon 的不变式“这些点表示一个多边形”要等到所有点都定义之后才能验证；也就是说，我们并没有——尽管这是极力推荐的做法——在构造函数里建立 Polygon 的不变式。我们考虑过去掉 add()，要求 Polygon 完全由一个至少含三个点的初始化列表来指定，但那样会让“程序自己生成一串点”的用法变得复杂。

13.9 Rectangle

屏幕上最常见的形状是矩形。原因之一是文化上的（我们的门、窗、画、墙、书架、纸张等等大多也是矩形的），原因之二是技术上的（把坐标限制在矩形区域内，比限制在任何其他形状的区域内都更简单）。无论如何，矩形是如此常见，以至于图形用户界面系统直接支持它们，而不是简单地把它们当作碰巧有四个角和直角的 Polygon。

```
struct Rectangle : Shape {
    Rectangle(Point xy, int ww, int hh);
    Rectangle(Point x, Point y);
    void draw_lines() const;

    int height() const { return h; }
    int width() const { return w; }
private:
    int h;      // 高
    int w;      // 宽
};
```

我们可以用两个点（左上角和右下角）来指定一个矩形，也可以用一个点（左上角）加上宽和高来指定。构造函数可以这样定义：

```
Rectangle::Rectangle(Point xy, int ww, int hh)
    : w{ww}, h{hh}
{
    if (h<=0 || w<=0)
        error("Bad rectangle: non-positive side");
    add(xy);
}

Rectangle::Rectangle(Point x, Point y)
    : w{y.x-x.x}, h{y.y-x.y}
{
    if (h<=0 || w<=0)
        error("Bad rectangle: first point is not top left");
    add(x);
}
```

每个构造函数都（用成员初始化语法，见 §9.4.4）适当地初始化成员 h 和 w，并（用 add()）把左上角那个点存进 Rectangle 的基类 Shape 里。此外，它还做一个简单的合理性检查：我们并不真的想要宽或高为负的 Rectangle。

有些图形与图形用户界面系统把矩形特殊对待，原因之一在于：判断哪些像素位于矩形内部的算法，比对 Polygon、Circle 等其他形状的同类算法要简单得多，因而也快得多。因此，“填充色”（fill color）——也就是矩形内部区域的颜色——这个概念用在矩形上比其他形状更普遍。我们可以在构造函数里设置填充色，也可以通过操作 set_fill_color() 来设置（它与颜色方面的其他服务一起由 Shape 提供）：

```
Rectangle rect00 {Point{150,100},200,100};
Rectangle rect11 {Point{50,50},Point{250,150}};
Rectangle rect12 {Point{50,150},Point{250,250}};     // 恰在 rect11 下方
Rectangle rect21 {Point{250,50},200,100};           // 恰在 rect11 右侧
Rectangle rect22 {Point{250,150},200,100};          // 恰在 rect21 下方

rect00.set_fill_color(Color::yellow);
rect11.set_fill_color(Color::blue);
rect12.set_fill_color(Color::red);
rect21.set_fill_color(Color::green);
```

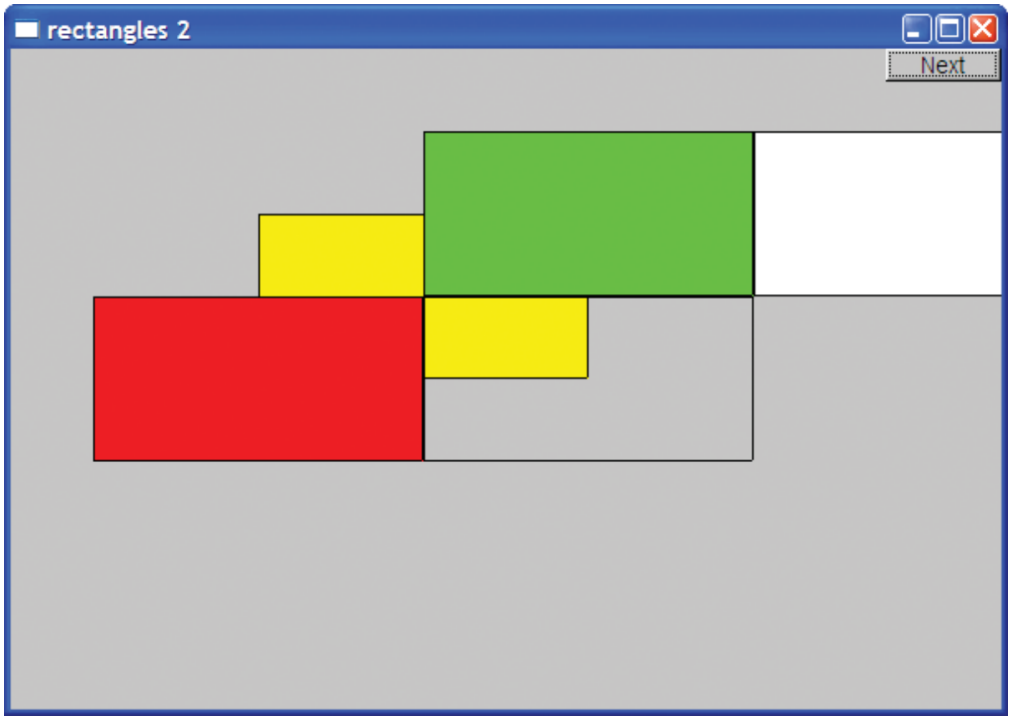
得到的结果是

没有设置填充色时，矩形是透明的；正因如此，你才能看到黄色 rect00 的一个角。

我们可以在窗口里移动图形（§14.2.3）。例如：

```
rect11.move(400,0);                    // 移到 rect21 右侧
rect11.set_fill_color(Color::white);
win12.set_label("rectangles 2");
```

得到的结果是



注意白色的 rect11 只有一部分落在窗口内。没放进去的部分被“裁剪”了，也就是说，它在屏幕上的任何地方都不显示。

还要注意各个形状是怎样一层层叠放起来的。做法就像把一张张纸摞在桌子上：你最先放下的那张在最底下。我们的 Window（§E.3）提供了一种重排形状的简单方法：你可以让窗口把某个形状放到最上面（用 Window::put_on_top()）。例如：

```
win12.put_on_top(rect00);
win12.set_label("rectangles 3");
```

得到的结果是

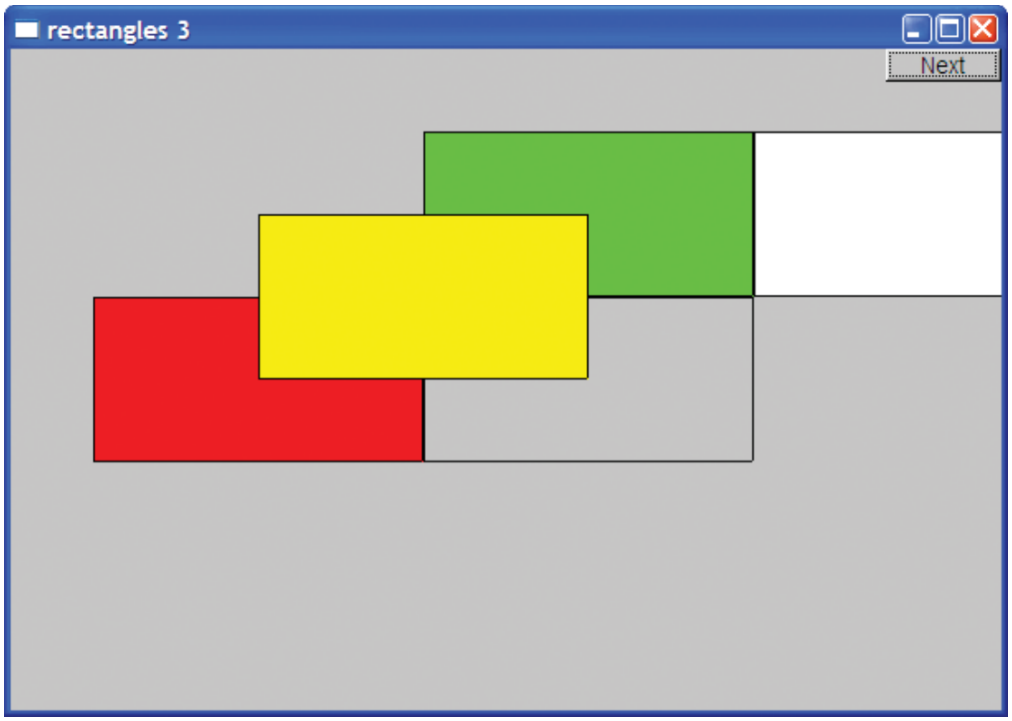
注意，尽管我们已经填充了（除一个之外的）所有这些矩形，构成矩形的那些线条仍然可见。如果不喜欢这些轮廓线，我们可以把它们去掉：

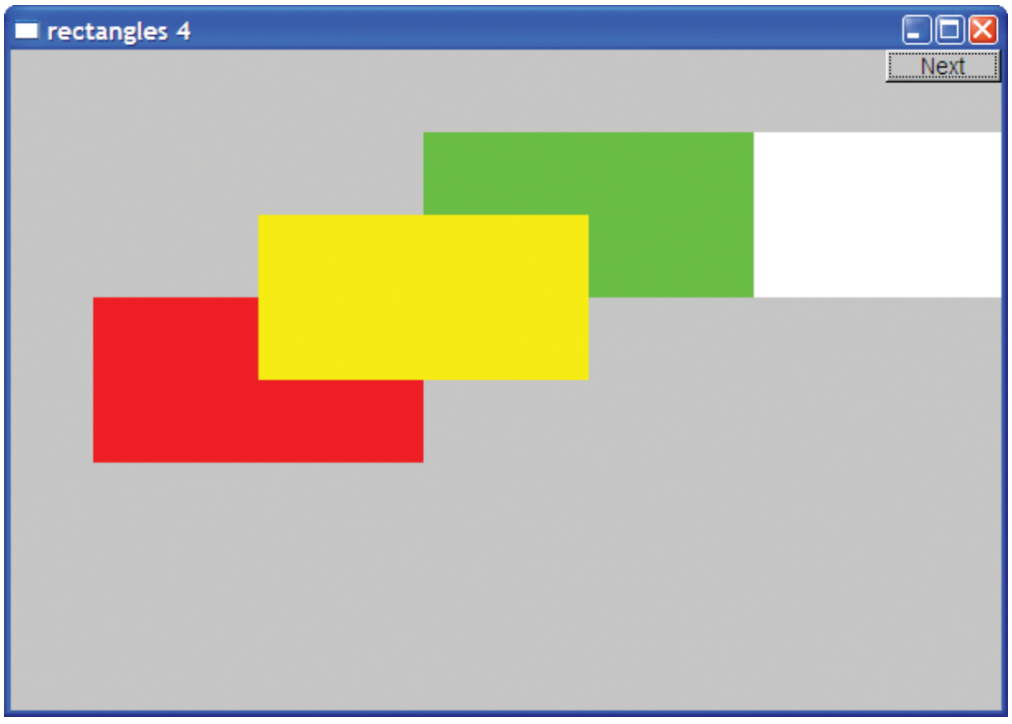
```
rect00.set_color(Color::invisible);
rect11.set_color(Color::invisible);
rect12.set_color(Color::invisible);
rect21.set_color(Color::invisible);
rect22.set_color(Color::invisible);
```

我们得到

注意，当填充色和线色都设置为 invisible 后，rect22 就再也看不见了。

由于必须同时处理线色和填充色，Rectangle 的 draw_lines() 写出来有点乱：





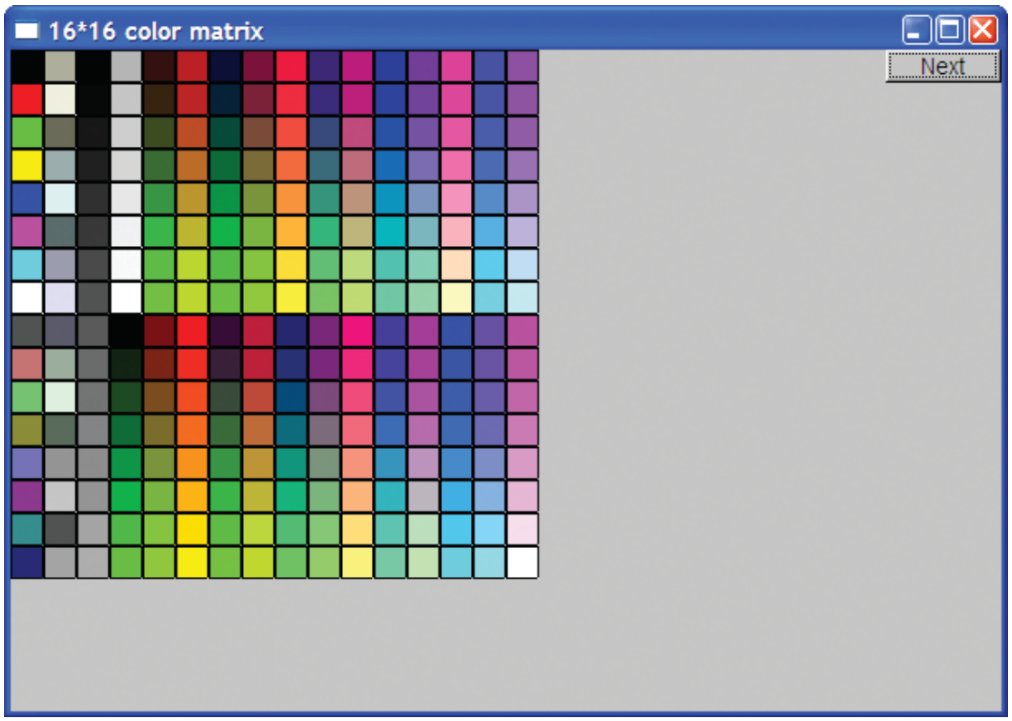

```
void Rectangle::draw_lines() const
{
    if (fill_color().visibility()) {    // 填充
        fl_color(fill_color().as_int());
        fl_rectf(point(0).x,point(0).y,w,h);
    }

    if (color().visibility()) {         // 画在填充之上的线条
        fl_color(color().as_int());
        fl_rect(point(0).x,point(0).y,w,h);
```

```
    }
}
```

如你所见，FLTK 提供了绘制矩形填充（`fl_rectf()`）和矩形轮廓（`fl_rect()`）的函数。默认情况下，两者都画（线条/轮廓画在填充之上）。

13.10 管理未命名对象

到目前为止，我们给自己创建的所有图形对象都起了名字。想要大量对象时，这样做就行不通了。举个例子，让我们为 FLTK 调色板中的 256 种颜色画一张简单的色卡；也就是说，创建 256 个带颜色的正方形，并把它们按 16×16 的矩阵画出来，以展示颜色值相近的颜色之间的关系。首先，结果如下：



给这 256 个正方形逐一起名不仅繁琐，而且愚蠢。左上角正方形显而易见的“名字”就是它在矩阵中的位置 $(0,0)$，任何其他正方形也类似地由坐标对 (i,j) 来标识（即“命名”）。这个例子需要的是与“对象矩阵”等价的东西。我们曾考虑使用 `vector<Rectangle>`，但结果表明它不够灵活。例如，拥有一个元素并非全部同类型的未命名对象（元素）集合会很有用。我们将在 §14.3 讨论这个灵活性问题。这里只给出我们的解决方案：一种既能保存命名对象、也能保存未命名对象的 `vector` 类型：

```
template<class T> class Vector_ref {
public:
    // ...
    void push_back(T&);       // 添加一个命名对象
    void push_back(T*);       // 添加一个未命名对象

    T& operator[](int i);     // 下标访问：读写
    const T& operator[](int i) const;

    int size() const;
};
```

它的用法与标准库 vector 非常相似：

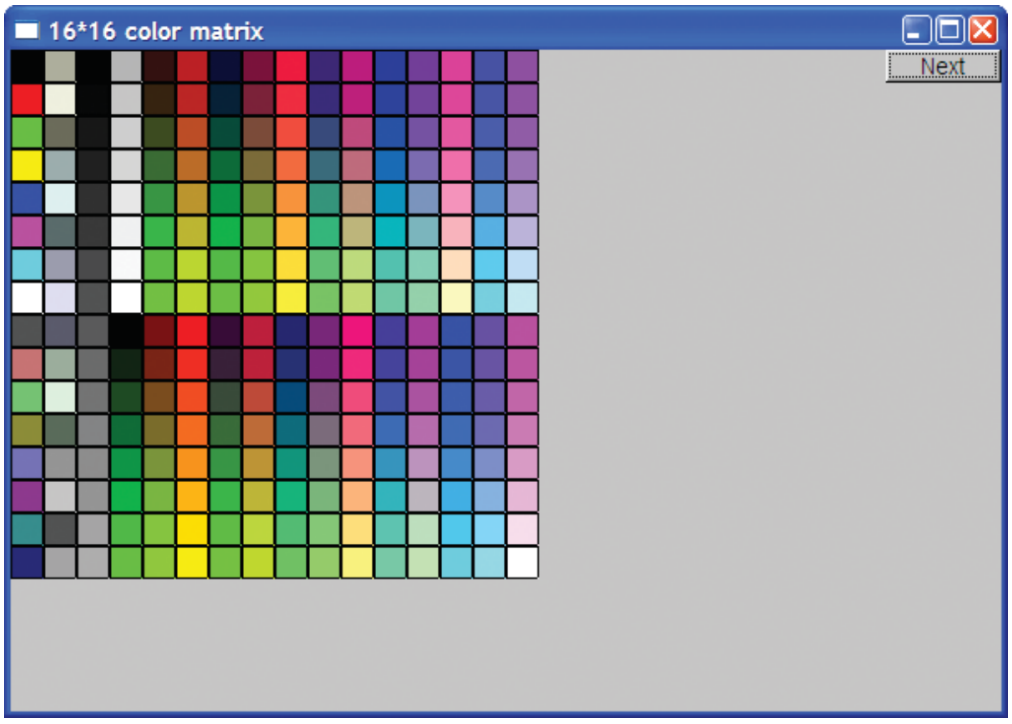
```
Vector_ref<Rectangle> rect;

Rectangle x {Point{100,200},Point{200,300}};
rect.push_back(x);              // 添加命名对象

rect.push_back(new Rectangle{Point{50,60},Point{80,90}});  // 添加未命名对象

for (int i=0; i<rect.size(); ++i) rect[i].move(10,10);     // 使用 rect
```

我们将在第 17 章解释 new 运算符，Vector_ref 的实现则见附录 E。目前只需知道：我们可以用它来保存未命名对象。new 运算符后跟一个类型名（此处为 Rectangle），后面可选地跟一个初始化列表（此处为 {Point{50,60},Point{80,90}}）。有经验的程序员听到我们在这个例子中没有引入内存泄漏，应该会松一口气。

有了 Rectangle 和 Vector_ref，我们就可以玩转颜色了。例如，可以画出前面展示过的那 256 种颜色的色卡：

```
Vector_ref<Rectangle> vr;

for (int i = 0; i<16; ++i)
    for (int j = 0; j<16; ++j) {
        vr.push_back(new Rectangle{Point{i*20,j*20},20,20});
        vr[vr.size()-1].set_fill_color(Color{i*16+j});
        win20.attach(vr[vr.size()-1]);
    }
```

我们创建了一个保存 256 个 Rectangle 的 Vector_ref，在窗口中把它们组织成 16×16 的矩阵。我们给这些 Rectangle 依次指定颜色 0、1、2、3、4 等等。每个 Rectangle 创建之后，就把它附着到窗口上，以便显示出来：

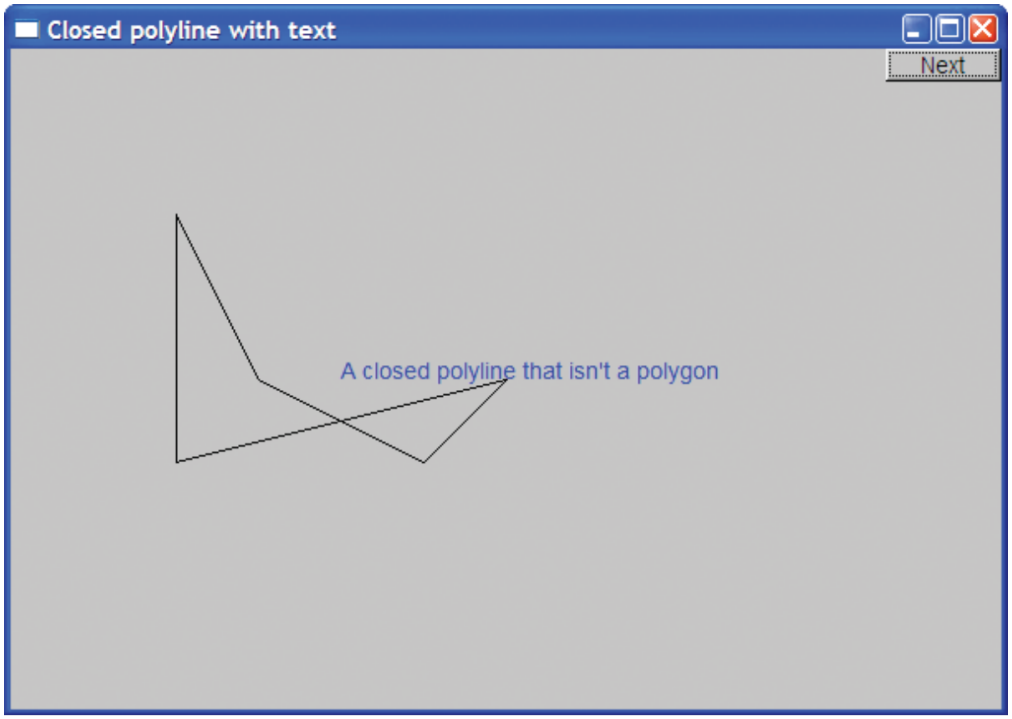


13.11 文本

显然,我们希望能在显示中添加文本。例如,我们可能想给 §13.8 中那个“古怪的”Closed_polyline 加上标注:

```
Text t {Point{200,200},"A closed polyline that isn't a polygon"};
t.set_color(Color::blue);
```

得到



基本上，一个 Text 对象定义了从某个 Point 开始的一行文本。该 Point 是文本的左下角。把字符串限制为单行是为了保证跨系统的可移植性。不要试图在字符串里放进换行符；它在你自己的窗口中未必会被当作换行来处理。字符串流（§11.4）对于组装要在 Text 对象中显示的字符串很有用（例子见 §12.7.7 和 §12.7.8）。Text 的定义如下：

```
struct Text : Shape {
    // 该点是第一个字母的左下角
    Text(Point x, const string& s)
        : lab{s}
        { add(x); }

    void draw_lines() const;

    void set_label(const string& s) { lab = s; }
    string label() const { return lab; }

    void set_font(Font f) { fnt = f; }
    Font font() const { return fnt; }

    void set_font_size(int s) { fnt_sz = s; }
    int font_size() const { return fnt_sz; }

private:
    string lab;     // 标签
    Font fnt {fl_font()};
    int fnt_sz {(fl_size()<14)?14:fl_size()} ;
};
```

如果你想让字体字符尺寸小于 14 或大于 FLTK 的默认值，就必须显式设置。这是“用一个检查来保护用户、使其免受底层库行为变化影响”的一个例子。此处的情况是：FLTK 的一次更新改变了其默认值，导致字符变得极小，从而破坏了既有程序，我们决定防止这个问题的发生。

我们把这两个初始值提供为成员初始化器（member initializer），而不是放在构造函数的初始化列表里，因为它们不依赖于构造函数的参数。

Text 有自己的 draw_lines()，因为只有 Text 类知道它的字符串是如何存储的：

```
void Text::draw_lines() const
{
    fl_draw(lab.c_str(),point(0).x,point(0).y);
}
```

字符颜色的确定方式与由线条构成的图形（如 Open_polyline 和 Circle）中线条的颜色完全一样，因此你可以用 set_color() 选择颜色，用 color() 查看当前使用的颜色。字符大小和字体的处理方式也类似。预定义的字体只有一小批：

```
class Font {   // 字符字体
public:
    enum Font_type {
        helvetica=FL_HELVETICA,
        helvetica_bold=FL_HELVETICA_BOLD,
        helvetica_italic=FL_HELVETICA_ITALIC,
        helvetica_bold_italic=FL_HELVETICA_BOLD_ITALIC,
        courier=FL_COURIER,
        courier_bold=FL_COURIER_BOLD,
        courier_italic=FL_COURIER_ITALIC,
        courier_bold_italic=FL_COURIER_BOLD_ITALIC,
        times=FL_TIMES,
        times_bold=FL_TIMES_BOLD,
        times_italic=FL_TIMES_ITALIC,
        times_bold_italic=FL_TIMES_BOLD_ITALIC,
        symbol=FL_SYMBOL,
        screen=FL_SCREEN,
        screen_bold=FL_SCREEN_BOLD,
        zapf_dingbats=FL_ZAPF_DINGBATS
    };

    Font(Font_type ff) :f{ff} { }
    Font(int ff) :f{ff} { }

    int as_int() const { return f; }
private:
    int f;
};
```

定义 Font 所用的类定义风格，与我们定义 Color（§13.4）和 Line_style（§13.5）时相同。

13.12 圆

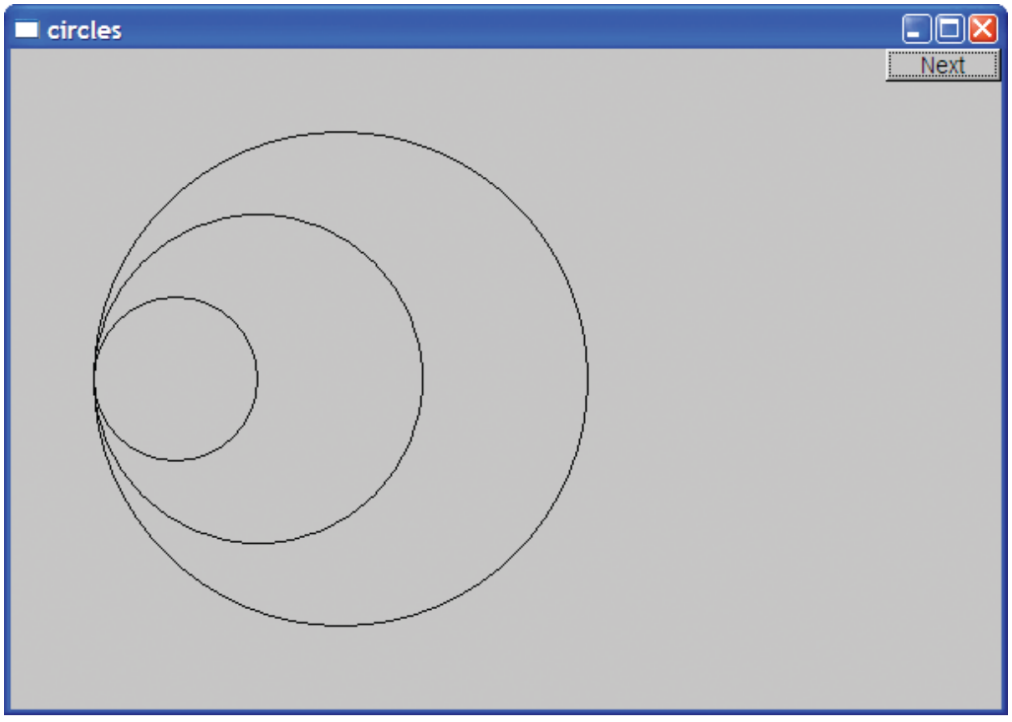
为了表明世界并不完全是矩形的，我们提供了 Circle 类和 Ellipse 类。一个 Circle 由圆心和半径定义：

```
struct Circle : Shape {
    Circle(Point p, int rr);      // 圆心和半径

    void draw_lines() const;

    Point center() const ;
    int radius() const { return r; }
    void set_radius(int rr)
    {
        set_point(0,Point{center().x-rr,center().y-rr});  // 保持
                                                          // 圆心不动
        r = rr;
    }
private:
    int r;
};
```

我们可以这样使用 Circle：

```
Circle c1 {Point{100,200},50};
Circle c2 {Point{150,200},100};
Circle c3 {Point{200,200},150};
```

这会画出三个大小不同的圆，它们的圆心排在一条水平线上：



Circle 实现的主要特别之处在于：保存的点不是圆心，而是圆外切正方形的左上角。两种存法都可以，但我们选择了 FLTK 为其优化的画圆例程所使用的那种。这样，Circle 又提供了一个例子：类可以用来呈现一个概念与实现所不同的（而且据信更好的）视图：

```
Circle::Circle(Point p, int rr)      // 圆心和半径
    :r{rr}
{
    add(Point{p.x-r,p.y-r});         // 保存左上角
}

Point Circle::center() const
{
    return {point(0).x+r, point(0).y+r};
}

void Circle::draw_lines() const
{
    if (color().visibility())
        fl_arc(point(0).x,point(0).y,r+r,r+r,0,360);
}
```

注意用 `fl_arc()` 来画圆。前两个参数指定左上角，接下来两个参数指定包围圆的最小矩形的宽和高，最后两个参数指定要绘制的起始角和终止角。画一个完整的圆要走过整整 360 度，但我们也可以用 `fl_arc()` 画圆的一部分（以及椭圆的一部分）；见习题 1。

13.13 椭圆

椭圆与 `Circle` 类似，但不是用半径定义，而是用长半轴和短半轴定义；也就是说，定义一个椭圆，我们要给出圆心的坐标、圆心到 x 轴上一点的距离，以及圆心到 y 轴上一点的距离：

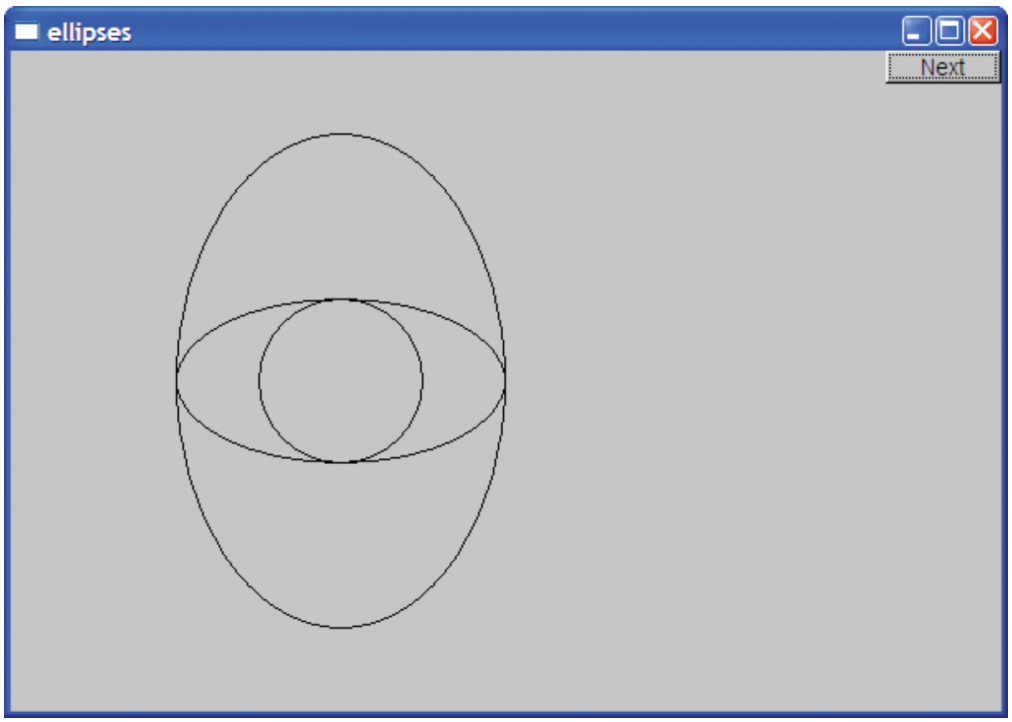
```
struct Ellipse : Shape {
    Ellipse(Point p, int w, int h);    // 圆心，以及距圆心的最大、最小距离

    void draw_lines() const;

    Point center() const;
    Point focus1() const;
    Point focus2() const;

    void set_major(int ww)
    {
        set_point(0,Point{center().x-ww,center().y-h});   // 保持
                                                          // 圆心不动
        w = ww;
    }
    int major() const { return w; }

    void set_minor(int hh)
    {
```

```
        set_point(0,Point{center().x-w,center().y-hh});    // 保持
                                                          // 圆心不动
        h = hh;
    }
    int minor() const { return h; }
private:
    int w;
    int h;
};
```

我们可以这样使用 Ellipse：

```
Ellipse e1 {Point{200,200},50,50};
Ellipse e2 {Point{200,200},100,50};
Ellipse e3 {Point{200,200},100,150};
```

这会画出三个圆心相同但轴长不同的椭圆：



注意，major()==minor() 的椭圆看上去与圆完全一样。

关于椭圆还有一种流行的观点：给定两个焦点，以及一个点到这两个焦点的距离之和。给定一个 Ellipse，我们可以计算出一个焦点。例如：

```
Point focus1() const
{
    if (h<=w)   // 焦点在 x 轴上:
        return {center().x+int(sqrt(double(w*w-h*h))),center().y};
    else        // 焦点在 y 轴上:
        return {center().x,center().y+int(sqrt(double(h*h-w*w)))};
}
```

为什么 Circle 不是 Ellipse？从几何上讲，每个圆都是椭圆，但并非每个椭圆都是圆。特别地，圆是两个焦点重合的椭圆。设想我们把 Circle 定义为 Ellipse 的派生类。这样做付出的代价是其表示中需要一个额外的值（圆由一个点和一个半径定义；椭圆则需要一个圆心和一对轴）。我们不喜欢用不着的空间开销，但 Circle 不作为 Ellipse 的主要原因是：如果不以某种方式禁用 set_major() 和 set_minor()，我们就无法这样定义它。毕竟，如果可以用 set_major() 把它变成 major()!=minor()，它就不再是（数学家所认可的）圆了——至少在我们那样做过之后它就不再是圆了。我们不能拥有一个有时属于一种类型（即 major()!=minor() 时）、有时又属于另一种类型（即 major()==minor() 时）的对象。我们能拥有的是一个有时看上去像圆的对象（一个 Ellipse）。而 Circle 则永远不会变成一个两轴不等的椭圆。

设计类时，我们必须小心，不要自作聪明，也不要被“直觉”误导而定义出在代码中讲不通的类。反过来，我们也必须确保类所表示的是某种连贯的概念，而不是一些数据成员和函数成员的杂凑。不加思考地把代码堆在一起、不去想我们究竟在表示什么概念，那是“乱砍”（hacking），其结果是写出的代码我们自己解释不清，别人也无法维护。如果你不觉得需要为别人着想，请记住：这里的“别人”可能就是几个月后的你自己。这样的代码调试起来也更困难。

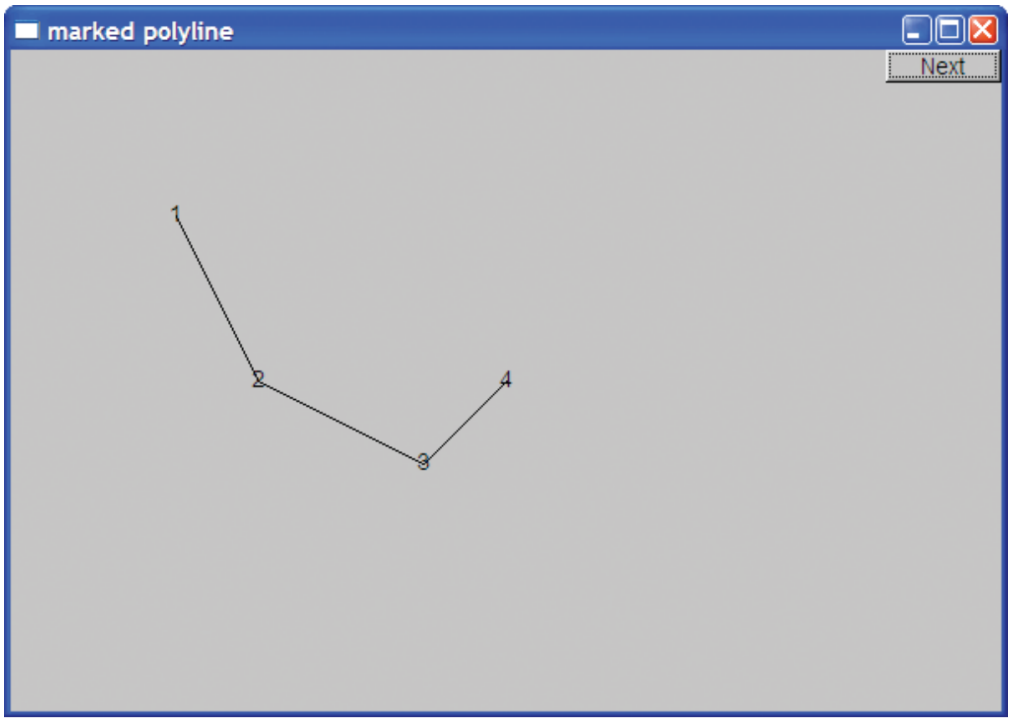
13.14 带标记的折线

我们常常想给图上的点加“标注”。显示图的一种方式是把它画成一条开放折线，因此我们需要的是在各个点处带“标记”的开放折线。Marked_polyline 做的正是这件事。例如：

```
Marked_polyline mpl {"1234"};
mpl.add(Point{100,100});
mpl.add(Point{150,200});
mpl.add(Point{250,250});
mpl.add(Point{300,200});
```

这会画出

Marked_polyline 的定义是




```
struct Marked_polyline : Open_polyline {
    Marked_polyline(const string& m) :mark{m} { if (m=="") mark = "*"; }
    Marked_polyline(const string& m, initializer_list<Point> lst);
    void draw_lines() const;
private:
    string mark;
};
```

通过从 Open_polyline 派生，我们“免费”获得了对 Point 的处理；唯一要做的是处理这些标记。特别是，draw_lines() 变成了

```
void Marked_polyline::draw_lines() const
{
    Open_polyline::draw_lines();
    for (int i=0; i<number_of_points(); ++i)
        draw_mark(point(i),mark[i%mark.size()]);
}
```

调用 Open_polyline::draw_lines() 负责画线，所以我们只需要处理“标记”。我们把标记提供为一个字符串，并按顺序使用它们：mark[i%mark.size()] 通过在创建 Marked_polyline 时提供的字符间循环，选出下一个要使用的字符。% 是模（求余）运算符。这个 draw_lines() 用一个小辅助函数 draw_mark() 在给定的点实际输出一个字母：

```
void draw_mark(Point xy, char c)
{
    constexpr int dx = 4;
    constexpr int dy = 4;

    string m {1,c};   // 保存单个字符 c 的字符串
    fl_draw(m.c_str(),xy.x-dx,xy.y+dy);
}
```

常量 dx 和 dy 用于把字母居中放在点的上方。构造 m 是为了让它只包含单个字符 c。

接受初始化列表的构造函数只是把列表转发给 Open_polyline 的初始化列表构造函数：

```
Marked_polyline(const string& m, initializer_list<Point> lst)
        :Open_polyline{lst},
        mark{m}
{
    if (m=="") mark = "*";
}
```

对空字符串的检查是必要的，以免 draw_lines() 试图访问一个并不存在的字符。

有了接受初始化列表的构造函数，我们可以把上面的例子简写为

```
Marked_polyline mpl {"1234",{{100,100}, {150,200}, {250,250}, {300,200}}};
```

13.15 Marks

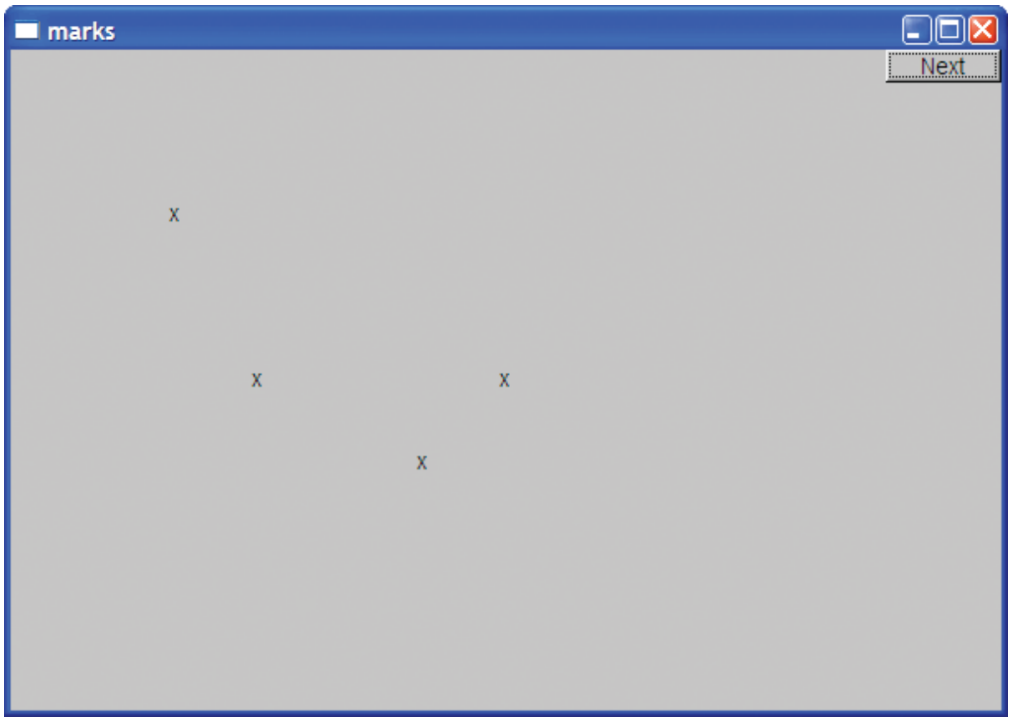
有时我们想显示标记但不用线把它们连起来。为此我们提供了 Marks 类。例如，我们可以把前面各个例子中用过的那四个点标记出来，而不用线连接：

```
Marks pp {"x",{{100,100}, {150,200}, {250,250}, {300,200}}};
```

这会画出

Marks 的一个明显用途是显示表示离散事件的数据——此时用线连接是不合适的。一个例子是一群人的（身高，体重）数据。

Marks 其实就是一个线条不可见的 Marked_polyline：




```
struct Marks : Marked_polyline {
    Marks(const string& m)
        :Marked_polyline{m}
    {
        set_color(Color{Color::invisible});
    }

    Marked_polyline(const string& m, initializer_list<Point> lst)
        : Marked_polyline{m,lst}
    {
        set_color(Color{Color::invisible});
    }
};
```

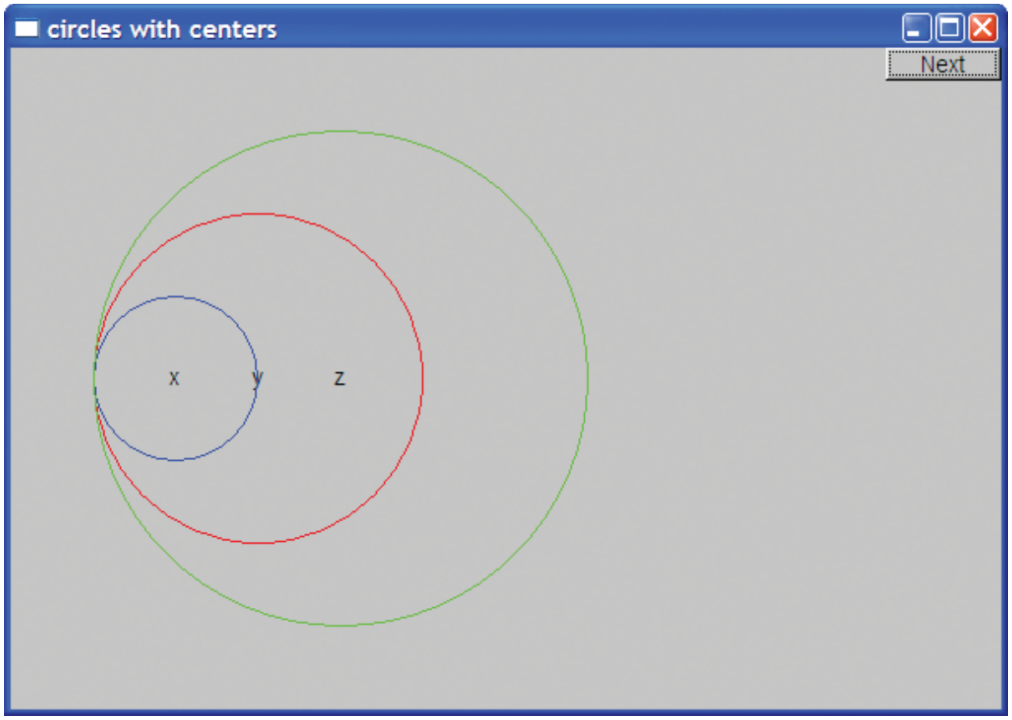
记号 `:Marked_polyline{m}` 用来初始化 `Marks` 对象中的 `Marked_polyline` 部分。这一记号是初始化成员所用语法（§9.4.4）的一个变体。

13.16 Mark

`Point` 只是 `Window` 中的一个位置。它既不是我们绘制的东西，也不是我们能看见的东西。如果想标记单个 `Point` 使我们能看见它，可以像 §13.2 那样用一对线来指示，或者使用 `Marks`。这有点罗嗦，所以我们有一个 `Marks` 的简化版本，用一个点和一个字符来初始化。例如，我们可以这样标记 §13.12 中各圆的圆心：

```
Mark m1 {Point{100,200},'x'};
Mark m2 {Point{150,200},'y'};
Mark m3 {Point{200,200},'z'};
c1.set_color(Color::blue);
c2.set_color(Color::red);
c3.set_color(Color::green);
```

这会画出



Mark 其实就是一个 Marks，只不过它的初始（通常也是唯一的）点在构造时立即给出：

```
struct Mark : Marks {
    Mark(Point xy, char c) : Marks{string{1,c}}
    {
        add(xy);
    }
};
```

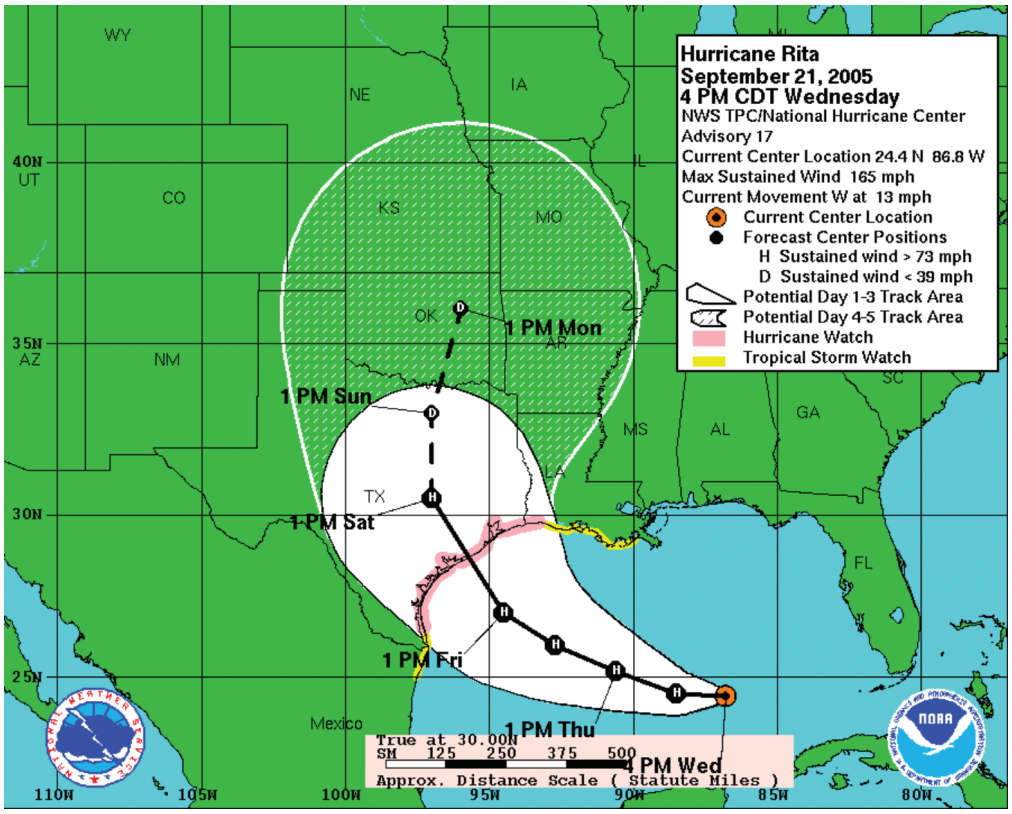
string{1,c} 是 string 的一个构造函数，把字符串初始化为只包含单个字符 c。

Mark 提供的全部功能，就是用一种便利的记号来创建一个 Marks 对象，其中唯一的一个点用单个字符标记。Mark 值得我们费劲去定义吗？还是它只是“多余且令人困惑的东西”？对此并没有一个明确的、合乎逻辑的答案。我们在这个问题上反复权衡过，但最终还是认为它对用户有用，而且定义它的工作量极小。

为什么用一个字符作为“标记”？我们本可以用任何小的形状，但字符提供了一套有用而简单的标记。能够使用多种“标记”来区分不同的点集合，常常是很有用的。诸如 x、o、+ 和 * 这样的字符围绕中心漂亮地对称。

13.17 图像

一台普通的个人计算机会在文件里保存成千上万幅图像，而且还可以通过网络访问数百万幅。自然，我们希望即使在相当简单的程序中也能显示其中一些图像。例如，下面是一幅图像（rita_path.gif），显示了飓风“丽塔”逼近得克萨斯湾海岸时的预计路径：



我们可以选择该图像的一部分，并把一张从太空中拍摄的“丽塔”的照片（rita.jpg）添加进来：

```
Image rita {Point{0,0},"rita.jpg"};
Image path {Point{0,0},"rita_path.gif"};
path.set_mask(Point{50,250},600,400);      // 选择可能的登陆区域

win.attach(path);
win.attach(rita);
```

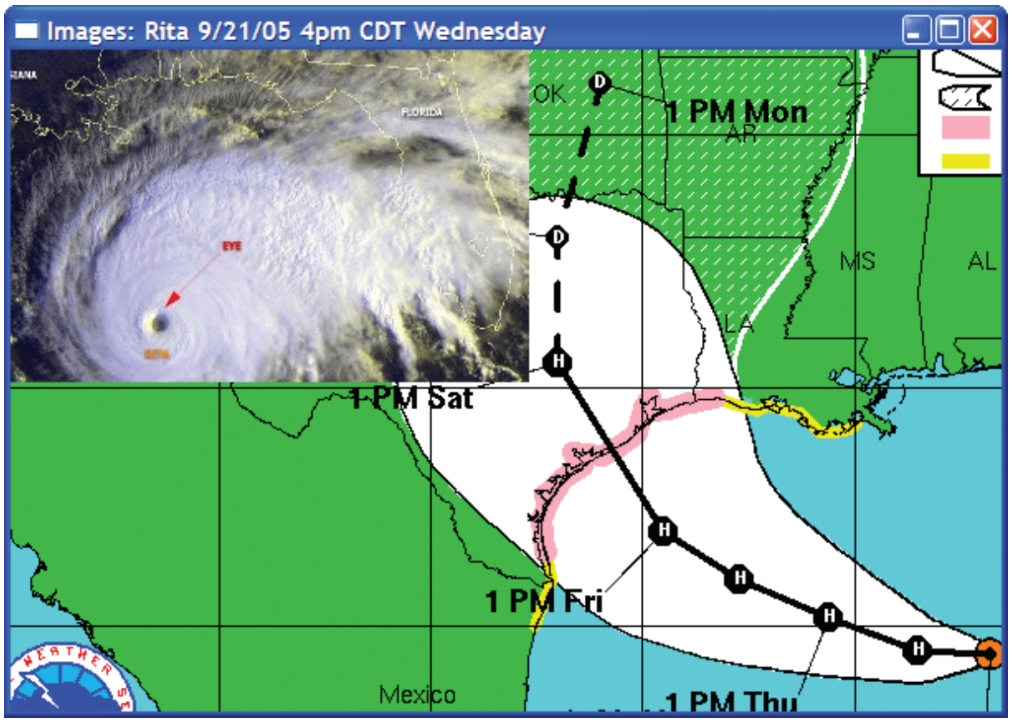
set_mask() 操作选择图像的一个子图片来显示。这里，我们从 rita_path.gif（加载为 path）中选出一幅 $(600, 400)$ 像素的图像，其最左上点位于 path 的点 $(50, 250)$。选择图像的一部分来显示实在太常见了，所以我们决定直接支持它。

图形按其被附着的顺序铺放，就像桌上一张张纸一样，所以我们把 path 放在“底层”的方法就是先于 rita 附着它。

图像可以用五花八门的格式编码。这里我们只处理最常见的两种，JPEG 和 GIF：

```
enum class Suffix { none, jpg, gif };
```

在我们的图形接口库中，内存中的图像表示为 Image 类的一个对象：




```
struct Image : Shape {
    Image(Point xy, string file_name, Suffix e = Suffix::none);
    ~Image() { delete p; }
    void draw_lines() const;
    void set_mask(Point xy, int ww, int hh)
                { w=ww; h=hh; cx=xy.x; cy=xy.y; }
private:
    int w,h;      // 相对位置 (cx,cy) 在图像内定义“掩码框”
    int cx,cy;
    Fl_Image* p;
    Text fn;
};
```

Image 的构造函数会尝试打开以其名字给定的文件。然后，它尝试用指定的编码来创建一幅图片，编码由可选参数给出，或者（更常见的）由文件后缀给出。如果图像无法显示（例如，找不到文件），Image 就显示一个 Bad_image。Bad_image 的定义如下：

```
struct Bad_image : Fl_Image {
    Bad_image(int h, int w) : Fl_Image{h,w,0} { }
    void draw(int x,int y, int, int, int, int) { draw_empty(x,y); }
};
```

图形库中对图像的处理相当复杂，但我们的图形接口类 Image 的主要复杂性在于构造函数中的文件处理：

```
// 有点过于繁琐的构造函数
// 因为图像文件相关的错误调试起来太痛苦了
Image::Image(Point xy, string s, Suffix e)
    :w{0}, h{0}, fn{xy,""}
{
    add(xy);

    if (!can_open(s)) {          // 我们能打开 s 吗？
        fn.set_label("cannot open \""+s+'"');
        p = new Bad_image(30,20);    // “错误图像”
        return;
    }

    if (e == Suffix::none) e = get_encoding(s);

    switch(e) {                  // 检查它是不是已知的编码
    case Suffix::jpg:
        p = new Fl_JPEG_Image{s.c_str()};
        break;
    case Suffix::gif:
        p = new Fl_GIF_Image{s.c_str()};
        break;
    default:                     // 不支持的图像编码
        fn.set_label("unsupported file type \""+s+'"');
        p = new Bad_image{30,20};    // “错误图像”
    }
}
```

我们用后缀来选择创建哪种对象来保存图像（`Fl_JPEG_Image` 或 `Fl_GIF_Image`）。我们用 `new` 创建该实现对象，并将其赋给一个指针。这是一个实现细节（关于运算符 `new` 和指针的讨论见第 17 章），它与 FLTK 的组织方式有关，在这里不具有根本性的重要意义。FLTK 使用 C 风格字符串，所以我们必须用 `s.c_str()` 而不是 `s` 本身。

现在，我们只需实现 `can_open()`，以测试能否打开一个具名文件进行读取：

```
bool can_open(const string& s)
    // 检查名为 s 的文件是否存在并可打开读取
{
    ifstream ff(s);
    return ff;
}
```

打开一个文件然后又把它关掉，是一种相当笨拙的办法，其用途是以可移植的方式把“打不开文件”这类错误与文件中数据格式相关的错误区分开来。

如果你有兴趣，可以查一查 `get_encoding()` 函数。它只是寻找一个后缀，然后在一个已知后缀的表中查找该后缀。那个查找表是一个标准库 `map`（见 §21.6）。

小试身手

1. 创建一个 800 × 1000 的 Simple_window。
2. 在该窗口最左侧的 800 × 800 部分放一个 8 × 8 的网格（这样每个正方形就是 100 × 100）。
3. 把从左上角开始沿对角线的八个正方形涂成红色（使用 Rectangle）。
4. 找一幅 200 × 200 像素的图像（JPEG 或 GIF），把它的三个副本放到网格上（每幅图像覆盖四个正方形）。如果找不到正好 200 见方的图像，就用 set_mask() 从一幅更大的图像中选出 200 × 200 的部分。不要遮住红色正方形。
5. 添加一幅 100 × 100 的图像。单击“Next”按钮时，让它从一个正方形移动到另一个正方形。只需把 wait_for_button() 放进一个循环，配上一些为图像挑一个新正方形的代码即可。

要点回顾

1. 为什么我们不“干脆”直接使用商业或开源图形库？
2. 使用我们的图形接口库做简单的图形输出，大约需要多少个类？
3. 使用图形接口库需要哪些头文件？
4. 哪些类定义的是封闭图形？
5. 为什么我们不干脆对每种形状都用 Line？
6. Point 的参数指什么？
7. Line_style 由哪些部分组成？
8. Color 由哪些部分组成？
9. 什么是 RGB？
10. 两条 Line 与一个包含两条线的 Lines 有何区别？
11. 对每个 Shape 可以设置哪些属性？
12. 由五个 Point 定义的 Closed_polyline 有多少条边？
13. 如果你定义了一个 Shape 但没有把它附着到 Window 上，会看到什么？
14. Rectangle 与有四个 Point（角点）的 Polygon 有何不同？
15. Polygon 与 Closed_polyline 有何不同？
16. 哪个在上：填充还是轮廓？
17. 为什么我们没有费心定义一个 Triangle 类（毕竟我们定义了 Rectangle）？
18. 如何把一个 Shape 移到 Window 中的另一个位置？
19. 如何用一行文本给 Shape 加标注？
20. 对 Text 中的一个文本串可以设置哪些属性？
21. 什么是字体？我们为什么关心它？
22. Vector_ref 是干什么的？如何使用它？
23. Circle 与 Ellipse 有什么区别？
24. 如果给定的文件名并不指向一个包含图像的文件，试图显示该 Image 会发生什么？
25. 如何显示图像的一部分？

术语

封闭图形（closed shape） 颜色（color） 椭圆（ellipse） 填充（fill） 字体（font）
字体大小（font size） GIF 图像（image） 图像编码（image encoding） 不可见（invisible）
JPEG 线（line） 线型（line style） 开放图形（open shape） 点（point）
多边形（polygon） 折线（polyline） 未命名对象（unnamed object） Vector_ref 可见（visible）

习题

对每道“定义一个类”的习题，请显示该类的几个对象，以证明它们能正常工作。

1. 定义一个类 `Arc`，画椭圆的一部分。提示：`fl_arc()`。
2. 画一个圆角方框。定义一个类 `Box`，由四条线和四段弧组成。
3. 定义一个类 `Arrow`，画一条带头部的线。
4. 定义函数 `n()`、`s()`、`e()`、`w()`、`center()`、`ne()`、`se()`、`sw()` 和 `nw()`。每个函数接受一个 `Rectangle` 参数并返回一个 `Point`。这些函数定义了矩形上和矩形内的“连接点”。例如，`nw(r)` 是名为 `r` 的 `Rectangle` 的西北（左上）角。
5. 对 `Circle` 和 `Ellipse` 定义习题 4 中的那些函数。把连接点放在图形上或图形外，但不要越出外接矩形。
6. 编写一个程序，画出类似 §12.6 中的类图。如果你先定义一个 `Box` 类——一个带文本标签的矩形——事情会简单一些。
7. 制作一张 RGB 色卡（例如，在网上搜索“RGB color chart”）。
8. 定义一个类 `Regular_hexagon`（正六边形是六条边长度都相等的六边形）。以圆心以及圆心到一个角点的距离作为构造函数参数。
9. 用 `Regular_hexagon` 平铺窗口的一部分（至少用八个六边形）。
10. 定义一个类 `Regular_polygon`。以圆心、边数（`>2`）以及圆心到一个角的距离作为构造函数参数。
11. 画一个 300×200 像素的椭圆。过椭圆的中心画一条 400 像素长的 x 轴和一条 300 像素长的 y 轴。标出两个焦点。标出椭圆上不在轴上的一个点。画出从两个焦点到该点的两条线。
12. 画一个圆。让一个标记在圆上移动（每按一次“Next”按钮就移动一点）。
13. 画出 §13.10 的颜色矩阵，但每个颜色块周围不要画线。
14. 定义一个直角三角形类。用八个不同颜色的直角三角形拼出一个八边形。
15. 用小直角三角形“平铺”一个窗口。
16. 重做上一题，但用六边形。
17. 重做上一题，但使用几种不同颜色的六边形。
18. 定义一个类 `Poly`，表示多边形，但在其构造函数中检查它的点确实构成一个多边形。提示：你必须把点提供给构造函数。
19. 定义一个类 `Star`。其中一个参数应是角点数。画出几颗角点数、线色和填充色各不相同的星形。

后记

第 12 章展示了如何做类的用户。本章把我们沿着程序员的“食物链”向上提升了一级：在这里，我们除了做工具的使用者，还成为了工具的建造者。

14 图形类设计

“功能实用、经久耐用、赏心悦目。”
（原文：*Functional, durable, beautiful.*）
——维特鲁威（Vitruvius）

图形学这两章的目的有两个：我们既想提供用于显示信息的实用工具，又想用这组图形界面类来演示通用的设计与实现技术。具体来说，本章会介绍一些界面设计的想法，以及继承这一概念。为了讲清楚，我们还得稍稍绕个弯，考察那些最直接支持面向对象编程的语言特性：类派生、虚函数和访问控制。我们并不认为设计可以脱离使用和实现来讨论，所以本书对设计的讨论相当具体。也许你最好把本章理解为“图形类的设计与实现”。

14.1 设计原则
- **14.1.1** 类型
- **14.1.2** 操作
- **14.1.3** 命名
- **14.1.4** 可变性

14.2 Shape
- **14.2.1** 抽象类
- **14.2.2** 访问控制
- **14.2.3** 绘制形状
- **14.2.4** 复制与可变性

14.3 基类与派生类
- **14.3.1** 对象布局
- **14.3.2** 派生类与定义虚函数
- **14.3.3** 覆盖
- **14.3.4** 访问
- **14.3.5** 纯虚函数

14.4 面向对象编程的好处

14.1 设计原则

我们的图形界面类的设计原则是什么？首先：这是个什么样的问题？“设计原则”究竟是什么，为什么我们要先看这些，而不是赶紧去干“画漂亮图”这件正经事？

14.1.1 类型

图形学是应用领域的一个例子。因此，我们在这里看到的，是如何把一组基本的应用概念和设施呈现给（像我们这样的）程序员的一个示例。如果这些概念被呈现得令人困惑、前后不一致、不完整，或者以其他方式在我们的代码中表达得很糟，那么生成图形输出的难度就会增加。我们希望自己的图形类能尽量减少程序员学习和使用它们所花的力气。

我们理想中的程序设计，是把应用领域的概念直接表示在代码里。这样，如果你理解这个应用领域，你也就理解代码；反过来也一样。例如：

- Window——操作系统所呈现的窗口
- Line——你在屏幕上看到的一条线
- Point——一个坐标点
- Color——你在屏幕上看到的颜色
- Shape——在我们的图形/图形用户界面世界观里，所有形状共同具备的东西

最后一个例子 Shape 与其他几个不同：它是一种泛化，一个纯粹抽象的概念。我们在屏幕上从来不会只看到一个“形状”；我们看到的总是某个具体的形状，比如一条线或一个六边形。你会发现，我们的类型定义也反映了这一点：试着创建一个 Shape 变量，编译器就会拦住你。

我们这组图形界面类构成一个库；这些类是要放在一起、组合起来使用的。它们也想作为范例，供你定义表示其他图形的类时参照，并作为这类类的构件。我们并不只是在定义一组互不相关的类，所以不能孤立地为每个类做设计决定。我们的类合在一起，呈现了一种“怎样做图形”的观点。我们必须保证这个观点足够优雅、足够连贯。考虑到这个库的规模，以及图形应用领域的庞大，我们不能指望完备。我们追求的是简单性和可扩展性。

事实上，没有任何类库能直接为它的应用领域的全部方面建模。这不仅不可能，而且没有意义。设想你要写一个显示地理信息的库。你想显示植被吗？国界、州界以及其他政治边界？公路系统？铁路？河流？突出显示社会与经济数据？气温和湿度的季节变化？上空的风场模式？航线？标出学校的位置？快餐“餐厅”的位置？风景名胜？“全都要！”对于一个综合性的地理应用来说也许是个好答案，但对单幅显示来说并不是答案。对于支持这类地理应用的库来说，它也许算个答案，但这样一个库不太可能还覆盖其他图形应用，比如徒手绘图、编辑照片图像、科学可视化和飞机控制显示。

所以，和以往一样，我们必须决定什么对我们重要。在这个例子里，我们必须决定想做好的是哪一种图形/图形用户界面。什么都想做，是失败的配方。一个好的库会从某个特定视角直接而清晰地为其应用领域建模，强调应用的某些方面，弱化其他方面。

我们在这里提供的类是为简单图形和简单图形用户界面而设计的。它们主要面向需要展示数据、展示来自数值/科学/工程应用的图形输出的用户。你可以在我们的类“之上”构建自己的类。如果这还不够，我们在实现中暴露了足够多的 FLTK 细节，让你了解怎样直接使用那个（或类似的“全功能”图形/图形用户界面）库——如果你愿意这样做的话。不过，如果你决定走这条路，请等到学完第 17 章和第 18 章。那两章包含关于指针和内存管理的信息，而成功直接使用大多数图形/图形用户界面库正需要这些知识。

一个关键的决定是提供大量“小”类，每个类只有很少的操作。例如，我们提供了 Open_polyline、Closed_polyline、Polygon、Rectangle、Marked_polyline、Marks 和 Mark，而本来我们可以只提供一个类（也许叫“polyline”），给它一大堆参数和操作，让我们指定某个对象是哪一种 polyline，甚至可以干脆把一个 polyline 从一种变成另一种。这种思路走到极端，就是把每一种形状都作为单个 Shape 类的一部分来提供。我们认为，使用许多小类最能贴切而有用地为我们的图形领域建模。一个“什么都有”的单个类，只会让用户陷在数据和选项里，而没有帮助理解、调试和提升性能的框架。

14.1.2 操作

我们为每个类只提供最少的操作。我们的理想是：接口最小，却仍能让我们完成想做的事。在需要更多便利的地方，我们总可以通过增加非成员函数，或者再加一个类来提供。

我们希望自己类的接口呈现出统一的风格。例如，不同类中执行类似操作的函数都取相同的名字、接受相同类型的参数，并且在可能时按相同的顺序要求这些参数。看看构造函数：如果一个形状需要一个位置，它就把 Point 作为第一个参数：

```
Line ln {Point{100,200},Point{300,400}};
Mark m {Point{100,200},'x'};       // 把单个点显示为 'x'
Circle c {Point{200,200},250};
```

处理点的所有函数都用 Point 类来表示点。这看起来理所当然，但许多库表现出混杂的风格。例如，设想一个画线的函数。我们可以采用两种风格之一：

```
void draw_line(Point p1, Point p2);     // 从 p1 到 p2（我们的风格）
void draw_line(int x1, int y1, int x2, int y2);     // 从 (x1,y1) 到 (x2,y2)
```

我们甚至可以两种都允许，但为了一致性、更好的类型检查和更好的可读性，我们只采用第一种风格。一致地使用 Point 还能让我们不至于把坐标对与另一种常见的整数对——宽度和高度——搞混。例如，看：

```
draw_rectangle(Point{100,200}, 300, 400);     // 我们的风格
draw_rectangle(100,200,300,400);     // 另一种写法
```

第一次调用用一个点、一个宽度和一个高度来画矩形。这还算容易猜出来，但第二次调用呢？那是由点 (100,200) 和 (300,400) 定义的矩形？还是由点 (100,200)、宽度 300、高度 400 定义的矩形？或者是某种完全不同的东西（尽管对某些人来说似乎也说得通）？一致地使用 Point 类型就避免了这种混乱。

顺带一提，如果一个函数需要宽度和高度，它们总是按这个顺序出现（就像我们总是在 y 坐标之前给出 x 坐标一样）。把这些小细节做一致，对易用性和避免运行期错误的帮助之大，出人意料。

逻辑上相同的操作取相同的名字。例如，每个把点、线等添加到某种形状上的函数都叫 add()，每个画线的函数都叫 draw_lines()。这种一致性帮助我们记忆（要记的细节更少），也帮助我们在设计新类时“照常规办”。有时，它甚至让我们能写出对许多不同类型都适用的代码，因为这些类型上的操作具有相同的模式。这样的代码称为泛型代码；参见第 19 章至第 21 章。

14.1.3 命名

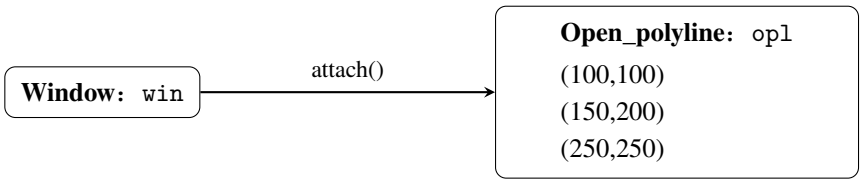
逻辑上不同的操作取不同的名字。这同样看似显然，但请想一想：为什么我们把 Shape “附着”（attach）到一个 Window 上，却把 Line “添加”（add）到一个 Shape 里？两种情况都是“把某个东西放进另一个东西里”，那么这种相似性不应该用一个共同的名字来体现吗？不应该。这种相似性掩盖了一个根本区别。看：

```
Open_polyline opl;
opl.add(Point{100,100});
opl.add(Point{150,200});
opl.add(Point{250,250});
```

这里，我们把三个点复制进 opl。调用 add() 之后，形状 opl 并不关心“我们的”那些点；它保留自己的副本。事实上我们很少自己留着点的副本——那件事交给形状去做。另一方面，看：

```
win.attach(opl);
```

这里，我们在窗口 win 和我们的形状 opl 之间建立了一种连接；win 并不复制 opl——它保留对 opl 的引用。所以，只要 win 还在使用 opl，让 opl 保持有效就是我们的责任。也就是说，当 win 正在使用 opl 时，我们绝不能退出 opl 的作用域。我们可以更新 opl，等 win 下次绘制 opl 时，我们的修改就会出现在屏幕上。我们可以用下图来说明 attach() 与 add() 的区别：



基本上，add() 采用值传递（复制），而 attach() 采用引用传递（共享同一个对象）。我们本可以选择把图形对象复制进 Window。但那样会带来一种不同的编程模型，而那种情况我们会用 add() 而不是 attach() 来表示。现在这样，我们只是把一个图形对象“附着”到一个 Window 上。这有重要的影响。例如，我们不能创建一个对象、把它附着上去、再让这个对象被销毁，然后指望结果程序还能正常工作：

```
void f(Simple_window& w)
{
    Rectangle r {Point{100,200},50,30};
    w.attach(r);
}    // 糟糕，r 的生命期到此结束

int main()
{
    Simple_window win {Point{100,100},600,400,"My window"};
    // . . .
    f(win);    // 自找麻烦
    // . . .
    win.wait_for_button();
}
```

等到我们从 f() 退出、走到 wait_for_button() 时，win 已经没有 r 可以引用和显示了。第 17 章会展示怎样在函数内部创建对象，并让它们在函数返回之后仍然存在。在那之前，我们必须避免附着那些活不到 wait_for_button() 调用的对象。我们有 Vector_ref（§13.10、§E.4）来帮忙。

注意，如果我们把 f() 声明为接受一个 const 引用的 Window 参数（正如 §8.5.6 所建议的），编译器本来是会阻止我们犯这个错误的：我们无法把 r 附着到一个 const Window 上，因为 attach() 需要修改 Window，以记录该 Window 对 r 的关注。

14.1.4 可变性

设计一个类时，“谁能修改数据（表示）?”和“怎样修改?”是我们必须回答的关键问题。我们尽量保证：对象状态的修改只由它自己的类来做。public/private 的区分对此至关重要，但后面我们也会展示需要更灵活、更微妙机制（protected）的例子。这意味着，我们不能只是给类一个数据成员——比如说一个叫 label 的 string——还必须考虑：构造之后是否应该允许修改它？如果允许，又怎样修改？我们还必须决定：除了本类的成员函数之外，是否需要有别的代码读取 label 的值；如果需要，又该怎样读。例如：

```
struct Circle {
    // . . .
private:
    int r;    // 半径
};


Circle c {Point{100,200},50};
c.r = -9;    // 可以吗？不行——编译期错误：Circle::r 是私有的
```

你可能在第 13 章已经注意到，我们决定阻止对大多数数据成员的直接访问。不直接暴露数据，使我们有机会检查“愚蠢”的值，比如半径为负的 Circle。为了实现的简洁，我们只有限地利用了这个机会，所以使用数值时务必小心。我们没有一贯而彻底地做检查，这既反映了想为讲解而保持代码简短的愿望，也因为我们知道：如果用户（你或我们）提供了“愚蠢”的值，结果不过是屏幕上出现一幅乱糟糟的图像，而不会破坏宝贵的数据。

我们把屏幕（看作一组 Window）纯粹当作输出设备。我们可以显示新对象、移除旧对象，但从不向“系统”询问我们自己不知道（或无法从我们为表示图像而建立的数据结构中知道）的信息。

14.2 Shape

Shape 类表示“可以出现在屏幕上的 Window 中的东西”这一一般概念：

- 它是把我们的图形对象与 Window 抽象联系起来的那个概念，而 Window 又提供了到操作系统和物理屏幕的连接。
- 它是处理颜色和画线所用线型的类。为此，它持有一个 Line_style、一个用于线条的 Color，以及一个用于填充的 Color。
- 它可以持有一串 Point，并且具有关于怎样绘制它们的基本概念。

有经验的设计者会看出，一个类同时做三件事，很可能在通用性上有问题。不过在这里，我们需要的方案要远比最一般的解法简单得多。

我们先给出完整的类，然后再讨论它的细节：

```
class Shape {    // 处理颜色和线型，并保存一个线序列
public:
    void draw() const;    // 处理颜色并绘制线条
    virtual void move(int dx, int dy);    // 将形状平移 +=dx、+=dy

    void set_color(Color col);
    Color color() const;

    void set_style(Line_style sty);
    Line_style style() const;

    void set_fill_color(Color col);
    Color fill_color() const;

    Point point(int i) const;    // 对点的只读访问
    int number_of_points() const;

    Shape(const Shape&) = delete;    // 防止复制
    Shape& operator=(const Shape&) = delete;

    virtual ~Shape() { }
protected:
    Shape() { }
    Shape(initializer_list<Point> lst);    // 把 lst 中的 Point 添加到此 Shape

    virtual void draw_lines() const;    // 绘制相应的线条
    void add(Point p);    // 把 p 加到 points
    void set_point(int i, Point p);    // points[i]=p;
private:
    vector<Point> points;    // 并非所有形状都用到
    Color lcolor {fl_color()};    // 线条与字符的颜色（带默认值）
    Line_style ls {0};
    Color fcolor {Color::invisible};    // 填充色
};
```

这是一个相对复杂的类，设计它是为了支持各种各样的图形类，并表示屏幕上“形状”这个一般概念。然而它仍然只有四个数据成员和 15 个函数。此外，这些函数都近乎平凡，好让我们把注意力集中在设计问题上。在本节余下的部分，我们会逐个走过这些成员，解释它们在设计中扮演的角色。

14.2.1 抽象类

```
protected:
    Shape() { }
    Shape(initializer_list<Point> lst);    // 把 Points 加入这个 Shape
```

我们来看 Shape 的构造函数。它们被声明为 protected，也就是说，只能在从 Shape 派生的类里直接使用（即用 :Shape 记法）。换言之，Shape 只能充当 Line、Open_polyline 这类类的基类。那个 protected: 的用意，是确保我们不会直接创建 Shape 对象。例如：

```
Shape ss;    // 错误：不能构造 Shape
```

Shape 是专门设计成基类的。就本例而言，如果允许人们直接创建 Shape 对象，也并不会出什么特别糟糕的事；但通过限制用法，我们为 Shape 日后可能的修改留出了余地——那些修改可能让它不适合直接使用。此外，禁止直接创建 Shape 对象，正好直接刻画了这样一个想法：我们不可能拥有或看到“一个笼统的形状”，能看到的只有具体的形状，例如 Circle 和 Closed_polyline。想一想！一个形状长什么样？唯一合理的回答是反问一句：“哪个形状？”我们用 Shape 表示的“形状”这一概念是抽象的。这是一个重要且常用的设计观念，所以我们不愿在程序里损害它。允许用户直接创建 Shape 对象，会破坏我们的理想：类应当直接表示概念。

默认构造函数把各成员设为默认值。这里又一次“露出”了实现所用的底层库 FLTK 的痕迹。不过，FLTK 对颜色和样式的概念并未被使用者直接提及；它们只是我们的 Shape、Color 和 Line_style 类实现的一部分。vector<Point> 默认是空向量。

接受初始化列表的构造函数同样使用这些默认初始化器，然后把其实参列表中的元素逐个 add() 进 Shape：

```
Shape::Shape(initializer_list<Point> lst)
{
    for (Point p : list) add(p);
}
```

一个类如果只能用作基类，它就是抽象类（abstract class）。另一种更常见的达到这个效果的办法叫纯虚函数（pure virtual function），见 §14.3.5。能用来创建对象的类——也就是抽象类的反面——叫具体类（concrete class）。注意，抽象和具体不过是给一种日常区别起的技术名字。我们可能去商店买相机。可是我们不能只说“要一台相机”就把它拿回家。什么牌子的相机？哪个具体型号？camera 这个词是个概括，指的是一个抽象概念。Olympus E-M5 指一种具体的相机，我们（用一大笔钱）可以买到它的某个特定实例：一台带唯一序列号的相机。所以，“相机”很像抽象（基）类；“Olympus E-M5”很像具体（派生）类；而我手里那台实际的相机（如果我买了的话）就很像一个对象。

这个声明是：

```
virtual ~Shape() { }
```

它定义了一个虚析构函数。目前我们不会用到它，所以把解释留到 §17.5.2，那里会展示一个用例。

14.2.2 访问控制

```
private:
    vector<Point> points;
    Color lcolor {fl_color()};    // 线条和字符的颜色（带默认值）
    Line_style ls {0};
    Color fcolor {Color::invisible};    // 填充色
```

Shape 把所有数据成员都声明为 private。这些成员的初始化器并不依赖构造函数实参，所以我直接在成员声明处给出了它们。和往常一样，vector 的默认值是“空”，因此不必显式写出；构造函数会套用这些默认值。

由于 Shape 的数据成员声明为 private，我们需要提供访问函数。做法有几种风格，我们选了一种自认为简单、方便且易读的：如果有一个成员表示属性 X，就提供一对函数 X() 和 set_X()，分别用于读取和写入。例如：

```
void Shape::set_color(Color col)
{
    lcolor = col;
}

Color Shape::color() const
{
    return lcolor;
}
```

这种风格的主要不便之处在于：不能给成员变量起与读取函数相同的名字。和往常一样，我们给函数挑了最方便的名字，因为它们是公有接口的一部分；私有变量叫什么名字则无关紧要得多。请注意我们用 const 表明读取函数不会修改自己的 Shape（§9.7.4）。

Shape 保存一个 Point 的 vector，名为 points，供派生类使用。我们提供函数 add() 把 Point 加入 points：

```
void Shape::add(Point p)    // 受保护
{
    points.push_back(p);
}
```

自然，points 一开始是空的。我们决定给 Shape 提供一套完整的功能接口，而不是让用户——甚至包括 Shape 派生类的成员函数——直接访问数据成员。对一些人来说，提供功能接口是理所当然的，因为他们认为把类的任何数据成员设为公有都是坏设计。对另一些人来说，我们的设计显得过于严格，因为我们不允许对派生类的所有成员直接进行写访问。

从 Shape 派生的形状（例如 Circle 和 Polygon）知道自己那些点是什么意思。基类 Shape 并不“理解”这些点，它只是保存它们。因此，派生类需要控制点是怎样加入的。例如：

- Circle 和 Rectangle 不允许用户添加点；那根本说不通。一个矩形多出一个点算什么？（§12.7.6）
- Lines 只允许成对添加点（不能添加单个点；§13.3）。
- Open_polyline 和 Marks 允许添加任意多个点。
- Polygon 只允许通过检查交点的 add() 来添加点（§13.8）。

我们把 add() 设为受保护的（即只能从派生类访问），以确保派生类掌握点是如何添加的。如果 add() 是公有的（谁都能加点），或者私有的（只有 Shape 能加点），就无法做到功能与我们的形状观念如此贴合。

同样，我们把 set_point() 设为受保护的。一般说来，只有派生类才知道一个点意味着什么，以及能否在不破坏不变式的前提下改动它。例如，若我们有一个由六个点定义的 Regular_hexagon 类，只改动其中一个点，得到的图形就“不再是正六边形”。反之，若改动矩形的一个点，结果仍然是矩形。事实上，在示例类和代码里我们并没有发现需要 set_point()，提供它只是为了确保“Shape 的每个属性都可读可设”这条规则成立。例如，若想要一个 Mutable_rectangle，我们可以从 Rectangle 派生，并给出改变点的操作。

我们把 Point 的 vector（points）设为私有，以防它被意外修改。为了让它有用，还需要提供对它的访问：

```
void Shape::set_point(int i, Point p)    // 未使用；目前不需要
{
    points[i] = p;
}

Point Shape::point(int i) const
{
    return points[i];
}

int Shape::number_of_points() const
{
    return points.size();
}
```

在派生类的成员函数中，这些函数是这样用的：

```
void Lines::draw_lines() const
    // 画出连接成对点的线段
{
    for (int i=1; i<number_of_points(); i+=2)
        fl_line(point(i-1).x,point(i-1).y,point(i).x,point(i).y);
}
```

你也许担心这么多琐碎的访问函数：它们难道不低效吗？会让程序变慢吗？会让生成的代码变大吗？不会，编译器会把它们全部“内联”消除掉。调用 number_of_points() 占用的内存字节数、执行的指令条数，与直接调用 points.size() 完全相同。

这些关于访问控制的考虑与决定很重要。我们本可以提供下面这个接近最小化的 Shape 版本：

```
struct Shape {     // 近乎最小的定义——过于简单——未被采用
    Shape();
    Shape(initializer_list<Point>);
    void draw() const;     // 处理颜色并调用 draw_lines
    virtual void draw_lines() const;  // 画出相应的线条
    virtual void move(int dx, int dy);  // 把形状移动 +=dx 和 +=dy
    virtual ~Shape();

    vector<Point> points;     // 并非所有形状都用

    Color lcolor;
    Line_style ls;
    Color fcolor;
};
```

那么，多出来的 12 个成员函数和两行访问说明（`private:` 与 `protected:`）带来了什么价值？基本答案是：保护表示，确保它不会以类设计者未预料到的方式改变，从而让我们能用更少的力气写出更好的类。这就是关于“不变式”的论证（§9.4.3）。这里我们会在定义 `Shape` 的派生类时指出这类好处。一个简单的例子是：早先版本的 `Shape` 用的是

```
Fl_Color lcolor;
int line_style;
```

这种做法后来被证明限制太大（`int` 线型无法优雅地支持线宽，`Fl_Color` 也无法表示“不可见”），并导致了一些凌乱的代码。如果这两个变量是公有的、并被用户代码使用，那么我们要改进接口库，就只得以破坏那些代码为代价（因为它们提到了 `lcolor` 和 `line_style` 这两个名字）。

此外，访问函数常常在记法上带来方便。例如 `s.add(p)` 就比 `s.points.push_back(p)` 更好读、更好写。

14.2.3 绘制形状

```
void draw() const;     // 处理颜色并调用 draw_lines
virtual void draw_lines() const;  // 恰当地画出线条
```

现在，除了 `Shape` 真正的核心，我们几乎已经描述了它的全部内容。`Shape` 最基本的任务就是绘制形状。我们可以去掉它的其他所有功能，甚至不让它拥有自己的数据，而不会造成重大的概念损失（见 §14.4）；但绘制是 `Shape` 的本职工作。它借助 FLTK 和操作系统的基本机制来完成绘制，而从用户的角度看，它只提供两个函数：

- `draw()` 先应用样式和颜色，然后调用 `draw_lines()`。
- `draw_lines()` 把像素放到屏幕上。

`draw()` 函数没有用什么新奇的技术。它只是调用 FLTK 函数，把颜色和样式设成 `Shape` 中指定的值，调用 `draw_lines()` 在屏幕上真正绘制，然后尽量把颜色和样式恢复成调用之前的样子：

```
void Shape::draw() const
{
    Fl_Color oldc = fl_color();
    // 没有可移植的好办法取回当前样式
    fl_color(lcolor.as_int());    // 设置颜色
    fl_line_style(ls.style(),ls.width());    // 设置样式
    draw_lines();
    fl_color(oldc);        // 重置颜色（恢复为之前的值）
    fl_line_style(0);      // 把线型重置为默认值
}
```

遗憾的是，FLTK 没有提供获取当前样式的办法，所以样式只是被重置为默认值。这就是我们有时为了简单和可移植而不得不接受的妥协。我们认为不值得在接口库里实现这项功能。

注意，Shape::draw() 不处理填充色，也不处理线条的可见性。这些由各个 draw_lines() 函数处理，因为它们更清楚该如何解释这些设置。原则上，所有颜色和样式的处理都可以下放给各个 draw_lines()，但那样会有大量重复。

现在考虑 draw_lines() 该怎么处理。稍微想一想就会发现，要让一个 Shape 函数画完每种形状所需的全部内容是很困难的。那样就得把每种形状的每一个像素都以某种方式存进 Shape 对象；如果沿用 vector<Point> 模型，我们要存的点会多得可怕。更糟的是，“屏幕”（也就是图形硬件）本来就保存着这些信息，而且做得更好。

为了避免这份额外的工作和额外的存储，Shape 换了另一种做法：它让每个 Shape（也就是每个从 Shape 派生的类）有机会自己定义“绘制它”意味着什么。Text、Rectangle 或 Circle 类可能各有巧妙的画法。事实上，大多数这样的类确实如此。毕竟，这些类“知道”自己究竟该表示什么。例如，Circle 由一个圆心和一个半径定义，而不是由一大堆线段定义。在需要的时候由圆心和半径生成 Circle 所需的那些位，其实并不太难，也不昂贵。所以 Circle 定义了自己的 draw_lines()，我们希望调用它而不是 Shape 的 draw_lines()。Shape::draw_lines() 声明里的那个 virtual 就是这个意思：

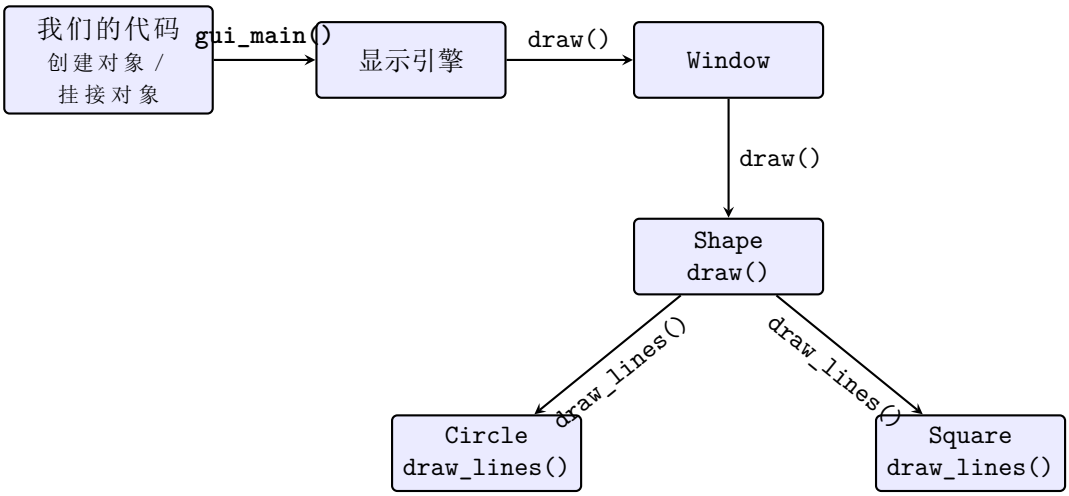
```
struct Shape {
    // . . .
    virtual void draw_lines() const;    // 让每个派生类自行定义 draw_lines()
    // . . .
};


struct Circle : Shape {
    // . . .
    void draw_lines() const;    // “覆盖” Shape::draw_lines()
    // . . .
};
```

所以，如果这个 Shape 是 Circle，Shape 的 draw_lines() 就必须以某种方式调用 Circle 的某个函数；如果是 Rectangle，就调用 Rectangle 的函数。draw_lines() 声明中的 virtual 一词保证了这一点：如果从 Shape 派生的类定义了自己的 draw_lines()（类型与 Shape 的 draw_lines() 相同），那么被

调用的是它，而不是 Shape 的 draw_lines()。第 13 章展示了 Text、Circle、Closed_polyline 等类是怎么做到的。在派生类中定义一个函数，使它能够通过基类提供的接口被使用，这叫做覆盖（overriding）。

注意，尽管 draw_lines() 在 Shape 中处于核心地位，它却是受保护的：它不打算让“普通用户”调用——那是 draw() 的职责——而只是一个“实现细节”，供 draw() 和 Shape 的派生类使用。

这就完成了 §12.2 中的显示模型。驱动屏幕的系统知道 Window；Window 知道 Shape，并能调用 Shape 的 draw()；最后，draw() 调用特定种类形状的 draw_lines()。用户代码中一次 gui_main() 调用就启动了显示引擎。



什么 gui_main()？到目前为止，我们的代码里其实还没见过 gui_main()。我们用的一直是 wait_for_button()，它以更简单的方式调用显示引擎。

Shape 的 move() 函数只是把其中保存的每个点相对于当前位置移动：

```
void Shape::move(int dx, int dy)    // 把形状移动 +=dx 和 +=dy
{
    for (int i = 0; i<points.size(); ++i) {
        points[i].x+=dx;
        points[i].y+=dy;
    }
}
```

和 draw_lines() 一样，move() 是虚函数，因为派生类可能有自己需要移动、而 Shape 并不知道的数据。例如参见 Axis（§12.7.3 与 §15.4）。

从逻辑上说，move() 对 Shape 并非必需；提供它只是为了方便，并多给一个虚函数的例子。每一种拥有未存于其 Shape 中的点的形状，都必须定义自己的 move()。

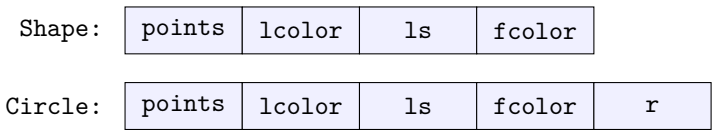
14.2.4　复制与可变性

```
Shape(const Shape&) =delete;      // 禁止复制
Shape& operator=(const Shape&) =delete;
```

Shape 类把复制构造函数和复制赋值都声明为删除的。其效果是消除本会由编译器默认生成的复制操作。例如：

```
void my_fct(Open_polyline& op, const Circle& c)
{
    Open_polyline op2 = op;     // 错误：Shape 的复制构造函数已删除
    vector<Shape> v;
    v.push_back(c);             // 错误：Shape 的复制构造函数已删除
    // . . .
    op = op2;                   // 错误：Shape 的赋值已删除
}
```

可是复制在太多地方都用得上！就看那个 push_back() 吧：不许复制，连 vector 都很难用（push_back() 会把它实参的一个副本放进向量里）。为什么有人要跟程序员过不去、禁止复制呢？只有当默认的复制操作很可能惹麻烦时，你才应该禁止它。my_fct() 就是一个“惹麻烦”的典型例子。我们不能把一个 Circle 复制进 v 中某个 Shape 大小的“槽位”里；Circle 有半径而 Shape 没有，所以 sizeof(Shape)<sizeof(Circle)。如果允许那句 v.push_back(c)，Circle 就会被“切片”（对象切片），此后对所得 Shape 元素的任何使用都很可能导致崩溃：Circle 的操作会假定存在一个并未被复制的半径成员（r）。



op2 的复制构造和对 op 的赋值也有完全相同的问题。考虑：

```
Marked_polyline mp {"x"};
Circle c(p,10);
my_fct(mp,c);    // Open_polyline 实参引用的是一个 Marked_polyline
```

现在，Open_polyline 的拷贝操作会把 mp 的 string 成员 mark 切掉，也就是“切片”（slice）。

基本上，类层次结构与按引用传递、默认拷贝这三者是凑不到一起的。当你设计的类打算充当层次结构中的基类时，就要像 Shape 那样，用 =delete 禁用它的拷贝构造函数和拷贝赋值。

切片（没错，这确实是个正经术语）并不是禁止拷贝的唯一理由。有不少概念最好不提供拷贝操作。要记住，图形系统必须记住每个 Shape 存放在哪里，才能把它显示到屏幕上。这正是我们把 Shape “附着”（attach）到 Window 上、而不是拷贝它的原因。举例来说，假如 Window 保存的只是 Shape 的一个副本，而不是对那个 Shape 的引用，那么对原对象的修改就不会影响这个副本。于是，如果我们改动了 Shape 的颜色，Window 不会察觉这一改动，仍会用未改变的颜色显示它的副本。从非常实际的意义上说，副本不如原件。

如果我们想拷贝那些已禁用默认拷贝操作的类型对象，可以写一个显式函数来完成这件事。这样的拷贝函数通常叫作 clone()。显然，只有当读取成员的函数足以表达构造副本所需的信息时，你才能写出 clone()；而对所有 Shape 来说，情况正是如此。

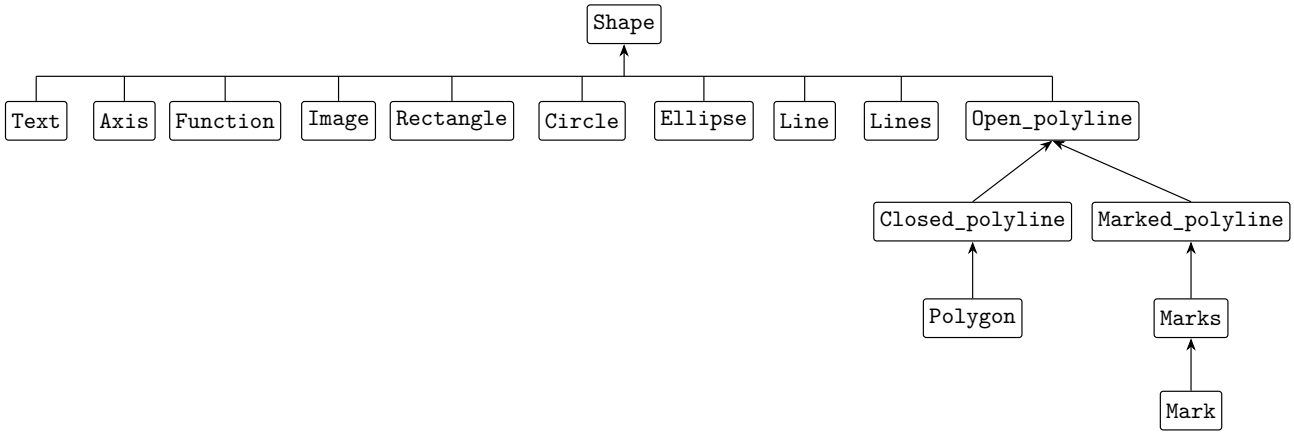
14.3　基类与派生类

下面我们从更技术化的角度来看基类与派生类；也就是说，仅在本节中，我们把讨论的重点从编程、应用设计和图形转向编程语言特性。在设计图形接口库时，我们依赖了三个关键的语言机制：

- 派生（derivation）：一种用一个类构造另一个类的方式，使新类可以替代原来的类使用。例如，`Circle` 派生自 `Shape`；换句话说，“`Circle` 是一种 `Shape`”，或者说“`Shape` 是 `Circle` 的基类”。派生类（这里是 `Circle`）除了自己的成员之外，还得到其基类（这里是 `Shape`）的全部成员。这常被称为继承（inheritance），因为派生类“继承”了基类的所有成员。在某些语境中，派生类称为子类（subclass），基类称为超类（superclass）。
- 虚函数（virtual function）：能够在基类中定义一个函数，并使得当用户调用基类函数时，实际调用的是派生类中同名同类型的函数。例如，当 `Window` 对一个实际上是 `Circle` 的 `Shape` 调用 `draw_lines()` 时，执行的是 `Circle` 的 `draw_lines()`，而不是 `Shape` 自己的 `draw_lines()`。这常被称为运行时多态（run-time polymorphism）、动态分派（dynamic dispatch）或运行时分派（run-time dispatch），因为调用哪个函数是在运行时根据所用对象的类型决定的。
- 私有成员与保护成员（private and protected members）：我们把类的实现细节保持为私有，以防被直接使用而使维护复杂化。这常被称为封装（encapsulation）。

继承、运行时多态和封装的运用，是对面向对象编程最常见的定义。因此，C++ 除了支持其他编程风格之外，还直接支持面向对象编程。例如，在第 20～21 章我们将看到 C++ 如何支持泛型编程。C++ 的关键机制借自 Simula67——并且明确致谢——那是第一种直接支持面向对象编程的语言（见第 22 章）。

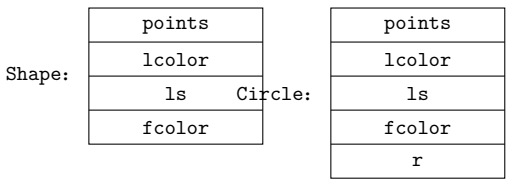
刚才一下子出现了不少技术术语！它们到底是什么意思？在我们的计算机上又是如何真正运作的？我们先画一张简单的图，表示图形接口各个类之间的继承关系：



箭头由派生类指向其基类。这类图有助于把类之间的关系可视化，也常常成为程序员黑板上的装饰。与商业框架相比，这只是一个只有 16 个类的很小的“类层次结构”，而且只有 `Open_polyline` 的众多后裔才使层次超过一层。显然，共同的基类（`Shape`）才是这里最重要的类，尽管它表示的是一个抽象概念，因此我们从不会直接创建一个“形状”对象。

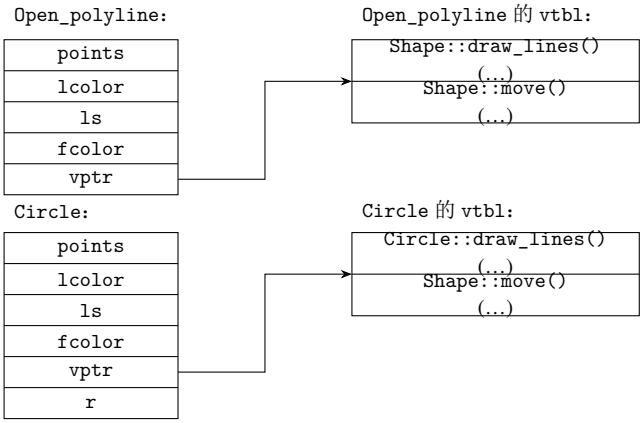
14.3.1 对象布局

对象在内存中是如何布局的？正如 §9.4.1 所见，类的成员决定了对象的布局：数据成员在内存中依次存放。使用继承时，派生类的数据成员只是接在基类的数据成员之后。例如：



Circle 拥有 Shape 的数据成员（毕竟它是一种 Shape），因此可以当作 Shape 使用。此外，Circle 还有“自己的”数据成员 r，它放在继承来的数据成员之后。

为了处理虚函数调用，Shape 对象中还需要（也确实有）一项额外的数据：用它指明当我们调用 Shape 的 draw_lines() 时，真正被调用的是哪个函数。通常的做法是加入一个函数表的地址。这个表通常称为 vtbl（“virtual table”，虚函数表），它的地址常称为 vptr（“virtual pointer”，虚指针）。指针将在第 17～18 章讨论；在这里，它们的作用类似引用。具体的实现可能给 vtbl 和 vptr 起不同的名字。把 vptr 和各个 vtbl 加入图中，就得到



由于 draw_lines() 是第一个虚函数，它占据 vtbl 的第一个槽位，后面是第二个虚函数 move() 的槽位。一个类想要多少个虚函数就可以有多少个；它的 vtbl 会按需变大（每个虚函数一个槽位）。这样，当我们调用 x.draw_lines() 时，编译器生成的调用会去执行 x 的 vtbl 中 draw_lines() 槽位里存放的函数。基本上，代码就是沿着图中的箭头走。所以如果 x 是一个 Circle，被调用的就是 Circle::draw_lines()。如果 x 的类型（比如 Open_polyline）完全沿用 Shape 所定义的 vtbl，那么被调用的就是 Shape::draw_lines()。同样，Circle 没有定义自己的 move()，所以当 x 是 Circle 时，x.move() 会调用 Shape::move()。基本上，为虚函数调用生成的代码只是找到 vptr，用它到达正确的 vtbl，再调用其中相应的函数。代价大约是两次内存访问加上一次普通函数调用的开销。这既简单又快速。

Shape 是抽象类，所以实际上不可能存在一个“只是 Shape”的对象；但 Open_polyline 的布局与“纯形状”完全相同，因为它既没有增加数据成员，也没有定义虚函数。每个含虚函数的类只有一个 vtbl，而不是每个对象一个，因此 vtbl 一般不会明显增加程序的目标代码大小。

注意，我们没有在图中画出任何非虚函数。这没有必要，因为这类函数的调用方式没有任何特别之处，它们也不会增大其类型对象的大小。

定义一个与基类中某个虚函数同名同类型的函数（例如 Circle::draw_lines()），使派生类的这个函数被放进 vtbl 取代基类的版本，这叫作覆盖（overriding）。例如，Circle::draw_lines() 覆盖了 Shape::draw_lines()。

我们为什么要讲 vtbl 和内存布局？要用面向对象编程，你需要了解这些吗？不需要。不过，很多人非常希望知道事物是如何实现的（我们就属于这一类），而且当人们不理解某件事时，各种神话就会冒出来。我们遇到过一些人，他们对虚函数怕得要命，“因为它们开销大”。为什么？有多大开销？跟什么比？开销会在哪里产生影响？我们讲解虚函数的实现模型，就是为了让你不必有这类恐惧。如果你确实需要一次虚函数调用（在运行时从若干备选方案中挑选），那么用其他语言特性也无法把这项功能写得更快或更省内存。这一点你可以自己验证。

14.3.2 派生类与虚函数的定义

我们用类名之后跟一个基类的方式，说明某个类是派生类。例如：

```
struct Circle : Shape { /* . . . */ };
```

默认情况下，struct 的成员是公有的（§9.3），这包括基类的公有成员。等价地，我们也可以写成

```
class Circle : public Shape { public: /* . . . */ };
```

这两种 Circle 的声明完全等价，但你可以就哪一种更好与人展开许多冗长而无结果的争论。我们认为，把时间花在其他话题上会更有成效。

要注意在需要 public 的时候别漏掉它。例如：

```
class Circle : Shape { public: /* . . . */ };    // 很可能是笔误
```

这会使 Shape 成为 Circle 的私有基类，从而使 Shape 的公有函数对 Circle 不可访问。这多半不是你的本意。好的编译器会对这个可能的错误给出警告。私有基类是有用途的，但那些超出了本书的范围。

虚函数必须在其类声明中声明为 virtual；但如果你把函数定义放在类外，那么在类外既不需要、也不允许再写 virtual 关键字。例如：

```
struct Shape {
    // . . .
    virtual void draw_lines() const;
    virtual void move();
    // . . .
};

virtual void Shape::draw_lines() const { /* . . . */ }    // 错误
void Shape::move() { /* . . . */ }    // 正确
```

14.3.3 覆盖

当你想要覆盖一个虚函数时，必须使用与基类中完全相同的名字和类型。例如：

```
struct Circle : Shape {
    void draw_lines(int) const;    // 很可能是笔误（多了个 int 参数？）
    void drawlines() const;    // 很可能是笔误（名字拼错了？）
    void draw_lines();    // 很可能是笔误（漏了 const？）
    // . . .
};
```

在这里，编译器会看到三个与 Shape::draw_lines() 无关的函数（因为名字不同或类型不同），它们不会覆盖该函数。好的编译器会对这些可能的错误给出警告。在覆盖函数里，你既不能、也不必额外声明什么来保证它真的覆盖了基类的函数。

draw_lines() 这个例子是真实的，因此细节可能不容易跟上，所以这里给出一个纯粹技术性的例子来说明覆盖：

```
struct B {
    virtual void f() const { cout << "B::f "; }
    void g() const { cout << "B::g "; }    // 非虚函数
};

struct D : B {
    void f() const { cout << "D::f "; }    // 覆盖 B::f
    void g() { cout << "D::g "; }
};

struct DD : D {
    void f() { cout << "DD::f "; }    // 没有覆盖 D::f（不是 const）
    void g() const { cout << "DD::g "; }
};
```

这里我们有一个小型的类层次结构，其中（只）有一个虚函数 f()。我们可以试着用它。特别是，我们可以试着调用 f() 和非虚函数 g()——g() 并不知道自己面对的是什么类型的对象，只知道它是一个 B（或由 B 派生的东西）：

```
void call(const B& b)
    // D 是一种 B，所以 call() 可以接受 D
    // DD 是一种 D，而 D 是一种 B，所以 call() 可以接受 DD
{
    b.f();
    b.g();
}

int main()
{
    B b;
    D d;
    DD dd;

    call(b);
    call(d);
    call(dd);

    b.f();
    b.g();

    d.f();
    d.g();

    dd.f();
    dd.g();
}
```

你会得到

```
B::f B::g D::f B::g D::f B::g B::f B::g D::f D::g DD::f DD::g
```

当你明白为什么会这样时，你就掌握了继承与虚函数的运作机制。

显然，要记住哪些派生类函数打算覆盖哪些基类函数可能很难。幸运的是，我们可以借助编译器来检查。我们可以显式声明某个函数是要做覆盖的。假设那些派生类函数本来就打算做覆盖，我们可以通过加上 override 来说明这一点，于是这个例子变成

```
struct B {
    virtual void f() const { cout << "B::f "; }
    void g() const { cout << "B::g "; }    // 非虚函数
};

struct D : B {
    void f() const override { cout << "D::f "; }    // 覆盖 B::f
    void g() override { cout << "D::g "; }    // 错误：没有可覆盖的虚函数 B::g
};

struct DD : D {
    void f() override { cout << "DD::f "; }    // 错误：没有覆盖 D::f（不是 const）
    void g() const override { cout << "DD::g "; }    // 错误：没有可覆盖的虚函数 D::g
};
```

在大型、复杂的类层次结构中，显式使用 override 尤其有用。

14.3.4 访问控制

C++ 提供了关于类成员访问的一个简单模型。一个类的成员可以是

- 私有（private）：如果成员是私有的，其名字只能由声明它的那个类的成员使用。
- 保护（protected）：如果成员是保护的，其名字只能由声明它的那个类的成员以及由该类派生的类的成员使用。
- 公有（public）：如果成员是公有的，其名字可以被所有函数使用。

用图形表示就是：

基类也可以是私有的、保护的或公有的：

- 如果类 D 的某个基类是私有的，那么该基类的公有成员名和保护成员名只能由 D 的成员使用。

- 如果类 D 的某个基类是保护的，那么该基类的公有成员名和保护成员名只能由 D 的成员以及由 D 派生的类的成员使用。
- 如果某个基类是公有的，那么它的公有成员名可以被所有函数使用。

这些定义忽略了“友元”（friend）的概念和一些次要细节，那些超出了本书的范围。如果你想成为一名“语言律师”（language lawyer），就需要研读 Stroustrup 的《The Design and Evolution of C++》和《The C++ Programming Language》，以及 ISO C++ 标准。我们不建议去做语言律师（即通晓语言定义中每个细枝末节的人）；做一名程序员（软件开发者、工程师、用户，随你喜欢怎么称呼真正使用这门语言的人）要有趣得多，通常对社会也更有用。

14.3.5 纯虚函数

抽象类（abstract class）是只能用作基类的类。我们用抽象类来表示抽象的概念；也就是说，我们把抽象类用于这样一些概念：它们是相关实体共同特征的概括。哲学的大部头著作曾试图精确地定义抽象概念（或抽象、概括……）。无论你在哲学上怎样定义它，抽象概念这一观念都极其有用。例子有“动物”（而不是任何一种特定的动物）、“设备驱动程序”（而不是任何一种特定设备的驱动程序）以及“出版物”（而不是任何一种特定的书或杂志）。在程序中，抽象类通常为一组相关的类（类层次结构）定义接口。

在 §14.2.1 中，我们看到了如何通过把构造函数声明为保护的来使一个类成为抽象类。还有另一种——而且常见得多——使类成为抽象类的方法：指明它的一个或多个虚函数需要在某个派生类中被覆盖。例如：

```
class B {    // 抽象基类
public:
    virtual void f() =0;    // 纯虚函数
    virtual void g() =0;
};

B b;    // 错误：B 是抽象类
```

这个奇怪的 =0 记法表示虚函数 B::f() 和 B::g() 是“纯”的；也就是说，它们必须在某个派生类中被覆盖。由于 B 含有纯虚函数，我们无法创建 B 类的对象。覆盖这些纯虚函数就能解决这个“问题”：

```
class D1 : public B {
public:
    void f() override;
    void g() override;
};

D1 d1;    // 正确
```

注意，除非所有纯虚函数都被覆盖，否则得到的类仍然是抽象类：

```
class D2 : public B {
public:
    void f() override;
    // 没有 g()
};

D2 d2;    // 错误：D2（仍然）是抽象的

class D3 : public D2 {
public:
    void g() override;
};

D3 d3;    // 正确
```

含有纯虚函数的类往往就是纯接口；也就是说，它们往往没有数据成员（数据成员会放在派生类里），因而也就没有构造函数（如果没有数据成员需要初始化，那就不太可能需要构造函数）。

14.4 面向对象编程的收益

当我们说 Circle 派生自 Shape、或者说 Circle 是一种 Shape 时，我们这样说是为了获得下面二者之一或二者兼有：

- 接口继承（interface inheritance）：一个期待 Shape 的函数（通常以引用作为参数）可以接受 Circle，并且可以通过 Shape 提供的接口来使用这个 Circle。
- 实现继承（implementation inheritance）：在定义 Circle 及其成员函数时，我们可以利用 Shape 提供的设施（例如数据成员和成员函数）。

不提供接口继承的设计（也就是说，在这种设计中，派生类的对象不能当作其公有基类的对象来使用）是一种糟糕且容易出错的设计。例如，我们可以定义一个名为 `Never_do_this` 的类，把 Shape 作为它的公有基类。然后我们就可以用一个函数覆盖 `Shape::draw_lines()`，而这个函数并不绘制图形，只是把图形的中心向左移动 100 像素。这个“设计”有着致命的缺陷：尽管 `Never_do_this` 提供了 Shape 的接口，它的实现却没有维持 Shape 所要求的语义（即含义与行为）。绝不要那样做！

接口继承之所以得名，是因为它的收益来自这样一些代码：它们使用基类提供的接口（“一个接口”；这里是 Shape），而不必了解派生类（“实现”；这里是从 Shape 派生的类）。

实现继承之所以得名，是因为它的收益来自基类（这里是 Shape）提供的设施给派生类（例如 Circle）的实现带来的简化。

请注意，我们的图形设计严重依赖于接口继承：“图形引擎”调用 `Shape::draw()`，后者又调用 Shape 的虚函数 `draw_lines()`，由它完成把图像真正画到屏幕上的实际工作。无论是“图形引擎”还是 Shape 类本身，都不知道存在哪些种类的图形。特别地，我们的“图形引擎”（FLTK 加上操作系统的图形设施）早在我们的图形类之前许多年就已经编写并编译好了！我们只需定义具体的图形，并通过 `attach()` 把它们作为 Shape 挂到 Window 上（`Window::attach()` 接受一个 `Shape&` 参数；见 §E.3）。此外，由于 Shape 类并不了解你的图形类，所以你每次定义一个新的图形接口类时都不需要重新编译 Shape。

换言之，我们可以在不修改现有代码的情况下向程序添加新的 Shape。这是软件设计、开发与维护的圣杯：在不修改系统的前提下扩展系统。我们能在不修改现有类的情况下做出的改动是有限度的（例如，Shape 提供的服务范围相当有限），而且这种技术并不适用于所有编程问题（例如，参见第 17～19 章，那里我们要定义 vector；继承在那里几乎派不上用场）。不过，接口继承仍然是设计和实现“面对变化依然健壮”的系统的最强大技术之一。

同样，实现继承也大有可为，但它并非万灵药。把有用的服务放在 Shape 里，就省去了在派生类中一遍又一遍重复劳动的麻烦。在真实世界的代码中，这一点可能极为重要。然而，这样做也有代价：只要 Shape 的接口发生任何改变，或者 Shape 数据成员的布局发生任何改变，都必须重新编译所有派生类及其用户。对于一个被广泛使用的库来说，这样的重新编译根本不可行。当然，有一些办法可以获得大部分收益而避免大部分问题；见 §14.3.5。

小试身手

遗憾的是，我们无法为检验对一般设计原则的理解构造一组小试身手，因此这里我们把重点放在支持面向对象编程的语言特性上。

1. 定义一个类 `B1`，其中有一个虚函数 `vf()` 和一个非虚函数 `f()`。在类 `B1` 内部定义这两个函数。实现每一个函数，让它输出自己的名字（例如 `B1::vf()`）。把这些函数设为 public。创建一个 `B1` 对象并调用这两个函数。
2. 从 `B1` 派生一个类 `D1`，并覆盖 `vf()`。创建一个 `D1` 对象，对它调用 `vf()` 和 `f()`。
3. 定义一个 `B1` 的引用（`B1&`），并用你刚才定义的 `D1` 对象初始化它。对这个引用调用 `vf()` 和 `f()`。
4. 现在为 `D1` 定义一个名为 `f()` 的函数，并重复第 1～3 步。解释结果。
5. 向 `B1` 添加一个名为 `pvf()` 的纯虚函数，并尝试重复第 1～4 步。解释结果。
6. 定义一个从 `D1` 派生的类 `D2`，并在 `D2` 中覆盖 `pvf()`。创建一个 `D2` 类的对象，并对它调用 `f()`、`vf()` 和 `pvf()`。
7. 定义一个类 `B2`，其中有一个纯虚函数 `pvf()`。定义一个类 `D21`，它有一个 `string` 数据成员和一个覆盖 `pvf()` 的成员函数；`D21::pvf()` 应当输出该字符串的值。定义一个类 `D22`，除数据成员是 `int` 之外，它与 `D21` 完全相同。定义一个函数 `f()`，它接受一个 `B2&` 参数，并对其参数调用 `pvf()`。分别用一个 `D21` 和一个 `D22` 调用 `f()`。

要点回顾

1. 什么是应用领域？
2. 命名的理想是什么？
3. 我们可以给什么命名？
4. Shape 提供哪些服务？
5. 抽象类与非抽象的类有什么区别？
6. 怎样才能使一个类成为抽象类？
7. 访问控制控制的是什么？
8. 把一个数据成员设为私有有什么好处？
9. 什么是虚函数？它与非虚函数有什么区别？
10. 什么是基类？
11. 是什么使一个类成为派生类？
12. 我们所说的对象布局指什么？
13. 你可以做些什么来使一个类更容易测试？

14. 什么是继承图？
15. 受保护成员与私有成员有什么区别？
16. 一个类的哪些成员可以从它的派生类访问？
17. 纯虚函数与其他虚函数有什么区别？
18. 你为什么会把成员函数设为虚函数？
19. 你为什么会把虚成员函数设为纯虚？
20. 覆盖（overriding）是什么意思？
21. 接口继承与实现继承有什么区别？
22. 什么是面向对象编程？

术语

抽象类	可变性	纯虚函数
访问控制	对象布局	子类
基类	面向对象编程	超类
派生类	重写	虚函数
分派	多态	虚函数调用
封装	私有	虚函数表
继承	受保护	
	公有	

习题

1. 定义两个类 `Smiley` 和 `Frowny`，二者都派生自类 `Circle`，并且都有两只眼睛和一张嘴。然后从 `Smiley` 和 `Frowny` 派生类，分别为它们加上一顶合适的帽子。
2. 试着复制一个 Shape。会发生什么？
3. 定义一个抽象类，并试着定义该类型的对象。会发生什么？
4. 定义一个类 `Immobile_Circle`，它与 `Circle` 完全相同，只是不能被移动。
5. 定义一个 `Striped_rectangle`：它不用填充，而是通过在矩形内部画一条条一个像素宽的横线来“填充”矩形（比如每隔一行画一条这样的线）。你可能需要调整线宽和行间距，才能得到你喜欢的图案。
6. 用 `Striped_rectangle` 中的技术定义一个 `Striped_circle`。
7. 用 `Striped_rectangle` 中的技术定义一个 `Striped_closed_polyline`（这需要一些算法上的创造性）。
8. 定义一个类 `Octagon` 表示正八边形。写一个测试，调用它的所有函数（你自己定义的，或从 Shape 继承来的）。
9. 定义一个 `Group`，作为 Shape 的容器，并对 `Group` 的各个成员施加合适的操作。提示：`Vector_ref`。用 `Group` 定义一个跳棋（checkers，即 draughts）棋盘，棋盘上的棋子可以在程序控制下移动。
10. 定义一个类 `Pseudo_window`，让它尽可能像 `Window`，但不必费太大力气。它应当有圆角、一个标签和控制图标。也许你可以添加一些假的“内容”，例如一幅图像。它不需要真的做任何事。让它出现在一个 `Simple_window` 中是可行的（而且确实值得推荐）。
11. 定义一个从 Shape 派生的 `Binary_tree` 类。把层数作为参数（levels==0 表示没有结点，levels==1 表示一个结点，levels==2 表示一个顶层结点带两个子结点，levels==3 表示一个顶层结点带两个

子结点、每个子结点又各带两个子结点，依此类推）。用一个小的圆圈表示结点。按惯例用线连接各结点。附注：在计算机科学中，树是从顶层结点向下生长的（有趣但合乎逻辑的是，这个结点常被称为根）。

12. 修改 `Binary_tree`，用虚函数来绘制它的结点。然后从 `Binary_tree` 派生一个新类，覆盖那个虚函数，改用另一种方式表示结点（例如三角形）。
13. 修改 `Binary_tree`，让它接受一个（或多个）参数，用来指明连接结点时使用哪种线（例如向下的箭头，或向上的红色箭头）。注意这个习题和上一个习题是怎样用两种不同的方式让类层次结构更灵活、更有用的。
14. 给 `Binary_tree` 添加一个向结点添加文本的操作。你可能需要修改 `Binary_tree` 的设计才能优雅地实现这一点。选择一种标识结点的方式；例如，你可以给出一个字符串“lrrlr”，表示沿二叉树向下依次向左、右、右、左、右（根结点既匹配开头的 l，也匹配开头的 r）。
15. 大多数类层次结构与图形无关。定义一个类 `Iterator`，它有一个纯虚函数 `next()`，返回 `double*`（见第 17 章）。然后从 `Iterator` 派生 `Vector_iterator` 和 `List_iterator`，使 `Vector_iterator` 的 `next()` 给出指向 `vector<double>` 下一个元素的指针，而 `List_iterator` 对 `list<double>` 做同样的事。用一个 `vector<double>` 初始化 `Vector_iterator`，第一次调用 `next()` 时给出指向其第一个元素的指针（如果有的话）。如果没有下一个元素，就返回 0。用一个函数 `void print(Iterator&)` 打印 `vector<double>` 和 `list<double>` 的元素，以此进行测试。
16. 定义一个类 `Controller`，它有四个虚函数 `on()`、`off()`、`set_level(int)` 和 `show()`。从 `Controller` 至少派生两个类。一个应当是简单的测试类，其中 `show()` 打印出该类被设为 on 还是 off，以及当前级别是多少。第二个派生类应当以某种方式控制 Shape 的线条颜色；“级别”的确切含义由你决定。试着再找出第三种可以用这样一个 `Controller` 类来控制的东西。
17. C++ 标准库中定义的异常，例如 `exception`、`runtime_error` 和 `out_of_range`（§5.6.3），被组织成一个类层次结构（其中有一个有用的虚函数 `what()`，返回一个据称解释出了什么问题的字符串）。在你的信息源中查找 C++ 标准异常类层次结构，并画出它的类层次图。

后记

软件的理想并不是构建一个无所不能的单一程序。理想是构建大量类，这些类紧密地反映我们的概念，并且协同工作，让我们能够优雅地构建自己的应用——相对于任务的复杂度而言投入的精力很小，性能足够，而且可以确信所产生的结果是正确的。这样的程序易于理解、易于维护，而那些只是为了尽快完成某项具体工作而草草拼凑起来的代码则不是这样。类、封装（由 `private` 和 `protected` 支持）、继承（由类派生支持）以及运行时多态（由虚函数支持），是我们用来构造系统的最强大工具之一。

15 函数与数据的绘制

"The best is the enemy of the good."
（至善乃善之敌。）
——伏尔泰

如果你身处任何实证领域，就需要把数据画成图。如果你所在的领域用数学来为现象建模，就需要把函数画成图。本章讨论生成这类图形的基本机制。和往常一样，我们既展示这些机制的用法，也讨论它们的设计。关键例子是绘制单参数函数的图形，以及显示从文件中读入的值。

15.1 引言
15.2 绘制简单函数
15.3 函数
15.3.1 默认实参
15.3.2 更多例子
15.3.3 lambda 表达式
15.4 坐标轴
15.5 近似
15.6 绘制数据
15.6.1 读取文件
15.6.2 总体布局
15.6.3 数据缩放
15.6.4 构建图形

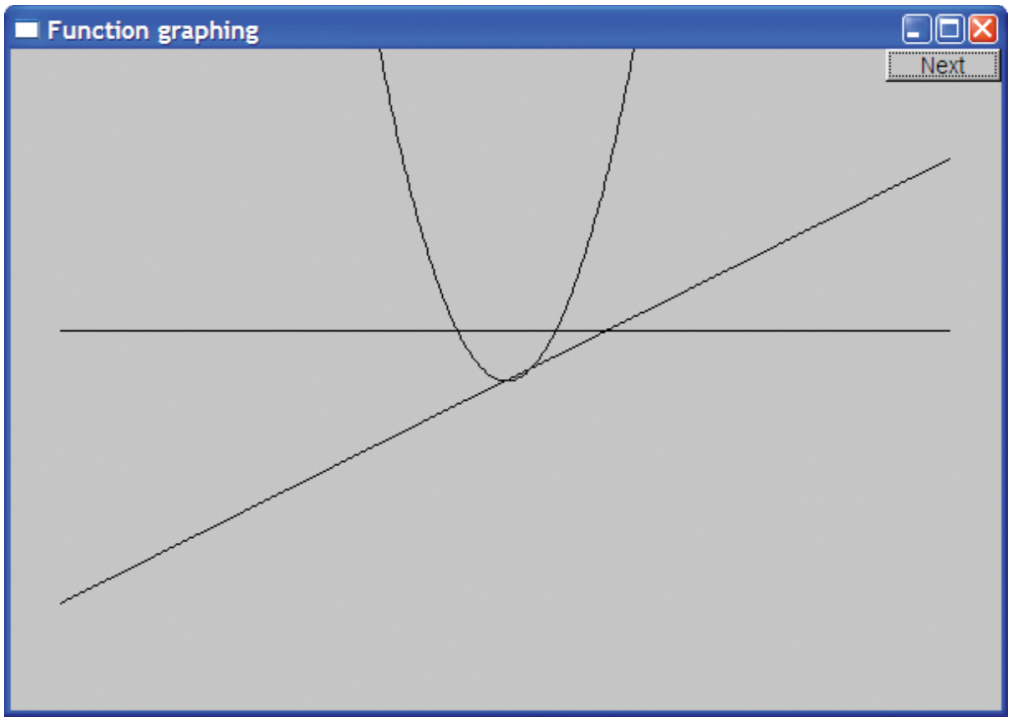
15.1 引言

与那些专业软件系统相比——如果你把这类可视化当成自己的主业，就会用到它们——这里介绍的设施还很原始。我们的首要目标不是输出的优雅，而是理解这类图形输出如何产生，以及其中用到的编程技术。你会发现，这里介绍的设计技术、编程技术和基本数学工具，比这些图形设施本身更有长期价值。因此，请不要过快略过这些代码片段——它们值得关注的地方，远不止计算并画出的那些形状。

15.2 绘制简单函数

我们开始吧。先看一些例子：我们能画出什么，以及画出它们需要什么代码。特别要注意其中用到的图形界面类。这里首先是一条抛物线、一条水平线和一条斜线：

其实，本章讲的是绘制函数图形，所以那条水平线并不只是一条水平线；它是绘制下面这个函数得到的结果：




```
double one(double) { return 1; }
```

这大概是我们能想到的最简单的函数：它是一个单参数函数，对任何参数都返回 1。既然计算返回值时用不到这个参数，我们就不必给它起名字。对每个传入 one() 的参数 x，得到的 y 值都是 1；也就是说，对所有 x，这条线都由 (x,y)==(x,1) 定义。

像所有入门阶段的数学论证一样，这一段有点琐碎、有点学究气，所以我们来看一个稍微复杂一点的函数：

```
double slope(double x) { return x/2; }
```

上面这个函数生成的就是那条斜线。对每个 x，我们得到 y 值 x/2。换句话说，(x,y)==(x,x/2)。两条线的交点是 (2,1)。

现在可以试点更有意思的东西：这个在本书中似乎反复出现的平方函数：

```
double square(double x) { return x*x; }
```

如果你还记得中学几何（就算不记得也没关系），这个函数定义了一条抛物线，最低点在 (0,0)，并且关于 y 轴对称。换句话说，(x,y)==(x,x*x)。因此，抛物线与那条斜线相接的最低点是 (0,0)。

下面是绘制这三个函数的代码：

```
constexpr int xmax = 600;               // 窗口大小
constexpr int ymax = 400;

constexpr int x_orig = xmax/2;          // (0,0) 的位置是窗口中心
constexpr int y_orig = ymax/2;
constexpr Point orig {x_orig,y_orig};

constexpr int r_min = -10;              // 区间 [-10:11)
constexpr int r_max = 11;

constexpr int n_points = 400;           // 区间内使用的点数

constexpr int x_scale = 30;             // 缩放系数
constexpr int y_scale = 30;

Simple_window win {Point{100,100},xmax,ymax,"Function graphing"};
Function s {one,r_min,r_max,orig,n_points,x_scale,y_scale};
Function s2 {slope,r_min,r_max,orig,n_points,x_scale,y_scale};
Function s3 {square,r_min,r_max,orig,n_points,x_scale,y_scale};

win.attach(s);
win.attach(s2);
win.attach(s3);
win.wait_for_button();
```

首先，我们定义一组常量，这样就不必用“魔法常量”把代码弄得乱七八糟。然后创建一个窗口，定义这几个函数，把它们附着到窗口上，最后把控制权交给图形系统去完成实际绘制。

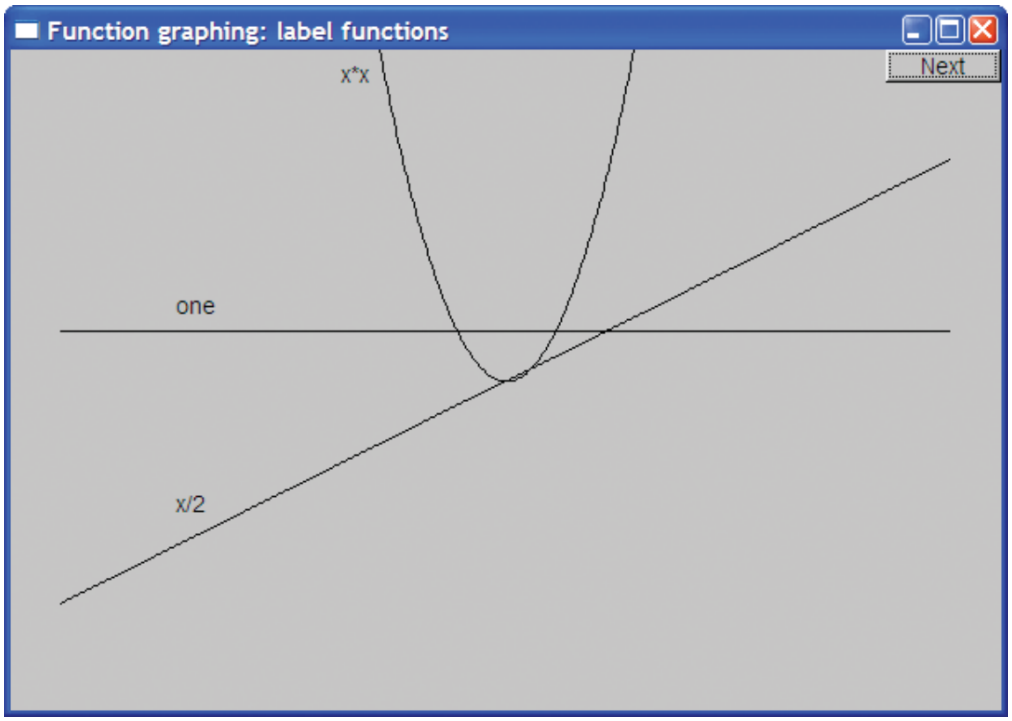
除了三个 `Function` 对象 `s`、`s2` 和 `s3` 的定义之外，其余都是重复的“样板”代码：

```
Function s {one,r_min,r_max,orig,n_points,x_scale,y_scale};
Function s2 {slope,r_min,r_max,orig,n_points,x_scale,y_scale};
Function s3 {square,r_min,r_max,orig,n_points,x_scale,y_scale};
```

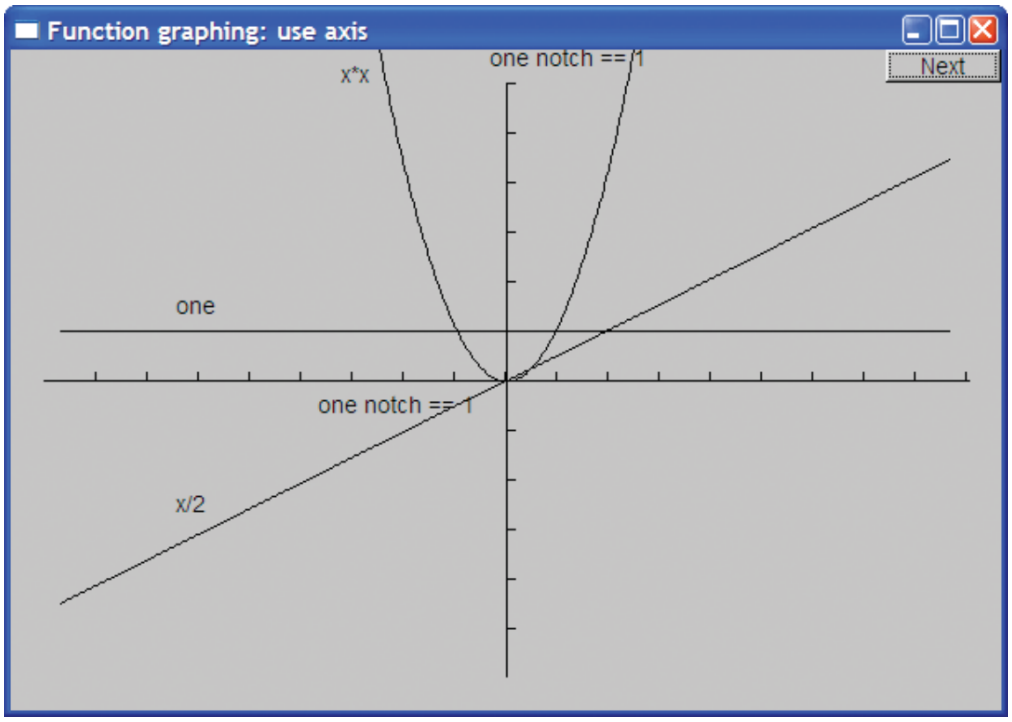
每个 `Function` 都指明它的第一个参数（一个接受单个 double 参数并返回 double 的函数）应当如何在窗口中绘制。第二、第三个参数给出 x 的范围（即被绘制函数的参数范围）。第四个参数（这里是 `orig`）告诉 `Function` 原点 (0,0) 应位于窗口中的哪个位置。

如果你觉得这么多参数容易混淆，我们同意。我们的理想是参数越少越好，因为参数一多既容易混淆，又给缺陷留下可乘之机。不过这里确实需要它们。最后三个参数我们稍后解释（§15.3）。不过先做一件事：给图加上标注：

我们总是尽量让图本身就能说明问题。人们不一定读周围的正文，而好的图又常被挪到别处，结果周围的正文就“失联”了。凡是作为图的一部分放进去的东西，最容易被注意到，而且——只要合理——也最可能帮助读者理解我们在展示什么。这里，我们只是给每条曲线加一个标注。“标注”的代码是三个 `Text` 对象（见 §13.11）：




```
Text ts {Point{100,y_orig-40},"one"};
Text ts2 {Point{100,y_orig+y_orig/2-20},"x/2"};
Text ts3 {Point{x_orig-100,20},"x*x"};
win.set_label("Function graphing: label functions");
win.wait_for_button();
```



从本章接下来的内容开始，我们将省略那些重复的代码：把形状附着到窗口、给窗口加标题，以及等待用户点击“Next”。

不过，这张图仍然不能令人满意。我们注意到 x/2 与 x*x 在 (0,0) 相接，one 与 x/2 在 (2,1) 相交，但这太隐晦了；我们需要坐标轴，给读者一个毫不隐晦的线索，说明图上发生了什么：

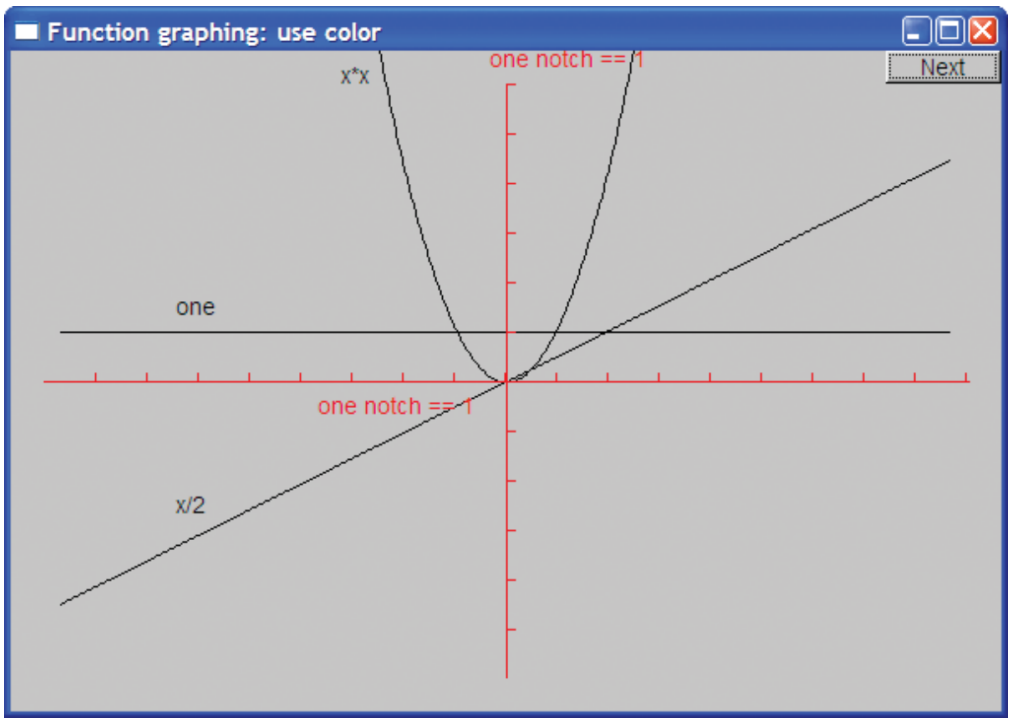
坐标轴的代码是两个 Axis 对象（§15.4）：

```
constexpr int xlength = xmax-40;    // 让轴比窗口略小一些
constexpr int ylength = ymax-40;

Axis x {Axis::x,Point{20,y_orig},
          xlength, xlength/x_scale, "one notch == 1"};
Axis y {Axis::y,Point{x_orig, ylength+20},
          ylength, ylength/y_scale, "one notch == 1"};
```

用 xlength/x_scale 作为刻度数，可以保证每个刻度代表 1、2、3 这样的数值。两条轴在 (0,0) 交叉是惯例。如果你更喜欢按显示数据的惯例（见 §15.6）把轴放在左边和下边，当然也可以那样做。区分坐标轴与数据的另一种办法是用颜色：

```
x.set_color(Color::red);
y.set_color(Color::red);
```

这样便得到



这样就可以了，不过出于审美上的考虑，我们大概会希望在顶部也留一点空白，好和底部、两侧的留白相称。把 x 轴的标注再往左挪一点，可能也是个更好的主意。我们故意留下这些瑕疵，好在这里提一句——总还有更多审美细节可以继续打磨。程序员手艺的一部分，就是知道何时停下来，把省下的时间用在更好的事情上（比如学一门新技术，或者睡觉）。记住：“最好的是好的敌人。”

15.3 函数

图形界面类 Function 的定义如下：

```
struct Function : Shape {
    // 函数参数不存储
    Function(Fct f, double r1, double r2, Point orig,
             int count = 100, double xscale = 25, double yscale = 25);
};
```

Function 是一个 Shape，它的构造函数生成大量线段，并把它们存放在自己的 Shape 部分中。这些线段近似表示函数 f 的值。f 的值在 [r1:r2) 区间内按等间距取值，计算 count 次：

```
Function::Function(Fct f, double r1, double r2, Point xy,
                   int count, double xscale, double yscale)
// 用 count 条线段绘制 f(x), x 在 [r1:r2) 内, (0,0) 显示在 xy 处
// x 坐标按 xscale 缩放, y 坐标按 yscale 缩放
{
    if (r2-r1<=0) error("bad graphing range");
    if (count <=0) error("non-positive graphing count");
    double dist = (r2-r1)/count;
    double r = r1;
    for (int i = 0; i<count; ++i) {
        add(Point{xy.x+int(r*xscale),xy.y-int(f(r)*yscale)});
        r += dist;
    }
}
```

xscale 和 yscale 分别用来缩放 x 坐标和 y 坐标。通常我们需要把数值缩放一下，才能让它们恰当地落进窗口的绘图区域。

注意，Function 对象并不保存传给其构造函数的那些值，因此我们事后既无法询问某个函数它的原点在哪里，也无法用不同的缩放比例重绘它，等等。它所做的只是保存点（在它的 Shape 中），并把自身画到屏幕上。如果希望 Function 在构造之后还能灵活修改，我们就必须把想要修改的值保存下来（见习题 2）。

我们用来表示函数参数的类型 Fct 是什么？它是标准库类型 std::function 的一个变体，这种类型可以“记住”一个函数，留待以后调用。Fct 要求其参数为 double，返回类型也是 double。

15.3.1 默认实参

注意 Function 构造函数的参数 xscale 和 yscale 是如何在声明中给定初始值的。这类初始化器称为默认实参（default argument）：如果调用者没有提供对应的值，就使用它们。例如：

```
Function s {one, r_min, r_max,orig, n_points, x_scale, y_scale};
Function s2 {slope, r_min, r_max, orig, n_points, x_scale};     // 没有 yscale
Function s3 {square, r_min, r_max, orig, n_points};     // 没有 xscale、没有 yscale
Function s4 {sqrt, r_min, r_max, orig};     // 没有 count、没有 xscale、没有 yscale
```

这等价于

```
Function s {one, r_min, r_max, orig, n_points, x_scale, y_scale};
Function s2 {slope, r_min, r_max,orig, n_points, x_scale, 25};
Function s3 {square, r_min, r_max, orig, n_points, 25, 25};
Function s4 {sqrt, r_min, r_max, orig, 100, 25, 25};
```

默认实参可以替代“提供多个重载函数”这种写法。我们本可以定义四个构造函数，而不必定义一个带三个默认实参的构造函数：

```
struct Function : Shape {           // 另一种写法：不用默认实参
    Function(Fct f, double r1, double r2, Point orig,
             int count, double xscale, double yscale);
    // y 使用默认缩放比例
    Function(Fct f, double r1, double r2, Point orig,
             int count, double xscale);
    // x 和 y 都使用默认缩放比例
    Function(Fct f, double r1, double r2, Point orig, int count);
    // count 使用默认值，x 或 y 使用默认缩放比例
    Function(Fct f, double r1, double r2, Point orig);
};
```

定义四个构造函数会更费事；而且在四构造函数版本里，默认值的性质藏在构造函数的定义中，不像在声明里那样一目了然。默认实参常用于构造函数，但对各种函数都有用。只能为末尾的参数定义默认实参。例如：

```
struct Function : Shape {
    Function(Fct f, double r1, double r2, Point orig,
             int count = 100, double xscale, double yscale);    // 错误
};
```

如果一个参数有默认实参，那么它后面的所有参数也都必须有默认实参：

```
struct Function : Shape {
    Function(Fct f, double r1, double r2, Point orig,
             int count = 100, double xscale=25, double yscale=25);
};
```

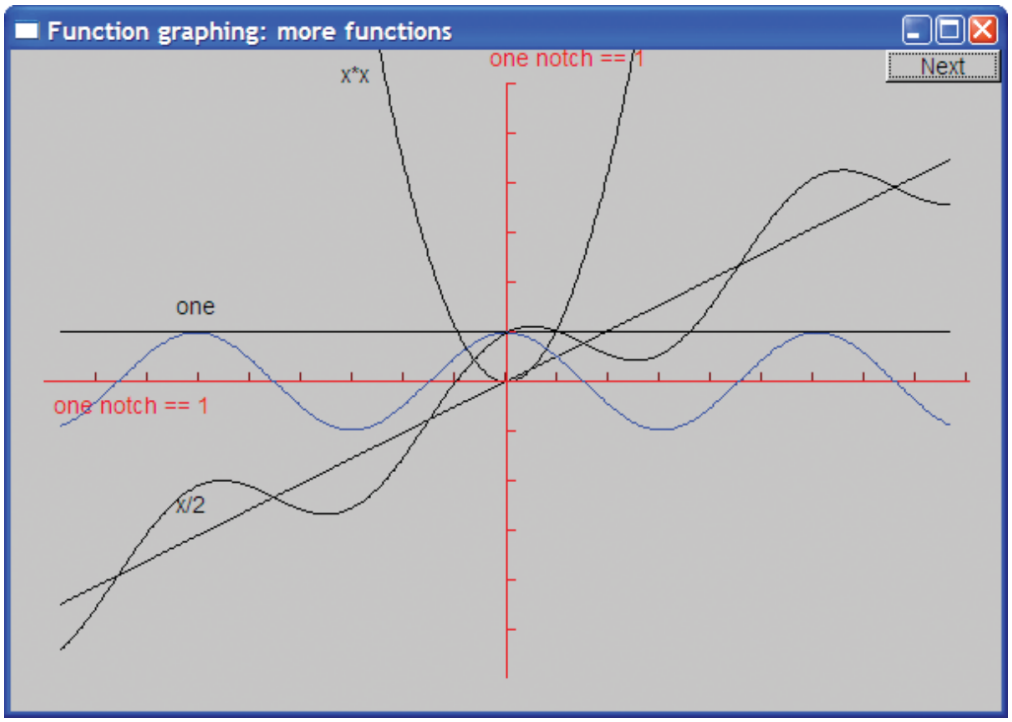
有时挑选好的默认实参很容易，例如 `string` 的默认值（空字符串）和 `vector` 的默认值（空向量）。在另一些情况下，比如 `Function`，选默认值就不那么容易；我们是经过一番试验和一次失败之后，才定下现在用的这几个值的。记住，你不一定要提供默认实参；如果觉得难选，就干脆让用户自己指定那个参数。

15.3.2　更多例子

我们又加了两个函数：一个是标准库中简单的余弦函数 cos，另一个——只是为了展示如何组合函数——是沿着 x/2 斜率的“斜余弦”：

```
double sloping_cos(double x) { return cos(x)+slope(x); }
```

结果如下：



代码如下：

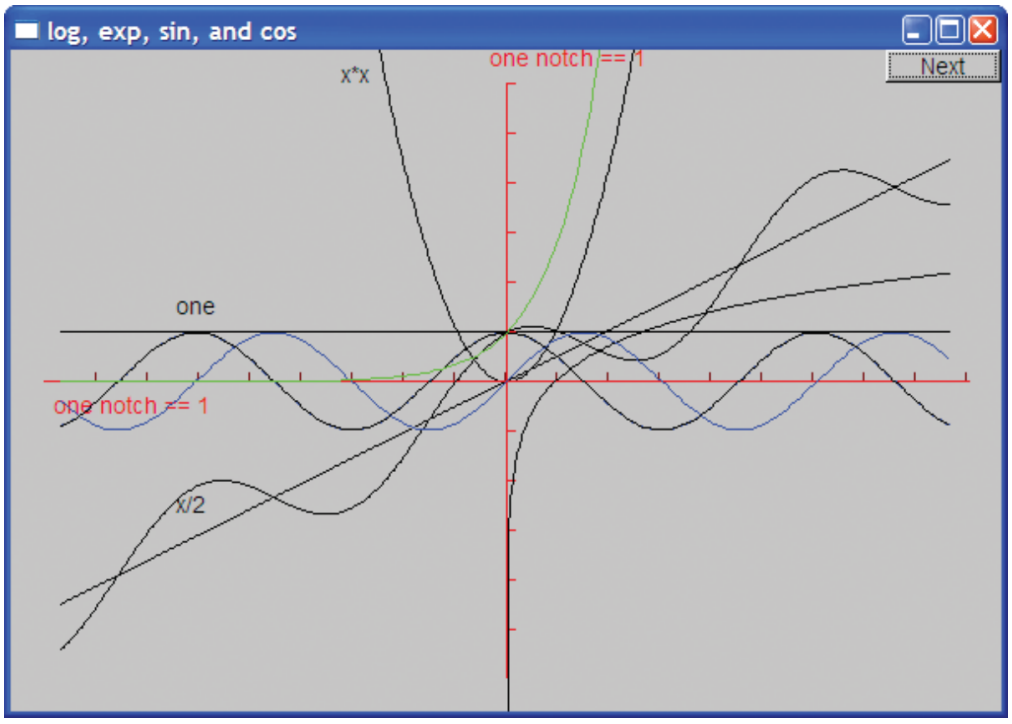
```
Function s4 {cos,r_min,r_max,orig,400,30,30};
s4.set_color(Color::blue);
Function s5 {sloping_cos, r_min,r_max,orig,400,30,30};
x.label.move(-160,0);
x.notches.set_color(Color::dark_red);
```

除了添加这两个函数，我们还移动了 x 轴的标注，并且（只是演示一下做法）稍微改变了它刻度的颜色。

最后，我们绘制一个对数函数、一个指数函数、一个正弦函数和一个余弦函数：

```
Function f1 {log,0.000001,r_max,orig,200,30,30};     // log() 自然对数，底为 e
Function f2 {sin,r_min,r_max,orig,200,30,30};        // sin()
f2.set_color(Color::blue);
Function f3 {cos,r_min,r_max,orig,200,30,30};        // cos()
Function f4 {exp,r_min,r_max,orig,200,30,30};        // exp() 指数函数 e^x
```

由于 log(0) 没有定义（从数学上说，是负无穷），我们把对数函数的区间起点设成一个小正数。结果是



我们没有给这些函数加标注，而是用了颜色。

标准数学函数，如 `cos()`、`sin()` 和 `sqrt()`，声明在标准库头文件 `<cmath>` 中。标准数学函数的清单见 §24.8 和 §B.9.2。

15.3.3　lambda 表达式

为了把一个函数作为参数传给 `Function` 而专门去定义它，有时会让人厌烦。因此，C++ 提供了一种记法，可以在需要的地方、也就是参数位置上，定义一个起函数作用的东西。例如，我们可以这样定义 `sloping_cos` 图形：

```
Function s5 {[](double x) { return cos(x)+slope(x); },
            r_min,r_max,orig,400,30,30};
```

`[](double x) { return cos(x)+slope(x); }` 是一个 lambda 表达式；也就是说，它是一个没有名字的函数，就在需要把它作为实参的地方就地定义。开头的 `[]` 称为 lambda 引导符（lambda introducer）。在引导符之后，lambda 表达式说明需要哪些实参（形参列表），以及要执行哪些动作（函数体）。返回类型可以从 lambda 的函数体推导出来。这里返回类型是 `double`，因为 `cos(x)+slope(x)` 的类型就是 `double`。如果我们愿意，也可以显式指定返回类型：

```
Function s5 {[](double x) -> double { return cos(x)+slope(x); },
            r_min,r_max,orig,400,30,30};
```

给 lambda 表达式指定返回类型很少有必要。主要原因是：lambda 表达式应当保持简单，以免成为错误和混乱的来源。如果一段代码做了相当重要的事情，就应该给它起个名字，而且多半还需要加注释，才能让原作者之外的人看懂。凡是不能轻松放进一两行的东西，我们都建议用有名字的函数来写。

lambda 引导符还可以用来让 lambda 表达式访问局部变量；见 §15.5。另见 §21.4.3。

15.4 坐标轴

凡是展示数据的地方（例如 §15.6.4），我们都会用到 Axis，因为一张图如果缺少让人理解其刻度的信息，通常就很可疑。一个 Axis 由一条线、这条线上的若干“刻度”（notch）以及一个文字标签组成。Axis 的构造函数会算出轴线，并（可选地）算出用作刻度的那些短线：

```
struct Axis : Shape {
    enum Orientation { x, y, z };
    Axis(Orientation d, Point xy, int length,
         int number_of_notches=0, string label = "");
    void draw_lines() const override;
    void move(int dx, int dy) override;
    void set_color(Color c);
    Text label;
    Lines notches;
};
```

label 和 notches 这两个对象被保留为公有，以便用户操作它们。例如，你可以让刻度与轴线使用不同的颜色，也可以用 move() 把标签挪到更方便的位置。Axis 就是一个由若干半独立对象组合而成的对象的例子。

Axis 构造函数放置这些线，并在 number_of_notches 大于 0 时添加“刻度”：

```
Axis::Axis(Orientation d, Point xy, int length, int n, string lab)
    :label(Point{0,0},lab)
{
    if (length<0) error("bad axis length");
    switch (d){
    case Axis::x:
    {
        Shape::add(xy);                     // 轴线
        Shape::add(Point{xy.x+length,xy.y});
        if (0<n) {                          // 添加刻度
            int dist = length/n;
            int x = xy.x+dist;
            for (int i = 0; i<n; ++i) {
                notches.add(Point{x,xy.y},Point{x,xy.y-5});
                x += dist;
            }
        }
        label.move(length/3,xy.y+20);       // 把标签放在线下方
        break;
    }
    case Axis::y:
    {
        Shape::add(xy);                     // y 轴向上延伸
        Shape::add(Point{xy.x,xy.y-length});
```

```
        if (0<n) {                          // 添加刻度
            int dist = length/n;
            int y = xy.y-dist;
            for (int i = 0; i<n; ++i) {
                notches.add(Point{xy.x,y},Point{xy.x+5,y});
                y -= dist;
            }
        }
        label.move(xy.x-10,xy.y-length-10); // 把标签放在顶部
        break;
    }
    case Axis::z:
        error("z axis not implemented");
    }
}
```

与许多真实世界的代码相比，这个构造函数非常简单；但请仔细看一遍，因为它并不完全平凡，而且演示了几项有用的技术。请注意，我们怎样把轴线存进 Axis 的 Shape 部分（用 Shape::add()），而刻度则存在一个单独的对象（notches）里。这样我们就能独立地操作轴线和刻度，例如给两者各自不同的颜色。类似地，标签被放在相对于其坐标轴的固定位置上，但由于它是一个独立对象，我们总可以把它挪到更好的地方。我们使用枚举 Orientation，为用户提供一种方便而不易出错的记法。

由于一个 Axis 有三个部分，当我们要把 Axis 当作整体来操作时，就必须提供相应的函数。例如：

```
void Axis::draw_lines() const
{
    Shape::draw_lines();
    notches.draw();     // 刻度用的颜色可能和线不同
    label.draw();       // 标签用的颜色可能和线不同
}
```

对刻度和标签，我们调用 draw() 而不是 draw_lines()，这样才能用上它们自己保存的颜色。轴线本身存放在 Axis::Shape 里，用的是那里保存的颜色。

我们可以分别设置轴线、刻度和标签的颜色，但从风格上说通常最好不要这样做，因此我们提供一个函数，把这三者设成同一种颜色：

```
void Axis::set_color(Color c)
{
    Shape::set_color(c);
    notches.set_color(c);
    label.set_color(c);
}
```

类似地，Axis::move() 把 Axis 的所有部分一起移动：

```
void Axis::move(int dx, int dy)
{
    Shape::move(dx,dy);
    notches.move(dx,dy);
    label.move(dx,dy);
}
```

15.5　近似

这里我们再给一个小例子，演示如何为函数绘图：我们把指数函数的计算过程“动画化”。目的有几个：帮你对数学函数建立感觉（如果你还没有这种感觉的话），展示图形可以用来图解计算过程，给你一些可读的代码，最后还要提醒一个计算中的常见问题。

计算指数函数的一种办法，是计算下面这个级数

$$e^x == 1 + x + x^2/2! + x^3/3! + x^4/4! + \cdots$$

这个级数中我们计算的项越多，得到的 e^x 值就越精确；也就是说，算的项越多，结果中在数学上正确的位数就越多。我们要做的是：逐项计算这个级数，并在每加进一项之后把结果画出来。这里的感叹号用的是通常的数学含义：阶乘。也就是说，我们按顺序画出下面这些函数：

```
exp0(x) = 0                 // 零项
exp1(x) = 1                 // 一项
exp2(x) = 1+x               // 两项；pow(x,1)/fac(1)==x
exp3(x) = 1+x+pow(x,2)/fac(2)
exp4(x) = 1+x+pow(x,2)/fac(2)+pow(x,3)/fac(3)
exp5(x) = 1+x+pow(x,2)/fac(2)+pow(x,3)/fac(3)+pow(x,4)/fac(4)
. . .
```

每一个函数都是对 e^x 的近似，而且比前一个略好一些。这里 pow(x,n) 是标准库函数，返回 x^n。标准库里没有阶乘函数，所以我们必须自己定义一个：

```
int fac(int n)          // 阶乘 factorial(n)；n!
{
    int r = 1;
    while (n>1) {
        r*=n;
        --n;
    }
    return r;
}
```

关于 fac() 的另一种实现，见习题 1。有了 fac()，我们就可以这样计算级数的第 n 项：

```
double term(double x, int n) { return pow(x,n)/fac(n); }    // 级数的第 n 项
```

有了 term()，要按 n 项的精度算出指数函数就很容易了：

```
double expe(double x, int n)            // x 的 n 项之和
{
    double sum = 0;
    for (int i=0; i<n; ++i) sum+=term(x,i);
    return sum;
}
```

下面用它来画点图。首先准备一些坐标轴和“真正的”指数函数，即标准库的 exp()，这样就能看出用 expe() 得到的近似有多接近：

```
Function real_exp {exp,r_min,r_max,orig,200,x_scale,y_scale};
real_exp.set_color(Color::blue);
```

可是怎样使用 expe() 呢？从编程的角度看，困难在于：我们的绘图类 Function 接受一个只带一个实参的函数，而 expe() 需要两个实参。就我们目前所见的 C++ 而言，这个问题没有真正优雅的解法。不过，lambda 表达式提供了一条路（§15.3.3）。请看：

```
for (int n = 0; n<50; ++n) {
    ostringstream ss;
    ss << "exp approximation; n==" << n ;
    win.set_label(ss.str());
    // 取下一个近似:
    Function e {[n](double x) { return expe(x,n); },
                r_min,r_max,orig,200,x_scale,y_scale};
    win.attach(e);
    win.wait_for_button();
    win.detach(e);
}
```

lambda 引导符 [n] 表示这个 lambda 表达式可以访问局部变量 n。这样一来，调用 expe(x,n) 时，n 在创建 Function 时就已经确定，而 x 则来自 Function 内部的每一次调用。

注意那个循环最后的 detach(e)。Function 对象 e 的作用域是 for 语句的那个块。每次进入这个块，我们都会得到一个名为 e 的新 Function；每次退出这个块，那个 e 就消失，由下一个取代。窗口不能记着旧的 e，因为它已经被销毁了。因此，detach(e) 保证窗口不会试图去画一个已销毁的对象。

这段代码首先给出一个窗口，其中只有坐标轴和用蓝色画出的“真正的”指数函数：

我们看到 $\exp(0)$ 等于 1，所以蓝色的“真正的指数函数”在 $(0,1)$ 处穿过 y 轴。

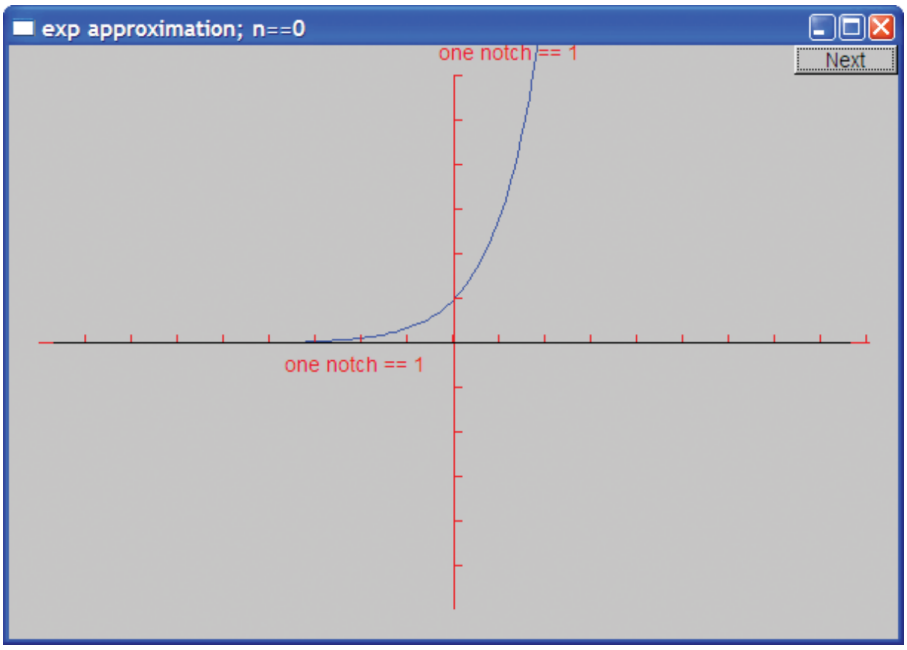
如果仔细看，你会发现我们其实把零项近似（exp0(x)==0）画成了一条黑线，正好压在 x 轴上。按一下“下一个”，就得到只用一项的近似。注意我们在窗口标签里显示了这次近似所用的项数：

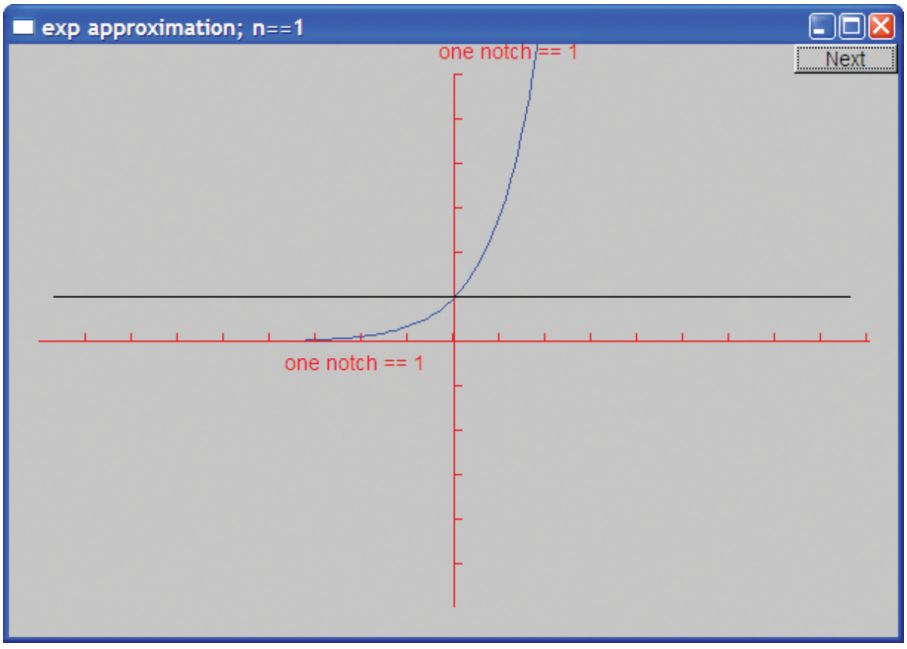
这就是函数 exp1(x)==1，即只用级数一项的近似。它在 $(0,1)$ 处与指数函数完全吻合，不过我们还能做得更好：

取两项（$1+x$）时，我们得到一条斜线，它在 $(0,1)$ 处穿过 y 轴。取三项（1+x+pow(x,2)/fac(2)）时，已经能看到开始收敛的迹象：

取十项时，效果已经相当不错，尤其是对大于 -3 的值：

如果我们不去细想，就可能以为：只要用越来越多的项，近似就会越来越好。然而，这里存在

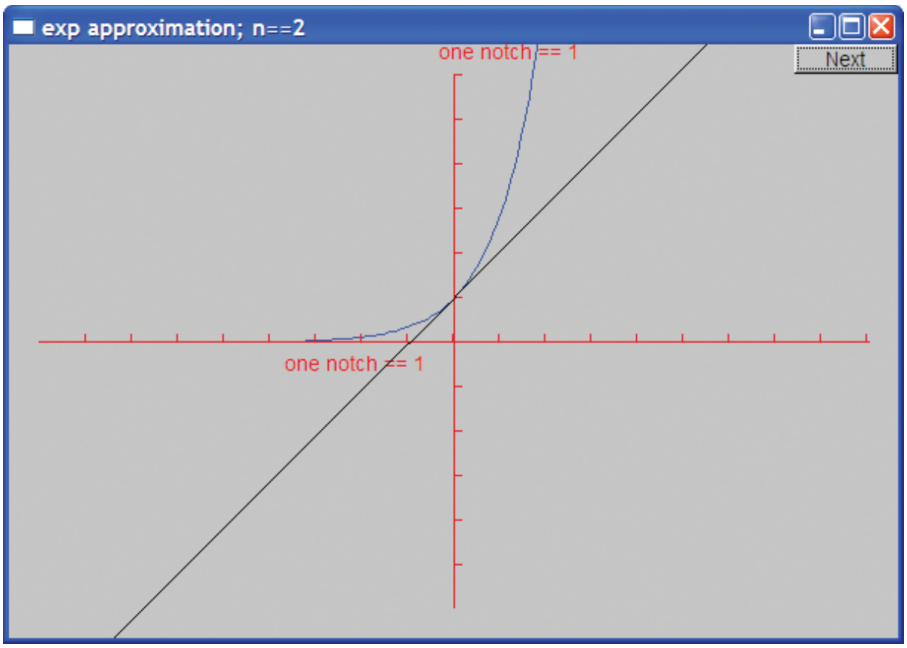


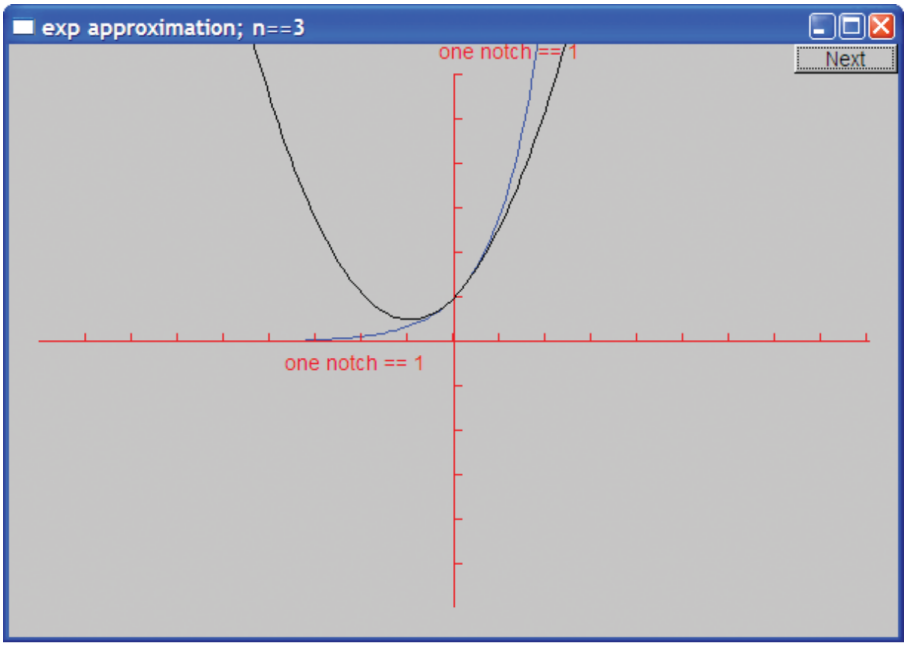


……是有限度的：从 13 项之后，就开始发生奇怪的事情。首先，近似的效果开始略微变差；到 18 项时，图上出现了竖直的线：

请记住，计算机的算术并不是纯粹的数学。浮点数不过是我们用固定位数所能得到的、对实数尽可能好的近似。往 `int` 里放一个过大的整数，它就会溢出；而 `double` 存的只是一个近似值。当我看到项数较多时出现的奇怪输出，我首先怀疑是计算开始产生无法用 `double` 表示的值，于是结果逐渐偏离数学上正确的答案。后来我才明白，是 `fac()` 产生了无法存进 `int` 的值。把 `fac()` 改成返回 `double`，问题就解决了。更多信息见第 5 章习题 11 和 §24.2。

最后这张图还很好地说明了一条原则：“看起来没问题”并不等于“测试过了”。把程序交给别人使用之前，先按照超出你最初认为合理的程度去测试它。除非你更有把握，否则让程序多跑一小会儿，或者换一组略有不同的数据，都可能造成真正的混乱——本例就是如此。



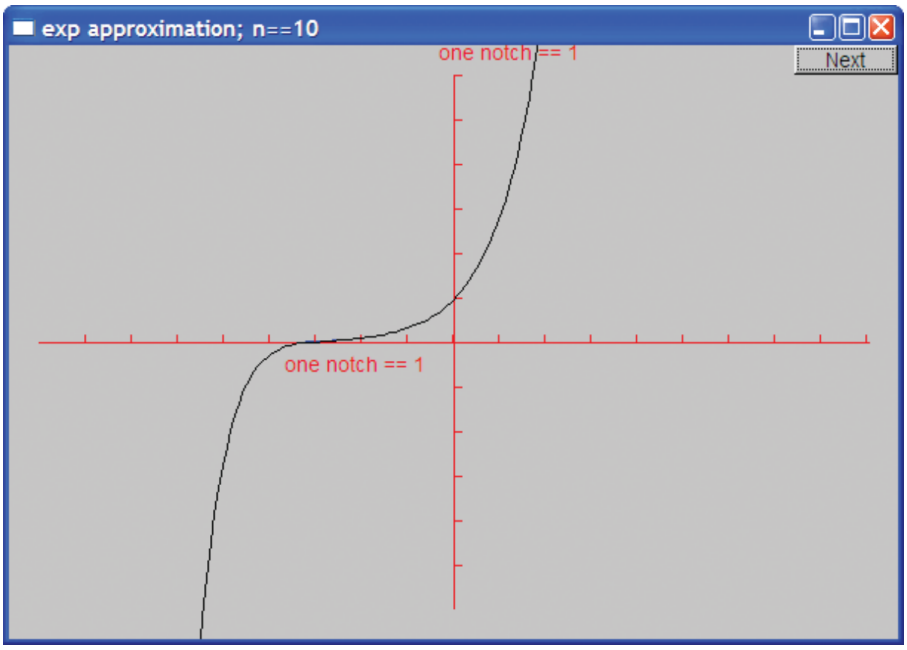


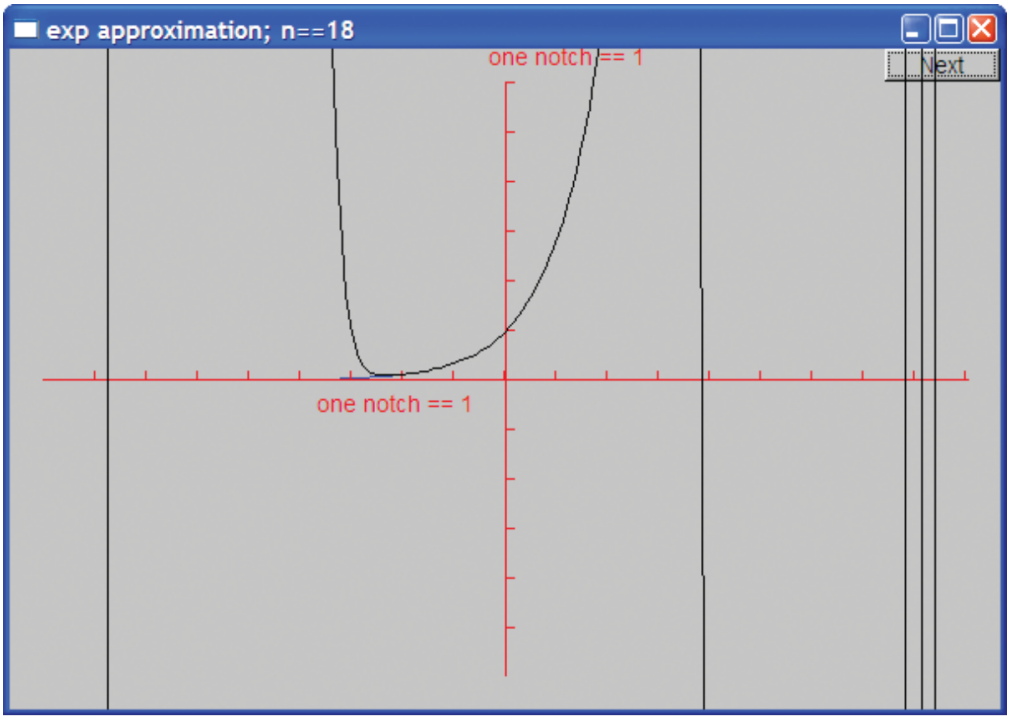
15.6 绘制数据

展示数据是一门技艺，需要高超的技能，也极受重视。做得好的时候，它把技术与艺术两方面结合起来，能显著增进我们对复杂现象的理解。然而也正因如此，绘图是一个极大的领域，其中大部分内容与编程技术无关。这里我们只给出一个简单的例子：显示从文件读入的数据。所示数据表示近一个世纪以来日本人口的年龄构成。2008 年线右侧的数据是预测值：

我们就用这个例子来讨论呈现这类数据所涉及的编程问题：

- 读取文件
- 缩放数据，使它适合窗口
- 显示数据
- 为图形添加标注

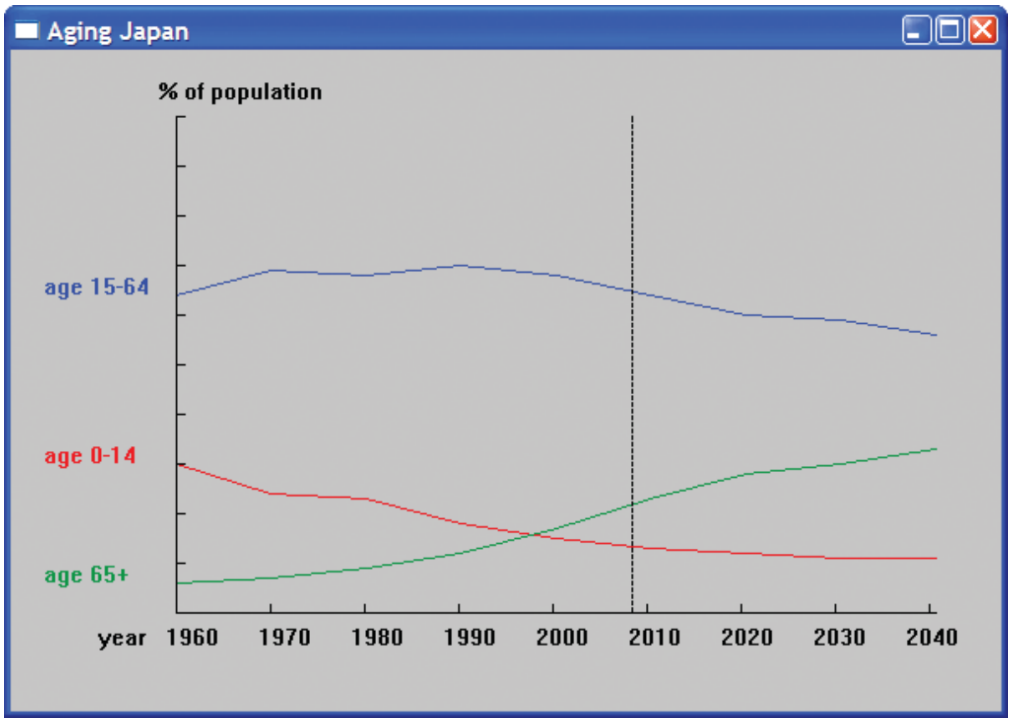




我们不打算深入艺术上的细节。说到底，这是“给极客看的图”，而不是“图形艺术”。当然，真正需要的时候，你可以在艺术上做得更好。

给定一组数据，我们必须考虑怎样把它显示得最好。为简化起见，我们只处理容易用二维方式显示的数据，而这已经涵盖了大多数人要处理的数据中的很大一部分。请注意，条形图、饼图以及诸如此类的常见图形，其实都只是用花哨方式显示的二维数据。三维数据常常可以通过下列方式处理：生成一系列二维图像；把若干二维图形叠加到同一个窗口中（“日本年龄”这个例子就是这么做的）；或者给各个数据点标注信息。如果想再进一步，我们就得编写新的图形类，或者改用另一个图形库。

所以，我们的数据基本上都是成对的值，例如（年份，儿童人数）。如果数据更多，例如（年份，儿童人数，成年人数，老年人数），我们只需决定要画哪一对值——或哪几对值。在我们的例子里，我们只画了（年份，儿童人数）、（年份，成年人数）和（年份，老年人数）。



看一组 (x,y) 值对有很多种方式。在考虑怎样画这样一组数据时，重要的是判断其中一个值在某种意义上是否是另一个值的函数。例如对于（年份，钢产量）这样一对值，把钢产量视为年份的函数、并把数据画成一条连续的折线，是相当合理的。`Open_polyline`（§13.6）就是画这类数据的显然选择。如果 y 不应看作 x 的函数，例如（人均国内生产总值，国家人口），则可以用 `Marks`（§13.15）来画彼此不相连的点。

现在回到我们的日本年龄分布这个例子。

15.6.1　读取文件

年龄分布文件由下面这样的一些行组成：

```
( 1960 : 30 64 6 )
(1970 : 24 69 7 )
(1980 : 23 68 9 )
```

冒号后面的第一个数是儿童（0-14 岁）在人口中所占的百分比，第二个是成年人（15-64 岁）的百分比，第三个是老年人（65 岁以上）的百分比。我们的任务就是把这些数读进来。请注意，数据的格式略有些不规则。和往常一样，这类细节得由我们自己来处理。

为了简化这项任务，我们先定义一个类型 `Distribution` 来存放一个数据项，并定义一个输入运算符来读取这样的数据项：

```
struct Distribution {
    int year, young, middle, old;
};

istream& operator>>(istream& is, Distribution& d)
    // 假定格式: ( year : young middle old )
{
    char ch1 = 0;
    char ch2 = 0;
    char ch3 = 0;
    Distribution dd;
    if (is >> ch1 >> dd.year
            >> ch2 >> dd.young >> dd.middle >> dd.old
            >> ch3) {
        if (ch1!= '(' || ch2!=':' || ch3!=')') {
            is.clear(ios_base::failbit);
            return is;
        }
    }
    else
        return is;
    d = dd;
    return is;
}
```

这是第 10 章那些思想的直接应用。如果这段代码你看不明白，请复习那一章。我们并不需要定义 Distribution 类型和 >> 运算符。但与“把数字读进来直接画”这种蛮干做法相比，它简化了代码。用 Distribution 把代码划分成几个逻辑部分，有助于理解和调试。不要不好意思“只是为了让代码更清晰”而引入类型。我们定义类，是为了让代码更直接地对应我们对代码中概念的思考方式。即便对只在局部很小范围内使用的“小”概念——比如表示某一年年龄分布的一行数据——这样做也很有帮助。

有了 Distribution，读取循环就变成

```
string file_name = "japanese-age-data.txt";
ifstream ifs {file_name};
if (!ifs) error("can't open ",file_name);

// . . .

for (Distribution d; ifs>>d; ) {
    if (d.year<base_year || end_year<d.year)
        error("year out of range");
    if (d.young+d.middle+d.old != 100)
        error("percentages don't add up");
    // . . .
}
```

也就是说，我们尝试打开文件 japanese-age-data.txt；如果找不到这个文件，就退出程序。像这里这样把文件名“硬编码”到源代码中，通常不是好主意；不过我们把这个程序看作一次性的小练习，所以不为它增添那些更适合长期使用的应用程序的设施。另一方面，我们确实把 japanese-age-data.txt 放进了具名的 string 变量，这样如果想把这段程序——或其中一部分代码——用于别的事情，修改起来就很容易。

读取循环会检查读到的年份是否落在预期范围内，以及各个百分比之和是否为 100。这是对数据的基本合理性检查。由于 >> 会检查每个数据项自身的格式，我们就没有在主循环里再做更多检查。

15.6.2 总体布局

那么，我们希望屏幕上出现什么？答案可以在 §15.6 的开头看到。这些数据似乎要求画三条 Open_polyline——每个年龄组一条。这些图形需要标注，我们决定在窗口左侧为每条线写一个“图注”。与另一种常见做法——把标签放在线本身上面的某处——相比，在这个例子里这样做更清楚。此外，我们用颜色区分各条曲线，并让标签与颜色相互对应。

我们想用年份标注 x 轴。穿过 2008 年的那条竖线表示图形从实际数据转入预测数据的位置。

我们决定直接用窗口的标签作为图形的标题。

把绘图代码写得既正确又好看，可能出人意料地棘手。主要原因是我们要做大量关于尺寸和偏移的琐碎计算。为了简化这些工作，我们先定义一组符号常量，用来规定我们怎样使用屏幕空间：

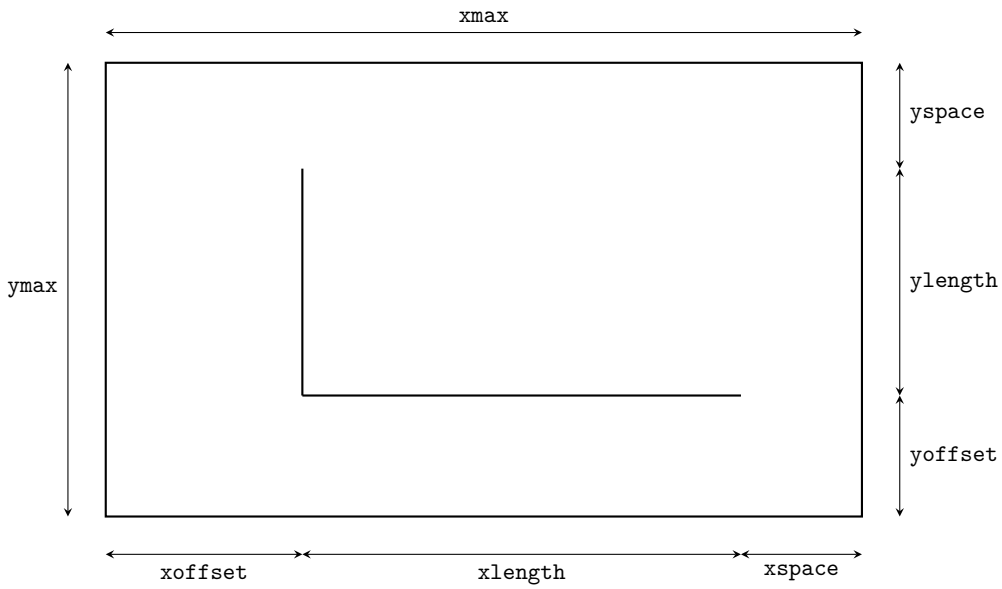
```
constexpr int xmax = 600;     // 窗口大小
constexpr int ymax = 400;

constexpr int xoffset = 100;  // 从窗口左侧到 y 轴的距离
constexpr int yoffset = 60;   // 从窗口底部到 x 轴的距离

constexpr int xspace = 40;    // 坐标轴之外留出的空间
constexpr int yspace = 40;

constexpr int xlength = xmax-xoffset-xspace;          // 坐标轴的长度
constexpr int ylength = ymax-yoffset-yspace;
```

大体上说，这定义了一个矩形空间（窗口），其中又包含另一个矩形（由坐标轴定义）：

我们发现，如果没有这样一幅“示意图”说明窗口中各个东西的位置，也没有定义这些位置的符号常量，我们就会迷失方向；一旦输出不符合我们的意愿，就会变得很沮丧。

15.6.3 缩放数据

接下来要定义怎样把数据装进那个空间。做法是对数据做缩放，使它适合坐标轴所定义的空间。为此需要缩放因子，也就是数据范围与坐标轴范围之比：




```
constexpr int base_year = 1960;
constexpr int end_year = 2040;

constexpr double xscale = double(xlength)/(end_year-base_year);
constexpr double yscale = double(ylength)/100;
```

我们希望缩放因子（xscale 和 yscale）是浮点数——否则计算可能受到严重的舍入误差影响。为了避免整数除法，我们在做除法之前先把长度转换成 double（§4.3.3）。

现在，要把一个数据点放到 x 轴上，只需减去它的基准值（1960），用 xscale 缩放，再加上 xoffset。y 值的处理与此类似。我们发现，反复做这件事时，总是记不住怎样才算做对。这也许只是平凡的计算，但琐碎而冗长。为了简化代码、把这个出错机会降到最低（也把令人沮丧的调试降到最低），我们定义一个小类来替我们完成这项计算：

```
class Scale {                   // 数据值到坐标的转换
    int cbase;                  // 坐标基准
    int vbase;                  // 值的基准
    double scale;
public:
    Scale(int b, int vb, double s) :cbase{b}, vbase{vb}, scale{s} { }
    int operator()(int v) const { return cbase + (v-vbase)*scale; } // 见 §21.4
};
```

我们要的是一个类，因为这个计算依赖三个常量值，而我们不希望不必要地重复它们。有了这个类，就可以定义

```
Scale xs {xoffset,base_year,xscale};
Scale ys {ymax-yoffset,0,-yscale};
```

请注意，我们把 ys 的缩放因子取成负数，以反映这样一个事实：y 坐标是向下增长的，而我们通常更愿意让较大的值对应图上较高的位置。现在可以用 xs 把一个年份转换成 x 坐标。同样，可以用 ys 把一个百分比转换成 y 坐标。

15.6.4 构建图形

终于，我们具备了以相当优雅的方式编写绘图代码所需的全部前提。我们先创建一个窗口，并放置坐标轴：

```
Window win {Point{100,100},xmax,ymax,"Aging Japan"};

Axis x {Axis::x, Point{xoffset,ymax-yoffset}, xlength,
    (end_year-base_year)/10,
    "year      1960      1970       1980      1990      "
    "2000          2010      2020      2030      2040"};
x.label.move(-100,0);

Axis y {Axis::y, Point{xoffset,ymax-yoffset}, ylength, 10,"% of population"};

Line current_year {Point{xs(2008),ys(0)},Point{xs(2008),ys(100)}};
current_year.set_style(Line_style::dash);
```

两轴交于 `Point{xoffset,ymax-yoffset}`，它代表 (1960,0)。请注意刻度是怎样摆放以反映数据的。y 轴上有十个刻度，每个刻度代表人口的 10%。x 轴上每个刻度代表十年，而刻度的确切数目由 `base_year` 与 `end_year` 计算得出，所以一旦改动这个范围，坐标轴就会自动重新计算。这正是避免在代码里使用“魔法常量”的一个好处。x 轴上的标签却违反了这条规则：它不过是反复摆弄标签字符串、直到数字正好落在各刻度下方的结果。要想做得更好，我们就得为每个“刻度”分别指定标签。

请注意标签字符串那种奇怪的排版方式。我们用了两个相邻的字符串字面量：

```
"year
1960
1970
1980
1990          "
"2000
2010
2020
2030
2040"
```

相邻的字符串字面量会被编译器拼接起来，因此它等价于：

```
"year     1960     1970     1980     1990     2000     2010     2020     2030     2040"
```

对于排布很长的字符串字面量来说，这是个有用的“技巧”，能让我们的代码更容易阅读。

`current_year` 是一条竖直线，用来把实测数据与预测数据分开。请注意 `xs` 和 `ys` 是怎样被用来把这条线放到恰到好处的位置并恰到好处地缩放的。

有了坐标轴，我们就可以来处理数据了。我们定义三个 `Open_polyline`，并在读取循环中把它们填满：

```
Open_polyline children;
Open_polyline adults;
Open_polyline aged;

for (Distribution d; ifs>>d; ) {
    if (d.year<base_year || end_year<d.year) error("year out of range");
    if (d.young+d.middle+d.old != 100)
        error("percentages don't add up");
    const int x = xs(d.year);
    children.add(Point{x,ys(d.young)});
    adults.add(Point{x,ys(d.middle)});
    aged.add(Point{x,ys(d.old)});
}
```

xs 和 ys 的运用使数据的缩放与摆放变得微不足道。像 Scale 这样的“小类”，对于简化记法、避免不必要的重复可能极其重要——从而提高可读性，也提高正确性的可能性。

为了让图形更容易看懂，我们给每条曲线加上标签并着色：

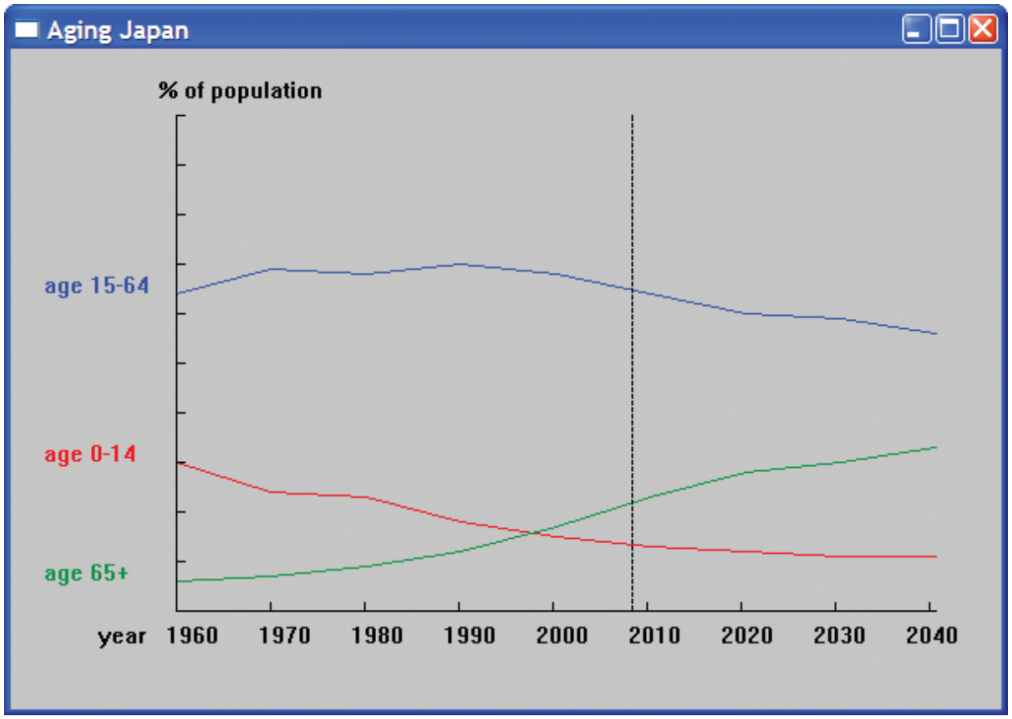
```
Text children_label {Point{20,children.point(0).y},"age 0-14"};
children.set_color(Color::red);
children_label.set_color(Color::red);

Text adults_label {Point{20,adults.point(0).y},"age 15-64"};
adults.set_color(Color::blue);
adults_label.set_color(Color::blue);

Text aged_label {Point{20,aged.point(0).y},"age 65+"};
aged.set_color(Color::dark_green);
aged_label.set_color(Color::dark_green);
```

最后，我们需要把各种 Shape 挂到 Window 上，并启动图形用户界面系统（§14.2.3）：

```
win.attach(children);
win.attach(adults);
win.attach(aged);

win.attach(children_label);
win.attach(adults_label);
win.attach(aged_label);

win.attach(x);
win.attach(y);
win.attach(current_year);

gui_main();
```

所有代码都可以放进 `main()` 里，但我们更愿意把辅助类 `Scale` 和 `Distribution` 连同 `Distribution` 的输入运算符一起留在外面。

万一你已经忘了我们做出来的东西长什么样，这里再给出一次输出：



小试身手

函数绘图练习：

1. 创建一个 600 × 600 的空 `Window`，标题为“Function graphs”。
2. 注意：你需要按课程网站上“安装 FLTK”的说明里指定的属性来建立项目。
3. 你需要把 `Graph.cpp` 和 `Window.cpp` 移入你的项目。
4. 添加一条 x 轴和一条 y 轴，长度均为 400，标注为“1 == 20 pixels”，并且每隔 20 像素一个刻度。两轴应相交于 (300,300)。
5. 把两条轴都设为红色。

在下面各题中，为每个要绘制的函数使用单独的 `Shape`：

1. 在区间 $[-10, 11]$ 内绘制函数 `double one(double x) { return 1; }`，把 (0,0) 放在 (300,300)，使用 400 个点，并且（在窗口内）不做缩放。
2. 改成使用 x 缩放 20、y 缩放 20。
3. 从现在起，所有图形都使用这个区间、这个缩放等设置。
4. 把 `double slope(double x) { return x/2; }` 加到窗口中。
5. 在 slope 曲线左下端点略上方的一点处，用 `Text` 给它加一个内容为“x/2”的标签。
6. 把 `double square(double x) { return x*x; }` 加到窗口中。
7. 往窗口里加一条余弦曲线（不要新写函数）。
8. 把余弦曲线设为蓝色。
9. 写一个函数 `sloping_cos()`，它把一条余弦加到 `slope()`（按上面的定义）上，并把它加到窗口中。

类定义练习：

1. 定义一个 struct Person，其中包含一个 string name 和一个 int age。
2. 定义一个 Person 类型的变量，用“Goofy”和 63 初始化它，并把它输出到屏幕（cout）。
3. 为 Person 定义输入（>>）和输出（<<）运算符；从键盘（cin）读入一个 Person，再把它输出到屏幕（cout）。
4. 给 Person 添加一个构造函数，初始化 name 和 age。
5. 把 Person 的表示设为私有，并提供 const 成员函数 name() 和 age() 来读取名字和年龄。
6. 修改 >> 和 <<，使它们适用于重新定义后的 Person。
7. 修改构造函数，检查 age 是否在 [0:150) 内，以及 name 是否不含下列任何字符：分号、冒号、双引号、单引号、方括号、星号、与号、脱字符、百分号、美元符号、井号、@ 号、感叹号。出错时使用 error()。测试。
8. 从输入（cin）读入一串 Person 到 vector<Person> 中；再把它们输出到屏幕（cout）。用正确的输入和错误的输入分别测试。
9. 把 Person 的表示改成用 first_name 和 second_name 代替 name。若没有同时提供名字和姓氏，就报错。切记同时修正 >> 和 <<。测试。

要点回顾

1. 什么是只有一个实参的函数？
2. 什么时候你会用（连续的）直线来表示数据？什么时候用（离散的）点？
3. 什么函数（数学公式）定义斜率？
4. 什么是抛物线？
5. 你怎样做出一条 x 轴？怎样做出一条 y 轴？
6. 什么是默认实参？什么时候会用到它？
7. 你怎样把若干函数加在一起？
8. 你怎样给绘制出来的函数着色并加标签？
9. 当我们说一个级数逼近某个函数时，是什么意思？
10. 为什么在写绘图代码之前要先勾画出图形的布局？
11. 你会怎样缩放图形，使输入数据能够放得下？
12. 你怎样不经反复试验就确定输入的缩放比例？
13. 为什么你要对输入进行格式化，而不是让文件里只放“那些数字”？
14. 你怎样规划图形的总体布局？又怎样在代码中体现这一布局？

术语

- 近似
- 函数
- 缩放
- 默认实参
- lambda
- 屏幕布局

习题

1. 下面是定义阶乘函数的另一种方式：

```
int fac(int n) { return n>1 ? n*fac(n-1) : 1; }    // n 的阶乘
```

它会先判定 `4>1`，于是 `fac(4)` 必定是 `4*fac(3)`，而后者显然就是 `4*3*fac(2)`，接着是 `4*3*2*fac(1)`，也就是 `4*3*2*1`。请试着看出它确实能工作。调用自身的函数称为递归（recursive）。§15.5 中的另一种实现称为迭代（iterative），因为它（用 `while`）逐个遍历各个值。请计算 0、1、2、3、4 直到 20 的阶乘，验证递归版 `fac()` 确实能工作，并且与迭代版 `fac()` 给出相同的结果。你更喜欢哪一种 `fac()` 实现？为什么？

2. 定义一个类 `Fct`，它与 `Function` 完全一样，只是会保存其构造函数的实参。为 `Fct` 提供“重置”操作，以便你能针对不同的区间、不同的函数等反复使用它。
3. 修改上一题中的 `Fct`，使它多接受一个实参，用来控制精度或别的什么。把该实参的类型做成模板参数，以获得额外的灵活性。
4. 在同一张图上绘制正弦（`sin()`）、余弦（`cos()`）、二者之和（`sin(x)+cos(x)`），以及二者平方之和（`sin(x)*sin(x)+cos(x)*cos(x)`）。务必提供坐标轴和标签。
5. 像 §15.5 那样“动画”演示级数 $1 - 1/3 + 1/5 - 1/7 + 1/9 - 1/11 + ...$。它称为莱布尼茨级数（Leibniz's series），收敛到 $\pi/4$。
6. 设计并实现一个条形图类。它的基本数据是一个 `vector<double>`，存放 N 个值；每个值用一个“条”来表示，条是一个矩形，其高度代表该值。
7. 扩展这个条形图类，使它能够给图本身以及各个条加标签。允许使用颜色。
8. 这里有一组以厘米为单位的身高，以及某个群体中具有该身高（四舍五入到最近的 5cm）的人数：(170,7)、(175,9)、(180,23)、(185,17)、(190,6)、(195,1)。你会怎样把这样的数据画成图？如果想不到更好的办法，就画条形图。记得提供坐标轴和标签。把数据放进文件，并从该文件中读取。
9. 再找一组身高的数据（1 英寸 = 2.54cm），用上一题的程序把它们画出来。例如，在网上搜索“height distribution”或“height of people in the United States”，忽略一大堆垃圾信息，或者问问朋友们的身高。理想情况下，换用新的数据集时你不必改动任何东西。由数据算出缩放比例是一个关键想法。从输入中读取标签也有助于在你想要重用代码时把改动降到最少。
10. 什么样的数据不适合用折线图或条形图？找出一个例子，并找出一种显示它的办法（例如显示为一组带标签的点）。
11. 找出两个或更多地点（例如英国剑桥和美国马萨诸塞州剑桥；叫“Cambridge”的城镇有很多）一年中各月的平均最高气温，并把它们画在一起。一如既往，注意坐标轴、标签、颜色的使用等。

后记

数据的图形化表示很重要。对一张精心绘制的图，我们就是比对用来绘制它的那组数字理解得更好。大多数人在需要画图时，会用别人写好的代码——一个库。这样的库是怎样构造出来的？手边没有库时又该怎么办？“一个普通的绘图工具”背后的基本想法是什么？现在你知道了：它既不是魔法，也不是什么高深莫测的事情。我们只讨论了二维图形；三维绘图在科学、工程、市场营销等领域也非常有用，而且可能更有趣。将来去探索它吧！

16　图形用户界面

"Computing is not about computers any more.
It is about living."
("计算不再只关乎计算机，它关乎生活。")
——Nicholas Negroponte

图形用户界面（graphical user interface，GUI）让用户可以通过按键、从菜单中选择、以各种方式录入数据，以及在屏幕上显示文本与图形实体，来与程序交互。我们与自己的计算机、与网站打交道时，习惯的正是这种方式。本章介绍如何编写代码来定义并控制一个 GUI 应用程序的基本知识。特别地，我们演示如何用回调（callback）编写与屏幕上实体交互的代码。我们的 GUI 设施是"架在"系统设施之上的。底层特性与接口放在附录 E 中介绍，那里用到第 17、18 章讲述的特性与技术。这里我们只关注用法。

16.1　用户界面的其他选择
16.2　"Next"按钮
16.3　一个简单的窗口
　　16.3.1　回调函数
　　16.3.2　等待循环
　　16.3.3　用 lambda 表达式作回调
16.4　按钮与其他控件
　　16.4.1　控件
　　16.4.2　按钮
　　16.4.3　`In_box` 与 `Out_box`
　　16.4.4　菜单
16.5　一个例子
16.6　控制反转
16.7　添加菜单
16.8　调试 GUI 代码

16.1　用户界面的其他选择

每个程序都有用户界面。运行在小型装置上的程序，输入可能只有几个按钮，输出则只是一盏闪烁的灯。还有些计算机只通过一根连线与外部世界相连。这里我们考虑最常见的情形：程序与一位盯着屏幕、使用键盘和指点设备（例如鼠标）的用户通信。在这种情形下，作为程序员，我们主要有三种选择：

- 使用控制台输入输出：对于输入简单、以文本为主（由命令和短小的数据项组成，例如文件名和简单的数据值）的技术性、专业性工作，这是一种很有竞争力的选择。如果输出是文本，我们可以把它显示在屏幕上，也可以存进文件。C++ 标准库的 iostream（第 10～11 章）提供了合适而方便的机制。如果需要图形输出，我们可以使用图形显示库（如第 12～15 章所示），而不必对编程风格作剧烈的改变。

- 使用图形用户界面（GUI）库：当我们希望用户交互建立在“操作屏幕上的对象”这一隐喻之上（指向、点击、拖放、悬停等）时，就这么做。这种风格常常（但并非总是）伴随着大量以图形方式显示的信息。用过现代计算机的人都见过这样用起来很方便的例子。谁想模仿 Windows/Mac 应用程序的“手感”，就必须采用 GUI 风格的交互。
- 使用 Web 浏览器界面：为此我们需要使用标记（版面）语言，例如 HTML，通常还要用脚本语言。演示具体做法超出了本书的范围，但对于需要远程访问的应用程序，这往往是最理想的方案。在这种情况下，程序与屏幕之间的通信仍然是文本的（使用字符流）。浏览器本身就是一个 GUI 应用程序：它把其中一部分文本翻译成图形元素，又把鼠标点击等动作翻译成可以送回程序的文本数据。

在许多人看来，使用 GUI 就是现代编程的精髓，有时甚至把与屏幕上对象的交互当成编程的核心关注点。我们不同意这种看法：GUI 只是一种 I/O 形式，而把应用程序的主体逻辑与 I/O 分离，是我们对软件的主要理想之一。只要可能，我们就希望在程序主体逻辑与负责输入输出的那部分代码之间保持清晰的接口。这样的分离使我们能够改变程序呈现给用户的方式，把程序移植到不同的 I/O 系统上，而且——最重要的是——能够分别思考程序的逻辑和它与用户的交互。

话虽如此，从好几个角度看，GUI 都是重要而有趣的。本章既要探讨把图形元素集成到应用程序中的各种方式，也要探讨如何避免让界面的琐事主导我们的思路。

16.2 “Next”按钮

第 12～15 章中用来驱动图形例子的那个“Next”按钮，我们是怎么提供的？在那里，我们用按钮在窗口中作图。显然，那是一种简单的 GUI 编程形式。事实上它简单到有人会说它算不上“真正的 GUI”。不过，我们还是来看看它是怎么做的，因为它会直接引向大家都认作 GUI 编程的那种编程方式。

我们在第 12～15 章中的代码习惯上写成这样：

```
// 创建对象和/或操作对象，把它们显示在 Window win 中：
win.wait_for_button();
// 创建对象和/或操作对象，把它们显示在 Window win 中：
win.wait_for_button();
// 创建对象和/或操作对象，把它们显示在 Window win 中：
win.wait_for_button();
```

每次执行到 `wait_for_button()`，我们都可以在屏幕上观察自己的对象，直到按下按钮，才取到程序下一部分的输出。从程序逻辑的角度看，这与一个把输出行写到屏幕上（控制台窗口）、时不时停下来接收键盘输入的程序没有区别。例如：

```
// 定义变量和/或计算值，产生输出
cin >> var;       // 等待输入
// 定义变量和/或计算值，产生输出
cin >> var;       // 等待输入
// 定义变量和/或计算值，产生输出
cin >> var;       // 等待输入
```

从实现的角度看，这两类程序差别很大。程序执行 `cin >> var` 时会停下来，等“系统”把你敲入的字符送回来。然而，负责照看屏幕、并在你使用鼠标时跟踪它的“系统”（图形用户界面系统）运行在相当不同的模型上：GUI 一直在记录鼠标在哪儿、用户在用鼠标做什么（点击等）。当程序需要一个动作时，它必须

- 告诉 GUI 要留意什么（例如“有人点击了‘Next’按钮”）
- 告诉 GUI 有人这样做时该做什么
- 一直等到 GUI 检测到程序所关心的动作

这里新颖而不同之处在于：GUI 并不只是返回到我们的程序；它被设计成对不同的用户动作作出不同的响应，例如点击许多按钮中的一个、缩放窗口、在窗口被另一个窗口遮挡之后重画它、弹出弹出式菜单。

作为起步，我们只想说：“当有人点击我的按钮时，请唤醒我。”也就是说，“当有人按下鼠标按钮、并且光标位于显示我那个按钮图像的矩形区域内时，请让我的程序继续执行。”这差不多是我们能想到的最简单的动作了。然而，这样的操作“系统”并没有提供，所以我们自己写了一个。看清它是怎么做的，是理解 GUI 编程的第一步。

16.3 一个简单的窗口

基本上，“系统”（由一个 GUI 库和操作系统共同构成）在持续跟踪鼠标的位置以及它的按钮是否被按下。程序可以表示自己对屏幕上的某个区域感兴趣，并请“系统”在“有值得注意的事情”发生时调用一个函数。在这个具体例子里，我们请系统在鼠标按钮“在我们的按钮上”被点击时调用我们的一个函数（“回调函数”）。为此我们必须

- 定义一个按钮
- 让它显示出来
- 定义一个供 GUI 调用的函数
- 把那个按钮和那个函数告诉 GUI
- 等待 GUI 调用我们的函数

我们就这么做。按钮是 `Window` 的一部分，所以（在 `Simple_window.h` 中）我们定义自己的类 `Simple_window`，让它包含成员 `next_button`：

```
struct Simple_window : Graph_lib::Window {
    Simple_window(Point xy, int w, int h, const string& title);
    void wait_for_button();                 // 简单的事件循环

private:
    Button next_button;                     //  “Next” 按钮
    bool button_pushed;                     // 实现细节

    static void cb_next(Address, Address);  // next_button 的回调
    void next();                            // next_button 被按下时要做的动作
};
```

显然，Simple_window 派生自 Graph_lib 的 Window。我们所有的窗口都必须直接或间接地派生自 Graph_lib::Window，因为正是这个类（通过 FLTK）把我们的窗口概念与系统的窗口实现连接起来。Window 实现的细节见 §E.3。

我们的按钮在 Simple_window 的构造函数中初始化：

```
Simple_window::Simple_window(Point xy, int w, int h, const string& title)
    :Window{xy,w,h,title},
    next_button{Point{x_max()-70,0}, 70, 20, "Next", cb_next},
    button_pushed{false}
{
    attach(next_button);
}
```

不出所料，Simple_window 把自己的位置（xy）、大小（w,h）和标题（title）交给 Graph_lib 的 Window 去处理。接着，构造函数用位置（Point{x_max()-70,0}，大致是右上角）、大小（70,20）、标签（N̈ext）和一个“回调”函数（cb_next）来初始化 next_button。前四个参数与我们为 Window 所做的完全对应：我们在屏幕上放一个矩形形状，并给它加上标签。

最后，我们把 next_button attach() 到自己的 Simple_window 上；也就是说，我们告诉窗口：它必须把按钮显示在自己的位置上，并确保 GUI 系统知道它的存在。

成员 button_pushed 是十分晦涩的实现细节；我们用它记录自上次执行 next() 以来按钮是否被按过。事实上，这里几乎一切都是实现细节，因此都声明为 private。忽略这些实现细节，我们看到

```
struct Simple_window : Graph_lib::Window {
    Simple_window(Point xy, int w, int h, const string& title);
    void wait_for_button();   // 简单的事件循环
    // . . .
};
```

也就是说，用户可以创建一个窗口，并等待它的按钮被按下。

16.3.1 回调函数

函数 cb_next() 是这里新的、有趣的部分。这就是我们希望 GUI 系统在检测到有人点击我们的按钮时调用的函数。既然我们把这个函数交给 GUI、让 GUI“回调我们”，它通常就被称作回调函数（callback function）。我们用前缀 cb_（表示“callback”）来表明 cb_next() 的用途。这只是为了帮助我们阅读——没有任何语言或库要求这种命名约定。显然，我们选 cb_next 这个名字，是因为它要作为“Next”按钮的回调。cb_next 的定义是一段难看的“样板代码”。

在展示那段代码之前，我们先想一想这里发生了什么：

图层的示例

我们的程序运行在好几“层”代码之上。它使用我们的图形库，而我们的图形库是用 FLTK 库实现的，FLTK 又是用操作系统的设施实现的。在一个系统中，层和子层可能还要多。总之，鼠标的设备驱动程序检测到的一次点击，必须以某种方式导致我们的函数 cb_next() 被调用。我们把 cb_next() 的地址和 Simple_window 的地址一层层向下传递；“下面”的某些代码于是在“Next”按钮被按下时调用 cb_next()。

GUI 系统（以及操作系统）要供用各种语言编写的程序使用，所以它不可能把所有用户都限制在漂亮的 C++ 风格里。特别地，它并不知道我们的 Simple_window 类或 Button 类。事实上，它根本不知道类和成员函数。回调函数所需的类型是这样选定的：从最低层的编程（包括 C 和汇编）也能使用它。回调函数不返回任何值，并接受两个地址作为参数。我们可以像下面这样声明一个遵守这些规则的 C++ 成员函数：

```
static void cb_next(Address, Address);   // next_button 的回调
```

关键字 static 在这里是为了确保 cb_next() 能像普通函数那样被调用，也就是说，不是作为针对某个特定对象的 C++ 成员函数被调用。让系统调用一个真正的 C++ 成员函数本来会好得多。然而回调接口必须能被许多语言使用，所以我们得到的只能是这样：一个静态成员函数。Address 参数表明 cb_next() 接受的是“内存中某个东西”的地址。大多数语言并不知道 C++ 的引用，所以我们不能用引用。编译器也不会被告知这些“东西”的类型是什么。这里我们贴近硬件，得不到语言通常提供的那种帮助。“系统”调用回调函数时，第一个参数是触发回调的那个 GUI 实体（Widget）的地址。我们不会用到第一个参数，所以懒得给它起名字。第二个参数是包含该 Widget 的窗口的地址；对 cb_next() 来说，那就是我们的 Simple_window。我们可以这样利用这条信息：

```
void Simple_window::cb_next(Address, Address pw)
    // 对位于 pw 的窗口调用 Simple_window::next()
{
    reference_to<Simple_window>(pw).next();
}
```

reference_to<Simple_window>(pw) 告诉编译器：pw 中的地址应当被看作某个 Simple_window 的地址；也就是说，我们可以把 reference_to<Simple_window>(pw) 当作对某个 Simple_window 的引用使用。第 17、18 章我们会回到内存寻址这个问题。在 §E.1 中我们给出 reference_to 的定义（到那时它已经微不足道了）。眼下我们只是高兴：终于得到了指向自己 Simple_window 的引用，可以按自己喜欢、也习惯了的方式访问数据和函数。最后，我们尽快离开这段依赖系统的代码，去调用自己的成员函数 next()。

我们本可以把想执行的所有代码都写进 cb_next()，但我们——像大多数优秀的 GUI 程序员一样——宁愿把杂乱的低层东西与漂亮的用户代码分开，所以我们用两个函数来处理一个回调：

- cb_next() 只是把系统对回调的约定映射成对普通成员函数的调用（next()）。
- next() 做我们想做的事（不必了解回调那些杂乱的约定）。

这里用两个函数的根本原因，是“一个函数只应完成一个逻辑动作”这条一般原则：cb_next() 把我们带出依赖系统的低层部分，next() 执行我们想要的动作。每当我们希望（“系统”）对某个窗口进行回调时，就定义这样一对函数；例如见 §16.5～7。在继续之前，我们复述一下这里发生的事情：

- 我们定义自己的 Simple_window。
- Simple_window 的构造函数把它的 next_button 注册到 GUI 系统。
- 当我们在屏幕上点击 next_button 的图像时，GUI 调用 cb_next()。
- cb_next() 把低层的系统信息转换成对我们窗口的成员函数 next() 的调用。
- next() 执行我们想作为按钮点击响应而完成的动作。

为了调用一个函数，这确实是一套相当繁复的办法。但请记住，我们处理的是把鼠标（或其他硬件设备）的动作传达给程序的基本机制。特别地：

- 通常有许多程序在同时运行。
- 程序是在操作系统之后很久才写成的。
- 程序是在 GUI 库之后很久才写成的。
- 程序可以用与操作系统不同的语言编写。
- 这套技术要处理各种交互（不只是我们这个小按钮的按下）。
- 一个窗口可以有许多按钮；一个程序可以有许多窗口。

不过，一旦理解了 next() 是如何被调用的，我们基本上也就理解了如何在一个带 GUI 接口的程序中处理每一个动作。

16.3.2 等待循环

那么，在这个最简单的例子里，每次按钮被“按下”时，我们希望 Simple_window 的 next() 做什么呢？基本上，我们需要一个操作，让程序在某一点停下来，使我们有机会看看到目前为止做成了什么；而且，我们还希望 next() 在等待之后让程序重新开始：

```
// 创建一些对象和/或操作一些对象，把它们显示在窗口中
win.wait_for_button();
// next() 使程序从这里继续往下执行
// 创建一些对象和/或操作一些对象
```

其实，这很容易做到。我们先定义 wait_for_button()：

```
void Simple_window::wait_for_button()
    // 修改过的事件循环：
    // 处理所有事件（按默认方式），当 button_pushed 变成 true 时退出
    // 这样就能在没有控制反转的情况下作图
{
    while (!button_pushed) Fl::wait();
    button_pushed = false;
    Fl::redraw();
}
```

像大多数 GUI 系统一样，FLTK 提供一个让程序停下来、直到有事情发生的函数。FLTK 的这个版本叫 wait()。其实 wait() 要做许多事情，因为只要有任何影响我们程序的事情发生，程序就会被唤醒。例如，在 Microsoft Windows 下运行时，窗口被移动、或在被别的窗口遮住之后重新可见时，重画窗口是程序的任务。处理缩放同样是 Window 的任务。Fl::wait() 按默认方式处理所有这些任务。每次 wait() 处理完一件事，它就会返回，好让我们的代码有机会做点什么。

所以，当有人点击我们的“Next”按钮时，wait() 调用 cb_next() 然后返回（回到我们的“等待循环”）。为了让 wait_for_button() 继续往下走，next() 只需把布尔变量 button_pushed 置为 true。这很容易：

```
void Simple_window::next()
{
    button_pushed = true;
}
```

当然，我们还需要在某处定义 button_pushed：

```
bool button_pushed;   // 在构造函数中初始化为 false
```

等待之后，wait_for_button() 需要把 button_pushed 复位，并 redraw() 窗口，以确保我们做的任何改动都能在屏幕上看到。所以它就是这么做的。

16.3.3 用 lambda 表达式作回调

于是，对 Widget 上的每个动作，我们都得定义两个函数：一个把系统对回调的概念映射过来，另一个执行我们想要的动作。考虑：

```
struct Simple_window : Graph_lib::Window {
    Simple_window{Point xy, int w, int h, const string& title};
    void wait_for_button();                 // 简单的事件循环

private:
    Button next_button;                     //  “Next” 按钮
    bool button_pushed;                     // 实现细节

    static void cb_next(Address, Address);  // next_button 的回调
    void next();                            // next_button 被按下时要做的动作
};
```

利用 lambda 表达式（§15.3.3），我们可以省去显式声明映射函数 `cb_next()` 的麻烦。取而代之，我们在 `Simple_window` 的构造函数中定义这个映射：

```
Simple_window::Simple_window(Point xy, int w, int h, const string& title)
    :Window{xy,w,h,title},
    next_button{Point{x_max()-70,0}, 70, 20, "Next",
                [](Address, Address pw) { reference_to<Simple_window>
                (pw).next(); }


},
button_pushed{false}
{
    attach(next_button);
}
```

16.4 Button 及其他 Widget

我们这样定义 `Button`：

```
struct Button : Widget {
    Button(Point xy, int w, int h, const string& label, Callback cb);
    void attach(Window&);
};
```

所以，一个 `Button` 就是一个 `Widget`，它带有位置（`xy`）、尺寸（`w,h`）、文本标签（`label`）和一个回调（`cb`）。基本上，凡是出现在屏幕上、并且关联着某个动作（例如一个回调）的东西，都是一个 `Widget`。

16.4.1 Widget

是的，widget 确实是一个技术术语。对 widget 来说，一个描述性更强、但不那么形象的名称是“控件”（control）。我们用 widget 来定义通过 GUI（图形用户界面）与程序交互的各种形式。我们的 `Widget` 接口类是这样的：

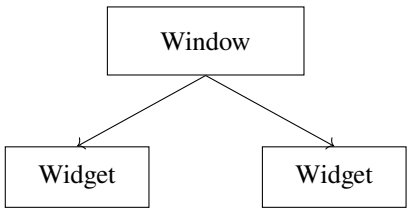
```
class Widget {
    // Widget 是 Fl_widget 的句柄——它 * 不是 * 一个 Fl_widget
    // 我们尽量让接口类与 FLTK 保持距离
public:
    Widget(Point xy, int w, int h, const string& s, Callback cb);
    virtual void move(int dx,int dy);
    virtual void hide();
    virtual void show();
    virtual void attach(Window&) = 0;
    Point loc;
    int width;
    int height;
    string label;
    Callback do_it;
protected:
    Window* own;    // 每个 Widget 都属于某个 Window
    Fl_Widget* pw;  // 与 FLTK 的 Widget 的连接
};
```

Widget 有两个有意思的函数，我们可以把它们用于 Button（也可以用于任何其他从 Widget 派生的类，例如 Menu；见 §16.7）：

- hide() 让 Widget 不可见。
- show() 让 Widget 重新可见。

Widget 一开始是可见的。

与 Shape 一样，我们可以在 Widget 所属的 Window 中 move() 它；而且必须先把它 attach() 到某个 Window 上，才能使用它。注意，我们把 attach() 声明成了纯虚函数（§14.3.5）：每个从 Widget 派生的类都必须定义“被附加到 Window 上”对它意味着什么。事实上，系统级的 widget 正是在 attach() 中创建的。Window 在实现自己的 attach() 时会调用这个 attach() 函数。基本上，把窗口和 widget 连接起来是一支微妙的小双人舞，双方都得做好自己那一份。结果是：窗口知道自己的各个 widget，而每个 widget 也知道自己的窗口：



注意，Window 并不知道自己处理的是哪些种类的 Widget。如 §14.4 所述，我们运用基本的面向对象编程，确保 Window 能够处理每一种 Widget。同样，Widget 也不知道自己处理的是哪种 Window。

我们一直有点马虎，把数据成员留成了可访问的。own 和 pw 这两个成员严格来说只供派生类的实现使用，所以我们把它们声明为 protected。

Widget 以及我们这里用到的各个 widget（Button、Menu 等）的定义都在 GUI.h 中。

16.4.2 Button

Button 是我们要处理的最简单的 Widget。它所做的只是在用户点击它时调用一个回调：

```
class Button : public Widget {
public:
    Button(Point xy, int ww, int hh, const string& s, Callback cb)
        :Widget{xy,ww,hh,s,cb} { }
    void attach(Window& win);
};
```

就这么简单。所有（相对）凌乱的 FLTK 代码都在 attach() 函数里。我们把这个解释放逐到了附录 E（在你读完第 17、18 章之前不要去看）。眼下你只需注意：定义一个简单的 Widget 并不特别困难。

我们不讨论按钮（以及其他 Widget）在屏幕上的外观如何这个有点复杂而凌乱的问题。麻烦在于，选择几乎无穷无尽，而且某些系统还强制规定了某些样式。另外，从编程技术的角度看，表达按钮的外观并不需要什么真正新的东西。如果你实在没辙了，我们要提醒你：把一个 Shape 放在按钮上面并不影响按钮的功能——而你已经知道怎样让一个形状看起来像任何东西。

16.4.3 In_box 与 Out_box

我们提供了两个 Widget，用来把文本输入程序、从程序输出：

```
struct In_box : Widget {
    In_box(Point xy, int w, int h, const string& s)
        :Widget{xy,w,h,s,0} { }
    int get_int();
    string get_string();
    void attach(Window& win);
};


struct Out_box : Widget {
    Out_box(Point xy, int w, int h, const string& s)
        :Widget{xy,w,h,s,0} { }
    void put(int);
    void put(const string&);
    void attach(Window& win);
};
```

In_box 可以接受键入其中的文本；我们可以用 get_string() 把这段文本当作字符串读出，也可以用 get_int() 当作整数读出。如果你想知道是否已有文本输入，可以用 get_string() 读一次，看结果是不是空字符串：

```
string s = some_inbox.get_string();
if (s == "") {
    // 处理缺失的输入
}
```

Out_box 用来向用户呈现某条消息。与 In_box 类似，我们可以 put() 整数或字符串。§16.5 给出了使用 In_box 和 Out_box 的例子。

我们本来可以提供 get_floating_point()、get_complex() 等等，但没有费这个劲，因为你可以取出字符串，把它塞进一个 stringstream，用那种方式做任何你喜欢的输入格式化（§11.4）。

16.4.4 Menu

我们提供一个非常简单的菜单概念：

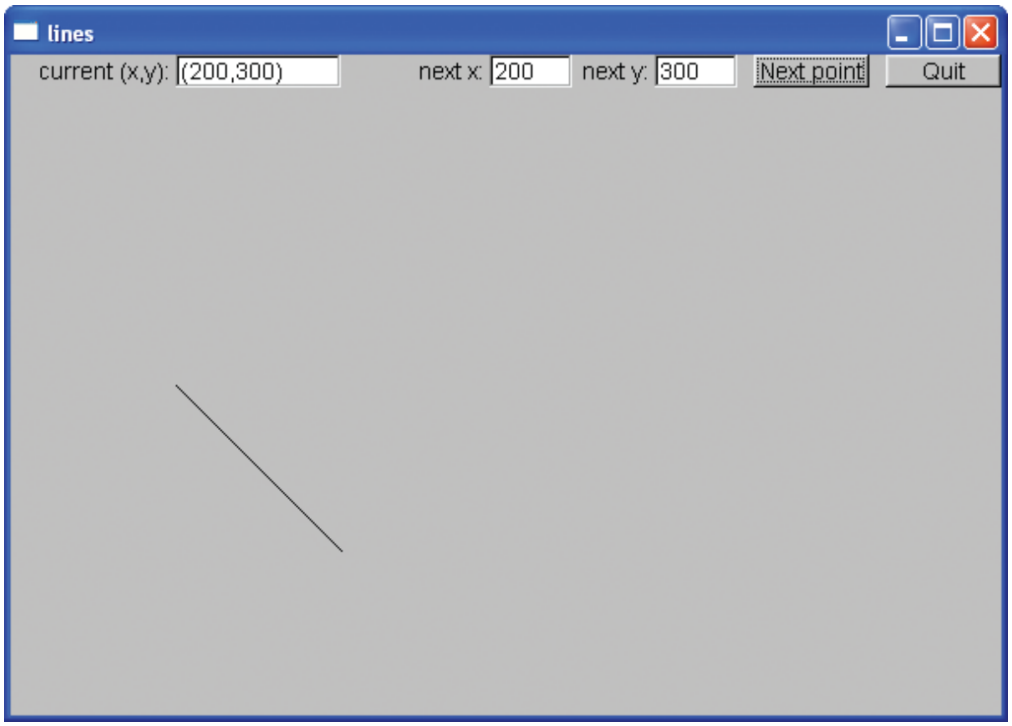
```
struct Menu : Widget {
    enum Kind { horizontal, vertical };
    Menu(Point xy, int w, int h, Kind kk, const string& label);
    Vector_ref<Button> selection;
    Kind k;
    int offset;
    int attach(Button& b);      // 将 Button 附加到 Menu
    int attach(Button* p);      // 将新的 Button 附加到 Menu
    void show()                 // 显示所有按钮
    {
        for (Button& b : selection) b.show();
    }
    void hide();                // 隐藏所有按钮
    void move(int dx, int dy);  // 移动所有按钮
    void attach(Window& win);   // 将所有按钮附加到窗口 win
};
```

Menu 本质上就是一个存放按钮的 vector。像往常一样，xy 这个 Point 是左上角。宽度和高度用来调整按钮加入菜单时的尺寸。例子见 §16.5 和 §16.7。每个菜单按钮（“菜单项”）都是一个独立的 Widget，作为 attach() 的实参交给 Menu。反过来，Menu 也提供一个 attach() 操作，把自己的所有 Button 附加到一个 Window 上。Menu 用一个 Vector_ref 管理它的各个 Button（§13.10、§E.4）。如果你想要“弹出式”菜单，只能自己动手做；见 §16.7。

16.5 一个例子

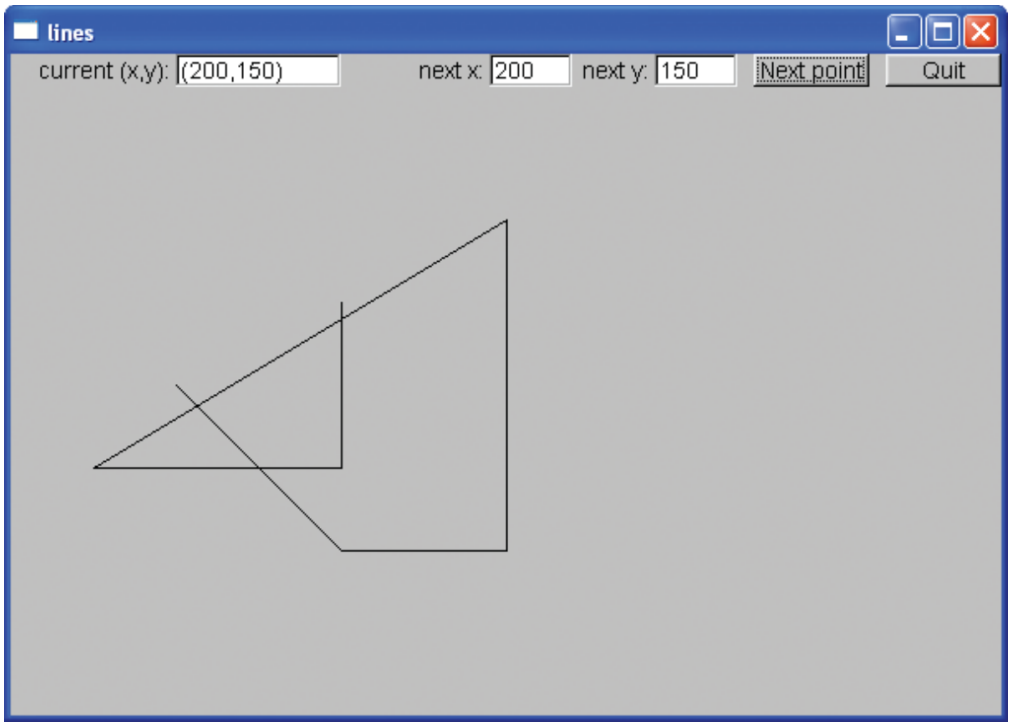
为了更好地体会基本的 GUI 设施，来看看一个简单应用的窗口，它涉及输入、输出和一点图形：

这个程序让用户显示一系列线段（一条开放折线；§13.6），这些线段由一系列坐标对给出。思路是：用户在“next x”和“next y”两个框里反复输入 (x,y) 坐标；每输入一对，就按一下“Next point”按钮。



一开始，“current (x,y)”框是空的，程序等待用户输入第一对坐标。输入之后，起点就出现在“current (x,y)”框里；此后每输入一对新坐标，就会画出一条线：从当前点（其坐标显示在“current (x,y)”框中）到新输入的 (x,y) 画一条线，而这个 (x,y) 就成为新的当前点。

这样画出的是一条开放折线。等用户玩腻了这个活动，还有“Quit”按钮可以退出。这一切都相当直截了当，而这个程序练了好几项有用的 GUI 设施：文本输入与输出、画线，以及多个按钮。上面的窗口显示的是输入两对坐标后的结果；输入七对之后，我们得到的是这样一幅图：



我们来定义一个类表示这样的窗口。这相当直截了当：

```
struct Lines_window : Window {
    Lines_window(Point xy, int w, int h, const string& title);
    Open_polyline lines;
private:
    Button next_button;     // 将 (next_x,next_y) 加入 lines
    Button quit_button;
    In_box next_x;
    In_box next_y;
    Out_box xy_out;
    void next();
    void quit();
};
```

折线用 Open_polyline 表示。按钮和输入框都做了声明（分别声明为 Button、In_box 和 Out_box），并且为每个按钮定义了一个实现所需动作的成员函数。我们决定去掉“样板式”的回调函数，改用 lambda。

Lines_window 的构造函数把一切初始化：

```
Lines_window::Lines_window(Point xy, int w, int h, const string& title)
    :Window{xy,w,h,title},
    next_button{Point{x_max()-150,0}, 70, 20, "Next point",
        [](Address, Address pw) {reference_to<Lines_window>(pw).next();}},
    quit_button{Point{x_max()-70,0}, 70, 20, "Quit",
        [](Address, Address pw) {reference_to<Lines_window>(pw).quit();}},
    next_x{Point{x_max()-310,0}, 50, 20, "next x:"},
    next_y{Point{x_max()-210,0}, 50, 20, "next y:"},
    xy_out{Point{100,0}, 100, 20, "current (x,y):"}
{
    attach(next_button);
    attach(quit_button);
    attach(next_x);
    attach(next_y);
    attach(xy_out);
    attach(lines);
}
```

也就是说，每个 widget 先被构造出来，然后附加到窗口上。

“Quit”按钮会删除窗口。做法是 FLTK 一个奇特的惯用法：干脆把它隐藏起来：

```
void Lines_window::quit()
{
    hide();    // FLTK 删除窗口的奇特惯用法
}
```

真正的活儿都在“Next point”按钮里：它读入一对坐标，更新 Open_polyline，更新位置读数，并重绘窗口：

```
void Lines_window::next()
{
    int x = next_x.get_int();
    int y = next_y.get_int();

    lines.add(Point{x,y});

    // 更新当前位置读数:
    ostringstream ss;
    ss << '(' << x << ',' << y << ')';
    xy_out.put(ss.str());

    redraw();
}
```

这一切都相当显然。我们用 get_int() 从 In_box 中取出整数坐标。我们用一个 ostringstream 来格式化要放进 Out_box 的字符串；str() 成员函数让我们取得 ostringstream 里的那个字符串。这里最后那次 redraw() 是必需的，用来把结果呈现给用户；在调用 Window 的 redraw() 之前，屏幕上一直保留着旧的图像。

那么这个程序有什么古怪、有什么与众不同呢？看看它的 main()：

```
#include "GUI.h"

int main()
try {
    Lines_window win {Point{100,100},600,400,"lines"};
    return gui_main();
}
catch(exception& e) {
    cerr << "exception: " << e.what() << '\n';
    return 1;
}
catch (...) {
    cerr << "Some exception\n";
    return 2;
}
```

那里基本上什么都没有！main() 的函数体只有我们的窗口 win 的定义，以及一次对函数 gui_main() 的调用。此外没有别的函数，没有 if、switch 或循环——完全没有第 6、7 章里那种代码——只有一个变量的定义和一次对 gui_main() 的调用，而 gui_main() 本身又只是调用 FLTK 的 run()。再往下看，我们会发现 run() 就是那个无限循环

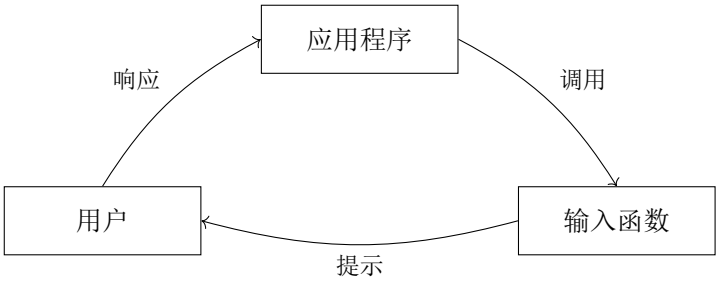
```
while(wait());
```

除了少数被推迟到附录 E 的实现细节，让我们的“lines”程序跑起来的代码我们已经全看过了，所有基本逻辑也都见过了。那么，到底发生了什么？

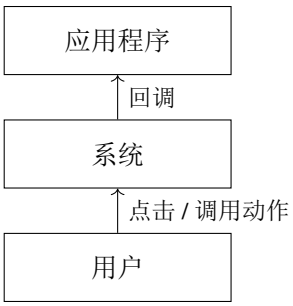
16.6 控制反转

实际发生的事情是：我们把对执行顺序的控制从程序手里移到了 widget 手里——用户激活哪个 widget，哪个就运行。例如，点击一个按钮，它的回调就运行。当这个回调返回时，程序就安定下来，等待用户做别的事。基本上，`wait()` 是告诉“系统”去留意各个 widget，并调用相应的回调。理论上，`wait()` 也可以告诉你这位程序员：是哪个 widget 请求了注意，然后把调用相应函数的事留给你做。然而，在 FLTK 和大多数其他 GUI 系统中，`wait()` 干脆直接调用相应的回调，省得你再去写选择回调的代码。

“常规程序”是这样组织的：



“GUI 程序”是这样组织的：



这种“控制反转”有一个含义：执行顺序完全由用户的操作决定。这使程序的组织与调试都变得更复杂。你很难预想用户会做什么，也很难预想一连串随机的回调可能产生哪些效果。于是系统性测试成了一场噩梦（见第 26 章）。应对这一问题的技术超出了本书的范围，但我们还是建议你：对由用户通过回调驱动的代码要格外小心。除了明显的控制流问题之外，还有可见性的问题，以及“哪个控件连着哪份数据”难以追踪的困难。为了尽量减少麻烦，务必让程序的图形用户界面部分保持简单，并增量式地构建图形用户界面程序，每完成一个阶段就测试一次。做图形用户界面程序时，几乎必须为对象及其交互画一些小图。

各个回调所触发的代码之间怎样通信？最简单的做法是让这些函数直接操作存放在窗口里的数据，§16.5 的例子就是这样。那里由“Next point”按钮调用的 `Lines_window` 的 `next()` 函数，从 `In_box`（`next_x` 和 `next_y`）读取数据，并更新 `lines` 成员变量和 `Out_box`（`xy_out`）。显然，由回调调用的函数可以做任何事：它可以打开文件、连接网络，等等。不过眼下我们只考虑简单情形：把数据保存在某个窗口里。

16.7 添加菜单

我们通过给“lines”程序加一个菜单，来考察“控制反转”带来的控制问题与通信问题。首先，我们只提供一个菜单，让用户可以改变 lines 成员变量中所有线条的颜色。我们加入菜单 color_menu 及其回调：

```
struct Lines_window : Window {
    Lines_window(Point xy, int w, int h, const string& title);
    Open_polyline lines;
    Menu color_menu;
    static void cb_red(Address, Address);       // 红按钮的回调
    static void cb_blue(Address, Address);      // 蓝按钮的回调
    static void cb_black(Address, Address);     // 黑按钮的回调
    // 动作:
    void red_pressed() { change(Color::red); }
    void blue_pressed() { change(Color::blue); }
    void black_pressed() { change(Color::black); }
    void change(Color c) { lines.set_color(c); }
    // ……同前……
};
```

把所有那些几乎一模一样的回调函数和“动作”函数都写出来，是很乏味的。不过这在概念上很简单，而要提供一套打字量明显更少的方案，则超出了本书的范围。如果你愿意，可以用 lambda（§16.3.3）省掉那些 cb_ 函数。菜单按钮被按下时，就把线条改成所要的颜色。

定义了 color_menu 成员之后，我们需要初始化它：

```
Lines_window::Lines_window(Point xy, int w, int h, const string& title)
    :Window(xy,w,h,title),
    // ……同前……
    color_menu{Point{x_max()-70,40},70,20,Menu::vertical,"color"}
{
    // ……同前……
    color_menu.attach(new Button{Point{0,0},0,0,"red",cb_red});
    color_menu.attach(new Button{Point{0,0},0,0,"blue",cb_blue});
    color_menu.attach(new Button{Point{0,0},0,0,"black",cb_black});
    attach(color_menu);
}
```

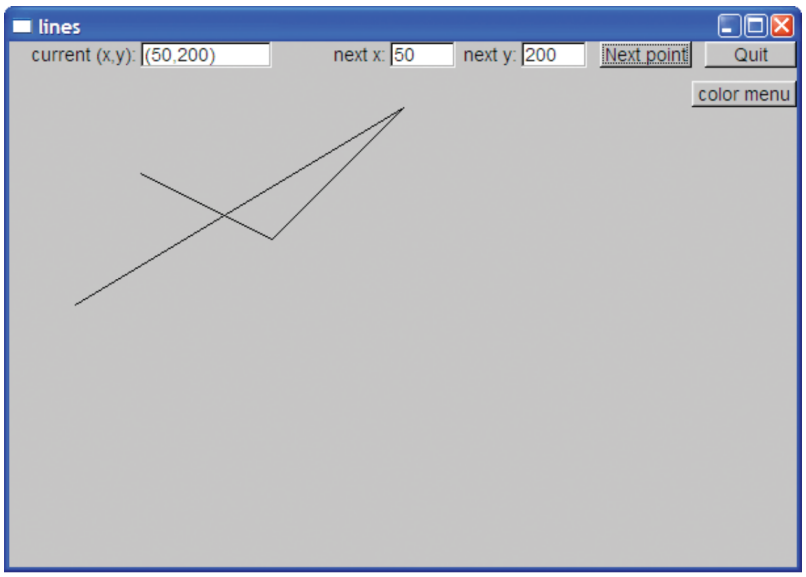
这些按钮是用 attach() 动态附加到菜单上的，需要时可以移除和/或替换。Menu::attach() 会调整按钮的大小和位置，并把它附加到窗口上。就这些，于是我们得到：

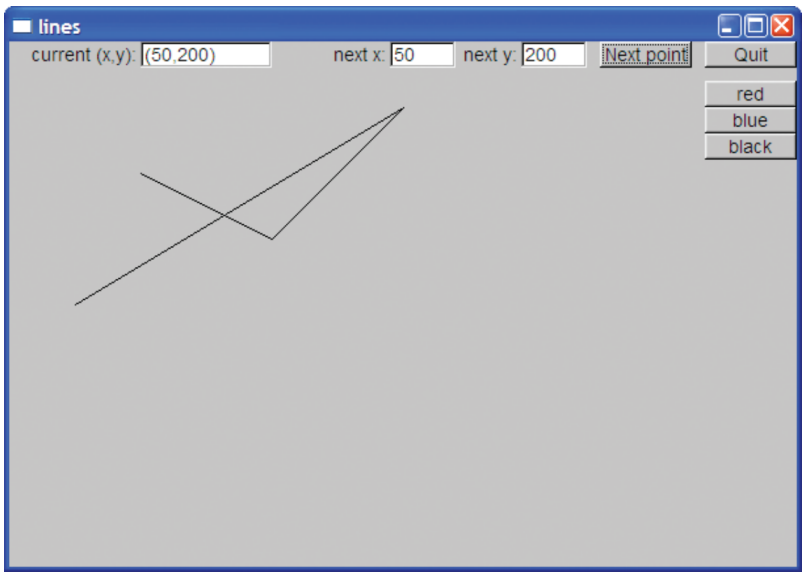
这样玩了一阵之后，我们认定真正想要的是一个“弹出式菜单”；也就是说，除了正在使用它的时候，我们不想把宝贵的屏幕空间花在菜单上。于是我们加了一个“color menu”按钮。按下它，颜色菜单就弹出来；一旦做出选择，菜单又重新隐藏，按钮重新出现。

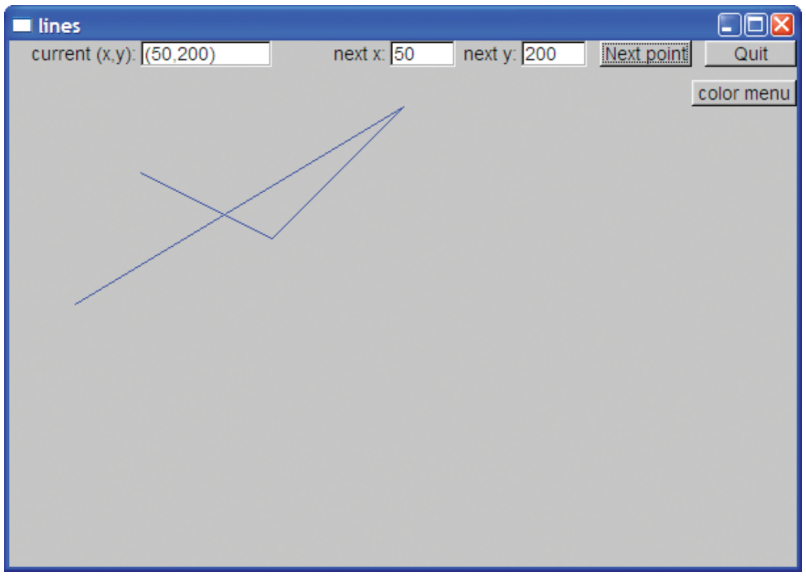
下面首先是加了几条线之后的窗口：

我们看到新的“color menu”按钮和一些（黑色）线条。按下“color menu”，菜单就出现了：

lines
current (x,y): no point
next x:
next y:
Next point
Quit
red
blue
black


lines
current (x,y): (50,200)
next x: 50
next y: 200
Next point
Quit
color menu


lines
current (x,y): (50,200)
next x: 50
next y: 200
Next point
Quit
red
blue
black



注意，“color menu”按钮此时隐藏了。在菜单用完之前我们不需要它。按下“blue”，我们就得到：线条现在变成了蓝色，“color menu”按钮又重新出现了。

为了做到这一点，我们添加了“color menu”按钮，并修改了各个“pressed”函数，让它们调整菜单和按钮的可见性。下面是全部修改完成后的完整 `Lines_window`：

```
struct Lines_window : Window {
    Lines_window(Point xy, int w, int h, const string& title);
private:
    // 数据:
    Open_polyline lines;
    // 控件:
    Button next_button;         // 把 (next_x,next_y) 加到 lines
    Button quit_button;         // 结束程序
    In_box next_x;
    In_box next_y;
    Out_box xy_out;
    Menu color_menu;
    Button menu_button;
    void change(Color c) { lines.set_color(c); }
    void hide_menu() { color_menu.hide(); menu_button.show(); }
    // 由回调触发的动作:
    void red_pressed() { change(Color::red); hide_menu(); }
    void blue_pressed() { change(Color::blue); hide_menu(); }
    void black_pressed() { change(Color::black); hide_menu(); }
    void menu_pressed() { menu_button.hide(); color_menu.show(); }
    void next();
    void quit();
    // 回调函数:
    static void cb_red(Address, Address);
    static void cb_blue(Address, Address);
    static void cb_black(Address, Address);
```

```
    static void cb_menu(Address, Address);
    static void cb_next(Address, Address);
    static void cb_quit(Address, Address);
};
```

注意，除构造函数之外所有成员都是私有的。基本上，这个 Window 类就是整个程序。所有事情都通过它的回调发生，因此不需要来自窗口外部的代码。我们把这些声明稍微整理了一下，希望让这个类更易读。构造函数为其所有子对象提供参数，并把它们附加到窗口上：

```
Lines_window::Lines_window(Point xy, int w, int h, const string& title)
    :Window{xy,w,h,title},
    next_button{Point{x_max()-150,0}, 70, 20, "Next point", cb_next},
    quit_button{Point{x_max()-70,0}, 70, 20, "Quit", cb_quit},
    next_x{Point{x_max()-310,0}, 50, 20, "next x:"},
    next_y{Point{x_max()-210,0}, 50, 20, "next y:"},
    xy_out{Point{100,0}, 100, 20, "current (x,y):"},
    color_menu{Point{x_max()-70,30},70,20,Menu::vertical,"color"},
    menu_button{Point{x_max()-80,30}, 80, 20, "color menu", cb_menu}
{
    attach(next_button);
    attach(quit_button);
    attach(next_x);
    attach(next_y);
    attach(xy_out);
    xy_out.put("no point");
    color_menu.attach(new Button{Point{0,0},0,0,"red",cb_red});
    color_menu.attach(new Button{Point{0,0},0,0,"blue",cb_blue});
    color_menu.attach(new Button{Point{0,0},0,0,"black",cb_black});
    attach(color_menu);
    color_menu.hide();
    attach(menu_button);
    attach(lines);
}
```

注意，初始化器的顺序与数据成员的声明顺序一致。这才是书写初始化器的正确顺序。事实上，成员初始化器总是按数据成员被声明的顺序执行。如果基类或成员的构造函数被乱序指定，有些编译器（很帮忙地）会给出警告。

16.8 调试图形用户界面代码

图形用户界面程序一旦开始工作，调试起来往往相当容易：所见即所得。然而，在第一批图形和控件开始出现在窗口里之前，甚至在窗口出现在屏幕上之前，常常有一段最令人沮丧的时期。试试这个 main()：

```
int main()
{
    Lines_window {Point{100,100},600,400,"lines"};
    return gui_main();
}
```

你看出错在哪里了吗？不管看没看出来，都应该试一试：这个程序能编译、能运行，但 Lines_window 并不会给你画线的机会，你最多只会在屏幕上看到一闪。怎样找出这类程序中的错误呢？

- 谨慎地使用久经考验的程序部件（类、函数、库）
- 简化所有新代码，从最简单的版本开始慢慢“生长”出程序，并逐行仔细检查代码
- 检查所有链接器设置
- 把代码与已经能工作的程序作比较
- 把代码讲给朋友听

你会发现，最难做的一件事是跟踪代码的执行。如果你学会了使用调试器，还有机会；但在这种情形里，仅仅插入“输出语句”不会有用——问题恰恰在于没有任何输出出现。即便是调试器也会遇到麻烦，因为同时有若干件事情在进行（“多线程”）——你的代码并不是唯一想与屏幕交互的代码。关键在于简化代码，并用系统化的方法去理解代码。

那么问题出在哪里？下面是正确的版本（来自 §16.5）：

```
int main()
{
    Lines_window win{Point{100,100},600,400,"lines"};
    return gui_main();
}
```

我们“忘了”给 Lines_window 起名字：win。既然实际上并不需要这个名字，那看起来也合理；但编译器随后认定，既然我们没有使用这个窗口，就可以立刻销毁它。哎呀！那个窗口存在了大约一毫秒。难怪我们没看见它。

另一个常见问题，是把一个窗口正好放在另一个窗口上面。这显然（或者说根本不显然）看起来就像只有一个窗口。另一个窗口到哪儿去了？我们可能花大量时间在代码里寻找并不存在的错误。把一个图形放在另一个图形上面，也会出现同样的问题。

最后——更糟的是——在使用图形用户界面库时，异常并不总能像我们希望的那样工作。既然我们的代码是由图形用户界面库管理的，我们抛出的异常可能永远到不了自己的处理程序——库或操作系统可能把它“吃掉”了（也就是说，它们可能依赖与 C++ 异常不同的错误处理机制，甚至可能完全不知道 C++ 的存在）。

调试中发现的常见问题包括：图形（Shape）和控件（Widget）因为没有附加而显示不出来；对象因为已经离开作用域而行为异常。想一想程序员可能怎样把菜单中按钮的创建与附加提取成一个函数：

```
// helper function for loading buttons into a menu
void load_disaster_menu(Menu& m)
{
    Point orig {0,0};
    Button b1 {orig,0,0,"flood",cb_flood};
    Button b2 {orig,0,0,"fire",cb_fire};
    // ……
    m.attach(b1);
    m.attach(b2);
    // ……
}


int main()
{
    // ……
    Menu disasters {Point{100,100},60,20,Menu::horizontal,"disasters"};
    load_disaster_menu(disasters);
    win.attach(disasters);
    // ……
}
```

这样做不行。那些按钮都是 `load_disaster_menu` 函数的局部对象，把它们附加到菜单上并不会改变这一点。解释见 §18.6.4（不要返回指向局部变量的指针），局部变量内存布局的图示见 §8.5.8。事情的实质是：`load_disaster_menu()` 返回之后，那些局部对象已经被销毁，而 `disasters` 菜单引用的是不存在的（已销毁的）对象。结果很可能出人意料，而且不好看。解决办法是使用由 `new` 创建的无名对象，而不要使用有名字的局部对象：

```
// helper function for loading buttons into a menu
void load_disaster_menu(Menu& m)
{
    Point orig {0,0};
    m.attach(new Button{orig,0,0,"flood",cb_flood});
    m.attach(new Button{orig,0,0,"fire",cb_fire});
    // ……
}
```

正确的解决办法甚至比这个（太常见的）错误还简单。

小试身手

1. 新建一个完整的项目，并按附录 D 所述设置 FLTK 的链接器设置。
2. 使用 `Graph_lib` 的设施，键入 §16.5 中的画线程序，并让它运行起来。
3. 按 §16.7 所述修改该程序，改用弹出式菜单，并让它运行起来。
4. 再修改该程序，为选择线条样式增加第二个菜单，并让它运行起来。

要点回顾

1. 你为什么会想要图形用户界面？

2. 什么时候你会想要非图形用户界面？
3. 什么是软件层？
4. 你为什么要把软件分层？
5. 在 C++ 中与操作系统通信时的根本问题是什么？
6. 什么是回调？
7. 什么是控件？
8. 控件还有个什么别的叫法？
9. 缩写 FLTK 是什么意思？
10. FLTK 怎么读？
11. 你还听说过哪些图形用户界面工具包？
12. 哪些系统用“控件（widget）”这个词，哪些更愿意用“控制（control）”？
13. 控件的例子有哪些？
14. 什么时候你会用输入框？
15. 输入框中存放的值的类型是什么？
16. 什么时候你会用按钮？
17. 什么时候你会用菜单？
18. 什么是控制反转？
19. 调试图形用户界面程序的基本策略是什么？
20. 为什么调试图形用户界面程序比调试“用流做输入输出的普通程序”更难？

术语

- button（按钮）
- dialog box（对话框）
- visible/hidden（可见/隐藏）
- callback（回调）
- GUI（图形用户界面）
- waiting for input（等待输入）
- console I/O（控制台输入输出）
- menu（菜单）
- wait loop（等待循环）
- control（控制）
- software layer（软件层）
- widget（控件）
- control inversion（控制反转）
- user interface（用户界面）

习题

1. 做一个 `My_window`，它有点像 `Simple_window`，只是多了两个按钮：next 和 quit。
2. 做一个窗口（基于 `My_window`），里面是一个 4×4 的方块按钮棋盘。按下某个按钮时，它执行一个简单的动作，比如在输出框中打印自己的坐标，或者变成稍微不同的颜色（直到另一个按钮被按下）。

3. 把一幅 Image 放在一个 Button 上面；按钮被按下时两者一起移动。用 std_lib_facilities.h 里的这个随机数生成器来为“图像按钮”挑选新位置：

```
#include<random>

inline int rand_int(int min, int max)
{
    static default_random_engine ran;
    return uniform_int_distribution<>{min,max}(ran);
}
```

它返回 $[min, max)$ 范围内的一个随机整数。

4. 做一个菜单，其菜单项分别画一个圆、一个正方形、一个等边三角形和一个六边形。再做一个（或两个）输入框，用来给出坐标对，并把按下菜单项所生成的图形放在该坐标处。抱歉，不支持拖放。
5. 写一个程序，画出你选定的一个图形，并在每次点击“Next”时把它移到一个新位置。新位置应当由从输入流读入的坐标对决定。
6. 做一个“模拟时钟”，也就是带指针走动的钟。通过一次库调用从操作系统取得一天中的时间。这个练习的主要部分是：找到能给你一天中时间的函数，以及一种等待一小段时间（例如时钟走一格所需的一秒）的方法，并学会根据你找到的文档使用它们。提示：clock()、sleep()。
7. 运用前面几个练习中发展出的技术，让一幅飞机的图像在窗口里“飞来飞去”。要有“Start”和“Stop”按钮。
8. 提供一个货币换算器。启动时从文件读取换算率。在输入窗口中输入金额，并提供选择源货币和目标货币的方式（例如一对菜单）。
9. 修改第 7 章的计算器，让它从输入框获取输入，并把结果显示在输出框中。
10. 提供一个程序，让你可以在一组函数（例如 sin() 和 log()）中选择，为这些函数提供参数，然后把它们画成图。

后记

图形用户界面是一个庞大的主题。其中很大一部分涉及风格，以及与既有系统的兼容性。此外，很大一部分还涉及令人眼花缭乱的各种控件（比如某个图形用户界面库提供几十种不同的按钮样式），这会让一位传统植物学家感到十分亲切。然而，这些大多与基本的编程技术无关，所以我们不朝那个方向继续。其他主题，比如缩放、旋转、变形、三维对象、阴影等等，需要图形学和/或数学方面的较高素养，而本书并不假定你具备这些素养。

有一件事你应该知道：大多数图形用户界面系统都提供一个“图形用户界面构建器”，让你以图形方式设计窗口布局，并以图形方式为按钮、菜单等指定回调和动作。对许多应用来说，这种构建器很值得一用，它可以减少编写“脚手架代码”（比如我们那些回调）的乏味。不过，始终要努力理解生成出来的程序是怎样工作的。有时生成的代码与你在本章看到的是等价的；有时则会用到更复杂和/或更昂贵的机制。

20 容器与迭代器

“Write programs that do one thing
and do it well. Write programs
to work together.”
（“写程序，只做一件事，并把它做好。写程序，让它们协同工作。”）
——Doug McIlroy

par

本章与下一章介绍 STL，即 C++ 标准库中“容器与算法”的那一部分。STL 是一个可扩展的框架，用来处理 C++ 程序中的数据。在给出第一个简单示例之后，我们介绍它的一般理想与基本概念。我们讨论迭代（iteration）、链表操作与 STL 容器。序列（sequence）与迭代器（iterator）这两个关键概念，把容器（数据）与算法（处理）联系在一起。本章为下一章介绍的那些通用、高效而实用的算法打下基础。作为示例，本章还给出一个文本编辑的框架，作为示例应用。

20.1 存储与处理数据
- **20.1.1** 处理数据
- **20.1.2** 泛化代码

20.2 STL 理念
20.3 序列与迭代器
- **20.3.1** 回到示例

20.4 链表
- **20.4.1** 链表操作
- **20.4.2** 迭代

20.5 再次泛化 vector
- **20.5.1** 容器遍历
- **20.5.2** auto

20.6 示例：简单的文本编辑器
- **20.6.1** 行
- **20.6.2** 迭代

20.7 vector、list 与 string
- **20.7.1** insert 与 erase

20.8 让我们的 vector 适配 STL
20.9 让内置数组适配 STL
20.10 容器概览
- **20.10.1** 迭代器类别

20.1 存储与处理数据

在深入处理更大的数据集合之前，我们先看一个简单示例，它指出了应对一大类数据处理问题的途径。杰克（Jack）和吉尔（Jill）各自在测量车速，并把结果记录为浮点值。杰克是按 C 程序员的习惯培养起来的，他把值存放在数组里；吉尔则把值存放在 vector 里。现在我们想在自己的程序中使用他们的数据。该怎么做呢？

我们可以让杰克和吉尔的程序把值写进文件，再由我们的程序把它们读回来。这样，我们就完全不必理会他们对数据结构和接口的选择。这种隔离往往是个好主意；如果决定这么做，我们便可以使用第 10～11 章介绍的技术来做输入，并用一个 `vector<double>` 来做计算。

可是，如果对我们想做的事来说，用文件并不是个好选择呢？假定数据采集代码被设计成函数调用的形式，每秒交付一组新数据。我们每秒调用一次杰克和吉尔的函数，让它们把数据交给我们处理：

```
double* get_from_jack(int* count);     // 杰克把 double 存入数组，并
                                       // 在 *count 中返回元素个数
vector<double>* get_from_jill();       // 吉尔填充 vector

void fct()
{
    int jack_count = 0;
    double* jack_data = get_from_jack(&jack_count);
    vector<double>* jill_data = get_from_jill();
    // . . . 处理 . . .
    delete[] jack_data;
    delete jill_data;
}
```

这里的假定是：数据存放在自由存储（free store）上，用完之后我们应当把它 delete 掉。另一个假定是：我们无法重写杰克和吉尔的代码，或者不愿这么做。

20.1.1 处理数据

这个示例显然经过了简化，但它与大量实际问题并无二致。如果我们能优雅地处理这个示例，就能处理极多常见的编程问题。这里的根本问题在于：我们无法控制“数据提供者”存放数据的方式。我们的任务是：要么按拿到手的形式直接使用数据，要么把它读进来、按我们更喜欢的方式存储。

我们想拿这些数据做什么？排序？找出最大值？求平均值？找出所有大于 65 的值？把吉尔的数据与杰克的比较？看看一共有多少个读数？可能性无穷无尽；真正写程序时，我们只需做所需的那项计算。这里我们只想做点事情，借此学会如何处理数据、如何做涉及大量数据的计算。先做一件真正简单的事：在每个数据集中找出值最大的元素。我们可以在 `fct()` 中把下面的代码插在“. . . 处理. . .”注释的位置：

```
// . . .
double h = -1;
double* jack_high;    // jack_high 将指向值最大的元素
double* jill_high;    // jill_high 将指向值最大的元素
for (int i=0; i<jack_count; ++i)
    if (h<jack_data[i]) {
        jack_high = &jack_data[i];    // 保存最大元素的地址
        h = jack_data[i];             // 更新“最大元素”
    }
h = -1;
for (int i=0; i<jill_data->size(); ++i)
    if (h<(*jill_data)[i]) {
        jill_high = &(*jill_data)[i];    // 保存最大元素的地址
        h = (*jill_data)[i];             // 更新“最大元素”
    }
cout << "Jill's max: " << *jill_high
     << "; Jack's max: " << *jack_high;
// . . .
```

注意我们访问吉尔的数据时所用的难看的记法：`(*jill_data)[i]`。函数 `get_from_jill()` 返回的是指向 vector 的指针，即 `vector<double>*`。要取到数据，必须先对指针解引用、得到那个 vector，即 `*jill_data`，然后才能对它做下标运算。不过，`*jill_data[i]` 并不是我们要的；它的含义是 `*(jill_data[i])`，因为 `[ ]` 的结合力比 `*` 更强。所以我们必须在 `*jill_data` 外面加圆括号，写成 `(*jill_data)[i]`。

试一试

如果你能修改吉尔的代码，你会怎样重新设计它的接口，以消除这种难看之处？

20.1.2 泛化代码

我们想要的是一种统一的访问和操作数据的方式，这样就不必每次遇到数据以略有不同的方式呈现时都另写一套代码。让我们把杰克和吉尔的代码当作例子，看看怎样才能使自己的代码更抽象、更统一。

显然，我们对杰克的数据所做的处理与对吉尔的数据所做的非常相似。不过，还有一些恼人的差异：`jack_count` 与 `jill_data->size()`，以及 `jack_data[i]` 与 `(*jill_data)[i]`。引入一个引用，就可以消除后一处差异：

```
vector<double>& v = *jill_data;
for (int i=0; i<v.size(); ++i)
    if (h<v[i]) {
        jill_high = &v[i];
        h = v[i];
    }
```

这与处理杰克数据的代码已经近得让人心痒。要写出一个既能为吉尔的数据、也能为杰克的数据做这项计算的函数，需要什么呢？我们可以想出好几种办法（见习题 3），但出于一般性的考虑——这种考虑在接下来两章中会变得清楚——我们选择了一种基于指针的方案：

```
double* high(double* first, double* last)
    // 返回指向 [first,last) 中值最大元素的指针
{
    double h = -1;
    double* high;
    for(double* p = first; p!=last; ++p)
        if (h<*p) { high = p; h = *p; }
    return high;
}
```

有了它，我们可以这样写：

```
double* jack_high = high(jack_data,jack_data+jack_count);
vector<double>& v = *jill_data;
double* jill_high = high(&v[0],&v[0]+v.size());
```

这看起来好多了。我们不必引入那么多变量，循环和循环体也只写一次（在 `high()` 中）。如果想知道最大值，看一下 `*jack_high` 和 `*jill_high` 即可。例如：

```
cout << "Jill's max: " << *jill_high
     << "; Jack's max: " << *jack_high;
```

注意，`high()` 依赖于 vector 把元素存放在数组中，因此我们才能用指向数组的指针来表达“找出值最大的元素”这个算法。

试一试

我们在这个小程序里留下了两个可能很严重的错误。其中一个可能导致崩溃；另一个则会在 `high()` 被用于许多本来可能用得上的其他程序时给出错误答案。下面介绍的一般技术会让这两个错误变得显而易见，并说明如何系统地避免它们。眼下，请先找出这两个错误，并提出补救办法。

这个 `high()` 函数的局限在于：它只是针对某个特定问题的解：

- 它只适用于数组。我们依赖于 vector 把元素存放在数组中，而存储数据的方式还有很多，例如 list 和 map（见 §20.4 和 §21.6.1）。
- 它可以用于 double 的 vector 和数组，却不能用于元素类型为其他的数组或 vector，例如 `vector<double*>` 和 `char[10]`。

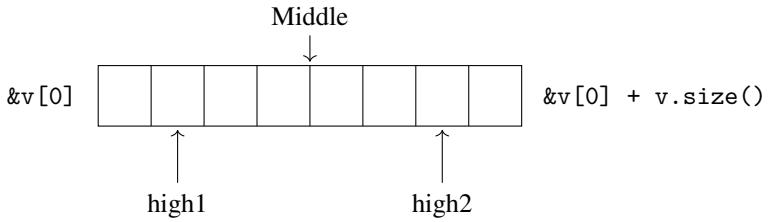
- 它找出值最大的元素，而人们想在这类数据上做的简单计算还有很多。

下面我们来探讨，怎样以高得多的通用性，在数据集合上支持这类计算。

请注意，当我们决定用指针来表达“找出值最大的元素”这个算法时，我们“顺便”把它泛化了，使它做的比我们要求的更多：我们既能（如我们所愿）找出数组或 vector 中值最大的元素，也能找出数组或 vector 的某一部分中值最大的元素。例如：

```
// . . .
vector<double>& v = *jill_data;
double* middle = &v[0]+v.size()/2;
double* high1 = high(&v[0], middle);            // 前半部分的最大值
double* high2 = high(middle, &v[0]+v.size());   // 后半部分的最大值
// . . .
```

这里 high1 将指向 vector 前半部分中值最大的元素，high2 将指向后半部分中值最大的元素。用图形表示，大致是这样：



我们给 high() 用的是指针参数。这有点低层，也容易出错。我们猜想，对许多程序员来说，找出 vector 中值最大元素的那个“显然”的函数会写成这样：

```
double* find_highest(vector<double>& v)
{
    double h = -1;
    double* high = 0;
    for (int i=0; i<v.size(); ++i)
        if (h<v[i]) { high = &v[i]; h = v[i]; }
    return high;
}
```

然而，这样写就得不到我们从 high() 那里“顺便”获得的灵活性——我们无法用 find_highest() 找出 vector 某一部分中值最大的元素。实际上，正是靠“摆弄指针”写出一个既能用于数组、也能用于 vector 的函数，我们才获得了实实在在的好处。我们会记住这一点：泛化可以带来能用于更多问题的函数。

20.2 STL 理念

C++ 标准库提供了一个把数据当作元素序列来处理的框架，称为 STL。STL 通常被认为是“standard template library”（标准模板库）的缩写。STL 是 ISO C++ 标准库中提供容器（如 vector、list 和 map）与泛型算法（如 sort、find 和 accumulate）的那一部分。因此，我们可以——而且确实——把 vector 这类设施同时算作“STL”和“标准库”的组成部分。另一些标准库特性，例如 ostream（第 10 章）和 C

风格字符串函数（§B.11.3），则不属于 STL。为了更好地领会和理解 STL，我们先考虑处理数据时必须面对的问题，以及我们对解决方案所抱的理想。

计算有两个主要方面：计算本身与数据。有时我们关注计算，谈论 if 语句、循环、函数、错误处理等；另一些时候我们关注数据，谈论数组、vector、string、文件等。然而，要把有用的活干完，两者都需要。大量数据若没有分析、可视化，并搜寻出“有趣的部分”，便是无法理解的。反过来，我们想计算多少都行，但除非有数据把我们的计算与某种真实的东西联系起来，否则计算将是枯燥而贫瘠的。此外，程序的“计算部分”必须与“数据部分”优雅地交互。

以这种方式谈论数据时，我们想的是大量数据：几十个 Shape、几百个温度读数、几千条日志记录、几百万个点、几十亿个网页，等等；也就是说，我们谈的是处理数据的容器、数据的流等。特别地，这里讨论的不是如何为一个小的对象（例如一个复数、一个温度读数或一个圆）最好地挑选几个值来表示它。关于这类类型，见第 9、11、14 章。

下面举几个简单例子，都是我们希望拿“大量数据”做的事：

- 按字典序排列单词。
- 在电话簿中根据姓名查找一个号码。
- 找出最高的温度。
- 找出所有大于 8800 的值。
- 找出值 17 第一次出现的位置。
- 按单元号对遥测记录排序。
- 按时间戳对遥测记录排序。
- 找出第一个大于“Petersen”的值。
- 找出最大的金额。
- 找出两个序列之间的第一处差异。
- 计算两个序列中元素的成对乘积。
- 找出一个月里每天的最高温度。
- 找出销售记录中排名前十的畅销品。
- 统计“Stroustrup”在 Web 上出现的次数。
- 计算所有元素之和。

注意，描述这些任务时，我们其实并没有提到数据是如何存储的。显然，要让这些任务有意义，我们必须处理 list、vector、文件、输入流之类的东西；但要谈论拿数据做什么，并不需要知道数据如何存储（或如何收集）的细节。重要的是值或对象的类型（元素类型）、我们访问这些值或对象的方式，以及我们想拿它们做什么。

这类任务非常常见。我们自然想简单而高效地写出完成这些任务的代码。反过来说，作为程序员，我们面临的问题是：

- 数据类型（“数据的种类”）有无穷多的变化。
- 存储数据元素集合的方式多得让人眼花缭乱。
- 我们想对数据集合做的任务种类繁多。

为了尽量减小这些问题的影响，我们希望自己的代码能利用各种类型之间、各种数据存储方式之间以及各种处理任务之间的共性。换句话说，我们想把自己的代码泛化，以应付这类变化。我们实在不想为每个问题从头手工打造解决方案；那将是乏味的时间浪费。

为了弄清我们希望写代码时得到什么样的支持，先来看看我们拿数据所做的事的一个更抽象的视角：

- 把数据收集到容器中
  - 例如 vector、list 和 array
- 组织数据
  - 为了打印
  - 为了快速访问
- 取回数据项
  - 按索引（例如第 42 个元素）
  - 按值（例如“age 字段”为 7 的第一条记录）
  - 按属性（例如“temperature 字段”大于 32 且小于 100 的所有记录）
- 修改容器
  - 添加数据
  - 删除数据
  - 排序（按某种准则）
- 执行简单的数值运算（例如把所有元素乘以 1.7）

我们希望做这些事时，不至于陷进关于容器之间的差异、访问元素方式的差异、元素类型之间的差异的细节泥潭。如果能做到这一点，我们离“简单而高效地使用大量数据”这个目标就前进了一大步。

回顾前几章介绍的编程工具与技术，可以看到我们（已经）能写出与所用数据类型无关的相似程序：

- 使用 int 与使用 double 并没有多大差别。
- 使用 `vector<int>` 与使用 `vector<string>` 并没有多大差别。
- 使用 double 的数组与使用 `vector<double>` 并没有多大差别。

我们希望把自己的代码组织成：只有当我们想做真正新的、不同的事情时，才需要写新代码。特别地，我们想为常见的编程任务提供代码，这样就不必每次找到一种新的数据存储方式、或找到一种略有不同的数据解释方式时，都重写一遍自己的解决方案。

- 在 vector 中查找一个值与在数组中查找一个值并没有多大差别。
- 忽略大小写查找一个 string，与区分大小写查找一个 string 并没有多大差别。
- 用精确值画实验数据图与用四舍五入后的值画数据图并没有多大差别。
- 复制一个文件与复制一个 vector 并没有多大差别。

我们想在这些观察的基础上继续前进，写出这样的代码：

- 易于阅读
- 易于修改
- 规整（regular）
- 简短（short）
- 快速（fast）

为了尽量减少编程工作量，我们希望做到：

- 对数据的统一访问
  - 与数据的存储方式无关
  - 与数据的类型无关
- 类型安全的数据访问
- 便于遍历数据
- 紧凑的数据存储
- 快速
  - 检索数据
  - 添加数据
  - 删除数据
- 最常见的一些算法有标准版本
  - 例如 `copy`、`find`、`search`、`sort`、`sum` 等

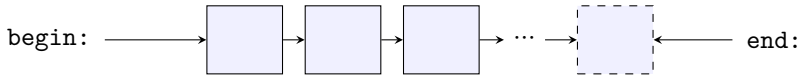
STL 提供的正是这些，而且远不止这些。我们不仅把它看成一套非常有用的设施，还要把它看成一个为最大灵活性与性能而设计的库的范例。STL 由 Alex Stepanov 设计，目的是为作用于数据结构的通用、正确而高效的算法提供一个框架。它的理想是数学那样的简洁、通用与优雅。

如果不采用这种具有明确表述的理想与原则的框架来处理数据，替代做法就是每个程序员凭当时觉得不错的主意，拿基本语言设施把每个程序都拼凑出来。那意味着大量额外工作。而且结果往往是一团没有原则的乱麻：很少能得到一个容易被原设计者以外的人理解的程序，而结果代码能用于其他场合，也只能靠碰运气。

既已考察了动机与理想，下面来看 STL 的基本定义，然后终于进入例子——这些例子将告诉我们如何逼近那些理想：为处理数据写出更好的代码，而且写得更轻松。

20.3 序列与迭代器

STL 的核心概念是序列（sequence）。从 STL 的角度看，一组数据就是一个序列。序列有起点，也有终点。我们可以从起点遍历到终点，并可选地读取或写入每个元素的值。我们用一对迭代器来标识序列的起点与终点。迭代器（iterator）是一个标识序列中某个元素的对象。我们可以把一个序列想象成这样：



这里，`begin` 和 `end` 都是迭代器，它们标识序列的起点与终点。STL 序列就是通常所说的“半开”（half-open）序列：由 `begin` 标识的元素属于该序列，而 `end` 迭代器指向序列末尾之后的那一个位置。这类序列（范围）的常用数学记法是 `[begin:end)`。从一个元素指向下一个元素的箭头表示：只要有了指向某个元素的迭代器，就能得到指向下一个元素的迭代器。

迭代器究竟是什么？迭代器是一个相当抽象的概念：

- 迭代器指向（引用）序列中的某个元素，或指向最后一个元素之后的那一个位置。
- 可以用 `==` 和 `!=` 比较两个迭代器。
- 可以用一元 `*` 运算符（“解引用”，即取“所指的内容”）取得迭代器所指元素的值。
- 可以用 `++` 得到指向下一个元素的迭代器。

例如，设 p 和 q 是指向同一序列中元素的迭代器：

标准迭代器的基本操作

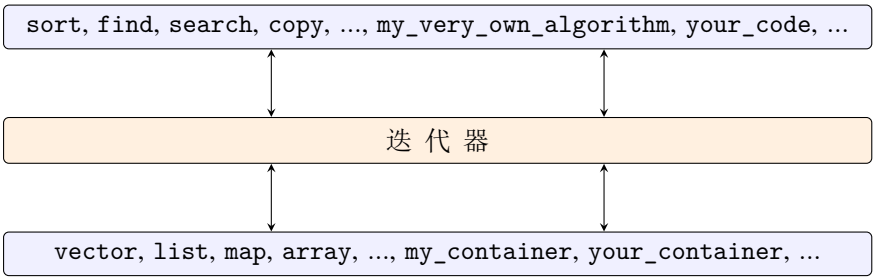
`p==q`	当且仅当 p 与 q 指向同一元素，或二者都指向最后一个元素之后的那一个位置时为真
`p!=q`	`!(p==q)`
`*p`	指向 p 所指的元素
`*p=val`	向 p 所指的元素写入
`val=*p`	从 p 所指的元素读取
`++p`	使 p 指向序列中的下一个元素，或指向最后一个元素之后的那一个位置

显然，迭代器的概念与指针的概念有关（§17.4）。事实上，指向数组元素的指针就是一种迭代器。不过，许多迭代器并不只是指针；例如，我们可以定义一个带范围检查的迭代器：当有人试图让它指向其 `[begin:end)` 序列之外，或对它解引用 `end` 时，它就抛出异常。事实证明，把迭代器当作抽象概念而不是某个具体类型，能给我们带来极大的灵活性与通用性。本章和下一章会给出若干例子。

试一试

写一个函数 `void copy(int* f1, int* e1, int* f2)`，把由 `[f1:e1)` 定义的 `int` 数组元素复制到另一个区间 `[f2:f2+(e1-f1))` 中。只使用上面提到的迭代器操作（不要用下标）。

迭代器用来把我们写的代码（算法）与我们的数据连接起来。写代码的人了解迭代器（而不必了解迭代器实际怎样取得数据的细节），提供数据的人只提供迭代器，而不把数据如何存储的细节暴露给所有用户。这样得到的结果简洁得令人愉快，并使算法与容器之间获得了相当程度的独立性。用 Alex Stepanov 的话说：“STL 的算法与容器之所以配合得如此之好，是因为它们对彼此一无所知。”它们共同了解的，只是由一对迭代器定义的序列。



换句话说，我的算法再也不必了解存储和访问数据的五花八门的方式，只需了解迭代器。反过来，如果我提供数据，也不必为了服务五花八门的用户而写各种代码，只需为我的数据实现一个迭代器。在最基本的层面上，一个迭代器仅由 `*`、`++`、`==` 和 `!=` 这几个运算符定义。这使它们既简单又快速。

STL 框架由大约 10 个容器和大约 60 个算法组成，它们通过迭代器连接起来（见第 21 章）。此外，许多组织和个人都按 STL 的风格提供容器与算法。STL 大概是目前最著名、使用最广的泛型编程（generic programming，§19.3.2）范例。只要掌握了基本概念和几个例子，其余部分你都能用起来。

20.3.1 回到那个例子

下面看看如何用 STL 的序列概念来表达“找出值最大的元素”这个问题：

```
template<typename Iterator>
Iterator high(Iterator first, Iterator last)
    // 返回一个迭代器，指向 [first:last) 中值最大的元素
{
    Iterator high = first;
    for (Iterator p = first; p!=last; ++p)
        if (*high<*p) high = p;
    return high;
}
```

注意，我们去掉了原来用来保存“目前所见最大值”的局部变量 h。当我们不知道序列元素的实际类型名时，用 -1 来初始化就显得完全没有道理、十分古怪。之所以如此，是因为它本来就没有道理、就是古怪！它还留下了一个迟早会发生的错误：在我们的例子里，-1 之所以能用，只是因为我们恰好没有负的速度值。我们早就知道，像 -1 这样的“魔法常量”不利于代码维护（§4.3.1、§7.6.1、§10.11.1 等）。这里我们看到，它们还会限制一个函数的用途，并可能成为“对解法思考不完整”的标志；也就是说，“魔法常量”可能——而且常常——是思维马虎的标志。

注意，这个“泛型”的 high() 可用于任何能用 < 比较的元素类型。例如，我们可以用 high() 在 vector<string> 中找出按字典序排在最后的 string（见习题 7）。

模板函数 high() 可用于任何由一对迭代器定义的序列。例如，我们可以把示例程序原样重写一遍：

```
double* get_from_jack(int* count);     // Jack 把 double 放入一个数组，
                                       // 并通过 *count 返回其中的元素个数
vector<double>* get_from_jill();       // Jill 填充这个 vector


void fct()
{
    int jack_count = 0;
    double* jack_data = get_from_jack(&jack_count);
    vector<double>* jill_data = get_from_jill();

    double* jack_high = high(jack_data,jack_data+jack_count);
    vector<double>& v = *jill_data;
    double* jill_high = high(&v[0],&v[0]+v.size());
    cout << "Jill's high " << *jill_high << "; Jack's high " << *jack_high;
    // . . .
    delete[] jack_data;
    delete jill_data;
}
```

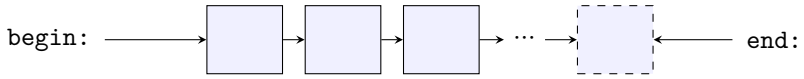
对这里的两次调用，high() 的模板实参类型 Iterator 都是 double*。除了（终于）把 high() 的代码写对之外，这与此前的解法表面上没有差别。准确地说，执行的代码没有差别，但代码的通用性有了极为重要的差别。模板版的 high() 可用于任何能用一对迭代器描述的序列。在细看 STL 的详细约定、以及它提供的那些能替我们省去编写常见棘手代码的有用标准算法之前，我们先来考虑另外几种存储数据元素集合的方式。

试一试

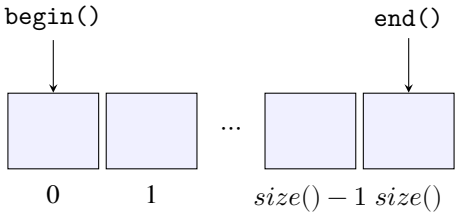
那个程序里我们又留下了一个严重错误。找出它、改正它，并针对这类问题提出一种通用的补救办法。

20.4 链表

再次考虑序列这一概念的图形表示：



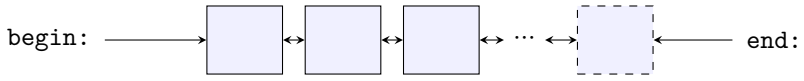
把它与我们在内存中想象 vector 的方式作个比较：



基本上，下标 0 标识的元素与迭代器 v.begin() 标识的是同一个元素；而下标 v.size() 标识的是最后一个元素之后的那一个元素，也就是迭代器 v.end() 所标识的元素。

vector 的元素在内存中连续存放。STL 的序列概念并不要求这一点；而且恰好有许多算法希望能在两个已有元素之间插入一个元素，而又不移动这些已有元素。抽象概念的图形表示暗示了这样一种可能：插入元素（以及删除元素）时不必移动其他元素。STL 的迭代器概念支持这一点。

STL 序列图最直接暗示的数据结构叫做链表（linked list）。抽象模型中的箭头通常实现为指针。链表的每个元素都处在一个“链节”（link）之中，链节由该元素以及一个或多个指针组成。链节只有一个指针（指向下一个链节）的链表叫单向链表（singly-linked list）；链节同时指向前一个和后一个链节的链表叫双向链表（doubly-linked list）。我们将勾勒双向链表的实现，C++ 标准库以 list 之名提供的正是这种链表。用图形表示，它可以是这样的：

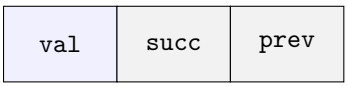


它可以用代码表示如下：

```
template<typename Elem>
struct Link {
    Link* prev;          // 前一个链节
    Link* succ;          // 后继（下一个）链节
    Elem val;            // 值
};
```

```
template<typename Elem> struct list {
    Link<Elem>* first;
    Link<Elem>* last;   // 最后一个链节之后的那一个位置
};
```

Link 的布局是



实现链表并把它们呈现给用户的方式有很多种。标准库版本的描述可以在附录 B 中找到。这里我们只概述 list 的关键性质——可以在不打扰已有元素的情况下插入和删除元素——然后演示如何遍历链表，并给出一个使用 list 的例子。

在思考链表时，我们强烈建议你画些小图，把所考虑的操作可视化。链表操作确实是一个“一图胜千言”的主题。

20.4.1 链表操作

链表需要哪些操作？

- vector 具有的那些操作（构造函数、size 等），但不含下标操作
- 插入（添加一个元素）与删除（移除一个元素）
- 某种可以用来引用元素、遍历链表的设施：迭代器

在 STL 中，那个迭代器类型是该类的成员，我们也照此办理：

```
template<typename Elem>
class list {
    // 表示与实现细节
public:
    class iterator;             // 成员类型：iterator

    iterator begin();           // 指向第一个元素的迭代器
    iterator end( );            // 指向最后一个元素之后那一个位置的迭代器

    iterator insert(iterator p, const Elem& v);    // 把 v 插入到 p 之后
    iterator erase(iterator p);                    // 从链表中移除 p

    void push_back(const Elem& v);      // 在末尾插入 v
    void push_front(const Elem& v);     // 在前端插入 v
    void pop_front();                   // 移除第一个元素
    void pop_back();                    // 移除最后一个元素

    Elem& front();                      // 第一个元素
    Elem& back();                       // 最后一个元素

    // . . .
};
```

正如“我们的”vector 不是完整的标准库 vector，这个 list 也不是标准库 list 的完整定义。这个 list 本身没有任何错误，它只是不完整。“我们的”list 的目的是让你理解链表是什么、链表可能怎样实现、以及怎样使用它的关键特性。更多信息见附录 B 或一本专家级的 C++ 图书。

迭代器在 STL list 的定义中处于中心地位。迭代器用来标识插入位置和要移除（删除）的元素，也用来在链表中“导航”，而不是使用下标。这种迭代器用法与我们在 §20.1 和 §20.3.1 中用指针遍历数组和 vector 的方式非常相似。这种迭代器风格是使用标准库算法的关键（§21.1–3）。

为什么 list 不用下标？我们可以为 list 提供下标，但那会是一个慢得出奇的操作：lst[1000] 要从第一个元素出发，沿途逐个访问链节，直到第 1000 个元素。如果确实想这样做，我们可以自己做（或使用 advance()；见 §20.6.2）。因此，标准库的 list 不提供那种看起来无害的下标语法。

我们把 list 的迭代器类型做成成员（嵌套类），因为它没有理由成为全局的：它只与链表一起使用。此外，这样做还使我们能把每个容器的迭代器类型都命名为 iterator。在标准库里，我们有 list<T>::iterator、vector<T>::iterator、map<K,V>::iterator，等等。

20.4.2 迭代

list 的迭代器必须提供 *、++、== 和 !=。由于标准库的 list 是双向链表，它还提供 --，用于朝链表前端“向后”迭代：

```
template<typename Elem>     // 要求 Element<Elem>() (§19.3.3)
class list<Elem>::iterator {
    Link<Elem>* curr;           // 当前链节
public:
    iterator(Link<Elem>* p) :curr{p} { }

    iterator& operator++() {curr = curr->succ; return *this; }  // 前进
    iterator& operator--() { curr = curr->prev; return *this; }  // 后退
    Elem& operator*() { return curr->val; }   // 取值（解引用）

    bool operator==(const iterator& b) const { return curr==b.curr; }
    bool operator!= (const iterator& b) const { return curr!=b.curr; }
};
```

这些函数又短又简单，效率也显然很高：没有循环，没有复杂的表达式，也没有“可疑的”函数调用。如果实现对你来说还不够清楚，快速看一眼上面的图即可。这个链表迭代器不过是一个指向链节的指针，外加所需的那些操作。注意，尽管 list<Elem>::iterator 的实现（代码）与我们用来充当 vector 和数组迭代器的简单指针差别很大，但各操作的含义（语义）完全相同。基本上，链表迭代器为 Link 指针提供了合适的 ++、--、*、== 和 !=。

现在再看 high()：

```
template<typename Iter>  // 要求 Input_iterator<Iter>() (§19.3.3)
Iterator high(Iter first, Iter last)
    // 返回一个迭代器，指向 [first,last) 中值最大的元素
{
    Iterator high = first;
    for (Iterator p = first; p!=last; ++p)
        if (*high<*p) high = p;
    return high;
}
```

（译注：原书此处模板函数 high() 的返回类型印作 Iterator，而形参类型名为 Iter，按上下文返回类型应为 Iter；代码依原书逐字保留。）

我们可以用它处理 list：

```
void f()
{
    list<int> lst; for (int x; cin >> x; ) lst.push_front(x);

    list<int>::iterator p = high(lst.begin(), lst.end());
    cout << "the highest value was " << *p << '\n';
}
```

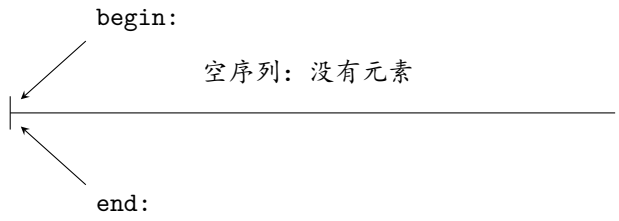
这里，Iterator 实参的“值”是 list<int>::iterator，++、* 和 != 的实现与数组的情形相比已经面目全非，但含义仍然相同。模板函数 high() 仍然遍历数据（这里是链表）并找出最大值。由于我们可以在链表的任何位置插入元素，我们用了 push_front() 把元素添加到前端，只是为了表明可以这么做。我们也完全可以像处理 vector 那样改用 push_back()。

试一试

标准库的 vector 不提供 push_front()。为什么？请为 vector 实现 push_front()，并把它与 push_back() 作比较。

现在，终于到了问这个问题的时候：“可要是链表是空的呢？”换句话说：“如果 lst.begin()==lst.end() 呢？”这种情况下，*p 就是对最后一个元素之后那个位置的解引用，也就是 lst.end()：灾难！或者——可能更糟——结果可能是一个随机值，却被误当成正确答案。

最后那种提法已经强烈暗示了解法：我们可以比较 begin() 和 end() 来判断链表是否为空——事实上，对任何 STL 序列，只要比较它的开头和末尾，就能判断它是不是空的：



这正是我们让 end 指向最后一个元素之后、而不是指向最后一个元素的深层原因：空序列不再是特殊情况。我们讨厌特殊情况，因为按定义，我们必须记得为它们编写特殊代码。

在我们这个例子里，可以这样用它：

```
list<int>::iterator p = high(lst.begin(), lst.end());
if (p==lst.end())            // 我们是否到达了末尾？
    cout << "The list is empty";
else
    cout << "the highest value is " << *p << '\n';
```

我们一贯用“把返回值与 end() 比较”来表示“未找到”，在 STL 算法中莫不如此。

由于标准库已经提供了 list，这里我们就不再深入它的实现。作为替代，我们简单看看链表适合做什么（如果你对链表的实现细节感兴趣，见习题 12–14）。

20.5 再次泛化 vector

显然，从 §20.3–4 的例子可以看出，标准库的 vector 有 iterator 成员类型，也有 begin() 和 end() 成员函数（和 std::list 一样）。可是在第 19 章里，我们没有为自己的 vector 提供这些东西。要让不同的容器能够按 §20.3 所介绍的 STL 泛型编程风格大致互换地使用，究竟需要做些什么？下面我们先勾勒方案（为简化起见，忽略分配器），然后再作解释：

```
template<typename T>   // 要求满足 Element<T>() (见 19.3.3)
class vector {
public:
    using size_type = unsigned long;
    using value_type = T;
    using iterator = T*;
    using const_iterator = const T*;

    // . . .
    iterator begin();
    const_iterator begin() const;
    iterator end();
    const_iterator end() const;

    size_type size();

    // . . .
};
```

using 声明为类型创建一个别名；也就是说，对我们的 vector 来说，iterator 是我们选作迭代器的那个类型 T* 的同义词、另一个名字。现在，对于一个名为 v 的 vector，我们可以写

```
vector<int>::iterator p = find(v.begin(), v.end(),32);
```

以及

```
for (vector<int>::size_type i = 0; i<v.size(); ++i) cout << v[i] << '\n';
```

要点在于：为了写出这样的代码，我们其实并不需要知道 iterator 和 size_type 命名的究竟是什么类型。特别是，上面这段代码是用 iterator 和 size_type 表达的，所以对于那些 size_type 不是 unsigned long 的 vector（许多嵌入式处理器上就不是），以及 iterator 不是普通指针而是类的情形（许多流行的 C++ 实现就是如此），它同样适用。

标准对 list 和其他标准容器的定义方式类似。例如：

```
template<typename T>
// 要求满足 Element<T>() (见 19.3.3)
class list {
public:
    class Link;
    using size_type = unsigned long;
    using value_type = T;
    class iterator;                // 见 20.4.2
    class const_iterator;   // 与 iterator 类似，但不允许写入元素

    // . . .
    iterator begin();
    const_iterator begin() const;
    iterator end();
    const_iterator end() const;

    size_type size();

    // . . .
};
```

这样一来，我们就能写出不关心自己用的是 list 还是 vector 的代码。标准库的所有算法都是按 iterator、size_type 这类成员类型名来定义的，因此它们无需不必要地依赖容器的实现，也无需依赖自己操作的究竟是哪一种容器（见第 21 章）。

除了对某个容器 C 写 C::iterator，我们常常更喜欢写 Iterator<C>。用一个简单的模板别名就能做到：

```
template<typename C>
using Iterator = typename C::iterator;
// Iterator<C> 的意思是 typename C::iterator
```

由于语言技术上的原因，我们必须给 C::iterator 加上 typename 前缀，才能说明 iterator 是一个类型；这也是我们更喜欢 Iterator<C> 的部分原因。类似地，我们定义

```
template<typename C>
using Value_type = typename C::value_type;
```

这样我们就可以写 Value_type<C>。这些类型别名并不在标准库里，但你可以在 std_lib_facilities.h 里找到它们。

using 声明是 C++11 的记法，也是 C 和 C++ 中所谓 typedef（§A.16）的推广。

20.5.1 容器遍历

有了 size()，我们就可以把 vector 从第一个元素一直遍历到最后一个元素。例如：

```
void print1(const vector<double>& v)
{
    for (int i = 0; i<v.size(); ++i)
        cout << v[i] << '\n';
}
```

这对链表行不通，因为 list 不提供下标运算。不过，用更简单的范围 for 循环（§4.6.1），我们可以遍历标准库的 vector 和 list。例如：

```
void print2(const vector<double>& v, const list<double>& lst)
{
    for (double x : v)
        cout << x << '\n';

    for (double x : lst)
        cout << x << '\n';
}
```

这对标准库容器以及我们“自己的”vector 和 list 都有效。为什么？诀窍是：范围 for 循环是按 begin() 和 end() 这两个函数定义的，它们分别返回指向我们 vector 元素中第一个位置和末尾之后位置的迭代器。范围 for 循环不过是用迭代器遍历序列的“语法糖”。当我们为自己的 vector 和 list 定义 begin() 和 end() 时，就“顺便”提供了范围 for 所需要的东西。

20.5.2 auto

当我们要为泛型结构写出循环时，给迭代器起名字可能真的很烦人。考虑：

```
template<typename T>
// 要求满足 Element<T>()
void user(vector<T>& v, list<T>& lst)
{
    for (vector<T>::iterator p = v.begin(); p!=v.end(); ++p) cout << *p << '\n';

    list<T>::iterator q = find(lst.begin(), lst.end(),T{42});
}
```

最烦人的一点是:编译器显然已经知道这个 list 的迭代器类型,也知道这个 vector 的 size_type。我们为什么非得把编译器已经知道的事再告诉它一遍?这么做只会惹恼我们当中那些打字不快的人,还给错误留下可乘之机。幸运的是,我们不必如此:可以把变量声明为 auto,意思是“用迭代器的类型作为这个变量的类型”:

```
template<typename T>
// 要求满足 Element<T>()
void user(vector<T>& v, list<T>& lst)
{
    for (auto p = v.begin(); p!=v.end(); ++p) cout << *p << '\n';

    auto q = find(lst.begin(), lst.end(),T{42});
}
```

这里 p 是 vector<T>::iterator,q 是 list<T>::iterator。几乎每个带初始化器的定义都可以用 auto。例如:

```
auto x = 123;   // x 是 int
auto c = 'y';   // c 是 char
auto& r = x;    // r 是 int&
auto y = r;     // y 是 int (引用被隐式解引用)
```

注意,字符串字面量的类型是 const char*,所以对字符串字面量使用 auto 可能带来不愉快的意外:

```
auto s1 = "San Antonio";        // s1 是 const char* (意外吧!?)
string s2 = "Fredericksburg";   // s2 是 string
```

当我们确切知道自己想要哪个类型时,明确写出来通常和用 auto 一样方便。

auto 的一个常见用途,是指定范围 for 循环中的循环变量。考虑:

```
template<typename C>
// 要求满足 Container<T>
void print3(const C& cont)
{
    for (const auto& x : cont)
        cout << x << '\n';
}
```

这里我们用 auto，因为给容器 cont 的元素类型起名字并不那么容易。我们用 const，是因为不打算写入容器的元素；用 &（引用），是为了防止元素太大、复制起来代价太高。

20.6 示例：一个简单的文本编辑器

链表的基本特点是：增删元素时不必移动链表中的其他元素。我们用一个简单的例子来说明这一点。请考虑：在简单的文本编辑器里，怎样表示一个文本文档的字符？这种表示应当让施加于文档的各种操作既简单又相当高效。

哪些操作？我们假定文档能放进计算机的主存。这样，我们就可以任选一种合用的表示，只在需要存入文件时把它转换成字节流；同样，也可以从文件读入字节流，再把它们转换成内存中的表示。这一点定下来之后，我们就可以专心选择一种方便的内存表示。基本上，我们的表示必须很好地支持五件事：

- 从输入字节流构造它
- 插入一个或多个字符
- 删除一个或多个字符
- 查找一个字符串
- 生成字节流，输出到文件或屏幕

最简单的表示是 vector<char>。可是要增删一个字符，我们就得移动文档中它后面的每一个字符。考虑：

```
This is he start of a very long document.
There are lots of . . .
```

把缺少的 t 补上，就得到

```
This is the start of a very long document.
There are lots of . . .
```

可是，如果这些字符存放在单个 vector<char> 里，我们就得把从 h 开始的每个字符都向右移一个位置。那将意味着大量的复制。

事实上，对于一份长达 7 万个字符的文档（比如本章，空格也计算在内），平均要搬动 35 000 个字符才能插入或删除一个字符。由此造成的实时延迟很可能被用户察觉，并让他们恼火。于是我们把这种表示方式“拆成”若干“块”（chunk），这样修改文档的一部分时就不必搬动大量字符。我们把文档表示成“行”的链表 list<Line>，其中 Line 是一个 vector<char>。例如：

```
This is the start of a very long document.          第 1 行
There are lots of . . .                             第 2 行
```

现在，当我们插入了那个 t 时，只需要搬动该行上其余的字符。而且，必要时我们还可以新增一行，而不必搬动任何字符。例如，我们可以在 document. 之后插入 This is a new line.，得到

```
This is the start of a very long document.
This is a new line.
There are lots of . . .
```

我们所需要做的，只是在中间插入一个新的“行”：

```
This is the start of a very long document.                    第 1 行
There are lots of . . .                                       第 2 行
This is a new line.                                           第 3 行
```

能在链表中插入新链接而不必搬动已有链接，从逻辑上说之所以重要，是因为可能有迭代器指向这些链接，或者有指针（和引用）指向这些链接中的对象。这类迭代器和指针不会因为插入或删除行而失效。例如，一个文本处理器可以维护一个 `vector<list<Line>::iterator>`，用来保存指向当前 `Document` 中每个标题和子标题开头的迭代器：

```
20.1  Storing and processing data                             第 1 行
20.2  STL ideals                                              第 307 行
20.3  Sequences and iterators                                 第 870 行
```

我们可以向“段落 20.2”添加行，而不会让指向“段落 20.3”的迭代器失效。

总之，我们使用“行的链表”，而不是“行的向量”或“所有字符的向量”，这既有逻辑上的理由，也有性能上的理由。请注意，适用这些理由的情形其实相当少见，所以“默认使用 vector”的经验法则仍然成立。要让链表优于向量，你需要一个具体的理由——即使你心里就是把数据看成一串元素！（见 §20.7。）链表是一个逻辑概念，在程序中你既可以用（链式）链表表示它，也可以用向量表示它。与我们日常所说的“列表”（例如待办清单、购物清单或日程表）最接近的 STL 对应物是序列（sequence），而大多数序列最好用向量来表示。

20.6.1 行

我们如何确定文档中什么是“行”？有三种显而易见的选择：

1. 依靠用户输入中的换行标记（例如 `'\n'`）。
2. 设法解析文档，并使用某种“自然的”标点（例如.）。
3. 把任何超过给定长度（例如 50 个字符）的行一分为二。

当然，还有一些不那么显而易见的选择。为简单起见，我们在这里采用第 1 种方案。

在我们的编辑器里，一份文档用 `Document` 类的对象来表示。把各种改进都去掉之后，我们的文档类型是这样的：

```
using Line = vector<char>;          // 一行就是一个字符向量

struct Document {
    list<Line> line;                // 一份文档就是一个行的链表
    Document() { line.push_back(Line{}); }
};
```

每个 `Document` 一开始都只有一行空行：`Document` 的构造函数造出一个空行，并把它压入行链表。

读取并切分成行可以这样做：

```
istream& operator>>(istream& is, Document& d)
{
    for (char ch; is.get(ch); ) {
        d.line.back().push_back(ch);            // 加入这个字符
        if (ch=='\n')
            d.line.push_back(Line{});           // 再加一行
    }
    if (d.line.back().size()) d.line.push_back(Line{});   // 末尾再加一个空行
    return is;
}
```

vector 和 list 都有成员函数 back()，它返回对最后一个元素的引用。要使用它，你必须确信容器里确实有最后一个元素可供 back() 引用——不要在空容器上使用它。这就是我们把 Document 定义成以空行结尾的原因。注意，我们保存了输入中的每一个字符，连换行符（'\n'）也保存。保存这些换行符会大大简化输出，但你必须小心定义字符计数（只数字符，得到的数目会包含空格和换行字符）。

20.6.2 迭代

如果文档只是一个 vector<char>，遍历它会很简单。可是，我们怎样遍历一个行的链表呢？显然，我们可以用 list<Line>::iterator 遍历这个链表。然而，如果我们想一个接一个地访问字符，而完全不理会行的边界，那该怎么办？我们可以提供一个专门为 Document 设计的迭代器：

```
class Text_iterator {       // 记录行以及行内的字符位置
    list<Line>::iterator ln;
    Line::iterator pos;
public:
    // 把迭代器置于第 ll 行的第 pp 个字符处:
    Text_iterator(list<Line>::iterator ll, Line::iterator pp)
        :ln{ll}, pos{pp} { }
    char& operator*() { return *pos; }
    Text_iterator& operator++();
    bool operator==(const Text_iterator& other) const
        { return ln==other.ln && pos==other.pos; }
    bool operator!=(const Text_iterator& other) const
        { return !(*this==other); }
};
```

```
Text_iterator& Text_iterator::operator++()
{
    ++pos;                                  // 前进到下一个字符
    if (pos==(*ln).end()) {
        ++ln;                               // 前进到下一行
       pos = (*ln).begin();                // 如果 ln==line.end() 就糟了，所以要保证不是这种情况
    }
    return *this;
}
```

为了让文本迭代器（Text_iterator）有用，我们需要给 Document 类配上常规的 begin() 和 end() 函数：

```
struct Document {
    list<Line> line;
    Text_iterator begin()       // 第一行的第一个字符
        { return Text_iterator(line.begin(), (*line.begin()).begin()); }
    Text_iterator end()         // 最后一行的最后一个字符之后的位置
    {
        auto last = line.end();
        --last;                 // 我们知道这份文档非空
        return Text_iterator(last, (*last).end());
    }
};
```

之所以要写 (*line.begin()).begin() 这种奇怪的记法，是因为我们要的是 line.begin() 所指对象的开头；也可以写成 line.begin()->begin()，因为标准库迭代器支持 ->。

现在，我们可以像这样遍历一份文档中的字符：

```
void print(Document& d)
{
    for (auto p : d) cout << *p;
}


print(my_doc);
```

把文档呈现为一个字符序列对许多事情都有用，但通常我们遍历文档是为了寻找某个比单个字符更具体的东西。例如，下面这段代码用来删除第 n 行：

```
void erase_line(Document& d, int n)
{
    if (n<0 || d.line.size()-1<=n) return;
    auto p = d.line.begin();
    advance(p,n);
    d.line.erase(p);
}
```

调用 advance(p,n) 会把迭代器 p 向前移动 n 个元素；advance() 是标准库函数，但我们也可以自己这样实现：

```
template<typename Iter>    // 要求满足 Forward_iterator<Iter>
void advance(Iter& p, int n)
{
    while (0<n) { ++p; --n; }
}
```

注意，advance() 可以用来模拟下标运算。事实上，对于名为 v 的 vector，p=v.begin(); advance(p,n); *p=x 大致等价于 v[n]=x。这里的“大致”是指：advance() 要费力地逐个越过前 n-1 个元素，而下标则直接到达第 n 个元素。对于链表，我们只能用这种费力的办法。这是为链表元素更灵活的布局所付出的代价。

对于既能前进又能后退的迭代器（例如链表的迭代器），给标准库的 advance() 传入负的参数会使迭代器向后移动。对于支持下标的迭代器（例如 vector 的迭代器），标准库的 advance() 会直接到达正确的元素，而不是用 ++ 慢慢挪过去。显然，标准库的 advance() 比我们写的要聪明一些。这一点值得注意：通常，标准库设施上所花费的精心与时间，是我们负担不起的，所以应当优先使用标准设施，而不是“自家酿造”。

试一试

试着修改 advance()，使它在你传入负参数时能“向后退”。

对用户来说，最直观的一类迭代大概就是搜索了。我们会搜索单个单词（例如 milkshake 或 Gavin），搜索那些不太容易被看成一个单词的字母序列（例如 secret\nhomestead——即一行以 secret 结尾，下一行以 homestead 开头），搜索正则表达式（例如 [bB]\w*ne——即一个大写或小写的 B，后面跟 0 个或多个字母，再跟 ne；见第 23 章），等等。下面我们用文档的布局来演示第二种情形：查找一个字符串。我们采用一个简单——并非最优——的算法：

- 在文档中找出搜索串的第一个字符。
- 看看这个字符以及其后的字符是否与搜索串匹配。
- 如果匹配，我们就完成了；如果不匹配，就继续寻找该首字符下一次出现的位置。

为了通用，我们采用 STL 的惯例，把要搜索的文本定义成由一对迭代器界定的序列。这样，我们的搜索函数既能用于整份文档，也能用于文档的任意片段。如果在文档中找到了这个字符串的一次出现，就返回指向其首字符的迭代器；如果没找到，就返回指向序列末尾的迭代器：

```
Text_iterator find_txt(Text_iterator first, Text_iterator last, const string& s)
{
    if (s.size()==0) return last;   // 空字符串不可能找到
    char first_char = s[0];
    while (true) {
        auto p = find(first,last,first_char);
        if (p==last || match(p,last,s)) return p;
        first = ++p;                // 看下一个字符
    }
}
```

用“返回序列末尾”来表示“没有找到”，是一项重要的 STL 惯例。`match()` 函数很平常，它只是比较两个字符序列。请试着自己写出来。用来在字符序列中查找字符的 `find()`，大概要算最简单的标准库算法了（§21.2）。我们可以像这样使用 `find_txt()`：

```
auto p = find_txt(my_doc.begin(), my_doc.end(), "secret\nhomestead");
if (p==my_doc.end())
    cout << "not found";
else {
    // 做一些事情
}
```

我们这个“文本处理器”及其操作都非常简单。显然，我们追求的是简单和合理的效率，而不是提供一个“功能齐全”的编辑器。不过，不要把这件事想得太容易：为任意字符序列提供高效的插入、删除和查找，绝不是一件小事。我们选这个例子，是为了说明 STL 概念——序列、迭代器、容器（例如 `list` 和 `vector`）——的强大与通用，同时说明一些 STL 编程惯例（技巧），例如用返回序列末尾来表示失败。注意，如果我们愿意，完全可以把 `Document` 发展成一个 STL 容器——通过提供 `Text_iterator`，我们已经完成了把 `Document` 表示为值序列的关键部分。

20.7 vector、list 与 string

我们为什么用 `list` 存放行、用 `vector` 存放字符？更准确地说，为什么用 `list` 表示行的序列、用 `vector` 表示字符的序列？还有，为什么不用 `string` 来存放一行？

我们可以把这个问题的形式稍微推广一点。到现在为止，我们已经见过四种存放字符序列的方式：

- `char[]`（字符数组）
- `vector<char>`
- `string`
- `list<char>`

面对一个具体问题，我们怎样在它们之间做选择？对于真正简单的任务，它们是可以互换的，也就是说，它们的接口非常相似。例如，给定一个迭代器，我们都能用 `++` 遍历其中元素，用 `*` 访问字符。如果回头看与 `Document` 有关的代码示例，我们实际上可以把 `vector<char>` 换成 `list<char>` 或 `string`，而不会有任何逻辑问题。这种可互换性从根本上说是好事，因为它让我们可以依据性能来做选择。不过，在考虑性能之前，我们应该先看看这些类型在逻辑上的性质：每一种能做哪些别的方式做不到的事？

- `Elem[]`：不知道自己的大小。没有 `begin()`、`end()`，也没有其他有用的容器成员函数。无法进行系统的范围检查。可以传给用 C 编写的函数和 C 风格函数。元素在内存中连续分配。数组的大小在编译时固定。比较（`==` 和 `!=`）和输出（`<<`）用的是指向数组首元素的指针，而不是元素本身。
- `vector<Elem>`：几乎什么都能做，包括 `insert()` 和 `erase()`。提供下标操作。插入和删除这类列表操作通常要移动元素（元素很大或元素很多时，这可能很低效）。可以做范围检查。元素在内存中连续分配。`vector` 可以扩充（例如用 `push_back()`）。`vector` 的元素（连续地）存放在一个数组中。比较运算符（`==`、`!=`、`<`、`<=`、`>` 和 `>=`）比较的是元素。
- `string`：提供所有常用而有用的操作，另外还提供专门的文本操作，例如拼接（`+` 和 `+=`）。元素保证在内存中连续存放。`string` 可以扩充。比较运算符（`==`、`!=`、`<`、`<=`、`>` 和 `>=`）比较的是元素。
- `list<Elem>`：除下标操作外，提供所有常用的操作。我们可以在不移动其他元素的前提下 `insert()` 和 `erase()`。每个元素需要多花两个字（用于链接指针）。`list` 可以扩充。比较运算符（`==`、`!=`、`<`、`<=`、`>` 和 `>=`）比较的是元素。

正如我们已经看到的（§17.2、§18.6），在尽可能低的层次上处理内存，以及同用 C 编写的代码接口时，数组是有用而且必要的（§27.1.2、§27.5）。除此以外，我们更愿意用 `vector`，因为它更容易使用、更灵活，也更安全。

试一试

这些差别在实际代码里意味着什么？请分别用 `char` 数组、`vector<char>`、`list<char>` 和 `string` 各定义一个值为 `"Hello"` 的对象，把它作为实参传给一个函数，输出传进来的这个字符串中字符的个数，在这个函数里把它与 `"Hello"` 比较（看看你传进来的到底是不是 `"Hello"`），再把它与 `"Howdy"` 比较，看按字典序哪一个在前。最后把这个实参复制到同类型的另一个变量中。

试一试

请对 `int` 数组、`vector<int>` 和 `list<int>` 重做上面那个“试一试”，它们的值都是 `{ 1, 2, 3, 4, 5 }`。

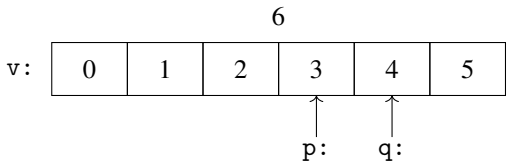
20.7.1 insert 与 erase

标准库的 `vector` 是我们默认的容器选择。它具备我们想要的大部分特性，因此只有在不得已时才改用别的容器。它的主要问题在于：执行列表操作（`insert()` 和 `erase()`）时习惯性地移动元素；当我们处理元素很多或元素很大的 `vector` 时，这可能代价高昂。不过，也不必为此过分担心。我们曾用 `push_back()` 把五十万个浮点值读入一个 `vector`，实测确认预分配并没有带来明显差别。在为了性能而做重大改动之前，一定要先测量；即使是专家，猜测性能也非常困难。

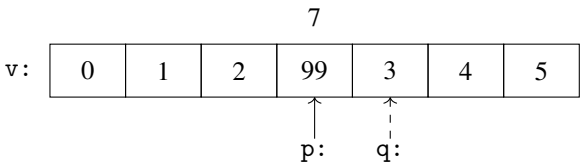
正如 §20.6 所指出的，移动元素还带来一个逻辑上的约束：对 `vector` 的元素做列表操作（例如 `insert()`、`erase()` 和 `push_back()`）时，不要持有指向这些元素的迭代器或指针：一旦元素被移动，你的迭代器或指针就会指向错误的元素，或者根本不指向任何元素。这是 `list`（以及 `map`，见 §21.6）相对于 `vector` 的主要优势。如果你需要的是一批大对象构成的集合，或者需要从程序中的许多地方指向这些对象，就应该考虑使用 `list`。

下面比较 vector 和 list 的 insert() 与 erase()。先看一个只为说明要点的例子：

```
vector<int>::iterator p = v.begin();     // 取一个 vector
++p; ++p; ++p;                           // 指向它的第 4 个元素
auto q = p;
++q;                                     // 指向它的第 5 个元素
```

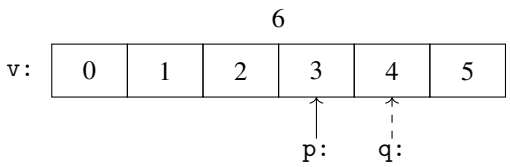



```
p = v.insert(p,99);     // p 指向被插入的元素
```



注意，q 现在失效了。随着 vector 大小的增长，元素可能已经被重新分配。如果 v 还有空闲容量，从而在原地增长，那么 q 很可能指向值为 3 的那个元素，而不是值为 4 的元素，但不要试图利用这一点。

```
p = v.erase(p);     // p 指向被删除元素之后的那个元素
```

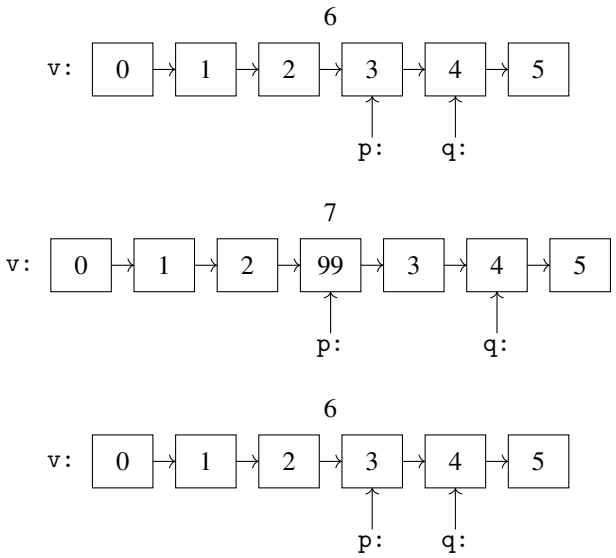


也就是说，先 insert() 再 erase() 掉刚插入的那个元素，我们又回到了起点，只是 q 已经失效。不过在这中间，插入点之后的所有元素都被移动了，而且随着 v 的增长，可能所有元素都被重新安置过。

作为对比，我们用 list 做完全相同的事情：

```
list<int>::iterator p = v.begin();     // 取一个 list
++p; ++p; ++p;                         // 指向它的第 4 个元素
auto q = p;
++q;                                   // 指向它的第 5 个元素


p = v.insert(p,99);     // p 指向被插入的元素
```

注意，q 仍然指向值为 4 的那个元素。




```
p = v.erase(p);     // p 指向被删除元素之后的那个元素
```

我们又一次回到了起点。不过，与 vector 不同，对 list 来说我们没有移动任何元素，q 自始至终都是有效的。

一个 list<char> 占用的内存至少是另外三种方案的三倍——在 PC 上，list<char> 每个元素用 12 字节，而 vector<char> 每个元素只用 1 字节。字符数量很大时，这个差别可能很显著。

在哪些方面 vector 优于 string 呢？看看前面列出的那些性质，似乎 string 能做到 vector 能做的一切，甚至更多。问题的一部分正在这里：既然 string 要做更多事情，它就更难优化。事实上，vector 往往针对 push_back() 这类“内存操作”进行优化，而 string 往往不是这样。相反，string 往往针对复制、针对短字符串的处理，以及针对与 C 风格字符串的交互进行优化。在文本编辑器的例子里，我们选择 vector，是因为我们用到了 insert() 和 delete()。不过那只是性能上的理由。逻辑上的主要区别在于：vector 可以存放几乎任何元素类型。只有在我们考虑字符的时候，才需要在两者之间做选择。总之，除非你需要 string 的操作，例如拼接或读取以空白分隔的单词，否则优先用 vector 而不是 string。

20.8 让 vector 适配 STL

在 §20.5 中加上 begin()、end() 和那些类型别名之后，我们的 vector 只缺 insert() 和 erase()，就和所需的 std::vector 近似到足够程度了：

```
template<typename T, typename A = allocator<T>>
        // requires Element<T>() && Allocator<A>() (§19.3.3)
class vector {
    int sz;              // 元素个数
    T* elem;             // 指向元素的指针
    int space;           // 元素个数加上空闲“槽位”的个数
    A alloc;             // 用 allocate 为元素管理内存
public:
    // ……第 19 章和 §20.5 中所有其他内容……
    using iterator = T*; // T* 是最简单的迭代器
    iterator insert(iterator p, const T& val);
    iterator erase(iterator p);
};
```

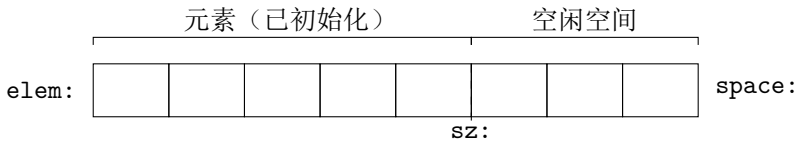
我们又一次用指向元素类型的指针 `T*` 作为迭代器类型。这是最简单的方案。提供带范围检查的迭代器留作习题（习题 18）。

通常，人们不会为把元素保存在连续存储中的数据类型（例如 `vector`）提供 `insert()` 和 `erase()` 这类列表操作。不过，对于较短的 `vector` 或元素个数较少的情形，`insert()` 和 `erase()` 这类列表操作极其有用，效率也高得出人意料。我们已经多次看到 `push_back()` 的用处，而它正是另一种传统上与列表联系在一起的操作。

基本上，`vector<T,A>::erase()` 的实现办法是：复制被删除元素之后的所有元素（删除、抹掉）。使用 §19.3.7 中 `vector` 的定义，再加上前面的补充，我们得到

```
template<typename T, typename A>
        // requires Element<T>() && Allocator<A>() (§19.3.3)
vector<T,A>::iterator vector<T,A>::erase(iterator p)
{
    if (p==end()) return p;
    for (auto pos = p+1; pos!=end(); ++pos)
        *(pos-1) = *pos;            // 把元素“向左复制一个位置”
    alloc.destroy(&*(end()-1));     // 销毁最后一个元素多出来的副本
    --sz;
    return p;
}
```

如果看一看图形化的表示，这类代码就更容易理解：



`erase()` 的代码相当简单，但动手在纸上画几个例子试试可能是个好主意。空的 `vector` 处理得对吗？为什么需要 `p==end()` 这个判断？如果删除的是 `vector` 的最后一个元素，会怎样？如果我们改用下标表示法，这段代码会更容易读吗？

实现 `vector<T,A>::insert()` 要稍微复杂一些：

```
template<typename T, typename A>
        // requires Element<T>() && Allocator<A>() (§19.3.3)
vector<T,A>::iterator vector<T,A>::insert(iterator p, const T& val)
{
    int index = p-begin();
    if (size()==capacity())
        reserve(size()==0?8:2*size());  // 确保有空间

    // 先把最后一个元素复制到未初始化的空间:
    alloc.construct(elem+sz,*back());
    ++sz;
    iterator pp = begin()+index;    // 放置 val 的位置
    for (auto pos = end()-1; pos!=pp; --pos)
        *pos = *(pos-1);            // 把元素向右复制一个位置
    *(begin()+index) = val;         // “插入” val
    return pp;
}
```

请注意：

- 迭代器不能指向它的序列之外，因此这种场合我们使用指针，例如 `elem+sz`。这也是分配器用指针而不是用迭代器来定义的原因之一。
- 使用 `reserve()` 时，元素可能被移动到新的内存区域。因此，我们必须记住待插入元素的下标，而不是指向它的迭代器。当 `vector` 重新分配元素时，指向该 `vector` 的迭代器都会失效——你可以把它们看作指向了旧的内存。
- 我们对分配器参数 `A` 的用法很直观，但并不准确。如果你真的需要实现一个容器，就得仔细阅读标准。
- 正是这类微妙之处，让我们尽可能避免处理低层的内存问题。当然，标准库的 `vector`——以及所有其他标准库容器——都把这些重要的语义细节处理对了。这也是宁可用标准库而不用“自家土制”东西的原因之一。

出于性能原因，你不会在十万个元素的 `vector` 中间使用 `insert()` 和 `erase()`；那种场合用 `list`（以及 `map`，见 §21.6）更好。不过，`insert()` 和 `erase()` 对所有 `vector` 都可使用，而且当你只是移动几个字的数据——甚至几十个字——时，它们的性能是无可匹敌的，因为现代计算机非常擅长这类复制；参见习题 20。如果列表里只是少量小元素，就不要用（链式的）`list` 来表示它。

20.9 让内建数组适配 STL

我们已经反复指出内建数组的弱点：一有机会它们就隐式转换成指针；不能用赋值来复制；不知道自己有多大（§18.6.2）；等等。我们也指出了它们的主要优点：它们几乎完美地模拟了物理内存。

为了兼得两者之长，我们可以构造一个数组容器，既有数组的优点，又没有数组的弱点。`array` 的一个版本曾作为技术报告（Technical Report，TR）的一部分被引入标准。由于 TR 中的特性并不要求每个实现都提供，你使用的实现里可能没有 `array`。不过，这个想法既简单又实用：

```
template <typename T, int N>
        // requires Element<T>()
struct array {
        // 与标准库的 array 不完全相同
    using value_type = T;
    using iterator = T*;
    using const_iterator = const T*;
    using size_type = unsigned int;      // 下标的类型

    T elems[N];
    // 不需要显式的构造/复制/销毁

    iterator begin() { return elems; }
    const_iterator begin() const { return elems; }
    iterator end() { return elems+N; }
    const_iterator end() const { return elems+N; }

    size_type size() const;

    T& operator[](int n) { return elems[n]; }
    const T& operator[](int n) const { return elems[n]; }

    const T& at(int n) const;            // 带范围检查的访问
    T& at(int n);                        // 带范围检查的访问

    T * data() { return elems; }
    const T * data() const { return elems; }
};
```

这个定义并不完整，也不完全符合标准，但它能让你明白思路。如果你的实现还没有提供标准 array，它也能给你一个可用的东西。如果实现提供了，它就在 <array> 头文件里。注意，由于 array<T,N> “知道”自己的大小是 N，我们可以（也确实）像 vector 那样提供赋值、==、!= 等操作。

举个例子，我们把 array 用在 §20.4.2 中 high() 的 STL 版本上：

```
void f()
{
    array<double,6> a = { 0.0, 1.1, 2.2, 3.3, 4.4, 5.5 };
    array<double,6>::iterator p = high(a.begin(), a.end());


    cout << "the highest value was " << *p << '\n';
}
```

需要注意的是，我们写 high() 时并没有想到 array。之所以能把 high() 用于 array，只是因为我们让两者都遵循了标准约定。

20.10 容器概览

STL 提供了相当多的容器：

标准容器

`vector`	连续分配的元素序列；把它用作默认容器
`list`	双向链表；需要在插入、删除元素时不移动已有元素时用它
`deque`	list 与 vector 的混合体；在你对算法和机器结构具备专家级认识之前，不要用它
`map`	平衡的有序树；需要按值访问元素时用它（见 §21.6.1-3）
`multimap`	平衡的有序树，一个键可以有多份副本；需要按值访问元素时用它（见 §21.6.1-3）
`unordered_map`	哈希表；map 的优化版本；用于大型 map，既需要高性能、又能设计出良好哈希函数时（见 §21.6.4）
`unordered_multimap`	哈希表，一个键可以有多份副本；multimap 的优化版本；用于大型 map，既需要高性能、又能设计出良好哈希函数时（见 §21.6.4）
`set`	平衡的有序树；需要记录一个个独立的值时用它（见 §21.6.5）
`multiset`	平衡的有序树，一个键可以有多份副本；需要记录一个个独立的值时用它（见 §21.6.5）
`unordered_set`	类似 `unordered_map`，但只保存值，不保存 (key,value) 对
`unordered_multiset`	类似 `unordered_multimap`，但只保存值，不保存 (key,value) 对
`array`	固定大小的数组，避开了与内置数组相关的大部分问题（见 §20.9）

你可以从书籍和在线文档中查到关于这些容器及其用法的大量补充信息。下面列出几个高质量的信息来源：

- Josuttis, Nicholai M. *The C++ Standard Library: A Tutorial and Reference*. Addison-Wesley, 2012. ISBN 978-0321623218。只使用第 2 版。
- Lippman, Stanley B.、Jose Lajoie 和 Barbara E. Moo. *C++ Primer*. Addison-Wesley, 2005. ISBN 0201721481。只使用第 5 版。
- Stroustrup, Bjarne. *The C++ Programming Language*. Addison-Wesley, 2012. ISBN 978-0321714114。只使用第 4 版。
- SGI 版 STL 实现与 iostream 库的文档：`www.sgi.com/tech/stl`。注意他们还提供了完整代码。

你是不是觉得受了骗？你觉得我们应该把容器及其用法的方方面面都给你讲清楚？这根本不可能。标准设施太多，有用的技术太多，有用的库也太多，你没法一次全部吸收。编程这个领域太丰富，没有人能掌握所有设施和技术——它也可以是一门高尚的技艺。作为程序员，你必须养成主动查找语言设施、库和技术方面新信息的习惯。编程是一个动态而快速发展的领域，只满足于自己已经知道、已经用惯的东西，就等于等着被甩在后面。“去查一查”是对很多问题的完全合理的回答；随着你的技能不断增长、成熟，它越来越经常地就是那个答案。

另一方面，你会发现：一旦理解了 vector、list 和 map，以及第 21 章介绍的标准算法，其他 STL 容器和 STL 风格的容器就都很容易使用。你也会发现，自己已经掌握了理解非 STL 容器以及使用它们的代码的基本知识。

什么是容器？STL 容器的定义在上述所有资料里都能找到。这里只给出一个非正式的定义。STL 容器具有以下性质：

- 它是一个元素序列 [begin():end())。
- 它提供复制元素的复制操作。复制可以通过赋值或复制构造函数完成。
- 它把元素类型命名为 value_type。
- 它有名为 iterator 和 const_iterator 的迭代器类型。迭代器提供 *、++（前缀和后缀）、== 和 !=，并具有相应的语义。list 的迭代器还提供 --，用于在序列中向后移动；这种迭代器称为双向迭代器。vector 的迭代器还提供 --、[ ]、+ 和 -，称为随机访问迭代器。（见 §20.10.1）
- 它提供 insert() 和 erase()、front() 和 back()、push_back() 和 pop_back()、size() 等操作；vector 和 map 还提供下标访问（例如 operator [ ]）。
- 它提供比较运算符（==、!=、<、<=、>、>=），比较的是元素。容器对 <、<=、>、>= 采用字典序：即从第一个元素开始按顺序比较。

这份清单的目的是给你一个总体印象。更多细节见附录 B。更精确的规格说明和完整清单，见 *The C++ Programming Language* 或标准本身。

有些数据类型提供了标准容器所要求的大部分功能，但并非全部。我们有时把它们称为“近似容器”。其中最有意思的几个是：

“近似容器”

T[n] 内置数组	没有 size() 等成员函数；有选择余地时，优先用 vector、string 或 array 这样的容器，而不是内置数组
string	只存放字符，但提供对文本处理有用的操作，例如连接（+ 和 +=）；优先用标准 string，而不是其他字符串
valarray	带向量运算的数值向量，但为了便于高性能实现而有许多限制；只有进行大量向量算术时才用它

此外，许多人和许多组织都实现过满足标准容器要求（或几乎满足）的容器。

如果你拿不准，就用 vector。除非你有充分的理由不用 vector，否则就用 vector。

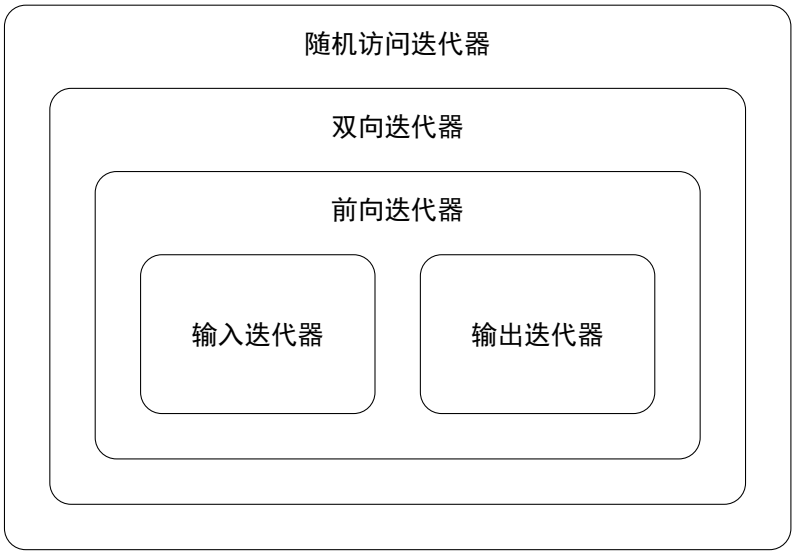
20.10.1 迭代器类别

我们一直把迭代器讲得好像所有迭代器都可以互换。如果你只做最简单的操作，例如把序列遍历一遍、每个值读一次，它们确实可以互换。如果想做更多的事，例如反向遍历或下标访问，就需要更高级的迭代器。

迭代器类别

输入迭代器 `input iterator`	可以用 `++` 向前遍历，用 `*` 读取元素值。istream 提供的就是这种迭代器；见 §21.7.2。如果 `(*p).m` 合法，可以用 `p->m` 作为简写。
输出迭代器 `output iterator`	可以用 `++` 向前遍历，用 `*` 写入元素值。ostream 提供的就是这种迭代器；见 §21.7.2。
前向迭代器 `forward iterator`	可以反复用 `++` 向前遍历，并用 `*` 读、写元素值（元素是 const 时当然不能写）。如果 `(*p).m` 合法，可以用 `p->m` 作为简写。
双向迭代器 `bidirectional iterator`	可以向前（`++`）、向后（`--`）遍历，并用 `*` 读、写元素值（元素是 const 时不能写）。list、map 和 set 提供的就是这种迭代器。如果 `(*p).m` 合法，可以用 `p->m` 作为简写。
随机访问迭代器 `random-access iterator`	可以向前（`++`）、向后（`--`）遍历，并用 `*` 或 `[ ]` 读、写元素值（元素是 const 时不能写）。可以用 `+` 给随机访问迭代器加一个整数，用 `-` 减一个整数。两个指向同一序列的随机访问迭代器相减，可以求出它们的距离。vector 提供的就是这种迭代器。如果 `(*p).m` 合法，可以用 `p->m` 作为简写。

从所提供的操作可以看出：凡是可以使用输出迭代器或输入迭代器的地方，都可以使用前向迭代器。双向迭代器也是前向迭代器，随机访问迭代器也是双向迭代器。用图形表示，迭代器类别之间的关系如下：



注意，迭代器类别并不是类，因此这个层次结构并不是用派生实现的类层次结构。

小试身手

1. 定义一个含十个元素 { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9 } 的 `int` 数组。
2. 用这十个元素定义一个 `vector<int>`。

3. 用这十个元素定义一个 list<int>。
4. 再定义一个 array、一个 vector 和一个 list，分别初始化为第一个 array、vector 和 list 的副本。
5. 把 array 中每个元素的值加 2；把 vector 中每个元素的值加 3；把 list 中每个元素的值加 5。
6. 写一个简单的 copy() 操作：

```
template<typename Iter1, typename Iter2>
        // 要求 Input_iterator<Iter1>() && Output_iterator<Iter2>()
Iter2 copy(Iter1 f1, Iter1 e1, Iter2 f2);
```

6. 它把 [f1,e1) 复制到 [f2,f2+(e1-f1))，并返回 f2+(e1-f1)，与标准库的 copy 函数完全一样。注意，如果 f1==e1，序列为空，因此没有什么可复制。
7. 用你的 copy() 把 array 复制到 vector，再把 list 复制到 array。
8. 用标准库的 find() 检查 vector 中是否含值 3，如果含就打印它的位置；再用 find() 检查 list 中是否含值 27，如果含就打印它的位置。第一个元素的“位置”是 0，第二个元素的位置是 1，依此类推。注意，如果 find() 返回序列末尾，就说明没有找到该值。

记住每完成一步都要测试。

要点回顾

1. 为什么不同的人写出的代码看起来不一样？举例说明。
2. 我们会对数据提出哪些简单问题？
3. 存储数据有哪几种不同方式？
4. 对一组数据项可以做哪些基本操作？
5. 对于数据的存储方式，有哪些理想目标？
6. 什么是 STL 序列？
7. 什么是 STL 迭代器？它支持哪些操作？
8. 怎样把迭代器移到下一个元素？
9. 怎样把迭代器移到前一个元素？
10. 如果把迭代器移到序列末尾之后，会发生什么？
11. 哪些种类的迭代器可以移到前一个元素？
12. 为什么把数据与算法分开是有用的？
13. 什么是 STL？
14. 什么是链表？它与 vector 的根本区别在哪里？
15. 链表中的“链接”是什么？
16. insert() 做什么？erase() 做什么？
17. 怎样判断一个序列是否为空？
18. list 的迭代器提供哪些操作？
19. 怎样用 STL 遍历一个容器？
20. 什么时候你会用 string 而不是 vector？
21. 什么时候你会用 list 而不是 vector？
22. 什么是容器？
23. begin() 和 end() 对容器应该起什么作用？
24. STL 提供哪些容器？

25. 什么是迭代器类别？STL 提供哪些种类的迭代器？
26. 随机访问迭代器提供了哪些双向迭代器没有的操作？

术语

algorithm、array container、auto、`begin()`、container、contiguous、doubly-linked list、element、empty sequence、`end()`、`erase()`、`insert()`、iteration、iterator、linked list、sequence、singly-linked list、`size_type`、STL、traversal、`using`、type alias、`value_type`

习题

1. 如果还没有做，请完成本章中所有的“试一试”练习。
2. 让 §20.1.2 中 Jack 与 Jill 的例子运行起来。用几个小文件的输入来测试它。
3. 看看回文的例子（§18.7）；用那套技术重做 §20.1.2 中 Jack 与 Jill 的例子。
4. 找出 §20.3.1 中 Jack 与 Jill 例子里的错误并修正，全程使用 STL 技术。
5. 为 `vector` 定义输入和输出运算符（`>>` 和 `<<`）。
6. 基于 §20.6.2 为 `Document` 写一个查找并替换的操作。
7. 在一个未排序的 `vector<string>` 中找出字典序最大的字符串。
8. 定义一个函数，统计一个 `Document` 中的字符数。
9. 定义一个程序，统计一个 `Document` 中的单词数。提供两个版本：一个把单词定义为“由空白分隔的字符序列”，另一个把单词定义为“连续的字母字符序列”。例如，按前一个定义，alpha.numeric 和 as12b 都算一个单词；按后一个定义，它们都算两个单词。
10. 定义单词计数程序的一个版本，让用户可以指定空白字符的集合。
11. 给定一个 `list<int>` 作为（按引用的）参数，创建一个 `vector<double>`，把 list 的元素复制进去。验证复制是完整且正确的。然后按值递增的顺序打印这些元素。
12. 补全 §20.4.1–2 中 `list` 的定义，让 `high()` 的例子能运行。要分配一个 `Link` 来表示末尾之后的位置。
13. 对 list 来说，我们其实并不需要一个“真正的”末尾之后 `Link`。修改上一题的解答，用 0 表示指向那个（并不存在的）末尾之后 `Link` 的指针（`list<Elem>::end()`）；这样，空 list 的大小就可以等于单个指针的大小。
14. 仿照 `std::list` 的风格定义一个单向链表 `slist`。由于 `slist` 没有反向指针，`list` 的哪些操作可以合理地从中去掉？
15. 定义一个 `pvector`，它像指针的 vector，但存放的是指向对象的指针，并且它的析构函数会删除每个对象。
16. 定义一个 `ovector`，它类似 `pvector`，但 `[ ]` 和 `*` 运算符返回的是所指向对象的引用，而不是指针。
17. 定义一个 `ownership_vector`，它像 `pvector` 一样存放指向对象的指针，但提供一种机制，让用户决定哪些对象归这个 vector 所有（即哪些对象会被析构函数删除）。提示：如果你第 13 章没有走神，这道题很简单。
18. 为 `vector` 定义一个带范围检查的迭代器（随机访问迭代器）。
19. 为 `list` 定义一个带范围检查的迭代器（双向迭代器）。
20. 运行一个小型计时实验，比较使用 `vector` 和 `list` 的代价。关于如何给程序计时，可以在 §26.6.1 找到说明。在 [0:N) 范围内生成 N 个随机 int 值。每生成一个 int，就把它插入一个 `vector<int>`（每次增加一个元素）。保持这个 vector 有序：即新值插在它前面所有小于等于它的值之后、所有

大于它的值之前。然后用 `list<int>` 存放这些 int，做同样的实验。对哪些 N，list 比 vector 快？试着解释你的结果。这个实验最初由 John Bentley 提出。

后记

如果有 N 种容器、M 件想对它们做的事，我们很容易最终写出 N*M 段代码。如果数据本身还有 K 种不同类型，甚至会写出 N*M*K 段代码。STL 解决这种膨胀的办法是：把元素类型作为参数（这就处理掉了 K 这个因子），并把对数据的访问与算法分开。让任何算法都通过迭代器去访问任何容器中的数据，我们就只需要 N+M 个算法。这是巨大的简化。例如，如果有 12 个容器和 60 个算法，蛮力做法需要 720 个函数，而 STL 的策略只需要 60 个函数和 12 个迭代器定义：我们一下子省掉了 90% 的工作量。事实上，这还低估了省下的工夫，因为许多算法要接受两对迭代器，而这两对迭代器不必是同一类型（例如见习题 6）。此外，STL 还提供了定义算法的约定，使编写正确、可组合的代码更容易，所以节省还要更多。

21 算法与映射

"In theory, practice is simple."
（理论上，实践很简单。）
——特里格维·里恩斯科格

本章结束我们对 STL 基本思想的介绍，也结束我们对它所提供设施的概览。在这里，我们把注意力放在算法上。我们的首要目标是向你介绍十来个最有用的算法——它们能为你省下几天、甚至几个月的工作。每个算法都配有用法示例，也配有它所支撑的编程技术。这里的第二个目标，是给你足够好用的工具：当标准库和其他现成库提供的东西不够用时，你能自己写出优雅而高效的算法。此外，我们还会再介绍三个容器：`map`、`set` 和 `unordered_map`。

21.1 标准库算法
21.2 最简单的算法：find()
  21.2.1 一些通用用法
21.3 通用搜索：find_if()
21.4 函数对象
  21.4.1 函数对象的抽象视角
  21.4.2 类成员上的谓词
  21.4.3 lambda 表达式
21.5 数值算法
  21.5.1 accumulate
  21.5.2 泛化 accumulate()
  21.5.3 内积
  21.5.4 泛化 inner_product()
21.6 关联容器
  21.6.1 map
  21.6.2 map 概览
  21.6.3 另一个 map 示例
  21.6.4 unordered_map
  21.6.5 set
21.7 复制
  21.7.1 copy
  21.7.2 流迭代器
  21.7.3 用 set 保持顺序
  21.7.4 copy_if
21.8 排序与查找
21.9 容器算法

21.1 标准库算法

标准库提供了大约 80 个算法。它们对某个人来说总有时候有用；我们关注其中一些对许多人常常有用的算法，也关注一些偶尔对某个人非常有用的算法：

选定的标准算法

`r=find(b,e,v)`	r 指向 $[b:e)$ 中 v 的第一次出现。
`r=find_if(b,e,p)`	r 指向 $[b:e)$ 中第一个使 p(x) 为真的元素 x。
`x=count(b,e,v)`	x 是 v 在 $[b:e)$ 中出现的次数。
`x=count_if(b,e,p)`	x 是 $[b:e)$ 中使 p(x) 为真的元素个数。
`sort(b,e)`	用 $<$ 对 $[b:e)$ 排序。
`sort(b,e,p)`	用 p 对 $[b:e)$ 排序。
`copy(b,e,b2)`	把 $[b:e)$ 复制到 $[b2:b2+(e-b))$；b2 之后最好有足够多的元素。
`unique_copy(b,e,b2)`	把 $[b:e)$ 复制到 $[b2:b2+(e-b))$；相邻的重复元素不复制。
`merge(b,e,b2,e2,r)`	把两个已排序序列 $[b2:e2)$ 与 $[b:e)$ 合并到 $[r:r+(e-b)+(e2-b2))$。
`r=equal_range(b,e,v)`	r 是已排序区间 $[b:e)$ 中值为 v 的子序列，本质上是对 v 的二分查找。
`equal(b,e,b2)`	$[b:e)$ 与 $[b2:b2+(e-b))$ 的元素都比较相等吗？
`x=accumulate(b,e,i)`	x 是 i 与 $[b:e)$ 中各元素之和。
`x=accumulate(b,e,i,op)`	与另一个 accumulate 类似，但“求和”用 op 计算。
`x=inner_product(b,e,b2,i)`	x 是 $[b:e)$ 与 $[b2:b2+(e-b))$ 的内积。
`x=inner_product(b,e,b2,i,op,op2)`	与另一个 inner_product 类似，但用 op 和 op2 代替 $+$ 与 $*$。

默认情况下，相等比较用 == 完成，排序则基于 <（小于）。标准库算法定义在头文件 <algorithm> 中。更多信息参见 §B.5，以及 §21.2 至 §21.5 中列出的资料。这些算法接受一个或多个序列。输入序列由一对迭代器界定；输出序列由指向其首元素的迭代器界定。通常，算法由一个或多个操作来参数化，这些操作既可以是函数对象，也可以是函数。算法通常通过返回输入序列的末尾来报告“失败”。例如，如果 find(b,e,v) 找不到 v，它就返回 e。

21.2 最简单的算法：find()

可以说，最简单的有用算法就是 find()。它在一个序列中查找具有给定值的元素：

```
template<typename In, typename T>
    // 要求 Input_iterator<In>()
    //   并且 Equality_comparable<Value_type<T>>() (§19.3.3)
In find(In first, In last, const T& val)
    // 返回 [first,last) 中第一个等于 val 的元素
{
    while (first!=last && *first != val) ++first;
    return first;
}
```

我们来看看 find() 的定义。当然，你可以在完全不知道它如何实现的情况下使用它——事实上我们已经用过了（例如 §20.6.2）。不过，find() 的定义体现了很多有用的设计思想，所以值得一看。

首先，find() 作用于由一对迭代器界定的序列。我们要在半开序列 $[first : last)$ 中查找值 val。find() 返回的结果是一个迭代器。这个结果要么指向序列中第一个值为 val 的元素，要么指向 last。返回指向序列中最后一个元素之后那个位置的迭代器，是 STL 报告“未找到”的最常见方式。所以我们可以这样使用 find()：

```
void f(vector<int>& v, int x)
{
    auto p = find(v.begin(),v.end(),x);
    if (p!=v.end()) {
        // 在 v 中找到了 x
    }
    else {
        // v 中没有 x
    }
    // . . .
}
```

这里和常见情形一样，序列由某个容器的全部元素构成（一个 STL vector）。我们把返回的迭代器与序列末尾作比较，看是否找到了目标值。返回值的类型，就是作为实参传入的那种迭代器。

为了避免写出返回的类型，我们用了 auto。用 auto 这个“类型”定义的对象，会获得其初始化器的类型。例如：

```
auto ch = 'c';      // ch 是 char
auto d = 2.1;       // d 是 double
```

auto 类型说明符在泛型代码中特别有用，比如在 find() 里，写出实际类型（这里是 vector<int>::iterator）可能很繁琐。

现在我们知道怎样使用 find()，因此也就知道怎样使用一大批遵循同样约定的其他算法。在继续看更多用法和更多算法之前，我们先仔细看看那个定义：

```
template<typename In, typename T>
    // 要求 Input_iterator<In>()
    //   并且 Equality_comparable<Value_type<T>>() (§19.3.3)
In find(In first, In last, const T& val)
    // 返回 [first,last) 中第一个等于 val 的元素
{
    while (first!=last && *first != val) ++first;
    return first;
}
```

你第一眼觉得那个循环很直观吗？我们不觉得。它其实已经是最小的、最高效的，并且直接表达了那个基本算法。不过，在你见过几个例子之前，它并不直观。我们把它按“笨办法”写出来，看看这一版与之相比如何：

```
template<typename In, typename T>
    // 要求 Input_iterator<In>()
    //   并且 Equality_comparable<Value_type<T>>() (§19.3.3)
In find(In first, In last, const T& val)
    // 返回 [first,last) 中第一个等于 val 的元素
{
    for (In p = first; p!=last; ++p)
        if (*p == val) return p;
    return last;
}
```

这两个定义在逻辑上等价，真正优秀的编译器会为两者生成同样的代码。但在现实中，许多编译器还不够好，既消除不了那个多余的变量（p），也无法重排代码把所有测试都集中到一处。为什么要为此费心还专门解释？一部分原因是，`find()` 第一种（也是我们偏好的）版本的风格已经非常流行，你必须理解它才能读懂别人的代码；另一部分原因是，性能恰恰对那种处理大量数据、又小又常用的函数最要紧。

试一试

你确定这两个定义逻辑等价吗？你怎样才能确定？试着为它们的等价性构造一个论证。做到这一点之后，再用一些数据把两者都试一遍。一位著名的计算机科学家（Don Knuth）曾说：“我只证明了算法正确，并没有测试它。”即使数学证明也可能出错。要有把握，你既需要推理，也需要测试。

21.2.1 一些通用用法

`find()` 算法是通用的。也就是说，它可以用于不同的数据类型。事实上，它在两种意义上都是通用的：它可以用于

- 任何 STL 风格的序列
- 任何元素类型

下面是一些例子（如果看糊涂了，可以查阅 §20.4 中的图）：

```
void f(vector<int>& v, int x)        // 适用于元素为 int 的 vector
{
    vector<int>::iterator p = find(v.begin(),v.end(),x);
    if (p!=v.end()) { /* 找到了 x */ }
    // . . .
}
```

在这里，find() 使用的迭代操作就是 vector<int>::iterator 的操作；也就是说，++（在 ++first 中）只是把指针移到内存中的下一个位置（vector 的下一个元素就存在那里），*（在 *first 中）则对该指针解引用。迭代器比较（在 first!=last 中）是指针比较，值比较（在 *first!=val 中）则只是比较两个整数。

我们换成 list 再试一次：

```
void f(list<string>& v, string x)        // 适用于元素为 string 的 list
{
    list<string>::iterator p = find(v.begin(),v.end(),x);
    if (p!=v.end()) { /* 找到了 x */ }
    // . . .
}
```

在这里，find() 使用的迭代操作是 list<string>::iterator 的操作。这些运算符具有所需的含义，所以逻辑与上面的 vector<int> 相同。不过实现差别很大：++（在 ++first 中）沿着元素的 Link 部分中的指针走到 list 下一个元素的存放位置，*（在 *first 中）则取出 Link 的值部分。迭代器比较（在 first!=last 中）是对 Link* 的指针比较，值比较（在 *first!=val 中）则用 string 的 != 运算符比较字符串。

所以 find() 极为灵活：只要我们遵守迭代器的简单规则，就能用它为我们能想到的任何序列、为我们愿意定义的任何容器查找元素。例如，我们可以用 find() 在 §20.6 定义的 Document 中查找一个字符：

```
void f(Document& v, char x)        // 适用于字符类型为 char 的 Document
{
    Text_iterator p = find(v.begin(),v.end(),x);
    if (p!=v.end()) { /* 找到了 x */ }
    // . . .
}
```

这种灵活性是 STL 算法的标志，也使它们比大多数人初遇它们时想象的更有用。

21.3 通用搜索：find_if()

我们其实并不那么经常查找某个具体的值。更常见的是，我们想找出满足某种条件的值。如果搜索的条件能由我们自己定义，find 操作就会有用得多。也许你想找一个大于 42 的值；也许你想比较字符串时不考虑大小写（大写还是小写）；也许你想找第一个奇数值；也许你想找地址字段为“17 Cherry Tree Lane”的那条记录。

基于用户提供的条件进行搜索的标准算法是 find_if()：

```
template<typename In, typename Pred>
    // 要求 Input_iterator<In>() && Predicate<Pred,Value_type<In>>()
In find_if(In first, In last, Pred pred)
{
    while (first!=last && !pred(*first)) ++first;
    return first;
}
```

很明显（把源码对比一下就知道了），它和 find() 一样，只是用 !pred(*first) 取代了 *first!=val；也就是说，一旦谓词 pred() 成功它就停止搜索，而不是等某个元素等于目标值才停止。

谓词是返回 true 或 false 的函数。显然，find_if() 要求谓词接受一个参数，这样它才能写出 pred(*first)。我们很容易写出检查值某种性质的谓词，比如“这个字符串里含有字母 x 吗?”“这个值大于 42 吗?”“这个数是奇数吗?”例如，我们可以像下面这样找出一个 int 的 vector 中第一个奇数值：

```
bool odd(int x) { return x%2; }          // % 是取模运算符

void f(vector<int>& v)
{
    auto p = find_if(v.begin(), v.end(), odd);
    if (p!=v.end()) { /* 找到了一个奇数 */ }
    // . . .
}
```

对那次 find_if() 调用，find_if() 会为每个元素调用 odd()，直到找到第一个奇数值。注意：把函数作为实参传递时，不要在函数名后面加 ()，因为那会调用它。

同样，我们可以像下面这样找出一个 list 中第一个大于 42 的元素：

```
bool larger_than_42(double x) { return x>42; }

void f(list<double>& v)
{
    auto p = find_if(v.begin(), v.end(), larger_than_42);
    if (p!=v.end()) { /* 找到了一个大于 42 的值 */ }
    // . . .
}
```

不过，最后一个例子并不令人满意。要是我们接下来想找大于 41 的元素呢？那得再写一个新函数。找大于 19 的元素呢？又得再写一个函数。一定有更好的办法！

如果我们想与任意值 v 比较，就需要设法让 v 成为 find_if() 的谓词的隐式参数。我们可能会试着这样写（选用 v_val 这个不太容易与其他名字冲突的名字）：

```
double v_val;        // larger_than_v() 用来与自己的参数比较的那个值
bool larger_than_v(double x) { return x>v_val; }
```

```
void f(list<double>& v, int x)
{
    v_val = 31;    // 为下一次调用 larger_than_v 把 v_val 设为 31
    auto p = find_if(v.begin(), v.end(), larger_than_v);
    if (p!=v.end()) { /* 找到了一个大于 31 的值 */ }

    v_val = x;     // 为下一次调用 larger_than_v 把 v_val 设为 x
    auto q = find_if(v.begin(), v.end(), larger_than_v);
    if (q!=v.end()) { /* 找到了一个大于 x 的值 */ }
    // . . .
}
```

真难看！我们相信，写这种代码的人早晚会自食其果，但我们同情他们的用户，也同情任何不得不维护这种代码的人。再说一遍：一定有更好的办法！

试一试

我们为什么如此厌恶那种对 v 的用法？请至少举出三种可能由此引发难以察觉的错误的方式。再列出三种你最不愿意看到这类代码的应用领域。

21.4 函数对象

于是，我们希望把一个谓词传给 find_if()，并且希望这个谓词把元素与某个以参数形式指定的值作比较。具体地说，我们希望写出类似这样的代码：

```
void f(list<double>& v, int x)
{
    auto p = find_if(v.begin(), v.end(), Larger_than(31));
    if (p!=v.end()) { /* 我们找到了一个大于 31 的值 */ }
    auto q = find_if(v.begin(), v.end(), Larger_than(x));
    if (q!=v.end()) { /* 我们找到了一个大于 x 的值 */ }
    // . . .
}
```

显然，Larger_than 必须是这样的东西：

- 我们可以把它当作谓词来调用，例如 pred(*first);
- 它能保存一个值（例如 31 或 x），供调用时使用。

为此我们需要一个“函数对象”（function object），也就是一个可以像函数那样使用的对象。之所以需要对象，是因为对象能保存数据，例如用来比较的那个值。例如：

```
class Larger_than {
    int v;
public:
    Larger_than(int vv) : v(vv) { }                 // 保存参数
    bool operator()(int x) const { return x>v; }    // 比较
};
```

有意思的是，这个定义让上面的例子能够按预期工作。现在只需弄清它为什么能工作。当我们写 Larger_than(31) 时，（显然）我们创建了一个 Larger_than 类的对象，其数据成员 v 中保存着 31。例如：

```
find_if(v.begin(),v.end(),Larger_than(31))
```

这里，我们把那个对象作为名为 pred 的形参传给 find_if()。对 v 的每个元素，find_if() 都会发起一次调用

```
pred(*first)
```

这会用实参 *first 调用我们函数对象的调用运算符，即 operator()。结果就是把元素的值 *first 与 31 作比较。

由此我们看到：函数调用可以看作一种运算符，即“() 运算符”，就像其他任何运算符一样。“() 运算符”也叫函数调用运算符或应用运算符。于是，pred(*first) 中的 ( ) 由 Larger_than::operator() 赋予含义，正如 v[i] 中的下标由 vector::operator[] 赋予含义一样。

21.4.1 函数对象的抽象视图

这里有一个机制，它让“函数”能够随身携带自己所需要的数据。显然，函数对象为我们提供了一种非常通用、强大而便利的机制。考虑一个更一般的函数对象概念：

```
class F {                                   // 函数对象的抽象示例
    S s;                                    // 状态
public:
    F(const S& ss) :s(ss) { /* 建立初始状态 */ }
    T operator() (const S& ss) const
    {
        // 用 ss 对 s 做点什么
        // 返回一个 T 类型的值 (T 通常是 void、bool 或 S)
    }
    const S& state() const { return s; }        // 显示状态
    void reset(const S& ss) { s = ss; }         // 重置状态
};
```

F 类的对象在其成员 s 中保存数据。如果需要，函数对象可以有多个数据成员。说某样东西保存着数据，另一种说法是它“带有状态”。创建 F 时，我们可以初始化这个状态；任何时候我们都可以读取这个状态。对 F，我们提供了一个操作 state() 来读取状态，另一个操作 reset() 来写入状态。不过，设计函数对象时，我们可以自由地提供任何我们认为合适的访问状态的方式。当然，我们也可以

用普通的函数调用记法直接或间接地调用函数对象。我们把 F 定义为被调用时接受一个实参，但我们可以定义参数个数任意多的函数对象。

使用函数对象是 STL 中最主要的参数化手段。我们用函数对象指明搜索中要找的东西（§21.3）、定义排序准则（§21.4.2）、指定数值算法中的算术运算（§21.5）、定义值相等意味着什么（§21.8），以及做更多的事。函数对象的使用是灵活性与通用性的主要来源。

函数对象通常非常高效。特别地，按值把小的函数对象传给模板函数，通常能带来最优的性能。原因很简单，但对那些更熟悉“把函数作为参数传递”的人来说却有些意外：通常，传函数对象生成的代码比传函数要小得多，也快得多！只有满足以下条件时这一点才成立：对象很小（大约零个、一个或两个字的数据），或者按引用传递；并且函数调用运算符很小（例如用 < 做一次简单比较），且被定义为内联（例如定义就写在类内部）。本章——以及本书——的大多数例子都遵循这一模式。小而简单的函数对象之所以性能高，根本原因在于它们保留了足够的类型信息，使编译器能生成最优的代码。即便是优化器并不高明的老编译器，也能为 Larger_than 中的比较生成一条简单的“大于”机器指令，而不是去调用函数。调用一次函数通常比执行一次简单的比较运算慢 10 到 50 倍。此外，函数调用的代码也比一次简单比较的代码大好几倍。

21.4.2　针对类成员的谓词

正如我们已经看到的，标准算法对基本类型（如 int 和 double）的元素序列工作得很好。然而在某些应用领域，存放类值的容器要常见得多。考虑一个在许多领域的应用中都十分关键的示例：按若干准则对记录排序：

```
struct Record {
    string name;        // 用标准 string，便于使用
    char addr[24];      // 为匹配数据库布局而沿用旧式写法
    // . . .
};


vector<Record> vr;
```

有时我们想按姓名对 vr 排序，有时又想按地址排序。除非这两种排序都能做得既优雅又高效，否则我们的技术在实际中价值有限。幸运的是，做到这一点很容易。我们可以写

```
// . . .
sort(vr.begin(), vr.end(), Cmp_by_name());          // 按姓名排序
// . . .
sort(vr.begin(), vr.end(), Cmp_by_addr());          // 按地址排序
// . . .
```

Cmp_by_name 是一个函数对象，它通过比较两个 Record 的 name 成员来比较这两个 Record；Cmp_by_addr 则通过比较 addr 成员来比较两个 Record。为了让用户能指定这样的比较准则，标准库的 sort 算法接受一个可选的第三参数来指定排序准则。Cmp_by_name() 创建一个 Cmp_by_name，供 sort() 用来比较 Record。这看起来不错——意思是说，我们并不介意维护这样写的代码。现在只要定义 Cmp_by_name 和 Cmp_by_addr 就行了：

```
// 用于 Record 对象的两种不同比较:
struct Cmp_by_name {
    bool operator()(const Record& a, const Record& b) const
        { return a.name < b.name; }
};

struct Cmp_by_addr {
    bool operator()(const Record& a, const Record& b) const
        { return strncmp(a.addr, b.addr, 24) < 0; }          // !!!
};
```

Cmp_by_name 类相当直观。它的函数调用运算符 operator() 只是用标准 string 的 < 运算符比较姓名串。然而 Cmp_by_addr 里的比较却很丑。原因是我们为地址选择了一种丑陋的表示：24 个字符的数组（且不以零结尾）。我们这样选，一方面是为了展示函数对象如何把丑陋而容易出错的代码隐藏起来，另一方面是因为这种表示曾被人当作挑战摆到我面前：“一个丑陋而重要的现实问题，STL 处理不了。”好吧，STL 能处理。比较函数使用了标准 C（和 C++）库函数 strncmp()，它比较定长字符数组，若第二个“字符串”按字典序排在第一个之后，就返回一个负数。哪天你真的需要做这种冷僻的比较（例如 §B.11.3），就去查查它。

21.4.3 lambda 表达式

在程序的一处定义函数对象（或函数）、再在另一处使用它，有时会有点麻烦。特别是，如果要执行的动作本身很容易写明、很容易理解，而且以后再也不需要，那就更烦人了。这种情况下，我们可以使用 lambda 表达式（§15.3.3）。理解 lambda 表达式的最好方式，也许就是把它看成一种简写记法：定义一个函数对象（一个带 operator() 的类），并立刻创建一个该类的对象。例如，我们本可以写成

```
// . . .
sort(vr.begin(), vr.end(),                    // 按姓名排序
    [](const Record& a, const Record& b)
        { return a.name < b.name; }
);
// . . .
sort(vr.begin(), vr.end(),                    // 按地址排序
    [](const Record& a, const Record& b)
        { return strncmp(a.addr, b.addr, 24) < 0; }
);
// . . .
```

在这种情况下，我们会想：用一个有名字的函数对象，代码是不是更好维护？也许 Cmp_by_name 和 Cmp_by_addr 还有别的用处。

不过，请回想 §21.4 中 find_if() 的例子。那里我们需要把一个操作作为参数传递，而这个操作还需要随身携带数据：

```
void f(list<double>& v, int x)
{
    auto p = find_if(v.begin(), v.end(), Larger_than(31));
    if (p!=v.end()) { /* 我们找到了一个大于 31 的值 */ }
    auto q = find_if(v.begin(), v.end(), Larger_than(x));
    if (q!=v.end()) { /* 我们找到了一个大于 x 的值 */ }
    // . . .
}
```

或者，等价地，我们也可以写成

```
void f(list<double>& v, int x)
{
    auto p = find_if(v.begin(), v.end(), [](double a) { return a>31; });
    if (p!=v.end()) { /* 我们找到了一个大于 31 的值 */ }
    auto q = find_if(v.begin(), v.end(), [&](double a) { return a>x; });
    if (q!=v.end()) { /* 我们找到了一个大于 x 的值 */ }
    // . . .
}
```

与局部变量 x 的比较，使 lambda 版本颇具吸引力。

21.5 数值算法

标准库算法大多处理数据管理方面的问题：复制、排序、查找数据，等等。不过，也有少数算法帮助进行数值计算。这些数值算法在你做计算时可能很重要，同时它们也示范了如何在 STL 框架内表达数值算法。

按 STL 风格提供的标准库数值算法只有四个：

数值算法

`x=accumulate(b,e,i)`	把一串值相加；例如对 $\{a,b,c,d\}$ 得到 $i+a+b+c+d$。结果 x 的类型就是初始值 i 的类型。
`x=inner_product(b,e,b2,i)`	把两个序列中成对的值相乘，再把结果求和；例如对 $\{a,b,c,d\}$ 和 $\{e,f,g,h\}$ 得到 $i+a*e+b*f+c*g+d*h$。结果 x 的类型就是初始值 i 的类型。
`r=partial_sum(b,e,r)`	产生由序列前 n 个元素之和构成的序列；例如对 $\{a,b,c,d\}$ 得到 $\{a,a+b,a+b+c,a+b+c+d\}$。
`r=adjacent_difference(b,e,b2,r)`	产生由序列中相邻元素之差构成的序列；例如对 $\{a,b,c,d\}$ 得到 $\{a,b-a,c-b,d-c\}$。

它们定义在 <numeric> 中。这里我们描述前两个，其余两个就留给你在需要时自己探索。

21.5.1 accumulate

最简单也最有用的数值算法是 accumulate()。在其最简单的形式中，它把一串值相加：

```
template<typename In, typename T>
    // 要求 Input_iterator<T>() && Number<T>()
T accumulate(In first, In last, T init)
{
    while (first!=last) {
        init = init + *first;
        ++first;
    }
    return init;
}
```

给定初始值 `init`，它只是把 `[first:last)` 序列中的每个值都加到 `init` 上，然后返回这个和。用来累计求和的变量 `init` 通常称为累加器（accumulator）。例如：

```
int a[] = { 1, 2, 3, 4, 5 };
cout << accumulate(a, a+sizeof(a)/sizeof(int), 0);
```

这会打印出 15，即 $0+1+2+3+4+5$（0 是初始值）。显然，`accumulate()` 可用于各种序列：

```
void f(vector<double>& vd, int* p, int n)
{
    double sum = accumulate(vd.begin(), vd.end(), 0.0);
    int sum2 = accumulate(p,p+n,0);
}
```

结果（那个和）的类型，就是 `accumulate()` 用来保存累加器的变量的类型。这带来了一定程度的灵活性，而这种灵活性有时很重要。例如：

```
void g(int* p, int n)
{
    int s1 = accumulate(p, p+n, 0);              // 把和累加到 int 中
    long sl = accumulate(p, p+n, long{0});       // 把这些 int 累加成 long
    double s2 = accumulate(p, p+n, 0.0);         // 把这些 int 累加成 double
}
```

在某些计算机上，`long` 的有效数字比 `int` 多。`double` 能表示比 `int` 更大（也更小）的数，但精度可能更低。我们会在第 24 章重新讨论取值范围与精度在数值计算中的作用。

用你想存放结果的那个变量作为初始值，是一种常见的惯用法，用来指定累加器的类型：

```
void f(vector<double>& vd, int* p, int n)
{
    double s1 = 0;
    s1 = accumulate(vd.begin(), vd.end(), s1);
    int s2 = accumulate(vd.begin(), vd.end(), s2);       // 糟糕
    float s3 = 0;
    accumulate(vd.begin(), vd.end(), s3);                // 糟糕
}
```

务必记住：要初始化累加器，并把 accumulate() 的结果赋给该变量。在这个例子中，s2 在被初始化之前就被用作初始值了，因此结果是未定义的。我们把 s3 传给了 accumulate()（按值传递，见§8.5.3），但结果从未赋给任何地方；那次编译纯粹是浪费时间。

21.5.2 推广 accumulate()

基本的、带三个参数的 accumulate() 执行加法。不过，还有许多其他有用的运算（例如乘、减）我们可能也想施加在一个序列上，因此 STL 提供了另一个带四个参数的 accumulate() 版本，可以指定要用的运算：

```
template<typename In, typename T, typename BinOp>
    // 要求 Input_iterator<In>() && Number<T>()
    //         && Binary_operator<BinOp,Value_type<In>,T>()
T accumulate(In first, In last, T init, BinOp op)
{
    while (first!=last) {
        init = op(init, *first);
        ++first;
    }
    return init;
}
```

任何接受两个累加器类型实参的二元运算都可以用在这里。例如：

```
vector<double> a = { 1.1, 2.2, 3.3, 4.4 };
cout << accumulate(a.begin(),a.end(), 1.0, multiplies<double>());
```

这会打印出 35.1384，即 $1.0 \times 1.1 \times 2.2 \times 3.3 \times 4.4$（1.0 是初始值）。这里提供的二元运算符 multiplies<double>() 是标准库中执行乘法的函数对象；multiplies<double> 做 double 的乘法，multiplies<int> 做 int 的乘法，等等。还有别的二元函数对象：plus（加法）、minus（减法）、divides（除法）和 modulus（取余数）。它们都定义在 <functional>（§B.6.2）中。

注意，对浮点数的连乘来说，显而易见的初始值是 1.0。

正如 sort() 的例子（§21.4.2）那样，我们常常关心的是类对象内部的数据，而不只是普通的内建类型。例如，给定单价和数量，我们可能想算出一批物品的总价：

```
struct Record {
    double unit_price;
    int units;          // 售出的数量
    // . . .
};
```

我们可以让传给 accumulate 的运算符从 Record 元素中取出数量，再把它乘到累加值上：

```
double price(double v, const Record& r)
{
    return v + r.unit_price * r.units;    // 计算价格并累加
}

void f(const vector<Record>& vr)
{
    double total = accumulate(vr.begin(), vr.end(), 0.0, price);
    // . . .
}
```

我们“偷懒”用了一个函数而不是函数对象来计算价格——只是为了说明我们也能那样做。我们通常更愿意用函数对象，

- 如果它们需要在两次调用之间保存某个值，或者
- 如果它们体量很小，以致内联能带来差别（至多几个原始操作）。

在这个例子中，我们本可以出于第二个理由选用函数对象。

试一试

定义一个 vector<Record>，用你自选的四条记录初始化它，并用上面的函数计算它们的总价。

21.5.3 内积

取两个向量，把下标相同的每一对元素相乘，再把所有这些乘积相加。这称为两个向量的内积（inner product），它在许多领域（例如物理和线性代数，见 §24.6）都是非常有用的运算。如果你更愿意看代码而不是文字，下面是 STL 版本：

```
template<typename In, typename In2, typename T>
    // 要求 Input_iterator<In> && Input_iterator<In2>
    //        && Number<T> (§19.3.3)
T inner_product(In first, In last, In2 first2, T init)
    // 注意：这就是我们把两个向量相乘（得到一个标量）的方式
{
    while(first!=last) {
        init = init + (*first) * (*first2);    // 把每对元素相乘
        ++first;
        ++first2;
    }
    return init;
}
```

这把内积的概念推广到了任意元素类型的任意序列。举个例子，考虑一个股票市场指数。它的工作方式是：取一组公司，给每家公司指定一个“权重”。例如，在我们最后一次查看时，道琼斯工业指数中美国铝业（Alcoa）的权重是 2.4808。要得到指数的当前值，就把每家公司的股价乘以其权重，再把所有得到的加权价格相加。显然，这就是价格序列与权重序列的内积。例如：

```
// 计算道琼斯工业指数：
vector<double> dow_price = {          // 每家公司的股价
    81.86, 34.69, 54.45,
    // . . .
};


list<double> dow_weight = {           // 每家公司在该指数中的权重
    5.8549, 2.4808, 3.8940,
    // . . .
};

double dji_index = inner_product(   // 把（权重，值）对相乘后相加
    dow_price.begin(), dow_price.end(),
    dow_weight.begin(),
    0.0);

cout << "DJI value " << dji_index << '\n';
```

注意，`inner_product()` 处理两个序列，却只接受三个实参：第二个序列只给出了起始位置。第二个序列的元素个数应当不少于第一个序列；否则就会发生运行时错误。就 `inner_product()` 而言，第二个序列比第一个序列元素多是可以的；那些“多余的”元素干脆不会被用到。

这两个序列不必是同一类型，元素类型也不必相同。为了说明这一点，我们用 `vector` 保存价格，用 `list` 保存权重。

21.5.4 推广 inner_product()

inner_product() 可以像 accumulate() 那样推广。不过对 inner_product() 我们需要两个额外的实参：一个用来把累加器与新值结合，完全和 accumulate() 一样；另一个用来结合每对元素的值：

```
template<typename In, typename In2, typename T, typename BinOp,
         typename BinOp2>
    // 要求 Input_iterator<In> && Input_iterator<In2> && Number<T>
    //         && Binary_operation<BinOp,T, Value_type<In>()
    //         && Binary_operation<BinOp2,T, Value_type<In2>()
T inner_product(In first, In last, In2 first2, T init, BinOp op, BinOp2 op2)
{
    while(first!=last) {
        init = op(init, op2(*first, *first2));
        ++first;
        ++first2;
    }
    return init;
}
```

在 §21.6.3 中，我们会回到道琼斯那个例子，并把推广后的 inner_product() 用作一个更优雅解法的一部分。

21.6 关联容器

在 vector 之后，最有用的标准库容器大概要算 map 了。map 是一个有序的（键，值）对序列，你可以根据键来查找对应的值；例如，my_phone_book["Nicholas"] 就可以是 Nicholas 的电话号码。在“人气竞赛”中唯一可能和 map 一争高下的是 unordered_map（见 §21.6.4），那是一种针对字符串键优化过的 map。与 map 和 unordered_map 类似的数据结构有很多名字，比如关联数组、散列表和红黑树。流行而有用的概念似乎总是有很多名字。在标准库中，我们把这类数据结构统称为关联容器（associative container）。

标准库提供八种关联容器：

关联容器

map	有序容器，存放（键，值）对
set	有序容器，存放键
unordered_map	无序容器，存放（键，值）对
unordered_set	无序容器，存放键
multimap	同一个键可以出现多次的 map
multiset	同一个键可以出现多次的 set
unordered_multimap	同一个键可以出现多次的 unordered_map
unordered_multiset	同一个键可以出现多次的 unordered_set

这些容器分别定义在 <map>、<set>、<unordered_map> 和 <unordered_set> 中。

21.6.1 map

考虑一个概念上很简单的任务：列出文本中各个单词出现的次数。最直观的做法是维护一张“已经见过的单词”表，同时记下每个单词被看到过多少次。每读到一个新单词，我们就检查以前是否见过它；如果见过，就把它的计数加一；如果没见过，就把它插入表中，并令其值为 1。用 list 或 vector 也能做到这一点，但那样每读一个单词都要查找一次，可能会很慢。map 存储键的方式让“某个键是否存在”变得很容易判断，于是我们任务中的查找部分就变得微不足道了：

```
int main()
{
    map<string,int> words;    // 保存（单词，频次）对
    for (string s; cin>>s; )
        ++words[s];    // 注意：words 用 string 作下标
    for (const auto& p : words)
        cout << p.first << ": " << p.second << '\n';
}
```

程序真正有趣的地方是 ++words[s]。从 main() 的第一行可以看出，words 是一个（string，int）对的 map，也就是说，words 把 string 映射到 int。换句话说，给定一个 string，words 能让我们访问与之对应的那个 int。所以，当我们用 string（其中保存着从输入读到的单词）对 words 做下标运算时，words[s] 就是与 s 对应的那个 int 的引用。来看一个具体的例子：

```
words["sultan"]
```

如果以前没有见过 "sultan" 这个字符串，那么 "sultan" 就会以 int 的默认值 0 进入 words。这时 words 中就有了一个条目（"sultan",0）。由此可知，如果以前没有见过 "sultan"，++words["sultan"] 就会把值 1 关联到字符串 "sultan" 上。具体来说：map 会发现没有找到 "sultan"，于是插入一个（"sultan",0）对，然后 ++ 把这个值加一，得到 1。

现在再看这个程序：++words[s] 把我们从输入得到的每个“单词”的值加一。一个新单词第一次出现时，得到的值是 1。现在这个循环的含义就清楚了：

```
for (string s; cin>>s; )
    ++words[s];    // 注意：words 用 string 作下标
```

它读入输入中的每个（以空白分隔的）单词，并统计每个单词出现的次数。剩下的工作只是把结果输出。与任何其他 STL 容器一样，我们可以遍历 map。map<string,int> 的元素类型是 pair<string,int>。pair 的第一个成员叫 first，第二个成员叫 second，因此输出循环就成了

```
for (const auto& p : words)
    cout << p.first << ": " << p.second << '\n';
```

作为测试，我们可以把 *The C++ Programming Language* 第一版的开篇文字输入给这个程序：

> *C++ is a general purpose programming language designed to make programming more enjoyable for the serious programmer. Except for minor details, C++ is a superset of the C programming language. In addition to the facilities provided by C, C++ provides flexible and efficient facilities for defining new types.*

得到的输出是

```
C: 1
C++: 3
C,: 1
Except: 1
In: 1
a: 2
addition: 1
and: 1
by: 1
defining: 1
designed: 1
details,: 1
efficient: 1
enjoyable: 1
facilities: 2
flexible: 1
for: 3
general: 1
is: 2
language: 1
language.: 1
make: 1
minor: 1
more: 1
new: 1
of: 1
programmer.: 1
programming: 3
provided: 1
provides: 1
purpose: 1
serious: 1
superset: 1
the: 3
to: 2
types.: 1
```

如果我们不想区分大写和小写字母，或者想消除标点，也可以做到：参见习题 13。

21.6.2 map 概览

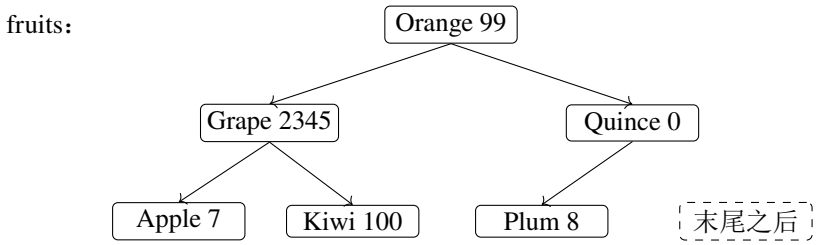
那么 map 究竟是什么？实现 map 的方法有很多，但 STL 的 map 实现往往是平衡二叉搜索树；更具体地说，是红黑树。我们不深入细节，不过现在你已经知道了这些术语，如果想知道更多，可以查文献或上网搜索。

树由结点构成（方式与链表由链接构成类似；见 §20.4）。一个 Node 保存一个键、与该键对应的值，以及指向两个后代 Node 的指针。

map 结点：

Key	`first`
Value	`second`
`Node*`	`left`
`Node*`	`right`
	…

下面是一个 `map<Fruit,int>` 在内存中可能的样子，这里假设我们向其中插入了 (Kiwi,100)、(Quince,0)、(Plum,8)、(Apple,7)、(Grape,2345) 和 (Orange,99)：



既然保存键值的那个 Node 成员名叫 `first`，二叉搜索树的基本规则就是

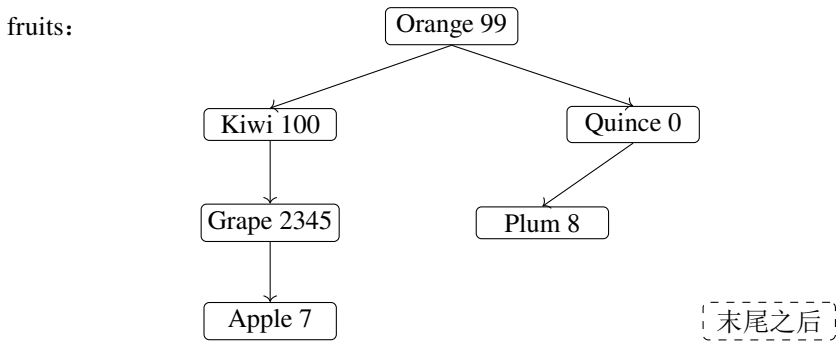
```
left->first<first && first<right->first
```

也就是说，对每个结点：

- 它的左子结点的键小于该结点的键；并且
- 该结点的键小于其右子结点的键。

你可以验证这棵树中每个结点都满足这一条件。正是这一条件让我们能够“从根开始沿树向下”查找。说来有趣，在计算机科学的文献里，树是从根向下生长的。在这个例子中，根结点是 (Orange, 99)。我们就这么一路比较着往下走，直到找到要找的东西，或者走到它本该在的位置。当（如上例）距根同样远的各子树所含的结点数目大致相同时，这棵树就称为平衡的。平衡能把到达一个结点平均需要访问的结点数降到最低。

一个 Node 还可能保存一些额外的数据，`map` 用这些数据来保持结点树的平衡。当每个结点左侧的后代与右侧的后代数目大致相当时，树就是平衡的。如果一棵有 N 个结点的树是平衡的，那么要找某个结点，最多只需查看 $\log_2 N$ 个结点。这比把键放在 `list` 里从头查找（这种线性查找平均要查看 $N/2$ 个结点，最坏情况是 N 个）要好得多。（另见 §21.6.4。）例如，看看下面这棵不平衡的树：



这棵树仍然满足“每个结点的键都大于其左子结点的键、小于其右子结点的键”这一条件：

```
left->first<first && first<right->first
```

然而这个版本的树是不平衡的，所以现在要“跳”三次才能到达 Apple 和 Kiwi，而在平衡的树里只需两次。对结点很多的树来说，这种差别可能非常显著，因此用来实现 map 的树都是平衡的。

要使用 map，我们并不需要懂树。不过，假定专业人员至少了解自己工具的基本原理，是合理的。我们真正必须理解的，是标准库为 map 提供的接口。下面是一个略作简化的版本：

```
template<typename Key, typename Value, typename Cmp = less<Key>>
    // requires Binary_operation<Cmp,Value>() (§19.3.3)
class map {
    // . . .
    using value_type = pair<Key,Value>;    // map 处理 (Key,Value) 对
    using iterator = sometype1;    // 类似于指向树结点的指针
    using const_iterator = sometype2;
    iterator begin();    // 指向第一个元素
    iterator end();    // 指向最后一个元素之后
    Value& operator[](const Key& k);    // 用 k 做下标
    iterator find(const Key& k);    // 有 k 对应的条目吗？
    void erase(iterator p);    // 删除 p 指向的元素
    pair<iterator, bool> insert(const value_type&);    // 插入一个 (键, 值) 对
    // . . .
};
```

真正的版本可以在 <map> 中找到。你可以把迭代器想象成类似 Node* 的东西，但不能指望你的实现真的用这个类型来实现迭代器。

它与 vector 和 list 的接口（§20.5 和 §B.4）明显相似。主要区别在于，迭代时元素是 pair——类型为 pair<Key,Value>。这个类型是又一个有用的 STL 类型：

```
template<typename T1, typename T2>
struct pair {    // std::pair 的简化版本
    using first_type = T1;
    using second_type = T2;
    T1 first;
    T2 second;
    // . . .
};


template<typename T1, typename T2>
pair<T1,T2> make_pair(T1 x, T2 y)
{
    return {x,y};
}
```

我们从标准中逐字抄来了 pair 的完整定义，以及它那个有用的辅助函数 make_pair()。

注意，遍历 map 时，元素会按键所定义的顺序出现。例如，如果遍历例子中的这些水果，我们会得到

```
(Apple,7) (Grape,2345) (Kiwi,100) (Orange,99) (Plum,8) (Quince,0)
```

我们插入这些水果的顺序无关紧要。

`insert()` 操作的返回值有点古怪，在简单程序里我们通常会忽略它。它是一个 pair：一个成员是指向该（键，值）元素的迭代器，另一个是 bool，若这次 `insert()` 调用确实插入了那个（键，值）对，它就为 true。如果键已经在 `map` 中，插入就会失败，bool 为 false。

注意，你可以通过提供第三个实参（`map` 声明中的 `Cmp`）来定义 `map` 所用顺序的含义。例如：

```
map<string, double, No_case> m;
```

`No_case` 定义的是忽略大小写的比较；见 §21.8。默认情况下，顺序由 `less<Key>` 定义，意思是“小于”。

21.6.3 另一个 map 示例

为了更好地体会 `map` 的用处，让我们回到 §21.5.3 的道琼斯例子。那段代码当且仅当每个权重都与对应的名字在 `vector` 中处于同一位置时才是正确的。这是一个隐含的约定，很容易成为难以察觉的 bug 的根源。解决这个问题的办法很多，其中一个很有吸引力：把每个权重与它所属公司的股票代码放在一起，例如 `("AA",2.4808)`。所谓“股票代码”（ticker symbol），就是在需要简短表示时用来代替公司名称的缩写。同样，我们也可以把公司的股票代码与它的股价放在一起，例如 `("AA",34.69)`。最后，

对我们这些不常与美国股市打交道的人来说，可以把公司的股票代码与对应的公司名放在一起，例如（“AA”，“Alcoa Inc.”）；也就是说，我们可以维护三个相互对应的 map。

首先建立（代码，价格）映射：

```
map<string,double> dow_price = {   // 道琼斯工业指数（代码，价格）
                                   // 最新报价见
                                   // www.djindexes.com
    {"MMM",81.86},
    {"AA",34.69},
    {"MO",54.45},
    // . . .
};
```

（代码，权重）映射：

```
map<string,double> dow_weight = {   // 道琼斯指数（代码，权重）
    {"MMM", 5.8549},
    {"AA",2.4808},
    {"MO",3.8940},
    // . . .
};
```

（代码，公司名）映射：

```
map<string,string> dow_name = {   // 道琼斯指数（代码，公司名）
    {"MMM","3M Co."},
    {"AA","Alcoa Inc."},
    {"MO","Altria Group Inc."},
    // . . .
};
```

有了这些 map，我们就可以方便地提取各种信息。例如：

```
double alcoa_price = dow_price["AAA"];      // 从 map 中读取值
double boeing_price = dow_price["BA"];

if (dow_price.find("INTC") != dow_price.end())    // 在 map 中查找表项
    cout << "Intel is in the Dow\n";
```

遍历 map 很容易。只需记住：键名为 `first`，值为 `second`：

```
// 输出道琼斯指数中每家公司的股价：
for (const auto& p : dow_price) {
    const string& symbol = p.first;         // “股票代码”
    cout << symbol << '\t'
         << p.second << '\t'
         << dow_name[symbol] << '\n';
}
```

我们甚至可以直接用 map 做一些计算。具体来说，我们可以像 §21.5.3 那样算出指数。我们需要从各自的 map 中取出股价和权重，再把它们相乘。对任意两个 `map<string,double>`，我们很容易写出一个完成此事的函数：

```
double weighted_value(
    const pair<string,double>& a,
    const pair<string,double>& b
)   // 提取值并相乘
{
    return a.second * b.second;
}
```

现在只需把这个函数接到 `inner_product()` 的推广版本上，就得到了指数的值：

```
double dji_index =
    inner_product(dow_price.begin(), dow_price.end(),    // 所有公司
                  dow_weight.begin(),                    // 它们的权重
                  0.0,                                   // 初始值
                  plus<double>(),                        // 相加（和平时一样）
                  weighted_value);                       // 提取值并相乘
```

为什么有人愿意把这类数据保存在 map 而不是 vector 里？我们用 map 把不同值之间的对应关系显式表达了出来，这是常见的原因之一。另一个原因是 map 按键所定义的顺序存放元素：我们遍历 `dow` 时，输出的代码是按字母顺序排列的；如果改用 vector，我们就得自己排序。而使用 map 最常见的原因，其实是想根据键来查找值。对于大型序列，用 `find()` 查找比在有序结构（例如 map）中查找要慢得多。

试一试

试一试

让这个小例子跑起来。然后按自己的喜好加入几家公司，权重也由你自定。

21.6.4 unordered_map

要在 vector 中查找一个元素，`find()` 必须从头开始逐个检查元素，直到找到值正确的那个，或者一直查到末尾。平均而言，代价与 vector 的长度（N）成正比；我们把这种代价记作 $O(N)$。

要在 map 中查找一个元素，下标运算符必须从根开始，沿树逐个检查元素，直到找到值正确的那个，或者走到某个叶结点。平均而言，代价与树的深度成正比。一棵存放 N 个元素的平衡二叉树，最大深度为 $\log_2 N$；因此代价是 $O(\log_2 N)$。与 $O(N)$ 相比，$O(\log_2 N)$——即代价与 $\log_2 N$ 成正比——其实已经相当好了：

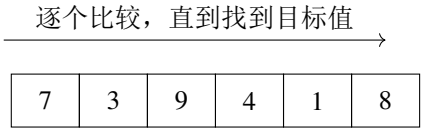
N	$\log_2 N$
15	4
128	7
1023	10
16,383	14

实际代价还取决于我们在查找中多快找到目标值，以及比较和迭代操作本身有多昂贵。追逐指针（map 的查找就是这样）通常比递增指针（`find()` 在 vector 中就是这样）要贵一些。

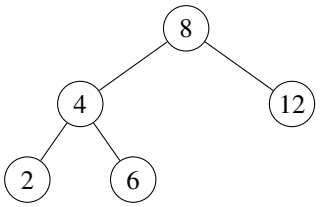
对于某些类型，特别是整数和字符串，我们还能做得比 map 的树查找更好。这里不展开细节，基本想法是：给定一个键，计算出一个指向 vector 的下标。这个下标称为散列值（hash value），使用这种技术的容器通常称为散列表（hash table）。可能的键的数目远大于散列表中的槽位数。例如，我们常用散列函数把数以十亿计的字符串映射到只有 1000 个元素的 vector 的下标上。这可能不太好办，但能够处理好，而且对实现大型 map 特别有用。散列表的主要优点在于：平均而言，一次查找的代价是（接近）常数，与表中元素的个数无关，即 $O(1)$。显然，对于大型 map（比如有 500,000 个网址的 map），这是一个明显的优势。关于散列查找的更多信息，可以查阅 `unordered_map` 的文档（网上有），或者任何一本数据结构基础教材（查找“散列表”和“散列”）。

我们可以用图来说明在（无序）vector、平衡二叉树和散列表中的查找：

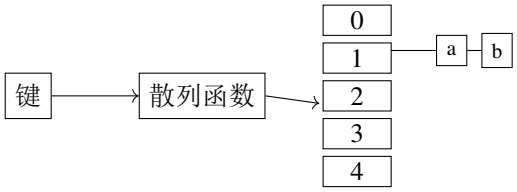
• 无序 vector 中的查找：



• map（平衡二叉树）中的查找：



• unordered_map（散列表）中的查找：



STL 的 `unordered_map` 是用散列表实现的，正如 STL 的 map 用平衡二叉树实现、STL 的 vector 用数组实现一样。STL 的部分价值就在于把这些存储和访问数据的方式，连同算法一起纳入一个共同的框架。经验法则是：

- 除非有充分理由，否则就用 vector。
- 如果需要根据值来查找（并且你的键类型有合理而高效的“小于”运算），就用 map。
- 如果需要在大型 map 中做大量查找，而且不需要有序遍历（并且能为键类型找到好的散列函数），就用 `unordered_map`。

这里我们不再详细描述 `unordered_map`。只要键的类型是 `string` 或 `int`，你就可以像用 map 一样使用 `unordered_map`，区别只在于：遍历元素时，元素不会按顺序排列。例如，我们可以把 §21.6.3 中道琼斯指数示例的一部分改写成这样：

```
unordered_map<string,double> dow_price;

for (const auto& p : dow_price) {
    const string& symbol = p.first;          // “股票代码”
    cout << symbol << '\t'
         << p.second << '\t'
         << dow_name[symbol] << '\n';
}
```

现在在 dow 中查找可能更快了。不过，这不会带来显著差别，因为这个指数里只有 30 家公司。假如我们保存的是纽约证券交易所所有公司的股价，也许就能看出性能差异。不过，我们会看到一个逻辑上的差异：遍历输出的结果不再按字母顺序排列。

从 C++ 标准的角度看，无序 map 是新生事物，还不完全是“一等公民”：它们定义在一份技术报告（Technical Report）里，而不是标准正文中。不过它们已经广泛可用；在不可用的地方，你常常能找到它们的前身，名字类似 hash_map。

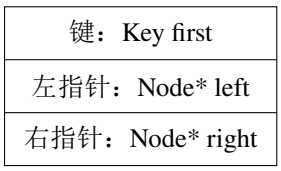
试一试

试一试

写一个小程序，使用 #include<unordered_map>。如果编译不过，说明你的 C++ 实现没有附带 unordered_map。如果你的 C++ 实现不提供 unordered_map，就得下载一份可用的实现（例如见 www.boost.org）。

21.6.5 set

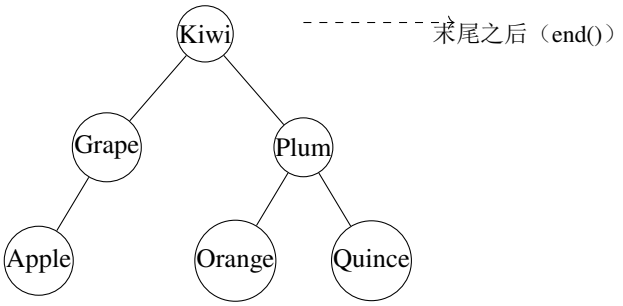
我们可以把 set 看成一种我们不关心其值的 map，或者更准确地说，是没有值的 map。set 的结点可以画成这样：



set 结点（Set node）：每个结点存放一个键，以及指向左、右子结点的两个指针。

我们可以把 map 示例（§21.6.2）中用到的水果集合表示成这样：

set 有什么用？碰巧有很多问题都要求我们记住某个值是否已经见过。记录哪些水果有货（与价格无关）就是一个例子；构造字典是另一个例子。另一种略有不同的用法是让 set 的元素成为“记录”，也就是说，元素是可能包含“大量”信息的对象——我们只是用其中一个成员作为键。例如：



水果集合（Fruits）：按字母顺序排列，依次为 Apple、Grape、Kiwi、Orange、Plum、Quince。

```
struct Fruit {
    string name;
    int count;
    double unit_price;
    Date last_sale_date;
    // . . .
};

struct Fruit_order {
    bool operator()(const Fruit& a, const Fruit& b) const
    {
        return a.name<b.name;
    }
};

set<Fruit, Fruit_order> inventory;    // 用 Fruit_order(x,y) 比较 Fruit
```

这里我们又一次看到，使用函数对象可以显著扩大 STL 组件所能处理的问题范围。

由于 set 没有值类型，它也不支持下表运算（`operator[]()`）。我们必须改用“列表操作”，例如 `insert()` 和 `erase()`。遗憾的是，map 和 set 也不支持 `push_back()`——原因很明显：新值插到哪里由 set 决定，而不是由程序员决定。请改用 `insert()`。例如：

```
inventory.insert(Fruit("quince",5));
inventory.insert(Fruit("apple",200,0.37));
```

与 map 相比，set 的一个优点是：可以直接使用从迭代器得到的值。因为 set 不像 map（§21.6.3）那样有（键，值）对，所以解引用运算符给出的是元素类型的值：

```
for (auto p = inventory.begin(); p!=inventory.end(); ++p)
    cout << *p << '\n';
```

当然，前提是你已经为 Fruit 定义了 <<。或者，我们也可以等价地写成

```
for (const auto& x : inventory)
    cout << x << '\n';
```

21.7 复制

在 §21.2 中，我们称 find() 是“最简单的有用算法”。自然，这一点是可以争论的。许多简单的算法都很有用——有些甚至写起来毫不费力。既然别人已经写好、并且替你调试过了，何必自己再写一遍呢？哪怕它再简单。论简单与实用，copy() 与 find() 不相上下。STL 提供了三个版本的 copy：

复制操作	
copy(b,e,b2)	把 [b:e) 复制到 [b2:b2+(e-b))
unique_copy(b,e,b2)	把 [b:e) 复制到 [b2:b2+(e-b))；不复制相邻的重复值
copy_if(b,e,b2,p)	把 [b:e) 复制到 [b2:b2+(e-b))，但只复制满足谓词 p 的元素

21.7.1 复制（copy）

基本的 copy 算法定义如下：

```
template<typename In, typename Out>
    // requires Input_iterator<In>() && Output_iterator<Out>()
Out copy(In first, In last, Out res)
{
    while (first!=last) {
        *res = *first;          // 复制元素
        ++res;
        ++first;
    }
    return res;
}
```

给定一对迭代器，copy() 把一段序列复制到另一段序列中，后者由指向其首元素的迭代器指定。例如：

```
void f(vector<double>& vd, list<int>& li)
    // 把 int 列表的元素复制到 double 的 vector 中
{
    if (vd.size() < li.size()) error("target container too small");
    copy(li.begin(), li.end(), vd.begin());
    // . . .
}
```

注意，copy() 的输入序列的类型可以与输出序列的类型不同。这正是 STL 算法一种很有用的通用性：它们适用于各种序列，而不对序列的实现做不必要的假设。我们没有忘记检查输出序列是否有足够的空间来容纳放进去的元素。检查这类大小是程序员的工作。STL 算法是为最大的通用性和最优的性能而编写的；它们（默认）不做范围检查，也不做其他可能代价高昂的保护性测试。有时你会希望它们做这些检查，但需要检查时，你可以像我们上面那样自己加上。

21.7.2 流迭代器

你一定听过“复制到输出”和“从输入复制”这样的说法。用这种方式来理解某些形式的 I/O 既自然又实用，而且我们真的可以用 copy() 来做这件事。

记住，序列是这样的东西：

- 有起点和终点
- 可以用 ++ 到达下一个元素
- 可以用 * 取得当前元素的值

我们可以很容易地用这种方式表示输入流和输出流。例如：

```
ostream_iterator<string> oo{cout};      // 给 *oo 赋值就是向 cout 写出

*oo = "Hello, ";                        // 意思是 cout << "Hello, "
++oo;                                   // 为下一次输出做好准备
*oo = "World!\n";                       // 意思是 cout << "World!\n"
```

你可以想象这该如何实现。标准库提供的 ostream_iterator 类型就是这样工作的；ostream_iterator<T> 是一个可以用来写出 T 类型值的迭代器。

类似地，标准库提供 istream_iterator<T> 类型，用于读入 T 类型的值：

```
istream_iterator<string> ii{cin};  // 读取 *ii 就是从 cin 读入一个 string

string s1 = *ii;                   // 意思是 cin>>s1
++ii;                              // 为下一次输入做好准备
string s2 = *ii;                   // 意思是 cin>>s2
```

有了 ostream_iterator 和 istream_iterator，我们就可以用 copy() 来完成 I/O。例如，我们可以像这样做一个“凑合能用”的字典程序：

```
int main()
{
    string from, to;
    cin >> from >> to;                  // 读取源文件名和目标文件名

    ifstream is {from};                 // 打开输入流
    ofstream os {to};                   // 打开输出流

    istream_iterator<string> ii {is};   // 为流建立输入迭代器
    istream_iterator<string> eos;       // 输入哨兵

    ostream_iterator<string> oo {os,"\n"};  // 为流建立输出迭代器

    vector<string> b {ii,eos};          // b 是用输入初始化的 vector
    sort(b.begin() ,b.end());           // 排序缓冲区
    copy(b.begin() ,b.end() ,oo);       // 把缓冲区复制到输出
}
```

迭代器 eos 是流迭代器对“输入结束”的表示。当 istream 到达输入末尾（通常称为 eof）时，它的 istream_iterator 将等于默认构造的 istream_iterator（这里名为 eos）。

注意，我们用一对迭代器初始化了 vector。作为容器的初始化器，一对迭代器 (a,b) 的意思是“把序列 [a:b) 读入容器”。当然，我们用的那对迭代器是 (ii,eos)——输入的开始和结束。这样我们就省去了显式使用 >> 和 push_back()。我们强烈建议不要采用下面这种替代写法：

```
vector<string> b(max_size);          // 不要猜测输入的数据量!
copy(ii,eos,b.begin());
```

试图猜测最大输入量的人，通常会发现自己猜少了，而由此产生的缓冲区溢出会给他们自己或他们的用户带来严重的问题。这类溢出也是安全问题的来源。

试一试

试一试

先让按原样写好的程序跑起来，并用一个小文件（比如只有几百个词的）测试它。然后试那个强烈不建议的版本——它要猜测输入量的大小，看看输入缓冲区 b 溢出时会发生什么。注意，最坏的情况是：溢出在你这个例子里并没有造成什么坏后果，于是你可能就想把它交付给用户了。

在我们这个小程序里，我们先把词读进来，然后再排序。当时这似乎是显然的做法，可是我们为什么要把词放到“错误的位置”上，然后再来排序呢？更糟的是，我们发现同一个词出现几次，就会把它存储并打印几次。

后一个问题可以通过用 unique_copy() 代替 copy() 来解决。unique_copy() 只是不复制相邻的相同值。例如，用普通的 copy() 时，程序会把

```
the man bit the dog
```

得到

```
bit
dog
man
the
the
```

如果改用 unique_copy()，程序会输出

```
bit
dog
man
the
```

这些换行是从哪里来的？用分隔符输出太常见了，所以 ostream_iterator 的构造函数允许你（可选地）指定一个字符串，在每输出一个值之后打印：

```
ostream_iterator<string> oo {os,"\n"};      // 为流建立输出迭代器
```

显然，对于给人读的输出，换行是常用的选择，但也许我们更喜欢用空格作分隔符？可以这样写

```
ostream_iterator<string> oo {os," "};      // 为流建立输出迭代器
```

这样得到的输出就是

```
bit dog man the
```

21.7.3 用 set 保持有序

还有一种更简单的办法得到这个输出：用 set 而不是 vector：

```
int main()
{
    string from, to;
    cin >> from >> to;                  // 读取源文件名和目标文件名

    ifstream is {from};                 // 建立输入流
    ofstream os {to};                   // 建立输出流

    set<string> b {istream_iterator<string>{is}, istream_iterator<string>{}};

    copy(b.begin(),b.end(), ostream_iterator<string>{os," "});   // 把缓冲区复制到输出
}
```

当我们把值插入 set 时，重复值会被忽略。此外，set 的元素保持有序，所以不需要排序。有了合适的工具，大多数任务都很容易。

21.7.4 copy_if

copy() 算法无条件地复制。unique_copy() 算法抑制相邻的相同元素。第三个 copy 算法只复制谓词为真的元素：

```
template<typename In, typename Out, typename Pred>
    // requires Input_iterator<In>() && Output_operator<Out>() &&
    // Predicate<Pred, Value_type<In>>()
Out copy_if(In first, In last, Out res, Pred p)
    // 复制满足谓词的元素
{
    while (first!=last) {
        if (p(*first)) *res++ = *first;
        ++first;
    }
    return res;
}
```

利用 §21.4 中的 Larger_than 函数对象，我们可以这样找出序列中所有大于 6 的元素：

```
void f(const vector<int>& v)
    // 复制所有值大于 6 的元素
{
    vector<int> v2(v.size());
    copy_if(v.begin(), v.end(), v2.begin(), Larger_than(6));
    // . . .
}
```

由于我的一个失误，这个算法在 1998 年的 ISO 标准中缺失了。这个错误现在已经得到纠正，但你仍然可能遇到没有 copy_if 的实现。如果真是这样，直接用本节给出的定义即可。

21.8 排序与查找

我们常常希望数据是有序的。这可以通过使用一种维护顺序的数据结构（例如 map 和 set）来实现，也可以通过排序来实现。STL 中最常用、最有用的排序操作，就是我们前面已经用过多次的 sort()。默认情况下，sort() 用 < 作为排序准则，但我们也可以提供自己的准则：

```
template<typename Ran>
    // requires Random_access_iterator<Ran>()
void sort(Ran first, Ran last);

template<typename Ran, typename Cmp>
    // requires Random_access_iterator<Ran>()
    // && Less_than_comparable<Cmp,Value_type<Ran>>()
void sort(Ran first, Ran last, Cmp cmp);
```

作为一个按用户指定的准则来排序的例子，我们来看看如何在不考虑大小写的情况下对字符串排序：

```
struct No_case {          // lowercase(x) < lowercase(y) 吗？
    bool operator()(const string& x, const string& y) const
    {
        for (int i = 0; i<x.length(); ++i) {
            if (i == y.length()) return false;       // y<x
            char xx = tolower(x[i]);
            char yy = tolower(y[i]);
            if (xx<yy) return true;                  // x<y
            if (yy<xx) return false;                 // y<x
        }
        if (x.length()==y.length()) return false;    // x==y
        return true;                                 // x<y (x 的字符更少)
    }
};
```

```
void sort_and_print(vector<string>& vc)
{
    sort(vc.begin(),vc.end(),No_case());

    for (const auto& s : vc)
        cout << s << '\n';
}
```

序列排好序之后，我们就不必再用 find() 从头查找了；我们可以利用这种有序性来做二分查找。二分查找大致是这样工作的：

假设我们要找的值是 x；先看中间的那个元素：

- 如果该元素的值等于 x，那就找到了！
- 如果该元素的值小于 x，那么值为 x 的任何元素必定在右边，于是我们去看右半部分（对那一半做二分查找）。
- 如果 x 的值小于该元素的值，那么值为 x 的任何元素必定在左边，于是我们去看左半部分（对那一半做二分查找）。
- 如果我们一直向左或向右走，到了最后一个元素仍未找到 x，那么就不存在具有该值的元素。

对于较长的序列，二分查找比 find()（线性查找）快得多。标准库中用于二分查找的算法是 binary_search() 和 equal_range()。我们说的“较长”是什么意思呢？这要看具体情况，但通常有十个元素，binary_search() 就已经比 find() 有优势了。对于一个 1000 个元素的序列，binary_search() 大约会比 find() 快 200 倍，因为它的代价是 $O(\log_2 N)$；参见 §21.6.4。

binary_search 算法有两个变体：

```
template<typename Ran, typename T>
bool binary_search(Ran first, Ran last, const T& val);

template<typename Ran, typename T, typename Cmp>
bool binary_search(Ran first, Ran last, const T& val, Cmp cmp);
```

这些算法要求并假定输入序列已经排好序。如果没有排序，就可能发生一些“有趣的事情”，比如死循环。binary_search() 只是告诉我们某个值是否存在：

```
void f(vector<string>& vs)          // vs 已经排好序
{
    if (binary_search(vs.begin(),vs.end(),"starfruit")) {
        // 找到了 starfruit
    }
    // . . .
}
```

因此，当我们只关心某个值是否在序列中时，binary_search() 是理想的选择。如果我们关心的是找到的是哪个元素，可以使用 lower_bound()、upper_bound() 或 equal_range()（§B.5.4、§23.4）。在关心找到哪个元素的情况下，原因通常在于：该元素是一个包含的信息比键更多的对象；可能存在多个具有相同键的元素；或者我们想知道是哪个元素满足了查找准则。

21.9 容器算法

所以我们用迭代器指定的元素序列来定义标准库算法。一个输入序列定义为一对迭代器 [b:e)，其中 b 指向序列的第一个元素，e 指向序列末尾之后的位置（§20.3）。输出序列则更简单，只用一个指向其首元素的迭代器来表示。例如：

```
void test(vector<int> & v)
{
    sort(v.begin(),v.end());   // 对 v 中从 v.begin() 到 v.end() 的元素排序
}
```

这样很好，也很通用。例如，我们可以只对半个 vector 排序：

```
void test(vector<int> & v)
{
    sort(v.begin(),v.begin()+v.size());     // 对 v 的前半部分元素排序
    sort(v.begin()+v.size(),v.end());       // 对 v 的后半部分元素排序
}
```

不过，指定元素范围有点罗嗦，而大多数时候我们是对整个 vector 排序，而不是只排一半。所以大多数时候，我们希望这样写：

```
void test(vector<int> & v)
{
    sort(v);   // 对 v 排序
}
```

标准库并没有提供 sort() 的这个变体，但我们可以自己定义：

```
template<typename C>  // 要求满足 Container<C>()
void sort(C& c)
{
    std::sort(c.begin(),c.end());
}
```

事实上，我们觉得它太有用了，于是把它加进了 `std_lib_facilities.h`。

输入序列这样处理很方便，但为了保持简单，我们倾向于把返回类型都留给迭代器。例如：

```
template<typename C, typename V>   // 要求满足 Container<C>()
Iterator<C> find(C& c, Val v)
{
    return std::find(c.begin(),c.end(),v);
}
```

自然，`Iterator<C>` 就是 C 的迭代器类型。

小试身手

每次操作（也就是本练习中的每一行所定义的）之后，都把该 vector 打印出来。

1. 定义一个结构体 `struct Item { string name; int iid; double value; /* . . . */ };`，创建一个 `vector<Item>` 类型的 vi，并用文件中的十个条目填充它。
2. 按 name 对 vi 排序。
3. 按 iid 对 vi 排序。
4. 按 value 对 vi 排序；按 value 递减的顺序（即最大值在前）打印它。
5. 插入 `Item("horse shoe",99,12.34)` 和 `Item("Canon S400", 9988,499.95)`。
6. 按 name 从 vi 中删除（erase）两个 Item。
7. 按 iid 从 vi 中删除（erase）两个 Item。
8. 用 `list<Item>` 而不是 `vector<Item>` 重做本练习。

现在再试试 map：

1. 定义一个 `map<string,int>`，名为 msi。
2. 向其中插入十个 (name,value) 对，例如 `msi["lecture"]=21`。
3. 用你选择的某种格式把 (name,value) 对输出到 cout。
4. 从 msi 中删除（erase）这些 (name,value) 对。
5. 写一个函数，从 cin 读入 value 对并把它们放进 msi。
6. 从输入读入十个对，填入 msi。
7. 把 msi 的元素输出到 cout。
8. 输出 msi 中（整数）值的总和。
9. 定义一个 `map<int,string>`，名为 mis。
10. 把 msi 中的值填入 mis；也就是说，如果 msi 中有元素 (”lecture”,21)，mis 中就应该有元素 (21,”lecture”)。
11. 把 mis 的元素输出到 cout。

更多 vector 用法：

1. 从文件中读入若干浮点值（至少 16 个）到名为 vd 的 `vector<double>` 中。
2. 把 vd 输出到 cout。
3. 创建一个 `vector<int>` 类型的 vi，使其元素个数与 vd 相同；把 vd 中的元素复制到 vi。
4. 把 (vd[i],vi[i]) 对输出到 cout，每行一对。
5. 输出 vd 中元素的总和。
6. 输出 vd 元素之和与 vi 元素之和的差。
7. 标准库中有一个名为 reverse 的算法，它以序列（一对迭代器）为参数；反转 vd，并把 vd 输出到 cout。
8. 计算 vd 中元素的平均值并输出。
9. 新建一个名为 vd2 的 `vector<double>`，把 vd 中所有小于（低于）平均值的元素复制到 vd2。
10. 对 vd 排序，然后再次输出。

要点回顾

1. 有哪些有用的 STL 算法实例？
2. `find()` 做什么？至少给出五个例子。
3. `count_if()` 做什么？
4. `sort(b,e)` 用什么作为排序准则？
5. STL 算法如何以一个容器作为输入参数？

6. STL 算法如何以一个容器作为输出参数？
7. STL 算法通常如何表示“未找到”或“失败”？
8. 什么是函数对象？
9. 函数对象与函数在哪些方面不同？
10. 什么是谓词？
11. `accumulate()` 做什么？
12. `inner_product()` 做什么？
13. 什么是关联容器？至少给出三个例子。
14. list 是关联容器吗？为什么不是？
15. 二叉树的基本排序性质是什么？
16. 一棵树是“平衡的”，这大致是什么意思？
17. map 的每个元素占多少空间？
18. vector 的每个元素占多少空间？
19. 既然有（有序的）map，为什么还会有人用 `unordered_map`？
20. set 与 map 有何不同？
21. multimap 与 map 有何不同？
22. 既然我们完全可以“就写一个简单的循环”，为什么还要用 `copy()` 算法？
23. 什么是二分查找？

术语

- `accumulate()`
- 算法（`algorithm`）
- `application()`
- 关联容器（associative container）
- 平衡树（balanced tree）
- `binary_search()`
- `copy()`
- `copy_if()`
- `equal_range()`
- `find()`
- `find_if()`
- 函数对象（function object）
- 泛型（generic）
- 哈希函数（hash function）
- `inner_product()`
- lambda
- `lower_bound()`
- `map`
- 谓词（predicate）
- 查找（searching）
- 序列（sequence）

- set
- sort()
- 排序（sorting）
- 流迭代器（stream iterator）
- unique_copy()
- unordered_map
- upper_bound()

习题

1. 通读本章，把其中所有你还没做过的“试一试”练习都做一遍。
2. 找一个可靠的 STL 文档来源，列出每一个标准库算法。
3. 自己实现 count()。测试它。
4. 自己实现 count_if()。测试它。
5. 如果我们不能用返回 end() 来表示“未找到”，那么我们该怎么办？重新设计并重新实现 find() 和 count()，让它们接受分别指向首元素和末元素的迭代器。把结果与标准版本作比较。
6. 在 §21.6.5 的 Fruit 例子中，我们把 Fruit 复制进了 set。如果我们不想复制 Fruit 呢？可以改用 set<Fruit*>。不过要这样做，就必须为该 set 定义一个比较操作。请用 set<Fruit*, Fruit_comparison> 实现 Fruit 例子，并讨论两种实现之间的差异。
7. 为 vector<int> 写一个二分查找函数（不使用标准库中的那个）。接口由你选择。测试它。你对自己的二分查找函数的正确性有多大信心？再为 list<string> 写一个二分查找函数，测试它。这两个二分查找函数彼此有多相似？如果你事先不知道 STL，你认为它们会有多相似？
8. 取 §21.6.1 的词频例子，修改它，使其按出现频率（而不是字典序）输出各行。例如输出行应当是 3: C++ 而不是 C++: 3。
9. 定义一个 Order 类，包含（客户）姓名、地址、数据和 vector<Purchase> 成员。Purchase 是一个类，包含（产品）名称、unit_price 和 count 成员。定义一个把 Order 从文件读入、写出到文件的机制，再定义一个打印 Order 的机制。创建一个至少含十个 Order 的文件，把它读入 vector<Order>，按（客户）姓名排序，再写回文件。另建一个至少含十个 Order 的文件，其中约三分之一与第一个文件中的相同，把它读入 list<Order>，按（客户）地址排序，再写回文件。用 std::merge() 把两个文件合并成第三个。
10. 计算上一个练习中两个文件里订单的总价值。单个 Purchase 的价值（当然）是它的 unit_price*count。
11. 提供一个图形用户界面，用于把 Order 输入到文件中。
12. 提供一个图形用户界面，用于查询 Order 文件；例如“查找来自 Joe 的所有订单”、“查找 Hardware 文件中订单的总价值”、“列出 Clothing 文件中的所有订单”。提示：先设计非图形用户界面，再在它之上构建图形用户界面。
13. 写一个程序，把文本文件“清理”干净，以便用于单词查询程序；也就是说，把标点替换成空白，把单词转成小写，把 don't 替换成 do not（等等），并去掉复数形式（例如 ships 变成 ship）。不要野心太大。例如，一般很难判断复数，所以如果同时找到 ship 和 ships，就只去掉末尾的 s。用这个程序处理一个真实的文本文件，其中至少要有 5000 个单词（例如一篇研究论文）。

14. 写一个程序（使用上一个练习的输出），回答这样的问题：“文件中 ship 出现了多少次？”“哪个单词出现得最频繁？”“文件中哪个单词最长？”“哪个最短？”“列出所有以 s 开头的单词。”“列出所有四个字母的单词。”
15. 为上一个练习中的程序提供一个图形用户界面。

后记

STL 是 ISO C++ 标准库中负责容器与算法的那一部分。因此它提供了非常通用、灵活而有用的基础工具。它可以为我们省下大量工作：重新发明轮子也许有趣，但很少有什么产出。除非有强有力的理由不这样做，否则请使用 STL 的容器和基本算法。更重要的是，STL 是泛型编程的一个范例，它展示了具体的问题和具体的解决方案如何催生出一套强大而通用的工具。如果你需要处理数据——大多数程序员都需要——STL 提供了一个范例、一组思想和一种常常有用的方法。

第四部分

拓宽视野

22 理想与历史

"When someone says, 'I want a programming language in which I need only say what I wish done,' give him a lollipop."

（当有人说：'我想要一种编程语言，在它里面我只需说出我想做什么'，那就给他一根棒棒糖吧。）

——艾伦·佩利斯（Alan Perlis）

本章极为简略、也极具选择性地讲述编程语言的历史，以及这些语言被设计来服务的那些理想。理想以及表达理想的语言，正是职业精神的基础。由于本书使用 C++，我们把重点放在 C++ 以及影响了 C++ 的语言上，目的是为本书所呈现的思想提供背景与视角。对于每一种语言，我们都会介绍它的设计者：语言并不只是抽象的创造物，而是个人为应对当时面临的问题而设计的具体解决方案。

22.1 历史、理想与职业精神

"历史是废话，"亨利·福特有句名言。相反的观点自古就被广泛引用："不了解历史的人注定要重蹈覆辙。"问题在于选择哪些历史该了解、哪些该丢弃："95% 的东西都是废话"是另一句相关的引语（我们赞同这一说法，尽管 95% 恐怕还是低估）。我们对历史与当前实践之关系的看法是：对历史缺乏一定理解，就不可能有职业精神。如果你对自己领域的背景了解太少，就容易轻信，因为任何领域的历史都散落着看似合理却行不通的想法。历史真正的"养分"，是那些在实践中证明了自己价值的想法与理想。

我们很想多谈谈更多语言以及各类软件（操作系统、数据库、图形学、网络、Web、脚本等）中关键思想的起源，但这些重要而又有用的软件与编程领域，只能请你在别处去寻找了。本书的篇幅只够让我们对编程语言的理想与历史做一点浮光掠影的探讨。

编程的终极目标始终是产出有用的系统。在关于编程技术与编程语言的激烈讨论中，这一点很容易被忘记。请不要忘记！如果需要提醒，不妨再翻回第 1 章看看。

22.1 历史、理想与职业精神
- **22.1.1** 编程语言的目标与哲学
- **22.1.2** 编程理想
- **22.1.3** 风格与范式

22.2 编程语言历史概述
- **22.2.1** 最早的语言
- **22.2.2** 现代语言的根源
- **22.2.3** Algol 家族
- **22.2.4** Simula
- **22.2.5** C
- **22.2.6** C++
- **22.2.7** 今天
- **22.2.8** 信息来源

22.1.1 编程语言的目标与哲学

什么是编程语言？编程语言应当为我们做什么？对“什么是编程语言”这个问题，常见的回答有以下几种：

- 指示机器的工具
- 描述算法的记号
- 程序员之间相互沟通的手段
- 做实验的工具
- 控制计算机化设备的手段
- 表达概念之间关系的方式
- 表达高层设计的手段

我们的回答是：“以上全是——而且还更多！”显然，我们这里想的是通用编程语言，本章通篇也是如此。此外还有专用语言和领域特定语言，它们服务于范围更窄、通常也定义得更精确的目标。

我们希望一种编程语言具有哪些好的性质呢？

- 可移植性
- 类型安全
- 有精确定义
- 高性能
- 能简洁地表达想法
- 一切便于调试的设施
- 一切便于测试的设施
- 能访问所有系统资源
- 平台无关性
- 能在所有平台上运行（如 Linux、Windows、智能手机、嵌入式系统）
- 几十年保持稳定
- 能对应用领域的变化迅速做出改进
- 易于学习
- 小巧
- 支持流行的编程风格（如面向对象编程和泛型编程）
- 一切有助于程序分析的东西
- 大量设施
- 有庞大社区的支持
- 对新手（学生、初学者）友好
- 为专家（如基础设施构建者）提供全面的设施
- 有大量可用的软件开发工具
- 有大量可用的软件组件（如库）
- 有开放的软件社区支持
- 得到主要平台厂商（Microsoft、IBM 等）的支持

遗憾的是，我们无法同时拥有这一切。这很令人遗憾，因为这些“性质”中的每一项客观上都很好：每一项都带来好处，而一种不提供它们的语言，会给用户增添额外的工作与麻烦。我们无法全部拥有的原因同样根本：其中若干性质彼此排斥。例如，你不可能既百分之百平台无关，又访问所有系

统资源；一个程序若访问了并非每个平台都有的资源，它就无法到处运行。同样，我们显然希望一种语言（以及使用它所需的工具和库）既小巧又易学，但只要还要为各种系统上的编程、为各种应用领域提供全面支持，这一点就无法做到。

理想正是在这里变得重要。理想指导着每一种语言、每一个库、每一件工具和每一位程序设计者都必须做出的技术选择与权衡。是的，当你编写程序时，你就是设计者，必须做出设计上的选择。

22.1.2 编程理想

《C++ 程序设计语言》的序言开头写道：“C++ 是一种通用编程语言，其设计目的是让严肃的程序员编程时更愉快。”什么？编程不就是为了交付产品吗？不是为了正确性、质量和可维护性吗？不是为了上市时间吗？不是为了效率吗？不是为了支持软件工程吗？当然，这些也都是，但我们不该忘记程序员。再看一个例子：Don Knuth 说过：“Alto 最好的地方在于，它在夜里不会跑得更快。”Alto 是施乐帕洛阿尔托研究中心（PARC）的一台计算机，是最早的“个人计算机”之一，与之相对的是供多人共享、白天要排队争抢使用机会的计算机。

我们用来编程的工具和技术，是为了让程序员——一个人——工作得更好、产出更好的结果。请不要忘记这一点。那么，我们能提出哪些准则，帮助程序员以最小的痛苦做出最好的软件呢？本书通篇都把我们的理想讲得很明确，所以这一节基本上只是一个总结。

我们之所以希望代码有良好的结构，主要原因是：正是结构让我们能够不必花费过多力气就做出修改。结构越好，改动就越容易：发现并修复错误、增加新特性、移植到新架构、让它运行得更快等等，莫不如此。这正是我们所说的“好”的含义。

本节余下的部分，我们将：

- 重新审视我们想要达到的目标，也就是我们希望自己的代码做到什么
- 介绍软件开发中的两种总体方法，并说明把两者结合起来胜过其中任何一种单独使用
- 考察在代码中体现出来的程序结构的几个关键方面：
  - 直接表达想法
  - 抽象层次
  - 模块化
  - 一致性与极简

理想是用来用的。它们是思考的工具，而不只是拿去取悦管理者和审查者的漂亮话。我们的程序应当去逼近理想。当我们在程序中卡住时，不妨退一步看看：问题是否源于偏离了某个理想；有时这会有所帮助。当我们评价一个程序时（最好在交付给用户之前），我们要找出那些将来可能引发问题的、对理想的偏离。尽可能广泛地运用理想，但要记住：实际的考虑（如性能和简单性）以及语言本身的弱点（没有哪种语言是完美的），常常使你只能做到对理想的一个良好近似。

理想可以在做具体技术决策时给我们指导。例如，我们不能孤立地、一个接一个地决定某个库的每个接口（§14.1）。那样做的结果会一团糟。相反，我们必须回到自己的基本原则，确定这个特定的库中什么才是重要的，然后产出一套一致的接口。理想情况下，我们还应当在文档和代码注释中阐明这个特定设计的设计原则与所做的权衡。

在项目开始时，回顾一下这些理想，看看它们与问题以及解决问题的早期想法有何关联。这可以是获得想法、打磨想法的好办法。到了设计与开发过程的后期，当你卡住时，退一步看看你的代码在哪些地方偏离理想最远——错误最可能潜伏在那里，设计问题也最可能出现在那里。这是对默认做法

的一种替代：反复盯着同一个地方，用同样的手法去找错误。“错误总是在你没看的地方——否则你早就找到它了。”

22.1.2.1 我们想要什么

一般来说，我们希望：

- 正确性：是的，要定义我们所说的“正确”是什么意思可能很难，但把这件事定义清楚，是完整工作的重要部分。通常，别人会替我们定义某个项目什么算正确，但接下来我们还得解释他们说的话。
- 可维护性：每一个成功的程序都会随时间而改变：它会被移植到新的硬件和软件平台，会被添加新设施而扩展，还会发现必须修复的新错误。下面关于程序结构的理想那几节讲的就是这个理想。
- 性能：性能（“效率”）是一个相对的说法，性能必须满足程序用途的需要。人们常说，高效的代码必然是低层的，而对良好高层代码结构的关注会导致低效。恰恰相反，我们发现，可接受的性能常常正是通过遵循我们所推荐的理想和方法获得的。STL 就是一个既抽象又非常高效的代码例子。性能不佳既可能来自对低层细节的痴迷，也可能来自对这些细节的轻视。
- 按时交付：把一个完美的程序晚一年交付，通常是不够好的。显然，人们期待的是不可能的东西，但我们确实需要在合理的时间内交付高质量的软件。有一种神话认为“按时完成”就意味着粗糙。恰恰相反，我们发现，重视良好的结构（如资源管理、不变式和接口设计）、面向可测试性的设计，以及使用合适的库（往往是为某个具体应用或应用领域设计的），是赶上截止日期的好办法。

这就引出了对代码结构的关注：

- 如果程序中存在错误（每个大程序都有错误），在结构清晰的程序中更容易找到它。
- 如果程序需要由新人来理解，或者需要以某种方式修改，清晰的结构远比一堆低层细节的乱麻容易理解，花费的力气也小得多。
- 如果程序遇到性能问题，调优一个高层程序（一个很好地逼近理想、结构定义良好的程序）往往比调优一个低层或混乱的程序更容易。首先，高层程序更可能被人理解；其次，高层程序常常远在低层程序之前就可以开始测试和调优。

请注意关于程序可理解性的那一点。任何有助于我们理解程序、有助于我们对它进行推理的东西都是好的。从根本上说，规则性胜于不规则性——只要这种规则性不是靠过度简化换来的。

22.1.2.2 总体方法

编写正确的软件有两种方法：

- 自底向上：只用已证明正确的组件来组合系统。
- 自顶向下：用假定含有错误的组件来组合系统，并捕获所有错误。

有意思的是，最可靠的系统会把这两种看似相反的方法结合起来。原因很简单：对于一个大型的现实系统，两种方法中的任何一种都无法提供所需的正确性、适应性和可维护性：

- 我们无法构建并“证明”足够多的基本组件，来消除所有错误来源。
- 在把它们组合成最终系统时，我们也无法完全补偿有错误的基本组件（库、子系统、类层次结构等）的缺陷。

然而，把两种方法的近似结合起来，能比任何单独一种提供更多：我们可以产出（或借用、购买）足够好的组件，使遗留的问题能靠错误处理和系统的测试来弥补。同时，如果我们不断构建更好的组件，系统中就会有更大的一部分可以用它们来构造，从而减少所需的“混乱的临时代码”。

测试是软件开发中不可缺少的一部分，第 26 章有较详细的讨论。测试就是对错误的系统化查找。“尽早测试、经常测试”是一句流行的口号。我们设法把程序设计得便于测试，并让错误更难“躲”在混乱的代码里。

22.1.2.3 直接表达想法

当我们表达某件事时——无论它是高层的还是低层的——理想的做法是在代码中直接表达它，而不是绕弯子。用代码直接表示我们的想法这一根本理想，有几个具体的变体：

- 在代码中直接表示想法。例如，把一个参数表示成具体的类型（如 `Month` 或 `Color`）比表示成更一般的类型（如 `int`）更好。
- 在代码中独立地表示相互独立的想法。例如，除少数例外，标准 `sort()` 能对任意标准容器中的任意元素类型排序……

……元素类型；排序、排序准则、容器和元素类型这几个概念是彼此独立的。假如我们造出的是一个“元素分配在自由存储上、元素类型派生自 `Object`、并且为 `vector::sort()` 定义了 `before()` 成员函数的对象 `vector`”，那么得到的 `sort()` 就会比现在不通用得多，因为我们对存储方式、类层次、可用的成员函数、排序方式等都作了假设。

- **在代码中直接表达想法之间的关系。**能够直接表达出来的最常见关系是继承（例如 `Circle` 是一种 `Shape`）和参数化（例如 `vector<T>` 表达的是所有 `vector` 的共同之处，与具体的元素类型无关）。
- **只在组合确实有意义的地方自由组合用代码表达的想法。**例如，`sort()` 允许我们使用多种元素类型和多种容器，但元素必须支持 `<` 运算（若不支持，就要用带一个额外参数的 `sort()` 来指定比较准则），而且被排序的容器必须支持随机访问迭代器。
- **用简单的方式表达简单的想法。**一味遵循上面这些理想，可能写出过度通用的代码。例如，我们可能得到分类学（继承结构）比任何人需要的都复杂的类层次，或者给每个（看似）简单的类都加上七个参数。为了避免每个用户都必须面对所有可能的复杂情况，我们尽量提供处理最常见或最重要情形的简单版本。例如，除了用 `op` 排序的通用版本 `sort(b,e,op)` 之外，我们还有隐式地用“小于”排序的 `sort(b,e)`；还可以提供 `sort(c)`（用“小于”排序标准容器）和 `sort(c,op)`（用 `op` 排序标准容器）。

22.1.2.4 抽象层次

我们倾向于在可行的最高抽象层次上工作，也就是说，我们的理想是把解决方案表达得尽可能通用。

例如，考虑怎样表示电话簿中的条目（就像我们可能存放在 PDA 或手机里的那样）。我们可以把一组（名字，值）对表示成 `vector<pair<string,Value_type>>`。不过，如果我们基本上总是用名字来访问这个集合，那么 `map<string,Value_type>` 就是更高一层的抽象，省掉了编写（和调试）访问函数的麻烦。反过来说，`vector<pair<string,Value_type>>` 本身又比两个数组 `string[max]` 和 `Value_type[max]` 的抽象层次更高——在后一种表示里，字符串与它的值之间的关系是隐含的。最低的抽象层次大概是“一个 `int`（元素个数）加两个 `void*`（指向某种表示形式，程序员知道它是什么，编译器不知道）”。在我们这个例子里，前面每一种建议都可以看作层次太低，因为它们关注的是值对的表示，而不是它的功能。我们可以通过定义一个直接反映某种用途的类来更靠近应用。例如，我们可以用一个 `Phonebook` 类来写应用代码，它的接口是为方便使用而设计的；而 `Phonebook` 类的实现可以采用上面任何一种表示。

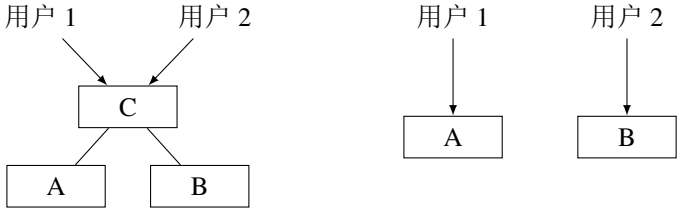
之所以偏好更高的抽象层次（前提是我们有合适的抽象机制，而且语言能以可接受的效率支持它），是因为这样的表述比在计算机硬件层面上表达出来的解法更接近我们思考和描述问题与解法的方式。

人们给出的降低抽象层次的理由通常是“效率”。只有确实需要时才应这样做（§25.2.2）。使用更低层（更原始）的语言特性并不一定带来更好的性能，有时反而会消除优化机会。例如，使用 `Phonebook` 类时，我们可以在多种实现之间选择，比如在 `string[max]` 加 `Value_type[max]` 与 `map<string,Value_type>` 之间选择：对某些应用前者更高效，对另一些应用后者更高效。当然，如果应用只涉及你的私人通讯录，性能不会是主要顾虑；但当我们必须跟踪并处理几百万条条目时，这种取舍就变得很有意思了。更严重的是，用了一段时间之后，低层特性的使用会耗掉程序员的时间，以致因为时间不够而错过（性能或其他方面的）改进机会。

22.1.2.5　模块化

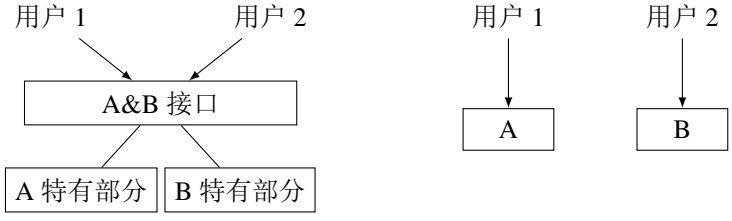
模块化是一种理想。我们希望用能够独立构建、独立理解和独立测试的“组件”（函数、类、类层次、库等）来组成系统。理想情况下，我们还希望这样设计和实现组件，使它们能在不止一个程序中使用（“复用”）。复用是指用别处已经用过、经过测试的组件来构建系统，也指这类组件的设计和使用。我们在讨论类、类层次、接口设计和泛型编程时已经涉及这一点。我们在“编程风格”（§22.1.3）中讲的内容，很多都与潜在的“可复用”组件的设计、实现和使用有关。请注意，并非每个组件都能用于不止一个程序；有些代码就是太专门了，不容易改造成可复用的。

代码中的模块化应当反映应用中重要的逻辑区分。仅仅把两个完全独立的类 A 和 B 塞进一个叫 C 的“可复用组件”里，并不能“提高复用”。由于 C 提供了 A 与 B 接口的并集，引入 C 反而使我们的代码更复杂了：



在这里，用户 1 和用户 2 都使用 C。除非你深入看一下 C 的内部，否则你可能以为用户 1 和用户 2 从共享一个流行组件中得到了好处。人们会（在这个例子里是错误地）假定共享（“复用”）带来的好处包括测试更充分、代码总量更少、用户群更大，等等。遗憾的是，除了有点过于简化之外，这种现象并不特别罕见。

那什么办法会有帮助呢？也许可以为 A 和 B 提供一个公共接口：



这两幅图分别想表达继承和参数化。在两种情况下，提供的接口都必须比 A 和 B 接口的简单并集更小，这样做才值得。换句话说，A 和 B 必须有根本的共同点，用户才能从中受益。请注意，我们又一次回到了接口（§9.7、§25.4.2），并由此涉及不变量（§9.4.3）。

22.1.2.6　一致性与极简主义

一致性和极简主义主要是表达想法时的理想，因此我们可能把它们当成只是外观问题而不予理会。然而，把混乱的设计优雅地呈现出来真的很难，所以对一致性和极简主义的要求可以用作设计准则，影响到程序最细微的细节：

- 如果你对某个特性的用处有疑问，就不要添加它。
- 确实要给相似的功能相似的接口（和名字），但只有在相似性是根本性的时候才这样做。
- 确实要给不同的功能不同的名字（可能还有不同的接口风格），但只有在差异是根本性的时候才这样做。

一致的命名、接口风格和实现风格有助于维护。代码一致时，新来的程序员不必为大型系统的每个部分学习一套新的约定。STL 就是一个例子（第 20 21 章、§B.4 6）。当这种一致性无法做到时（例如对于年代久远的代码或用其他语言写的代码），可以考虑提供一个与程序其余部分风格一致的接口。否则，那种外来（“奇怪”“糟糕”）的风格就会传染到程序中每一个需要访问这段碍眼代码的部分。

保持极简主义和一致性的一种办法是仔细（且一致地）为每个接口编写文档。这样，不一致和重复更容易被发现。记录前置条件、后置条件和不变量尤其有用，认真关注资源管理和错误报告也同样有用。一致的错误处理和资源管理策略对简单性至关重要（§19.5）。

对一些程序员来说，关键的设计原则是 KISS（“Keep It Simple, Stupid”，保持简单，笨蛋）。我们甚至听人声称 KISS 是唯一值得遵循的设计原则。不过，我们更喜欢不那么耸动的说法，比如“让简单的事情保持简单”，以及“保持简单：尽可能简单，但不能更简单”。后一句引自阿尔伯特 • 爱因斯坦，它说明简化有可能过头——超过某个界限就不再合理，反而损害设计。显而易见的问题是：“对谁简单？跟什么相比简单？”

22.1.3　风格/范式

设计和实现程序时，我们力求风格一致。C++ 支持四种可以视为根本的风格：

- 过程式编程
- 数据抽象
- 面向对象编程
- 泛型编程

这些风格有时（有点浮夸地）被称为“编程范式”。此外还有许多“范式”，例如函数式编程、逻辑编程、基于规则的编程、基于约束的编程和面向方面的编程。但 C++ 并不直接支持它们，而且一本入门书也不可能面面俱到，所以我们把这些留给“今后的工作”，连同我们必须省略的、关于确实讲到的那些范式/风格的大量细节：

- **过程式编程**：把程序组织成对参数进行操作的函数的想法。例子是数学函数库，如 `sqrt()` 和 `cos()`。C++ 通过函数的概念支持这种编程风格（第 8 章）。能够选择按值、按引用和按 const 引用传递参数，这一点是最有价值的。数据常常组织成用 `struct` 表示的数据结构。这里不使用显式的抽象机制（例如类的私有数据成员或成员函数）。请注意，这种编程风格——以及函数——是其他每一种风格不可分割的组成部分。
- **数据抽象**：先为一类应用领域提供一组合适的类型，然后用这些类型编写程序的想法。矩阵是一个经典的例子（§24.3 6）。显式的数据隐藏（例如使用类的私有数据成员）被大量使用。标准库的 `string` 和 `vector` 是常见的例子，它们显示了数据抽象与泛型编程所使用的参数化之间的紧密关系。之所以叫“抽象”，是因为类型是通过接口使用的，而不是直接访问它的实现。

- **面向对象编程**：把类型组织成层次结构，以便在代码中直接表达它们之间关系的想法。经典的例子是第 14 章的 Shape 层次。当这些类型确实具有根本的层次关系时，这显然很有价值。然而，一直存在严重的过度使用倾向，也就是说，人们为那些并非出于根本原因而属于一起的类型建立层次结构。当人们要派生时，先问为什么：想表达什么？基类与派生类的区分在这个具体情形里对我有什么帮助？
- **泛型编程**：把具体的算法“提升”到更高抽象层次的想法——通过添加参数来表达哪些部分可以变化而不改变算法的本质。第 20 章的 high() 例子就是一个简单的提升例子。STL 中的 find() 和 sort() 算法是用泛型编程以非常通用的形式表达的经典算法。参见第 20 21 章以及下面的例子。

全都用上！人们谈起编程风格（“范式”）时，常常把它们说成简单而彼此割裂的选项：要么用泛型编程，要么用面向对象编程。如果你的目标是以最好的方式表达问题的解法，你就会组合使用多种风格。所谓“最好”，我们指的是易读、易写、易维护，并且效率足够高。看一个例子：经典的“图形例子”源自 Simula（§22.2.4），通常被看作面向对象编程的例子。第一种解法可能像这样：

```
void draw_all(vector<Shape*>& v)
{
    for(int i = 0; i<v.size(); ++i) v[i]->draw();
}
```

这看起来确实“挺面向对象”。它关键地依赖于类层次，也依赖于虚函数调用能为每个给定的 Shape 找到正确的 draw() 函数；也就是说，对 Circle 调用 Circle::draw()，对 Open_polyline 调用 Open_polyline::draw()。但 vector<Shape*> 基本上是一个泛型编程构造：它依赖一个在编译时确定的参数（元素类型）。我们可以用一个简单的标准库算法来表达对所有元素的迭代，从而强调这一点：

```
void draw_all(vector<Shape*>& v)
{
    for_each(v.begin(),v.end(),mem_fun(&Shape::draw));
}
```

for_each() 的第三个参数是一个函数，要对前两个参数所指定序列中的每个元素调用它（§B.5.1）。现在，这第三次函数调用被假定为用 f(x) 语法调用的普通函数（或函数对象），而不是用 p->f() 语法调用的成员函数。所以，我们用标准库函数 mem_fun()（§B.6.2）来说明我们真正想调用的是一个成员函数（虚函数 Shape::draw()）。要点在于，for_each() 和 mem_fun() 作为模板，并不那么“像面向对象”；它们显然属于我们通常认为的泛型编程。更有意思的是，mem_fun() 是一个返回类对象的独立（模板）函数。换句话说，它很容易被归类为纯数据抽象（没有继承），甚至过程式编程（没有数据隐藏）。因此，我们可以说这一行代码用到了 C++ 所支持的四种基本风格的关键方面。

可是，我们为什么要写“画出所有图形”例子的第二个版本呢？它做的事情跟第一个版本从根本上一样；那样写甚至还多花几个字符！我们可以争辩说，用 for_each() 表达循环“更明显、更不容易出错”，但对很多人来说这算不上很有说服力的理由。更好的理由是：“for_each() 说的是要做什么（对一个序列进行迭代），而不是怎么做。”不过，对大多数人来说，真正有说服力的理由很简单：“它有用”：它指出了通往一种推广的道路（符合最好的泛型编程传统），使我们能解决更多问题。图形为什么放在 vector 里？为什么不是 list？为什么不是一般的序列？于是我们可以写出第三个（也更通用的）版本：

```
template<class Iter> void draw_all(Iter b, Iter e)
{
    for_each(b,e,mem_fun(&Shape::draw));
}
```

现在它对各种图形序列都能工作了。特别是，我们甚至可以针对 Shape 数组的元素调用它：

```
Point p {0,100};
Point p2 {50,50};
Shape* a[] = { new Circle(p,50), new Triangle(p,p2,Point(25,25)) };
draw_all(a,a+2);
```

我们还可以提供一个用起来更简单的版本，把它限制为只作用于容器：

```
template<class Cont> void draw_all(Cont& c)
{
    for (auto& p : c) p->draw();
}
```

甚至可以（使用 C++14 的概念，§19.3.3）：

```
void draw_all(Container& c)
{
    for (auto& p : c) p->draw();
}
```

要点仍然是：这段代码显然是面向对象的、泛型的，而且非常像普通的过程式代码。它在类层次和各个容器的实现上依赖于数据抽象。由于没有更好的说法，使用最合适的风格组合来编程一直被称为多范式编程。不过，我已经倾向于把它就看成编程：“范式”主要反映的是关于问题如何求解的受限视角，以及我们用来表达解法的编程语言的弱点。我预测，随着技术、编程语言和支持工具的大幅改进，编程会有光明的未来。

22.2 编程语言史概述

最初的时候，程序员是用凿子把 0 和 1 刻在石头上的！嗯，差不多是这样。这里，我们（几乎）从头开始，快速介绍编程语言史上的一些重大进展，它们都与使用 C++ 编程有关。

编程语言非常多。语言发明的速度至少是每十年 2000 种，“语言死亡”的速度也差不多。这里，我们通过简要提及十种语言来覆盖近 60 年。更多信息见 http://research.ihost.com/hopl/HOPL.html。在那里你可以找到三届 ACM SIGPLAN HOPL（History of Programming Languages，编程语言史）会议所有论文的链接。这些论文经过广泛的同行评议，因此比一般的网上信息来源可信、完整得多。我们这里讨论的语言在 HOPL 上都有人介绍过。请注意，如果你把一篇著名论文的完整标题输入网络搜索引擎，很有可能找到那篇论文。此外，这里提到的大多数计算机科学家都有个人主页，你可以在上面找到关于他们工作的大量信息。

本章对一种语言的介绍必然非常简短：提到的每种语言——以及数百种没有提到的语言——都值得写一整本书。我们对一种语言提到什么也高度挑剔。希望你把这当成一种挑战，去学得更多，而不

是想“原来语言 X 就这点东西！”请记住，这里提到的每种语言都是一项重大成就，都对我们这个世界作出了重要贡献。在这么短的篇幅里，我们根本不可能公正地对待这些语言——但完全不提会更糟。我们本来想为每种语言提供一点代码，但抱歉，这里不是做这种项目的地方（见习题 5 和习题 6）。

太常见的情况是，一项人工制品（例如一种编程语言）被呈现为它“本来就是那样”，或者被说成某个匿名“开发过程”的产物。这歪曲了历史：通常——尤其是在早期和形成期——一种语言是一个或（通常）多个人物的理想、工作、个人品味和外部约束的产物。因此，我们强调与这些语言相关的关键人物。IBM、贝尔实验室、剑桥大学等机构并不设计语言；设计语言的是这些机构中的个人——通常是与朋友和同事合作。

请注意一个常常扭曲我们历史观的奇怪现象。著名科学家和工程师的照片，大多是在他们成名、成为杰出人物之后拍摄的：国家科学院院士、皇家学会会士、圣约翰骑士团成员、图灵奖得主等等——换句话说，是在他们作出最辉煌工作几十年之后。几乎所有人直到晚年都是（或曾是）本行业最多产的人之一。然而，当你回顾自己最喜欢的语言特性和编程技术的诞生时，试着去想象一个年轻人（科学和工程界中的女性至今仍然远远不够多），正在盘算自己手头的钱够不够请女朋友去一家像样的餐馆；或者想象一位家长，正在考虑能否把一篇关键论文投到一个会议，而会议的时间和地点又能跟年轻家庭的假期结合起来。花白的胡须、秃掉的头顶和过时的衣着，都是很久以后才有的事。

22.2.1 最早的语言

1949 年起，第一批“现代”存储程序电子计算机出现时，每一台都有自己的语言。算法的表达（比如计算行星轨道）与特定机器的指令之间是一一对应的。显然，科学家（当时的用户大多是科学家）手头有写着数学公式的笔记，但程序却是一串机器指令。最早的原始清单是十进制或八进制数——与它们在计算机存储器中的表示完全一致。后来出现了汇编程序和“自动编码”（auto code）；也就是说，人们开发出这样的语言：机器指令和机器设施（如寄存器）都有符号化的名字。于是程序员可以写“LD R0 123”，把地址 123 处存储器的内容装入寄存器 0。不过，每台机器都有自己的指令集和自己的语言。

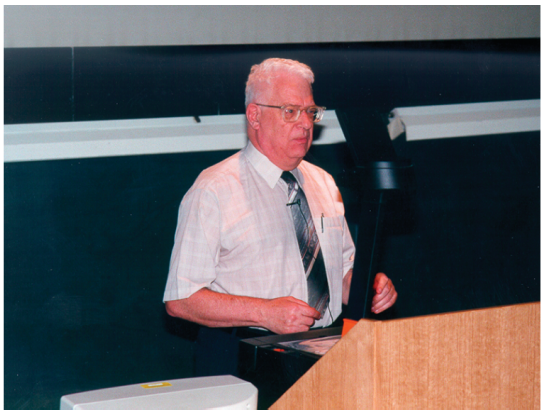
剑桥大学计算机实验室的大卫 • 惠勒（David Wheeler）是代表那个时代编程语言设计者的当然人选。1949 年，他写出了有史以来第一个真正在存储程序计算机上运行的程序（就是我们在 §4.4.2.1 见过的“平方表”程序）。他是大约十位有资格声称写出了第一个编译器（针对某台机器专用的“自动编码”）的人之一。函数调用是他发明的（是的，就连这么看似简单的东西，也得有人在某个时刻发明出来）。1951 年他写了一篇关于如何设计库的精彩论文；那篇论文至少超前了 20 年！他与莫里斯 • 威尔克斯（Maurice Wilkes，去查查他）和 D. J. Gill 合著了第一本关于编程的书。他获得了第一个计算机科学博士学位（1951 年于剑桥），后来又在硬件（缓存体系结构和早期局域网）和算法（例如 TEA 加密算法 [§25.5.6] 和“Burrows-Wheeler 变换”[bzip2 使用的压缩算法]）方面作出了重大贡献。大卫 • 惠勒恰好是 Bjarne Stroustrup 的博士论文导师——计算机科学是一门年轻的学科。大卫 • 惠勒一些最重要的工作是他在读研究生时做的。他后来成为剑桥的教授和皇家学会会士。

参考文献

Burrows, M. 与 David Wheeler，“A Block Sorting Lossless Data Compression Algorithm”，Technical Report 124, Digital Equipment Corporation, 1994。

Bzip2 链接：`www.bzip.org/`。

- Cambridge Ring 网站：`http://koo.corpus.cam.ac.uk/projects/earlyatm/cr82`。
- Campbell-Kelly, Martin. “David John Wheeler.” *Biographical Memoirs of Fellows of the Royal Society*，第 52 卷，2006 年。（他的技术传记。）



- EDSAC：`http://en.wikipedia.org/wiki/EDSAC`。
- Knuth, Donald. *The Art of Computer Programming*. Addison-Wesley，1968 年，以及之后的许多修订版。请到各卷索引中查找“David Wheeler”。
- TEA 链接：`http://en.wikipedia.org/wiki/Tiny_Encryption_Algorithm`。
- Wheeler, D. J. “The Use of Sub-routines in Programmes.” *Proceedings of the 1952 ACM National Meeting*。（这就是 1951 年那篇关于库设计的论文。）
- Wilkes, M. V., D. Wheeler, and D. J. Gill. *Preparation of Programs for an Electronic Digital Computer*. Addison-Wesley，1951 年；1957 年第 2 版。这是第一本关于编程的书。

22.2.2　现代语言的根源

下面这张图列出了早期的一些重要语言：

这些语言之所以重要，部分原因在于它们当年（有些直到今天）被广泛使用，或者因为它们成了现代重要语言的祖先——而且往往是同名的直系后代。本节讨论三种早期语言——Fortran、COBOL 和 Lisp——大多数现代语言都可追溯到它们。

22.2.2.1　Fortran

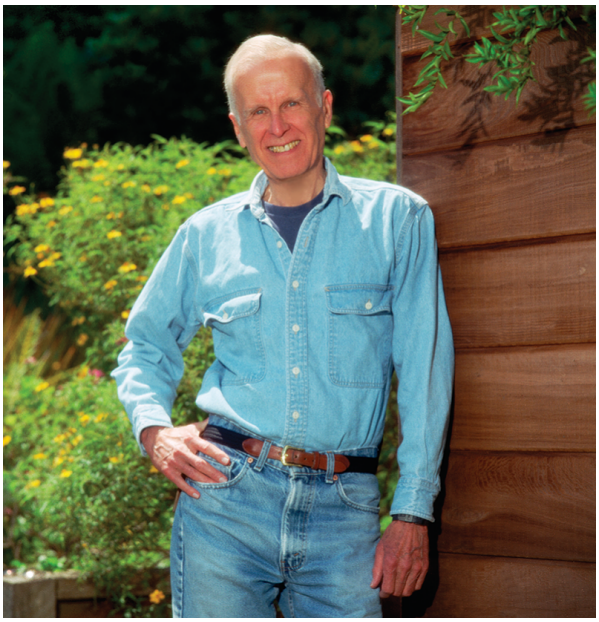
Fortran 于 1956 年问世，这可以说是编程语言发展史上最重要的一步。“Fortran”代表“Formula Translation”（公式翻译），其基本想法是：用一种为人而不是为机器设计的记号，生成高效的机器码。Fortran 记号的范本，是科学家和工程师用数学解题时写下的东西，而不是（当时还很新的）电子计算机所提供的机器指令。

从今天的视角看，可以把 Fortran 看作第一次尝试用代码直接表示一个应用领域。它让程序员能够像在教科书里见到的那样写线性代数。Fortran 提供了数组、循环和标准数学函数（使用标准数学记号，例如 $x + y$ 和 $\sin(x)$），有标准数学函数库和 I/O 机制，用户还可以自己定义函数和库。

这种记号在很大程度上与机器无关，因此 Fortran 代码常常只需稍加修改就能从一台计算机搬到另一台计算机。相对当时的技术水平，这是巨大的进步。因此，Fortran 被视为第一个高级编程语言。

当时人们认为，从 Fortran 源代码生成的机器码必须接近最优效率：机器有房间那么大，极其昂贵（相当于一队优秀程序员年薪的许多倍），按现代标准慢得荒唐（比如每秒 10 万条指令），内存也小得离谱（比如 8K 字节）。可人们确实在用这些机器运行有用的程序，因此不能让记号的改进（它能提高程序员的生产率和可移植性）妨碍这一点。

Fortran 在其目标领域，即科学与工程计算中，取得了巨大成功，并且从那时起一直在持续演进。Fortran 语言的主要版本有 II、IV、77、90、95、03。今天究竟 Fortran77 还是 Fortran90 用得更广，仍有争论。



Fortran 的第一个定义和第一个实现，是由 IBM 一个以 John Backus 为首的团队完成的：“我们并不知道自己想要什么，也不知道该怎么做。它就是那么长出来的。”他当时怎么可能知道呢？此前从未有

人做过这样的事。但在这个过程中，他们开发或发现了编译器的基本结构：词法分析、语法分析、语义分析和优化。直到今天，Fortran 在数值计算的优化方面仍然领先。在最初的 Fortran 之后还出现了一样东西：用来描述文法的记号，即巴科斯-诺尔范式（BNF）。它最早用于 Algol60（§22.2.3.1），如今大多数现代语言都在使用它。第 6 章和第 7 章讲文法时，用的就是 BNF 的一个版本。

很久以后，John Backus 开创了编程语言的一个全新分支（“函数式编程”），主张用数学方式编程，而不是采用基于读写内存位置的机器视角。请注意，纯数学里没有赋值这个概念，甚至没有“动作”。你只是“简单地”陈述在一组条件下必须为真的事情。函数式编程的部分根源在 Lisp（§22.2.2.3），函数式编程的一些思想也反映在 STL 中（第 21 章）。

参考文献

- Backus, John. “Can Programming Be Liberated from the von Neumann Style?” *Communications of the ACM*，1977 年。（他的图灵奖演讲。）
- Backus, John. “The History of FORTRAN I, II, and III.” *ACM SIGPLAN Notices*，第 13 卷第 8 期，1978 年。编程语言史会议专刊。
- Hutton, Graham. *Programming in Haskell*. Cambridge University Press，2007 年。ISBN 0521692695。
- ISO/IEC 1539. *Programming Languages —Fortran*。（“Fortran95”标准。）
- Paulson, L. C. *ML for the Working Programmer*. Cambridge University Press，1991 年。ISBN 0521390222。

22.2.2.2　COBOL

COBOL（“The Common Business-Oriented Language”，即面向商业的通用语言）对商业程序员的意义，正如 Fortran 对科学程序员的意义（而且有时至今仍然如此）。它的重点在数据操作：

- 复制
- 存储与检索（记录保存）
- 打印（报表）

计算与运算（在 COBOL 的核心应用领域里，这种看法常常是对的）被视为次要的事情。人们曾希望、也曾宣称，COBOL 与“商业英语”如此接近，以至于经理们自己就能编程，而程序员很快就会变得多余。这些年来，热衷削减编程成本的管理者一再重复这个希望。但它从来就没有哪怕一丁点真实性。

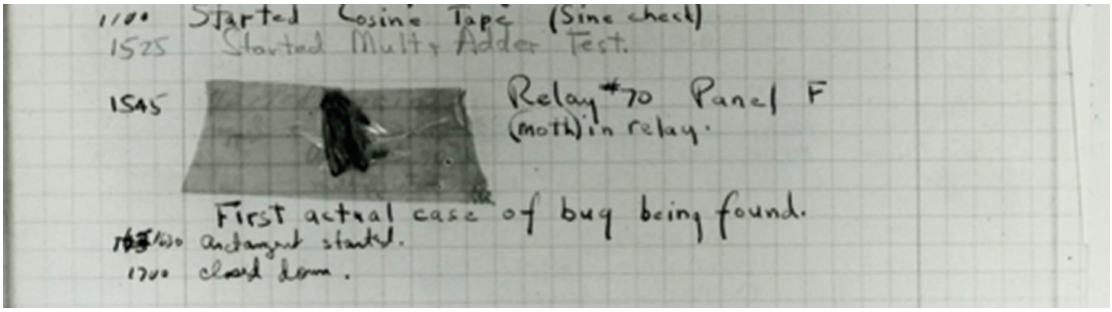
COBOL 最初是由一个委员会（CODASYL）在 1959-60 年设计的，发起方是美国国防部和一批大型计算机制造商，目的是满足与商业有关的计算需求。这个设计直接建立在 Grace Hopper 发明的 FLOW-MATIC 语言之上。她的贡献之一，是采用接近英语的语法（与 Fortran 开创、至今仍占主导的数学记号相对）。和 Fortran 一样——也和所有成功的语言一样——COBOL 经历了持续演进。主要修订版本有 60、61、65、68、70、80、90 和 04。

Grace Murray Hopper 在耶鲁大学获得数学博士学位。第二次世界大战期间，她为美国海军工作，参与了最早一批计算机的项目。在早期计算机产业干了几年之后，她又回到了海军：

> “美国海军后备少将 Grace Murray Hopper 博士是一位了不起的女性，她出色地应对了为最早一批计算机编程的种种挑战。在她作为软件开发概念领域领袖的一生中，她推动了从原始编程技术向使用复杂编译器的转变。她相信，‘我们一直都是这么做的’未必是继续做下去的好理由。”

——Anita Borg，1994 年“Grace Hopper Celebration of Women in Computing”会议

人们常把“第一个把计算机里的错误称为 bug 的人”这一功劳记在 Grace Murray Hopper 名下。她确实是较早使用这个词的人之一，并且留下了一份书面记录：



可以看出，那只 bug 是真的（一只飞蛾），而且它直接影响了硬件。现代的大多数 bug 似乎都出在软件里，视觉上的吸引力也差得远。

参考文献

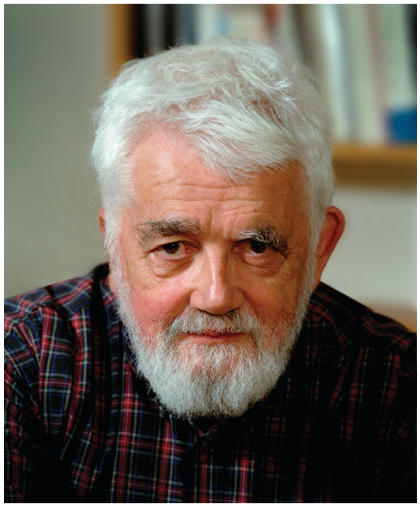
- G. M. Hopper 的传记：`http://tergestesoft.com/~eddysworld/hopper.htm`。
- ISO/IEC 1989:2002. *Information Technology —Programming Languages —COBOL.*
- Sammet, Jean E. “The Early History of COBOL.” *ACM SIGPLAN Notices*，第 13 卷第 8 期，1978 年。编程语言史会议专刊。

22.2.2.3 Lisp

Lisp 最初由 MIT 的 John McCarthy 于 1958 年设计，用于链表和符号处理（它的名字就来自“LISt Processing”）。Lisp 起初是解释执行的，而非编译执行（现在也常常如此）。Lisp 方言有几十种（很可能是上百种）。事实上，人们常说“Lisp 一词隐含复数”。目前最流行的方言是 Common Lisp 和 Scheme。这个语言家族一直是（现在也仍然是）人工智能（AI）研究的主力（不过交付的产品常常是用 C 或 C++ 写的）。Lisp 的主要灵感来源之一，是（数学上的）lambda 演算。

Fortran 和 COBOL 是专为帮助解决各自应用领域中的现实问题而设计的。Lisp 社群则更关心编程本身以及程序的优雅。这些努力往往是成功的。Lisp 是第一种把自身定义与硬件分离、并把语义建立在某种数学之上的语言。如果说 Lisp 有特定的应用领域，那它要难界定得多：“AI”和“符号计算”并

不像“商业处理”和“科学编程”那样，能清晰地对应到日常任务。Lisp（以及 Lisp 社群）的思想可以在许多更现代的语言中找到，尤其是函数式语言。



John McCarthy 在加州理工学院获得数学学士学位，在普林斯顿大学获得数学博士学位。你也许会注意到，编程语言的设计者中有很多数学专业出身的人。在 MIT 做出令人难忘的工作之后，McCarthy 于 1962 年转到斯坦福，帮助创建了斯坦福人工智能实验室。人们普遍认为“人工智能”一词是他创造的，他对该领域也贡献良多。

参考文献

- Abelson, Harold, and Gerald J. Sussman. *Structure and Interpretation of Computer Programs*, Second Edition. MIT Press，1996 年。ISBN 0262011530。
- ANSI INCITS 226-1994（原 ANSI X3.226:1994）。*American National Standard for Programming Language —Common LISP*。
- McCarthy, John. “History of LISP.” *ACM SIGPLAN Notices*，第 13 卷第 8 期，1978 年。编程语言史会议专刊。
- Steele, Guy L., Jr. *Common Lisp: The Language*. Digital Press，1990 年。ISBN 1555580416。
- Steele, Guy L., Jr., and Richard Gabriel. “The Evolution of Lisp.” *Proceedings of the ACM History of Programming Languages Conference (HOPL-2). ACM SIGPLAN Notices*，第 28 卷第 3 期，1993 年。

22.2.3 Algol 家族

1950 年代末，许多人觉得编程正变得过于复杂、过于随意、也过于不科学。他们觉得编程语言种类多得没有必要，而且这些语言在构建时对通用性和可靠的基本原理关注不够。这种情绪此后多次浮现；当时一群人聚集在 IFIP（国际信息处理联合会）的主持下，仅用几年就创造出一门新语言，彻底改变了我们思考语言及其定义的方式。大多数现代语言——包括 C++——都大大受惠于这项努力。

22.2.3.1 Algol60

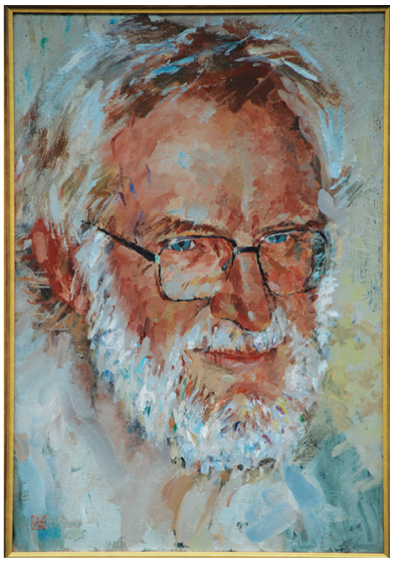
由 IFIP 2.1 工作组努力促成的“ALGOrithmic Language”（算法语言）Algol，是现代编程语言概念的一次突破：

- 词法作用域
- 用文法定义语言
- 语法规则与语义规则的清晰分离
- 语言定义与实现的清晰分离
- 系统地使用（静态的，即编译期的）类型
- 直接支持结构化编程

“通用编程语言”这一概念本身，是随 Algol 一起出现的。在此之前，语言要么是科学的（如 Fortran），要么是商业的（如 COBOL），要么是表处理的（如 Lisp）、仿真的，等等。在这些语言当中，Algol60 与 Fortran 关系最密切。

遗憾的是，Algol60 从未在学术圈之外获得大规模使用。业界许多人觉得它“太古怪”，Fortran 程序员嫌它“太慢”，COBOL 程序员说它“不支持商业处理”，Lisp 程序员说它“不够灵活”，业界大多数人（包括掌管工具投资的管理者）说它“太学术”，许多美国人则说它“太欧洲”。这些批评大多是对的。例如，Algol60 报告根本没有定义任何 I/O 机制！不过，类似的批评几乎可以加在当时任何一种语言身上——而 Algol 在许多方面树立了新标准。

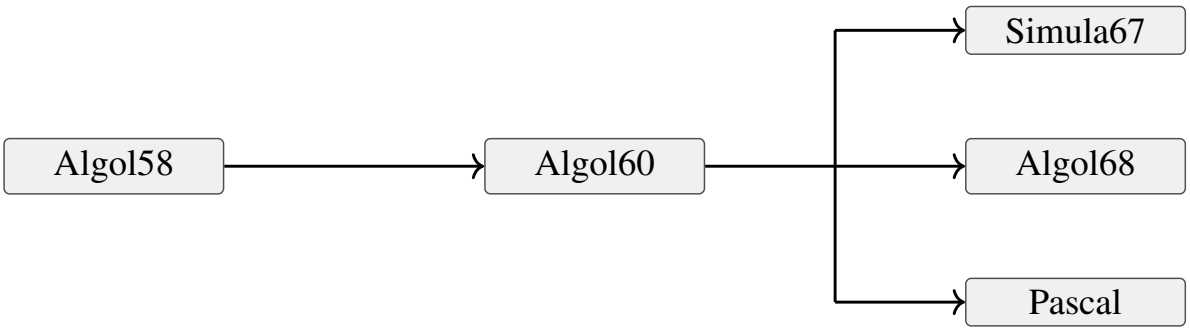
Algol60 有一个问题：没人知道该怎么实现它。这个问题由 Peter Naur（Algol60 报告的编辑）和 Edsger Dijkstra 领导的一个程序员团队解决了：



Peter Naur 在哥本哈根大学接受的是天文学教育，先后在哥本哈根技术大学（DTH）和丹麦计算机制造商 Regnecentralen 工作。他很早就（1950–51 年）在英国剑桥的计算机实验室学会了编程（丹麦那时还没有计算机），后来拥有一段横跨学术界与产业界的卓越职业生涯。他是 BNF（巴科斯-诺尔范式）的共同发明人，而 BNF 正是用来描述文法的；他也是用形式化方法推理程序的早期倡导者（Bjarne Stroustrup 最早大约在 1971 年，就是从 Peter Naur 的技术文章里学会了不变式的用法）。Naur 始终对计算抱有一种深思熟虑的见解，总在考虑编程中人的方面。事实上，他后来的工作完全可以算作哲学的一部分（只不过他认为传统学院哲学纯属胡说）。他是哥本哈根大学第一位 datalogi 教授（丹麦语 datalogi 最好译作“信息学”；Peter Naur 讨厌“计算机科学”这个说法，认为它名不副实——计算主要不是关于计算机的）。

Edsger Dijkstra 是计算机科学史上另一位不世出的大师。他在莱顿学习物理，但早期的计算工作是在阿姆斯特丹的 Mathematisch Centrum 做的。后来他在不少地方工作过，包括埃因霍温理工大学、Burroughs 公司和得克萨斯大学（奥斯汀）。除了在 Algol 上的开创性工作，他还是把数学逻辑用于编程和算法的先驱与坚定倡导者，并且是 THE 操作系统的设计者与实现者之一——THE 是最早系统地处理并发的操作系统之一。THE 代表“Technische Hogeschool Eindhoven”，即 Dijkstra 当时工作的那所大学。可以说，他最著名的论文是“Go-To Statement Considered Harmful”，该文令人信服地揭示了非结构化控制流的问题。

Algol 的家族树令人印象深刻：



请注意 Simula67 和 Pascal。这两种语言是许多（很可能是大多数）现代语言的祖先。

参考文献

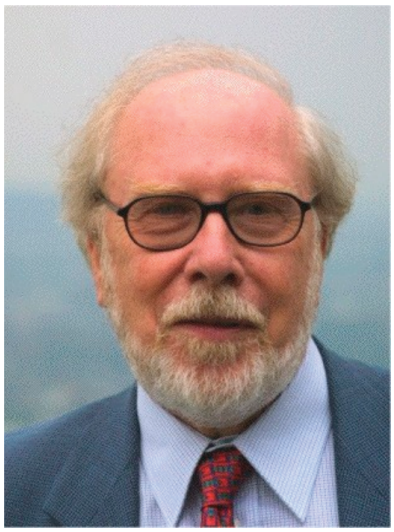
- Dijkstra, Edsger W. “Algol 60 Translation: An Algol 60 Translator for the x1 and Making a Translator for Algol 60.” Report MR 35/61. Mathematisch Centrum (Amsterdam)，1961 年。
- Dijkstra, Edsger. “Go-To Statement Considered Harmful.” *Communications of the ACM*，第 11 卷第 3 期，1968 年。
- Lindsey, C. H. “The History of Algol68.” *Proceedings of the ACM History of Programming Languages Conference (HOPL-2). ACM SIGPLAN Notices*，第 28 卷第 3 期，1993 年。
- Naur, Peter, ed. “Revised Report on the Algorithmic Language Algol 60.” A/S Regnecentralen (Copenhagen)，1964 年。
- Naur, Peter. “Proof of Algorithms by General Snapshots.” *BIT*，第 6 卷，1966 年，第 310–16 页。（大概是最早一篇讲如何证明程序正确的论文。）
- Naur, Peter. “The European Side of the Last Phase of the Development of ALGOL 60.” *ACM SIGPLAN Notices*，第 13 卷第 8 期，1978 年。编程语言史会议专刊。
- Perlis, Alan J. “The American Side of the Development of Algol.” *ACM SIGPLAN Notices*，第 13 卷第 8 期，1978 年。编程语言史会议专刊。
- van Wijngaarden, A., B. J. Mailloux, J. E. L. Peck, C. H. A. Koster, M. Sintzoff, C. H. Lindsey, L. G. L. T. Meertens, and R. G. Fisker, eds. *Revised Report on the Algorithmic Language Algol 68* (Sept. 1973). Springer-Verlag，1976 年。

22.2.3.2 Pascal

Algol 家族树里提到的 Algol68，是一个庞大而雄心勃勃的项目。和 Algol60 一样，它出自“Algol 委员会”（IFIP 第 2.1 工作组）之手，但它花了“永远那么久”才完成，许多人等得不耐烦，怀疑这个项目永远拿不出有用的东西。委员会中的一位成员 Niklaus Wirth 决定干脆自己设计和实现 Algol 的后继者。与 Algol68 相反，这门叫 Pascal 的语言是对 Algol60 的简化。

Pascal 于 1970 年完成，结果确实简单，也因此有些死板。人们常说它只是为了教学而设计的，但早期论文把它描述为当时超级计算机上 Fortran 的替代品。Pascal 确实容易学，在出现了一个可移植性很好的实现之后，它作为教学语言变得非常流行，但事实证明它对 Fortran 并不构成威胁。

Pascal 是苏黎世瑞士联邦理工学院（ETH）的 Niklaus Wirth 教授（照片分别摄于 1969 年和 2004 年）的工作成果。他的博士（电气工程与计算机科学）出自加州大学伯克利分校，他一生都与加州保持着联系。Wirth 教授是世界上最接近“职业语言设计者”的人。在 25 年间，他设计并实现了



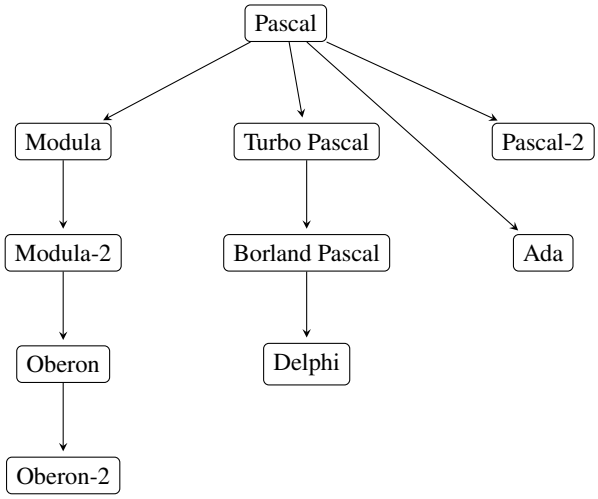
- Algol W
- PL/360
- Euler
- Pascal
- Modula
- Modula-2
- Oberon
- Oberon-2
- Lola（一种硬件描述语言）

尼可劳斯 • 维尔特（Niklaus Wirth）把这称为自己对简洁性的无尽追求。他的工作影响极大。研究这一系列语言，是一件极有意思的练习。维尔特教授是唯一一位在 HOPL 会议上介绍过两门语言的人。

最终，纯粹的 Pascal 被证明对工业应用来说过于简单、也过于僵硬。20 世纪 80 年代，它之所以免于消亡，主要靠的是安德斯 • 海尔斯伯格（Anders Hejlsberg）的工作。海尔斯伯格是 Borland 公司的三位创始人之一。他先设计并实现了 Turbo Pascal（除其他改进外，还提供了更灵活的参数传递机制），后来又加入了类似 C++ 的对象模型（但只有单继承，并配有一套漂亮的模块机制）。他受教于哥本哈

根技术大学，彼得·诺尔（Peter Naur）偶尔在那里授课——世界有时候真是小得很。海尔斯伯格后来为 Borland 设计了 Delphi，为微软设计了 C#。

（必然经过简化的）Pascal 家族树如下：



参考文献

Borland/Turbo Pascal. `http://en.wikipedia.org/wiki/Turbo_Pascal`.

Hejlsberg, Anders, Scott Wiltamuth, and Peter Golde. *The C# Programming Language*, Second Edition. Microsoft .NET Development Series. ISBN 0321334434.

Wirth, Niklaus. “The Programming Language Pascal.” *Acta Informatics*, Vol. 1 Fasc 1, 1971.

Wirth, Niklaus. “Design and Implementation of Modula.” *Software—Practice and Experience*, Vol. 7 No. 1, 1977.

Wirth, Niklaus. “Recollections about the Development of Pascal.” Proceedings of the ACM History of Programming Languages Conference (HOPL-2). ACM SIGPLAN Notices, Vol. 28 No. 3, 1993.

Wirth, Niklaus. *Modula-2 and Oberon*. Proceedings of the Third ACM SIGPLAN Conference on the History of Programming Languages (HOPL-III). San Diego, CA, 2007. `http://portal.acm.org/toc.cfm?id=1238844`.

22.2.3.3 Ada

Ada 编程语言当初的设计目标，是成为满足美国国防部（U.S. Department of Defense）全部编程需求的语言。具体来说，它要成为一门为嵌入式系统编程交付可靠、可维护代码的语言。它最明显的祖先语言是 Pascal 和 Simula（见 §22.2.3.2 与 §22.2.4）。设计 Ada 的小组由让·伊什比亚（Jean Ichbiah）领导——他曾担任 Simula 用户组的主席。Ada 的设计强调：

- 数据抽象（但 1995 年之前没有继承）
- 强静态类型检查
- 语言直接支持并发

Ada 的设计目标是成为软件工程在编程语言中的化身。因此，美国国防部并没有亲自设计这门语言；它设计的是一套设计语言的繁复流程。大量人员和组织参与了这个设计过程：过程经过一系列竞赛向前推进，先产生最好的规格说明，再产生体现获胜规格说明思想的最好的语言。这项庞大的工程持续了 20 年（1975—1998），从 1980 年起由一个名为 AJPO（Ada 联合项目办公室）的部门管理。

1979 年，由此产生的语言以奥古斯塔 • 埃达 • 洛夫莱斯夫人（Lady Augusta Ada Lovelace，诗人拜伦勋爵的女儿）命名。可以说洛夫莱斯夫人是近代（对“近代”的某种定义之下）第一位程序员，因为她在 19 世纪 40 年代曾与查尔斯 • 巴贝奇（Charles Babbage，剑桥的卢卡斯数学教授——那是牛顿的教席！）合作研制一台革命性的机械计算机。遗憾的是，巴贝奇的机器作为实用工具并不成功。

由于这套繁复的流程，Ada 一直被视为“委员会设计”语言的终极代表。获胜设计团队的首席设计师、来自法国霍尼韦尔 • 布尔（Honeywell Bull）公司的让 • 伊什比亚坚决否认这一点。不过，根据我与他的讨论，我怀疑：要不是受到流程如此强的约束，他本来能设计出一门更好的语言。

多年来，美国国防部要求军用应用必须使用 Ada，于是有了这样的说法：“Ada 不只是个好主意，它是法律！”最初，Ada 的使用只是“被要求”，但当许多项目获得“豁免”而改用其他语言（通常是

C++）之后，美国国会通过了一项法律，要求在多数军用应用中使用 Ada。面对商业与技术的现实，那项法律后来被撤销了。世界上极少有人像比亚内 • 斯特劳斯特鲁普那样，作品被美国国会立法禁止。

话虽如此，我们仍要坚持：Ada 是一门比它的名声好得多的语言。我们猜想，如果美国国防部对它的使用、以及对使用它的确切方式（应用开发流程标准、软件开发工具、文档等）没有那么强硬，它本可以取得明显更大的成功。直到今天，Ada 在航空航天应用以及类似的先进嵌入式系统应用领域仍然重要。

Ada 于 1980 年成为军用标准，1983 年成为 ANSI 标准（第一个实现也是 1983 年完成的——比第一个标准晚了三年！），1987 年成为 ISO 标准。为形成 1995 年的 ISO 标准，这份 ISO 标准经过了大幅（但当然是兼容的）修订。值得注意的改进包括：并发机制更灵活，并且支持继承。

参考文献

Barnes, John. *Programming in Ada 2005*. Addison-Wesley, 2006. ISBN 0321340787.

Consolidated Ada Reference Manual, consisting of the international standard (ISO/IEC 8652:1995). Information Technology —Programming Languages —Ada, as updated by changes from Technical Corrigendum 1 (ISO/IEC 8652:1995:TC1:2000).

Ada information page: `www.adaic.org/`.

Whitaker, William A. *ADA —The Project: The DoD High Order Language Working Group*. Proceedings of the ACM History of Programming Languages Conference (HOPL-2). ACM SIGPLAN Notices, Vol. 28 No. 3, 1993.

22.2.4 Simula

Simula 是 20 世纪 60 年代早期到中期由克里斯滕 • 尼高（Kristen Nygaard）和奥勒-约翰 • 达尔（Ole-Johan Dahl）在挪威计算中心与奥斯陆大学开发的。Simula 无疑属于 Algol 语言家族。事实上，Simula 几乎是 Algol60 的完全超集。然而，我们之所以把 Simula 单独挑出来特别讨论，是因为今天被称为“面向对象编程”的多数基本思想都源自它。它是第一门提供继承和虚函数的语言。“类”（class）一词表示“用户定义类型”，“虚”（virtual）一词表示那种可以通过基类提供的接口来覆盖并调用的函数——这两个词都来自 Simula。

Simula 的贡献不限于语言特性。它还带来了一套表述清晰的面向对象设计观念，其基础是用代码为现实世界的现象建模：

- 把想法表示为类（class）与类对象（class object）。
- 把层次关系表示为类层次（继承）。

于是，程序变成了一组相互作用的对象，而不是一个铁板一块的整体。

克里斯滕 • 尼高——Simula67 的共同发明人（与他一起的是左边戴眼镜的奥勒-约翰 • 达尔）——从多数标准看（包括身高）都是一位巨人，他的热情与慷慨也同样出众。他构想出了面向对象编程与设计的根本思想，尤其是继承，并在几十年里不断追索它们的含义。他从不满足于简单、短期而短视的答案。他持续参与社会事务，坚持了几十年。挪威之所以没有加入欧盟，可以说有他相当大的功劳：他把欧盟看作一个潜在的、中央集权的官僚噩梦，会对位于联盟远边缘的小国——挪威——的需求漠不关心。20 世纪 70 年代中期，克里斯滕 • 尼高在丹麦奥胡斯大学的计算机科学系度过了相当长的时间（当时比亚内 • 斯特劳斯特鲁普正在那里攻读硕士学位）。

克里斯滕 • 尼高的硕士学位是奥斯陆大学的数学学位。他于 2002 年去世，就在他与终身好友奥勒-约翰 • 达尔一起获得 ACM 图灵奖（可以说是计算机科学家最高的职业荣誉）之前一个月。

奥勒-约翰·达尔是一位更传统的学者。他对规格说明语言和形式化方法很感兴趣。1968 年，他成为奥斯陆大学第一位信息学（计算机科学）正教授。

2000 年 8 月，达尔和尼高被挪威国王授予圣奥拉夫勋章指挥官称号。看来连真正的极客也能在自己的家乡得到认可！

参考文献

Birtwistle, G., O-J. Dahl, B. Myhrhaug, and K. Nygaard. *SIMULA Begin*. Studentlitteratur (Lund, Sweden), 1979. ISBN 9144062125.

Holmevik, J. R. “Compiling SIMULA: A Historical Study of Technological Genesis.” *IEEE Annals of the History of Computing*, Vol. 16 No. 4, 1994, pp. 25–37.

Krogdahl, S. “The Birth of Simula.” Proceedings of the HiNC 1 Conference in Trondheim, June 2003 (IFIP WG 9.7, in cooperation with IFIP TC 3).

Nygaard, Kristen, and Ole-Johan Dahl. “The Development of the SIMULA Languages.” *ACM SIGPLAN Notices*, Vol. 13 No. 8, 1978. Special Issue: History of Programming Languages Conference.

SIMULA Standard. DATA Processing —Programming Languages —SIMULA. Swedish Standard, Stockholm, Sweden (1987). ISBN 9171622349.

22.2.5 C

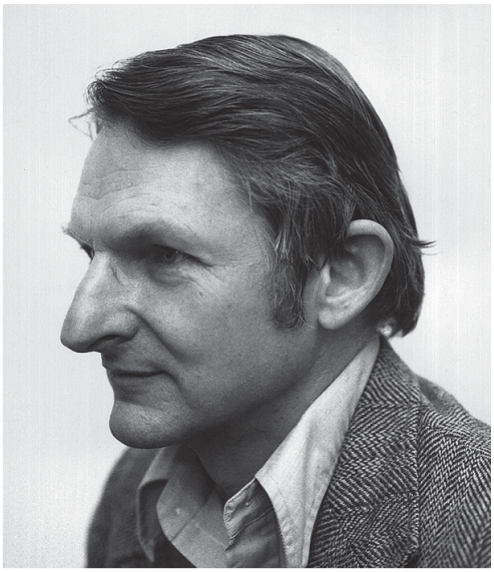
1970 年，人们“普遍认为”，严肃的系统编程——特别是操作系统的实现——必须用汇编代码完成，而且不可能做到可移植。这与 Fortran 之前科学编程的处境很相似。若干个人和小组开始向这种正统观念发起挑战。从长远来看，C 编程语言（第 27 章）是这些努力中最成功的一个，而且优势明显。

丹尼斯·里奇（Dennis Ritchie）在新泽西州默里山的贝尔电话实验室计算机科学研究中心设计并实现了 C 编程语言。C 的美妙之处在于，它是一门刻意保持简单的编程语言，紧紧贴近硬件的根本特性。如今的大部分复杂性（其中多数为了兼容性也重现于 C++ 中）都是在他最初的设计之后加上去的，而且有几项还是在他反对的情况下加上的。C 成功的一部分原因在于它很早就被广泛使用，但它真正的力量在于把语言特性直接映射到硬件设施上（见 §25.4–5）。丹尼斯·里奇把 C 简洁地描述为“一门强类型、但检查很弱的语言”；也就是说，C 有一个静态的（编译期）类型系统，如果一个程序以与其定义不符的方式使用某个对象，那就是不合法的。然而，C 编译器无法检查这一点。在 C 编译器必须在 48K 字节内存中运行的时代，这种安排是说得通的。C 投入使用后不久，人们就设计了一个名为 lint 的程序，它在编译器之外单独检查程序是否符合类型系统。

与肯·汤普森（Ken Thompson）一起，丹尼斯·里奇是 Unix 的共同发明人，而 Unix 是有史以来影响最大的操作系统。C 过去是——现在也是——与 Unix 操作系统联系在一起，并通过 Unix 与 Linux 及开源运动联系在一起。

在贝尔实验室计算机科学研究中心，丹尼斯·里奇工作了 40 年。他毕业于哈佛大学（物理学专业）；他在哈佛大学的应用数学博士学位始终没有拿到，因为他要么忘了、要么拒绝支付一笔很小的（60 美元）注册费。

早年，即 1974—1979 年，贝尔实验室的许多人影响了 C 的设计与采用。道格 • 麦基尔罗伊（Doug McIlroy）是大家最喜欢的批评者、讨论伙伴和点子大王。他影响了 C、C++、Unix，以及远不止这些的东西。



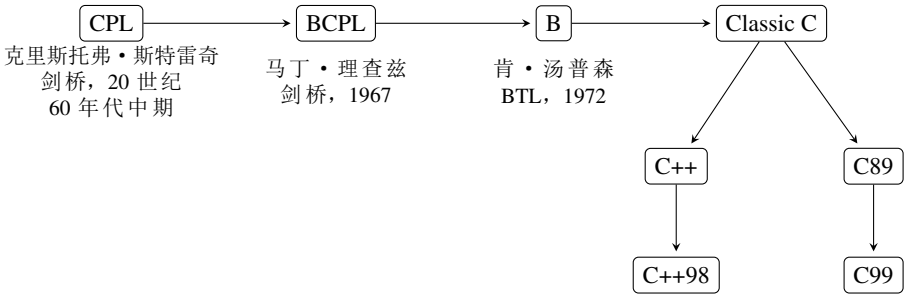
布莱恩·克尼汉（Brian Kernighan）是一位出类拔萃的程序员和作家。他的代码和文章都是清晰性的典范。本书的风格部分源自他那部杰作《C 程序设计语言》（The C Programming Language，以两位作者布莱恩·克尼汉和丹尼斯·里奇的名字缩写，被称为“K&R”）中的教学章节。

光有好想法是不够的；要让想法在大范围内真正有用，就必须把它们化到最简单的形式，并清晰地表达出来，使目标读者中的大量人群都能理解。罗嗦是这种表达的大敌之一；含糊其辞和过度抽象同样是。纯粹主义者常常对这种普及的成果嗤之以鼻，更偏爱以只有专家才看得懂的方式呈现“原创结果”。我们不这样看：把一个不平凡但有价值的想法装进初学者的脑袋，这件事很难，却是专业水准成长所必需的，对社会整体也有价值。

多年来，布莱恩·克尼汉参与了许多有影响力的编程与出版项目。两个例子是 AWK——一门早期脚本语言，名字取自其作者（Aho、Weinberger 和 Kernighan）的姓氏首字母——以及 AMPL，即“一种数学编程语言”（A Mathematical Programming Language）。

布莱恩·克尼汉目前是普林斯顿大学的教授；他当然是一位出色的教师，专长是把本来复杂的话题讲清楚。他在贝尔实验室计算机科学研究中心工作了 30 多年。贝尔实验室后来成为 AT&T 贝尔实验室，再后来又分拆为 AT&T 实验室和朗讯贝尔实验室。他毕业于多伦多大学（物理学专业）；他的博士学位是普林斯顿大学的电气工程博士。

C 语言家族树如下：



C 的起源可以追溯到英国那个从未完成的 CPL 项目、马丁·理查兹（Martin Richards）从剑桥大学休假访问 MIT 期间做出的 BCPL（Basic CPL）语言，以及肯·汤普森做出的解释型语言 B。后来，C 由 ANSI 和 ISO 标准化，并且从 C++ 接受了很多影响（例如函数实参检查和 const）。

CPL 是剑桥大学与伦敦帝国学院（Imperial College）的合作项目。最初，项目在剑桥进行，所以“C”在官方意义上代表“Cambridge”（剑桥）。帝国学院加入之后，“C”的官方解释变成了“Combined”（联合）。而在实际上（至少我们听到的说法如此），“C”一直代表“Christopher”，取自 CPL 的主要设计者克里斯托弗·斯特雷奇（Christopher Strachey）。

参考文献

Brian Kernighan 的主页：`http://cm.bell-labs.com/cm/cs/who/bwk` 与 `www.cs.princeton.edu/~bwk/`。

Dennis Ritchie 的主页：`http://cm.bell-labs.com/cm/cs/who/dmr`。

ISO/IEIC 9899:1999. *Programming Languages —C*.（C 标准。）

Kernighan, Brian, and Dennis Ritchie. *The C Programming Language*. Prentice Hall, 1978. Second Edition, 1988. ISBN 0131103628.

贝尔实验室计算机科学研究中心成员名单：`http://cm.bell-labs.com/cm/cs/alumni.html`。

Ritchards, Martin. *BCPL —The Language and Its Compiler*. Cambridge University Press, 1980. ISBN 0521219655.

Ritchie, Dennis. “The Development of the C Programming Language. Proceedings of the ACM History of Programming Languages Conference (HOPL-2). ACM SIGPLAN Notices, Vol. 28 No. 3, 1993.

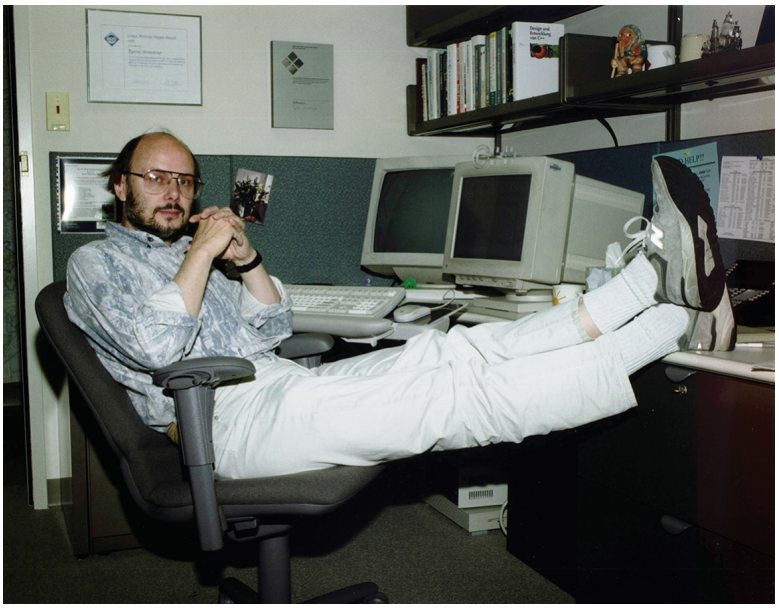
Salus, Peter. *A Quarter Century of UNIX*. Addison-Wesley, 1994. ISBN 0201547775.

22.2.6 C++

C++ 是一门通用编程语言，偏重于系统编程，它：

- 是更好的 C
- 支持数据抽象
- 支持面向对象编程
- 支持泛型编程

C++ 最初由比亚内·斯特劳斯特鲁普（Bjarne Stroustrup）在贝尔电话实验室位于新泽西州默里山的计算机科学研究中心设计并实现，也就是说，就在丹尼斯·里奇、布莱恩·克尼汉、肯·汤普森、道格·麦基尔罗伊等 Unix 名家的走廊另一头。



Bjarne Stroustrup 在家乡丹麦奥胡斯（Aarhus）的大学获得硕士学位（数学与计算机科学方向）。随后他前往剑桥，在 David Wheeler 指导下获得博士学位（计算机科学）。C++ 的主要贡献在于：

- 让抽象技术对主流项目而言负担得起、易于管理；
- 在效率至关重要的应用领域，率先使用面向对象编程与泛型编程技术。

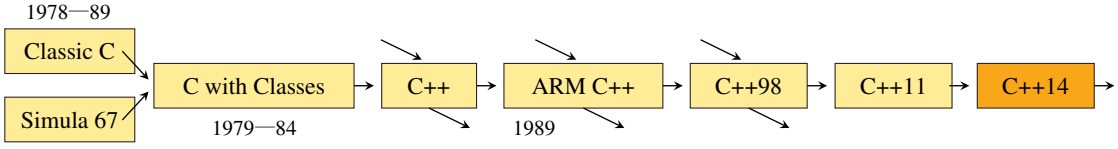
在 C++ 之前，这些技术（人们常笼统地把它们归在“面向对象编程”这个名目之下）在工业界大多鲜为人知。正如 Fortran 之前的科学计算、C 之前的系统编程一样，当时“众所周知”，这些技术对实际应用来说过于昂贵，对“普通程序员”来说也过于复杂，难以掌握。

C++ 的工作始于 1979 年，1985 年发布商业版本。在最初的设计与实现之后，Bjarne Stroustrup 与贝尔实验室及其他地方的朋友一起继续开发它，直到 1990 年正式启动标准化工作。此后，C++ 的定义先由 ANSI（美国国家标准机构）负责，自 1991 年起改由 ISO（国际标准化组织）负责。Bjarne Stroustrup 在其中发挥了主要作用，担任负责新语言特性的关键小组的主席。第一个国际标准（C++98）于 1998 年获批，第二个标准于 2011 年获批（C++11）。下一个 ISO 标准将是 C++14，再下一个有时称为 C++1y，可能会成为 C++17。

C++ 在最初十年的成长之后最重要的进展是 STL——标准库中用于容器和算法的设施。它是数十年工作的成果（主要出自 Alexander Stepanov 之手），目标是产出最通用、最高效的软件，其灵感来自数学的美与实用。

Alex Stepanov 是 STL 的发明者，也是泛型编程的先驱。他毕业于莫斯科大学，研究过机器人、算法等诸多领域，使用过多种语言（包括 Ada、Scheme 和 C++）。自 1979 年起，他先后在美国的学术界和工业界工作，尤其是在 GE 实验室、AT&T 贝尔实验室、惠普、Silicon Graphics 和 Adobe。

C++ 的族谱是这样的：



“C with Classes”是 Bjarne Stroustrup 对 C 与 Simula 思想的最初综合。在它的后继者 C++ 实现之后，它随即消亡。

讨论语言时，人们常常关注优雅和高级特性。然而，C 和 C++ 并不是靠这些成为计算史上最成功的两种语言的。它们的长处是灵活性、性能和稳定性。大型软件系统要存活几十年，常常会耗尽硬件资源，也常常遭遇完全出乎意料的需求变化。C 和 C++ 能够在这种环境中繁荣发展。我们最喜欢的 Dennis Ritchie 的一句话是：“有些语言是为了证明某个观点而设计的，另一些则是为了解决某个问题而设计

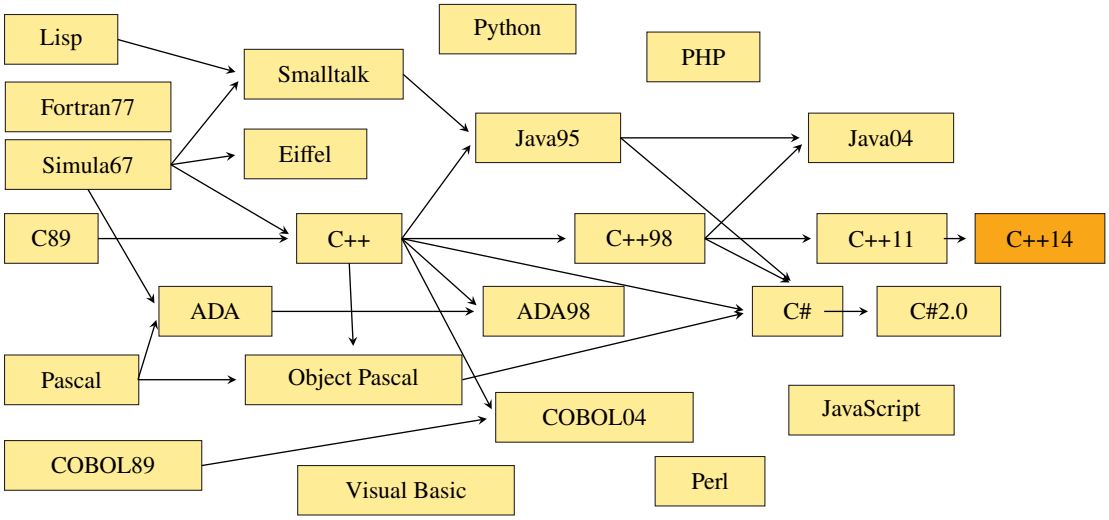
的。”他所说的“另一些”，主要指 C。Bjarne Stroustrup 喜欢说：“连我都知道怎样设计一门比 C++ 更漂亮的语言。”C++ 的目标——和 C 一样——不是抽象的美（尽管能得到时我们非常欣赏），而是实用。

参考文献

- Alexander Stepanov 的论文：www.stepanovpapers.com。
- Bjarne Stroustrup 的主页：www.stroustrup.com。
- ISO/IEC 14882:2011. Programming Languages —C++.（C++ 标准。）
- Stroustrup, Bjarne. “A History of C++: 1979−1991.” Proceedings of the ACM History of Programming Languages Conference (HOPL-2). ACM SIGPLAN Notices, Vol. 28 No. 3, 1993.
- Stroustrup, Bjarne. The Design and Evolution of C++. Addison-Wesley, 1994. ISBN 0201543303.
- Stroustrup, Bjarne. The C++ Programming Language, Fourth Edition. Addison-Wesley, 2013. ISBN 978-0321563842.
- Stroustrup, Bjarne. A Tour of C++. Addison-Wesley, 2013. ISBN 978-0321958310.
- Stroustrup, Bjarne. “C and C++: Siblings”; “C and C++: A Case for Compatibility”; and “C and C++: Case Studies in Compatibility.” The C/C++ Users Journal. July, Aug., and Sept. 2002.
- Stroustrup, Bjarne. “Evolving a Language in and for the Real World: C++ 1991−2006.” Proceedings of the Third ACM SIGPLAN Conference on the History of Programming Languages (HOPL-III). San Diego, CA, 2007. http://portal.acm.org/toc.cfm?id=1238844.

22.2.7　今天

当前人们在用哪些编程语言，用来做什么？这个问题真的很难回答。当代语言的族谱——即使只取最简略的形式——也相当拥挤而凌乱：



事实上，我们在网上（以及其他地方）找到的大多数统计数字，比传言好不了多少，因为它们度量的东西与“使用”只有很弱的关联，例如包含某编程语言名称的网页帖子数、编译器的出货量、学术论文数、图书销量等等。这类度量全都偏向新语言而轻视成熟语言。再说，什么才算程序员？是每天使用编程语言的人吗？只是为了学习而写小程序的学员算不算？只谈论编程的教授算不算？几乎每年写一个程序的物理学家算不算？一个职业程序员——几乎按定义——每周要用好几种编程语言，那么他被统计很多次还是只统计一次？我们看到，对这些问题，不同的统计各有各的答案。

不过，我们觉得有义务给你一个看法：2014 年全世界大约有 1000 万名职业程序员。这个看法依据的是 IDC（一家数据收集公司）、与出版商和编译器供应商的讨论，以及各种网络来源。你尽可以挑刺，但我们知道：只要对“程序员”给出一个还算合理的定义，这个数字就大于 100 万、小于 1 亿。他们用哪些语言？Ada、C、C++、C#、COBOL、Fortran、Java、PERL、PHP、Python 和 Visual Basic 大概（只是大概）占全部程序的 90% 以上。

除了这里提到的语言，我们还能列出几十种甚至几百种。除非是为了对有趣或重要的语言尽量公平，我们看不出这样做有什么意义。需要时请自己去查找信息。职业程序员会好几门语言，并按需要学习新的语言。不存在适合所有人、所有应用的“唯一真语言”。事实上，我们能想到的所有大型系统都使用了不止一种语言。

22.2.8 信息来源

上面每种语言的介绍都附有一份参考列表。下面这些参考则涵盖多种语言：

- 更多语言设计者的链接与照片：www.angelfire.com/tx4/cus/people/。
- 一些语言的示例：http://dmoz.org/Computers/Programming/Languages/。

教材

- Scott, Michael L. Programming Language Pragmatics. Morgan Kaufmann, 2000. ISBN 1558604421.
- Sebesta, Robert W. Concepts of Programming Languages. Addison-Wesley, 2003. ISBN 0321193628.

历史类著作

- Bergin, T. J., and R. G. Gibson, eds. History of Programming Languages —II. Addison-Wesley, 1996. ISBN 0201895021.
- Hailpern, Brent, and Barbara G. Ryder, eds. Proceedings of the Third ACM SIGPLAN Conference on the History of Programming Languages (HOPL-III). San Diego, CA, 2007. http://portal.acm.org/toc.cfm?id=1238844.
- Lohr, Steve. Go To: The Story of the Math Majors, Bridge Players, Engineers, Chess Wizards, Maverick Scientists and Iconoclasts—The Programmers Who Created the Software Revolution. Basic Books, 2002. ISBN 978-0465042265.
- Sammet, Jean. Programming Languages: History and Fundamentals. Prentice Hall, 1969. ISBN 0137299885.
- Wexelblat, Richard L., ed. History of Programming Languages. Academic Press, 1981. ISBN 0127450408.

要点回顾

1. 历史有哪些用处？
2. 编程语言有哪些用处？举例说明。
3. 列出编程语言的一些客观上优良的基本性质。
4. 我们所说的抽象是什么意思？更高的抽象层次又是什么意思？
5. 我们对代码的四个高层次理想是什么？
6. 列出高层次编程的一些潜在优势。
7. 什么是复用？它能带来什么好处？
8. 什么是过程式编程？给出一个具体例子。
9. 什么是数据抽象？给出一个具体例子。

10. 什么是面向对象编程？给出一个具体例子。
11. 什么是泛型编程？给出一个具体例子。
12. 什么是多范式编程？给出一个具体例子。
13. 第一段程序是什么时候在存储程序计算机上运行的？
14. 什么工作使 David Wheeler 引人注目？
15. John Backus 的第一门语言的主要贡献是什么？
16. Grace Murray Hopper 设计的第一门语言是什么？
17. John McCarthy 主要从事计算机科学的哪个领域？
18. Peter Naur 对 Algol60 有哪些贡献？
19. 什么工作使 Edsger Dijkstra 引人注目？
20. Niklaus Wirth 设计并实现了哪些语言？
21. Anders Hejlsberg 设计了哪些语言？
22. Jean Ichbiah 在 Ada 项目中担任什么角色？
23. Simula 开创了哪种编程风格？
24. Kristen Nygaard 在（奥斯陆以外的）哪里任教？
25. 什么工作使 Ole-Johan Dahl 引人注目？
26. Ken Thompson 是哪个操作系统的主要设计者？
27. 什么工作使 Doug McIlroy 引人注目？
28. Brian Kernighan 最著名的书是哪一本？
29. Dennis Ritchie 在哪里工作？
30. 什么工作使 Bjarne Stroustrup 引人注目？
31. Alex Stepanov 在设计 STL 时用过哪些语言？
32. 列出 §22.2 未介绍的十种语言。
33. Scheme 是哪门语言的方言？
34. C++ 两个最著名的祖先是哪两门语言？
35. C++ 中的“C”代表什么？
36. Fortran 是首字母缩写吗？如果是，代表什么？
37. COBOL 是首字母缩写吗？如果是，代表什么？
38. Lisp 是首字母缩写吗？如果是，代表什么？
39. Pascal 是首字母缩写吗？如果是，代表什么？
40. Ada 是首字母缩写吗？如果是，代表什么？
41. 哪门编程语言最好？

术语

本章的“术语”其实是语言、人物和组织。

语言

- Ada
- Algol
- BCPL
- C
- C++

- COBOL
- Fortran
- Lisp
- Pascal
- Scheme
- Simula

人物

- Charles Babbage
- John Backus
- Ole-Johan Dahl
- Edsger Dijkstra
- Anders Hejlsberg
- Grace Murray Hopper
- Jean Ichbiah
- Brian Kernighan
- John McCarthy
- Doug McIlroy
- Peter Naur
- Kristen Nygaard
- Dennis Ritchie
- Alex Stepanov
- Bjarne Stroustrup
- Ken Thompson
- David Wheeler
- Niklaus Wirth

组织

- 贝尔实验室
- Borland
- 剑桥大学（英格兰）
- ETH（瑞士联邦理工学院）
- IBM
- MIT
- 挪威计算中心
- 普林斯顿大学
- 斯坦福大学
- 哥本哈根技术大学
- 美国国防部
- 美国海军

习题

1. 给“编程”下定义。

2. 给“编程语言”下定义。
3. 翻看本书各章的章首小图。哪些小图来自计算机科学家？用一段话概括这些科学家各自的贡献。
4. 翻看本书各章的章首小图。哪些小图不是来自计算机科学家？指出每个人来自哪个国家、从事什么领域的工作。
5. 用本章提到的每一种语言写一个“Hello, World!”程序。
6. 对本章提到的每一种语言，找一本流行的教材，看看它用什么程序作为第一个完整程序。把那个程序用其他所有语言写一遍。提醒：这很容易变成一个上百个程序的项目。
7. 我们显然“漏掉”了许多重要的语言。特别是 C++ 之后的发展，我们基本上都略去了。列出你认为应该介绍的 5 门现代语言，并仿照本章各语言的写法，为其中 3 门各写一页半。
8. C++ 用来做什么？为什么？写一份 10 到 20 页的报告。
9. C 用来做什么？为什么？写一份 10 到 20 页的报告。
10. 挑一门语言（不是 C 或 C++），写一份 10 到 20 页的介绍，说明它的起源、目标和设施。给出大量具体例子。谁在使用它？用来做什么？
11. 目前谁担任剑桥大学的卢卡斯数学教授（Lucasian Chair）?
12. 本章提到的语言设计者中，谁有数学学位？谁没有？
13. 本章提到的语言设计者中，谁有博士学位？在哪个领域？谁没有博士学位？
14. 本章提到的语言设计者中，谁获得过图灵奖？图灵奖是什么？找出这里提到的获奖者实际的图灵奖颁奖词。
15. 写一个程序：给定一个（名称, 年份）对的文件，例如 (Algol,1960) 和 (C,1974)，把各个名称画在一条时间轴上。
16. 修改上一题的程序，使它读取（名称, 年份,(祖先)）三元组的文件，例如 (Fortran,1956,())、(Algol,1960,(Fortran)) 和 (C++,1985,(C,Simula))，并把这些语言画在一条时间轴上，用箭头从祖先指向后代。用这个程序画出 §22.2.2 和 §22.2.7 中那些图的改进版本。

后记

显然，无论是编程语言的历史，还是推动人们追求更好软件的那些理想，我们都只是触及了表面。我们认为历史与理想足够重要，因此对此深感遗憾。我们希望已经传达出我们的一部分兴奋之情，也让你对“追求更好的软件、更好的编程”这一事业的宏大有所体会——它正是通过编程语言的设计与实现体现出来的。话虽如此，请记住：编程——开发高质量的软件——才是根本的、重要的主题；编程语言只是为此服务的工具。

23 文本处理

"Nothing is so obvious that it's obvious . . . The use of the word 'obvious' indicates the absence of a logical argument."

（没有什么东西是显而易见到不言自明的……使用"显而易见"这个词，恰恰说明缺少了逻辑论证。）

——埃罗尔·莫里斯

本章主要讨论如何从文本中提取信息。我们把大量知识以词语的形式存放在文档里，例如书籍、电子邮件或"印刷"表格；可到后来，又不得不把它提取成更适合计算的形式。这里，我们回顾文本处理中最常用的标准库设施：`string`、`iostream` 和 `map`。接着，我们介绍正则表达式（`regex`），用它来表达文本中的模式。最后，我们演示如何用正则表达式在文本中查找并提取特定的数据元素，例如 ZIP 码（邮政编码），以及如何校验文本文件的格式。

23.1 文本
23.2 字符串
23.3 I/O 流
23.4 映射
  23.4.1 实现细节
23.5 一个问题
23.6 正则表达式的思想
  23.6.1 原始字符串字面量
23.7 用正则表达式搜索
23.8 正则表达式语法
  23.8.1 字符与特殊字符
  23.8.2 字符类
  23.8.3 重复
  23.8.4 分组
  23.8.5 择一
  23.8.6 字符集与范围
  23.8.7 正则表达式错误
23.9 用正则表达式匹配
23.10 参考文献

23.1 文本

我们几乎无时无刻不在处理文本。我们的书里满是文本，我们在计算机屏幕上看到的东西大多也是文本，而我们的源代码同样是文本。我们的各种沟通渠道里都充斥着词语。两个人之间交流的一切都可以表示成文本，但也不必走极端。图像和声音通常最好还是用图像和声音来表示（也就是一堆二进制位），除此之外的几乎一切，都适合用程序去做文本分析和变换。

从第 3 章起我们就一直在用 `iostream` 和 `string`，所以这里只简要回顾这两个库。映射（§23.4）对文本处理特别有用，因此我们给出一个用它分析电子邮件的例子。回顾完这些之后，本章就转向用正则表达式在文本中搜索模式（§23.5–10）。

23.2 字符串

string 中保存着一个字符序列，并提供若干有用的操作，例如向 string 添加一个字符、求 string 的长度、拼接 string。其实标准 string 提供的操作相当多，但大多数只在你必须于底层做相当复杂的文本处理时才有用。这里我们只提几个比较有用的。若需要，你可以在手册或专家级教材中查阅它们的细节（以及完整的 string 操作集合）。它们定义在 <string> 中（注意：不是 <string.h>）：

选定的 string 操作

`s1 = s2`	把 s2 赋给 s1；s2 可以是 string，也可以是 C 风格字符串。
`s += x`	在末尾追加 x；x 可以是字符、string 或 C 风格字符串。
`s[i]`	下标访问。
`s1+s2`	拼接；结果 string 中的字符，是 s1 中各字符的副本后面接上 s2 中各字符的副本。
`s1==s2`	比较 string 的值；s1 或 s2 中可以有且仅有一个是 C 风格字符串。还有 `!=`。
`s1<s2`	按字典序比较 string 的值；s1 或 s2 中可以有且仅有一个是 C 风格字符串。还有 `<=`、`>` 和 `>=`。
`s.size()`	s 中字符的个数。
`s.length()`	s 中字符的个数。
`s.c_str()`	s 中字符的 C 风格版本。
`s.begin()`	指向第一个字符的迭代器。
`s.end()`	指向 s 末尾之后位置的迭代器。
`s.insert(pos,x)`	在 `s[pos]` 之前插入 x；x 可以是 string 或 C 风格字符串。s 会扩展，为 x 中的字符腾出空间。
`s.append(x)`	在 s 的最后一个字符之后插入 x；x 可以是 string 或 C 风格字符串。s 会扩展，为 x 中的字符腾出空间。
`s.erase(pos)`	删除 s 中从 `s[pos]` 开始的所有尾随字符。s 的大小变为 pos。
`s.erase(pos,n)`	从 `s[pos]` 开始删除 n 个字符。s 的大小变为 `max(pos,size-n)`。
`pos = s.find(x)`	在 s 中查找 x；x 可以是字符、string 或 C 风格字符串；pos 是找到的第一个字符的下标，或者 `string::npos`（s 末尾之外的位置）。
`in>>s`	从 in 读入一个以空白分隔的单词到 s。
`getline(in,s)`	从 in 读入一行到 s。
`out<<s`	把 s 写到 out。

I/O 操作在第 10 章和第 11 章中讲解，并在 §23.3 中做了小结。注意，向 string 输入的操作会按需扩展 string，因此不会发生溢出。

insert() 和 append() 操作可能会移动字符，以便为新字符腾出空间。erase() 操作则把字符在 string 中“向前”移动，确保我们删除字符的位置不会留下空隙。

标准库的 string 其实是一个模板，名为 basic_string，它支持多种字符集，例如 Unicode，可提供成千上万个字符（除了“普通字符”之外，还有 £、Ω、μ、δ 以及笑脸、音符之类的符号）。例如，如果你有一个用来保存 Unicode 字符的类型，比如 Unicode，就可以写

```
basic_string<Unicode> a_unicode_string;
```

我们一直在用的标准 string，也就是 string，不过是普通 char 的 basic_string：

```
using string = basic_string<char>;      // string 就是 basic_string<char> (§20.5)
```

这里我们不讨论 Unicode 字符或 Unicode 字符串，但如果你需要，可以自行查阅；你会发现它们（在语言层面、在 string 中、在 iostream 中、在正则表达式中）的处理方式与普通字符和字符串大体相同。如果你需要使用 Unicode 字符，最好向有经验的人请教；要想真正有用，你的代码不仅要遵守语言规则，还要遵循某些系统约定。

在文本处理的语境中，有一点很重要：几乎任何东西都可以表示成字符串。例如，就在这一页上，数 12.333 就是表示成由六个字符组成的字符串（前后还有空白）。如果我们读入这个数，就必须先把这些字符转换成浮点数，才能对它做算术运算。于是就需要在值与字符串之间做双向转换。在 §11.4 中，我们看到了如何用 ostringstream 把整数变成字符串。这一技术可以推广到任何具有 << 运算符的类型：

```
template<typename T> string to_string(const T& t)
{
    ostringstream os;
    os << t;
    return os.str();
}
```

例如：

```
string s1 = to_string(12.333);
string s2 = to_string(1+5*6-99/7);
```

现在 s1 的值是 "12.333"，s2 的值是 "17"。事实上，to_string() 不仅可用于数值，还可用于任何具有 << 运算符的类 T。

反方向的转换——从字符串到数值——差不多同样简单，也同样有用：

```
struct bad_from_string : std::bad_cast {  // 报告字符串转换错误的类
    const char* what() const override
    {
        return "bad cast from string";
    }
};

template<typename T> T from_string(const string& s)
{
    istringstream is {s};
    T t;
    if (!(is >> t)) throw bad_from_string{};
    return t;
}
```

例如：

```
double d = from_string<double>("12.333");

void do_something(const string& s)
try
{
    int i = from_string<int>(s);
    // . . .
}
catch (bad_from_string e) {
    error("bad input string",s);
}
```

from_string() 比 to_string() 多出来的复杂性，源于一个字符串可以表示多种类型的值。这意味着我们必须指明想从字符串中提取哪种类型的值。这也意味着我们正在查看的字符串未必保存着我们期望的那种类型的值的表示。例如：

```
int d = from_string<int>("Mary had a little lamb");          // 哎呀!
```

所以存在出错的可能，我们用异常 bad_from_string 来表示它。在 §23.9 中我们会演示：from_string()（或等价的函数）对于严肃的文本处理是必不可少的，因为我们需要从文本字段中提取数值。在 §16.4.3 中，我们看到过等价的函数 get_int() 是如何用在图形用户界面代码中的。

请注意 to_string() 与 from_string() 在功能上多么相似。事实上，它们大致互为逆运算；也就是说（忽略空白、舍入等细节），对每个“合理的类型 T”都有

```
s==to_string(from_string<T>(s))        // 对所有 s
```

以及

```
t==from_string<T>(to_string(t))        // 对所有 t
```

这里的“合理”是指：T 应当定义了默认构造函数、>> 运算符，以及与之匹配的 << 运算符。

还要注意，to_string() 和 from_string() 的实现都借助 stringstream 来完成所有繁重的工作。基于这一点，人们定义了任意两个具有匹配的 << 与 >> 运算的类型之间的通用转换操作：

```
template<typename Target, typename Source>
Target to(Source arg)
{
    stringstream interpreter;
    Target result;
    if (!(interpreter << arg)                    // 把 arg 写入流
            || !(interpreter >> result)          // 从流中读回 result
            || !(interpreter >> std::ws).eof())  // 流中还剩着字符吗？
        throw runtime_error{"to<>() failed"};
    return result;
}
```

那个古怪而巧妙的 !(interpreter>>std::ws).eof() 会读掉我们在提取结果之后可能残留在 stringstream 中的空白。空白是允许的，但输入中不应再有别的字符，而我们可以通过判断是否已到“文件末尾”来检查这一点。所以，如果我们想从字符串中读出一个 int，那么 to<int>("123") 和 to<int>("123 ") 都会成功，而 to<int>("123.5") 会失败，因为末尾多了一个.5。

23.3 I/O 流

既然说到字符串与其他类型之间的联系，就自然要谈到 I/O 流。I/O 流库并不只是做输入输出，它还在字符串格式与内存中的类型之间进行转换。标准库的 I/O 流提供了读取、写入和格式化字符串的设施。iostream 库在第 10 章和第 11 章中讲述，所以这里只做小结：

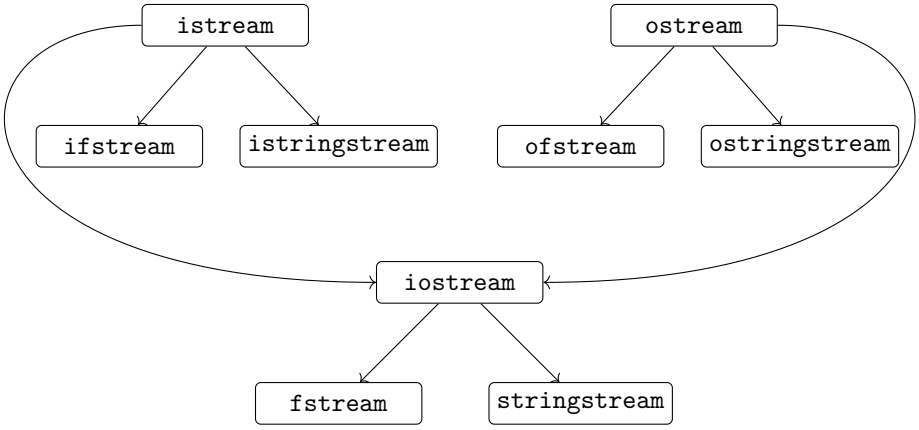
流 I/O

in >> x	按 x 的类型从 in 读入到 x。
out << x	按 x 的类型把 x 写到 out。
in.get(c)	从 in 读入一个字符到 c。
getline(in,s)	从 in 读入一行到 string s。

标准流被组织成一个类层次结构（§14.3）：

这些类合起来，使我们能够对文件和字符串（以及任何能被弄得像文件或字符串的东西，例如键盘和屏幕；见第 10 章）进行 I/O。正如第 10 章和第 11 章所述，iostream 提供了相当精细的格式化设施。图中的箭头表示继承关系（见 §14.3），因此，例如一个 stringstream 既可以当作 iostream 使用，也可以当作 istream 或 ostream 使用。

与 string 一样，iostream 也可以用于 Unicode 这样更大的字符集，处理方式与普通字符大体相同。请再次注意：如果你需要使用 Unicode 的 I/O，最好向有经验的人请教；要想真正有用，你的代码不仅要遵守语言规则，还要遵循某些系统约定。



23.4 映射

关联数组（映射、哈希表）对大量文本处理来说是“关键”（key，双关语）。原因很简单：当我们处理文本时，我们在收集信息，

而这些信息往往与文本字符串联系在一起，例如姓名、地址、邮政编码、社会保障号码、职务名称等。即使其中有些文本字符串可以转换成数值，把它们当作文本处理、并用这些文本进行标识，通常也更方便、更简单。第 21.6 节的单词计数例子就是一个很好的简单示例。如果你对 map 的用法还不放心，请在往下读之前重读 §21.6。

考虑电子邮件。我们经常搜索和分析电子邮件消息与邮件日志——通常借助某个程序（例如 Thunderbird 或 Outlook）。多数情况下，这些程序让我们不必看到消息的完整源码，但关于谁发送、谁接收、消息沿途经过了哪里，以及更多其他内容的信息，都是以消息头中文本的形式呈现在程序面前的。这些信息就构成了一条完整的消息。分析消息头的工具有成千上万种。大多数工具先使用正则表达式（如 §23.5–9 所述）提取信息，再用某种关联数组把相关的消息关联起来。例如，我们常常搜索一个邮件文件，把同一发件人、同一主题或者含有某个特定主题信息的消息全部收集起来。

下面，我们用一个非常简化的邮件文件来说明从文本文件中提取数据的一些技术。其中的消息头是取自 www.faqs.org/rfcs/rfc2822.html 的真实 RFC2822 消息头。请看：

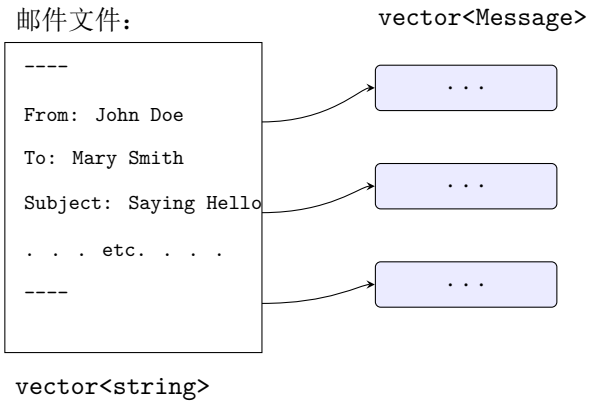
```
xxx
xxx
----
From: John Doe <jdoe@machine.example>
To: Mary Smith <mary@example.net>
Subject: Saying Hello
Date: Fri, 21 Nov 1997 09:55:06 -0600
Message-ID: <1234@local.machine.example>

This is a message just to say hello.
So, "Hello".
----
From: Joe Q. Public <john.q.public@example.com>
To: Mary Smith <@machine.tld:mary@example.net>, , jdoe@test.example
Date: Tue, 1 Jul 2003 10:52:37 +0200
Message-ID: <5678.21-Nov-1997@example.com>
```

```
Hi everyone.
----
To: "Mary Smith: Personal Account" <smith@home.example>
From: John Doe <jdoe@machine.example>
Subject: Re: Saying Hello
Date: Fri, 21 Nov 1997 11:00:00 -0600
Message-ID: <abcd.1234@local.machine.tld>
In-Reply-To: <3456@example.net>
References: <1234@local.machine.example> <3456@example.net>

This is a reply to your reply.
----
----
```

基本上，我们对文件做了缩略，丢弃了大部分信息；同时，为了便于分析，我们用一行只含 —-（四个连字符）的内容来结束每一条消息。我们将编写一个小“玩具程序”，找出“John Doe”发送的所有消息，并输出它们的“Subject”。只要能做成这件事，我们就能做成许多有意思的事。

首先，我们必须考虑：是要对数据做随机访问，还是只让数据依次流过输入流、边流边分析。我们选择前者，因为在真实的程序里，我们多半会对多个发件人感兴趣，或者对某个发件人的多条信息感兴趣。而且，随机访问实际上是两种做法中较难的一种，因此它能让我们考察更多的技术。尤其是，我们又可以重新用上迭代器了。

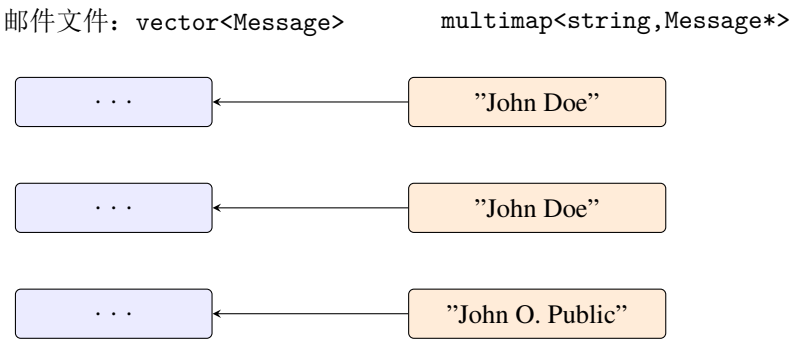
我们的基本想法是：把整个邮件文件读入一个结构（我们把它称为 `Mail_file`）。这个结构将保存邮件文件的所有行（放在一个 `vector<string>` 中），以及每条消息起止位置的指示信息（放在一个 `vector<Message>` 中）：



在此基础上，我们再加上迭代器以及 `begin()` 和 `end()` 函数，这样就能以惯常的方式遍历各行、遍历各条消息。这段“样板代码”会让我们方便地访问各条消息。有了它，我们就可以编写那个“玩具程序”，把每个发件人的所有消息归拢到一起，使它们易于一起访问：

最后，我们把“John Doe”发来的所有消息的 Subject 头都输出来，以演示我们建立的这些访问结构的用法。

我们用到了许多基本的标准库设施：




```
#include<string>
#include<vector>
#include<map>
#include<fstream>
#include<iostream>
using namespace std;
```

我们把 Message 定义为一对指向 vector<string>（也就是我们的行向量）的迭代器：

```
typedef vector<string>::const_iterator Line_iter;

class Message {   // 一条 Message 指向一条消息的首行和末行
    Line_iter first;
    Line_iter last;
public:
    Message(Line_iter p1, Line_iter p2) :first{p1}, last{p2} { }
    Line_iter begin() const { return first; }
    Line_iter end() const { return last; }
    // . . .
};
```

我们把 Mail_file 定义为一个结构，用来保存文本行和消息：

```
using Mess_iter = vector<Message>::const_iterator;

struct Mail_file {                  // 一个 Mail_file 保存某个文件的所有行
                                    // 并简化对消息的访问
    string name;                    // 文件名
    vector<string> lines;           // 按顺序保存的各行
    vector<Message> m;              // 按顺序保存的各条消息

    Mail_file(const string& n);     // 把文件 n 读入 lines

    Mess_iter begin() const { return m.begin(); }
    Mess_iter end() const { return m.end(); }
};
```

注意我们是怎样给这些数据结构加上迭代器的：这样就能方便地按部就班地遍历它们。这里我们其实不会用到标准库算法，但如果我们想用，迭代器已经备好了。

要在消息中查找并提取信息，我们需要两个辅助函数：

```
// 找出一条 Message 中发件人的名字;
// 如果找到就返回 true,
// 如果找到，就把发件人的名字放进 s:
bool find_from_addr(const Message* m, string& s);

// 返回该 Message 的主题；如果没有主题，则返回 "":
string find_subject(const Message* m);
```

最后，我们可以编写代码来从文件中提取信息：

```
int main()
{
    Mail_file mfile {"my-mail-file.txt"};    // 从文件初始化 mfile

    // 先把每个发件人的消息收集到一个 multimap 中:
    multimap<string, const Message*> sender;

    for (const auto& m : mfile) {
        string s;
        if (find_from_addr(&m,s))
            sender.insert(make_pair(s,&m));
    }

    // 现在遍历这个 multimap,
    // 提取 John Doe 的消息的主题:
    auto pp = sender.equal_range("John Doe <jdoe@machine.example>");
    for(auto p = pp.first; p!=pp.second; ++p)
        cout << find_subject(p->second) << '\n';
}
```

让我们详细考察 map 的用法。我们之所以用 multimap（§20.10、§B.4），是因为想把来自同一地址的多条消息归拢到一处。标准库的 multimap 正好能做到这一点（它使访问具有相同键的元素变得容易）。显然（也很典型），我们的任务分为两部分：

- 建立 map。
- 使用 map。

我们遍历所有消息，并用 insert() 把它们插入 multimap，从而建立起这个 multimap：

```
for (const auto& m : mfile) {
    string s;
    if (find_from_addr(&m,s))
        sender.insert(make_pair(s,&m));
}
```

进入 map 的是一个（键，值）对，我们用 `make_pair()` 来生成它。我们用“自制的”`find_from_addr()` 找出消息发送者的名字。

为什么我们先要把 `Message` 放进一个 `vector`，然后再去建立 `multimap`？为什么不直接把 `Message` 放进 map？原因既简单又根本：

- 首先，我们建立一个通用的结构，它可以用于许多事情。
- 然后，我们把它用于某个特定的应用。

这样一来，我们就积累起一组或多或少可复用的组件。假如我们当初直接在 `Mail_file` 里建一个 map，那么每当我们想完成另一件不同的事情时，就不得不重新定义它。特别是，我们的 `multimap`（它的名字叫 sender，这个命名是有讲究的）是按照消息的 Address 字段排序的。大多数其他应用不会觉得这个顺序特别有用：它们可能关注的是 Return 字段、Recipients、Copy-to 字段、Subject 字段、时间戳，等等。

这种分阶段（或者说分层，因为这些部分有时就叫层）构建应用的方式，可以极大地简化程序的设计、实现、文档编写和维护。关键在于：每一部分只做一件事，并且以直截了当的方式去做。另一方面，想一次把所有事情都做完，那就得靠小聪明了。显然，我们那个“从邮件头中提取信息”的程序只是一个很小的应用示例。把不同的东西分开、模块化、逐步构建应用——这些做法的价值会随着程序规模的增长而增大。

要提取信息，我们只需用 `equal_range()` 函数（§B.4.10）找出所有键为 `"John Doe"` 的条目，然后遍历 `equal_range()` 返回的区间 `[first,second)` 中的所有元素，用 `find_subject()` 把主题提取出来：

```
auto pp = sender.equal_range("John Doe <jdoe@machine.example>");
for (auto p = pp.first; p!=pp.second; ++p)
    cout << find_subject(p->second) << '\n';
```

当我们遍历一个 map 的元素时，得到的是一系列（键，值）对；与所有 pair 一样，第一个元素（这里是 string 键）称为 `first`，第二个元素（这里是 Message 值）称为 `second`（§21.6）。

23.4.1 实现细节

显然，我们需要实现这些用到的函数。本来很想把它留作习题，好省下几棵树（纸），但我们还是决定把这个例子做完整。`Mail_file` 的构造函数打开文件，并构造 `lines` 和 `m` 两个向量：

```
Mail_file::Mail_file(const string& n)
    // 打开名为 n 的文件
    // 从 n 中读出各行，存入 lines
    // 在 lines 中找出各条消息，并把它们组装进 m
    // 为简单起见，假定每条消息都以一行 ---- 结束
{
    ifstream in {n};                    // 打开文件
    if (!in) {
        cerr << "no " << n << '\n';
        exit(1);                        // 终止程序
    }

    for (string s; getline(in,s); )     // 建立行的向量
        lines.push_back(s);

    auto first = lines.begin();         // 建立 Message 的向量
    for (auto p = lines.begin(); p!=lines.end(); ++p) {
        if (*p == "----") {             // 消息结束
            m.push_back(Message(first,p));
            first = p+1;                // ---- 不属于消息
        }
    }
}
```

这里的错误处理很简陋。如果这是一个我们打算交给朋友使用的程序，那就得做得更好。

试一试

我们是认真的：请真的运行这个示例，并确保你理解了结果。什么叫“更好的错误处理”？请修改 `Mail_file` 的构造函数，让它能处理与 `---` 的使用有关的常见格式错误。

在我们能更好地识别文件中的信息（用正则表达式，见 §23.6–10）之前，`find_from_addr()` 和 `find_subject()` 这两个函数只是简单的占位实现：

```
int is_prefix(const string& s, const string& p)
    // p 是 s 的开头部分吗？
{
    int n = p.size();
    if (string(s,0,n)==p) return n;
    return 0;
}
```

```
bool find_from_addr(const Message* m, string& s)
{
    for (const auto& x : m)
        if (int n = is_prefix(x, "From: ")) {
            s = string(x,n);
            return true;
        }
    return false;
}

string find_subject(const Message* m)
{
    for (const auto& x : m)
        if (int n = is_prefix(x, "Subject: ")) return string(x,n);
    return "";
}
```

注意我们使用子串的方式：`string(s,n)` 构造一个由 s 从 `s[n]` 起的尾部（即 `s[n]..s[s.size()-1]`）构成的 string；而 `string(s,0,n)` 构造一个由字符 `s[0]..s[n-1]` 构成的 string。由于这些操作实际上会构造新的 string 并复制字符，在性能重要的地方应当慎用。

为什么 `find_from_addr()` 和 `find_subject()` 这两个函数差别这么大？例如，一个返回 bool，另一个返回 string。它们之所以不同，是因为我们想说明一点：

- `find_from_addr()` 能区分“找到了一行地址、但地址为空（””）”与“根本没有地址行”这两种情况。在前一种情况下，`find_from_addr()` 返回 true（因为它找到了一行地址），并把 s 置为”” （因为这个地址碰巧是空的）。在后一种情况下，它返回 false（因为不存在地址行）。
- `find_subject()` 无论是主题为空，还是根本没有主题行，都返回””。

`find_from_addr()` 做出的这种区分有用吗？必要吗？我们认为这种区分可能有用，而且我们肯定应当意识到它的存在。在数据文件中查找信息时，这种区分会一再出现：我们找到想找的字段了吗？字段里有有用的内容吗？在真实的程序里，`find_from_addr()` 和 `find_subject()` 两个函数都会按照 `find_from_addr()` 的风格来写，以便用户能做出这种区分。

这个程序没有针对性能做优化，但对大多数用途来说大概已经够快了。特别是，它只读取输入文件一次，而且它不保留……

该文件文本的多份副本。对于大文件，把 `multimap` 换成 `unordered_multimap` 也许是个好主意，但除非你实际测量，否则永远也不会知道。

关于标准库关联容器（`map`、`multimap`、`set`、`unordered_map` 和 `unordered_multimap`）的介绍见 §21.6。

23.5 一个问题

输入输出流和字符串帮助我们读写字符序列、存储字符序列，并协助完成基本的操作。然而，我们经常需要对文本做这样的操作：必须考虑某个字符串的上下文，或者涉及许多相似的字符串。考虑一个简单的例子。取一条电子邮件消息（一个由单词构成的序列），看看其中是否包含美国的州名缩写和邮政编码（两个字母后跟五位数字）：

```
for (string s; cin>>s; ) {
    if (s.size()==7
        && isalpha(s[0]) && isalpha(s[1])
        && isdigit(s[2]) && isdigit(s[3]) && isdigit(s[4])
        && isdigit(s[5]) && isdigit(s[6]))
        cout << "found " << s << '\n';
}
```

这里，若 x 是字母则 isalpha(x) 为真，若 x 是数字则 isdigit(x) 为真（见 §11.6）。

这个简单的（过于简单的）解决方案存在几个问题：

- 它很罗嗦（四行代码，八次函数调用）。
- 我们会漏掉（有意漏掉？ ）每一个没有用空白与上下文隔开的邮政编码（例如 "TX77845"、TX77845–1234 和 ATX77845）。
- 我们会漏掉（有意漏掉?）每一个字母与数字之间带空格的邮政编码（例如 TX 77845）。
- 我们会接受（有意接受?）每一个字母为小写的邮政编码（例如 tx77845）。
- 如果我们决定查找另一种格式的邮政编码（例如 CB3 0FD），就必须把代码整个重写。

一定存在更好的办法！在揭示那个办法之前，我们先来考虑一下：如果决定固守“老一套简单做法”，靠写更多代码来处理更多情况，会碰到哪些问题。

- 如果想处理不止一种格式，我们就得开始添加 if 语句或 switch 语句。
- 如果想同时处理大写和小写，我们就得显式转换（通常转成小写），或者再加上一个 if 语句。
- 我们需要以某种方式（什么方式?）描述所要查找内容的前后文。这意味着我们必须处理单个字符而不是字符串，而这又意味着我们会失去 iostream 提供的许多好处（见 §7.8.2）。

如果你愿意，可以试着自己把那样的代码写出来，但很明显，沿着这条路走下去，我们会陷入一堆处理各种特殊情况的 if 语句所造成的混乱。即便对这么简单的例子，我们也必须处理备选情况（例如五位和九位的邮政编码）。对许多其他例子，我们还必须处理重复（例如任意数量的数字后跟一个感叹号，如 123! 和 123456!）。最终，我们还得同时处理前缀和后缀。正如我们已观察到的（见 §11.1-2），人们在输出格式上的口味，并不受程序员对规则性和简单性的愿望所限制。只要想想人们写日期的五花八门的方式：

```
2007-06-05
June 5, 2007
jun 5, 2007
5 June 2007
6/5/2007
5/6/07
. . .
```

到了这一步——如果不是更早的话——有经验的程序员就会宣称：“一定存在更好的办法!”（比写更多普通代码更好的办法），然后动手去寻找它。最简单也最流行的解决方案，就是使用所谓的正则表达式（regular expression）。正则表达式是大量文本处理工作的支柱，是 Unix grep 命令的基础（见习题 8），也是那些大量用于此类处理的编程语言（如 AWK、PERL 和 PHP）的重要组成部分。

我们将要使用的正则表达式是 C++ 标准库的一部分。它们与 PERL 中的正则表达式兼容。这使得许多解释、教程和手册都能为我们所用。例如，可以看 C++ 标准委员会的工作文件（在网上搜索

“WG21”）、John Maddock 的 `boost::regex` 文档，以及大多数 PERL 教程。这里，我们将描述正则表达式的基本概念，以及一些最基本、最有用的用法。

试一试

试一试

上一段和再上一段“不经意地”用了几个名称和缩写，却没有做任何解释。花点时间上网浏览一下，弄清我们指的是什么。

23.6 正则表达式的概念

正则表达式的基本思想是：它定义了一个模式，我们可以在文本中查找这个模式。想一想，怎样才能简洁地描述一个简单的美国邮政编码（例如 `TX77845`）的模式。下面是第一次尝试：

```
wwddddd
```

这里，`w` 表示“任意字母”，`d` 表示“任意数字”。我们用 `w`（word，单词）而不用 `l`（letter，字母），因为 `l` 太容易与数字 1 混淆。这个记法在这个简单例子里行得通，不过让我们拿九位邮政编码格式（例如 `TX77845-5629`）试一试。这样如何

```
wwddddd-dddd
```

看起来没问题，可是为什么 `d` 表示“任意数字”，而 `-` 表示“普普通通的”短横线呢？我们总该以某种方式表明 `w` 和 `d` 是特殊的：它们代表字符类，而不是它们自身（`w` 表示“a 或 b 或 c 或…”，`d` 表示“1 或 2 或 3 或…”）。这太微妙了。让我们按 C++ 中一贯表示特殊字符的方式，在一个表示字符类名字的字母前面加上反斜杠（例如字符串字面量里的 `\n` 表示换行）。这样我们就得到

```
\w\w\d\d\d\d\d-\d\d\d\d
```

这有点难看，但至少不含糊，而且反斜杠让人一眼就看出“有不同寻常的事情在发生”。这里，我们靠简单地重复一个字符来表示它的重复。这可能有点繁琐，而且容易出错。快问一句：短横线前的五位数字和短横线后的四位数字，我们真的写对了吗？写对了，但我们从没有在任何地方明确写出 5 和 4，所以你非得数一遍才能确认。我们可以在字符后面加一个计数来表示重复。例如：

```
\w2\d5-\d4
```

不过，我们确实需要某种语法，来表明那个模式中的 2、5、4 是计数，而不只是字母数字字符 2、5 和 4。让我们把计数放进花括号里来表示：

```
\w{2}\d{5}-\d{4}
```

这样一来，左花括号就和 `\`（反斜杠）一样成了特殊字符，但这也无可奈何，我们总能应付。

到目前为止一切顺利，但我们还得处理两个更麻烦的细节：邮政编码中的最后四位数字是可选的。我们总得能够说明：`TX77845` 和 `TX77845-5629` 都接受。表达这一点有两种基本方式：

\w{2}\d{5} 或 \w{2}\d{5}-\d{4}
以及
\w{2}\d{5}，并可选地后跟 -\d{4}

为了简洁而精确地表达这一点，我们首先必须表达分组（或称子模式）的概念，这样才能谈论模式 \w{2}\d{5}-\d{4} 的各个部分。按惯例，我们用圆括号表示分组：

```
(\w{2}\d{5})(-\d{4})
```

现在我们把模式分成了两个子模式，于是只需说明想对它们做什么。和往常一样，引入一项新设施的代价是引入另一个特殊字符：(现在也“特殊”了，正如 \ 和左花括号一样。按惯例，| 用来表示“或”（备选），? 用来表示有条件的（可选的），所以我们也许可以写成

```
(\w{2}\d{5})|(\w{2}\d{5}-\d{4})
```
以及
```
(\w{2}\d{5})(-\d{4})?
```

与计数记法中的花括号（例如 \w{2}）一样，我们把问号（?）用作后缀。例如，(-\d{4})? 的意思是“可选地后跟 -\d{4}”，也就是说，我们接受以短横线开头、后跟四位数字的内容作为后缀。实际上，五位邮政编码模式（\w{2}\d{5}）外面的圆括号并没有派上任何用场，所以我们完全可以把它去掉：

```
\w{2}\d{5}(-\d{4})?
```

为了给出 §23.5 中所提问题的完整解决方案，我们还可以在两个字母后面加上一个可选的空格：

```
\w{2} ?\d{5}(-\d{4})?
```

那个“?”看起来有点怪，但它当然是一个空格字符后跟一个 ?，表示该空格字符是可选的。如果我们不希望空格隐蔽得像个 bug，可以把它放进圆括号里：

```
\w{2}( )?\d{5}((-\d{4})?
```

如果有人觉得这仍然太晦涩，我们可以为空白字符发明一种记法，例如 \s（s 代表“空格”）。这样我们就可以写成

```
\w{2}\s?\d{5}(-\d{4})?
```

但如果有人在两个字母后写了两个空格呢？按目前定义的模式，TX77845 和 TX 77845 都能接受，TX 77845 却不行。这有点微妙。我们需要能够表示“零个或多个空白字符”，于是引入后缀 * 表示“零个或多个”，得到

```
\w{2}\s*\d{5}(-\d{4})?
```

如果你跟着逻辑一步步走下来，这一切都说得通。这种模式记法合乎逻辑，而且极其简洁。此外，我们并不是随意做出这些设计选择的：这种特定的记法极为常见、极受欢迎。对许多文本处理任务来说，你都需要读懂并写出这种记法。是的，它看起来有点像一只猫从键盘上走过；是的，打错一个字符（哪怕是一个空格）就会彻底改变含义；但请你还是习惯它吧。我们提不出明显更好的方案，而这种记法自从最初为 Unix grep 命令引入以来，三十多年间一直风靡一时——即便在当时，它也并非全新的东西。

23.6.1 原始字符串字面量

注意那些正则表达式模式里的反斜杠。要把一个反斜杠（\）放进 C++ 字符串字面量，我们必须先在它前面加一个反斜杠。考虑我们的邮政编码模式：

```
\w{2}\s*\d{5}(-\d{4})?
```

要把那个模式表示成字符串字面量，我们必须写成

```
"\\w{2}\\s*\\d{5}(-\\d{4})?"
```

往前想一步就会意识到：我们想要匹配的许多模式都包含双引号（"）。要把双引号放进字符串字面量，我们必须先在它前面加一个反斜杠。这很快就会变得难以收拾。事实上，在实际使用中，这个“特殊字符问题”如此烦人，以至于 C++ 和其他语言引入了原始字符串字面量（raw string literal）的概念，以便对付现实中的正则表达式模式。在原始字符串字面量中，反斜杠就只是一个反斜杠字符（而不是转义字符），双引号也只是一个双引号字符（而不是字符串的结束）。作为原始字符串字面量，我们的邮政编码模式就变成

```
R"(\w{2}\s*\d{5}(-\d{4})?)"
```

R"(开始这个字符串，)" 结束它，所以这个字符串的 22 个字符是

```
\w{2}\s*\d{5}(-\d{4})?
```

不计末尾的那个零。

23.7 用正则表达式搜索

现在，我们用上一节的邮政编码模式，在一个文件中查找邮政编码。程序先定义模式，然后逐行读取文件，搜索这个模式。如果程序在某一行中发现了该模式的一次出现，它就输出行号以及找到的内容：

```
#include <regex>
#include <iostream>
#include <string>
#include <fstream>
using namespace std;

int main()
{
    ifstream in {"file.txt"};                    // 输入文件
    if (!in) cerr << "no file\n";

    regex pat {R"(\w{2}\s*\d{5}(-\d{4})?)"};    // 邮政编码模式

    int lineno = 0;
```

```
    for (string line; getline(in,line); ) {   // 把输入行读入输入缓冲区
        ++lineno;
        smatch matches;                       // 匹配到的字符串放在这里
        if (regex_search(line, matches, pat))
            cout << lineno << ": " << matches[0] << '\n';
    }
}
```

这需要稍微详细地解释一下。标准库的正则表达式位于 <regex> 中。有了它，我们就可以定义一个模式 pat：

```
regex pat {R"(\w{2}\s*\d{5}(-\d{4})?)"};     // 邮政编码模式
```

正则表达式模式是一种字符串，因此我们可以用字符串来初始化它。这里我们用的是原始字符串字面量。不过，regex 并不只是一个字符串：当你初始化一个 regex（或给它赋值）时所创建的那套相当精巧的模式匹配机制是隐藏起来的，超出了本书的范围。然而，一旦我们用邮政编码模式初始化了一个 regex，就可以把它应用到文件的每一行：

```
smatch matches;
if (regex_search(line, matches, pat))
    cout << lineno << ": " << matches[0] << '\n';
```

regex_search(line,matches,pat) 在 line 中搜索任何与 pat 中保存的正则表达式相匹配的内容；如果找到匹配，就把它们存入 matches。自然，如果没有找到匹配，regex_search(line,matches,pat) 返回 false。

matches 变量的类型是 smatch。其中的 s 代表“sub”（子）或“string”（字符串）。基本上，一个 smatch 就是由 string 类型的子匹配构成的 vector。第一个元素（这里是 matches[0]）是完整的匹配。只要 i<matches.size()，我们就可以把 matches[i] 当作字符串处理。因此，如果对某个正则表达式来说子模式最多有 N 个，我们就会发现 matches.size()==N+1。

那么，什么是子模式？一个好的初步回答是“模式中圆括号里的任何内容”。看看模式 \w{2}\s*\d{5}(-\d{4})?，我们看到邮政编码四位扩展部分外面的圆括号。这是我们能看到的唯一子模式，所以我们猜测（而且猜对了）matches.size()==2。我们还可以猜测，那最后四位数字很容易访问。例如：

```
for (string line; getline(in,line); ) {
    smatch matches;
    if (regex_search(line, matches, pat)) {
        cout << lineno << ": " << matches[0] << '\n';       // 整个匹配
        if (1<matches.size() && matches[1].matched)
            cout  << "\t: " << matches[1] << '\n';          // 子匹配
    }
}
```

严格说来，我们本来不必测试 1<matches.size()，因为我们已经仔细看过模式了，但我们就是想偏执一点（因为我们一直在 pat 里试验各种模式，它们并不都只有一个子模式）。要问某个子匹配是否成功，可以查看它的 matched 成员，这里是 matches[1].matched。顺便说一句：当 matches[i].matched 为 false 时，未匹配的子模式 matches[i] 打印出来是空字符串。同样，不存在的子模式（例如上面模式中的 matches[17]）也会被当作未匹配的子模式处理。

我们用下面这个文件试了这个程序

```
address TX77845
ffff tx 77843 asasasaa
ggg TX3456-23456
howdy
zzz TX23456-3456sss ggg TX33456-1234
cvzcv TX77845-1234 sdsas
xxxTx77845xxx
TX12345-123456
```

然后得到了这样的输出

```
pattern: "\w{2}\s*\d{5}(-\d{4})?"
1: TX77845
2: tx 77843
5: TX23456-3456
          : -3456
6: TX77845-1234
          : -1234
7: Tx77845
8: TX12345-1234
          : -1234
```

请注意，我们

- 没有被以 ggg 开头的那一行里格式不合规的“邮政编码”骗过去（那一行有什么问题?）
- 只用 zzz 那一行找到了第一个邮政编码（我们只要求每行找到一个）
- 在第 5 行和第 6 行找到了正确的后缀
- 在第 7 行的一串 x 当中找到了“隐藏”的邮政编码
- 在 TX12345-123456 中（不幸?）也找到了“隐藏”的邮政编码

23.8 正则表达式语法

我们已经看到一个相当基本的正则表达式匹配示例。现在该更系统、更完整地考察正则表达式（就它们在 `regex` 库中使用的形式而言）了。

正则表达式（regexp 或 regex）本质上是一门描述字符模式的小语言。它强大（富有表现力）而简洁，正因如此也可能相当晦涩。经过几十年的使用，它已经积累了许多微妙的特性，还形成了若干种方言。这里我们只描述目前使用最广的那种方言（PERL 方言）中一个（较大且有用的）子集。如果你需要更多东西来表达自己的意思，或者要读懂别人写的正则表达式，就到网上去找：教程（质量参差不齐）和规范资料应有尽有。

该库还支持 ECMAScript、POSIX、awk、grep 和 egrep 等记法，以及大量搜索选项。这些有时极为有用，特别是当你需要匹配用另一种语言规定的某个模式时。如果你觉得自己需要超出这里所述基本设施的能力，可以去查那些选项。不过请记住：“用上最多的特性”并不是优秀编程的目标。只要能做到，就体谅一下那个不得不阅读并理解你代码的可怜维护者（也许就是几个月后的你自己）：写代码不要做不必要的聪明，尽可能避开晦涩的特性。

23.8.1 字符与特殊字符

正则表达式指定一个模式，用来匹配字符串中的字符。默认情况下，模式里的一个字符匹配字符串中它自身。例如，正则表达式（模式）`"abc"` 会匹配 `Is there an abc here?` 中的 `abc`。

正则表达式的真正威力来自“特殊字符”以及模式中具有特殊含义的字符组合：

具有特殊含义的字符

`.`	任意单个字符（一个“通配符”）
`[`	字符类（character class）
`{`	计数（count）
`(`	开始分组
`)`	结束分组
`\`	使下一个字符具有特殊含义
`*`	零次或多次
`+`	一次或多次
`?`	可选（零次或一次）
`|`	备选（或）
`^`	行首；取反（negation）
`$`	行尾

例如：

```
x.y
```

匹配任何以 `x` 开头、以 `y` 结尾的三字符字符串，例如 `xxy`、`x3y` 和 `xay`，但不匹配 `yxy`、`3xy` 和 `xy`。

注意，`{...}`、`*`、`+` 和 `?` 都是后缀运算符。例如，`\d+` 的意思是“一个或多个十进制数字”。

如果你想在模式中使用某个特殊字符，就必须用反斜杠把它“转义”；例如，在模式中 + 是一次或多次运算符，而 \+ 是一个加号。

23.8.2 字符类

最常见的那些字符组合，都用一种简洁的形式表示成“特殊字符”：

用于字符类的特殊字符

\d	一个十进制数字	[[:digit:]]
\l	一个小写字母	[[:lower:]]
\s	一个空白（空格、制表符等）	[[:space:]]
\u	一个大写字母	[[:upper:]]
\w	一个字母（a-z 或 A-Z）或一个数字（0-9）或一个下划线（_）	[[:alnum:]]
\D	不是 \d	[^[:digit:]]
\L	不是 \l	[^[:lower:]]
\S	不是 \s	[^[:space:]]
\U	不是 \u	[^[:upper:]]
\W	不是 \w	[^[:alnum:]]

注意，大写形式的特殊字符表示“不是该特殊字符的小写版本”。特别地，\W 的意思是“不是字母”，而不是“一个大写字母”。

第三栏中的条目（例如 [[:digit:]]）给出的是使用较长名字的另一种语法。

与 string 和 iostream 库一样，regex 库能够处理大型字符集，例如 Unicode。也像 string 和 iostream 的情况一样，我们提到这一点，只是为了让你在需要时知道去哪里寻求帮助、去哪里找更多信息。Unicode 文本处理超出了本书的范围。

23.8.3 重复

重复出现的那些模式，由下面这几个后缀运算符来指定：

重复

{n}	恰好 n 次
{n,}	n 次或更多
{n,m}	至少 n 次、至多 m 次
*	零次或多次，即 {0,}
+	一次或多次，即 {1,}
?	可选（零次或一次），即 {0,1}

例如：

```
Ax*
```

匹配一个 A 后面跟零个或多个 x，例如

```
A
Ax
Axx
Axxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

如果你想要至少出现一次，那就该用 + 而不是 *。例如：

```
Ax+
```

匹配一个 A 后面跟一个或多个 x，例如

```
Ax
Axx
Axxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

但不匹配

```
A
```

零次或一次（即“可选”）这一常见情形用问号表示。例如：

```
\d-?\d
```

匹配两个数字，它们中间可以有一个短横线，例如

```
1-2
12
```

但不匹配

```
1—2
```

要规定恰好出现多少次，或者规定一个出现次数范围，就使用花括号。例如：

```
\w{2}-\d{4,5}
```

匹配恰好两个字母，后跟一个短横线（-），再跟四位或五位数字，例如

```
Ab-1234
XX-54321
22-54321
```

但不匹配

```
Ab-123
?b-1234
```

是的，数字也属于 \w 字符。

23.8.4 分组

要把一个正则表达式指定为一个子模式，可以用圆括号把它分组。例如：

```
(\d*:)
```

这定义了一个子模式：零个或多个数字，后跟一个冒号。分组可以用在更精巧的模式中。例如：

```
(\d*:)?(\d+)
```

这表示一段可选且可以为空的数字序列，后跟一个冒号，再后跟一段由一个或多个数字组成的序列。难怪人们要发明一种简洁而精确的方式来说这类事情！

23.8.5 备选

“或”字符（|）用来指定一个备选项。例如：

```
Subject: (FW:|Re:)?(.*)
```

它能识别这样的电子邮件主题行：后面可选地跟 FW: 或 Re:，再跟零个或多个字符。例如：

```
Subject: FW: Hello, world!
Subject: Re:
Subject: Norwegian Blue
```

但不匹配

```
SUBJECT: Re: Parrots
Subject  FW: No subject!
```

空的备选项是不允许的：

```
(|def)          // 错误
```

不过，我们可以一次指定几个备选项：

```
(bs|Bs|bS|BS)
```

23.8.6 字符集与范围

特殊字符为最常见的若干字符类提供了简写：数字（\d）；字母、数字和下划线（\w）；等等（§23.7.2）。不过，定义我们自己的字符类既容易又常常有用。例如：

[\w @]	一个单词字符、一个空格，或者一个 @
[a-z]	从 a 到 z 的小写字母
[a-zA-Z]	从 a 到 z 的大写或小写字母
[Pp]	一个大写或小写的 P

（续）

[\w\-]	一个单词字符或一个短横线（单独的-表示“范围”）
[asdfghjkl;']	美式 QWERTY 键盘中间一行的字符
[.]	一个点
[.[{(*+?^$]	在正则表达式中具有特殊含义的一个字符

在字符类的说明中，-（短横线）用来指定一个范围，例如 [1-3]（即 1、2 或 3）和 [w-z]（即 w、x、y 或 z）。请谨慎使用这类范围：并非每种语言都有相同的字母，也并非每种字母编码都有相同的顺序。如果你想用的范围不是英文字母和数字中最常见字符的子范围，就去查文档。

注意，我们可以在字符类的说明中使用特殊字符，例如 \w（意思是“任意单词字符”）。那么，怎样才能把一个反斜杠（\）放进字符类里呢？和往常一样，用反斜杠把它“转义”：\\。

当一个字符类说明的第一个字符是 ^ 时，这个 ^ 表示“取反”。例如：

[^aeiouy]	不是英语元音
[^\d]	不是数字
[^aeiouy]	一个空格、一个 ^，或者一个英语元音

在最后一个正则表达式中，^ 并不是 [之后的第一个字符，所以它只是一个普通字符，而不是取反运算符。正则表达式可以很微妙。

正则表达式的实现还提供一组命名字符类，供匹配时使用。例如，如果你想匹配任意字母数字字符（也就是字母或数字：a-z 或 A-Z 或 0-9），可以用正则表达式 [[:alnum:]]。这里 alnum 是一个字符集合（字母数字字符集）的名字。一个非空、带引号的字母数字字符串，其模式可以写成 "[[:alnum:]]+"。要把这个正则表达式放进普通字符串字面量，我们必须转义其中的引号：

```
string s {"\" [[:alnum:]]+\""};
```

此外，要把那个字符串字面量放进 regex，我们还必须把反斜杠再转义一次：

```
regex s {"\\\" [[:alnum:]]+\\\""};
```

使用原始字符串字面量更简单：

```
regex s2 {R" (" [[:alnum:]]+")"};
```

对于含有反斜杠或双引号的模式，最好使用原始字符串字面量。在许多应用中，大多数模式都属于这种情况。

使用正则表达式会带来一大堆记法约定。不管怎样，下面列出标准的字符类：

字符类	
alnum	任意字母数字字符
alpha	任意字母字符
blank	任意不是行分隔符的空白字符
cntrl	任意控制字符
d	任意十进制数字
digit	任意十进制数字
graph	任意图形字符
lower	任意小写字符
print	任意可打印字符
punct	任意标点字符
s	任意空白字符
space	任意空白字符
upper	任意大写字符
w	任意单词字符（字母数字字符加上下划线）
xdigit	任意十六进制数字字符

regex 的一种实现可能提供更多字符类；但如果你决定使用这里未列出的命名类，一定要核实它对你要用的场合是否足够可移植。

23.8.7 正则表达式的错误

如果我们指定一个非法的正则表达式，会发生什么？考虑：

```
regex pat1 {"(|ghi)"};            // 缺失备选项
regex pat2 {"[c-a]"};             // 不是范围
```

当我们把一个模式赋给 regex 时，这个模式会受到检查；如果因为模式非法或过于复杂，正则表达式匹配器无法用它进行匹配，就会抛出一个 bad_expression 异常。

下面这个小程序有助于体会正则表达式匹配：

```
#include <regex>
#include <iostream>
#include <string>
#include <fstream>
#include<sstream>
using namespace std;

// 从输入接受一个模式，以及一组文本行
// 检查该模式，并搜索符合该模式的行

int main()
{
    regex pattern;

    string pat;
    cout << "enter pattern: ";
    getline(cin,pat);           // 读入模式

    try {
        pattern = pat;      // 这会检查 pat
        cout << "pattern: " << pat << '\n';
    }
    catch (bad_expression) {
        cout << pat << " is not a valid regular expression\n";
        exit(1);
    }

    cout << "now enter lines:\n";
    int lineno = 0;

    for (string line; getline(cin,line); ) {
        ++lineno;
        smatch matches;


        if (regex_search(line, matches, pattern)) {
            cout << "line " << lineno << ": " << line << '\n';
            for (int i = 0; i<matches.size(); ++i)
                cout << "\tmatches[" << i << "]: "
                     <<  matches[i] << '\n';
        }
        else
            cout << "didn't match\n";
    }
}
```

试一试

让这个程序跑起来，并用它试一些模式，例如 `abc`、`x.*x`、`(.*)`、`[)]*` 和 `\w+ \w+( Jr)?`。

23.9 用正则表达式匹配

正则表达式有两种基本用法：

- 在（长度任意的）数据流中查找与某个正则表达式匹配的字符串——`regex_search()` 会把它的模式当作该数据流的子串来查找。
- 把一个正则表达式与一个（长度已知的）字符串匹配——`regex_match()` 要求它的模式与整个字符串完全匹配。

§23.6 中查找邮政编码用的是“查找”。这里我们来看一个“匹配”的例子。考虑从下面这样一张表中提取数据：

KLASSE	ANTAL DRENGE	ANTAL PIGER	ELEVER IALT
0A	12	11	23
1A	7	8	15
1B	4	11	15
2A	10	13	23
3A	10	12	22
4A	7	7	14
4B	10	5	15
5A	19	8	27
6A	10	9	19
6B	9	10	19
7A	7	19	26
7G	3	5	8
7I	7	3	10
8A	10	16	26
9A	12	15	27
0MO	3	2	5
0P1	1	1	2
0P2	0	5	5
10B	4	4	8
10CE	0	1	1
1MO	8	5	13
2CE	8	5	13
3DCE	3	3	6
4MO	4	1	5
6CE	3	4	7
8CE	4	4	8
9CE	4	9	13
REST	5	6	11
Alle klasser	184	202	386

这张表（2007 年 Bjarne Stroustrup 曾就读的小学各班级的学生人数）是从一个网页上摘下来的；在原来的环境里它看起来漂亮整洁，也是我们需要分析的那类数据中相当典型的一种：

- 它有数值字段。
- 它有字符字段，其中的字符串只有了解这张表来龙去脉的人才看得懂。（这里用了丹麦语，这一点就更明显了。）

- 这些字符串中包含空格。
- 这张表数据的各个“字段”由一个“分隔标志”隔开，在这里是制表符。

我们特意选这张表，因为它“相当典型”而又“不太难”；不过要注意一个微妙之处：我们其实分不清空格和制表符，只能把这个问题留给代码去解决。

我们将用正则表达式来说明怎样：

- 检查这张表排得对不对（即每一行的字段数是否正确）；
- 检查各数加起来对不对（最后一行声称是上方各列之和）。

如果这些都能做到，那几乎什么都能做了！例如，我们可以生成一张新表，把首位数字相同（表示年级：一年级以 1 开头）的各行合并起来；也可以看看这些年里学生人数是在增加还是减少（见习题 10 和 11）。

要分析这张表，我们需要两个模式：一个用于标题行，一个用于其余各行：

```
regex header {R"(^[\w ]+(           [\w ]+)*$)"};
regex row {R"(^[\w ]+(                \d+)(  \d+)(   \d+)$)"};
```

请记住，我们称赞过正则表达式语法的简洁和实用，但并没有说它便于初学者理解。事实上，正则表达式素有“只写语言”的名声，而这名声名副其实。让我们从标题行开始。由于它不含任何数值数据，我们本可以把第一行直接丢掉，不过——为了练练手——还是来解析它。它由四个“词字段”（“字母数字字段”）组成，彼此用制表符隔开。这些字段里可以包含空格，所以不能简单地用 `\w` 来指定它的字符。我们改用 `[\w ]`，即一个词字符（字母、数字或下划线）或一个空格。一个或多个这样的字符写作 `[\w ]+`。我们希望其中第一个出现在行首，于是得到 `^[\w ]+`。“帽子”（`^`）表示“行首”。其余每个字段都可以表示为一个制表符后跟若干词字符：`(  [\w ]+)`。现在取其中任意多个，后面再跟一个行尾：`(  [\w ]+)*$`。美元符号（`$`）表示“行尾”。

请注意，我们没法看出制表符真的是制表符；不过在这个例子里，排版时它们被展开，反倒把自己暴露了出来。

现在看这个练习里更有意思的部分：从那些行中提取数值数据所用的模式。第一个字段和前面一样：`^[\w ]+`。它后面恰好跟着三个数值字段，每个前面都有一个制表符，即 `(  \d+)`，于是我们得到

```
^[\w ]+(          \d+)(          \d+)(          \d+)$
```

把它放进原始字符串字面量之后，就是

```
R"(^[\w ]+(          \d+)(          \d+)(          \d+)$)"
```

现在只要用上这些模式就行了。首先我们只验证这张表的版面：

```
int main()
{
    ifstream in {"table.txt"};      // 输入文件
    if (!in) error("no input file\n");

    string line;                    // 输入缓冲区
    int lineno = 0;

    regex header {R"(^[\w ]+(           [\w ]+)*$)"};      // 标题行
    regex row {R"(^[\w ]+(              \d+)(  \d+)(   \d+)$)"};    // 数据行

    if (getline(in,line)) {         // 检查标题行
        smatch matches;
        if (!regex_match(line, matches, header))
            error("no header");
    }
    while (getline(in,line)) {      // 检查数据行
        ++lineno;
        smatch matches;
        if (!regex_match(line, matches, row))
            error("bad line",to_string(lineno));
    }
}
```

为了简短，我们省略了 `#include`。由于我们要检查每一行上的所有字符，所以用 `regex_match()` 而不是 `regex_search()`。两者的区别恰恰在于：`regex_match()` 必须匹配输入中的每一个字符才算成功，而 `regex_search()` 则在输入中寻找能匹配的子串。本想说 `regex_search()` 却错写成 `regex_match()`（或者反过来），会是个极难查出的 bug。不过，这两个函数对 `matches` 参数的用法完全相同。

现在可以接着核对表里的数据了。我们保留男生（“drenge”）列和女生（“piger”）列学生人数的合计。每一行我们都要检查最后一个字段（“ELEVER IALT”）是否真的是前两个字段之和。最后一行（“Alle klasser”）声称是上方各列之和。为了核对这一点，我们修改 `row`，把文本字段变成一个子匹配，这样才能认出“Alle klasser”：

```
int main()
{
    ifstream in {"table.txt"};          // 输入文件
    if (!in) error("no input file");

    string line;                        // 输入缓冲区
    int lineno = 0;

    regex header {R"(^[\w ]+(           [\w ]+)*$)"};      // 标题行
    regex row {R"(^[\w ]+(              \d+)(  \d+)(   \d+)$)"};  // 数据行

    if (getline(in,line)) {             // 检查标题行
```

```
        smatch matches;
        if (regex_match(line, matches, header)) {
            error("no header");
        }
    }

    // 各列合计:
    int boys = 0;
    int girls = 0;

    while (getline(in,line)) {
        ++lineno;
        smatch matches;
        if (!regex_match(line, matches, row))
            cerr << "bad line: " << lineno << '\n';

        if (in.eof()) cout << "at eof\n";

        // 检查本行:
        int curr_boy = from_string<int>(matches[2]);
        int curr_girl = from_string<int>(matches[3]);
        int curr_total = from_string<int>(matches[4]);
        if (curr_boy+curr_girl != curr_total) error("bad row sum \n");

        if (matches[1]=="Alle klasser") {      // 最后一行
            if (curr_boy != boys) error("boys don't add up\n");
            if (curr_girl != girls) error("girls don't add up\n");
            if (!(in>>ws).eof()) error("characters after total line");
            return 0;
        }

        // 更新合计:
        boys += curr_boy;
        girls += curr_girl;
    }

    error("didn't find total line");
}
```

最后一行在语义上与其他行不同——它是它们的总和。我们靠它的标签（“Alle klasser”）来认出它。我们决定在它之后不再接受任何非空白字符（使用 §23.2 中 `to<>()` 的技术），并且如果找不到这一行就报错。

我们用 §23.2 的 `from_string()` 从数据字段中提取整数值。前面已经检查过这些字段只由数字组成，所以不必再检查字符串到整数的转换是否成功。

23.10 参考文献

正则表达式是一种流行而有用的工具。许多编程语言、许多格式都提供它。它有优雅的理论支撑（基于形式语言），也有高效的实现技术（基于状态机）。正则表达式的完整一般性、它的理论、它的实现以及状态机的一般用法，都超出了本书的范围。不过，这些主题在计算机科学课程中相当标准，正则表达式又如此流行，所以要找到更多资料并不难（如果你需要，或者只是感兴趣）。

更多资料见：

- Aho, Alfred V., Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*, Second Edition（通常称为“龙书”）. Addison-Wesley, 2007. ISBN 0321547985.
- Cox, Russ. “Regular Expression Matching Can Be Simple and Fast (but Is Slow in Java, Perl, PHP, Python, Ruby, . . .).” `http://swtch.com/~rsc/regexp/regexp1.html`.
- Maddock, J. *boost::regex* documentation. `www.boost.org/`.
- Schwartz, Randal L., Tom Phoenix, and Brian D. Foy. *Learning Perl*, Fourth Edition. O’Reilly, 2005. ISBN 0596101058.

小试身手

1. 查清 `regex` 是否随你的标准库一起提供。提示：试试 `std::regex` 和 `tr1::regex`。
2. 让 §23.7 的那个小程序跑起来；这可能涉及怎样设置项目选项和/或命令行选项，以便链接正则表达式库并使用正则表达式的头文件。
3. 用第 2 题的程序测试 §23.7 中的那些模式。

要点回顾

1. 我们在哪里能找到“文本”？
2. 标准库中最常用于文本分析的设施有哪些？
3. `insert()` 是插到其位置（或迭代器）之前还是之后？
4. 什么是 Unicode？
5. 怎样在字符串表示与其他类型之间来回转换？
6. 设 `s` 是 `string`，`cin>>s` 与 `getline(cin,s)` 有什么区别？
7. 列出标准流。
8. `map` 的键是什么？举例说明有用的键类型。
9. 怎样遍历 `map` 的元素？
10. `map` 与 `multimap` 有什么区别？`multimap` 缺少哪一个有用的 map 操作，为什么？
11. 前向迭代器需要哪些操作？
12. 空字段与不存在的字段有什么区别？各举两例。
13. 为什么表示正则表达式需要一个转义字符？
14. 怎样把一个正则表达式放进 `regex` 变量？
15. `\w+\s\d{4}` 匹配什么？举三个例子。要用哪个字符串字面量来初始化带该模式的 `regex` 变量？
16. 在程序里怎样判断一个字符串是不是合法的正则表达式？
17. `regex_search()` 做什么？
18. `regex_match()` 做什么？
19. 在正则表达式中怎样表示字符点（.）？
20. 在正则表达式中怎样表示“至少三个”这一概念？

21. 7 是 \w 字符吗？下划线 _ 呢？
22. 大写字符的记法是什么？
23. 怎样指定自己的字符集合？
24. 怎样提取一个整数字段的值？
25. 怎样用正则表达式表示一个浮点数？
26. 怎样从一个匹配中提取浮点值？
27. 什么是子匹配？怎样访问它？

术语

match（匹配） multimap（多重映射） pattern（模式）
regex_match() regex_search() regular expression（正则表达式）
search（查找） smatch sub-pattern（子模式）

习题

1. 让电子邮件文件的例子跑起来；用一个你自己造的更大的文件测试它。务必放入那些很可能触发错误的消息，例如带有两行地址的消息、若干地址相同和/或主题相同的消息，以及空消息。还要用某个完全不符合该程序规格的东西来测试它，例如一个不含 ---- 行的大文件。
2. 增加一个 multimap，用它保存主题。让程序从键盘读入一个字符串，并打印出所有以该字符串为主题的消息。
3. 修改 §23.4 的电子邮件例子，改用正则表达式来查找主题和发件人。
4. 找一个真实的电子邮件消息文件（其中包含真实邮件），修改电子邮件例子，从用户输入的发件人名字中提取主题行。
5. 找一个大邮件文件（数千条消息），然后对使用 multimap 的版本和把该 multimap 换成 unordered_multimap 的版本分别计时。注意我们的应用并没有利用 multimap 的有序性。
6. 写一个在文本文件中查找日期的程序。按“行号: 行内容”的格式输出至少含一个日期的每一行。先从一个简单格式的正则表达式开始，例如 12/24/2000，用它测试程序；然后再加入更多格式。
7. 写一个程序（与上一题类似），在文件中查找信用卡号。稍微做点调研，弄清实际使用的信用卡格式有哪些。
8. 修改 §23.8.7 的程序，让它接受一个模式和一个文件名作为输入。输出应当是所有包含该模式匹配的行，并带行号（行号: 行内容）。如果没有找到匹配，则不应产生任何输出。
9. 利用 eof()（§B.7.2）可以判断表中哪一行是最后一行。用它来（试着）简化 §23.9 的表检查程序。务必用“表格之后还有空行”的文件以及“末尾完全没有换行符”的文件来测试你的程序。
10. 修改 §23.9 的表检查程序，让它写出一张新表，把首位数字相同（表示年级：一年级以 1 开头）的各行合并起来。
11. 修改 §23.9 的表检查程序，看看这些年里学生人数是在增加还是减少。
12. 写一个程序，以查找含日期各行的程序（习题 6）为基础，找出所有日期并把它们重排成 ISO 的 yyyy-mm-dd 格式。程序应接受一个输入文件，产生一个输出文件；除日期格式改变之外，输出文件应与输入文件完全相同。
13. 点（.）能匹配 '\n' 吗？写个程序弄清楚。
14. 写一个像 §23.8.7 那样的程序，通过键入模式来做模式匹配实验。不过，让它把文件读进内存（用换行符 '\n' 表示换行），这样就可以试验跨行的模式。测试它并记录十几个测试模式。
15. 描述一个无法用正则表达式表示的模式。

16. 仅供高手：证明上一题中找到的模式确实不是正则表达式。

后记

人们很容易陷入这样一种看法：计算机和计算都只是关于数字的，计算是数学的一种形式。显然并非如此。只要看看你的计算机屏幕：上面满是文字和图片，也许它还在忙着播放音乐。对每一个应用来说，使用合适的工具都很重要。在 C++ 的语境中，这意味着使用合适的库。对于文本处理，正则表达式库常常是关键工具——也别忘了映射（`map`）和标准算法。

24 数值计算

"For every complex problem there is an answer that is clear, simple, and wrong."
（对每一个复杂的问题，都有一个清晰、简单而又错误的答案。）
——H. L. Mencken

本章概述了一些支持数值计算的基本语言设施与库设施。我们讨论大小、精度和截断这几个基本问题。本章的核心内容是讨论多维数组——既有 C 风格的数组，也有一个 N 维的矩阵库。我们介绍随机数，因为在测试、模拟和游戏中经常需要它。最后，我们列出标准数学函数，并简要介绍标准库复数（complex）的基本功能。

24.1 引言
24.2 大小、精度与溢出
24.2.1 数值极限
24.3 数组
24.4 C 风格多维数组
24.5 Matrix 库
24.5.1 维数与访问
24.5.2 一维 Matrix
24.5.3 二维 Matrix
24.5.4 Matrix 的输入输出
24.5.5 三维 Matrix
24.6 一个例子：求解线性方程组
24.6.1 经典高斯消元法
24.6.2 选主元
24.6.3 测试
24.7 随机数
24.8 标准数学函数
24.9 复数
24.10 参考文献

24.1 引言

对有些人来说，数值计算——也就是严肃的数值计算——就是一切，许多科学家、工程师和统计学家都属于这一类。对许多人来说，数值计算有时是必不可少的，偶尔与物理学家合作的计算机科学家就属于这一类。对大多数人来说，除了整数和浮点数的简单算术之外，很少需要数值计算。本章的目的是讨论处理简单数值问题所需的语言技术细节。我们并不试图讲授数值分析，也不打算细讲浮点运算的精微之处；这些主题远远超出了本书的范围，而且与应用领域中特定领域的主题交织在一起。这里，我们介绍

- 与内置类型大小固定有关的问题，例如精度和溢出
- 数组：既包括内置的多维数组概念，也包括更适合数值计算的 Matrix 库
- 对随机数最基本的介绍

- 标准库数学函数
- 复数

重点在于 Matrix 库，它使矩阵（多维数组）的处理变得轻而易举。

24.2 大小、精度与溢出

当我们使用内置类型和通常的计算技术时，数被存放在大小固定的内存中；也就是说，整数类型（int、long 等）只是数学意义上整数（whole number）的近似，而浮点类型（float、double 等）也只是数学意义上实数的近似。这意味着，从数学的观点看，有些计算是不精确的，甚至是错误的。考虑：

```
float x = 1.0/333;
float sum = 0;
for (int i=0; i<333; ++i) sum+=x;
cout << setprecision(15) << sum << "\n";
```

运行这段代码，我们得到的并不是某些人天真地期望的 1，而是

```
0.999999463558197
```

我们预料到的正是这样的结果。这里看到的是舍入误差（rounding error）造成的影响。一个浮点数只有固定数目的二进制位，所以只要指定一个计算，让它表示结果所需的位数超过硬件所能提供的位数，我们就总能“骗过”它。例如，有理数 1/3 无法精确地表示为十进制小数（无论我们用多少位小数）。1/333 也一样，因此当我们把 333 个 x 相加（x 是机器对 1/333 的最佳 float 近似）时，得到的东西与 1 略有不同。只要大量使用浮点数，舍入误差就会出现；唯一的问题在于误差是否显著影响结果。

一定要检查结果是否合理。计算时，你必须对合理的结果是什么样子有个概念，否则很容易被某个“愚蠢的 bug”或计算错误所欺骗。要意识到出现舍入误差的可能；如果拿不准，就去请教专家，或者查阅数值计算方面的资料。

试一试

把例子中的 333 换成 10，再运行一次这个例子。你预期会得到什么结果？实际又得到了什么结果？事先提醒过你了！

整数大小固定所带来的影响，可能以更富戏剧性的方式显现出来。原因在于，浮点数按定义就是（实）数的近似，所以它们往往损失精度（即丢失最低有效位）。而整数则往往会溢出（即丢失最高有效位）。这就使浮点错误变得隐蔽（初学者常常注意不到），而整数错误则变得壮观（通常想不注意都难）。记住，我们更希望错误尽早、尽可能醒目地暴露出来，这样我们才能修正它们。

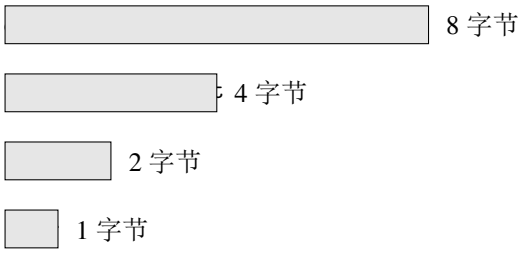
考虑一个整数问题：

```
short int y = 40000;
int i = 1000000;
cout << y << "    " << i*i << "\n";
```

运行它，我们得到输出

```
-25536      -727379968
```

这也在预料之中。这里看到的是溢出的效果。整数类型只能表示（相对）较小的整数。要精确地表示我们需要的每一个数，并且以适合高效计算的方式表示，位数根本就不够。在这里，2 字节的 `short` 整数无法表示 40,000，而 4 字节的 `int` 也无法表示 1,000,000,000,000。C++ 内置类型的确切大小（§A.8）取决于硬件和编译器；对变量 `x` 或类型 `x`，`sizeof(x)` 会告诉你 `x` 的大小（以字节计）。按定义，`sizeof(char)==1`。我们可以这样说明各种类型的大小：



这些尺寸是 Windows 上使用 Microsoft 编译器时的情形。C++ 提供了各种大小的整数和浮点数，但除非你有非常充分的理由另作选择，否则就坚持用 `char`、`int` 和 `double`。在大多数（当然不是全部）程序里，其余的整数类型和浮点类型带来的麻烦超过它们本身的价值。

你可以把一个整数赋给一个浮点变量。如果这个整数大于该浮点类型所能表示的范围，你就会损失精度。例如：

```
cout << "sizes: " << sizeof(int) << ' ' << sizeof(float) << '\n';
int x = 2100000009;      // 大整数
float f = x;
cout << x << ' ' << f << '\n';
cout << setprecision(15) << x << ' ' << f << '\n';
```

在我们的机器上，它产生了

```
Sizes: 4 4
2100000009 2.1e+009
2100000009 2100000000
```

`float` 和 `int` 占用的空间一样大（4 字节）。`float` 由一个“尾数”（通常是 0 到 1 之间的值）和一个指数（$mantissa \times 10^{exponent}$）来表示，所以它无法精确表示最大的 `int`。（如果我们硬要这么做，那么在占掉表示指数所需的空间之后，尾数还能放在哪里呢？）`f` 尽其所能近似地表示了 2100000009，这是它应该做的。然而，最后那个 9 是它无法精确表示的——我们当然正是因此才选了这个数。

另一方面，当你把一个浮点数赋给一个整数时，得到的是截断（truncation）；也就是说，小数部分——小数点后的那些数字——被直接丢弃了。例如：

```
float f = 2.8;
int x = f;
cout << x << ' ' << f << '\n';
```

`x` 的值将是 2，而不会是你按“四舍五入规则”所想象的那样为 3。C++ 的 float 到 int 转换是截断，而不是舍入。

进行计算时，你必须留意可能发生的溢出和截断，C++ 不会替你捕获这类问题。考虑：

```
void f(int i, double fpd)
{
    char c = i;           // 是的：char 确实就是非常小的整数
    short s = i;          // 注意：int 可能放不进 short int
    i = i+1;              // 如果 i 本来已是最大的 int 呢？
    long lg = i*i;        // 注意：long 未必比 int 更大
    float fps = fpd;      // 注意：大的 double 可能放不进 float
    i = fpd;              // 截断：例如 5.7 -> 5
    fps = i;              // 可能损失精度（对于非常大的 int 值）
}

void g()
{
    char ch = 0;
    for (int i = 0; i<500; ++i)
        cout << int(ch++) << '\t';
}
```

如果有疑问，就去检查、去实验！不要只是绝望，也不要只读文档。除非你有经验，否则很容易误解与数值计算有关的高度技术性的文档。

试一试

运行 `g()`。修改 `f()`，让它打印出 `c`、`s`、`i` 等。用各种值对它进行测试。

整数及其转换的表示方式将在 §25.5.3 进一步考察。只要可能，我们就更愿意把自己限制在少数几种数据类型上，这有助于把混乱降到最低。例如，程序中不用 `float` 而只用 `double`，就消除了 double 到 float 转换问题的可能性。事实上，我们更愿意把计算限制在 `int`、`double` 和 `complex`（见 §24.9）上，把字符限制在 `char` 上，把逻辑实体限制在 `bool` 上。其余的算术类型，只有在不得不处理时我们才去处理。

24.2.1 数值极限

在 `<limits>`、`<climits>`、`<limits.h>` 和 `<float.h>` 中，每个 C++ 实现都规定了内置类型的各种性质，这样程序员就能利用这些性质检查界限、设置哨兵值等。这些值列在 §B.9.1 中，对底层工具的构建者可能至关重要。如果你觉得自己需要它们，那你可能离硬件太近了，不过它们还有别的用途。例如，人们常常对语言实现的某些方面感到好奇，比如“`int` 有多大？”或者“`char` 是有符号的吗？”想在系统文档里找到确定而正确的答案可能很难，而标准只规定了最低要求。不过，写一个给出答案的程序却轻而易举：

```
cout << "number of bytes in an int: " << sizeof(int) << '\n';
cout << "largest int: " << INT_MAX << '\n';
cout << "smallest int value: " << numeric_limits<int>::min() << '\n';

if (numeric_limits<char>::is_signed)
    cout << "char is signed\n";
else
    cout << "char is unsigned\n";

char ch = numeric_limits<char>::min();    // 最小的正值
cout << "the char with the smallest positive value: " << ch << '\n';
cout << "the int value of the char with the smallest positive value: "
     << int(ch) << '\n';
```

当你编写的代码打算在多种硬件上运行时，让程序能够获得这类信息有时会变得极有价值。否则，通常只能把答案手工写死在程序里，从而埋下维护隐患。

这些极限值在你想检测溢出时也很有用。

24.3　数组

数组（array）是一个元素序列，其中的元素可以通过索引（位置）来访问。表示这个一般概念的另一个词是向量（vector）。这里我们特别关注的，是元素本身也是数组的那种数组：多维数组。多维数组的一个常用名称是矩阵（matrix）。名称如此多样，说明这个一般概念既流行又实用。标准库的 `vector`（§B.4）、`array`（§20.9）以及内置数组（§A.8.2）都是一维的。那么，如果我们需要二维（例如一个矩阵）呢？如果我们需要七维呢？

我们可以这样来可视化一维数组和二维数组：

v	0	1	2	3

一个向量（例如 `Matrix<int> v(4)`），也称一维数组，甚至 $1 \times N$ 矩阵

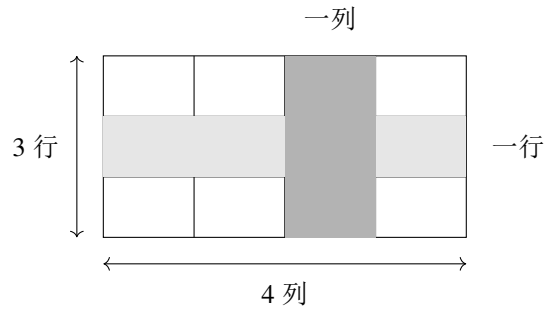
m				

一个 3×4 矩阵（例如 `Matrix<int,2> m(3,4)`），也称二维数组

数组是大多数计算（“数字捣弄”，number crunching）的基础。大多数有意思的科学计算、工程计算、统计计算和金融计算都严重依赖数组。

我们常说数组由行和列组成：

一列（column）是具有相同第一个（x）坐标的元素序列。一行（row）是具有相同第二个（y）坐标的元素集合。



一个 3 × 4 矩阵，也称二维数组；深色格标出一列，浅色带标出一行

24.4 C 风格多维数组

C++ 的内置数组可以用作多维数组。我们只需把多维数组看作数组的数组，也就是以数组为元素的数组。例如：

```
int ai[4];                  // 一维数组
double ad[3][4];            // 二维数组
char ac[3][4][5];           // 三维数组
ai[1] = 7;
ad[2][3] = 7.2;
ac[2][3][4] = 'c';
```

这种做法继承了内置一维数组的优点，也继承了它的缺点：

优点

- 直接映射到硬件
- 对底层操作效率高
- 有直接的语言支持

问题

- C 风格多维数组是数组的数组（见下文）。
- 大小固定（即在编译时固定）。如果要在运行时确定大小，就只能使用自由存储（free store）。
- 无法干净地传递。数组稍受刺激就会变成指向其首元素的指针。
- 没有范围检查。和往常一样，数组不知道自己有多大。
- 没有数组操作，甚至连赋值（复制）都没有。

内置数组广泛用于数值计算。它们同时也是 bug 和复杂性的一个主要来源。对大多数人来说，编写和调试它们实在是一件痛苦的事。如果你不得不用它们，就去查资料（例如《The C++ Programming Language》）。不幸的是，C++ 的多维数组与 C 共享，所以“外面”有大量代码在使用它们。

最根本的问题在于，你无法干净地传入多维数组，只能退回到指针，并自己显式计算元素在多维数组中的位置。例如：

```
void f1(int a[3][5]);                          // 只适用于 [3][5] 矩阵
void f2(int [ ][5], int dim1);                 // 第 1 维可以是一个变量
void f3(int [5 ][ ], int dim2);                // 错误：第 2 维不能是变量
void f4(int[ ][ ], int dim1, int dim2);        // 错误（而且这样也根本不行）
void f5(int* m, int dim1, int dim2)            // 别扭，但能用
{
    for (int i=0; i<dim1; ++i)
        for (int j = 0; j<dim2; ++j) m[i*dim2+j] = 0;
}
```

这里，我们虽然知道 m 是一个二维数组，却仍把它当作 int* 传入。只要第二维必须是变量（一个参数），就根本没有办法告诉编译器 m 是一个 (dim1,dim2) 数组，于是我们只能传入一个指针，指向保存它的那块内存的起始处。表达式 m[i*dim2+j] 的实际意思就是 m[i,j]，但因为编译器不知道 m 是二维数组，我们只好自己计算 m[i,j] 在内存中的位置。

在我们看来，这种做法太复杂、太原始，也太容易出错。它还可能很慢，因为显式地计算元素位置会让优化变得更困难。我们不再花篇幅把这一切都教给你，而是集中讲一个 C++ 库，它消除了内置数组的这些麻烦。

24.5 Matrix 库

我们究竟想从面向数值计算的数组/矩阵中得到哪些基本的“东西”？

- “我的代码看起来应该非常像我那本数学/工程教材里讲的数组。”
- 或者是关于向量、矩阵、张量的写法。
- 编译期和运行期都做检查。
- 任意维数的数组。
- 每一维上元素个数任意的数组。
- 数组是正规的变量/对象。
- 可以到处传递它们。
- 常见的数组运算：
  - 下标：( )
  - 切片：[ ]
  - 赋值：=
  - 缩放运算（+=、-=、*=、%= 等）
  - 融合的向量运算（例如 res[i] = a[i]*c+b[2]）
  - 点积（res = sum of a[i]*b[i]；又称 inner_product）
- 基本上，它把传统的数组/向量记号变成你本来得辛辛苦苦自己写出来的代码（而且运行起来至少和你自己写的一样高效）。
- 需要时可以自己扩展它（它的实现里没有用任何“魔法”）。

Matrix 库做的正是这些，而且只做这些。如果你想要更多东西，比如高级的数组函数、稀疏数组、对内存布局的控制等等，就必须自己写，或者（更好的做法是）用一个更贴近你需求的库。不过，许多这类需求都可以通过在 Matrix 之上构建算法和数据结构来满足。Matrix 库并不属于 ISO C++ 标准库。你可以在本书支持站点上找到它，文件名是 Matrix.h。它在一个名为 Numeric_lib 的命名空间中定义

自己的设施。我们之所以选择“matrix”这个名字，是因为“vector”和“array”在 C++ 库里被用得更加泛滥。matrix 的复数是 matrices（matrixes 是较少见的形式）。当 Matrix 指某个 C++ 语言实体时，我们用 Matrixes 作复数，以免混淆。Matrix 库的实现用到了高级技术，这里不做描述。

24.5.1 尺寸与访问

考虑一个简单的例子：

```
#include "Matrix.h"
using namespace Numeric_lib;

void f(int n1, int n2, int n3)
{
    Matrix<double,1> ad1(n1);              // 元素是 double；一维
    Matrix<int,1> ai1(n1);                 // 元素是 int；一维
    ad1(7) = 0;                            // 用 ( ) 加下标 —— Fortran 风格
    ad1[7] = 8;                            // 用 [ ] 也可以 —— C 风格
    Matrix<double,2> ad2(n1,n2);           // 二维
    Matrix<double,3> ad3(n1,n2,n3);        // 三维
    ad2(3,4) = 7.5;                        // 真正的多维下标
    ad3(3,4,5) = 9.2;
}
```

所以，当你定义一个 Matrix（一个 Matrix 类的对象）时，你要指定元素类型和维数。显然，Matrix 是一个模板，元素类型和维数就是模板参数。给 Matrix 一对实参（例如 `Matrix<double,2>`）得到的是一个类型（一个类），你可以通过提供实参来定义该类型的对象（例如 `Matrix<double,2> ad2(n1,n2);`），这些实参指定各维的大小。于是 ad2 就是一个二维数组，两维大小分别为 n1 和 n2，也就是一个 $n1 \times n2$ 的矩阵。要从一维 Matrix 中取出一个声明元素类型的元素，用一个下标；要从二维 Matrix 中取出一个元素，用两个下标；依此类推。

与内置数组和 vector 一样，我们的 Matrix 下标从 0 开始（而不像 Fortran 数组那样从 1 开始）；也就是说，Matrix 的元素编号为 [0,max)，其中 max 是元素的个数。

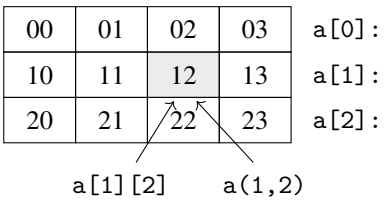
这很简单，也是“照搬教材”。如果你对此感到困难，该去看一本合适的数学教材，而不是程序员手册。这里唯一的“巧妙之处”是：声明 Matrix 时可以省略维数——“一维”是默认值。还要注意，我们既可以用 [ ] 加下标（C 和 C++ 风格），也可以用 ( ) 加下标（Fortran 风格）。两者都提供，使我们能更好地处理多维情形。[x] 这种下标写法总是只取一个下标，得到 Matrix 中相应的那一行；如果 a 是一个 N 维 Matrix，那么 a[x] 就是一个 N-1 维 Matrix。(x,y,z) 这种下标写法取一个或多个下标，得到 Matrix 中相应的元素；下标的个数必须等于维数。

下面看看我们犯错时会发生什么：

```
void f(int n1, int n2, int n3)
{
    Matrix<int,0> ai0;                      // 错误：没有 0 维矩阵
    Matrix<double,1> ad1(5);
    Matrix<int,1> ai(5);
    Matrix<double,1> ad11(7);
    ad1(7) = 0;                             // Matrix_error 异常（7 越界）
    ad1 = ai;                               // 错误：元素类型不同
    ad1 = ad11;                             // Matrix_error 异常（维数不同）
    Matrix<double,2> ad2(n1);               // 错误：缺少第 2 维的长度
    ad2(3) = 7.5;                           // 错误：下标个数不对
    ad2(1,2,3) = 7.5;                       // 错误：下标个数不对
    Matrix<double,3> ad3(n1,n2,n3);
    Matrix<double,3> ad33(n1,n2,n3);
    ad3 = ad33;                             // 正确：元素类型相同、维数相同
}
```

维数的声明与使用之间的不匹配，我们在编译期就能抓住。越界错误则在运行期抓住，并抛出 Matrix_error 异常。

第一维是行、第二维是列，所以我们用 (row,column) 给一个二维矩阵（二维数组）加下标。也可以写成 [row][column]，因为用一个下标给二维矩阵加下标，得到的就是作为该行的一维矩阵。可以这样来想象：



这个 Matrix 在内存中按“行优先”顺序存放：

00	01	02	03	10	11	12	13	20	21	22	23

Matrix “知道”自己的维数，所以对于一个作为参数传入的 Matrix，我们可以非常简单地访问它的元素：

```
void init(Matrix<int,2>& a)       // 把每个元素初始化成一个有特征的值
{
    for (int i=0; i<a.dim1(); ++i)
        for (int j = 0; j<a.dim2(); ++j)
            a(i,j) = 10*i+j;
}

void print(const Matrix<int,2>& a)    // 逐行输出元素
{
    for (int i=0; i<a.dim1(); ++i) {
        for (int j = 0; j<a.dim2(); ++j)
            cout << a(i,j) <<'\t';
        cout << '\n';
    }
}
```

所以，dim1() 是第 1 维中元素的个数，dim2() 是第 2 维中元素的个数，依此类推。元素的类型和维数是 Matrix 类型的一部分，所以我们无法写出一个接受任意 Matrix 作参数的函数（但可以写一个模板来做这件事）：

```
void init(Matrix& a);    // 错误：缺少元素类型和维数
```

注意，Matrix 库并不提供矩阵运算，比如把两个 4 维 Matrix 相加，或者把一个二维 Matrix 与一个一维 Matrix 相乘。要优雅而高效地做到这些，目前超出了这个库的范围。各种设计的矩阵库都可以构建在 Matrix 库之上（见习题 12）。

24.5.2 一维 Matrix

对最简单的 Matrix，也就是一维（1D）Matrix，我们能做些什么？

因为一维是默认值，我们可以在声明中省略维数：

```
Matrix<int,1> a1(8);   // a1 是一个 int 的一维 Matrix
Matrix<int> a(8);      // 意思是 Matrix<int,1> a(8);
```

所以，a 和 a1 是同一类型（Matrix<int,1>）。我们可以问它的大小（元素的个数）和维度（某一维中元素的个数）。对一维 Matrix 来说，这两者是同一回事。

```
a.size();              // Matrix 中元素的个数
a.dim1();              // 第 1 维中元素的个数
```

我们还可以按元素在内存中的排布方式取得它们，也就是取得指向第一个元素的指针：

```
int* p = a.data();        // 把数据取成指向（内置）数组的指针
```

把 Matrix 的数据传给接受指针参数的 C 风格函数时，这很有用。我们还可以加下标：

```
a(i);       // 第 i 个元素 (Fortran 风格), 但会做范围检查
a[i];       // 第 i 个元素 (C 风格), 会做范围检查
a(1,2);     // 错误: a 是一维 Matrix
```

算法常常只涉及 Matrix 的一部分。这样的一“部分”称为 slice()（子 Matrix 或一段元素），我们提供两个版本：

```
a.slice(i);         // 从 a[i] 到最后一个元素
a.slice(i,n);       // 从 a[i] 到 a[i+n-1] 的 n 个元素
```

下标和切片既可以出现在赋值的左边，也可以出现在右边。它们引用其所属 Matrix 的元素，而不复制这些元素。例如：

```
a.slice(4,4) = a.slice(0,4);       // 把 a 的前一半赋给后一半
```

例如，如果 a 一开始是

{ 1 2 3 4 5 6 7 8 }

那么得到

{ 1 2 3 4 1 2 3 4 }

注意，最常见的切片是 Matrix 的“前几个元素”和“最后几个元素”；也就是说，a.slice(0,j) 是区间 [0:j)，而 a.slice(j) 是区间 [j:a.size())。特别地，上面的例子最容易写成

```
a.slice(4) = a.slice(0,4);       // 把 a 的前一半赋给后一半
```

也就是说，这种记号是偏向常见情形的。你可以指定 i 和 n，使 a.slice(i,n) 超出 a 的范围。不过，得到的切片只会引用 a 中实际存在的那些元素。例如，a.slice(i,a.size()) 引用区间 [i:a.size())，而 a.slice(a.size()) 和 a.slice(a.size(),2) 都是空 Matrix。这对许多算法来说恰好是个有用的约定。我们借用的是数学中的这个约定。显然，a.slice(i,0) 是空 Matrix。我们不会故意这么写，但有些算法如果能把 n 恰巧为 0 的 a.slice(i,n) 当作空 Matrix（而不是一个我们必须回避的错误），就会更简单。

我们还有（对 C++ 对象来说很常见的）复制操作，它会复制所有元素：

```
Matrix<int> a2 = a;     // 复制初始化
a = a2;                 // 复制赋值
```

我们还可以把一个内置运算施加到 Matrix 的每个元素上：

```
a *= 7;                 // 缩放: 对每个 i 执行 a[i]*=7 (还有 +=、-=、/= 等)
a = 7;                  // 对每个 i 执行 a[i]=7
```

只要元素类型支持该运算符，这对每个赋值运算符和每个复合赋值运算符都成立（=、+=、-=、/=、*=、%=、^=、&=、|、>>=、<<=）。我们还可以把一个函数施加到 Matrix 的每个元素上：

```
a.apply(f);                 // 对每个元素 a[i] 执行 a[i]=f(a[i])
a.apply(f,7);               // 对每个元素 a[i] 执行 a[i]=f(a[i],7)
```

复合赋值运算符和 apply() 修改的是其 Matrix 参数的元素。如果我们想要的是把结果作为返回值新建一个 Matrix，可以用

```
b = apply(abs,a);    // 新建一个 Matrix，使 b(i)==abs(a(i))
```

这里的 abs 是标准库的绝对值函数（§24.8）。基本上，apply(f,x) 与 x.apply(f) 的关系，就像 + 与 += 的关系。例如：

```
b = a*7;                // 对每个 i，b[i] = a[i]*7
a *= 7;                 // 对每个 i，a[i] = a[i]*7
y = apply(f,x);         // 对每个 i，y[i] = f(x[i])
x.apply(f);             // 对每个 i，x[i] = f(x[i])
```

这里我们得到 a==b 和 x==y。

在 Fortran 中，后一种 apply 称为“广播（broadcast）”函数，通常写作 f(x) 而不是 apply(f,x)。为了让这个设施对每个函数 f 都可用（而不像 Fortran 那样只对少数几个选定的函数可用），我们需要给这个“广播”操作起个名字，于是我们（重新）使用了 apply。

此外，为了与成员版本 apply 的两参数形式 a.apply(f,x) 相配，我们提供

```
b = apply(f,a,x);       // 对每个 i，b[i]=f(a[i],x)
```

例如：

```
double scale(double d, double s) { return d*s; }
b = apply(scale,a,7);       // 对每个 i，b[i] = a[i]*7
```

注意，“自由”的 apply() 接受一个由其参数产生结果的函数；apply() 随后用这个结果来初始化结果 Matrix。它通常不修改被施加的 Matrix。成员版本 apply() 则不同：它接受的函数会修改自己的参数，也就是说，它会修改被施加的 Matrix 的元素。例如：

```
void scale_in_place(double& d, double s) { d *= s; }
b.apply(scale_in_place,7);    // 对每个 i，b[i] *= 7
```

我们还提供了传统数值库中最有用的一些函数：

```
Matrix<int> a3 = scale_and_add(a,8,a2);     // 融合乘加
int r = dot_product(a3,a);                  // 点积
```

scale_and_add() 运算常被称为融合乘加（fused multiply-add），简称 fma；它的定义是：对 Matrix 中的每个 i，result(i)=arg1(i)*arg2+arg3(i)。点积又称 inner_product，在 §21.5.3 中描述；它的定义是：对 Matrix 中的每个 i，result+=arg1(i)*arg2(i)，其中 result 的初值为 0。

一维数组非常常见；你可以把一维数组表示成内置数组、vector 或 Matrix。如果你需要所提供的那些矩阵运算（例如 *=），或者该 Matrix 必须与更高维的 Matrix 交互，那就用 Matrix。

你可以这样解释这样一个库的用处：“它更贴合数学上的写法”，或者“它省得你为每个元素写那些循环”。不管怎么说，写出来的代码明显更短，写错的机会也更少。Matrix 的这些运算——比如复制、给所有元素赋值、对所有元素做运算——省得我们去读或写一个循环（也省得我们担心循环是不是写得完全正确）。

Matrix 支持两个构造函数，用于把内置数组中的数据复制进 Matrix。例如：

```
void some_function(double* p, int n)
{
    double val[] = { 1.2, 2.3, 3.4, 4.5 };
    Matrix<double> data(p,n);
    Matrix<double> constants(val);
    // . . .
}
```

当程序的其他部分不使用 Matrix、而是以数组或 vector 的形式把数据交给我们时，这两个构造函数常常很有用。

注意，编译器能推断出被初始化数组的元素个数，所以我们定义 constants 时不必给出元素个数——它是 4。另一方面，只给一个指针时编译器无法知道元素个数，所以对 data 我们必须同时指定指针（p）和元素个数（n）。

24.5.3 二维 Matrix

Matrix 库的总体想法是：不同维数的 Matrix 其实非常相似，只是在你需要明确指定维数的地方有所不同，因此我们关于一维 Matrix 所说的大部分内容都适用于二维 Matrix：

```
Matrix<int,2> a(3,4);

int s = a.size();              // 元素总数
int d1 = a.dim1();             // 第 1 维中元素的个数
int d2 = a.dim2();             // 第 2 维中元素的个数
int* p = a.data();             // 把数据取成一个指向 C 风格数组的指针
```

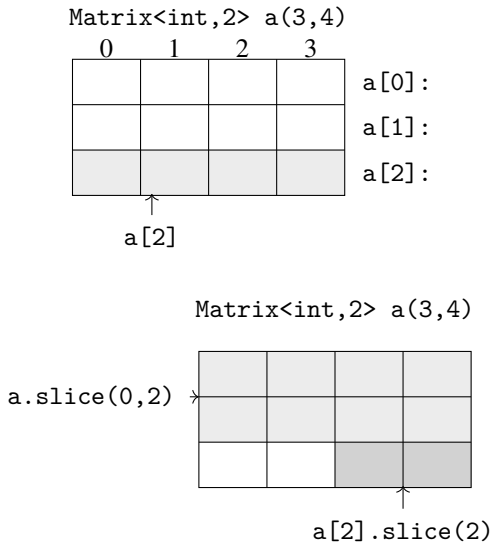
我们可以问元素的总数，以及每一维中元素的个数。我们还可以取得一个指针，指向按矩阵方式在内存中排布的那些元素。

我们可以加下标：

```
a(i,j);                        // (i,j) 个元素 (Fortran 风格)，但会做范围检查
a[i];                          // 第 i 行 (C 风格)，会做范围检查
a[i][j];                       // (i,j) 个元素 (C 风格)
```

对二维 Matrix，用 [i] 加下标得到的是作为第 i 行的一维 Matrix。这意味着我们可以取出各行，把它们传给需要一维 Matrix 甚至内置数组（`a[i].data()`）的运算和函数。注意 `a(i,j)` 可能比 `a[i][j]` 更快，不过这很大程度上取决于编译器和优化器。

我们还可以取切片：




```
a.slice(i);                    // 从 a[i] 到最后一行的那些行
a.slice(i,n);                  // 从 a[i] 到 a[i+n-1] 的那些行
```

注意，二维 Matrix 的切片本身也是一个二维 Matrix（可能行数更少）。

分布式运算与一维 Matrix 相同。这些运算并不关心我们怎样组织元素；它们只是按元素在内存中排定的顺序，作用于所有元素：

```
Matrix<int,2> a2 = a;          // 复制初始化
a = a2;                        // 复制赋值
a *= 7;                        // 缩放（还有 +=、-=、/= 等）
a.apply(f);                    // 对每个元素 a(i,j) 执行 a(i,j)=f(a(i,j))
a.apply(f,7);                  // 对每个元素 a(i,j) 执行 a(i,j)=f(a(i,j),7)
b=apply(f,a);                  // 新建一个 Matrix，使 b(i,j)==f(a(i,j))
b=apply(f,a,7);                // 新建一个 Matrix，使 b(i,j)==f(a(i,j),7)
```

事实证明，交换两行常常很有用，所以我们提供了这个操作：

```
a.swap_rows(1,2);              // 交换 a[1] 和 a[2] 两行
```

没有 `swap_columns()`。如果需要，你自己写（习题 11）。由于行优先的布局，行和列并不是完全对称的概念。这种不对称还体现在：[i] 得到的是一个行（我们也没有提供列选择运算符）。在 (i,j) 中，第一个下标 i 选择行。这种不对称也反映了深刻的数学性质。

二维的“东西”似乎无穷无尽，因此它们显然是二维 Matrix 的明显应用对象：

```
enum Piece { none, pawn, knight, queen, king, bishop, rook };
Matrix<Piece,2> board(8,8);     // 一个棋盘

const int white_start_row = 0;
const int black_start_row = 7;

Matrix<Piece> start_row
          = {rook, knight, bishop, queen, king, bishop, knight, rook};

Matrix<Piece> clear_row(8);      // 8 个取默认值的元素
```

clear_row 的初始化利用了 none==0，以及元素默认初始化为 0 这一事实。

我们可以像这样使用 start_row 和 clear_row：

```
board[white_start_row] = start_row;              // 重设白方棋子
for (int i = 1; i<7; ++i) board[i] = clear_row;  // 清空棋盘中部
board[black_start_row] = start_row;              // 重设黑方棋子
```

注意，当我们用 [i] 取出一个行时，得到的是一个左值（§4.3）；也就是说，我们可以对 board[i] 的结果赋值。

24.5.4 Matrix 的输入输出

Matrix 库为 1D 和 2D 的 Matrix 提供了非常简单的 I/O：

```
Matrix<double> a(4);
cin >> a;
cout << a;
```

这会读入四个用空白分隔的 double，整组由花括号界定；例如：

```
{ 1.2 3.4 5.6 7.8 }
```

输出与输入非常相似，所以你可以把写出去的内容再读回来。

二维 Matrix 的 I/O 只是读写一串由花括号界定的 1D Matrix。例如：

```
Matrix<int,2> m(2,2);
cin >> m;
cout << m;
```

这会读入

```
{
{ 1 2 }
{ 3 4 }
}
```

输出会非常相似。

Matrix 的 << 和 >> 运算符主要是为了让简单程序的编写保持简单。用于更高级的用途时，你很可能需要把它们替换成自己的版本。因此，Matrix 的 << 和 >> 放在 MatrixIO.h 头文件里（而不是 Matrix.h），这样你在使用 Matrix 时不必包含它。

24.5.5 三维 Matrix

基本上，三维（以及更高维）的 Matrix 与二维 Matrix 一样，只是维数更多。考虑：

```
Matrix<int,3> a(10,20,30);

a.size();       // 元素总数
a.dim1();       // 第 1 维中元素的个数
a.dim2();       // 第 2 维中元素的个数

a.dim3();                           // 第 3 维的元素个数
int* p = a.data();                  // 把数据提取为指向 C 风格数组的指针
a(i,j,k);                           // 第 (i,j,k) 个元素 (Fortran 风格)，带范围检查
a[i];                               // 第 i 行 (C 风格)，带范围检查
a[i][j][k];                         // 第 (i,j,k) 个元素 (C 风格)
a.slice(i);                         // 从第 i 行到最后一行的各行
a.slice(i,j);                       // 从第 i 行到第 j 行的各行
Matrix<int,3> a2 = a;               // 拷贝初始化
a = a2;                             // 拷贝赋值
a *= 7;                             // 缩放 (以及 +=、-=、/= 等)
a.apply(f);                         // 对每个元素 a(i,j,k)：a(i,j,k)=f(a(i,j,k))
a.apply(f,7);                       // 对每个元素 a(i,j,k)：a(i,j,k)=f(a(i,j,k),7)
b=apply(f,a);                       // 生成新 Matrix，使 b(i,j,k)==f(a(i,j,k))
b=apply(f,a,7);                     // 生成新 Matrix，使 b(i,j,k)==f(a(i,j,k),7)
a.swap_rows(7,9);                   // 交换行：a[7] <-> a[9]
```

如果你理解二维 Matrix，你就理解三维 Matrix。例如，这里的 a 是三维的，所以 a[i] 是二维的（只要 i 在范围内），a[i][j] 是一维的（只要 j 在范围内），而 a[i][j][k] 是 int 元素（只要 k 在范围内）。

我们倾向于把世界看成三维的。这就使三维 Matrix 在建模中有显而易见的用途（例如用笛卡尔网格做物理仿真）：

```
int grid_nx;            // 网格分辨率；在启动时设定
int grid_ny;
int grid_nz;
Matrix<double,3> cube(grid_nx, grid_ny, grid_nz);
```

如果再加入时间作为第四维，我们就得到一个需要四维 Matrix 的四维空间。依此类推。

想了解更高级的 Matrix 版本（支持一般的 N 维矩阵），参见 *The C++ Programming Language* 第 29 章。

24.6　例子：解线性方程组

数值计算的代码，如果你理解它所表达的数学，就有意义；如果不理解，它往往看起来完全是无意义的乱码。如果你学过线性代数基础，本节的例子应当相当简单；如果没学过，就把它看成一个示例：把教科书上的解法逐字转写成代码，几乎不改措辞。

这里选这个例子，是为了演示 `Matrix` 的一个相当真实、也相当重要的用途。我们将求解如下形式的一组（任意一组）线性方程组：

$$\begin{aligned} a_{1,1}x_1 + \cdots + a_{1,n}x_n &= b_1 \\ &\vdots \\ a_{n,1}x_1 + \cdots + a_{n,n}x_n &= b_n \end{aligned}$$

这里，x 表示 n 个未知数；a 和 b 是给定的常数。为简单起见，我们假定未知数和常数都是浮点值。目标是求出同时满足这 n 个方程的未知数取值。这些方程可以紧凑地用矩阵和两个向量表示：

$$\mathbf{Ax} = \mathbf{b}$$

这里，A 是由各系数定义的 $n \times n$ 方阵：

$$A = \begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ \vdots & & \vdots \\ a_{n,1} & \cdots & a_{n,n} \end{bmatrix}$$

向量 x 和 b 分别是未知数向量和常数向量：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix}, \quad \text{以及} \quad \mathbf{b} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

这个方程组可能有零个、一个或无穷多个解，取决于矩阵 A 的系数和向量 b。求解线性方程组有多种方法。我们采用一种经典方案，称为高斯消元（Gaussian elimination，见 Freeman 与 Phillips 的 *Parallel Numerical Algorithms*；Stewart 的 *Matrix Algorithms, Volume I*；以及 Wood 的 *Introduction to Numerical Analysis*）。首先，我们对 A 和 b 作变换，使 A 成为上三角矩阵。所谓上三角，是指 A 对角线以下的所有系数都为零。换言之，方程组变成这样：

$$\begin{bmatrix} a_{1,1} & \cdots & a_{1,n} \\ 0 & \ddots & \vdots \\ 0 & \cdots & a_{n,n} \end{bmatrix} \begin{bmatrix} x_1 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ \vdots \\ b_n \end{bmatrix}$$

这很容易做到。要把 $a(i,j)$ 位置化为零，可以将第 i 行的方程乘以一个常数，使 $a(i,j)$ 等于第 j 列中的另一个元素，比如 $a(k,j)$。这样做之后，我们只须把两个方程相减，于是 $a(i,j) == 0$，而第 i 行中的其他值也相应地改变。

如果我们能让所有对角系数都非零，那么方程组有唯一解，可以用“回代”求出。最后一个方程很容易求解：

$$a_{n,n}x_n = b_n$$

显然，$x[n]$ 就是 $b[n]/a(n,n)$。然后从方程组中消去第 n 行，继续求 $x[n-1]$ 的值，依此类推，直到算出 $x[1]$ 的值。对每个 n，我们都要除以 $a(n,n)$，所以对角值必须非零。如果不成立，回代法就会失败，这意味着方程组无解或有无穷多个解。

24.6.1 经典高斯消元

现在来看看表达这一算法的 C++ 代码。首先，我们给将要使用的两个 `Matrix` 类型约定别名，以简化记号：

```
typedef Numeric_lib::Matrix<double, 2> Matrix;
typedef Numeric_lib::Matrix<double, 1> Vector;
```

接下来表达我们想要的计算：

```
Vector classical_gaussian_elimination(Matrix A, Vector b)
{
    classical_elimination(A, b);
    return back_substitution(A, b);
}
```

也就是说，我们（用按值传递）复制输入 A 和 b，调用一个函数求解方程组，再用回代算出要返回的结果。关键在于，我们对问题的分解以及解答所用的记号，都直接来自教科书。为完成解答，我们还要实现 `classical_elimination()` 和 `back_substitution()`。同样，解法就在教科书里：

```
void classical_elimination(Matrix& A, Vector& b)
{
    const Index n = A.dim1();
    // 从第 1 列遍历到倒数第 2 列
    // 在对角线以下的各元素中填入 0:
    for (Index j = 0; j<n-1; ++j) {
        const double pivot = A(j,j);
        if (pivot == 0) throw Elim_failure(j);
        // 在第 i 行对角线以下的每个元素中填入 0:
        for (Index i = j+1; i<n; ++i) {
            const double mult = A(i,j) / pivot;
            A[i].slice(j) = scale_and_add(A[j].slice(j), -mult, A[i].slice(j));
            b(i) -= mult*b(j);        // 对 b 作相应的修改
        }
    }
}
```

所谓“主元”（pivot），就是当前正在处理的那一行对角线上的元素。它必须非零，因为我们要除以它；如果它是零，我们就通过抛出异常来放弃：

```
Vector back_substitution(const Matrix& A, const Vector& b)
{
    const Index n = A.dim1();
    Vector x(n);

    for (Index i = n-1; i>= 0; --i) {
        double s = b(i)-dot_product(A[i].slice(i+1),x.slice(i+1));
        if (double m = A(i,i))
            x(i) = s/m;
        else
            throw Back_subst_failure(i);
    }
    return x;
}
```

24.6.2 选主元

我们可以避开除以零的问题，并通过调整行的次序让零值和小值远离对角线，从而得到更稳健的解。所谓“更稳健”，是指对舍入误差不那么敏感。然而，随着我们不断在对角线下方填入零，各数值也在变化，因此我们还必须在过程中重排次序，才能使小值远离对角线（也就是说，不能先把矩阵重排好，再用经典算法）：

```
void elim_with_partial_pivot(Matrix& A, Vector& b)
{
    const Index n = A.dim1();
    for (Index j = 0; j<n; ++j) {
        Index pivot_row = j;
        // 找一个合适的主元:
        for (Index k = j+1; k<n; ++k)
            if (abs(A(k,j)) > abs(A(pivot_row,j))) pivot_row = k;
        // 如果找到了更好的主元，就交换这两行:
        if (pivot_row!=j) {
            A.swap_rows(j,pivot_row);
            std::swap(b(j), b(pivot_row));
        }

        // 消元:
        for (Index i = j+1; i<n; ++i) {
            const double pivot = A(j,j);
            if (pivot==0) error("can't solve: pivot==0");
            const double mult = A(i,j)/pivot;
            A[i].slice(j) = scale_and_add(A[j].slice(j), -mult, A[i].slice(j));
            b(i) -= mult*b(j);
        }
    }
}
```

我们用 swap_rows() 和 scale_and_multiply() 让代码更接近常规写法，也省得自己写显式循环。

24.6.3 测试

显然，我们必须测试代码。幸运的是，有一个简单的办法：

```
void solve_random_system(Index n)
{
    Matrix A = random_matrix(n);        // 见 §24.7
    Vector b = random_vector(n);
    cout << "A = " << A << '\n';
    cout << "b = " << b << '\n';

    try {
        Vector x = classical_gaussian_elimination(A, b);
        cout << "classical elim solution is x = " << x << '\n';
        Vector v = A*x;
        cout << " A*x = " << v << '\n';
    }
    catch(const exception& e) {
        cerr << e.what() << '\n';
    }
}
```

有三种途径会进入 catch 子句：

- 代码中的缺陷（不过我们是乐观主义者，认为没有缺陷）
- 某个输入让 classical_elimination 出错（在很多情况下 elim_with_partial_pivot 会做得更好）
- 舍入误差

不过，我们的测试并没有我们希望的那样真实，因为真正随机的矩阵不太可能给 classical_elimination 造成麻烦。

为了验证解答，我们打印出 $A*x$，它应当等于 b（或者考虑到舍入误差，在我们的目的下足够接近）。正因为存在舍入误差，我们才没有简单地写成

```
if (A*x!=b) error("substitution failed");
```

由于浮点数只是实数的近似，我们不得不接受近似正确的答案。一般来说，对浮点计算的结果使用 == 和 != 最好避免：浮点本身就是一种近似。

Matrix 库没有定义矩阵与向量相乘，所以我们为这个程序补上了：

```
Vector operator*(const Matrix& m, const Vector& u)
{
    const Index n = m.dim1();
    Vector v(n);
    for (Index i = 0; i<n; ++i) v(i) = dot_product(m[i],u);
    return v;
}
```

又一次，一个简单的 Matrix 操作替我们完成了大部分工作。Matrix 的输出操作来自 MatrixIO.h，如 §24.5.4 所述。random_matrix() 和 random_vector() 两个函数是对随机数的简单运用（§24.7），留作习题。Index 是 Matrix 库所用索引类型的类型别名（§A.16）。我们用一条 using 声明把它引入作用域：

```
using Numeric_lib::Index;
```

24.7 随机数

如果你向人要一个随机数，多数人会说 7 或 17，所以有人提出这两个是“最随机”的数。人们几乎从不回答 0。0 被视为一个规整的整数，因此不被看作“随机”，反倒可以算“最不随机”的数。从数学角度看，这纯属无稽之谈：随机的并不是某个单独的数。我们经常需要、并且经常称之为随机数的那种东西，是一个数序列，它服从某种分布，而且你很难根据前面几个数预测序列中的下一个数。这样的数在测试（这是生成大量测试用例的一种办法）、游戏（这是确保下一次运行不同于上一次运行的一种办法）和仿真（我们可以让仿真实体在其参数允许的范围内以“随机”方式行动）中最有用。

作为实用工具和数学问题，随机数已经达到相当高的精细程度，以匹配其在现实世界中的重要性。这里我们只触及满足简单测试与仿真所需的基础。在头文件 <random> 中，标准库提供了一套精致的随机数生成设施，可以匹配多种数学分布。标准库的随机数设施基于两个基本概念：

- 引擎（随机数引擎）：引擎是一个函数对象，它生成一个均匀分布的整数序列。
- 分布：分布是一个函数对象，它接受来自引擎的值序列作为输入，按照某个数学公式生成一个值序列。

例如，考虑 §24.6.3 中用到的 random_vector() 函数。调用 random_vector(n) 会生成一个 Matrix<double,1>，其中有 n 个 double 类型的元素，取值是 $[0:n)$ 范围内的随机值：

```
Vector random_vector(Index n)
{
    Vector v(n);
    default_random_engine ran{};                    // 生成整数
    uniform_real_distribution<> ureal{0,max};    // 把 int 映射为 double
                                                    // 范围 [0:max)
    for (Index i = 0; i < n; ++i)
        v(i) = ureal(ran);
    return v;
}
```

默认引擎（default_random_engine）简单、运行开销小，对日常使用足够了。对于更专业的用途，标准库提供多种引擎，它们具有更好的随机性质，运行开销也各不相同。例如 linear_congurential_engine、mersenne_twister_engine 和 random_device。如果你想用这些引擎，或者一般地说，如果你需要比 default_random_engine 更好的东西，那就得读点资料了。想了解你的系统随机数生成器的质量，请做习题 10。

来自 std_lib_facilities.h 的两个随机数生成器定义如下：

```
int randint(int min, int max)
{
    static default_random_engine ran;
    return uniform_int_distribution<>{min,max}(ran);
}

int randint(int max)
{
    return randint(0,max);
}
```

这些简单的函数非常有用，不过为了试试别的东西，我们来生成一个正态分布：

```
auto gen = bind(normal_distribution<double>{15,4.0},
                                default_random_engine{});
```

标准库中来自 <functional> 的 bind() 会构造一个函数对象，调用它时，它会以自己的第二个实参调用第一个实参。所以在这里，gen() 使用 default_random_engine，按照均值为 15、方差为 4.0 的正态分布返回值。我们可以这样使用它：

```
vector<int> hist(2*15);
for (int i = 0; i < 500; ++i)        // 生成 500 个值的直方图
    ++hist[int(round(gen()))];


for (int i = 0; i != hist.size(); ++i) {    // 输出直方图
    cout << i << '\t';
    for (int j = 0; j != hist[i]; ++j)
        cout << '*';
    cout << '\n';
}
```

输出如下：

```
0
1
2
3       **
4       *
5       *****
6       ****
7       ****
8       ******
9       ************
10    ***************************
11    ***************************
12    ************************************
13    ********************************************************
14    ************************************************
15    ********************************************************
16    ******************************
17    ************************************************
18    ***********************************
19    *********************************
20    **************
21    ************
22    ************
23    *******
24    ******
25    *
26    *
27
28
29
```

正态分布（normal distribution）非常常见，又称高斯分布（Gaussian distribution），或者（原因显而易见）干脆叫“钟形曲线”。其他分布还有 `bernoulli_distribution`、`exponential_distribution` 和 `chi_squared_distribution`。关于它们的介绍可以在《C++ 程序设计语言》中找到。整数分布返回闭区间 $[a:b]$ 内的值，而实数（浮点）分布返回开区间 $[a:b)$ 内的值。

默认情况下，一个引擎（可能只有 `random_device` 例外）在每次运行程序时都给出相同的序列。这对最初的调试最为方便。如果我们想让同一个引擎给出不同的序列，就得用不同的值去初始化它。这样的初始值习惯上称为“种子”（seed）。例如：

```
auto gen1 = bind(uniform_int_distribution<>{0,9},
    default_random_engine{});
auto gen2 = bind(uniform_int_distribution<>{0,9},
    default_random_engine{10});
auto gen3 = bind(uniform_int_distribution<>{0,9},
    default_random_engine{5});
```

为了得到不可预测的序列，人们常常拿当前时刻（精确到最后一纳秒；§26.6.1）或者类似的东西当作种子。

24.8 标准数学函数

标准数学函数（cos、sin、log 等）由标准库提供，它们的声明位于 <cmath> 中。

标准数学函数

abs(x)	绝对值
ceil(x)	不小于 x 的最小整数
floor(x)	不大于 x 的最大整数
sqrt(x)	平方根；x 必须非负
cos(x)	余弦
sin(x)	正弦
tan(x)	正切
acos(x)	反余弦；结果非负
asin(x)	反正弦；返回最接近 0 的结果
atan(x)	反正切
sinh(x)	双曲正弦
cosh(x)	双曲余弦
tanh(x)	双曲正切
exp(x)	以 e 为底的指数
log(x)	自然对数（以 e 为底）；x 必须为正
log10(x)	以 10 为底的对数

标准数学函数为 float、double、long double 和 complex（§24.9）类型的实参提供。如果你做浮点计算，会发现这些函数很有用。若需要更多细节，相关文档随处可见；你手边的在线文档就是个不错的起点。

如果一个标准数学函数无法得出数学上有效的结果，它就会设置变量 errno。例如：

```
errno = 0;
double s2 = sqrt(-1);
if (errno) cerr << "something went wrong with something somewhere";
if (errno == EDOM)              // 定义域错误
    cerr << "sqrt() not defined for negative argument";
pow(very_large,2);              // 不是个好主意
if (errno==ERANGE)              // 值域错误
    cerr << "pow(" << very_large << ",2) too large for a double";
```

如果你做严肃的数学计算，就必须在得到结果之后检查 errno，确保它仍然是 0。如果不是 0，就说明出了问题。请查阅手册或在线文档，弄清哪些数学函数会设置 errno，以及它们各自把 errno 设成什么值。

正如例子所示，errno 非零只意味着“出了点问题”。非标准库的函数在出错时也常常会设置 errno，所以你必须更仔细地看 errno 的值，才能大致判断究竟哪里出了错。如果你在调用某个标准库函数之后立即检查 errno，并且在调用之前已确认 errno==0，那么就可以像例子中对待 EDOM 和 ERANGE 那样信赖这些值。EDOM 表示定义域错误（domain error，即结果方面的问题），ERANGE 表示值域错误（range error，即实参方面的问题）。

基于 errno 的错误处理相当原始，它可以追溯到最早的（1975 年版）C 数学函数。

24.9 复数

复数（complex number）在科学与工程计算中用得很多。我们假定：如果你需要它们，就已经了解它们的数学性质，因此这里只演示复数在 ISO C++ 标准库中怎样表达。复数及其相关标准数学函数的声明位于 <complex> 中：

```
template<class Scalar> class complex {
    // complex 就是一对标量值，本质上是一对坐标
    Scalar re, im;
public:
    constexpr complex(const Scalar & r, const Scalar & i) :re(r), im(i) { }
    constexpr complex(const Scalar & r) :re(r),im(Scalar ()) { }
    complex() :re(Scalar ()), im(Scalar ()) { }

    constexpr Scalar real() { return re; }       // 实部
    constexpr Scalar imag() { return im; }       // 虚部

    // 运算符: = += -= *= /=
};
```

标准库的 `complex` 保证支持标量类型 `float`、`double` 和 `long double`。除了 `complex` 的成员和标准数学函数（§24.8）之外，`<complex>` 还提供了一大批有用的运算：

复数的运算符

`z1+z2`	加法
`z1-z2`	减法
`z1*z2`	乘法
`z1/z2`	除法
`z1==z2`	相等
`z1!=z2`	不等
`norm(z)`	`abs(z)` 的平方
`conj(z)`	共轭：若 z 为 {re,im}，则 conj(z) 为 (re,-im)
`polar(rho,theta)`	由极坐标 (rho,theta) 构造一个复数
`real(z)`	实部
`imag(z)`	虚部
`abs(z)`	又称 rho
`arg(z)`	又称 theta
`out << z`	复数输出
`in >> z`	复数输入

注意：`complex` 不提供 `<` 和 `%`。

请把 `complex<T>` 完全当作内置类型（例如 `double`）来用。例如：

```
using cmplx = complex<double>;  // 有时 complex<double> 太罗嗦

void f(cmplx z, vector<cmplx>& vc)
{
    cmplx z2 = pow(z,2);
    cmplx z3 = z2*9.3+vc[3];
    cmplx sum = accumulate(vc.begin(), vc.end(), cmplx{});
    // . . .
}
```

请记住，我们习惯用于 `int` 和 `double` 的运算并非都适用于 `complex`。例如：

```
if (z2<z3)                    // 错误：复数没有 < 运算符
```

注意，C++ 标准库复数的表示（布局）与 C 和 Fortran 中相应的类型兼容。

24.10 参考文献

基本上，本章讨论的这些问题——例如舍入误差、`Matrix` 运算和复数算术——单独来看既没有意思，也说不通。我们只是描述 C++ 为那些既需要数学概念与技术、又了解它们的人做数值计算所提供的（部分）支持。

如果你在这些方面有点生疏，或者只是好奇，我们可以推荐一些信息来源：

- MacTutor 数学史档案（The MacTutor History of Mathematics archive），`http://www-gap.dcs.st-and.ac.uk/~history`
  - 对任何喜欢数学、或者只是需要用数学的人，都是一个极好的链接
  - 对想看看数学“人性的一面”的人也是极好的链接；例如：唯一一位获得过奥运会奖牌的大数学家是谁？
  - 著名数学家：传记、成就
  - 奇珍（Curio）
    - 著名曲线
    - 著名问题
    - 数学主题
      - 代数
      - 分析
      - 数与数论
      - 几何与拓扑
      - 数学物理
      - 数学天文学
  - 数学史
  - ……
- Freeman, T. L., and Chris Phillips. *Parallel Numerical Algorithms*. Prentice Hall, 1992.

- Gullberg, Jan. *Mathematics — From the Birth of Numbers*. W. W. Norton, 1996. ISBN 039304002X. 堪称讲基础而实用数学的最令人愉快的书之一。这是一本（罕见的）既能当闲书读、又能用来查矩阵之类具体主题的数学书。
- Knuth, Donald E. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, Third Edition. Addison-Wesley, 1998. ISBN 0201896842.
- Stewart, G. W. *Matrix Algorithms, Volume I: Basic Decompositions*. SIAM, 1998. ISBN 0898714141.
- Wood, Alistair. *Introduction to Numerical Analysis*. Addison-Wesley, 1999. ISBN 020194291X.

小试身手

1. 打印 `char`、`short`、`int`、`long`、`float`、`double`、`int*` 和 `double*` 的大小（用 `sizeof`，不要用 `<limits>`）。
2. 打印 `sizeof` 报告的下列对象的大小：`Matrix<int> a(10)`、`Matrix<int> b(100)`、`Matrix<double> c(10)`、`Matrix<int,2> d(10,10)`、`Matrix<int,3> e(10,10,10)`。
3. 打印第 2 题中每个 `Matrix` 的元素个数。
4. 写一个程序，从 `cin` 读入 `int`，并输出每个 `int` 的 `sqrt()`；如果对某个 x 来说 `sqrt(x)` 不合法，就输出“no square root”（也就是说，要检查 `sqrt()` 的返回值）。
5. 从输入读入十个浮点值，放进一个 `Matrix<double>` 里。`Matrix` 没有 `push_back()`，所以要小心处理输入的 `double` 个数不对的情况。打印这个 `Matrix`。
6. 计算 $[0, n) \times [0, m)$ 的乘法表，用二维 `Matrix` 表示。从 `cin` 读入 n 和 m，并把这张表打印得好看些（假定 m 足够小，结果能排在一行里）。
7. 从 `cin` 读入十个 `complex<double>`（没错，`cin` 支持对 `complex` 用 `>>`），放进一个 `Matrix`。计算并输出这十个复数的和。
8. 读入六个 `int`，放进 `Matrix<int,2> m(2,3)`，再把它们打印出来。

要点回顾

1. 谁在用数值计算？
2. 什么是精度（precision）？
3. 什么是溢出（overflow）？
4. `double` 的常见大小是多少？`int` 呢？
5. 怎样检测溢出？
6. 在哪里能找到数值极限，例如最大的 `int`？
7. 什么是数组？什么是行？什么是列？
8. 什么是 C 风格的多维数组？
9. 对矩阵计算而言，语言支持（例如一个库）应当具备哪些理想性质？
10. 矩阵的“维”（dimension）是什么？
11. 一个矩阵可以有多少维（在理论上、在数学上）？
12. 什么是切片（slice）？
13. 什么是广播运算（broadcast operation）？举出几个。
14. Fortran 风格与 C 风格的下标有什么不同？
15. 怎样把某个运算施加到矩阵的每个元素上？举例说明。
16. 什么是融合运算（fused operation）？
17. 给出点积（dot product）的定义。

18. 什么是线性代数？
19. 什么是高斯消去法？
20. 什么是主元（pivot）？（线性代数里的？还是“现实生活中”的？）
21. 是什么让一个数成为随机的？
22. 什么是均匀分布？
23. 在哪里能找到标准数学函数？它们是为哪些实参类型定义的？
24. 复数的虚部是什么？
25. -1 的平方根是什么？

术语

`array`（数组）、C、`column`（列）、复数（complex number）、维（dimension）、点积（dot product）、逐元素运算（element-wise operation）、`errno`、Fortran、融合运算（fused operation）、虚部（imaginary）、`Matrix`、多维（multidimensional）、随机数（random number）、实部（real）、行（row）、缩放（scaling）、大小（size）、`sizeof`、切片（slicing）、下标访问（subscripting）、均匀分布（uniform distribution）

习题

1. `a.apply(f)` 和 `apply(f,a)` 所接受的函数实参 f 是不一样的。为这两种用法各写一个 `triple()` 函数，并分别用它们把数组 { 1 2 3 4 5 } 的元素变成三倍。再定义一个单独的 `triple()` 函数，使它能同时用于 `a.apply(triple)` 和 `apply(triple,a)`。解释一下：为什么让每个交给 `apply()` 使用的函数都写成那种样子可能是个坏主意。
2. 把习题 1 重做一遍，但改用函数对象（function object），而不是函数。提示：`Matrix.h` 里有例子。
3. 仅限专家级（用本书介绍的设施无法做到）：写一个 `apply(f,a)`，使它既能接受 `void (T&)`、`T (const T&)`，也能接受与它们对应的函数对象。提示：`Boost::bind`。
4. 让高斯消去法程序跑起来；也就是说，把它补全、编译，并用一个简单的例子测试它。
5. 拿 `A={ {0 1} {1 0} }` 和 `b={ 5 6 }` 去试高斯消去法程序，看它失败。然后再试 `elim_with_partial_pivot()`。
6. 在高斯消去法的例子里，用循环替换向量运算 `dot_product()` 和 `scale_and_add()`。测试，并说说代码的清晰程度怎么样。
7. 不使用 `Matrix` 库重写高斯消去法程序；也就是说，用内置数组或 `vector`，而不用 `Matrix`。
8. 给高斯消去法做动画演示。
9. 重写非成员的 `apply()` 函数，让它返回一个 `Matrix`，元素类型为所施加函数的返回类型；也就是说，`apply(f,a)` 应当返回 `Matrix<R>`，其中 R 是 f 的返回类型。警告：这个解答需要用到本书未介绍的模板知识。
10. 你的 `default_random_engine` 有多随机？写一个程序，输入两个整数 n 和 d，调用 `randint(n)` 共 d 次，并记录结果。输出 $[0:n)$ 中每个值被抽到的次数，再“目测”这些计数有多接近。用较小的 n 和较小的 d 试一试，看看只抽少量随机数是否会带来明显的偏差。
11. 写一个 `swap_columns()`，与 §24.5.3 的 `swap_rows()` 相对应。显然，为此你得读懂现有的部分 `Matrix` 库代码。不必太担心效率：不可能让 `swap_columns()` 跑得像 `swap_rows()` 一样快。
12. 实现下面两个函数：

```
Matrix<double> operator*(Matrix<double,2>&,Matrix<double>&);
```

和

```
Matrix<double,N> operator+(Matrix<double,N>&,Matrix<double,N>&)
```

如有需要，就在教科书中查一下它们的数学定义。

后记

如果你对数学不太自在，那你多半不喜欢这一章，而且多半会选择一个不太需要本章这些信息的行业。反过来，如果你确实喜欢数学，我们希望你能体会到：数学的基本概念可以在代码中被表示得多么贴近。

25　嵌入式系统编程

"'Unsafe' means 'Somebody may die.'"
"所谓'不安全'，意思就是'可能有人会死'。"
——安全官

本章给出嵌入式系统编程的概貌。也就是说，我们讨论的主题主要与为"小装置"编写程序有关——这些装置看起来不像带屏幕和键盘的传统计算机。我们重点关注"贴近硬件"地工作所需要的原则、编程技术、语言设施和编码标准。涉及的主要语言问题包括资源管理、内存管理、指针与数组的使用以及位操作。我们强调安全用法，也强调对最低层特性的替代方案。本章不打算介绍专门的机器体系结构或对硬件设备的直接访问——那是专门文献和手册的任务。作为示例，我们给出一个加密/解密算法的实现。

25.1　嵌入式系统

世界上大多数计算机并不能一眼就认出是计算机。它们只是某个更大的系统或"小装置"的一部分。例如：

- 汽车：一辆现代汽车可能装有几十台计算机，分别控制燃油喷射、监测发动机性能、调节收音机、控制刹车、监视轮胎是否充气不足、控制挡风玻璃雨刷，等等。
- 电话：一部手机里至少有两台计算机；其中一台往往专门用于信号处理。
- 飞机：一架现代飞机上的计算机负责从运行乘客娱乐系统到为使飞行性能最佳而摆动翼尖的各种任务。
- 照相机：有些照相机带有五个处理器，甚至每个镜头都有自己独立的处理器。
- 信用卡（"智能卡"那一类）
- 医疗设备的监视器与控制器（例如 CAT 扫描仪）
- 电梯（升降机）
- PDA（个人数字助理）
- 打印机控制器
- 音响系统
- MP3 播放器
- 厨房电器（例如电饭煲和面包机）
- 电话交换机（通常由数千台专用计算机构成）
- 泵控制器（用于水泵、油泵等）
- 焊接机器人：有些用于人类焊工无法进入的狭窄或危险场所
- 风力涡轮机：有些能发出数兆瓦的电力，高达 200m（650ft）
- 海堤闸门控制器
- 装配线质量监测器
- 条形码阅读器
- 汽车装配机器人
- 离心机控制器（用于许多医学分析过程）
- 磁盘驱动器控制器

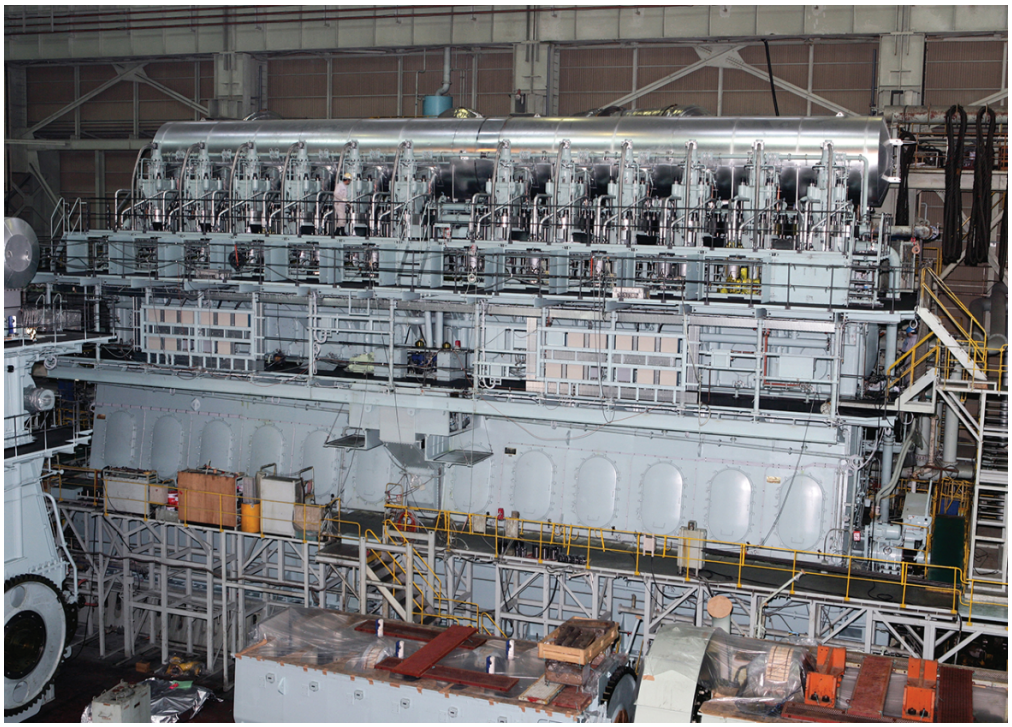
25.1 嵌入式系统
25.2 基本概念
25.2.1 可预测性
25.2.2 理想
25.2.3 与故障共存
25.3 内存管理
25.3.1 自由存储的问题
25.3.2 通用自由存储的替代方案
25.3.3 池示例
25.3.4 栈示例
25.4 地址、指针与数组
25.4.1 未受检的转换
25.4.2 一个问题：失灵的接口
25.4.3 一种解决方案：接口类
25.4.4 继承与容器
25.5 位、字节与字
25.5.1 位与位运算
25.5.2 bitset
25.5.3 有符号与无符号
25.5.4 位操作
25.5.5 位域
25.5.6 示例：简单加密
25.6 编码标准
25.6.1 编码标准应当是什么？
25.6.2 示例规则
25.6.3 真实的编码标准

这些计算机是更大系统的组成部分。这样的“大系统”通常看起来不像计算机，我们一般也不把它们当成计算机。看到一辆汽车沿街驶来时，我们不会说：“看，那是一个分布式计算机系统！”嗯，汽车确实也是一个分布式计算机系统，但它的运行与机械、电子和电气部件结合得如此紧密，以致我们无法孤立地看待其中的计算机。对这些计算的约束（时间上的与空间上的），乃至程序正确性的定义，都无法与更大的系统分开。嵌入式计算机常常控制着某个物理设备，而计算机行为的正确性就被定义为该物理设备的正确运行。考虑一台大型船用柴油机：

请注意 5 号气缸顶部的那位工程师。这是一台很大的发动机，是驱动最大船舶的那一类发动机。这样一台发动机如果出了故障，你会在早晨报纸的头版上读到相关消息。在这台发动机上，每个气缸盖上都装着一套由三台计算机组成的气缸控制系统。每套气缸控制系统通过两个独立的网络连接到发动机控制系统（另外三台计算机）。发动机控制系统再连接到控制室，工程师可以在那里通过专用的图形用户界面系统与它通信。整个系统还可以由航运公司的控制中心通过无线电（经卫星）远程监视。更多例子见第 1 章。

那么，从程序员的角度看，运行在这台发动机所包含的计算机里的程序有什么特别之处？更一般地说，对于各种嵌入式系统，有哪些问题会变得突出，而我们在“普通程序”中通常不必太担心？

- 通常，可靠性至关重要：故障可能非常惊人、代价高昂（比如“数十亿美元”），还可能致命（对于失事船上的人，或其所处环境中的动物）。



- 通常，资源（内存、处理器周期、电能）是有限的：这在发动机计算机上大概算不上问题，但想一想智能手机、传感器、空间探测器上的计算机等等。在这个双处理器 2GHz、8GB 内存的笔记本电脑随处可见的世界里，飞机或空间探测器中一台关键计算机可能只有 60MHz 和 256KB，而一个小装置的 RAM 甚至不到 1MHz、只有几百字。为抵御环境危害（振动、碰撞、不稳定的供电、高温、严寒、潮湿、工人踩踏等）而设计的计算机，通常比驱动学生笔记本电脑的处理器慢得多。
- 通常，实时响应必不可少：如果喷油器错过一个喷油周期，一个输出 100,000Hp 的极复杂系统就可能遇到麻烦；错过几个周期——也就是在一秒左右的时间里没能正确工作——直径可达 33ft（10m）、重达 130 吨的螺旋桨就可能开始出现异常。你绝对不希望发生这种事。
- 通常，系统必须连续多年无故障运行：也许系统运行在绕地球飞行的通信卫星上，也许系统本身太便宜、数量又太多，以致任何明显的返修率都会让制造商破产（想想 MP3 播放器、带嵌入式芯片的信用卡和汽车喷油器）。在美国，骨干电话交换机强制要求的可靠性指标是 20 年停机 20 分钟（更别提每次想改程序就把这样的交换机停掉）。
- 通常，现场维护不可行或极少进行：大约每隔一年，当船的其他部件需要维护、而且所需的计算机专家正好在合适的时间到达合适的地点时，你可以把一艘大船开进港口来维护它的计算机。计划外的现场维护是不可行的（船在太平洋中央遭遇大风暴时，不允许存在缺陷）。你根本无法派人去修理一颗环绕火星飞行的空间探测器。

很少有什么系统会同时受到上述所有约束，而任何受到其中哪怕一种约束的系统，都属于专家的领域。我们的目标不是把你变成“速成专家”——试图那样做既很愚蠢，也很不负责任。我们的目标是让你了解基本的问题，以及解决这些问题所涉及的基本概念，使你能够体会构建这类系统所需的某些技能。也许你会对获取这些宝贵的技能产生兴趣。设计与实现嵌入式系统的人，对我们技术文明的许多方面都至关重要。在这个领域，专业人士可以大有作为。

这些内容对新手有意义吗？对 C++ 程序员有意义吗？两个答案都是肯定的。嵌入式系统处理器的数量远远超过传统 PC。编程工作中很大一部分与嵌入式系统编程有关，所以你的第一份真正的工作可

能就涉及嵌入式系统编程。此外，本节开头那份嵌入式系统示例清单，取自本人亲眼见过用 C++ 完成的项目。

25.2 基本概念

嵌入式系统中所包含的计算机，其编程在很多方面与普通编程并无不同，因此本书介绍的大部分思想都适用。不过侧重点往往不同：我们必须根据任务的约束调整对编程语言设施的使用，而且常常要在最低层操作硬件：

- 正确性：这比通常情况更加重要。“正确性”并不只是一个抽象概念。在嵌入式系统的语境中，程序正确与否不仅意味着产生正确的结果，还意味着在正确的时间、以正确的顺序产生结果，并且只使用可接受的资源集合。理想情况下，什么算“正确”的细节会被仔细地规定下来，但这样的规定往往要经过一些实验才能完成。而关键的实验常常只有在整个系统（运行该程序的计算机只是其中一部分）建成之后才能进行。为一个嵌入式系统完整地规定正确性，可能同时是极其困难又极其重要的。在这里，“极其困难”可能意味着“在可用的时间和资源下不可能做到”；我们必须利用一切可用的工具和技术尽力而为。所幸，在某个给定领域里，规范、仿真、测试等技术的水平可以相当可观。而“极其重要”可能意味着“一旦失败就会造成伤害或毁灭”。
- 容错：我们必须仔细规定一个程序应当处理哪些情形。例如，对一个普通的学生程序来说，如果我们演示时把电源线踢掉，你大概会觉得不公平。断电并不属于普通 PC 应用应当处理的情形。然而，对嵌入式系统来说断电并不罕见，有些系统还被要求能够应对这种情况。例如，系统中的关键部分可能配有双电源、备用电池等。更糟的是，对某些应用来说，“我以为硬件是正常工作的”根本不能算理由。在长时间、大范围的各种条件下，硬件就是不会一直正常工作。例如，某些电话交换机和某些航空航天应用在编写时就假定：计算机内存中的某一位迟早会“决定”改变自己的值（例如从 0 变成 1）。或者它会“决定”自己喜欢 1 这个值，从而拒绝把它改成 0。只要内存足够大、使用时间足够长，这种错误行为最终总会发生。如果把内存暴露在强辐射下——比如地球大气层之外的那种辐射——它会来得更早。当我们开发一个系统（无论是否嵌入式）时，必须决定要提供哪种程度的硬件故障容忍。通常的默认做法是假定硬件按规范正常工作。而当我们面对更关键的系统时，这个假设就必须修改。
- 不停机：嵌入式系统通常必须长时间运行，既不能修改软件，也不能靠了解实现细节的熟练操作员干预。“长时间”可能是几天、几个月、几年，甚至硬件的整个寿命。这并不是嵌入式系统独有的，但它与绝大多数“普通应用”以及本书（到目前为止）的所有例子和习题都不同。这种“必须永远运行”的要求意味着要重视错误处理和资源管理。什么是“资源”？资源就是机器只能有限供应的东西；程序通过某种显式动作（“获取资源”“分配”）从系统取得资源，并通过显式或隐式的方式把它归还（“释放”“回收”“去分配”）。资源的例子包括内存、文件句柄、网络连接（套接字）和锁。属于长时间运行系统的程序必须释放它需要的每一项资源，只有少数它永久拥有的除外。例如，一个每天忘记关闭一个文件的程序，在大多数操作系统上活不过一个月左右。一个每天漏掉 100 字节未去分配的程序，一年就会浪费 32K 以上——这足以让一个小装置在几个月后崩溃。这类资源“泄漏”最讨厌的地方在于：程序会完好地运行好几个月，然后突然停止工作。如果程序终将崩溃，我们希望它尽早崩溃，以便补救问题。特别是，我们希望它在交给用户之前很久就崩溃。

- *实时约束*：如果某个响应必须在截止时间之前发生，我们可以把该嵌入式系统归为*硬实时*。如果响应大多数时候必须在截止时间之前发生，但偶尔的超时是可以承受的，我们就把它归为*软实时*。软实时的例子有车窗控制器和立体声放大器。车窗动作延迟零点几秒，人不会察觉；只有受过训练的听者才能听出音高变化上毫秒级的延迟。硬实时的例子是喷油器：它必须相对于活塞运动在精确的时刻“喷油”。即使时间偏差只有零点几毫秒，性能也会受损，发动机开始变差；严重的定时问题可能使发动机完全停转，从而可能导致事故或灾难。
- *可预测性*：这是嵌入式系统代码中的一个关键概念。显然，这个词有许多直观含义，但在这里——在嵌入式系统编程的语境中——我们采用一个专门的技术含义：如果某个操作在给定的计算机上每次执行所花的时间都相同，并且所有这类操作所花的时间也相同，那么该操作就是可预测的。例如，当 `x` 和 `y` 是整数时，`x+y` 每次执行所花的时间都相同，而 `xx+yy` 在 `xx` 和 `yy` 是另外两个整数时所花的时间也与它相同。通常，我们可以忽略与机器体系结构有关的执行速度细微变化（例如由缓存和流水线造成的差异），而直接依赖所需时间存在固定的常数上限这一事实。不可预测的操作（就这个词的这层含义而言）不能用于硬实时系统，在所有实时系统中也必须极其谨慎地使用。不可预测操作的经典例子是对一个元素个数未知且不易界定上界的列表做线性查找（例如 `find()`）。只有当我们能够可靠地预测元素个数、或者至少预测最大元素个数时，这样的查找在硬实时系统中才是可接受的；也就是说，要在给定的固定时间内保证响应，我们必须——也许要借助代码分析工具——计算出通向截止时间的每一条可能代码序列所需的时间。
- *并发*：嵌入式系统通常必须响应来自外部世界的事件。这就导致程序中许多事情“同时”发生，因为它们对应于真正同时发生的真实事件。同时处理若干动作的程序称为*并发*程序或*并行*程序。遗憾的是，并发这个引人入胜、困难而又重要的问题超出了本书的范围。

25.2.1 可预测性

从可预测性的角度看，C++ 相当不错，但并非完美。C++ 语言中的所有设施（包括虚函数调用）都是可预测的，只有以下例外：

- 使用 `new` 和 `delete` 的自由存储分配（见 §25.3）
- 异常（§19.5）
- `dynamic_cast`（§A.5.7）

对硬实时应用来说，必须避免使用这些设施。`new` 和 `delete` 的问题在 §25.3 中详细描述，那些问题是根本性的。请注意，标准库的 `string` 和标准容器（`vector`、`map` 等）会间接使用自由存储，因此它们也是不可预测的。`dynamic_cast` 的问题属于当前实现的问题，而不是根本性的问题。

异常的问题在于：面对某一个 `throw`，程序员如果不查看大段代码，就无法知道要花多长时间才能找到匹配的 `catch`，甚至不知道是否存在这样的 `catch`。在嵌入式系统程序中，最好确实有 `catch`，因为我们不能指望总有一位 C++ 程序员守在旁边准备使用调试器。异常的问题原则上可以用工具解决：对每个 `throw`，工具准确告诉你将调用哪个 `catch`、这个 `throw` 要花多长时间才能到达那里；但在目前，这还是个研究问题。所以如果你需要可预测性，就只能凑合使用基于返回码的错误处理，以及其他老式、繁琐但可预测的技术。

25.2.2 理想

编写嵌入式系统程序时存在一种危险：对性能和可靠性的追求会把程序员带回到只使用低级语言设施的老路上去。这种策略对个别小的代码片段是可行的。然而，它很容易把总体设计搞得一团糟，让人难以对正确性有信心，并增加构建系统所需的时间和金钱。

一如既往，我们的理想是在问题约束允许的范围内、在尽可能高的抽象层次上工作。不要沦落到去写华而不实的汇编代码！一如既往，尽可能把想法直接表达在代码里（在给定全部约束的前提下）。一如既往，努力写出最清晰、最干净、最易维护的代码。不到万不得已不要优化。对嵌入式系统来说，性能（时间上或空间上的）往往是必需的，但试图从每一小段代码里榨出性能是错误的做法。而且，对许多嵌入式系统来说，关键在于正确并且足够快；超过“足够快”之后，系统只是空闲着，直到需要下一个动作。试图让每几行代码都尽可能高效，会花掉大量时间，引入大量缺陷，而且常常错失优化机会，因为算法和数据结构变得难以理解、难以修改。例如，那种“低层优化”的做法常常错失内存优化的机会：几乎相同的代码出现在许多地方，却因为一些偶然的差异而无法共享。

约翰·本特利（John Bentley）以代码极其高效而闻名，他提出了两条“优化定律”：

- 第一定律：别做。
- 第二定律（仅供专家）：现在还别做。

在优化之前，先确保你理解了这个系统。只有到那时，你才有信心说它是——或者可以变得——正确而可靠的。把注意力放在算法和数据结构上。系统的一个早期版本能运行之后，再根据需要仔细测量并调整它。幸运的是，令人愉快的意外并不罕见：干净的代码有时运行得足够快，也不占用过多内存。不过别指望这一定成立——要测量。令人不快的意外同样并不罕见。

25.2.3 与故障共存

设想我们要设计并实现一个不允许失败的系统。所谓“不失败”，我们不妨定为“无需人工干预就能运行一个月”。我们必须防范哪些故障呢？太阳爆发这类情况可以排除，系统被大象踩踏大概也可以排除。然而，一般来说我们无法预知会出什么问题。对某个具体系统，我们可以而且必须假定哪类错误比其他错误更常见。例如：

- 电源浪涌 / 断电
- 连接器因振动而脱出插槽
- 系统被坠落的碎片击中，压坏处理器
- 系统跌落（磁盘可能因撞击而损坏）
- X 射线使某些内存位以语言定义下不可能的方式改变取值

瞬态错误通常最难发现。瞬态错误是指“有时候”发生、但并非程序每次运行都会发生的错误。例如，我们听说有一款处理器只在温度超过 130°F（54°C）时才会出错。按设计它根本不应该变得那么热；但当系统在车间里（无意中且偶尔地）被盖住时，它确实变得那么热，而在实验室测试时从来没有过热。

在实验室之外出现的错误最难修复。你很难想象其中涉及的设计与实现工作量：让喷气推进实验室（JPL）的工程师诊断火星巡视器上的软件和硬件故障（信号以光速传播，单程就要 20 分钟），并在弄清问题之后更新软件来修复它。

领域知识——也就是关于某个系统、它的环境以及它的用途的知识——对于设计并实现一个对错误有良好抵御能力的系统是必不可少的。这里我们只谈一些一般性的内容。请注意，我们在此提到的每一条“一般性内容”，都曾是成千上万篇论文和数十年研究与开发的主题。

- *防止资源泄漏*：不要泄漏。明确你的程序使用了哪些资源，并确保（完美地）节约使用它们。任何泄漏最终都会杀死你的系统或子系统。最基本的资源是时间和内存。通常，程序还会使用其他资源，例如锁、通信通道和文件。
- *复制*：如果某个系统在功能上关键地依赖某项硬件资源（例如一台计算机、一个输出设备、一个轮子），设计者就面临一个基本选择：系统是否应当包含该关键资源的多个副本？我们可以接受硬件损坏时系统失败，也可以提供一个备用件并让软件切换到备用件上使用。例如，船用柴油机的喷油器控制器就是由冗余网络连接的三台互为备份的计算机。请注意，“备用件”不必与原件完全相同（例如，空间探测器可能有一副主用的强天线和一副较弱的备份天线）。还要注意，当系统正常工作时，“备用件”通常还可以用来提升性能。
- *自检*：知道程序（或硬件）什么时候出错。硬件部件（例如存储设备）在这方面能帮上大忙：它们监视自身的错误、纠正小错误并报告重大故障。软件可以检查其数据结构的一致性、检查不变式（§9.4.3），并依赖内部的“健全性检查”（断言）。遗憾的是，自检本身也可能不可靠，必须小心避免报告错误这件事本身又引起错误——要彻底检查错误检查代码，真的很难。
- *让出错的代码有一条快速退路*：让系统模块化。把错误处理建立在模块之上：每个模块都有特定的任务要完成。如果一个模块……

它判定自己无法完成自己的任务时，可以把这一情况报告给另一个模块。让错误处理保持在一个模块内部，并且保持简单（这样它才更可能既正确又高效），同时让另一个模块负责处理严重错误。一个好的可靠系统是模块化的、多层次的。在每一层，严重错误都被报告给上一层的模块——最终也许报告给人。一个被告知发生了严重错误（另一个模块自己处理不了的错误）的模块，随后可以采取恰当的措施——也许包括重启那个检测到错误的模块，或者改跑一个不那么复杂（但更健壮）的“备份”模块。对于一个给定的系统，“一个模块”究竟指什么，是整个系统设计的一部分；不过你可以把它看成一个类、一个库、一个程序，或者一台计算机上的所有程序。

- 监视各子系统，以防它们自己无法或没有察觉到问题。在多层次的系统中，较高层可以监视较低层。许多真正不允许出故障的系统（例如船用发动机或空间站控制器）都为关键子系统准备了三份副本。这种三重化不只是为了有两个备份，还为了让“哪个子系统行为异常”的争议能靠 2 比 1 表决来裁定。在难以建立多层次组织的地方（也就是可能不允许出故障的系统或子系统的最高层），三重化尤其有用。

我们可以尽情设计，并尽我们所知小心地实现，但系统仍然会行为异常。在把系统交付给用户之前，必须对它进行系统而彻底的测试；参见第 26 章。

25.3 内存管理

计算机中两种最基础的资源是时间（用来执行指令）和空间（用来存放数据和代码的内存）。在 C++ 中，有三种为存放数据而分配内存的方式（§17.4、§A.4.2）：

- 静态内存：由链接器分配，只要程序还在运行就一直存在
- 栈（自动）内存：调用函数时分配，从函数返回时释放
- 动态（堆）内存：由 `new` 分配，由 `delete` 释放以便重复利用

我们不妨从嵌入式系统编程的角度来考察这几种方式。特别地，我们将从这样的任务的角度来考虑内存管理：这些任务认为可预测性（§25.2.1）至关重要，例如硬实时编程和安全性至关重要的编程。

静态内存在嵌入式系统编程中不构成特殊问题：一切都在程序开始运行之前、也就是在系统部署之前很久就已安排妥当。

栈内存可能成为问题，因为有可能用得太多，不过这一点并不难处理。系统设计者必须确定：程序的任何一次执行都不会让栈增长到超过可接受的限度。这通常意味着必须限制函数调用的最大嵌套层数；也就是说，我们必须能够证明一条调用链（例如 f1 调用 f2，……，fn）绝不会太长。在某些系统中，这导致禁止使用递归调用。对某些系统和某些递归函数来说，这样的禁令是合理的，但它并不是根本性的。例如，我知道 `factorial(10)` 最多调用 `factorial` 十次。不过，嵌入式系统程序员很可能还是愿意采用 `factorial` 的迭代实现（§15.5），以杜绝任何疑虑或意外。

动态内存分配通常被禁止或被严格限制；也就是说，`new` 或者被禁止，或者其使用被限制在启动阶段，而 `delete` 被禁止。基本原因是

- 可预测性：自由存储区的分配不可预测；也就是说，它不保证是常量时间的操作。通常它确实不是：在许多 `new` 的实现中，在分配并释放了大量对象之后，分配一个新对象所需的时间可能急剧增加。
- 碎片：自由存储区可能产生碎片；也就是说，在分配和释放对象之后，剩余未使用的内存可能被“碎片化”成许多小的未使用空间“空洞”。这些空洞毫无用处，因为每个空洞都太小，装不下应用所用的那种对象。因此，可用的自由存储区大小可能远远小于初始自由存储区大小减去已分配对象大小所得的值。

下一节将说明这种不可接受的状况是如何产生的。要点在于：对于硬实时系统或安全性至关重要的系统，我们必须避免那种同时使用 `new` 和 `delete` 的编程技术。接下来的几节将说明，如何用栈和池系统地避开自由存储区带来的问题。

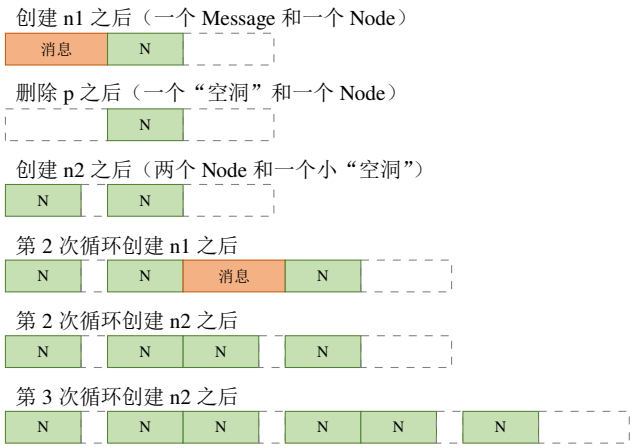
25.3.1 自由存储区的问题

`new` 到底有什么问题？其实，真正的问题出在把 `new` 和 `delete` 一起使用。请看下面这一串分配与释放操作的结果：

```
Message* get_input(Device&);    // 在自由存储区上创建一个 Message

while (/* ... */) {
    Message* p = get_input(dev);
    // ...
    Node* n1 = new Node(arg1,arg2);
    // ...
    delete p;
    Node* n2 = new Node (arg3,arg4);
    // ...
}
```

每绕循环一圈，我们就创建两个 `Node`，而在这个过程中还会创建一个 `Message` 再把它删掉。作为根据来自某个“设备”的输入构建数据结构的一部分，这样的代码并不罕见。看着这段代码，我们也许会以为每绕一圈会“消耗”`2*sizeof(Node)` 字节的内存（再加上自由存储区的开销）。可惜，并不

能保证内存的“消耗”仅限于预期且期望的 2*sizeof(Node) 字节。事实上，情况多半并非如此。假设有一个简单的（但并非不现实的）内存管理器。再假设 Message 比 Node 略大一些。我们可以像下面这样把自由空间的使用情况画出来：橙色表示 Message，绿色表示 Node，纯白色表示“空洞”（也就是“未使用的空间”）：

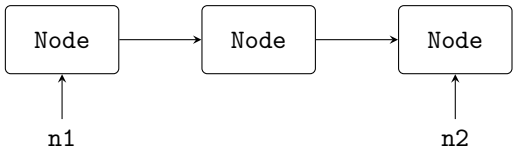


橙色＝ Message，绿色＝ Node，白色虚线＝空洞（未使用的空间）

于是，每执行一次循环，我们都在自由存储区上留下一些未使用的空间（“一个空洞”）。那也许只有几个字节，但如果我们无法利用这些空洞，后果就和内存泄漏一样糟糕——而即便是很小的泄漏，最终也会拖垮长期运行的程序。内存中的自由空间散落成许多小得无法用来分配新对象的“空洞”，这叫做内存碎片。基本上，自由存储区管理器最终会用光所有大到足以容纳程序所用那类对象的空洞，只剩下小得没有用处的空洞。对几乎所有大量使用 new 和 delete 的长期运行的程序来说，这都是一个严重问题；长运行程序中很大一部分内存被不可用的碎片占据，这种情况并不少见。这通常会使执行 new 所需的时间急剧增加，因为它必须在大量对象和碎片中搜索一块大小合适的空间。显然，这不是嵌入式系统所能接受的行为。在那些设计得过于天真的非嵌入式系统中，这也可能成为严重问题。

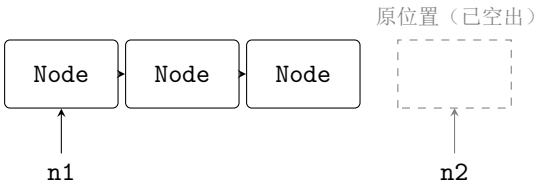
为什么“语言”或“系统”不能处理这个问题呢？换个问法：我们难道不能干脆把程序写成不产生这类“空洞”的样子吗？我们先来考察对付内存中所有这些无用的小“空洞”的最显而易见的办法：移动各个 Node，让所有自由空间压实成一整块连续区域，好用它来分配更多对象。

可惜，“系统”做不到这一点。原因在于 C++ 代码直接引用内存中的对象。例如，指针 n1 和 n2 里存的是真正的内存地址。如果我们移动了它们所指的对象，这些地址就不再指向正确的对象了。假设我们（在某个地方）保存了指向所创建节点的指针。我们可以像下面这样表示数据结构中相关的部分：



现在我们移动对象来压实内存，好让所有未使用的内存都集中到一处：

可惜，我们在移动对象时没有更新指针，结果把这些指针弄得一团糟。为什么不在移动对象的同时更新指针呢？我们可以写一个程序来做这件事，但前提是我们知道数据结构的细节。一般来说，“系统”（C++ 的运行时支持系统）根本不知道指针在哪儿；也就是说，给定一个对象，“程序里现在有哪些指针指向这个对象？”这个问题没有好的答案。即使这个问题能轻易解决，这种做法（称为压实式垃



圾回收）也并不总是正确的做法。例如，为了工作良好，它通常需要比程序实际所需内存多一倍以上的内存，才能跟踪指针并在内存中搬移对象。嵌入式系统上可能没有这么多额外内存。此外，高效的压实式垃圾回收器很难做得可预测。

我们当然可以针对自己的数据结构回答“指针在哪儿？”这个问题，然后压实它们。那样做可行，但更简单的办法是从一开始就避免碎片。在刚才的例子中，我们本可以先把两个 Node 都分配好，再分配消息：

```
while ( ... ) {
    Node* n1 = new Node;
    Node* n2 = new Node;
    Message* p = get_input(dev);
    // ... 把信息存进节点 ...
    delete p;
    // ...
}
```

然而，重新安排代码来避免碎片通常并不容易。要可靠地做到这一点，往好了说也是非常困难的，而且往往与良好代码的其他规则相冲突。因此，我们宁愿把自由存储区的使用限制在那些从一开始就不会造成碎片的方式上。通常，预防问题胜过解决问题。

试一试

试一试

把上面的程序补全，打印出所创建对象的地址和大小，看看你机器上是否出现“空洞”、以及是怎样出现的。如果你有时间，还可以画出像上面那样的内存布局图，以便更好地看清到底发生了什么。

25.3.2 通用自由存储区的替代方案

所以，我们不能造成碎片。那该怎么办呢？第一个简单的观察是：new 本身不会造成碎片；它需要 delete 才能造出空洞。因此我们从禁止 delete 开始。这意味着，一个对象一旦被分配，就会永远成为程序的一部分。

在没有 delete 的情况下，new 是可预测的吗？也就是说，所有 new 操作花费的时间都一样吗？是的，在一切常见的实现中都是如此，但标准实际上并没有保证这一点。通常，嵌入式系统有一串启动代码，它在初次上电或重启之后把系统置为“准备就绪”状态。在这段时间里，我们可以按自己喜欢的方式分配内存，只要不超过允许的上限。我们可以决定在启动期间使用 new。另一种做法（或者同时并用）是为将来预留全局（静态）内存。出于程序结构的考虑，全局数据通常最好避免，但用这种语言机制来预分配内存可能是明智的。这方面的确切规则应当写入系统的编码规范（见 §25.6）。

有两种数据结构对可预测的内存分配特别有用：

- 栈：栈是这样一种数据结构：你可以分配任意数量的内存（不超过给定的最大尺寸），并且只能释放最近一次分配；也就是说，栈只能在顶部增长和收缩。不可能出现碎片，因为两次分配之间不可能出现“空洞”。
- 池：池是一组尺寸相同的对象。只要分配的对象数不超过池所能容纳的数量，我们就可以分配和释放对象。不可能出现碎片，因为所有对象的尺寸都相同。

对栈和池来说，分配和释放都是可预测而且快速的。

因此，对于硬实时系统或关键系统，我们可以按需要自己定义栈和池。更好的做法是：我们应当能够直接使用由别人规定、实现并测试过的栈和池（只要其规范符合我们的需要）。

注意，C++ 标准容器（vector、map 等）以及标准 string 都不能使用，因为它们间接用到了 new。你可以构建（购买或借用）“类标准”容器，使其可预测；但你的实现自带的那套默认容器并没有针对嵌入式系统的用途做约束。

注意，嵌入式系统通常有非常严格的可靠性要求，因此无论我们选择哪种方案，都必须确保不会因为退回到大量直接使用低层设施而使编程风格倒退。代码里满是指针、显式类型转换之类的东西，要保证它们正确，难到了不合理的程度。

25.3.3 池示例

池是这样一种数据结构：我们可以从中分配某一给定类型的对象，稍后再释放（归还）这些对象。池包含的对象数量有一个上限；这个数量在创建池时指定。用绿色表示“已分配的对象”，蓝色表示“可供分配为对象的空间”，我们可以像下面这样把池画出来：

一个 Pool 可以这样定义：

```
template<typename T, int N>
class Pool {                    // N 个类型为 T 的对象组成的池
public:
    Pool();                     // 创建一个由 N 个 T 组成的池
    T* get();                   // 从池中取出一个 T；若无空闲 T 则返回 0
    void free(T*);              // 把 get() 发出的一个 T 还给池
    int available() const;      // 空闲 T 的个数
private:
    // 存放 T[N] 的空间，以及记录哪些 T 已分配、哪些未分配的数据
    //  （例如一个空闲对象链表）
};
```

每个 Pool 对象都有一种元素类型和一个最大对象数量。我们可以这样使用 Pool：

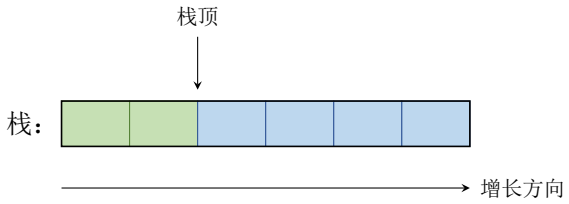
```
Pool<Small_buffer,10> sb_pool;
Pool<Status_indicator,200> indicator_pool;

Small_buffer* p = sb_pool.get();
// ...
sb_pool.free(p);
```

程序员的责任是确保池永远不会被耗尽。“确保”的确切含义取决于应用。对某些系统，程序员必须把代码写成这样：除非确实有对象可分配，否则绝不调用 `get()`。在另一些系统上，程序员可以检查 `get()` 的结果，并在结果为 0 时采取某种补救措施。一个典型的例子是一套被设计成最多同时处理 100 000 通呼叫的电话系统。每通呼叫都会分配某种资源，例如一个拨号缓冲区。如果系统的拨号缓冲区用完了（例如 `dial_buffer_pool.get()` 返回 0），系统就拒绝建立新连接（并且可能“掐断”少数已有的呼叫以腾出容量）。想打电话的人可以稍后再试。

自然，我们的 `Pool` 模板只是池这一普遍想法的一种变体。例如，在对内存分配的限制不那么严酷的地方，我们可以定义这样的池：其元素数量在构造函数中指定；甚至可以定义这样的池：如果我们需要比最初指定的更多的对象，其元素数量还可以在以后更改。

25.3.4 栈示例

栈是这样一种数据结构：我们可以从中分配内存块，并释放最近一次分配的块。用绿色表示“已分配的内存”，蓝色表示“可供分配的空间”，我们可以像下面这样把栈画出来：



如图所示，这个栈朝右边“增长”。

我们可以像定义对象池那样，定义一个对象栈：

```
template<typename T, int N>
class Stack {                    // N 个类型为 T 的对象组成的栈
    // ...
};
```

然而，大多数系统都需要分配大小各异的对象。栈能做到这一点，而池不能，所以下面我们展示如何定义一个栈，从它那里分配大小各异的“原始”内存，而不是固定大小的对象：

```
template<int N>
class Stack {                    // N 字节的栈
public:
    Stack();                     // 创建一个 N 字节的栈
    void* get(int n);            // 从栈中分配 n 个字节;
                                 // 若无空闲空间则返回 0
    void free();                 // 把 get() 返回的最后一个值还给栈
    int available() const;       // 可用字节数
private:
    // 存放 char[N] 的空间，以及记录哪些空间已分配、哪些未分配的数据
    //  (例如一个栈顶指针)
};
```

由于 get() 返回一个 void*，指向所需数量的字节，把这些内存转换成我们想要的那种对象就成了我们自己的事。我们可以这样使用这样的栈：

```
Stack<50*1024> my_free_store;    // 50K 存储空间，用作栈

void* pv1 = my_free_store.get(1024);
int* buffer = static_cast<int*>(pv1);


void* pv2 = my_free_store.get(sizeof(Connection));
Connection* pconn = new(pv2) Connection(incoming,outgoing,buffer);
```

关于 static_cast 的用法，见 §17.8。new(pv2) 这种写法叫作“定位 new”（placement new），它的意思是“在 pv2 所指的空间里构造一个对象”。它不分配任何东西。这里的前提是：类型 Connection 有一个能够接受参数列表 (incoming,outgoing,buffer) 的构造函数。如果没有这样的构造函数，程序就编译不过。

当然，我们的 Stack 模板只是“栈”这一普遍想法的一种变体。例如，在内存分配方面的限制不那么苛刻（不那么 Draconian）的场合，我们可以定义这样的栈：可供分配的字节数在构造函数中指定。

25.4 地址、指针与数组

可预测性是某些嵌入式系统的需要；可靠性则是所有系统都关心的事。这促使人们尽量避免那些已被证明容易出错的语言特性和编程技术（至少在嵌入式系统编程的语境里如此，尽管未必在所有场合都如此）。这方面最主要的“嫌疑犯”是粗心大意地使用指针。有两类问题格外突出：

- 显式（不加检查、不安全）的类型转换
- 把指向数组元素的指针传来传去

前一类问题通常只要严格限制显式类型转换（强制转换）的使用就能解决。指针与数组的问题则更微妙，需要真正理解，最好用（简单的）类或库设施来处理（例如 array，见 §20.9）。因此，本节着重讨论后一类问题该怎样处理。

25.4.1 未检查的转换

物理资源（例如外部设备的控制寄存器）以及针对它们的最基本的软件控制，在底层系统中通常都位于特定地址上。我们不得不把这些地址写进程序，并给这类数据一个类型。例如：

```
Device_driver* p = reinterpret_cast<Device_driver*>(0xffb8);
```

另见 §17.8。做这类编程时，你手边得摊着手册、或者开着在线文档。硬件资源——也就是资源寄存器（们）的地址（用整数表示，而且常常是十六进制整数）——与“操作该硬件资源的软件所用的指针”之间的对应关系十分脆弱。你必须靠自己想清楚，编译器帮不上多少忙（因为这不是编程语言层面的问题）。通常，从 `int` 到指针类型的一次简单的（难看的、完全不加检查的）`reinterpret_cast`，就是从应用程序通到其真正重要的硬件资源的那条链路中最关键的一环。

凡是显式转换（`reinterpret_cast`、`static_cast` 等，见 §A.5.7）并非必需的地方，就不要用它。这类转换（强制转换）真正必要的场合，远比那些主要经验来自 C 和 C 风格 C++ 的程序员通常以为的要少得多。

25.4.2 一个问题：有缺陷的接口

正如 §18.6.1 提到的，数组常常以“指向某个元素的指针”（往往是指向首元素的指针）的形式传给函数。这样一来，数组就“丢掉”了自己的大小，于是接收函数无法直接判断这个指针到底指向多少个元素——甚至无法判断它是否指向任何元素。这是许多微妙而且难以修复的错误的根源。这里，我们考察这类数组/指针问题的几个例子，并给出另一种做法。我们先看一个设计得很糟（但遗憾的是并不罕见）的接口，然后再逐步改进它。请看：

```
void poor(Shape* p, int sz)              // 糟糕的接口设计
{
    for (int i = 0; i<sz; ++i) p[i].draw();
}


void f(Shape* q, vector<Circle>& s0)   // 非常糟糕的代码
{
    Polygon s1[10];
    Shape s2[10];
    // 初始化
    Shape* p1 = new Rectangle{Point{0,0},Point{10,20}};
    poor(&s0[0],s0.size());          // #1 (把 vector 里的数组传进去)
    poor(s1,10);                     // #2
    poor(s2,20);                     // #3
    poor(p1,1);                      // #4
    delete p1;
    p1 = 0;
    poor(p1,1);                      // #5
    poor(q,max);                     // #6
}
```

函数 poor() 是糟糕接口设计的一个例子：它提供的接口让调用者有大把机会犯错，却几乎没给实现者留下任何防御这类错误的机会。

试一试

试一试

在往下读之前，先试着数一数：你能在 f() 里找出多少个错误？具体地说，poor() 的哪几个调用会让程序崩溃？

乍一看，这些调用都挺像样；可正是这类代码让程序员熬过一个个调试的长夜，也让质量工程师做噩梦。

1. 传入了错误的元素类型，例如 poor(&s0[0],s0.size())。另外，s0 也可能是空的，这时 &s0[0] 就是错的。
2. 使用了“魔法常量”（此处是对的）：poor(s1,10)。此外，元素类型也不对。
3. 使用了“魔法常量”（此处是错的）：poor(s2,20)。
4. 正确（很容易验证）：第一次调用 poor(p1,1)。
5. 传入了空指针：第二次调用 poor(p1,1)。
6. 可能正确：poor(q,max)。仅看这段代码片段，我们无法确定。要判断 q 是否指向一个至少有 max 个元素的数组，我们必须找到 q 和 max 的定义，并确定它们在我们使用它们的这一点的取值。

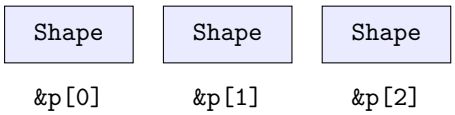
每一种情况下，错误本身都很简单。我们遇到的并不是什么微妙的算法问题或数据结构问题。问题在于：poor() 的接口把一个以指针形式传入的数组牵扯了进来，这就打开了一整类问题的可能性。你也许已经体会到，我们使用 p1、s0 这类“技术性”却毫无帮助的名字，是怎样把问题掩盖起来的。不过，那些便于记忆却具有误导性的名字，会让这类问题更难被发现。

理论上，编译器能查出其中少数几个错误（例如 p1==0 时的第二次 poor(p1,1) 调用），但说实在的，这个例子之所以没酿成灾难，只是因为编译器抓住了“定义抽象类 Shape 的对象”这一企图。然而这与 poor() 的接口问题无关，所以我们不该因此太安心。下面我们改用 Shape 的一个非抽象变体，以免被别的事情分散注意力、偏离接口问题本身。

为什么 poor(&s0[0],s0.size()) 这个调用是错的？&s0[0] 指的是一个 Circle 数组的第一个元素，它的类型是 Circle*。我们需要的是 Shape*，而我们传进去的确实是指向“派生自 Shape 的类的对象”的指针（这里是 Circle*）。这显然是可以接受的：正是靠这种转换，我们才能进行面向对象编程，通过共同的接口（这里是 Shape）访问各种类型的对象（§14.2）。可是，poor() 并不只是把这个 Shape* 当作指针来用；它把它当数组用，靠下标一路扫过这个数组：

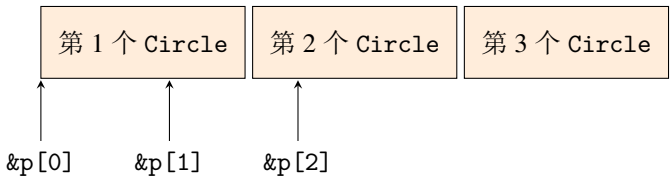
```
for (int i = 0; i<sz; ++i) p[i].draw();
```

也就是说，它依次查看从内存位置 &p[0]、&p[1]、&p[2] 等开始的那些对象：



从内存地址上看，这些指针彼此相隔 sizeof(Shape)（§17.3.1）。对 poor() 的调用者来说不幸的是，sizeof(Circle) 比 sizeof(Shape) 大，所以内存布局可以画成这样：

也就是说，poor() 是在用一个指向某个 Circle 中间某个位置的指针去调用 draw()！这很可能立刻造成灾难（崩溃）。



poor(s1,10) 这个调用更隐蔽。它依赖一个“魔法常量”，所以立刻就该被怀疑为维护隐患；但这里还有更深一层的问题。使用 Polygon 数组之所以没有立刻遭遇我们在 Circle 上看到的问题，唯一的原因是 Polygon 没有给它的基类 Shape 增加数据成员（而 Circle 增加了；见 §13.8 和 §13.12）。也就是说，sizeof(Shape)==sizeof(Polygon)；更一般地说，Polygon 与 Shape 有相同的内存布局。换句话说，我们只是“运气好”；Polygon 的定义稍有改动就会导致崩溃。所以 poor(s1,10) 能用，但它是一个迟早会发作的错误。这绝对算不上高质量的代码。

我们在这里看到的，就是下面这条一般性语言规则在实现层面的原因：“D 是一种 B”并不意味着“Container<D> 是一种 Container<B>”（§19.3.3）。例如：

```
class Circle : public Shape { /* . . . */ };

void fv(vector<Shape>&);
void f(Shape &);

void g(vector<Circle>& vd, Circle & d)
{
    f(d);          // 可以：从 Circle 到 Shape 的隐式转换
    fv(vd);        // 错误：无法从 vector<Circle> 转换为 vector<Shape>
}
```

好吧，使用 poor() 确实是很糟糕的代码；可是这样的代码能算嵌入式系统代码吗？换句话说，在那些安全性或性能至关重要的领域里，这类问题该不该让我们操心？我们能不能把它当作“非关键系统的程序员才会碰上的隐患”，耸耸肩说一句“别那么做”就算了？嗯，许多现代嵌入式系统都严重依赖图形用户界面，而图形界面几乎总是按我们例子中那种面向对象的方式组织起来的。例子包括 iPod 的用户界面、某些手机的界面，以及从各种“小玩意儿”一直到飞机的操作员显示屏。另一个例子是：同类小装置（比如各种电动机）的控制器可以构成一个经典的类层次。换句话说，这类代码——尤其是这类函数声明——恰恰是我们应当操心的代码。我们需要一种更安全的方式来传递关于数据集合的信息，同时又不引入其他严重问题。

所以，我们不想再以“指针加大小”的形式把内置数组传给函数。那我们该怎么做？最简单的办法是传递容器的引用，例如 vector 的引用。我们为

```
void poor(Shape* p, int sz);
```

所看到的那些问题，对于

```
void general(vector<Shape>&);
```

根本不可能发生。如果你所处的编程环境允许使用 std::vector（或与之相当的设施），那就在接口里始终如一地使用 vector（或相当的东西）；绝不要以“指针加大小”的形式传递内置数组。

如果你没法把自己限制在 vector 或相当的设施之内，那你就进入了一片更困难的领域：那里的解决办法涉及并不简单的技术和语言特性——尽管我们提供的那个类（Array_ref）用起来很直接。

25.4.3 解决方案：接口类

遗憾的是，在许多嵌入式系统里我们不能使用 std::vector，因为它依赖自由存储（free store）。要解决这个问题，可以给 vector 一个特殊实现，或者（更容易）使用一个行为像 vector、但不做内存管理的容器。在勾勒这样一个接口类之前，先想一想我们希望它具备哪些性质：

- 它是对内存中对象的引用（它不拥有对象，不分配对象，也不删除对象，等等）。
- 它“知道”自己的大小（因此有可能做范围检查）。
- 它“知道”自己元素的精确类型（因此不会成为类型错误的来源）。
- 它传递（拷贝）起来的开销与一个（指针，计数）对一样小。
- 它不会隐式转换成指针。
- 很容易表达接口对象所描述的元素范围的一个子范围。
- 它用起来和内置数组一样方便。

我们只能做到近似“和内置数组一样方便”。我们并不希望它方便到让错误开始变得容易发生。

下面就是这样一个类：

```
template<typename T>
class Array_ref {
public:
    Array_ref(T* pp, int s) :p{pp}, sz{s} { }
    T& operator[ ](int n) { return p[n]; }
    const T& operator[ ](int n) const { return p[n]; }
    bool assign(Array_ref a)
    {
        if (a.sz!=sz) return false;
        for (int i=0; i<sz; ++i) { p[i]=a.p[i]; }
        return true;
    }
    void reset(Array_ref a) { reset(a.p,a.sz); }
    void reset(T* pp, int s) { p=pp; sz=s; }
    int size() const { return sz; }
    // 默认的拷贝操作:
    // Array_ref 不拥有任何资源
    // Array_ref 具有引用语义
private:
    T* p;
    int sz;
};
```

Array_ref 已经接近最小化了：

- 没有 push_back()（那会需要自由存储），也没有 at()（那会需要异常）。
- Array_ref 是一种引用，所以拷贝它只是拷贝 (p,sz)。
- 用不同的数组初始化，可以得到类型相同但大小不同的 Array_ref。
- 用 reset() 更新 (p,size)，可以改变一个已有 Array_ref 的大小（许多算法需要指定子范围）。
- 没有迭代器接口（但需要时很容易加上）。事实上，从概念上说，Array_ref 非常接近“由两个迭代器描述的范围”。

Array_ref 不拥有自己的元素；它不做内存管理；它只是一个用来访问和传递元素序列的机制。在这一点上，它与标准库的 array（§20.9）不同。

为了便于创建 Array_ref，我们提供了几个有用的辅助函数：

```
template<typename T> Array_ref<T> make_ref(T* pp, int s)
{
    return (pp) ? Array_ref<T>{pp,s} : Array_ref<T>{nullptr,0};
}
```

如果我们用一个指针来初始化 Array_ref，就必须显式地给出大小。这显然是个弱点，因为它给了我们一个给错大小的机会。它还给了我们另一个机会：使用一个由“派生类数组隐式转换成基类指针”得来的指针，例如把 Polygon[10] 转成 Shape*（正是 §25.4.2 里那个糟糕透顶的老问题）。不过有时候，我们只能相信程序员。

我们决定对空指针格外小心（因为它们是常见的问题来源），并且对空 vector 也采取了类似的预防措施：

```
template<typename T> Array_ref<T> make_ref(vector<T>& v)
{
    return (v.size()) ? Array_ref<T>{&v[0],v.size()} : Array_ref<T>{nullptr,0};
}
```

这里的想法是传递 vector 内部的那个元素数组。我们在这里关心 vector，尽管在 Array_ref 能派上用场的那类系统里，vector 往往并不合适。原因在于：它与那些可以在那里使用的容器（例如基于内存池的容器，见 §25.3.3）共享一些关键性质。

最后，我们来处理编译器已知大小的内置数组：

```
template <typename T, int s> Array_ref<T> make_ref(T (&pp)[s])
{
    return Array_ref<T>{pp,s};
}
```

这个古怪的 T(&pp)[s] 写法把参数 pp 声明为“对含有 s 个 T 型元素的数组的引用”。这让我们可以用一个数组来初始化 Array_ref，并且记住它的大小。我们无法声明空数组，所以不必测试元素个数是否为零：

```
Polygon ar[0];              // 错误：没有元素
```

有了 Array_ref，我们就可以试着重写那个例子：

```
void better(Array_ref<Shape> a)
{
    for (int i = 0; i<a.size(); ++i) a[i].draw();
}

void f(Shape* q, vector<Circle>& s0)
{
    Polygon s1[10];
    Shape s2[20];
    // 初始化
    Shape* p1 = new Rectangle{Point{0,0},Point{10,20}};
    better(make_ref(s0));        // 错误：需要 Array_ref<Shape>
    better(make_ref(s1));        // 错误：需要 Array_ref<Shape>
    better(make_ref(s2));        // 可以（不需要转换）
    better(make_ref(p1,1));      // 可以：一个元素
    delete p1;
    p1 = 0;
    better(make_ref(p1,1));      // 可以：零个元素
    better(make_ref(q,max));     // 可以（如果 max 是合适的）
}
```

我们看到了这些改进：

- 代码更简单了。程序员很少需要考虑大小，而需要时，大小只出现在一个特定的地方（创建 Array_ref 处），而不是散落在代码各处。
- Circle[] 到 Shape[]、以及 Polygon[] 到 Shape[] 的转换中的类型问题被抓住了。
- s1 和 s2 元素个数错误的问题被隐式地处理掉了。
- max（以及其他指针的元素计数）的潜在问题变得更显眼——它是我们唯一必须明确写出大小的地方。
- 我们隐式而系统地处理了空指针和空 vector。

25.4.4 继承与容器

可是，如果我们想把一组 Circle 当作一组 Shape 来处理呢？也就是说，如果我们真的想让 better()（它是我们的老朋友 draw_all() 的一个变体；见 §19.3.2、§22.1.3）来处理多态呢？嗯，基本上做不到。在 §19.3.3 和 §25.4.2 里我们已经看到，类型系统有非常充分的理由拒绝把 vector<Circle> 当作 vector<Shape> 接受。出于同样的理由，它也拒绝把 Array_ref<Circle> 当作 Array_ref<Shape> 接受。如果你记不清原因，最好重读一遍 §19.3.3，因为这一点相当根本，尽管它会带来不便。

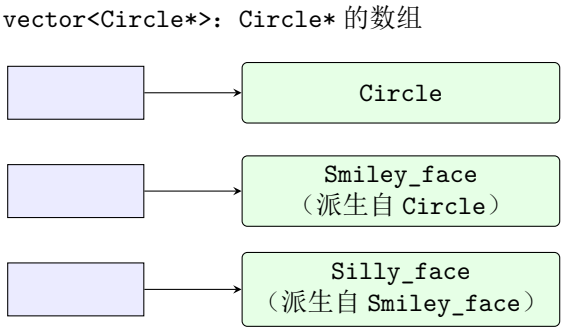
此外，为了保持运行时的多态行为，我们必须通过指针（或引用）来操作多态对象：better() 里 a[i].draw() 中的那个点号就是线索。一看到那个点号而不是箭头（->），我们就该预料到多态会有问题。

那我们能做些什么？首先，我们必须使用指针（或引用），而不是直接使用对象，所以我们会试着使用 Array_ref<Circle*>、Array_ref<Shape*> 等，而不是 Array_ref<Circle>、Array_ref<Shape> 等。

不过，我们仍然不能把 Array_ref<Circle*> 转换成 Array_ref<Shape*>，因为转换之后我们可能往那个 Array_ref<Shape*> 里放入不是 Circle* 的元素。但这里有一个漏洞：

- 这里我们并不想修改自己的 Array_ref<Shape*>，我们只想把那些 Shape 画出来！这是一个有趣而有用的特例：我们反对 Array_ref<Circle*> 到 Array_ref<Shape*> 转换的理由，并不适用于“不修改 Array_ref<Shape*>”的情形。
- 所有指针数组的布局都相同（与它们指向何种对象无关），所以我们不会陷入 §25.4.2 里的布局问题。

也就是说，把 Array_ref<Circle*> 当作不可变的 Array_ref<Shape*> 来处理，本身没有任何问题。所以，我们“只需”找到一种办法，把 Array_ref<Circle*> 当作不可变的 Array_ref<Shape*> 来对待。请看：



把那个 Circle* 数组当作不可变的 Shape* 数组（来自一个 Array_ref）来处理，在逻辑上没有任何问题。

我们似乎已经闯进了“专家领域”。事实上，这个问题确实很棘手，用目前介绍过的工具无法解决。不过，我们还是来看看：要做出一个近乎完美的替代品，来取代我们那种有缺陷——却又太受欢迎——的接口风格（指针加元素个数，见 §25.4.2），需要付出什么。请记住：不要仅仅为了证明自己有多聪明就闯进“专家领域”。大多数情况下，更好的策略是找一个库，让某位专家替你把设计、实现和测试都做完。

首先，我们把 better() 改写成使用指针的版本，并保证我们不会“乱动”作为实参的那个容器：

```
void better2(const Array_ref<Shape*const> a)
{
    for (int i = 0; i<a.size(); ++i)
        if (a[i])
            a[i]->draw();
}
```

现在我们处理的是指针，所以应当检查空指针。为了确保 better2() 不会通过 Array_ref 以不安全的方式修改我们的数组和 vector，我们加了两个 const。第一个 const 保证我们不会对自己的 Array_ref 施加修改性（变异）操作，例如 assign() 和 reset()。第二个 const 放在 * 之后，表示我们要的是“常量指针”（而不是“指向常量的指针”）；也就是说，即使我们有可用的操作，也不想修改那些元素指针。

接下来，我们必须解决那个核心问题：怎样表达这样一个想法——Array_ref<Circle*> 可以被转换成

- 某种类似 Array_ref<Shape*> 的东西（这样我们才能在 better2() 里使用它）

- 但只能转换成 Array_ref<Shape*> 的不可变版本

我们可以给 Array_ref 增加一个转换运算符来做到这一点：

```
template<typename T>
class Array_ref {
public:
    // 与前面相同
    template<typename Q>
    operator const Array_ref<const Q>()
    {
        // 检查元素的隐式转换:
        static_cast<Q>(*static_cast<T*>(nullptr));      // 检查元素转换
        return Array_ref<const Q>{reinterpret_cast<Q*>(p),sz};  // 转换 Array_ref
    }
    // 与前面相同
};
```

这东西让人头疼，但大致上是这样：

- 这个运算符会转换成 Array_ref<const Q>，对每个类型 Q 都成立，前提是我们能把 Array_ref<T> 的元素转换成 Array_ref<Q> 的元素（我们并不使用那次转换的结果，只是检查元素类型可以转换）。
- 我们用蛮力（reinterpret_cast）取得指向所需元素类型的指针，从而构造出新的 Array_ref<const Q>。蛮力的办法往往要付出代价；在这个例子里，代价就是：绝不要对使用了多重继承的类（§A.12.4）使用 Array_ref 转换。
- 请注意 Array_ref<const Q> 里的那个 const：正是它保证我们无法把 Array_ref<const Q> 拷贝进一个普通的、可变的 Array_ref<Q>。

我们确实警告过你，这是“专家领域”，而且“让人头疼”。不过，这个版本的 Array_ref 用起来很简单（棘手的只是定义/实现）：

```
void f(Shape* q, vector<Circle*>& s0)
{
    Polygon* s1[10];
    Shape* s2[20];
    // 初始化
    Shape* p1 = new Rectangle{Point{0,0},10};
    better2(make_ref(s0));          // 正确：转换为 Array_ref<Shape*const>
    better2(make_ref(s1));          // 正确：转换为 Array_ref<Shape*const>
    better2(make_ref(s2));          // 正确（无需转换）
    better2(make_ref(p1,1));        // 错误
    better2(make_ref(q,max));       // 错误
}
```

用指针的尝试都会出错，因为它们是 Shape*，而 better2() 要求的是 Array_ref<Shape*>；也就是说，better2() 需要的是“存放指针的对象”，而不是一个指针。如果想把指针传给 better2()，就得先把它们放进容器（例如内置数组或 vector）再传进去。对于单个指针，可以用那个别扭的写法

`make_ref(&p1,1)`。但对于（多于一个元素的）数组，除了创建一个存放对象指针的容器，没有别的办法。

总之，我们可以为数组的弱点造出简单、安全、易用而高效的接口。这正是本节的主要目标。有人提议把“每个问题都可以靠再加一层间接来解决”（语出 David Wheeler）当作“计算机科学第一定律”。我们解决这个接口问题，走的也正是这条路。

25.5 位、字节与字

前面我们谈过硬件层面的内存概念，比如位（bit）、字节（byte）和字（word），但在一般性的编程中，这些并不是我们常常惦记的东西。我们想的是特定类型的对象，例如 `double`、`string`、`Matrix` 和 `Simple_window`。这里，我们要看的是这样一个编程层次：在这个层次上，必须更清楚地意识到底层内存的实际情形。

如果你对自己关于整数二进制表示和十六进制表示的知识没有把握，现在也许是复习 §A.2.1.1 的好时机。

25.5.1 位与位运算

把字节看作 8 个位的序列：

7:	6:	5:	4:	3:	2:	1:	0:
1	0	1	0	0	1	1	1

注意字节中把位从右（最低有效位）向左（最高有效位）编号的约定。现在，把字看作 4 个字节的序列：

3:	2:	1:	0:
0xff	0x10	0xde	0xad

同样，我们从右向左编号，也就是从最低有效字节到最高有效字节。这些图把现实世界简化了：曾经有字节为 9 位的计算机（不过我们十来年没见过一台了），而字只有 2 个字节的机器也不少见。不过，只要你记得在利用“8 位”和“4 个字节”之前先查一查自己系统的手册，就不会有事。

在打算移植的代码里，要用 `<limits>`（§24.2.1）确认你关于大小的假设是正确的。可以把断言放进代码里，让编译器去检查：

```
static_assert(4<=sizeof(int),"ints are too small");
static_assert(!numeric_limits<char>::is_signed,"char is signed");
```

`static_assert` 的第一个实参是一个被假定为真的常量表达式。如果它不为真，也就是断言失败，编译器就会把第二个实参（一个字符串）写进错误信息。

在 C++ 里怎么表示一组位？答案取决于我们需要多少位，以及希望哪些操作既方便又高效。我们可以把整数类型当作位的集合来用：

- `bool` ——1 位，但占用整整一个字节的空间
- `char` ——8 位
- `short` ——16 位

- `int` ——通常是 32 位，但许多嵌入式系统是 16 位
- `long int` ——32 位或 64 位（但至少与 `int` 一样多）
- `long long int` ——32 位或 64 位（但至少与 `long` 一样多）

上面给出的大小是典型值，但不同实现可能不同，所以如果需要确知，就测一测。此外，标准库还提供了处理位的几种方式：

- `std::vector<bool>` ——需要多于 `8*sizeof(long)` 位时
- `std::bitset` ——需要多于 `8*sizeof(long)` 位时
- `std::set` ——具名位的无序集合（见 §21.6.5）
- 一个文件——大量的位（见 §25.5.6）

此外，还有两个语言特性可以用来表示位：

- 枚举（`enum`）；见 §9.5
- 位域（bitfield）；见 §25.5.5

表示“位”的方式如此之多，恰恰说明了一件事：计算机内存里的一切归根到底都是一组位，于是人们总忍不住要提供各式各样的办法来看待位、给位命名、对位做运算。注意，内置设施处理的是一组位数固定的位（例如 8、16、32 和 64），这样计算机就能用硬件直接提供的运算、以最优的速度对它们做逻辑运算。相比之下，标准库设施提供的位数是任意的。这可能限制性能，但不要预先对效率问题下判断：只要你选的位数与底层硬件匹配得好，库设施可以——而且常常确实——被优化得跑得很好。

我们先来看整数。对整数，C++ 提供的基本上就是硬件直接实现的那些按位逻辑运算。这些运算作用于操作数的每一位：

按位运算

`|`	或	若 x 的第 n 位或 y 的第 n 位为 1，则 `x|y` 的第 n 位为 1。
`&`	与	若 x 的第 n 位与 y 的第 n 位都为 1，则 `x&y` 的第 n 位为 1。
`^`	异或	若 x 与 y 的第 n 位中恰有一个为 1（而不是两个都为 1），则 `x^y` 的第 n 位为 1。
`<<`	左移	`x<<s` 的第 n 位是 x 的第 $n+s$ 位。
`>>`	右移	`x>>s` 的第 n 位是 x 的第 $n-s$ 位。
`~`	求补	`~x` 的第 n 位是 x 的第 n 位的反面。

你也许会觉得把“异或”（`^`，有时写作 xor）算作一种基本运算有点奇怪。不过，在许多图形和加密代码里，它正是最关键的运算。

编译器不会把按位逻辑 `<<` 误当成输出运算符，但你可能会搞混。为了避免混淆，请记住：输出运算符把 `ostream` 作为左操作数，而按位逻辑运算符把整数作为左操作数。

注意，`&` 与 `&&` 不同，`|` 与 `||` 也不同：按位运算符对操作数的每一位单独运算（§A.5.5），结果的位数与操作数一样多；而 `&&` 和 `||` 只返回 `true` 或 `false`。

我们来看几个例子。位模式通常用十六进制记法表示。对于半个字节（4 位），有

十六进制	位	十六进制	位
0x0	0000	0x8	1000
0x1	0001	0x9	1001
0x2	0010	0xa	1010
0x3	0011	0xb	1011
0x4	0100	0xc	1100
0x5	0101	0xd	1101
0x6	0110	0xe	1110
0x7	0111	0xf	1111

对于不超过 9 的数，我们本来可以用十进制，但用十六进制能帮我们记住：此刻想的是位模式。对字节和字，十六进制就真正有用了。一个字节中的各位可以用两位十六进制数表示。例如：

十六进制字节	位
0x00	0000 0000
0x0f	0000 1111
0xf0	1111 0000
0xff	1111 1111
0xaa	1010 1010
0x55	0101 0101

所以，为了让事情尽量简单，我们使用无符号数（§25.5.3），可以这样写：

```
unsigned char a = 0xaa;
unsigned char x0 = ~a;       // a 的按位取反
```

```
 a:   1  0  1  0  1  0  1  0   0xaa
~a:   0  1  0  1  0  1  0  1   0x55
```

```
unsigned char b = 0x0f;
unsigned char x1 = a&b;      // a 与 b
```

```
 a:   1  0  1  0  1  0  1  0   0xaa
 b:   0  0  0  0  1  1  1  1   0xf
a&b:  0  0  0  0  1  0  1  0   0xa
```

```
unsigned char x2 = a^b;      // 异或：a xor b
```

（译注：原书此行印作 `x4 == a>>2;`，按上下文应为赋值 `x4 = a>>2;`，属排印错误；代码依原书保留。）

```
  a:   1  0  1  0  1  0  1  0   0xaa
  b:   0  0  0  0  1  1  1  1   0xf
a^b:   1  0  1  0  0  1  0  1   0xa5
```

```
unsigned char x3 = a<<1;   // 左移 1
```

```
  a:   1  0  1  0  1  0  1  0   0xaa
a<<1:  0  1  0  1  0  1  0  0   0x54
```

注意，有一个 0 从第 0 位（最低有效位）之外“移进来”，填补这个字节。最左边的那一位（第 7 位）则干脆消失了。

```
unsigned char x4 == a>>2;     // 右移 2
```

```
  a:   1  0  1  0  1  0  1  0   0xaa
a>>2:  0  0  1  0  1  0  1  0   0x2a
```

注意，有两个 0 从第 7 位（最高有效位）之外“移进来”，填补这个字节。最右边的 2 位（第 1 位和第 0 位）则干脆消失了。

我们可以这样画出位模式，而且培养对位模式的感觉确实有好处，但这样画很快就让人厌烦了。下面这个小程序把整数转换成它们的位表示：

```
int main()
{
    for (int i; cin>>I; )
        cout << dec << i << "=="
             << hex << "0x" << i << "=="
             << bitset<8*sizeof(int)>{i} << '\n';
}
```

为了打印这个整数的各个位，我们用一个标准库 `bitset`：

```
bitset<8*sizeof(int)>{i}
```

一个 `bitset` 就是一串固定数目的位。这里我们用的位数是 `int` 中位的个数——`8*sizeof(int)`——并用我们的整数 `i` 来初始化这个 `bitset`。

试一试

试一试

让这个位的例子跑起来，再试几个值，以培养对二进制表示和十六进制表示的感觉。如果你被负数的表示搞糊涂了，先读 §25.5.3，然后再试一遍。

（译注：原书此行印作 `for (int i; cin>>I; )`，循环中使用的变量应为小写 `i`，`I` 疑为排印之误；代码依原书保留。）

25.5.2 bitset

标准库模板类 bitset（来自 <bitset>）用来表示和操作位的集合。每个 bitset 大小固定，在构造时指定：

```
bitset<4> flags;
bitset<128> dword_bits;
bitset<12345> lots;
```

一个 bitset 默认被初始化为“全零”，但通常都会给它一个初始值；bitset 的初始值可以是一个无符号整数，也可以是由 0 和 1 组成的字符串。例如：

```
bitset<4> flags = 0xb;
bitset<128> dword_bits {string{"1010101010101010"}};
bitset<12345> lots;
```

这里 lots 将全为零，而 dword_bits 前面是 112 个 0，后面是我们显式指定的那 16 位。如果你用一个含有 '0' 和 '1' 之外的字符的字符串来初始化，就会抛出 std::invalid_argument 异常：

```
string s;
cin>>s;
bitset<12345> my_bits{s};  // 可能抛出 std::invalid_argument
```

我们可以对 bitset 使用常用的位操作运算符。假定 b1、b2 和 b3 都是 bitset：

```
b1 = b2&b3;      // 与
b1 = b2|b3;      // 或
b1 = b2^b3;      // 异或
b1 = ~b2;        // 求补
b1 = b2<<2;      // 左移
b1 = b2>>3;      // 右移
```

基本上，对于位运算（按位逻辑运算），bitset 的行为就像一个位数任意、由用户指定大小的 unsigned int（§25.5.3）。你能对 unsigned int 做的（算术运算除外），都能对 bitset 做。特别是，bitset 对 I/O 很有用：

```
cin>>b;                                  // 从输入读取一个 bitset
cout<<bitset<8>{'c'};                    // 输出字符 'c' 的位模式
```

向 bitset 读入时，输入流会去找 0 和 1。请看：

```
10121
```

它被读成 101，剩下的 21 留在流里没有读。

与字节和字一样，bitset 的位也是从右向左编号的（从最低有效位到最高有效位），因此，例如第 7 位的数值是 2^7：

对 bitset 来说，编号不只是一个约定，因为 bitset 支持对位做下标运算。例如：

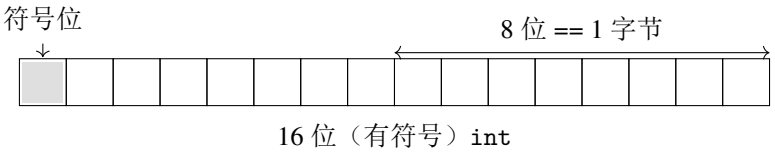
```
7: 6: 5: 4: 3: 2: 1: 0:
1  0  1  0  0  1  1  1
```

```
int main()
{
    constexpr int max = 10;
    for (bitset<max> b; cin>>b; ) {
        cout << b << '\n';
        for (int i =0; i<max; ++i) cout << b[i];  // 逆序
        cout << '\n';
    }
}
```

如果你需要关于 bitset 更完整的图景，就去查在线文档、手册，或者专家级的教科书。

25.5.3 有符号与无符号

像大多数语言一样，C++ 同时支持有符号整数和无符号整数。无符号整数在内存中的表示很直接：bit0 表示 1，bit1 表示 2，bit2 表示 4，以此类推。但有符号整数就带来一个问题：怎么区分正数和负数？C++ 给硬件设计者留了一些自由，但几乎所有实现都使用二进制补码（two's complement）表示。最左边（最高有效位）被当作“符号位”：



如果符号位是 1，这个数就是负数。几乎普遍采用二进制补码表示。为了节省篇幅，我们看看在 4 位整数里会怎样表示有符号数：

正数：	0	1	2	4	7
	0000	0001	0010	0100	0111
负数：	1111	1110	1101	1011	1000
	−1	−2	−3	−5	−8

对 $-(x+1)$，它的位模式可以这样描述：把 x 的各位取补（也就是 ~x；见 §25.5.1）。

到目前为止，我们一直只使用有符号整数（例如 int）。一套稍好些的规则是：

- 数值使用有符号整数（例如 int）。
- 位的集合使用无符号整数（例如 unsigned int）。

这条经验法则不算坏，但很难坚持，因为有些人偏爱用无符号整数做某些形式的算术，而我们有时不得不使用他们的代码。特别是，由于一些可以追溯到 C 语言早期（那时 int 是 16 位、每一位都很金贵）的历史原因，vector 的 v.size() 返回的是无符号整数。例如：

```
vector<int> v;
// . . .
for (int i = 0; i<v.size(); ++i) cout << v[i] << '\n';
```

一个“好心”的编译器可能会提醒我们：我们在混用有符号值（即 i）和无符号值（即 v.size()）。混用有符号变量和无符号变量可能带来灾难。例如，循环变量 i 可能溢出；也就是说，v.size() 可能比有符号 int 能表示的最大值还大。于是 i 会先到达有符号 int 中表示正整数的最大值（即 2 的“int 的位数减 1”次方再减 1，例如 $2^{15}-1$）。接下来的一次 ++ 无法给出下一个更大的整数，反而会得到一个负值。这个循环就永远不会结束！每次到达最大的整数，我们就会从最小的负 int 值重新开始。所以，对 16 位的 int 来说，如果 v.size() 是 32×1024 或更大，这个循环就是一个（很可能非常严重的）错误；对 32 位的 int 来说，问题会在 i 达到 $2\times1024\times1024\times1024$ 时出现。

所以，从技术上说，本书中的大多数循环都写得马虎，都可能引起问题。换句话说，对于嵌入式系统，我们本该要么验证过循环永远不会到达那个临界点，要么把它换成另一种形式的循环。为了避免这个问题，我们可以使用 vector 提供的 size_type、迭代器，或者范围 for 语句：

```
for (vector<int>::size_type i = 0; i<v.size(); ++i) cout << v[i] << '\n';

for (vector<int>::iterator p = v.begin(); p!=v.end(); ++p) cout << *p << '\n';

for (int x : v) cout << x << '\n';
```

size_type 保证是无符号的，所以第一种（无符号整数）写法比上面的 int 版多一位可用。这可能很重要，但它仍然只多出一位的范围（也就是能把可做的迭代次数翻倍）。使用迭代器的循环就没有这种限制。

试一试

试一试

下面这个例子看起来无害，其实是一个死循环：

```
void infinite()
{
    unsigned char max = 160;        // 非常大
    for (signed char i=0; i<max; ++i) cout << int(i) << '\n';
}
```

运行它，并解释为什么。

基本上，把无符号整数当作整数来用（而不是只把它们当作位的集合，即不用 +、−、*、/）有两个理由：

- 为了取得那额外一位的精度
- 为了表达“这个整数不可能是负数”这一逻辑性质

前者正是程序员使用无符号循环变量所得到的好处。

同时使用有符号类型和无符号类型的问题在于：在 C++ 中（和 C 中一样），它们相互转换的方式既出人意料，又很难记住。请看：

```
unsigned int ui = -1;

int si = ui;
int si2 = ui+2;
unsigned ui2 = ui+2;
```

令人意外的是，第一个初始化居然成功，ui 得到值 4294967295——也就是与有符号整数 -1（“全 1”）具有相同表示（位模式）的无符号 32 位整数。有些人觉得这很巧妙，把 -1 当作“全 1”的简写；另一些人则觉得这是个问题。同样的转换规则也适用于从无符号到有符号，所以 si 得到值 -1。如我们所料，si2 变成 1（-1+2 == 1），ui2 也一样。ui2 的结果应该会让你愣一下：4294967295+2 怎么会是 1？把 4294967295 看成十六进制数（0xffffffff）就清楚了：4294967295 是最大的无符号 32 位整数，所以 4294967297 无法用 32 位整数表示——不管有符号还是无符号。因此我们要么说 4294967295+2 溢出了，要么（更准确地说）说无符号整数支持模算术；也就是说，32 位整数上的算术是模 2^{32} 的算术。

到这里为止，前面讲的都清楚吗？即使清楚了，我们也希望已经让你相信：在无符号整数上玩弄那一点额外的精度，是在玩火。它容易让人糊涂，因此是潜在的错误来源。

如果整数溢出会发生什么？考虑下面这段程序：

```
Int i = 0;
while (++i) print(i);     // 把 i 作为整数打印出来，后面跟一个空格
```

它会打印出什么样的值序列？显然，这取决于 Int 的定义（这一次大写的 I 并不是笔误）。对于位数有限的整数类型，迟早会溢出。如果 Int 是无符号的（例如 unsigned char、unsigned int 或 unsigned long long），那么 ++ 就是模运算，因此表示完能表示的最大数之后就回到 0（循环随即结束）。如果 Int 是有符号整数（例如 signed char），数值会突然变成负数，然后慢慢回升到 0（循环在那里结束）。例如对 signed char，我们会看到 1 2 ... 126 127 −128 −127 ... −2 −1。

整数溢出时到底发生什么？答案是：我们就像位数足够那样继续算，但把结果中装不进目标整数的那部分丢弃。这种策略会丢掉最左边（最高有效）的若干位。我们执行赋值时看到的也是同样的效果：

```
int si = 257;             // 放不进 char
char c = si;              // 隐式转换成 char
unsigned char uc = si;
signed char sc = si;
print(si); print(c); print(uc); print(sc); cout << '\n';

si = 129;                 // 放不进 signed char
c = si;
uc = si;
sc = si;
print(si); print(c); print(uc); print(sc);
```

我们得到

```
257     1     1     1
129     -127     129     -127
```

对这个结果的解释是：257 比能装进 8 位的数多 2（255 是“8 个 1”），而 129 比能装进 7 位的数多 2（127 是“7 个 1”），于是符号位被置上了。顺便说一句：这个程序说明我们机器上的 char 是有符号的（c 的行为与 sc 相同，而与 uc 不同）。

试一试

试一试

在纸上把这些位模式画出来。先用纸笔推算，当 si=128 时答案会是什么；然后运行程序，看看你的机器是否同意。

顺便说一句：我们为什么要引入那个 print() 函数？我们本来可以写

```
cout << i << ' ';
```

然而，如果 i 是 char，这样输出的就是字符，而不是它的整数值。因此，为了统一处理所有整数类型，我们定义了：

```
template<typename T> void print(T i) { cout << i << '\t'; }
void print(char i) { cout << int(i) << '\t'; }
void print(signed char i) { cout << int(i) << '\t'; }
void print(unsigned char i) { cout << int(i) << '\t'; }
```

总之：你可以像使用有符号整数那样使用无符号整数（包括普通的算术运算），但只要可以，就避免这么做，因为它很微妙，而且容易出错。

- 尽量不要只是为了多得到一位精度就去用无符号。
- 如果你需要多一位，很快你就需要再多一位。

遗憾的是，你没法完全避开无符号算术：

- 标准库容器的下标使用的是无符号类型。
- 有些人就是喜欢无符号算术。

25.5.4 位操作

我们为什么要摆弄位？嗯，我们大多数人其实并不想。“摆弄位”（bit fiddling）层次低又容易出错，所以只要有别的办法，我们就会用别的办法。然而位既基础又非常有用，因此我们很多人没法假装它们不存在。这话听起来有点消极、有点让人泄气，但这是有意的。有些人确实很喜欢摆弄位和字节，所以值得记住：摆弄位是不得已时才做的事（过程中很可能还挺好玩），但位不该充斥在你的代码里。

引用 John Bentley 的话：“玩位的人会被咬”（People who play with bits will be bitten），“玩字节的人会被 bytten”（People who play with bytes will be bytten）。

那么，什么时候需要摆弄位？有时我们应用中的自然对象本身就是位，于是应用领域里的一些自然操作就是位操作。这类领域的例子包括硬件指示器（“标志”）、底层通信（必须从字节流中取出各种类型的值）、图形（必须由多层图像合成画面）以及加密（见下一节）。

例如，考虑怎样从一个整数中取出（底层的）信息（也许是因为我们想按二进制 I/O 的方式把它当字节发送出去）：

```
void f(short val)                       // 假定 short 是 16 位、2 字节
{
    unsigned char right = val&0xff;     // 最右边（最低有效）字节
    unsigned char left = val>>8;        // 最左边（最高有效）字节
    // . . .
    bool negative = val&0x8000;         // 符号位
    // . . .
}
```

这类操作很常见，称为“移位与掩码”（shift and mask）。我们用 << 或 >> 来“移位”，把想处理的位挪到字的最右端（最低有效端），在那里容易操作。我们用按位与（&）配合一个位模式（这里是 0xff）来“掩码”，把结果中不想要的位消掉（置零）。

当我们想给位起名字时，常常使用枚举。例如：

```
enum Printer_flags {
    acknowledge=1,
    paper_empty=1<<1,
    busy=1<<2,
    out_of_black=1<<3,
    out_of_color=1<<4,
    // . . .
};
```

这就让每个枚举项的值正好等于它的名字所表示的值：

out_of_color	16	0x10	0001 0000
out_of_black	8	0x8	0000 1000
busy	4	0x4	0000 0100
paper_empty	2	0x2	0000 0010
acknowledge	1	0x1	0000 0001

这样的值很有用，因为它们可以彼此独立地组合起来：

```
unsigned char x = out_of_color | out_of_black;   // x 变成 24 (16+8)
x |= paper_empty;                               // x 变成 26 (24+2)
```

注意 | 与等号连用可以读作“置位”（或“置若干位”）。类似地，& 可以读作“这一位置位了吗?”。例如：

```
if (x & out_of_color) {          // out_of_color 置位了吗？(是的，置位了)
    // . . .
}
```

我们仍然可以用 & 来做掩码：

```
unsigned char y = x & (out_of_color | out_of_black);   // y 变成 24
```

现在 y 里有一份从 x 的第 4 位和第 3 位（out_of_color 和 out_of_black）复制过来的位。

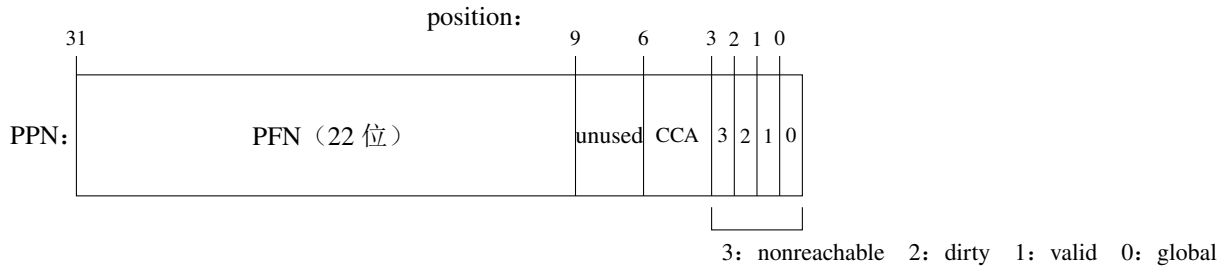
把枚举当作一组位来用是非常常见的做法。这样做时，我们需要一次转换，才能把按位逻辑运算的结果“放回”枚举里。例如：

```
Flags z = Printer_flags(out_of_color | out_of_black);  // 这个显式转换是必需的
```

之所以需要这次显式转换，是因为编译器无法知道 out_of_color | out_of_black 的结果是不是 Flags 变量的合法值。编译器的怀疑是有道理的：毕竟没有任何枚举项的值是 24（out_of_color | out_of_black）；但在这种情况下，我们知道这个赋值是合理的（而编译器不知道）。

25.5.5　位域

前面提到过，硬件接口是位频繁出现的一个领域。通常，一个接口被定义成一批大小不一的位和数的混合体。这些“位和数”通常都有名字，并且位于字的特定位置上，这个字常称为设备寄存器（device register）。C++ 有一种专门的语言设施来处理这种固定布局：位域（bitfield）。考虑操作系统深处的页面管理器所使用的页号。下面是一张来自操作系统手册的示意图：



这个 32 位的字被用作两个数值字段（一个是 22 位，一个是 3 位）和四个标志（各 1 位）。这几个数据块的大小和位置都是固定的。中间甚至还有一个未使用（也没有名字）的“字段”。我们可以把它表示成一个结构体：

```
struct PPN {                        // R6000 物理页号
    unsigned int PFN : 22;          // 页框号 (Page Frame Number)
    int : 3;                        // 未使用
    unsigned int CCA : 3;           // 缓存一致性算法
    bool nonreachable : 1;
    bool dirty : 1;
    bool valid : 1;
    bool global : 1;
};
```

我们不得不查手册才知道 PFN 和 CCA 应当按无符号整数解释；除此之外，几乎可以直接照着这张图写出那个结构体。位域从左到右填充一个字。位数写在冒号后面，用整数值给出。你不能指定绝对位置（例如第 8 位）。如果位域“消耗”的位比一个字能容纳的还多，放不下的字段就进入下一个字。但愿这正是你想要的结果。定义好之后，位域用起来和其他变量完全一样：

```
void part_of_VM_system(PPN* p)
{
    // . . .
    if (p->dirty) {          // 内容有改动
        // 复制到磁盘
        p->dirty = 0;
    }
    // . . .
}
```

位域主要省去了移位和掩码的麻烦，不必再为了取到字中间的信息而折腾。例如，给定一个名为 pn 的 PPN，我们可以这样取出 CCA：

```
unsigned int x = pn.CCA;          // 取出 CCA
```

如果我们用一个名为 pni 的 int 表示同样的位，那也可以写成

```
unsigned int y = (pni>>4)&0x7;   // 取出 CCA
```

也就是说，先把 pn 右移，让 CCA 落到最低的那三位上，再用 0x7（即最后三位全为 1）把其他位掩掉。如果你去看机器码，很可能会发现这两行生成的代码完全相同。

这种“缩写汤”（CCA、PPN、PFN）在这一层的代码里很典型，离开上下文就没什么意义。

25.5.6 一个例子：简单加密

作为在数据的位与字节表示这一层上操作数据的例子，我们来看一个简单的加密算法：微型加密算法（Tiny Encryption Algorithm，TEA）。它最初由剑桥大学的 David Wheeler 编写（§22.2.1）。它很小，但抵御非期望解密的保护非常出色。

不要盯着代码看太久（除非你真的想看，并且愿意冒头痛的风险）。我们给出这段代码，只是让你感受一下真实世界里有用的位操作代码是什么味道。如果你想专门研究加密，那需要另一本教科书。关于这个算法的更多资料和其他语言中的变体，见 http://en.wikipedia.org/wiki/Tiny_Encryption_Algorithm 以及英格兰布拉德福德大学 Simon Shepherd 教授的 TEA 网站。这段代码并不打算自我解释（里面没有注释！）。

加密与解密（enciphering/deciphering，也叫 encryption/decryption）的基本想法很简单。我想把一段文本发给你，但不想让别人读到。因此我按某种方式变换这段文本，使不知道我究竟改了什么的人读不懂它，同时又使你能逆转我的变换，读出原文。这就叫加密。加密要用一个算法（必须假定有个不请自来的窃听者也知道它）和一个称为“密钥”的字符串。你和我都有密钥（我们希望那个窃听者没有）。你收到密文后，用密钥解密，也就是把我发出的“明文”还原出来。

TEA 以一个含两个 unsigned long 的数组（v[0]、v[1]）为参数，其中存放要加密的 8 个字符；以另一个含两个 unsigned long 的数组（w[0]、w[1]）接收加密后的输出；还有一个含四个 unsigned long 的数组（k[0] 到 k[3]），那是密钥：

```
void encipher(
    const unsigned long *const v,
    unsigned long *const w,
    const unsigned long *const k)
{
    static_assert(sizeof(long)==4,"size of long wrong for TEA");
    unsigned long y = v[0];
    unsigned long z = v[1];
    unsigned long sum = 0;
    const unsigned long delta = 0x9E3779B9;
    for (unsigned long n = 32; n-->0; ) {
        y += (z<<4 ^ z>>5) + z^sum + k[sum&3];
        sum += delta;
        z += (y<<4 ^ y>>5) + y^sum + k[sum>>11 & 3];
    }
    w[0]=y;
    w[1]=z;
}
```

注意，所有数据都是无符号的，这样我们就能对它们做按位运算，而不用担心与负数相关的特殊处理带来意外。移位（<< 和 >>）、异或（^）以及按位与（&）承担了主要工作，另外还顺带用了普通的（无符号）加法。这段代码是专门为 long 占 4 个字节的机器写的。代码里到处是“魔法”常量（例如它假定 sizeof(long) 是 4）。这通常不是个好主意，但这一个软件正好能写在一张纸上。写成数学公式，它装得进一个信封的背面，或者像最初设想的那样，装进一个记性好的程序员的脑袋里。David Wheeler 希望旅行时不用带笔记、笔记本电脑之类的东西也能加密。除了小，这段代码还很快。变量 n 决定迭代次数：迭代次数越多，加密越强。据我们所知，n==32 的 TEA 从未被攻破。

下面是相应的解密函数：

```
void decipher(
    const unsigned long *const v,
    unsigned long *const w,
    const unsigned long *const k)
{
    static_assert(sizeof(long)==4,"size of long wrong for TEA");
    unsigned long y = v[0];
    unsigned long z = v[1];
    unsigned long sum = 0xC6EF3720;
    const unsigned long delta = 0x9E3779B9;

    // sum = delta<<5; 一般地 sum = delta * n
    for (unsigned long n = 32; n-- > 0; ) {
        z -= (y << 4 ^ y >> 5) + y ^ sum + k[sum>>11 & 3];
        sum -= delta;
        y -= (z << 4 ^ z >> 5) + z ^ sum + k[sum&3];
    }
```

```
    w[0]=y;
    w[1]=z;
}
```

我们可以这样使用 TEA，生成一个要通过不安全的连接发送出去的文件：

```
int main()                          // 发送方
{
    const int nchar = 2*sizeof(long);     // 64 位
    const int kchar = 2*nchar;            // 128 位
    string op;
    string key;
    string infile;
    string outfile;
    cout << "please enter input file name, output file name, and key:\n";
    cin >> infile >> outfile >> key;
    while (key.size()<kchar) key += '0';    // 补齐密钥
    ifstream inf(infile);
    ofstream outf(outfile);
    if (!inf || !outf) error("bad file name");
    const unsigned long* k =
            reinterpret_cast<const unsigned long*>(key.data());
    unsigned long outptr[2];
    char inbuf[nchar];
    unsigned long* inptr = reinterpret_cast<unsigned long*>(inbuf);
    int count = 0;
    while (inf.get(inbuf[count])) {
        outf << hex;                        // 使用十六进制输出
        if (++count == nchar) {
            encipher(inptr,outptr,k);
            // 用前导零补齐:
            outf << setw(8) << setfill('0') << outptr[0] << ' '
                 << setw(8) << setfill('0') << outptr[1] << ' ';
            count = 0;
        }
    }
    if (count) {    // 补齐
        while(count != nchar) inbuf[count++] = '0';
        encipher(inptr,outptr,k);
        outf << outptr[0] << ' ' << outptr[1] << ' ';
    }
}
```

核心的一小段代码是那个 `while` 循环，其余都只是辅助。`while` 循环把字符读进输入缓冲区 `inbuf`，每当攒够 TEA 所需的 8 个字符，就把它们交给 `encipher()`。TEA 并不关心字符；事实上它根本不知道自己加密的是什么。例如，你也可以加密一张照片或一段电话通话。TEA 只关心你给它 64 位（两

个 `unsigned long`），好让它产生对应的 64 位。所以，我们取 `inbuf` 的指针，把它转换成 `unsigned long*`，再传给 TEA。对密钥我们也照做；TEA 会使用密钥的前 128 位（四个 `unsigned long`），所以我们把用户的输入“补齐”，确保有 128 位。最后一条语句用零填充文本，凑成 TEA 所需的 64 位（8 字节）的整数倍。

密文怎么发送？我们可以自由选择；但由于它“只是位”，而不是 ASCII 或 Unicode 字符，我们没法真的把它当作普通文本处理。二进制 I/O（见 §11.3.2）是一种选择，不过这里我们决定把输出的字按十六进制数打印出来：

```
5b8fb57c 806fbcce 2db72335 23989d1d 991206bc 0363a308
8f8111ac 38f3f2f3 9110a4bb c5e1389f 64d7efe8 ba133559
4cc00fa0 6f77e537 bde7925f f87045f0 472bad6e dd228bc3
a5686903 51cc9a61 fc19144e d3bcde62 4fdb7dc8 43d565e5
f1d3f026 b2887412 97580690 d2ea4f8b 2d8fb3b7 936cfa6d
6a13ef90 fd036721 b80035e1 7467d8d8 d32bb67e 29923fde
197d4cd6 76874951 418e8a43 e9644c2a eb10e848 ba67dcd8
7115211f dbe32069 e4e92f87 8bf3e33e b18f942c c965b87a
44489114 18d4f2bc 256da1bf c57b1788 9113c372 12662c23
eeb63c45 82499657 a8265f44 7c866aae 7c80a631 e91475e1
5991ab8b 6aedbb73 71b642c4 8d78f68b d602bfe4 d1eadde7
55f20835 1a6d3a4b 202c36b8 66a1e0f2 771993f3 11d1d0ab
74a8cfd4 4ce54f5a e5fda09d acbdf110 259a1a19 b964a3a9
456fd8a3 1e78591b 07c8f5a2 101641ec d0c9d7e1 60dbeb11
b9ad8e72 ad30b839 201fc553 a34a79c4 217ca84d 30f666c6
d018e61c d1c94ea6 6ca73314 cd60def1 6e16870e 45b94dc0
d7b44fcd 96e0425a 72839f71 d5b6427c 214340f9 8745882f
0602c1a2 b437c759 ca0e3903 bd4d8460 edd0551e 31d34dd3
c3f943ed d2cae477 4d9d0b61 f647c377 0d9d303a ce1de974
f9449784 df460350 5d42b06c d4dedb54 17811b5f 4f723692
14d67edb 11da5447 67bc059a 4600f047 63e439e3 2e9d15f7
4f21bbbe 3d7c5e9b 433564f5 c3ff2597 3a1ea1df 305e2713
9421d209 2b52384f f78fbae7 d03c1f58 6832680a 207609f3
9f2c5a59 ee31f147 2ebc3651 e017d9d6 d6d60ce2 2be1f2f9
eb9de5a8 95657e30 cad37fda 7bce06f4 457daf44 eb257206
418c24a5 de687477 5c1b3155 f744fbff 26800820 92224e9d
43c03a51 d168f2d1 624c54fe 73c99473 1bce8fbb 62452495
5de382c1 1a789445 aa00178a 3e583446 dcbd64c5 ddda1e73
fa168da2 60bc109e 7102ce40 9fed3a0b 44245e5d f612ed4c
b5c161f8 97ff2fc0 1dbf5674 45965600 b04c0afa b537a770
9ab9bee7 1624516c 0d3e556b 6de6eda7 d159b10e 71d5c1a6
b8bb87de 316a0fc9 62c01a3d 0a24a51f 86365842 52dabf4d
372ac18b 9a5df281 35c9f8d7 07c8f9b4 36b6d9a5 a08ae934
239efba5 5fe3fa6f 659df805 faf4c378 4c2048d6 e8bf4939
31167a93 43d17818 998ba244 55dba8ee 799e07e7 43d26aef
d5682864 05e641dc b5948ec8 03457e3f 80c934fe cc5ad4f9
0dc16bb2 a50aa1ef d62ef1cd f8fbbf67 30c17f12 718f4d9a
43295fed 561de2a0
```

试一试

试一试

密钥是 bs；那么明文是什么？

任何安全专家都会告诉你，把明文和密文文件存在一起是个愚蠢的主意，还会对填充、对使用只有两个字母的密钥之类的事发表意见；但这是一本编程书，而不是一本计算机安全书。

我们测试这些程序的办法是：读入密文，再把原文取回来。写程序时，能做一个简单的正确性测试总是件好事。

下面是解密程序的核心部分：

```
unsigned long inptr[2];
char outbuf[nchar+1];
outbuf[nchar]=0;   // 结尾符
unsigned long* outptr = reinterpret_cast<unsigned long*>(outbuf);
inf.setf(ios_base::hex ,ios_base::basefield);    // 使用十六进制输入

while (inf>>inptr[0]>>inptr[1]) {
    decipher(inptr,outptr,k);
    outf<<outbuf;
}
```

注意这里对 `inf.setf(ios_base::hex, ios_base::basefield)` 的使用，它用于读入十六进制数。解密时，我们把输出缓冲区 `outbuf` 当作位来处理，这是借助强制类型转换（cast）做到的。

TEA 算不算嵌入式系统编程的例子？它并非专门针对嵌入式系统，但你可以想象，凡是需要保密或进行金融交易的地方都可能用到它——那会包括许多“小设备”。不管怎样，TEA 展示了好的嵌入式系统代码的许多特征：它建立在一个我们充分理解（数学上）的模型之上，这让我们对它的正确性有把握；它很小、很快，而且直接依赖硬件特性。`encipher()` 和 `decipher()` 的接口风格不太合我们的口味。不过 `encipher()` 和 `decipher()` 当初设计成既是 C 函数又是 C++ 函数，因此凡是 C 不支持的 C++ 设施都不能用。此外，那些“魔数常量”直接来自对数学公式的手工翻译。

25.6 编码标准

错误的来源有很多。最严重、也最难纠正的错误，与高层设计决策有关，比如总体的错误处理策略、对某些标准的符合程度（或缺乏符合）、算法、数据的表示方式等。这些不是我们在这里要处理的问题。我们关注的是由写得很差的代码引起的错误，也就是那些以不必要的易错方式使用编程语言设施、或者以掩盖其含义的方式表达想法的代码。

编码标准试图通过定义一种“内部风格”（house style）来解决后一类问题，这种风格引导程序员使用 C++ 语言中某个被认为适合特定应用的子集。例如，一个有硬实时约束的嵌入式系统、或者一个需要“永远”运行的系统，其编码标准可能禁止使用 `new`。通常，编码标准还力求保证两名程序员写出的代码比他们各自自由选用所有可能风格时更为相似。例如，编码标准可能要求循环一律使用 `for` 语句（从而禁用 `while` 语句）。这能让代码更统一，而在大型项目中，这对维护可能很重要。请注意，编码标准的目标是针对特定种类的程序员、为特定种类的编程改进代码。不存在适合所有 C++ 应用和所有 C++ 程序员的唯一编码标准。

因此，编码标准试图解决的问题，是源于我们表达解决方案的方式的问题，而不是源于我们试图用应用解决的问题本身固有复杂性的问题。可以说，编码标准试图处理的是附带复杂性（incidental complexity），而不是固有复杂性（inherent complexity）。

这类附带复杂性的主要来源有：

- 过分聪明的程序员：他们使用自己并未真正理解的特性，或以复杂难懂的解法为乐；
- 受教育不足的程序员：他们不使用最合适的语言与库特性；

- 编程风格上不必要的差异：使完成类似任务的代码看起来不同，令维护者困惑；
- 不合适的编程语言：导致使用那些不适合特定应用领域或特定程序员群体的语言特性；
- 库的使用不足：导致对低层资源大量临时拼凑式的操作；
- 不合适的编码标准：造成额外工作，或禁绝某类问题的最佳解法，于是自身反倒成了这些标准本意要解决的问题之源。

25.6.1 好的编码标准应该是什么样的？

好的编码标准应当帮助程序员写出好的代码；也就是说，它应当通过回答大量细小问题来帮助程序员，而这些问题原本需要每位程序员逐个、临时地花时间去决定。有一句古老的工程师谚语说："形式带来自由。"理想情况下，编码标准应当是规定性的（prescriptive），明确指出应该做什么。这看起来显而易见，但许多编码标准只是一串禁令，在遵守了一长串"不要"之后，却对接下来该做什么毫无指引。仅仅被告知不要做什么，很少有帮助，往往还令人恼火。

好的编码标准中的规则应当是可验证的，最好能由程序来验证；也就是说，代码写完之后，我们应当能够看着它轻松回答："我有没有违反自己编码标准中的任何一条规则？"

好的编码标准应当为规则给出理由。不能只告诉程序员"我们这儿就是这么做的！"。一旦这样做，他们就会反感。更糟的是，程序员总会设法绕过编码标准中被他们认为毫无意义、妨碍自己把活干好的部分。不要指望你会喜欢编码标准的每一点。即使最好的编码标准也是一种折中，而且多数标准都会禁止某些被认为会引发问题的做法——哪怕这些做法从未给你带来过麻烦。例如，不一致的命名规则是混乱之源，但不同的人对某些命名约定有强烈的偏爱，对另一些则强烈反感。比如，我认为标识符的 `CamelCodingStyle` 风格"丑得离谱"，强烈偏好更简洁、天生更易读的 `underscore_style`，很多人也同意。另一方面，很多讲道理的人并不同意。显然，没有任何命名标准能让所有人满意；但在这个问题上，如同在许多其他问题上一样，一致的风格肯定好过没有标准。

总结如下：

- 好的编码标准是为特定应用领域和特定程序员群体设计的。
- 好的编码标准既是规定性的，也是限制性的。
- 推荐一些"基础"库设施，往往是指定性规则最有效的用法。
- 编码标准是一组关于代码应当长什么样的规则：
  - 通常规定命名与缩进规则，例如"采用 Stroustrup 排版"。
  - 通常规定语言的某个子集，例如"不要使用 `new` 或 `throw`"。
  - 通常规定注释规则，例如"每个函数都必须有注释说明它做什么"。
  - 常常要求使用某些库，例如"使用 `<iostream>` 而不是 `<stdio.h>`"，或者"使用 `vector` 和 `string` 而不是内置数组和 C 风格字符串"。
- 多数编码标准的共同目标是改进：
  - 可靠性
  - 可移植性
  - 可维护性
  - 可测试性
  - 可复用性
  - 可扩展性

  - 可读性
- 好的编码标准胜过没有标准。我们不会在没有编码标准的情况下启动一个大型（多人、多年）工业项目。
- 糟糕的编码标准可能比没有标准更糟。例如，把编程限制在类似 C 子集范围内的 C++ 编码标准是有害的。不幸的是，糟糕的编码标准并不罕见。
- 所有编码标准都不受程序员欢迎，即使是好的标准。多数程序员希望完全按自己喜欢的方式写代码。

25.6.2 规则示例

下面我们列出一些规则，让你体会一下编码标准是什么样子。当然，我们挑选的是希望对你也有用的规则。不过，我们从未见过哪份真实世界的编码标准能在 35 页以内讲完，多数要长得多。因此，这里并不试图给你一套完整的规则。而且，每一份好的编码标准都是为特定应用领域和特定程序员群体设计的。所以，我们并不假装这些规则具有普适性。

这些规则都编号，并附有（简短的）理由。许多规则带有例子，以便理解。我们区分两类规则：建议（recommendation），程序员偶尔可以决定不予理会；硬性规则（firm rule），必须遵守。在真实的一套规则中，硬性规则通常（只有）凭主管的书面许可才能违反。每违反一条建议或硬性规则，都要求在代码中加一条注释。规则的任何例外都可以列在该规则之中。硬性规则在编号中用大写字母 R 标识，建议用编号中的小写字母 r 标识。

这些规则分为以下几类：

- 通用
- 预处理
- 命名与排版
- 类规则
- 函数与表达式规则
- 硬实时
- 关键系统

“硬实时”和“关键系统”两类规则只适用于被划入相应类别的项目。

与真实世界的好编码标准相比，我们的术语定义不够明确（例如，“关键”到底指什么？），规则也过于简短。这些规则与 JSF++ 规则（见 25.6.3 节）之间的相似并非偶然——我曾参与制定 JSF++ 规则。不过，本书中的代码示例并不符合下面的规则——毕竟，本书的代码不是关键嵌入式系统代码。

通用规则

R100：任何单个函数或类所含的逻辑源代码行（不含注释）不得超过 200 行。

理由：长函数和长类往往很复杂，因而难以理解和测试。

r101：任何单个函数或类都应能显示在一屏之内，并只承担单一的、合乎逻辑的用途。

理由：只看到一个函数或类的一部分的程序员，更容易忽略某个问题。试图完成多项逻辑功能的函数，往往比只做一件事的函数更长、更复杂。

R102：所有代码都应符合 ISO/IEC 14882:2011(E) 标准 C++。

理由：语言扩展或偏离 ISO/IEC 14882 的写法往往更不稳定、规定得不够明确，并且会限制可移植性。

预处理规则

R200：除使用 #ifdef 和 #ifndef 进行源码控制外，不得使用任何宏。

理由：宏不遵守作用域和类型规则。在肉眼检查源代码文本时，宏的使用并不明显。

R201：#include 只能用于包含头文件（*.h）。

理由：#include 用于访问接口声明，而不是实现细节。

R202：所有 #include 指令都应位于所有非预处理声明之前。

理由：出现在文件中间的 #include 更容易被读者忽略，也会因同一名字在不同位置被解析成不同含义而造成不一致。

R203：头文件（*.h）不得包含非 const 变量的定义，也不得包含非 inline、非模板函数的定义。

理由：头文件应当包含接口声明，而不是实现细节。不过，常量常被视为接口的一部分；有些非常简单的函数出于性能考虑需要是 inline 的（因而要放在头文件里）；而当前的模板实现要求完整的模板定义放在头文件中。

命名与排版

R300：必须使用缩进，且在同一源文件内保持一致。

理由：可读性与风格。

R301：每条新语句都应从新的一行开始。

理由：可读性。

示例：

```
int a = 7; x = a+7; f(x,9);   // 违规

int a = 7;                    // 正确
x = a+7;                      // 正确
f(x,9);                       // 正确
```

示例：

```
if (p<q) cout << *p;  //违规
```

示例：

```
if (p<q)
    cout << *p;   // 正确
```

R302：标识符应取描述性的名字。

- 标识符可以包含常用缩写与首字母缩略词。
- 按惯例使用时，x、y、i、j 等都是描述性的。
- 使用 number_of_elements 风格，而不是 numberOfElements 风格。
- 不得使用匈牙利命名法。
- 只有类型名、模板名和命名空间名以大写字母开头。
- 避免过长的名字。

示例：Device_driver 和 Buffer_pool。

理由：可读性。

注意：以一条下划线开头的标识符，按 C++ 标准是保留给语言的实现使用的，因此被禁止。

例外：调用经过批准的库时，可以使用该库中的名字。

R303：标识符之间不得仅仅因下列差异而不同：

- 大小写的混用
- 下划线字符的有无
- 字母 O 与数字 0 或字母 D 互换
- 字母 I 与数字 1 或字母 l 互换
- 字母 S 与数字 5 互换
- 字母 Z 与数字 2 互换
- 字母 n 与字母 h 互换

示例：`Head` 和 `head` `//违规`

理由：可读性。

R304：不得有完全由大写字母和下划线构成的标识符。

示例：`BLUE` 和 `BLUE_CHEESE` `//违规`

理由：全大写字母被广泛用于经批准的库的 `#include` 文件中可能用到的宏。

例外：用于 `#include` 卫哨（guard）的宏名。

函数与表达式规则

r400：内层作用域中的标识符不应与外层作用域中的标识符相同。

示例：

```
int var = 9; { int var = 7; ++var; }  //违规：var 遮蔽了 var
```

理由：可读性。

R401：声明应放在尽可能小的作用域中。

理由：让初始化与使用彼此靠近，能最大限度减少混淆的可能；让变量离开作用域还能释放其资源。

R402：变量都应初始化。

示例：

```
int var;                          // 违规：var 未初始化
```

理由：未初始化的变量是常见的错误来源。

例外：立即从输入中读入取值的变量不必初始化。

注意：许多类型（如 `vector` 和 `string`）都有默认构造函数，可以保证初始化。

R403：不得使用强制类型转换（cast）。

理由：强制类型转换是常见的错误来源。

例外：可以使用 `dynamic_cast`。

例外：具名转换（named cast）可用于把硬件地址转换为指针，也可用于把从程序外部来源（例如图形用户界面库）得到的 `void*` 转换为适当类型的指针。

R404：接口中不得使用内置数组；也就是说，作为函数实参的指针应被假定为指向单个元素。传递数组请使用 `Array_ref`。

理由：数组是以指针方式传递的，其元素个数不会随之前传到被调函数。此外，隐式的数组到指针转换与隐式的派生类到基类转换相结合，可能导致内存被破坏。

类规则

R500：没有公有数据成员的类用 `class`；没有私有数据成员的类用 `struct`。不要使用既有公有数据成员又有私有数据成员的类。

理由：清晰。

r501：如果一个类有析构函数，或者有指针类型、引用类型的成员，那么它必须定义或禁止复制构造函数与复制赋值。

理由：析构函数通常会释放某种资源。对于指针成员、引用成员以及带有析构函数的类，默认的复制语义很少能“做对事”。

R502：如果一个类有虚函数，它就必须有虚析构函数。

理由：类之所以有虚函数，是为了能通过基类接口使用它。只通过该基类了解某个对象的函数可能会 delete 这个对象，而派生类需要有机会（在自己的析构函数中）做清理。

r503：只接受一个实参的构造函数必须声明为 explicit。

理由：避免令人意外的隐式转换。

硬实时规则

R800：不得使用异常。

理由：不可预测。

R801：`new` 只应在启动阶段使用。

理由：不可预测。

例外：placement new（按标准含义）可用于从栈中分配的内存。

R802：不得使用 `delete`。

理由：不可预测；还可能造成内存碎片。

R803：不得使用 `dynamic_cast`。

理由：不可预测（假定采用常见的实现技术）。

R804：除 `std::array` 外，不得使用标准库容器。

理由：不可预测（假定采用常见的实现技术）。

关键系统规则

R900：自增和自减运算不得用作子表达式。

示例：

```
int x = v[++i];      // 违规
```

示例：

```
++i;
int x = v[i];         // 可以
```

理由：这样的自增容易被忽略。

R901：代码不应依赖算术表达式层级以下的优先级规则。

示例：

```
x = a*b+c;     // 可以
```

示例：

```
if ( a<b || c<=d)  // 违规：应给 (a<b) 和 (c<=d) 加括号
```

理由：在 C/C++ 功底较弱的程序员所写的代码中，反复发现由优先级造成的混淆。

我们有意在编号中留出空缺，这样就能在不改动已有规则编号、并且仍能通过编号看出总体分类的前提下加入新规则。规则往往以编号为人所知，所以重新编号会遭到用户抵制。

25.6.3 真正的编码规范

存在大量 C++ 编码规范。大多数是公司内部的，并未广泛公开。在很多情况下，这大概是件好事——也许对那些公司自己的程序员例外。下面列出的这些规范，只要在它们适用的领域里恰当使用，就能带来一些好处：

- **Google C++ 风格指南**：http://google-styleguide.googlecode.com/svn/trunk/cppguide.xml。一份风格相当老旧、限制颇多但在不断演进的风格指南。
- Lockheed Martin 公司，《Joint Strike Fighter Air Vehicle Coding Standards for the System Development and Demonstration Program》（系统开发与演示计划用联合攻击战斗机飞行器编码标准）。文件编号 2RDU00001 Rev C，2005 年 12 月。俗称“JSF++”；这是洛克希德·马丁航空公司为飞行器（也就是“飞机”）软件写的一套规则。这些规则确实是由那些生产“人命关天”的软件的程序员编写、也为他们所用的。www.stroustrup.com/JSF-AV-rules.pdf。
- Programming Research 公司，《High-integrity C++ Coding Standard Manual》2.4 版。www.programmingresearch.com。
- Herb Sutter、Andrei Alexandrescu，《C++ Coding Standards: 101 Rules, Guidelines, and Best Practices》。Addison-Wesley，2004 年。ISBN 0321113586。这更像一份“元编码规范”：它给出的不是具体规则，而是关于哪些规则好、为什么好的指导。

要注意：熟悉你的应用领域、你的编程语言以及相关的编程技术，这是任何东西都替代不了的。对大多数应用——当然也包括大多数嵌入式系统编程——你还需要熟悉操作系统和/或硬件体系结构。如果你需要用 C++ 做底层编程，不妨看看 ISO C++ 委员会关于性能的报告（ISO/IEC TR 18015，www.stroustrup.com/performanceTR.pdf）；他们/我们所说的“性能”，主要指的就是“嵌入式系统编程”。

嵌入式系统领域里充斥着各种语言方言和专有语言，但只要有可能，就应当使用标准化的语言（例如 ISO C++）、工具和库。这样能把你需要学习的东西减到最少，也能让你的工作更有可能长久。

小试身手

1. 运行下面这段代码：

   ```
   int v = 1; for (int i = 0; i<sizeof(v)*8; ++i) { cout << v << ' '; v <<=1; }
   ```

2. 把 v 声明为 `unsigned int`，再运行一遍。
3. 用十六进制字面量定义若干个 `short unsigned int`，使它们分别具有下列位模式：
   (a). 所有位都置位
   (b). 最低位（最低有效位）置位
   (c). 最高位（最高有效位）置位
   (d). 最低字节置位
   (e). 最高字节置位
   (f). 每隔一位置位（且最低位为 1）
   (g). 每隔一位置位（且最低位为 0）

4. 把每一个都按十进制和十六进制打印出来。
5. 只使用位操作运算（`|`、`&`、`<<`）和（仅有的）字面量 1 和 0，完成第 3、4 题。

要点回顾

1. 什么是嵌入式系统？举出十个例子，其中至少要有三个不是本章提到过的。
2. 嵌入式系统有什么特别之处？举出五个常见的关注点。
3. 请给出可预测性（predictability）在嵌入式系统语境下的定义。
4. 为什么嵌入式系统可能难以维护和修理？
5. 为什么为了性能而优化系统可能不是个好主意？
6. 为什么我们更偏爱抽象层次较高的做法，而不是低层代码？
7. 什么是瞬态错误（transient error）？我们为什么特别怕它？
8. 怎样设计一个系统，使它在发生故障后仍能存活？
9. 为什么我们无法防止一切故障？
10. 什么是领域知识？举出一些应用领域的例子。
11. 为什么编写嵌入式系统程序需要领域知识？
12. 什么是子系统？举例说明。
13. 从 C++ 语言的角度看，存储有哪三种？
14. 你会在什么时候愿意使用自由存储（free store）？
15. 为什么在嵌入式系统中使用自由存储往往不可行？
16. 在嵌入式系统中，什么时候可以安全地使用 `new`？
17. 在嵌入式系统的语境下，`std::vector` 有什么潜在问题？
18. 在嵌入式系统的语境下，异常有什么潜在问题？
19. 什么是递归函数调用？为什么有些嵌入式系统程序员避免使用它？他们改用的是什么？
20. 什么是内存碎片（memory fragmentation）？
21. （在编程的语境下）什么是垃圾收集器？
22. 什么是内存泄漏？它为什么会成为问题？
23. 什么是资源？举例说明。
24. 什么是资源泄漏？我们怎样才能系统地防止它？
25. 为什么我们无法轻易地把对象从内存中的一个位置移到另一个位置？
26. 什么是栈（stack）？
27. 什么是池（pool）？
28. 为什么使用栈和池不会导致内存碎片？
29. 为什么需要 `reinterpret_cast`？它为什么很糟糕？
30. 为什么指针作为函数实参是危险的？举例说明。
31. 使用指针和数组会带来哪些问题？举例说明。
32. 在接口中，除了使用（指向数组的）指针，还有哪些替代做法？
33. 什么是“计算机科学第一定律”？
34. 什么是位（bit）？
35. 什么是字节（byte）？
36. 一个字节通常有多少位？
37. 对位的集合，我们有哪些运算？

38. 什么是“异或”（exclusive or）？它为什么有用？
39. 我们怎样表示一个位的集合（序列，或者随便你叫它什么）？
40. 按惯例，我们怎样给一个字（word）中的位编号？
41. 按惯例，我们怎样给一个字中的字节编号？
42. 什么是字（word）？
43. 一个字通常有多少位？
44. 0xf7 的十进制值是多少？
45. 0xab 对应的是怎样的位序列？
46. 什么是 `bitset`？什么时候会需要它？
47. `unsigned int` 与 `signed int` 有什么区别？
48. 什么时候你会更愿意用 `unsigned int` 而不是 `signed int`？
49. 如果要循环处理的元素个数非常多，你会怎样写这个循环？
50. 把一个 `unsigned int` 赋值为 -3 之后，它的值是多少？
51. 为什么我们会想要直接操作位和字节（而不是更高层的类型）？
52. 什么是位域（bitfield）？
53. 位域有什么用途？
54. 什么是加密（enciphering）？我们为什么要用它？
55. 你能加密一张照片吗？
56. TEA 代表什么？
57. 怎样把一个数按十六进制形式输出？
58. 编码标准的目的是什么？列出制定编码标准的理由。
59. 为什么不可能有普遍适用的编码标准？
60. 列出一条好的编码标准应具备的一些性质。
61. 编码标准怎么会造成危害？
62. 列出至少十条你自己喜欢（觉得有用）的编码规则，并说明它们为什么有用。
63. 为什么我们避免使用 `ALL_CAPITAL` 形式的标识符？

术语

地址（address）、位（bit）、位域（bitfield）、`bitset`、编码标准（coding standard）、嵌入式系统（embedded system）、加密（encryption）、异或（exclusive or）、小装置（gadget）、垃圾收集器（garbage collector）、硬实时（hard real time）、泄漏（leak）、池（pool）、可预测性（predictability）、实时（real time）、资源（resource）、软实时（soft real time）、无符号（unsigned）

习题

1. 如果你还没有做，请完成本章的“试一试”练习。
2. 列出一批能用十六进制记法拼出来的单词：把 0 读作 o，把 1 读作 l，把 2 读作 to，等等；例如 Foo1 和 Beef。交上去评分之前，请把其中的粗俗字眼去掉。
3. 用一个 32 位有符号整数，分别以下列位模式初始化并打印结果：全零；全一；1 与 0 交替（最左边为 1）；0 与 1 交替（最左边为 0）；110011001100……模式；001100110011……模式；全一字节与全零字节交替、且以全一字节开头；全一字节与全零字节交替、且以全零字节开头。再用一个 32 位无符号整数重做这道题。
4. 把按位逻辑运算符 `&`、`|`、`^`、`~` 加入第 7 章的计算器。

5. 写一个无限循环，并运行它。
6. 写一个不容易被认出是无限循环的无限循环。如果循环在把某种资源彻底耗尽之后就终止，那它其实不算真正的无限循环，这种写法可以接受。
7. 写出 0 到 400 的十六进制值；再写出 -200 到 200 的十六进制值。
8. 写出你键盘上每个字符的数值。
9. 不使用任何标准头文件（例如 `<limits>`），也不查文档，算出 `int` 有多少位，并判定在你的实现上 `char` 是有符号的还是无符号的。
10. 看一看 §25.5.5 的位域例子。写一个例子：先初始化一个 PPN，然后读出并打印每个字段的值，接着改变每个字段的值（给字段赋值）并打印结果。重做这道题，但把 PPN 信息存放在一个 32 位无符号整数里，并用位操作运算符（§25.5.4）来访问这个字中的位。
11. 重做上一题，但把各个位放在一个 `bitset<32>` 里。
12. 写出 §25.5.6 中那个例子的明文。
13. 用 TEA（§25.5.6）在两台计算机之间“安全地”通信。用电子邮件勉强也算数。
14. 实现一个简单的 `vector`，它最多能容纳 N 个元素，元素从池中分配。用 N==1000、元素类型为 `int` 的情况测试它。
15. 用 `new` 分配 10,000 个大小随机、位于 [1000:0) 字节范围内的对象，测量所需的时间（§26.6.1）；再用 `delete` 释放它们，测量所需的时间。做两遍：一遍按分配的逆序释放，一遍按随机顺序释放。然后做同样的事：从池中分配 10,000 个 500 字节的对象并释放它们。再做同样的事：在栈上分配 10,000 个大小随机、位于 [1000:0) 字节范围内的对象，然后（按逆序）释放它们。比较这些测量结果。每种测量至少做三次，以确保结果一致。
16. 拟定 20 条编码风格规则（不要只照抄 §25.6 中的那些）。把它们应用到你最近写的一个 300 行以上的程序上。写一段简短的（一两页）评论，谈谈应用这些规则的体会：你发现代码中的错误了吗？代码变得更清晰了吗？有没有一些代码反而变得不那么清晰？然后根据这些经验修改这套规则。
17. 在 §25.4.3–4 中，我们提供了一个类 `Array_ref`，并声称它能让数组元素的访问更简单、更安全；特别是，我们声称它能正确处理继承。请试着用各种办法，在不使用强制类型转换、也不使用其他涉及未定义行为的操作的前提下，借助一个 `Array_ref<Shape*>` 把一个 `Rectangle*` 放进 `vector<Circle*>`。这应该是做不到的。

后记

那么，嵌入式系统编程基本上就是“摆弄位”（bit fiddling）吗？完全不是，特别是当你刻意把摆弄位减到最少、以免它成为正确性上的隐患时。不过，一个系统里总有些地方“必须摆弄”位和字节；问题只在于在哪儿摆弄、怎么摆弄。在大多数系统中，底层代码可以也应当被局部化。我们接触过的许多最有趣的系统都是嵌入式的，而一些最有趣、最具挑战性的编程任务也正是在这个领域里。

26 测试

"I have only proven the code correct, not tested it."
"我只证明了代码是正确的，并没有测试过它。"
——唐纳德·克努特

本章讨论测试以及为正确性而进行的设计。这些话题都很大，我们只能触及其表面。重点在于给出一些实用的想法与技术，用来测试程序中的各个单元，例如函数和类。我们会讨论接口的使用，以及怎样挑选要针对接口运行的测试。我们强调：设计系统时就应当设法简化测试，并且从开发的最早阶段就开始测试。文中还会简要讨论证明程序正确，以及处理性能问题。

26.1 我们想要什么
- **26.1.1** 告诫

26.2 证明
26.3 测试
- **26.3.1** 回归测试
- **26.3.2** 单元测试
- **26.3.3** 算法与非算法
- **26.3.4** 系统测试
- **26.3.5** 找出不成立的假设

26.4 为测试而设计
26.5 调试
26.6 性能
- **26.6.1** 计时

26.7 参考文献

26.1 我们想要什么

我们来做一个小实验。写一个二分查找（binary search）。现在就写，不要等到本章末尾，也不要等到下一节之后。动手试一试很重要。就现在！二分查找是在一个有序序列中进行的查找，它从中间开始：

- 如果中间的元素等于我们要查找的值，查找就结束了。
- 如果中间的元素小于我们要查找的值，我们就看右半部分，对右半部分再做一次二分查找。
- 如果中间的元素大于我们要查找的值，我们就看左半部分，对左半部分再做一次二分查找。
- 结果既要能表明查找是否成功，又要能让我们修改找到的那个元素，例如一个下标、一个指针或一个迭代器。

比较（排序）准则用小于（`<`）。数据结构、调用方式、返回结果的方式都随你，但查找的代码必须自己写。在这种少见的情况下，用别人写好的函数反而会适得其反，即使你恰当地注明了出处。特别是不要用标准库算法（`binary_search` 或 `equal_range`），在多数场合它们本会是你的第一选择。花多少时间都可以。

好了，现在你已经写好了自己的二分查找函数。如果还没写，请回到上一段。你怎么知道自己写的查找函数是正确的？如果还没有，请写下你确信这段代码正确的理由。你对这些推理有多大信心？你的论证中是否有些部分可能比较薄弱？

那只是一段极其简单的代码，实现的是一个非常规整、众所周知的算法。而你的编译器大约有 20 万行代码，你的操作系统有 1000 万到 5000 万行代码，你下次度假或开会要乘坐的飞机上，安全攸关的代码有 50 万到 200 万行。这些数字让你安心吗？你为二分查找函数所用的那些技术，放到真实世界的软件规模上还管用吗？

奇怪的是，面对这么多复杂的代码，大多数软件在大多数时候都能正确工作。我们不会把游戏泛滥的消费级 PC 上运行的任何东西算作“关键”软件。更重要的是，安全攸关的软件几乎总能正确工作。近十年来，我们想不起有哪架飞机或哪辆汽车是因为软件故障而坠毁或撞毁的。银行软件被一张金额为 0.00 的支票搞得严重混乱的故事，如今已经很久远了；这类事情基本上不再发生。可软件正是像你这样的人写的。你知道自己会犯错；我们都会犯错。那么“他们”是怎么做对的？

最根本的答案是：“我们”已经学会如何用不可靠的部件搭出可靠的系统。我们努力让每个程序、每个类、每个函数都正确，但第一次尝试通常都会失败。于是我们调试、测试、重新设计，尽可能多地找出并清除错误。然而，在任何不那么简单的系统里，总有些错误还潜伏着。我们知道这一点，却找不到它们——更准确地说，以我们能够且愿意付出的时间和精力，无法把它们全部找出来。接着我们再一次重新设计系统，使它能够从意外事件和“不可能”事件中恢复。结果可能就是极其可靠的系统。注意，这样可靠的系统里可能仍然藏着错误——通常确实如此——也仍会偶尔表现得不如我们所愿。但它们不会崩溃，并且总能提供最低限度可接受的服务。例如，在需求异常高的时候，电话系统也许无法接通每一个呼叫，但它从来不会连大量呼叫都接不通。

我们也可以很哲学地讨论：一个我们已经预想到并做了防范的意外错误，是否还算错误。不过还是别这样了。对系统构建者来说，更有利、更有成效的做法是“只管”想清楚怎样让系统更可靠。

26.1.1 告诫

测试是一个很大的话题。关于测试该怎么做，有好几个流派；不同的行业和应用领域在测试上也有不同的传统和标准。这很自然——电子游戏和航空电子软件确实不需要相同的可靠性标准——但它也导致术语和工具上的差异，令人困惑。请把本章当作个人项目的想法来源；如果在大型系统的测试中遇到问题，也可以把它当作一种理想参照。大型系统的测试涉及多种工具与组织结构的组合，在这里逐一描述没有多大意义。

26.2 证明

等一下！我们为什么不直接证明程序是正确的，而要围着测试忙来忙去？正如艾兹格·迪杰斯特拉（Edsger Dijkstra）一针见血地指出的：“测试能揭示错误的存在，却不能揭示错误的不存在。”于是就产生了一个显而易见的愿望：证明程序是正确的，“就像数学家证明定理那样”。

不幸的是，证明不那么简单的程序是正确的，超出了当前的技术水平（在极受约束的应用领域之外）；证明本身也可能出错（数学家写出的证明同样会出错）；而程序证明这整个领域又是一个高深的话题。所以我们尽力把程序组织得便于推理，说服自己它们是正确的。但同时我们也要测试（§26.3），并设法让代码对残留的错误有抵御能力（§26.4）。

26.3 测试

在 §5.11 中，我们把测试描述为“一种系统地查找错误的方法”。下面来看看具体的做法。

人们把单元测试与系统测试区分开来。“单元”是像函数或类这样、构成完整程序一部分的东西。如果我们把这些单元单独拿来测试，那么一旦发现错误，就知道该到哪里去找问题的原因：错误一定在我们正在测试的单元里（或者在我们用来执行测试的代码里）。这与系统测试形成对照：在系统测试中我们测试一个完整的系统，所能知道的只是错误“在系统的某个地方”。通常，在单元测试做得很好的前提下，系统测试中发现的错误与单元之间不受欢迎的相互作用有关。它们比单个单元内部的错误更难找，修复起来往往也更昂贵。

显然，一个单元（比如一个类）可以由其他单元（比如函数和其他类）组成，而一个系统（比如一个电子商务系统）也可以由其他系统（比如一个数据库、一个图形用户界面、一个网络系统和一个订单校验系统）组成，所以单元测试与系统测试之间的区分并不像你想的那么清楚。但总的想法是：把单元测好，我们就为自己省了事，也为最终用户省了痛苦。

看待测试的一种方式是：任何不那么简单的系统都是由单元搭起来的，而这些单元本身又由更小的单元搭成。于是我们先测试最小的单元，再测试由它们组成的单元，这样一路往上，直到测试完整个系统；也就是说，“系统”不过就是最大的那个单元（直到我们把它当作某个更大系统的单元来用）。

那么，我们先来考虑如何测试一个单元（例如一个函数、一个类、一个类层次或一个模板）。测试人员区分白箱测试（能看到被测对象的详细实现）与黑箱测试（只能看到被测对象的接口）。我们不会把这种区分看得太重；尽可以去读你测试的对象的实现。但要记住，以后可能有人来重写那份实现，所以尽量不要依赖接口中没有保证的东西。事实上，测试任何东西时的基本想法都是：往它的接口上扔我们能扔的一切，看它是否做出合理的响应。

提到有人（也许就是你自己）可能在你测试之后改动代码，就引出了回归测试。基本上，每当你做了改动，就必须重新测试，确保没有弄坏什么。所以，改进了一个单元之后，你要重跑它的单元测试；而在把整个系统交给别人（或者自己拿它去做真正的事情）之前，你要运行完整的系统测试。

运行这样的系统完整测试通常称为回归测试（regression testing），因为它通常包含运行那些以前发现过错误的测试，看看这些错误是否仍然被修复着。如果没有，程序就“退化”了，需要再次修复。

26.3.1 回归测试

把过去对发现错误有用的测试积累成一大套，是为一套有效的系统测试集打基础的主要工作。假设你有用户，他们会给你发缺陷报告。永远不要扔掉缺陷报告！专业人士用缺陷跟踪系统来保证这一点。无论如何，一份缺陷报告要么说明系统中有错误，要么说明用户对系统的理解有错误。无论哪种情况，它都有用。

通常，一份缺陷报告里包含太多无关信息，处理它的第一件事就是写出能重现所报告问题的最小的程序。这往往要删掉提交代码的大部分内容：特别是，我们设法去掉那些不影响错误的库和应用程序代码。找出这个最小测试程序，常常有助于我们把缺陷定位到系统的代码中；而这个最小程序正是要加入回归测试集的东西。找出最小程序的办法是不断删除代码，直到错误消失——然后把它最后删掉的那一点代码再放回去。如此反复，直到没有可删的候选。

只因为旧的缺陷报告就运行成百上千（甚至上万）个测试，看上去也许不太系统，但我们在这里真正做的，是系统地利用用户和开发者的经验。回归测试集是开发团队“组织记忆”的重要组成部分。

对于一个大型系统，我们根本无法指望原来的开发者还留在身边，来解释设计与实现的细节。正是回归测试集，防止一个系统逐渐偏离开发者和用户一致认可的应有行为。

26.3.2 单元测试

好，话说得够多了！来看一个具体的例子：我们来测试二分查找。下面是 ISO 标准（§25.3.3.4）中的规格说明：

```
template<class ForwardIterator, class T>
bool binary_search(ForwardIterator first, ForwardIterator last,
    const T& value);

template<class ForwardIterator, class T, class Compare>
bool binary_search(ForwardIterator first, ForwardIterator last,
    const T& value, Compare comp);
```

- **要求（Requires）**：`[first,last)` 中的元素 `e` 相对于表达式 `e<value` 与 `!(value<e)`（或 `comp(e,value)` 与 `!comp(value,e)`）是划分好的。此外，对 `[first,last)` 中的所有元素 `e`，`e<value` 蕴含 `!(value<e)`，或 `comp(e,value)` 蕴含 `!comp(value,e)`。
- **返回（Returns）**：若区间 `[first,last)` 中存在满足相应条件的迭代器 `i`：`!(*i<value) && !(value<*i)`，或 `comp(*i,value)==false && comp(value,*i)==false`，则返回 `true`。
- **复杂度（Complexity）**：最多进行 `log(last-first)+2` 次比较。

没人说过形式化（好吧，半形式化）的规格说明对初学者来说是容易读的。不过，如果你真的按本章开头我们强烈建议的那样，动手设计并实现了一个二分查找，那么你对二分查找做什么、该怎么测试它，就已经相当有数了。这个（标准）版本接受一对前向迭代器（§20.10.1）和一个值作为参数，如果该值在迭代器所定义的区间内就返回 `true`。迭代器必须界定一个有序序列。比较（排序）准则是 `<`。带比较准则作为额外参数的第二个 `binary_search` 版本，我们留作练习。

这里我们只处理编译器抓不到的错误，所以像下面这样的例子属于别人的问题：

```
binary_search(1,4,5);                   // 错误：int 不是前向迭代器
vector<int> v(10);
binary_search(v.begin(),v.end(),"7");   // 错误：不能在 int 的 vector 中查找 string
binary_search(v.begin(),v.end());       // 错误：漏掉了要查找的值
```

我们怎样才能系统地测试 `binary_search()`？显然，我们不能把它的每一种可能参数都试一遍，因为每一种可能的参数意味着每一种可能类型的值的每一种可能序列——那将是无穷多个测试！所以，我们必须挑选测试，而为了挑选，还需要一些做出选择的原则：

- 测试容易犯的错误（找出最多的错误）。
- 测试后果严重的错误（找出潜在后果最严重的错误）。

所谓“严重”，指的是会带来最可怕后果的错误。一般来说这是个模糊的概念，但对具体的程序可以讲清楚。例如，孤立地看一个二分查找，所有错误都差不多糟；但如果我们把那个 `binary_search()` 用在一个对每个答案都仔细复核的程序里，那么它给出错误答案，可能远比它陷入死循环而根本不返回要容易接受。在这种情况下，我们会花更大力气去骗 `binary_search()` 进入死循环（或很长的循环），而不是去骗它给出错误答案。注意我们这里用了“骗”这个字。测试——除了别的以外——是一种把创

造性思维用在“我们怎样才能让这段代码出毛病？”这个问题上的练习。最好的测试人员不仅有条理，而且相当狡猾（当然，是为了正当目的）。

26.3.2.1 测试策略

我们怎样去打破 binary_search()？先从看它的要求开始，也就是它对输入做了哪些假定。不幸的是，从我们测试者的角度看，标准里明确写着 [first,last) 必须是有序序列；也就是说，保证这一点是调用者的责任，所以我们不能靠给它未排序的输入、或给它一个 last<first 的 [first,last) 来公平地打破 binary_search()。注意，binary_search() 的要求并没有说：如果我们给它的输入不满足要求，它会怎样。标准的其他部分说，那种情况下它可以抛出异常，但并不要求它这样做。不过，这些事实在我们测试 binary_search() 的调用时值得记住，因为调用者没能建立 binary_search() 这类函数的前置要求，是错误的一个常见来源。

我们可以设想 binary_search() 会出现下面几类错误：

- 永不返回（例如死循环）
- 崩溃（例如错误的解引用、无限递归）
- 值在序列中却找不到
- 值不在序列中却找到了

此外，我们还记住了用户可能犯错的几种“机会”：

- 序列没有排序（例如 {2,1,5,-7,2,10}）。
- 序列不是合法的序列（例如 binary_search(&a[100], &a[50],77)）。

对于一个简单的调用 binary_search(p1,p2,v)，实现者可能怎样犯错（好让测试者去找）？错误常常出在“特殊情况”上。特别是，在考虑（任何种类的）序列时，我们总要检查开头和结尾。尤其是空序列必须总是被测到。那么，我们来看几个按题意排好序的整数数组：

```
{ 1,2,3,5,8,13,21 }                 // 一个“普通序列”
{ }                                 // 空序列
{ 1 }                               // 只有一个元素
{ 1,2,3,4 }                         // 元素个数为偶数
{ 1,2,3,4,5 }                       // 元素个数为奇数
{ 1, 1, 1, 1, 1, 1, 1 }             // 所有元素都相等
{ 0,1,1,1,1,1,1,1,1,1,1,1,1 }       // 开头有一个不同的元素
{ 0,0,0,0,0,0,0,0,0,0,0,0,0,1 }     // 末尾有一个不同的元素
```

有些测试序列最好由程序生成：

- vector<int> v1;
  for (int i=0; i<100000000; ++i) v.push_back(i); //一个非常大的序列
- 元素个数随机的若干序列
- 元素随机（但仍然有序）的若干序列

这并不像我们希望的那样系统。毕竟，我们是“随手挑”了一些序列。不过，我们用了一些相当通用的经验规则，它们在处理值的集合时常常有用；请考虑：

- 空集
- 小集合
- 大集合

- 分布极端的集合
- “我们关心的东西”出现在末尾附近的集合
- 含重复元素的集合
- 元素个数为偶数和为奇数的集合
- 用随机数生成的集合

我们用随机序列，只是为了看看能不能碰巧（也就是，找出一个错误）撞上我们没想到的东西。这是蛮力技术，但就我们的时间而言相对便宜。

为什么“奇数与偶数”？嗯，很多算法会把输入序列划分开，例如分成前半和后半，而程序员可能只考虑了奇数或偶数的情况。更一般地说，当我们划分一个序列时，划分点就成了某个子序列的末尾，而我们知道，错误往往出现在序列的末尾附近。

一般说来，我们要找的是

- 极端情形（很大、很小、输入分布奇怪，等等）
- 边界条件（任何靠近极限的东西）

这到底意味着什么，取决于我们正在测试的那个具体程序。

26.3.2.2 一个简单的测试框架

我们的测试有两类：应当成功的测试（例如查找一个在序列中的值）和应当失败的测试（例如在一个空序列中查找一个值）。对我们的每个序列，我们都构造一些应成功和应失败的测试。我们从最简单、最明显的做法开始，然后逐步改进，直到对二分查找这个例子来说足够好：

```
vector<int> v { 1,2,3,5,8,13,21 };
if (binary_search(v.begin(),v.end(),1) == false) cout << "failed";
if (binary_search(v.begin(),v.end(),5) == false) cout << "failed";
if (binary_search(v.begin(),v.end(),8) == false) cout << "failed";
if (binary_search(v.begin(),v.end(),21) == false) cout << "failed";
if (binary_search(v.begin(),v.end(),-7) == true) cout << "failed";
if (binary_search(v.begin(),v.end(),4) == true) cout << "failed";
if (binary_search(v.begin(),v.end(),22) == true) cout << "failed";
```

这很重复、很乏味，但开头这样也行。事实上，许多简单的测试不过是一长串这样的调用。这种朴素做法有一个优点：极其简单。就连测试组里最新的成员也能往测试集里加一个新测试。不过，我们通常可以做得更好。例如，这里某个测试失败时，我们并不知道是哪个测试失败。这不可接受。而且，写测试并不是退回到“剪切粘贴”式编程的借口。我们需要像对待其他代码一样，考虑测试代码的设计。于是：

```
vector<int> v { 1,2,3,5,8,13,21 };
for (int x : {1,5,8,21,-7,2,44})
    if (binary_search(v.begin(),v.end(),x) == false) cout << x << " failed";
```

假定我们最终会有几十个测试，这就大不一样了。测试真实世界的系统时，我们常常有成千上万个测试，所以精确说明哪个测试失败了是必不可少的。

在继续之前，请注意另一个（半系统的）测试技术的例子：我们用正确的值来测试，其中一些取自序列的两端，一些取自“中间”。对这个序列，我们本可以试遍所有值，但通常那不是一个现实的选

择。对于应当失败的值，我们从两端各取一个，再在中间取一个。这同样不算完全系统，但我们开始看到一个模式：只要处理值的序列或值的范围，它就有用——而这非常常见。

这些测试有什么问题？

- 我们（起初）反复写了同样的东西。
- 我们（起初）手工给测试编号。
- 输出非常少（帮助不大）。

这样看了一阵之后，我们决定把测试作为数据保存在文件里。每个测试包含一个标识标签、一个要查找的值、一个序列和一个预期结果。例如：

```
{ 27 7 { 1 2 3 5 8 13 21} 0 }
```

这是 27 号测试。它在序列 1,2,3,5,8,13,21 中查找 7，预期结果是 0（即 false）。为什么我们把测试输入放在文件里，而不是直接写进测试程序的正文？嗯，在这个例子里，我们本可以把测试直接敲进程序文本，但把大量数据放进源代码文件会显得凌乱；而且我们常常用程序来生成测试用例。机器生成的测试用例通常放在数据文件里。另外，现在我们可以写一个测试程序，用各种不同的测试用例文件来运行它：

```
struct Test {
    string label;
    int val;
    vector<int> seq;
    bool res;
};


istream& operator>>(istream& is, Test& t);    // 使用所描述的格式


int test_all(istream& is)
{
    int error_count = 0;
    for (Test t; is>>t; ) {
        bool r = binary_search(t.seq.begin(), t.seq.end(), t.val);
        if (r !=t.res) {
            cout << "failure: test " << t.label
                 << " binary_search: "
                 << t.seq.size() << " elements, val==" << t.val
                 << " -> " << t.res << '\n';
            ++error_count;
        }
    }
    return error_count;
}
```

```
int main()
{
    int errors = test_all(ifstream("my_tests.txt"));
    cout << "number of errors: " << errors << "\n";
}
```

下面是用上面列出的那些序列写出的一些测试输入：

```
{ 1.1 1 { 1 2 3 5 8 13 21 } 1 }
{ 1.2 5 { 1 2 3 5 8 13 21 } 1 }
{ 1.3 8 { 1 2 3 5 8 13 21 } 1 }
{ 1.4 21 { 1 2 3 5 8 13 21 } 1 }
{ 1.5 -7 { 1 2 3 5 8 13 21 } 0 }
{ 1.6 4 { 1 2 3 5 8 13 21 } 0 }
{ 1.7 22 { 1 2 3 5 8 13 21 } 0 }

{ 2 1 { } 0 }

{ 3.1 1 { 1 } 1 }
{ 3.2 0 { 1 } 0 }
{ 3.3 2 { 1 } 0 }
```

这里就能看出我们为什么用字符串标签而不用数字：这样我们可以用更灵活的方式给测试“编号”——这里用的是十进制，用来表示针对同一个序列的几个不同测试。更复杂的格式可以省去在测试数据文件里重复写同一个序列的麻烦。

26.3.2.3 随机序列

在选择用于测试的值时，我们设法智胜实现者（往往就是我们自己），去选那些集中在已知错误可能藏身之处的值（例如复杂的条件序列、序列的末尾、循环，等等）。然而，我们编写和调试代码时做的也是同样的事。所以，我们在设计测试时可能重复设计中的某个逻辑错误，从而完全漏掉一个问题。这也是让开发者以外的人参与测试设计的一个理由。有一种技术偶尔能帮上这个忙：直接生成（大量）随机值。例如，下面这个函数用 §24.7 和 `std_lib_facilities.h` 中的 `randint()`，把一条测试描述写到 `cout`：

```
void make_test(const string& lab, int n, int base, int spread)
    // 把一个带标签 lab 的测试描述写到 cout
    // 生成一个由 n 个元素组成的序列，从 base 开始
    // 相邻元素之间的平均距离在 [0:spread) 内均匀分布
{
    cout << "{ " << lab << " " << n << " { ";
    vector<int> v;
    int elem = base;
    for (int i = 0; i<n; ++i) {    // 生成元素
        elem+= randint(spread);
        v.push_back(elem);
    }
    int val = base+ randint(elem-base);    // 生成要查找的值
    bool found = false;
    for (int i = 0; i<n; ++i) {
        // 打印元素，并查看 val 是否被找到
        if (v[i]==val) found = true;
        cout << v[i] << " ";
    }
    cout << "} " << found << " }\n";
}
```

注意，我们没有用 binary_search 去判断随机的 val 是否在那个随机序列里。我们不能用被测的东西来确定一个测试的正确取值。

其实，二分查找并不是蛮力随机数测试法特别合适的例子。我们怀疑这样找不出任何“手工”测试没抓到的错误，但这种技术常常有用。不管怎样，我们来生成几个随机测试：

```
int no_of_tests = randint(100);            // 大约生成 50 个测试
for (int i=0; i<no_of_tests; ++i) {
    string lab = "rand_test_";
    make_test(lab+to_string(i),            // to_string 见 §23.2


        randint(500),              // 元素个数
        0,                         // 基数
        randint(50));              // 跨度
}
```

基于随机数生成的测试，在我们需要检验大量操作的累积效果时特别有用——某个操作的结果取决于先前各操作是如何处理的，也就是说，系统是有状态的；参见 §5.2。

随机数对 binary_search 之所以没有多大用处，是因为对同一序列的每次搜索都与其他搜索无关。当然，这个说法假设 binary_search 的实现没有干出什么致命的蠢事，比如修改它自己的序列。对此我们有更好的测试办法（习题 5）。

26.3.3 算法与非算法

我们一直以 `binary_search()` 作为例子。它是一个真正的算法，具备以下特点：

- 对输入有明确规定的需求；
- 对输入有明确规定的效果（本例中没有任何效果）；
- 不依赖任何不是其显式输入的对象；
- 不受环境施加的严重约束（例如没有规定的时间、空间或资源共享要求）。

它有着显而易见且明确陈述的前置条件和后置条件（§5.10）。换句话说，它是测试者的梦想。可惜我们常常没有这么幸运：我们不得不测试混乱的代码，而这些代码（充其量）只是由一段颇为马虎的英文说明和几张图定义出来的。

等等！我们是不是在这里把逻辑搞马虎了？当我们对代码应该做什么还没有精确的规格说明时，怎么能谈正确性和测试呢？问题在于，软件中需要完成的很多事情，并不容易用完全清晰的数学术语来规定。而且在许多理论上可以那样规定的情形中，所需的数学也超出了编写和测试这些代码的程序员的能力。于是我们只能接受这样的局面：理想是完全精确的规格说明，现实则是某个人（比如我们）在真实世界的条件和时间压力下所能做到的程度。

那么，假设你有一个混乱的函数必须测试。我们所说的“混乱”是指：

- 输入：它对（显式或隐式）输入的需求规定得没有我们希望的那样好。
- 输出：它（显式或隐式）的输出规定得没有我们希望的那样好。
- 资源：它对资源（时间、内存、文件等）的使用规定得没有我们希望的那样好。

所谓“显式或隐式”，是指我们不仅要看形式参数和返回值，还要看它对全局变量、iostream、文件、自由存储区内存分配等的任何影响。那么，我们能做些什么呢？首先，这样的函数几乎可以肯定太长了——否则我们本可以更清楚地陈述它的需求和效果。也许我们说的是一个长达五页的函数，或者以复杂而不明显的方式使用“辅助函数”的函数。你可能觉得五页对一个函数来说已经很多了。确实很多，但我们见过比这长得多得多的函数。不幸的是，它们并不罕见。

如果这是我们自己的代码，而且我们有时间，我们首先会设法把这样一个“混乱的函数”拆成更小的函数，让它们更接近我们对“规格良好的函数”的理想，并先测试这些小函数。不过在这里，我们假定目标是测试软件——也就是系统地找出尽可能多的错误——而不只是（在发现错误时）把它修好。

那么，我们要找什么？作为测试者，我们的工作是找出错误。错误可能藏在哪儿？什么样的代码容易含有错误？

- 对“其他代码”的隐蔽依赖：留意全局变量、非 `const` 引用参数、指针等的使用。
- 资源管理：留意内存管理（`new` 和 `delete`）、文件使用、锁等。
- 留意循环：检查结束条件（如 `binary_search()` 那样）。
- `if` 语句和 `switch` 语句（常称为“分支”）：检查其逻辑中的错误。

下面我们逐个看例子。

26.3.3.1 依赖

```
int do_dependent(int a, int& b)         // 混乱的函数
// 无约束的依赖
{
    int val;
    cin>>val;
    vec[val] += 10;
    cout << a;
    b++;
    return b;
}
```

要测试 do_dependent()，我们不能只是合成若干组参数，然后看它对这些参数做了什么。我们必须考虑到它使用了全局变量 cin、cout 和 vec。在这个小小的荒唐函数里，这一点相当明显，但在真实的代码中，它可能隐藏在大段代码里。幸运的是，有一些软件能帮我们找出这类依赖。不幸的是，这类软件并不总是容易获得或得到广泛使用。假设我们没有分析软件帮忙，那就逐行检查这个函数，列出它所有的依赖。

要测试 do_dependent()，我们必须考虑：

- 它的输入：
  - a 的值
  - b 的值，以及 b 所引用的那个 int 的值
  - 来自 cin 的输入（送入 val）以及 cin 的状态
  - cout 的状态
  - vec 的值，特别是 vec[val] 的值
- 它的输出：
  - 返回值
  - b 所引用的那个 int 的值（我们把它自增了）
  - cin 的状态（当心流状态和格式状态）
  - cout 的状态（当心流状态和格式状态）
  - vec 的状态（我们给 vec[val] 赋了值）
  - vec 可能抛出的任何异常（vec[val] 可能越界）

这张清单很长。事实上，它比函数本身还长。这很能说明我们为什么不喜欢全局变量，为什么对非 const 引用（以及指针）心存疑虑。一个只读取自己的参数、并把结果作为返回值产生的函数，确实有非常可取之处：我们很容易理解和测试它。

一旦输入和输出都确定下来，我们基本上就回到了 binary_search() 的情形。我们只需用（针对显式输入和隐式输入的）输入值来生成测试，看看它们是否给出（同时考虑隐式输出和显式输出的）期望输出。对 do_dependent()，我们大概会先用一个非常大的 val 和一个负的 val 来试试，看看会发生什么。看来 vec 最好是一个带范围检查的 vector（否则我们很容易制造出真正糟糕的错误）。当然，我们会查看文档中对所有这些输入和输出的说明；但面对这样一个混乱的函数，我们几乎不指望规格说明是完整而精确的，所以我们干脆先把函数弄坏（也就是找出错误），然后再开始追问什么才是正确的。通常，这样的测试和追问应当导向重新设计。

26.3.3.2 资源管理

```
void do_resources1(int a, int b, const char* s)   // 混乱的函数
// 无约束的资源使用
{
    FILE* f = fopen(s,"r");                  // 打开文件（C 风格）
    int* p = new int[a];                     // 分配一些内存
    if (b<=0) throw Bad_arg();               // 也许抛出异常
    int* q = new int[b];                     // 再分配一些内存
    delete[] p;                              // 释放 p 所指的内存
}
```

要测试 do_resources1()，我们必须考虑每一个获取的资源是否都得到了妥善处置，也就是说，每个资源要么被释放，要么被移交给别的函数。

这里，显而易见的是：

- 名为 s 的文件没有关闭；
- 如果 b<=0 或者第二次 new 抛出异常，为 p 分配的内存就泄漏了；
- 如果 0<b，为 q 分配的内存就泄漏了。

此外，我们始终应当考虑打开文件的操作可能失败。为了得到这个糟糕的结果，我们故意使用了一种非常老派的编程风格（fopen() 是打开文件的标准 C 方式）。如果写成下面这样，本可以让测试者的工作更简单：

```
void do_resources2(int a, int b, const char* s)      // 不那么混乱的函数
{
    ifstream is(s);                      // 打开文件
    vector<int> v1(a);                   // 创建 vector（拥有内存）
    if (b<=0) throw Bad_arg();           // 也许抛出异常
    vector<int> v2(b);                   // 再创建一个 vector（拥有内存）
}
```

现在每个资源都由一个对象拥有，该对象的析构函数会释放它。想一想怎样把函数写得更简单（更干净），有时正是获得测试思路的好办法。§19.5.2 中的“资源获取即初始化”（RAII）技术，为这类资源管理问题提供了一般的策略。

请注意，资源管理并不只是检查每一块分配的内存是否都被释放。有时我们会从别处接收资源（例如作为参数），有时又会把资源传出函数（例如作为返回值）。判断这类情形中怎样才算正确，可能相当困难。考虑：

```
FILE* do_resources3(int a, int* p, const char* s)      // 混乱的函数
// 无约束的资源传递
{
    FILE* f = fopen(s,"r");
    delete p;
    delete var;
    var = new int[27];
    return f;
}
```

do_resources3() 把（据称已）打开的文件作为返回值传回去，这样做对吗？do_resources3() 把作为参数 p 传进来的内存删除了，这样做对吗？我们还加了一处非常狡猾的全局变量 var（显然是个指针）的用法。基本上，把资源传入传出函数既常见又有用，但要判断这样做是否正确，需要了解某种资源管理策略。谁拥有这个资源？应该由谁删除或释放它？文档应当清楚而简单地回答这些问题。（做梦吧。）无论哪种情况，资源传递都是错误滋生的沃土，也是测试的诱人目标。

注意我们（故意）用一个全局变量把资源管理的例子弄得更复杂了。当我们开始把各种可能的错误来源混在一起时，事情会变得非常混乱。作为程序员，我们力求避免这种情况。作为测试者，我们则把这类例子当作容易得手的目标。

26.3.3.3 循环

我们在讨论 binary_search() 时已经看过循环。基本上，大多数错误都出在两端：

- 循环开始时，一切是否都正确初始化了？
- 我们是否正确地在最后一种情形（通常是最后一个元素）处结束？

下面是一个我们把事情弄错的例子：

```
int do_loop(const vector<int>& v)        // 混乱的函数
// 无约束的循环
{
    int i;
    int sum;
    while(i<=vec.size()) sum+=v[i];
    return sum;
}
```

这里有三处明显的错误。（是哪三处？）此外，好的测试者会立刻发现给 sum 累加时存在溢出的可能：

- 许多循环要处理数据，当输入很大时，它们可能引起某种溢出。

一个著名且特别讨厌的循环错误——缓冲区溢出——属于可以通过系统地追问关于循环的两个关键问题来发现的那一类：

```
char buf[MAX];              // 固定大小的缓冲区

char* read_line()           // 危险地马虎
{
    int i = 0;
    char ch;
    while(cin.get(ch) && ch!='\n') buf[i++] = ch;
    buf[i+1] = 0;
    return buf;
}
```

当然，你不会写出这样的东西！（为什么不会？`read_line()` 到底错在哪里？）然而，遗憾的是它很常见，而且有许多变体，例如

```
// 危险地马虎:
gets(buf);              // 把一行读入 buf
scanf("%s",buf);        // 把一行读入 buf
```

在你的文档里查一查 `gets()` 和 `scanf()`，然后像躲避瘟疫一样避开它们。所谓“危险”，是指这类缓冲区溢出是计算机上“破解”——也就是入侵——的常用手段。正因为如此，许多实现现在会对 `gets()` 及其近亲发出警告。

26.3.3.4 分支

显然，当我们必须做出选择时，就可能做错选择。这使得 `if` 语句和 `switch` 语句成为测试者的好靶子。有两类主要问题值得注意：

- 是否覆盖了所有可能情况？
- 正确的动作是否与正确的情况相对应？

考虑下面这个荒唐的函数：

```
void do_branch1(int x, int y)        // 混乱的函数
// 无约束地使用 if
{
    if (x<0) {
        if (y<0)
            cout << "very negative\n";
        else
            cout << "somewhat negative\n";
    }
    else if (x>0) {
        if (y<0)
            cout << "very positive\n";
        else
            cout << "somewhat positive\n";
    }
}
```

这里最明显的错误是，我们“忘了”x 为 0 的情况。在针对零（或针对正值与负值）做测试时，零常常被忘掉，或者被错误地归入某一类（例如被当作负数）。此外，这里还潜伏着一个更隐蔽（但并不罕见）的错误：(x>0 && y<0) 和 (x>0 && y>=0) 对应的动作“不知怎么”被弄反了。这种错误在剪切粘贴式编辑中屡见不鲜。

if 语句的用法越复杂，这类错误就越容易出现。从测试者的角度看，我们会检查这样的代码，并设法确保每个分支都被测试到。对 do_branch1()，显而易见的测试集是

```
do_branch1(-1,-1);
do_branch1(-1, 1);
do_branch1(1,-1);
do_branch1(1,1);
do_branch1(-1,0);
do_branch1(0,-1);
do_branch1(1,0);
do_branch1(0,1);
do_branch1(0,0);
```

基本上，这就是在我们注意到 do_branch1() 用 < 和 > 与 0 比较之后，采取的暴力式“把所有备选情况都试一遍”的做法。要抓住 x 为正时那些错误的动作，我们必须把调用与期望的输出结合起来。

处理 switch 语句与处理 if 语句在本质上是相似的。

```
void do_branch1(int x, int y)   // 混乱的函数
// 无约束地使用 switch
{
    if (y<0 && y<=3)
        switch (x) {
        case 1:
            cout << "one\n";
            break;
        case 2:
            cout << "two\n";
        case 3:
            cout << "three\n";
    }
}
```

这里我们犯了四个典型错误：

- 我们对错误的变量做了范围检查（检查的是 y 而不是 x）。
- 我们漏掉了一个 break 语句，导致 x==2 时执行了错误的动作。
- 我们漏掉了 default 分支（以为用那个 if 语句已经处理好了）。
- 我们写成了 y<0，而本意是说 0<y。

作为测试者，我们总要寻找未被处理的情况。请注意，“仅仅把问题修好”是不够的。它可能在我们不注意的时候再次出现。作为测试者，我们要编写能系统地抓住错误的测试。如果我们只是把这个简单的代码修好，我们的修复很可能本身就是错的，结果要么没解决问题，要么引入了新的、不同的

错误。查看代码的目的其实不是发现错误（尽管那总是有用的），而是设计一组合适的测试，用来抓住所有错误（或者更现实地说，抓住许多错误）。

注意，循环带有一个隐式的“if”：它要测试我们是否已经到达末尾。因此循环也是分支语句。当我们查看含有分支的程序时，第一个问题总是：“每个分支我们都覆盖（测试）到了吗？”令人惊讶的是，在真实代码中这并不总是做得到（因为在真实代码中，函数是按其他函数的需要被调用的，未必会以所有可能的方式被调用）。因此，测试者常问的一个问题是：“你们的代码覆盖率是多少？”而回答最好是：“我们测试了大多数分支。”接着再解释为什么其余分支难以到达。100% 的覆盖率才是理想。

26.3.4 系统测试

测试任何重要的系统都是一项技术活。例如，控制电话系统的计算机要在专门建造的机房里测试，那里摆满了一排排机柜，用计算机模拟数万人的通话流量。这类系统耗资数百万，由技艺高超的工程师团队完成。一台主电话交换机部署之后，应当连续工作 20 年，停机时间最多 20 分钟（任何原因都算，包括停电、洪水和地震）。这里我们不再细讲——那难度大概相当于教一个物理系大一学生计算火星探测器的轨道修正——但我们会试着给你一些思路，它们对较小的项目有用，也有助于理解较大系统的测试。

首先，请记住，测试的目的是找出错误，尤其是那些可能频繁出现、可能很严重的错误。它并不只是编写并运行数量最多的测试。这意味着，对被测系统有一定了解是非常可取的。有效的系统测试甚至比单元测试更依赖对应用领域的知识（领域知识）。开发一个系统，需要的不仅是对编程语言问题和计算机科学的了解，还需要理解应用领域以及使用这些应用的人。我们认为，这一点正是激励我们与代码打交道的重要原因：我们能见到这么多有趣的应用，遇到有趣的人。

要对一个完整系统进行测试，必须用它的所有部件（单元）把它构建起来。这要花相当多的时间，因此许多系统测试每天只运行一次（常常在夜里，在开发者本该睡觉的时候），而且要在所有单元测试都完成之后。回归测试在这里是一个关键组成部分。程序中最容易发现错误的区域是新代码，以及早先发现过错误的代码区域。因此，运行旧的测试集合（回归测试）是必不可少的；没有它们，大型系统永远不会稳定下来。我们引入新错误的速度会和我们消除旧错误的速度一样快。

请注意，我们理所当然地认为：当我们修好一些错误时，会顺便引入一些新错误。我们希望新错误的数量少于我们消除的旧错误，并且新错误的后果不那么严重。然而，至少在我们重新运行回归测试、并为新代码补充新测试之前，我们必须假定系统是被我们的错误修复弄坏了的。

26.3.5 找出不成立的假设

`binary_search` 的规格说明清楚地指出，被搜索的序列必须是有序的。这使我们失去了许多做“狡猾”单元测试的机会。但显然，仍有一些机会可以写出坏代码而我们没有设计测试去发现它们（系统测试除外）。我们能否利用对系统“单元”（函数、类等）的理解来设计更好的测试？

不幸的是，最简单的回答是“不能”。作为纯粹的测试者，我们不能改动代码；而要发现对接口需求（前置条件）的违反，必须有人在每次调用之前、或者作为每次调用的实现的一部分来做检查（见§5.5）。不过，如果我们在测试自己的代码，就可以插入这类检查。如果我们只是测试者，而写代码的人愿意听我们的（情况并不总是如此），我们可以把这些未检查的需求告诉他们，让他们确保这些需求得到检查。

再考虑一次 `binary_search`：我们无法测试输入序列 `[first:last)` 确实是一个序列，也无法测试它是有序的（§26.3.2.2）。不过，我们可以写一个进行检查的函数：

```
template<class Iter, class T>
bool b2(Iter first, Iter last, const T& value)
{
    // 检查 [first:last) 是否是一个序列:
    if (last<first) throw Bad_sequence();

    // 检查序列是否有序:
    if (2<=last-first)
        for (Iter p = first+1; p<last; ++p)
            if (*p<*(p-1)) throw Not_ordered();

    // 一切正常，调用 binary_search:
    return binary_search(first,last,value);
}
```

binary_search 没有内置这类检查，原因包括以下几点：

- 对 last<first 的检查对前向迭代器无法进行；例如 std::list 的迭代器没有 <（§B.3.2）。一般来说，并没有真正好的办法来检验一对迭代器是否定义了一个序列（从 first 开始迭代、指望碰到 last，并不是个好主意）。
- 扫描序列以检查值是否有序，比执行 binary_search 本身要昂贵得多（binary_search 的真正目的，正是不必像 std::find 那样盲目遍历序列去找某个值）。

那么我们能做什么呢？我们可以在测试时用 b2 替换 binary_search（不过只适用于以随机访问迭代器调用 binary_search 的情形）。

另一种做法是，让 binary_search 的实现者插入一些代码，由测试者来启用：

```
template<class Iter, class T>           // 注意：含伪代码
bool binary_search (Iter first, Iter last, const T& value)
{
    if (test enabled) {
        if (Iter is a random access iterator) {
            // 检查 [first:last) 是否是一个序列:
            if (last<first) throw Bad_sequence();
        }
        // 检查序列是否有序:
        if (first!=last) {
            Iter prev = first;
            for (Iter p = ++first; p!=last; ++p, ++prev)
                if (*p<*prev) throw Not_ordered();
        }
    }
    // 现在执行 binary_search
}
```

由于“测试已启用”的含义取决于代码测试是如何组织的（特定的系统、特定的组织各有不同），我们把它留作伪代码：在测试你自己的代码时，你完全可以只用一个 test_enabled 变量。“迭代器是

随机访问迭代器”这项检查也被我们留作伪代码，因为我们还没有讲解“迭代器特征”（iterator traits）。如果你确实需要这样的检查，请到更深入的 C++ 教材里查阅迭代器特征。

26.4 为测试而设计

当我们开始写一个程序时，我们知道希望它最终完整而正确。我们也知道，要做到这一点就必须测试它。因此，我们力求从第一天起就为正确性和测试而设计。事实上，许多优秀的程序员把“尽早测试、经常测试”当作口号，在还没想好怎样测试之前不写任何代码。尽早考虑测试有助于从一开始就避免错误（也有助于日后发现错误）。我们认同这一理念。有些程序员甚至在实现一个单元之前就先写好它的单元测试。

§26.3.2.1 中的示例以及 §26.3.3 中的几个示例，说明了以下关键要点：

- 使用定义良好的接口，这样才能为这些接口的使用编写测试。
- 提供一种把操作用文本表示出来的办法，使它们能被保存、分析和重放。输出操作也是如此。
- 在调用方的代码中嵌入对原本不受检查的假设（断言）的检查，以便在系统测试之前就捕获错误的实参。
- 尽量减少依赖，并让依赖关系保持明确。
- 有清晰的资源管理策略。

从理念上说，这可以看作让单元测试技术也能用于子系统乃至整个系统。

如果性能无关紧要，我们就可以让对（原本不受检查的）假设（要求、前置条件）的检查一直开着。然而，通常总有理由不让这些检查系统地执行。例如，我们已经看到，检查一个序列是否有序，既复杂，又比调用 `binary_sort` 昂贵得多。因此，最好是设计这样一个系统：允许我们有选择地启用和停用这类检查。对许多系统来说，即使在最终的（交付）版本里，也最好保留相当多较便宜的检查：有时“不可能”的事就是会发生，而我们宁愿从一条具体的错误消息中得知它，也不愿只看到程序莫名崩溃。

26.5 调试

调试（debugging）既是技术问题，也是态度问题。两者之中，态度更重要。请重读第 5 章。注意调试与测试有何不同：两者都能抓住缺陷，但调试要随意得多，而且通常关心的是消除已知缺陷和实现功能。凡能使调试更像测试的事，都应该去做。说我们热爱测试有点夸张，但我们肯定讨厌调试。良好的早期单元测试以及为测试而设计，有助于把调试减到最少。

26.6 性能

程序仅仅正确还不够，还不足以有用。即使假设它具备足够的功能而称得上有用，它也必须提供适当的性能。好的程序“足够高效”，也就是说，在可用资源条件下它能在可接受的时间内运行。请注意，绝对的效率是没有意义的；而对让程序跑得快这件事的痴迷，会因把代码复杂化（导致更多缺陷和更多调试）、并使维护（包括移植和性能调优）变得更困难、更昂贵，从而严重损害开发工作。

那么，我们怎么知道一个程序（或程序中的某个单元）是否“足够高效”呢？抽象地讲，我们无法知道；而对许多程序来说，硬件足够快，这个问题根本不会出现。我们见过一些已交付的产品是用

调试模式编译的（即运行速度比必要的慢约 25 倍），目的是对部署之后发生的错误给出更好的诊断信息（即使是最好的代码，在与“别处”开发的代码共存时也会遇到这种情况）。

因此，“它足够高效吗?”这个问题的答案是：“测量有趣的测试用例要花多长时间。”要做到这一点，你显然必须足够了解最终用户，从而知道他们会认为什么算“有趣”，以及这类有趣的用法可以接受多长时间。从逻辑上说，我们只要用秒表给测试计时，并检查没有哪个测试耗费了不合理的时间。当我们使用 system_clock（§26.6.1）之类的设施替我们计时，并且能把测试所花的时间自动与“何为合理”的估计相比较时，这种做法就变得可行了。另一种做法（或者作为补充）是记录测试所花的时间，并与先前的测试运行比较。这样我们就得到了一种针对性能的回归测试。

一些最糟糕的性能缺陷由糟糕的算法引起，并且可以通过测试发现。使用大数据集进行测试的原因之一，就是让低效的算法暴露出来。举例来说，假设某个应用需要求矩阵各行的元素之和（使用第 24 章的 Matrix 库）。有人提供了一个合适的函数：

```
double row_sum(Matrix<double,2> m, int n);     // m[n] 中元素之和
```

现在有人用它来生成一个和向量，其中 v[n] 是前 n 行元素之和：

```
double row_accum(Matrix<double,2> m, int n)   // m[0:n) 中元素之和
{
    double s = 0;
    for (int i=0; i<n; ++i) s+=row_sum(m,i);
    return s;
}

// 计算 m 各行的累积和:
vector<double> v;
for (int i = 0; i<m.dim1(); ++i) v.push_back(row_accum(m,i+1));
```

你可以设想这段代码是某个单元测试的一部分，或者是在系统测试驱动应用时执行的一部分。无论哪种情况，只要矩阵真的很大，你就会注意到一件怪事：所需时间基本上随 m 的大小的平方增长。为什么？我们所做的是先加第一行的所有元素，再加第二行的所有元素（又访问了一遍第一行的全部元素），然后加第三行的所有元素（又访问了第一行和第二行的全部元素），如此等等。

如果你觉得这个例子已经很糟，那就想想：假如 row_sum() 必须访问数据库来取得数据，会是什么情形。从磁盘读取比从主存读取慢几千倍。

现在你可能会抱怨：“没人会写出这么蠢的东西!”很遗憾，我们见过糟糕得多的代码；而且从性能角度看，糟糕的算法埋在应用代码里时通常并不那么容易看出来。你第一眼扫过那段代码时，看出性能问题了吗？除非你专门去找某一类问题，否则问题可能相当难发现。下面是取自某台服务器的一个简单的真实例子：

```
for (int i=0; i<strlen(s); ++i) {
    // . . . 对 s[i] 做点什么 . . .
}
```

这里的 s 常常是一个约 2 万个字符的字符串。

并非所有性能问题都与糟糕的算法有关。事实上（正如我们在 §26.3.3 中指出的），我们写的许多代码根本不能算作严格意义上的算法。这类“非算法”的性能问题通常归入“设计不佳”这个大类别，其中包括

- 反复重新计算同样的信息（例如上面求各行元素之和的问题）
- 反复检查同一个事实（例如在循环中每次使用下标时都检查它是否在范围内，或者某个实参在函数之间原封不动地传递时反复检查它）
- 反复访问磁盘（或网络）

注意这里的“反复”是（反复出现的）反复。显然，我们指的是“不必要的反复”，但要点在于：除非某件事做了很多次，否则它不会对性能产生影响。我们完全赞成彻底检查函数实参和循环变量，但如果我们对同一批值做了一百万次同样的检查，这些多余的检查就很可能损害性能。如果我们通过测量发现性能确实受损，就会设法看看能否去掉某项重复的动作。除非你确信性能真的是个问题，否则不要这么做。过早优化是许多缺陷和大量时间浪费的根源。

26.6.1 计时

你怎么知道一段代码是否足够快？你怎么知道一次操作要花多长时间？说起来，在很多真正要紧的场合，你只要看一眼钟（秒表、挂钟或手表）就行。这既不科学也不精确，但如果用秒表计时根本不可行（因为操作太快），你往往就可以断定这个程序已经足够快了。痴迷于性能并不可取。

如果你需要测量更小的时间增量，或者无法守着秒表干等，就需要让计算机来帮忙；它知道时间，也能把时间告诉你。例如在 Unix 系统上，只要在命令前面加上 `time`，系统就会打印出所花的时间。你可以用 `time` 来弄清楚编译一个 C++ 源文件 `x.cpp` 需要多长时间。通常你这样编译：

```
g++ x.cpp
```

要给这次编译计时，只需加上 `time`：

```
time g++ x.cpp
```

这会编译 `x.cpp`，同时把所花的时间打印到屏幕上。这是给小程序计时的一种简单而有效的办法。记住每次都要多跑几遍计时，因为机器上的“其他活动”可能干扰结果。如果三次得到大致相同的结果，通常就可以相信这个结果。

但如果你想测量只花几毫秒的事情呢？如果你想自己对程序的某一部分做更细致的测量呢？你可以使用 `<chrono>` 中的标准库设施。例如，要测量函数 `do_something()` 所花的时间，可以这样写代码：

```
#include <chrono>
#include <iostream>
using namespace std;

int main()
{
    int n = 10000000;              // 把 do_something() 重复 n 次

    auto t1 = system_clock::now();         // 开始计时

    for (int i = 0; i<n; i++) do_something();    // 计时循环

    auto t2 = system_clock::now();                  // 结束计时
    cout << "do_something() " << n << " times took "
```

```
        << duration_cast<milliseconds>(t2-t1).count() << "milliseconds\n";
}
```

system_clock 是标准计时器之一，system_clock::now() 返回调用它那一刻的时间点（time_point）。两个 time_point 相减（这里是 t2-t1），就得到一个时间长度（duration）。我们可以用 auto 来省去 duration 和 time_point 类型的细节——如果你对时间的看法只是手表上看到的那样，这些类型会复杂得令人吃惊。事实上，标准库的计时设施最初是为高级物理应用设计的，比大多数用户需要的灵活、通用得多。

要得到以某个特定时间单位（如秒、毫秒或纳秒）表示的时长，我们用转换函数 duration_cast 把它转换（“cast”）成该单位。之所以需要 duration_cast 之类的东西，是因为不同的系统和不同的时钟用不同的单位计量时间。别忘了 .count()。正是它从既包含时钟滴答数、又包含其单位的 duration 中提取出单位个数（“时钟滴答数”）。

system_clock 适合测量从几分之一秒到几秒的间隔。不要试图用它来测量小时级的时长。

再说一遍：凡是不能重复三次得到大致相同结果的时间测量，都不要相信。“大致相同”是什么意思？“误差在 10% 以内”是个合理的答案。记住现代计算机很快：每秒十亿条指令是寻常事。这意味着，除非你能把某件事重复几万次，或者它做的事真的很慢（比如写磁盘、访问网络），否则你根本测不出任何东西。对后一种情况，你只需让它重复几百次，但你要担心的是：同时发生的事情太多，你可能看不懂结果。

26.7 参考文献

- Stone, Debbie, Caroline Jarrett, Mark Woodroffe, Shailey Minocha. *User Interface Design and Evaluation*（《用户界面设计与评估》）. Morgan Kaufmann, 2005. ISBN 0120884364.
- Whittaker, James A. *How to Break Software: A Practical Guide to Testing*（《如何攻破软件：实用测试指南》）. Addison-Wesley, 2003. ISBN 0321194330.

小试身手

让 binary_search 的测试跑起来：

1. 为 §26.3.2.2 中的 Test 实现输入运算符。
2. 为 §26.3 中的各个序列补全一个测试文件：
   (a). { 1 2 3 5 8 13 21 } 普通序列
   (b). { }
   (c). { 1 }
   (d). { 1 2 3 4 } 元素个数为偶数
   (e). { 1 2 3 4 5 } 元素个数为奇数
   (f). { 1 1 1 1 1 1 1 } 所有元素相等
   (g). { 0 1 1 1 1 1 1 1 1 1 1 1 1 1 } 起始处元素不同
   (h). { 0 0 0 0 0 0 0 0 0 0 0 0 0 1 } 末尾元素不同
3. 参照 §26.3.1.3，补全一个生成如下序列的程序：
   (a). 一个非常大的序列（你认为多大算非常大？为什么？）
   (b). 十个元素个数随机的序列
   (c). 十个元素个数分别为 0、1、2……9 的随机序列（但仍保持有序）

4. 对字符串序列重复这些测试，例如 { Bohr Darwin Einstein Lavoisier Newton Turing }。

要点回顾

1. 列出一些应用，并简要说明若其中出现缺陷，最坏可能发生什么；例如飞机控制——坠毁：231 人死亡，损失价值 5 亿美元的设备。
2. 我们为什么不直接证明自己的程序正确？
3. 单元测试与系统测试有什么区别？
4. 什么是回归测试，它为什么重要？
5. 测试的目的是什么？
6. `binary_search` 为什么不直接检查自己的要求？
7. 如果我们无法检查所有可能的错误，那么主要该查找哪些种类的错误？
8. 在操作元素序列的代码中，缺陷最可能出现在哪里？
9. 为什么测试大的数值是个好主意？
10. 为什么我们常常把测试表示为数据而不是代码？
11. 我们为什么以及在什么时候会使用大量基于随机值的测试？
12. 为什么用图形用户界面（GUI）的程序难以测试？
13. 要孤立地测试一个“单元”需要什么？
14. 可测试性与可移植性之间有什么联系？
15. 为什么测试类比测试函数更难？
16. 测试可重复为什么很重要？
17. 当测试者发现某个“单元”依赖未检查的假设（前置条件）时，他能做什么？
18. 设计者或实现者可以做什么来改进测试？
19. 测试与调试有何不同？
20. 性能什么时候重要？
21. 举出两个（或更多）例子，说明如何（轻易地）造成糟糕的性能问题。

术语

- assumptions（假设）
- black-box testing（黑盒测试）
- branching（分支）
- design for testing（为测试而设计）
- inputs（输入）
- outputs（输出）
- post-condition（后置条件）
- pre-condition（前置条件）
- proof（证明）
- regression（回归）
- resource usage（资源使用）
- state（状态）
- `system_clock`（系统时钟）
- system test（系统测试）
- test coverage（测试覆盖率）

- test harness（测试框架）
- testing（测试）
- timing（计时）
- unit test（单元测试）
- white-box testing（白盒测试）

习题

1. 用 §26.3.2.1 给出的测试运行你在 §26.1 中写的二分查找算法。
2. 修改 `binary_search` 的测试，使它能处理任意元素类型。然后用字符串序列和浮点序列测试它。
3. 对接受比较准则的那个 `binary_search` 版本重做习题 1。列出这个额外实参带来的、可能引入错误的新机会。
4. 设计一种测试数据格式，使你只定义一次序列就能对它运行多个测试。
5. 在 `binary_search` 的测试集合中增加一个测试，设法抓住（不太可能发生的）`binary_search` 修改了序列这种错误。
6. 对第 7 章的计算器做最小改动，让它从文件读入、把输出写到文件（或者使用操作系统重定向 I/O 的设施）。然后为它设计一套相当全面的测试。
7. 测试 §20.6 中的“简单文本编辑器”。
8. 为第 12～15 章的图形界面库增加一个基于文本的接口。例如，字符串 `Circle(Point(0,1),15)` 应生成一次对 `Circle(Point(0,1),15)` 的调用。用这个文本接口画一幅“儿童画”：一座带屋顶、两扇窗和一扇门的二维房子。
9. 为图形界面库增加一种基于文本的输出格式。例如，当执行调用 `Circle(Point(0,1),15)` 时，应在某个输出流上产生 `Circle(Point(0,1),15)` 这样的字符串。
10. 用习题 9 中的文本接口为图形界面库编写一个更好的测试。
11. 对 §26.6 中的求和示例计时，分别取维数为 100、10,000、1,000,000 和 10,000,000 的方阵 `m`。元素值用 [−10:10) 范围内的随机值。重写 `v` 的计算，改用更高效（不是 $O(N^2)$）的算法，并比较计时结果。
12. 写一个程序，生成随机浮点数并用 `std::sort()` 排序。测量排序 500,000 个 `double` 和 5,000,000 个 `double` 所用的时间。
13. 重复上一习题的实验，但改用长度在 [0:100) 范围内的随机字符串。
14. 重复上一习题，但改用 `map` 而不是 `vector`，这样就不必排序。

后记

作为程序员，我们梦想写出漂亮、直接就能跑的程序——最好第一次尝试就成功。现实并非如此：把程序弄对很难，而在我们（以及同事）不断改进它们的过程中让它们保持正确也很难。测试——包括为测试而设计——是确保我们交付的系统真正能工作的主要手段。在这个技术高度发达的世界里，每当一天结束，我们都真该在心里感念一下那些（常常被遗忘的）测试人员。

27 C 程序设计语言

“C is a strongly typed,
weakly checked,
programming language.
（*C* 是一门强类型、弱检查的编程语言。）”
——丹尼斯·里奇（Dennis Ritchie）

本章从熟悉 C++ 的读者视角出发，简要概述 C 语言及其标准库。文中列出 C 中缺少的 C++ 特性，并举例说明 C 程序员在没有这些特性时如何应对。本章还介绍 C 与 C++ 不兼容之处，并讨论二者的互操作。为便于说明，文中给出 I/O、链表操作、内存管理和字符串处理方面的示例。

27.1 C 与 C++：兄弟
27.1.1 C/C++ 兼容性
27.1.2 C 中缺少的 C++ 特性
27.1.3 C 标准库
27.2 函数
27.2.1 没有函数名重载
27.2.2 函数实参类型检查
27.2.3 函数定义
27.2.4 从 C++ 调用 C 与从 C 调用 C++
27.2.5 函数指针
27.3 细微的语言差异
27.3.1 struct 标记命名空间
27.3.2 关键字
27.3.3 定义
27.3.4 C 风格强制转换
27.3.5 void* 的转换
27.3.6 enum
27.3.7 命名空间
27.4 自由存储
27.5 C 风格字符串
27.5.1 C 风格字符串与 const
27.5.2 字节操作
27.5.3 示例：strcpy()
27.5.4 一个风格问题
27.6 输入/输出：stdio
27.6.1 输出
27.6.2 输入
27.6.3 文件
27.7 常量与宏
27.8 宏
27.8.1 类函数宏
27.8.2 语法宏
27.8.3 条件编译
27.9 示例：侵入式容器

27.1 C 与 C++：兄弟

C 语言由丹尼斯·里奇（Dennis Ritchie）在贝尔实验室设计并实现，并因布莱恩·克尼汉（Brian Kernighan）与里奇合著的《C 程序设计语言》（The C Programming Language，俗称“K&R”）一书而广为流行；可以说，这本书至今仍是最好的 C 语言入门读物，也是编程领域的名著之一（§22.2.5）。C++ 最初的定义文本，是在里奇提供的 1980 年版 C 语言定义文本基础上改写而成的。经过最初这次分家之后，两种语言各自继续演化。与 C++ 一样，C 如今也由 ISO 标准定义。

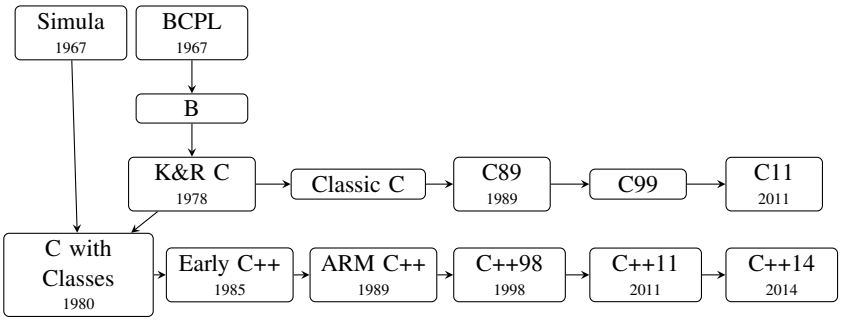
我们把 C 基本上视为 C++ 的一个子集。因此，从 C++ 的角度看，描述 C 的问题可以归结为两点：

- 说明 C 在哪些地方不是 C++ 的子集。
- 说明 C 中缺少哪些 C++ 特性，以及可以用哪些设施和技术来弥补。

从历史上看，现代 C 与现代 C++ 是兄弟。二者都是“经典 C”（Classic C）的直系后代；这个 C 方言因克尼汉与里奇的《C 程序设计语言》第一版而流行，后来又加上了结构体赋值与枚举。

今天使用的 C 版本在很大程度上仍是 C89（也就是 K&R 第二版所描述的那个版本），本章介绍的也正是它。目前仍有一些地方在使用经典 C，也有一些在使用 C99，但在你掌握了 C++ 和 C89 之后，这些都不该给你带来麻烦。

C 与 C++ 都“诞生”于新泽西州默里山（Murray Hill）的贝尔实验室计算机科学研究中心（有一段时间，我的办公室与丹尼斯·里奇、布莱恩·克尼汉的办公室只隔几个门，斜对着走廊）：



两种语言现在都由 ISO 标准委员会定义和控制。两者都各自有许多在用的受支持实现。通常，一个实现同时支持两种语言，具体使用哪一种由编译器开关或源文件后缀决定。两者可用的平台数量都超过其他任何语言。两者最初都是为主要面向硬系统编程任务而设计，如今也仍大量用于这类任务，例如：

- 操作系统内核
- 设备驱动程序
- 嵌入式系统
- 编译器
- 通信系统

等价的 C 程序与 C++ 程序之间没有性能差异。

与 C++ 一样，C 的使用极为广泛。合在一起，C/C++ 社区是地球上最大的软件开发社区。

27.1.1 C/C++ 兼容性

常能听到人们提到“C/C++”。然而并不存在这样一门语言，使用“C/C++”这种说法通常是无知的表现。我们只在讨论 C/C++ 兼容性问题、以及谈论庞大的 C/C++ 共享技术社区时，才使用“C/C++”。

C++ 在很大程度上是 C 的超集，但并不完全是。只有极少数例外：既属于 C 又属于 C++ 的构造，在两种语言中含义（语义）相同。C++ 的设计目标是“尽可能接近 C，但不更近”：

- 便于迁移
- 便于共存

大多数不兼容之处都与 C++ 更严格的类型检查有关。

一个合法 C 程序但不是合法 C++ 程序的例子，是把 C++ 关键字（而非 C 关键字）用作标识符（见 §27.3.2）：

```
int class(int new, int bool);        /* C 合法，C++ 不合法 */
```

而某个构造在两种语言中都合法、语义却不同的例子比较难找，下面就是一个：

```
int s = sizeof('a');                          /* sizeof(int)：C 中通常为 4，C++ 中为 1 */
```

字符字面量（如 `'a'`）的类型在 C 中是 `int`，在 C++ 中是 `char`。不过，对于 `char` 变量 `ch`，两种语言中都有 `sizeof(ch)==1`。

与兼容性和语言差异有关的信息并不怎么令人兴奋，也没有什么新的编程技巧可学。你也许会喜欢 `printf()`（§27.6），但除了这个可能的例外（以及几处蹩脚的极客式幽默），本章枯燥至极。它的目的很简单：让你在需要时能读、能写 C。这包括指出那些对经验丰富的 C 程序员显而易见、而 C++ 程序员通常想不到的陷阱。我们希望你能以最小的痛苦学会避开这些陷阱。

大多数 C++ 程序员迟早都要处理 C 代码，正如大多数 C 程序员迟早都要处理 C++ 代码。本章介绍的内容对大多数 C 程序员来说并不陌生，但其中有些会被视为“专家级”。原因很简单：并非每个人对什么是“专家级”都有一致看法，我们只是描述真实代码中常见的做法。也许弄懂兼容性问题，可以成为一条廉价的捷径，让你凭空赢得“C 专家”的名声。但请记住：真正的专业能力在于用好一门语言（这里是 C），而不在于记住那些晦涩的语言规则（考察兼容性问题时暴露出来的正是这类规则）。

参考文献

- ISO/IEC 9899:1999. *Programming Languages —C*. 该标准定义 C99；多数实现实现的是 C89（通常带少量扩展）。
- ISO/IEC 9899:2011. *Programming Languages —C*. 该标准定义 C11。
- ISO/IEC 14882:2011. *Programming Languages —C++*.
- Kernighan, Brian W., and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 1988. ISBN 0131103628.
- Stroustrup, Bjarne. “Learning Standard C++ as a New Language.” *C/C++ Users Journal*, 1999 年 5 月。
- Stroustrup, Bjarne. “C and C++: Siblings”；“C and C++: A Case for Compatibility”；“C and C++: Case Studies in Compatibility.” *The C/C++ Users Journal*, 2002 年 7 月、8 月、9 月。

Stroustrup 的论文最容易在我的出版目录主页上找到。

27.1.2 C 中缺少的 C++ 特性

从 C++ 的角度看，C（即 C89）缺少很多特性，例如：

- 类与成员函数：改用 `struct` 与全局函数。
- 派生类与虚函数：改用 `struct`、全局函数和函数指针（§27.2.3）。
- 模板与内联函数：改用宏（§27.8）。
- 异常：改用错误码、错误返回值等。
- 函数重载：给每个函数起一个不同的名字。
- `new/delete`：改用 `malloc()/free()`，并把初始化与清理的代码分开写。
- 引用：改用指针。
- `const`、`constexpr`，或在常量表达式中使用函数：改用宏。
- `bool`：改用 `int`。

- `static_cast`、`reinterpret_cast`、`const_cast`：改用 C 风格强制转换，例如用 `(int)a` 而不是 `static_cast<int>(a)`。

大量有用的代码都是用 C 写的，所以这份清单应当提醒我们：没有任何一项语言特性是绝对必需的。大多数语言特性——甚至大多数 C 语言特性——只是为了给程序员提供方便（仅此而已）。毕竟，只要有足够的时间、聪明才智和耐心，任何程序都可以用汇编语言写出来。请注意，由于 C 与 C++ 共享一个非常接近真实机器的机器模型，它们很适合模拟各种编程风格。

本章余下的部分将说明，在没有这些特性的情况下如何编写有用的程序。我们使用 C 的基本建议是：

- 用 C 提供的设施，去模拟那些 C++ 特性本来要支持的编程技术。
- 写 C 时，就写在 C++ 的 C 子集里。
- 使用能确保函数实参检查的编译器警告级别。
- 对于大型程序，使用 lint（见 §27.2.2）。

C/C++ 不兼容的许多细节相当晦涩，技术性也很强。不过，要读写 C，你其实不必记住其中大部分：

- 当你使用了 C 中没有的 C++ 特性时，编译器会提醒你。
- 如果你遵守上面这些规则，就不太会遇到在 C 与 C++ 中含义不同的东西。

在所有这些 C++ 设施都缺席的情况下，有些设施在 C 中反而变得更重要：

- 数组与指针
- 宏
- `typedef`（C 与 C++98 中相当于简单 using 声明的设施；见 §20.5、§A.16）
- `sizeof`
- 强制转换

本章会给出这类用法的一些例子。

`//` 注释是我从 C 的祖先 BCPL 引入 C++ 的，当时我实在受够了敲 `/* . . . */` 注释。大多数 C 方言（包括 C99 和 C11）都接受 `//` 注释，所以放心用大概没问题。不过，在本章那些意在作为 C 的示例里，我们只使用 `/* . . . */` 注释。C99 和 C11 又引入了一些 C++ 特性（也引入了几项与 C++ 不兼容的特性），但这里我们坚持用 C89，因为它使用得广泛得多。

27.1.3 C 标准库

自然，依赖类与模板的 C++ 标准库设施在 C 中是没有的，包括：

- `vector`
- `map`
- `set`
- `string`
- STL 算法：例如 `sort()`、`find()`、`copy()`
- `iostreams`
- `regex`

针对这些情况，通常会有一些基于数组、指针和函数的 C 库以供使用，算是一种补偿。C 标准库的主要部分包括：

- `<stdlib.h>`：通用工具（例如 `malloc()` 和 `free()`；见 §27.4）

- `<stdio.h>`：标准 I/O；见 §27.6
- `<string.h>`：C 风格字符串操作与内存操作；见 §27.5
- `<math.h>`：标准浮点数学函数；见 §24.8
- `<errno.h>`：供 `<math.h>` 使用的错误码；见 §24.8
- `<limits.h>`：各整数类型的长度；见 §24.2
- `<time.h>`：日期与时间；见 §26.6.1
- `<assert.h>`：调试断言；见 §27.9
- `<ctype.h>`：字符分类；见 §11.6
- `<stdbool.h>`：布尔宏

完整说明请参阅一本好的 C 教材，例如 K&R。所有这些库（以及头文件）在 C++ 中也都可用。

27.2 函数

在 C 中：

- 同一个名字只能有一个函数。
- 函数参数类型检查是可选的。
- 没有引用（因此也没有按引用传递）。
- 没有成员函数。
- 没有内联函数（C99 除外）。
- 还有另一种函数定义语法。

除此之外，情况与你在 C++ 中习惯的差不太多。下面我们来看看这意味着什么。

27.2.1 不允许函数重载

考虑下面这段代码：

```
void print(int);                        /* 打印一个 int */
void print(const char*);                /* 打印一个字符串 */   /* 错误！ */
```

第二个声明是错误的，因为不能有两个同名的函数。所以你只好自己想出一对合适的名字：

```
void print_int(int);                    /* 打印一个 int */
void print_string(const char*);         /* 打印一个字符串 */
```

有人偶尔把这一点说成优点：现在你不会误用错误的函数去打印一个 `int` 了！我们显然不接受这种说法；而且缺少重载函数确实让泛型编程的想法难以实现，因为泛型编程依赖于语义相似的函数具有相同的名字。

27.2.2 函数参数类型检查

考虑下面这段代码：

```
int main()
{
    f(2);
}
```

C 编译器会接受这段代码：调用一个函数之前，你不必先声明它（不过你可以也应该声明）。某处可能存在 f() 的一个定义。那个 f() 也可能在另一个翻译单元里；但如果哪儿都没有，链接器就会报错。

遗憾的是，另一个源文件里的那个定义可能是这样：

```
/* other_file.c: */
int f(char* p)
{
    int r = 0;
    while (*p++) r++;
    return r;
}
```

链接器不会报告这个错误。你会得到运行时错误，或者某个莫名其妙的结果。

怎样处理这类问题呢？一个实用的答案是：始终如一地使用头文件。如果你调用或定义的每个函数都在某个头文件里声明，并且需要时总是把这个头文件用 #include 包含进来，我们就能得到检查。不过在大型程序里，这一点很难做到。因此，大多数 C 编译器都有一些选项，能对调用未声明的函数给出警告：把它们打开。另外，从 C 诞生之初就有一些可用于检查各种一致性问题的程序，它们通常称为 lint。对每个不那么简单的 C 程序都用一用 lint。你会发现，lint 会把你推向一种与使用 C++ 子集相当接近的 C 用法。促使 C++ 设计出来的一个观察结果是：编译器可以轻松检查 lint 所检查的很大一部分内容（但不是全部）。

在 C 中，你也可以要求检查函数参数。做法很简单：声明函数时把参数类型写出来（就跟 C++ 一样）。这样的声明称为函数原型（function prototype）。但是要提防那些没有指定参数的函数声明：它们不是函数原型，也不意味着会做函数参数检查：

```
int g(double);                  /* 原型——类似 C++ 的函数声明 */
int h();                        /* 不是原型——参数类型未指定 */


void my_fct()
{
    g();                    /* 错误：缺少参数 */
    g("asdf");              /* 错误：参数类型不对 */
    g(2);                   /* 没问题：2 会转换成 2.0 */
    g(2,3);                 /* 错误：多了一个参数 */
    h();                    /* 编译器认为没问题！结果可能出乎意料 */
    h("asdf");              /* 编译器认为没问题！结果可能出乎意料 */
    h(2);                   /* 编译器认为没问题！结果可能出乎意料 */
    h(2,3);                 /* 编译器认为没问题！结果可能出乎意料 */
}
```

h() 的声明没有指定任何参数类型。这并不意味着 h() 不接受参数；它的意思是“接受任意一组参数，并希望它们对被调用的那个函数来说是正确的”。再说一遍：好的编译器会给出警告，lint 会抓住这个问题。

C++	C 中的等价写法
`void f(); //首选`	`void f(void);`
`void f(void);`	`void f(void);`
`void f(. . .); //接受任意参数`	`void f(); /* 接受任意参数 */`

对于没有函数原型位于当前作用域内的情形，有一套特殊的参数转换规则。例如，char 和 short 会转换成 int，float 会转换成 double。如果你需要知道（比方说）long 会怎样，请到一本好的 C 教材里查。我们的建议很简单：不要调用没有原型的函数。

请注意，尽管编译器会允许传入类型错误的参数（例如给 int 类型的形参传一个 char*），但使用这样一个类型错误的参数仍然是一个错误。正如 Dennis Ritchie 所说：“C 是一门强类型、弱检查的编程语言。”

27.2.3 函数定义

定义函数的方式与 C++ 完全一样，而且这样的定义本身就是函数原型：

```
double square(double d)
{
    return d*d;
}


void ff()
{
    double x = square(2);              /* 没问题：把 2 转换成 2.0 再调用 */
    double y = square();               /* 缺少参数 */
    double y = square("Hello");        /* 错误：参数类型不对 */
    double y = square(2,3);            /* 错误：参数太多 */
}
```

不带参数的函数定义并不是函数原型：

```
void f() { /* 做点什么 */ }


void g()
{
    f(2);       /* 在 C 中没问题；在 C++ 中是错误 */
}
```

如果把下面这个声明

```
void f();          /* 未指定参数类型 */
```

读成“f() 可以接受任意数量、任意类型的参数”，那在我看来非常奇怪。作为回应，我发明了一种新写法，用关键字 void 把“什么都没有”明确写出来（void 是一个四个字母的单词，意思是“没有”）：

```
void f(void);  /* 不接受任何参数 */
```

不过我很快就后悔了，因为这种写法看起来古怪，而且在统一进行参数类型检查之后完全是多余的。更糟的是，Dennis Ritchie（C 之父）和 Doug McIlroy（贝尔实验室计算机科学研究中心的最高审美仲裁者；见 §22.2.5）都说它是“一桩丑事”。不幸的是，这桩丑事在 C 社区里变得非常流行。不过在 C++ 中不要用它，那里它不仅难看，而且在逻辑上也是多余的。

C 还提供了第二种、Algol 60 风格的函数定义，其中参数类型（可选地）与参数名分开指定：

```
int old_style(p,b,x) char* p; char b;
{
    /* . . . */
}
```

这种“旧式定义”比 C++ 还早，它不是函数原型。默认情况下，没有声明类型的参数就是 int。所以 x 是 old_style() 的一个 int 参数。我们可以这样调用 old_style()：

```
old_style();                          /* 没问题：所有参数都没有 */
old_style("hello", 'a', 17);          /* 没问题：所有参数类型都对 */
old_style(12, 13, 14);                /* 没问题：12 的类型不对， */
                                      /* 但也许 old_style() 根本不用 p */
```

编译器应当接受这些调用（不过我们希望它对第一个和第三个给出警告）。

我们关于函数参数检查的建议是：

- 始终如一地使用函数原型（使用头文件）。
- 调整编译器的警告级别，使参数类型错误能被抓住。
- 使用（一些）lint。

这样得到的代码同时也是 C++ 代码。

27.2.4 从 C++ 调用 C，也从 C 调用 C++

只要两个编译器本来就是为此设计的，你就可以把用 C 编译器编译出的文件与用 C++ 编译器编译出的文件链接在一起。例如，你可以用 GNU 的 C 和 C++ 编译器（GCC），把分别由 C 和 C++ 生成的目标文件链接在一起。你也可以用微软的 C 和 C++ 编译器（MSC++），把分别由 C 和 C++ 生成的目标文件链接在一起。这种做法很常见、也很有用，因为它让你能使用的库比仅用这两种语言中的一种所能得到的要多得多。

C++ 提供的类型检查比 C 更严格。特别是，C++ 的编译器和链接器会检查 f(int) 与 f(double) 这两个函数的定义和使用是否一致——即使它们位于不同的源文件中。C 的链接器不做这类检查。要从 C++ 调用一个用 C 定义的函数，并且让一个用 C++ 定义的函数能被 C 调用，我们需要告诉编译器自己在做什么：

```
// 从 C++ 调用 C 函数:
extern "C" double sqrt(double);   // 按 C 函数的约定来链接

void my_c_plus_plus_fct()
{
    double sr = sqrt(2);
}
```

基本上，extern "C" 是告诉编译器使用 C 的链接约定。除此之外，从 C++ 的角度看一切照常。事实上，C++ 标准库里的 sqrt(double) 通常就是 C 标准库里的 sqrt(double)。要让一个函数能以这种方式从 C++ 调用，C 程序这边不需要做任何事。C++ 只是去适应 C 的链接约定。

我们也可以借助 extern "C" 让一个 C++ 函数能被 C 调用：

```
// 可被 C 调用的 C++ 函数:
extern "C" int call_f(S* p, int i)
{
    return p->f(i);
}
```

在 C 程序里，我们现在可以像这样间接调用成员函数 f()：

```
/* 从 C 调用 C++ 函数:  */
int call_f(S* p, int i);
struct S* make_S(int,const char*);

void my_c_fct(int i)
{
    /* . . . */
    struct S* p = make_S(x, "foo");
    int x = call_f(p,i);
    /* . . . */
}
```

为此，C 这边既不需要、也不可能提到 C++。

这种互操作带来的好处是显而易见的：代码可以混合使用 C 和 C++ 来编写。特别是，C++ 程序可以使用用 C 写成的库，而 C 程序也可以使用用 C++ 写成的库。此外，大多数语言（尤其是 Fortran）都有与 C 互相调用的接口。

在上面的例子里，我们假定 C 和 C++ 可以共享 p 所指向的那个类对象。对大多数类对象来说这确实成立。特别是，如果你有这样一个类：

```
// 在 C++ 中:
class complex {
    double re, im;
public:
    // 所有常规操作
};
```

你就可以把指向对象的指针在 C 与 C++ 之间来回传递。你甚至可以在 C 程序里用一个声明来访问 `re` 和 `im`：

```
/* 在 C 中: */
struct complex {
    double re, im;
    /* 没有任何操作 */
};
```

任何语言内部关于布局的规则都可能很复杂，而跨语言布局的规则甚至难以明确规定。不过，你可以在 C 和 C++ 之间传递内置类型，也可以传递（不含虚函数的）类（结构体）。如果一个类含有虚函数，你只应传递指向其对象的指针，把实际的操作留给 C++ 代码。`call_f()` 就是这样一个例子：`f()` 可能是虚函数，那样的话这个例子就演示了如何从 C 调用虚函数。

除了坚持使用内置类型之外，最简单也最安全的类型共享方式，是在一个 C/C++ 共用的头文件里定义一个 struct。不过这种策略严重限制了 C++ 能被怎样使用，所以我们并不把自己局限于此。

27.2.5 指向函数的指针

如果我们在 C 里想用面向对象的技术（§14.2-4），能做些什么呢？基本上，我们需要一个虚函数的替代品。对大多数人来说，最先想到的办法是使用一个带“类型字段”的 struct，由这个字段描述给定对象表示的是哪种形状。例如：

```
struct Shape1 {
    enum Kind { circle, rectangle } kind;
    /* . . . */
};

void draw(struct Shape1* p)
{
    switch (p->kind) {
    case circle:
        /* 按圆形绘制 */
        break;
    case rectangle:
        /* 按矩形绘制 */
        break;
    }
}
```

```
int f(struct Shape1* pp)
{
    draw(pp);
    /* . . . */
}
```

这样能行。不过有两个麻烦：

- 对每个“伪虚”函数（例如 `draw()`），我们都得写一个新的 switch 语句。
- 每当我们添加一种新形状，都得修改每一个“伪虚”函数（例如 `draw()`），在 switch 语句里加上一个 case 分支。

第二个问题相当麻烦，因为它意味着我们无法把“伪虚”函数作为库的一部分提供出去，因为用户不得不经常修改这些函数。最有效的替代办法要用到指向函数的指针：

```
typedef void (*Pfct0)(struct Shape2*);
typedef void (*Pfct1int)(struct Shape2*,int);

struct Shape2 {
    Pfct0 draw;
    Pfct1int rotate;
    /* . . . */
};


void draw(struct Shape2* p)
{
    (p->draw)(p);
}

void rotate(struct Shape2* p, int d)
{
    (p->rotate)(p,d);
}
```

这个 Shape2 可以像 Shape1 一样使用。

```
int f(struct Shape2* pp)
{
    draw(pp);
    /* . . . */
}
```

稍微多做一点工作，对象就不必为每个“伪虚函数”各保存一个函数指针。它也可以保存一个指针，指向一个由函数指针组成的数组（C++ 中的虚函数差不多就是这样实现的）。在真实程序里使用这类方案，主要问题在于把所有那些函数指针的初始化都做对。

27.3 次要的语言差异

本节举出一些细小的 C/C++ 差异；如果你从没听说过它们，就可能被绊倒。这些差异很少会严重影响编程，因为一般都有显而易见的变通办法。

27.3.1 struct 标记的名字空间

在 C 中，结构体的名字（C 里没有 class 关键字）与其他标识符处在不同的名字空间里。因此，结构体的每个名字（称为结构体标记，structure tag）前面都必须加上关键字 struct。例如：

```
struct pair { int x,y; };
pair p1;                    /* 错误：作用域中没有标识符 pair */
struct pair p2;             /* 正确 */
int pair = 7;               /* 正确：结构体标记 pair 不在此作用域中 */
struct pair p3;             /* 正确：结构体标记 pair 没有被那个 int 遮蔽 */
pair = 8;                   /* 正确：pair 指的是那个 int */
```

说来令人惊讶，多亏一个狡猾的兼容性手法，这在 C++ 中也能成立。让变量（或函数）与结构体同名，是 C 中相当常见的一种惯用法，不过我们并不推荐。

如果你不想在每个结构体名字前面都写 struct，可以用 typedef（§20.5）。下面这种惯用法很常见：

```
typedef struct { int x,y; } pair;
pair p1 = { 1, 2 };
```

总的来说，在 C 中 typedef 更常见、也更有用，因为在 C 里你没有“定义带相关操作的新类型”这个选项。

在 C 中，嵌套结构体的名字被放在嵌套它的那个结构体所在的作用域里。例如：

```
struct S {
    struct T { /* . . . */ };
    /* . . . */
};

struct T x;     /* 在 C 中正确（在 C++ 中不正确） */
```

在 C++ 中，你会写成

```
S::T x;             // 在 C++ 中正确（在 C 中不正确）
```

只要可能，就不要在 C 里嵌套结构体：它的作用域规则与大多数人天真（而且合理）的预期并不一样。

27.3.2 关键字

C++ 中的许多关键字在 C 里并不是关键字（因为 C 不提供相应功能），在 C 中可以当作标识符使用：

C++ 关键字，它们在 C 里不是关键字

alignas	class	inline	private	true
alignof	compl	mutable	protected	try
and	concept	namespace	public	typeid
and_eq	const_cast	new	reinterpret_cast	typename
asm	constexpr	noexcept	requires	using
bitand	delete	not	static_assert	virtual
bitor	dynamic_cast	not_eq	static_cast	wchar_t
bool	explicit	nullptr	template	xor
catch	export	operator	this	xor_eq
char16_t	false	or	thread_local	
char32_t	friend	or_eq	throw	

在 C 中不要把这些名字用作标识符，否则你的代码无法移植到 C++。如果你在某个头文件里用了其中一个名字，这个头文件就无法从 C++ 使用。

有些 C++ 关键字在 C 中是宏：

C++ 关键字，它们在 C 中是宏

and	bitor	false	or	wchar_t
and_eq	bool	not	or_eq	xor
bitand	compl	not_eq	true	xor_eq

在 C 中，它们定义在 `<iso646.h>` 和 `<stdbool.h>` 里（`bool`、`true`、`false`）。不要利用“它们在 C 中是宏”这一点。

27.3.3 定义

C++ 允许的定义位置比 C89 多。例如：

```
for (int i = 0; i<max; ++i) x[i] = y[i];        // 在 C 中不允许定义 i

while (struct S* p = next(q)) {                 // 在 C 中不允许定义 p
    /* . . . */
}

void f(int i)
{
    if (i< 0 || max<=i) error("range error");
    int a[max];           // 错误：在 C 中不允许语句之后再声明
    /* . . . */
}
```

C（C89）不允许在 `for` 语句的初始化部分声明变量，不允许在条件中声明，也不允许在块内某条语句之后再声明。我们只好写成类似这样：

```
int i;
for (i = 0; i<max; ++i) x[i] = y[i];

struct S* p;
while (p = next(q)) {
    /* . . . */
}

void f(int i)
{
    if (i< 0 || max<=i) error("range error");
    {
        int a[max];
        /* . . . */
    }
}
```

在 C++ 中，未初始化的声明就是定义；在 C 中，它只是声明，所以同一个名字可以出现两次：

```
int x;
int x;          /* 在 C 中定义或声明一个名为 x 的整数；在 C++ 中错误 */
```

在 C++ 中，一个实体必须恰好定义一次。如果这两个 `int` 位于不同的翻译单元，事情就更有意思了：

```
/* 在文件 x.c 中： */
int x;

/* 在文件 y.c 中： */
int x;
```

无论是 C 还是 C++ 编译器，都不会从 x.c 或 y.c 中挑出毛病。但是，如果把 x.c 和 y.c 当作 C++ 编译，链接器会报“重复定义”错误。如果把 x.c 和 y.c 当作 C 编译，链接器会接受这个程序，并（按照 C 的规则，这是正确的）认为只存在一个 x，由 x.c 和 y.c 中的代码共享。如果你想要一个共享全局变量 x 的程序，就要显式说明：

```
/* 在文件 x.c 中： */
int x = 0;                    /* 定义 */

/* 在文件 y.c 中： */
extern int x;                /* 声明，不是定义 */
```

更好的做法是使用头文件：

```
/* 在文件 x.h 中： */
extern int x;                 /* 声明，不是定义 */

/* 在文件 x.c 中： */
#include "x.h"
int x = 0;                    /* 定义 */

/* 在文件 y.c 中： */
#include "x.h"
/* x 的声明在头文件中 */
```

更好的做法是：避免使用全局变量。

27.3.4 C 风格的强制类型转换

在 C（以及 C++）中，可以用下面这种最简记号把一个值 v 显式转换成类型 T：

```
(T)v
```

这种“C 风格转换”或“旧式转换”受到打字差的人和思维马虎的人的喜爱，因为它最简短，而且你不需要知道从 v 造出一个 T 需要做些什么。另一方面，维护程序员有理由害怕这种风格的转换，因为它几乎看不出来，对写代码者的意图不留下任何线索。C++ 的转换（具名转换或模板风格转换，见 §A.5.7）正是为了让显式类型转换容易发现（显得难看）并且具体。在 C 中你别无选择：

```
int* p = (int*)7;           /* 重新解释位模式：reinterpret_cast<int*>(7) */
int x = (int)7.5;           /* 截断 double：static_cast<int>(7.5) */
```

```
typedef struct S1 { /* . . . */ } S1;
typedef struct S2 { /* . . . */ } S2;
S2 a;
const S2 b;             /* C 中允许未初始化的 const */

S1* p = (S1*)&a;     /* 重新解释位模式：reinterpret_cast<S1*>(&a) */
S2* q = (S2*)&b;     /* 去掉 const：const_cast<S2*>(&b) */
S1* r = (S1*)&b;     /* 去掉 const 并改变类型；很可能是 bug */
```

即使在 C 中，我们也犹豫是否推荐宏（§27.8），但用下面的方式表达意图也许是个好主意：

```
#define REINTERPRET_CAST(T,v) ((T)(v))
#define CONST_CAST(T,v) ((T)(v))

S1* p = REINTERPRET_CAST (S1*,&a);
S2* q = CONST_CAST(S2*,&b);
```

这不会带来 `reinterpret_cast` 和 `const_cast` 所做的类型检查，但它确实让这些天生难看的操作变得显眼，也让程序员的意图变得明确。

27.3.5 `void*` 的转换

在 C 中，`void*` 可以作为赋值或初始化的右操作数，用来给任何指针类型的变量赋值或初始化；在 C++ 中则不行。例如：

```
void* alloc(size_t x);          /* 分配 x 个字节 */

void f (int n)
{
    int* p = alloc(n*sizeof(int));
    /* 在 C 中正确；在 C++ 中错误 */
    /* . . . */
}
```

这里，`alloc()` 返回的 `void*` 被隐式转换成 `int*`。在 C++ 中，我们必须把那一行改写成

```
int* p = (int*)alloc(n*sizeof(int));     /* 在 C 和 C++ 中都正确 */
```

我们用了 C 风格的转换（§27.3.4），这样它在 C 和 C++ 中都是合法的。

为什么 `void*` 到 `T*` 的隐式转换在 C++ 中不合法？因为这类转换可能不安全：

```
void f()
{
    char i = 0;
    char j = 0;
    char* p = &i;
    void* q = p;
    int* pp = q;              /* 不安全；在 C 中合法，在 C++ 中错误 */
    *pp = -1;                 /* 从 &i 开始覆写内存 */
}
```

这里我们甚至无法确定被覆写的是哪块内存。也许是 j 和 p 的一部分？也许是为管理 f() 调用而使用的某些内存（f 的栈帧）？无论这里覆写的是什么数据，调用 f() 都是坏消息。

注意，（相反方向的）T* 到 void* 的转换是完全安全的——你无法为它构造出上面那样的恶劣例子——而且这种转换在 C 和 C++ 中都允许。

遗憾的是，void* 到 T* 的隐式转换在 C 中很常见，它可能是真实代码里最主要的 C/C++ 兼容性问题（见 §27.4）。

27.3.6 enum

在 C 中，不用转换就能把 int 赋给 enum。例如：

```
enum color { red, blue, green };
int x = green;                /* 在 C 和 C++ 中都正确 */
enum color col = 7;    /* 在 C 中正确；在 C++ 中错误 */
```

一个推论是：在 C 中，我们可以对枚举类型的变量使用自增（++）和自减（--）。这也许很方便，但确实带来一种危险：

```
enum color x = blue;
++x;           /* x 变成 green；在 C++ 中错误 */
++x;           /* x 变成 3；在 C++ 中错误 */
```

“掉出”枚举量列表的末尾，未必就是我们想要的结果。

注意，与结构体标记一样，枚举的名字也有自己的名字空间，所以每次使用时都必须加上关键字 enum：

```
color c2 = blue;                      /* 在 C 中错误：color 不在作用域；在 C++ 中正确 */
enum color c3 = red;            /* 正确 */
```

27.3.7 名字空间

C 中没有（C++ 意义上的）名字空间。那么，在大型 C 程序中要避免名字冲突时该怎么办？通常人们使用前缀或后缀。例如：

```
/* 在 bs.h 中： */
typedef struct bs_string { /* . . . */ } bs_string;       /* Bjarne 的 string */
typedef int bs_bool ;                                    /* Bjarne 的 Boolean 类型 */

/* 在 pete.h 中： */
typedef char* pete_string;            /* Pete 的 string */
typedef char pete_bool ;              /* Pete 的 Boolean 类型 */
```

这种技术如此流行，以至于使用一两个字母的前缀通常是个坏主意。

27.4 自由存储

C 不提供处理对象的 new 和 delete 运算符。要使用自由存储，就得使用处理内存的函数。最重要的几个函数定义在“通用工具”标准头 <stdlib.h> 中：

```
void* malloc(size_t sz);                    /* 分配 sz 个字节 */
void free(void* p);                           /* 释放 p 所指向的内存 */
void* calloc(size_t n, size_t sz);     /* 分配 n*sz 个字节并初始化为 0 */
void* realloc(void* p, size_t sz);  /* 把 p 所指向的内存重新分配到大小为 sz 的空间 */
```

typedef size_t 是一种无符号类型，也定义在 <stdlib.h> 中。

为什么 malloc() 返回 void*？因为 malloc() 根本不知道你想把哪种类型的对象放进那块内存。初始化是你自己的事。例如：

```
struct Pair {
    const char* p;
    int val;
};

struct Pair p2 = {"apple",78};
struct Pair* pp = (struct Pair*) malloc(sizeof(Pair));     /* 分配 */
pp->p = "pear";              /* 初始化 */
pp->val = 42;
```

注意，我们不能写

```
*pp = {"pear", 42};        /* 错误：C 和 C++98 都不允许 */
```

在 C 和 C++ 中都不行。不过，在 C++ 中，我们会为 Pair 定义一个构造函数，然后写

```
Pair* pp = new Pair("pear", 42);
```

在 C 中（C++ 中不行，见 §27.3.4），可以省略 malloc() 前面的转换，但我们不推荐这样做：

```
int* p = malloc(sizeof(int)*n);     /* 避免这样写 */
```

省略转换相当流行，因为这样可以少打些字，而且能抓住那种少见的错误：（不合法地）在使用 malloc() 之前忘了包含 <stdlib.h>。不过，它也可能抹掉一条视觉线索，让人看不出大小算错了：

```
p = malloc(sizeof(char)*m);     /* 很可能是 bug——装不下 m 个 int */
```

不要在 C++ 程序中使用 `malloc()/free()`；`new/delete` 不需要转换，会处理初始化（构造函数）和清理（析构函数），会（通过异常）报告内存分配错误，而且同样快。不要 `delete` 由 `malloc()` 分配的对象，也不要 `free()` 由 `new` 分配的对象。例如：

```
int* p = new int[200];
// . . .
free(p);        // 错误

X* q = (X*)malloc(n*sizeof(X));
// . . .
delete q;       // 错误
```

这也许会工作，但不是可移植的代码。此外，对于带构造函数或析构函数的对象，把 C 风格与 C++ 风格的自由存储管理混在一起，是酿成灾难的配方。

`realloc()` 函数通常用于扩充缓冲区：

```
int max = 1000;
int count = 0;
int c;
char* p = (char*)malloc(max);
while ((c=getchar())!=EOF) {          /* 读入：忽略 eof 行上的字符 */
    if (count==max-1) {               /* 需要扩充缓冲区 */
        max += max;                   /* 缓冲区大小翻倍 */
        p = (char*)realloc(p,max);
        if (p==0) quit();
    }
    p[count++] = c;
}
```

关于 C 输入操作的说明，参见 §27.6.2 与 §B.11.2。

`realloc()` 函数未必会把原有的分配搬到新分配的内存里。千万不要对由 `new` 分配的内存使用 `realloc()`。

如果使用 C++ 标准库，大致等价的代码是

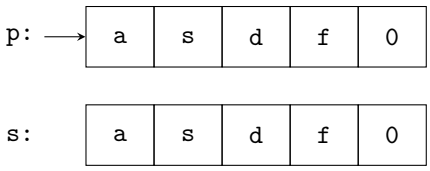
```
vector<char> buf;
char c;
while (cin.get(c)) buf.push_back(c);
```

关于输入与分配策略的更详尽讨论，参见论文“Learning Standard C++ as a New Language”（文献列表见 §27.1）。

27.5　C 风格字符串

在 C 中，字符串（在 C++ 文献里常称为 C 字符串或 C 风格字符串）是以 0 结尾的字符数组。例如：

```
char* p = "asdf";
char s[] = "asdf";
```



在 C 中，我们不能有成员函数，不能重载函数，也不能为结构体定义运算符（例如 ==）。由此可知，我们需要一组（非成员）函数来操作 C 风格字符串。C 与 C++ 标准库在 <string.h> 中提供了这类函数：

```
size_t strlen(const char* s);                          // 计算字符数
char* strcat(char* s1, const char* s2);                // 把 s2 接到 s1 末尾
int strcmp(const char* s1, const char* s2);            // 按字典序比较
char* strcpy(char* s1, const char* s2);                // 把 s2 复制到 s1
char* strchr(const char* s, int c);                    // 在 s 中查找 c
char* strstr(const char* s1, const char* s2);          // 在 s1 中查找 s2
char* strncpy(char*, const char*, size_t n);           // strcpy, 最多 n 个字符
char* strncat(char*, const char, size_t n);            // strcat, 最多 n 个字符
int strncmp(const char*, const char*, size_t n);       // strcmp, 最多 n 个字符
```

这并不是函数的全部，但都是最有用、最常用的。下面简要说明它们的用法。

我们可以比较字符串。相等运算符（==）比较的是指针值；标准库函数 strcmp() 比较的才是 C 风格字符串的值：

```
const char* s1 = "asdf";
const char* s2 = "asdf";

if (s1==s2) {   /* s1 与 s2 指向同一个数组吗？ */
                /* （这通常并不是你想要的结果） */
}

if (strcmp(s1,s2)==0) {
    /* s1 与 s2 保存的是相同的字符吗？ */
}
```

strcmp() 对其两个参数做三路比较。给定上面的 s1 与 s2，strcmp(s1,s2) 会返回 0，表示完全匹配。如果 s1 按字典序排在 s2 之前，它会返回一个负数；如果 s1 按字典序排在 s2 之后，则返回一个正数。“字典序”一词大致就是“像查字典那样”的意思。例如：

```
strcmp("dog","dog")==0
strcmp("ape","dodo")<0      // 在字典中 "ape" 排在 "dodo" 之前
strcmp("pig","cow")>0       // 在字典中 "pig" 排在 "cow" 之后
```

指针比较 s1==s2 的值并不保证为 0（假）。实现可能决定把所有字符字面量的副本存放在同一块内存里，于是我们会得到 1（真）。通常，比较 C 风格字符串的正确选择是 strcmp()。

我们可以用 strlen() 求 C 风格字符串的长度：

```
int lgt = strlen(s1);
```

注意，strlen() 统计字符时不计结尾的 0。这里 strlen(s1)==4，但存储 äsdf 需要 5 个字节。这一点小小的差别正是许多“差一错误”的根源。

我们可以把一个 C 风格字符串（包括结尾的 0）复制到另一个：

```
strcpy(s1,s2);   // 把 s2 中的字符复制到 s1
```

你的任务是确保目标字符串（数组）有足够空间容纳源字符串中的字符。

strncpy()、strncat() 与 strncmp() 是 strcpy()、strcat() 与 strcmp() 的受限版本：它们最多只处理 n 个字符，n 是它们的第三个参数。注意，如果源字符串中的字符多于 n 个，strncpy() 不会复制结尾的 0，结果就不再是合法的 C 风格字符串。

strchr() 与 strstr() 在第一个参数所指的字符串中查找第二个参数，并返回指向匹配处第一个字符的指针。与 find() 一样，它们从左向右搜索字符串。

这些简单函数能完成如此多的事，而犯个小错又如此容易，这实在令人惊叹。考虑一个简单问题：把用户名与地址连接起来，中间放一个 @ 字符。用 std::string 可以这样写：

```
string s = id + '@' + addr;
```

如果用标准的 C 风格字符串函数，我们可以写成

```
char* cat(const char* id, const char* addr)
{
    int sz = strlen(id)+strlen(addr)+2;
    char* res = (char*) malloc(sz);
    strcpy(res,id);
    res[strlen(id)+1] = '@';
    strcpy(res+strlen(id)+2,addr);
    res[sz-1]=0;
    return res;
}
```

我们写对了吗？谁来 free() cat() 返回的那个字符串？

试一试

测试 cat()。为什么是 2？我们在 cat() 里留下了一个初学者水平的性能错误：把它找出来并消除。我们还“忘记”给代码写注释了。请补上合适的注释——假定读者已经了解标准的 C 字符串函数。

27.5.1 C 风格字符串与 const

考虑：

```
char* p = "asdf";
p[2] = 'x';
```

这在 C 中合法，在 C++ 中不合法。在 C++ 中，字符串字面量是常量、不可变的值，所以 `p[2]='x'`（想把值改成 äsxf）是非法的。不幸的是，很少有编译器能抓住导致问题的那次对 p 的赋值。运气好的话运行时会出错，但别指望这一点。相反，应当这样写：

```
const char* p = "asdf";         // 现在你不能通过 p 改写 "asdf"
```

这条建议对 C 和 C++ 都适用。

C 的 `strchr()` 有一个类似、但更难以察觉的问题。考虑：

```
char* strchr(const char* s, int c);     /* 在常量 s 中查找 c（这不是 C++ 的声明） */
const char aa[] = "asdf";               /* aa 是一个常量数组 */
char* q = strchr(aa, 'd');              /* 找到 'd' */
*q = 'x';                               /* 把 aa 中的 'd' 改成 'x' */
```

同样，这在 C 和 C++ 中都是非法的，但 C 编译器抓不住它。有时这被称为“变质”（transmutation）：它把 const 变成非 const，违背了人们对代码的合理假设。

在 C++ 中，这个问题通过标准库对 `strchr()` 的不同声明得到解决：

```
char const* strchr(const char* s, int c);     // 在常量 s 中查找 c
char* strchr(char* s, int c);                 // 在 s 中查找 c
```

`strstr()` 同理。

27.5.2　字节操作

在遥远的黑暗时代（20 世纪 80 年代初），`void*` 尚未发明，C（以及 C++）程序员用字符串操作来处理字节。如今，基本的内存操作标准库函数以 `void*` 作为参数类型与返回类型，以提醒用户：他们正在直接操作本质上无类型的原始内存：

```
/* 从 s2 复制 n 个字节到 s1（类似 strcpy）： */
void* memcpy(void* s1, const void* s2, size_t n);

/* 从 s2 复制 n 个字节到 s1（[s1:s1+n) 可与 [s2:s2+n) 重叠）： */
void* memmove(void* s1, const void* s2, size_t n);

/* 比较 s2 与 s1 的 n 个字节（类似 strcmp）： */
int memcmp(const void* s1, const void* s2, size_t n);

/* 在 s 的前 n 个字节中查找 c（c 被转换为 unsigned char）： */
void* memchr(const void* s, int c, size_t n);

/* 把 c（转换为 unsigned char）复制到 s 所指的前 n 个字节中的每一个： */
void* memset(void* s, int c, size_t n);
```

不要在 C++ 中使用这些函数。特别是，`memset()` 通常会干扰构造函数所提供的保证。

27.5.3 示例：strcpy()

strcpy() 的定义既出名又声名不佳，它是 C（以及 C++）所能达到的那种简洁风格的典型例子：

```
char* strcpy(char* p, const char* q)
{
    while (*p++ = *q++);
    return p;
}
```

为什么这确实能把 C 风格字符串 q 复制到 p 里，解释留给你。后自增在 §A.5 中描述：p++ 的值是 p 自增之前的值。

试一试

这个 strcpy() 的实现正确吗？请解释原因。

如果你解释不了，我们就不会把你当作 C 程序员（无论你在其他语言的编程上多么在行）。每种语言都有自己的惯用法，而这就是 C 的惯用法之一。

27.5.4 一个风格问题

我们不动声色地在一个由来已久、常被激烈争论、但基本无关紧要的风格问题上选了边。我们这样声明指针：

```
char* p;            // p 是指向 char 的指针
```

而不是这样：

```
char *p;            /* p 是某种东西，对它解引用可以得到一个 char */
```

空白放在哪里对编译器完全无关紧要，但程序员在乎。我们的风格（在 C++ 中常见）强调被声明变量的类型，而另一种风格（在 C 中更常见）强调变量的用法。注意，我们不建议在一条声明里声明多个变量：

```
char c, *p, a[177], *f();   /* 合法，但令人困惑 */
```

这类声明在旧代码里并不少见。相反，应当使用多行声明，并利用多出来的横向空间写注释和初始化器：

```
char c = 'a';           /* 用 f() 输入时的终止字符 */
char* p = 0;            /* f() 最后读到的字符 */
char a[177];            /* 输入缓冲区 */
char* f();              /* 读入缓冲区 a；返回指向第一个读到字符的指针 */
```

另外，要选择有意义的名称。

27.6 输入输出：stdio

C 中没有 iostream，所以我们使用 <stdio.h> 中定义的 C 标准 I/O，通常简称为 stdio。cin 与 cout 在 stdio 中的对应物是 stdin 与 stdout。stdio 与 iostream 可以在同一个程序里混用（针对同一批 I/O 流），但我们不推荐这样做。如果你觉得非混不可，请查阅专家级教材中关于 stdio 与 iostream 的内容（尤其是 ios_base::sync_with_stdio()）。另见 §B.11。

27.6.1 输出

stdio 中最流行、最有用的函数是 printf()。printf() 最基本的用法就是打印一个（C 风格）字符串：

```
#include<stdio.h>

void f(const char* p)
{
    printf("Hello, World!\n");
    printf(p);
}
```

这并不特别有趣。有趣之处在于：printf() 可以接受任意多个参数，而开头那个字符串控制着这些额外参数是否被打印、以及怎样打印。printf() 在 C 中的声明是这样的：

```
int printf(const char* format, . . . );
```

. . . 的意思是“以及可选的更多参数”。我们可以这样调用 printf()：

```
void f1(double d, char* s, int i, char ch)
{
    printf("double %g string %s int %d char %c\n", d, s, i, ch);
}
```

这里，%g 的意思是“用通用格式打印一个浮点数”，%s 是“打印一个 C 风格字符串”，%d 是“用十进制数字打印一个整数”，%c 是“打印一个字符”。每个这样的格式说明符取用下一个尚未使用的参数，于是 %g 打印 d，%s 打印 s，%d 打印 i，%c 打印 ch。完整的 printf() 格式列表见 §B.11.2。

不幸的是，printf() 不是类型安全的。例如：

```
char a[] = { 'a', 'b' };            /* 没有结尾的 0 */

void f2(char* s, int i)
{
    printf("goof %s\n", i);         /* 错误，且不会被捕获 */
    printf("goof %d: %s\n", i);     /* 错误，且不会被捕获 */
    printf("goof %s\n", a);         /* 错误，且不会被捕获 */
}
```

最后一个 printf() 的效果很有意思：它会打印 a[1] 之后内存中的每一个字节，直到遇到 0 为止。那可能是很多字符。

缺乏类型安全，是我们在 stdio 与 iostream 之间更偏好 iostream 的原因之一，尽管 stdio 在 C 与 C++ 中的行为完全相同。另一个原因是 stdio 函数不可扩展：你无法像用 iostream 那样扩展 printf() 来打印自己类型的值。例如，你没有办法定义自己的 %Y 来打印某个结构体 Y。

printf() 有一个有用的版本，它把文件描述符作为第一个参数：

```
int fprintf(FILE* stream, const char* format, . . . );
```

例如：

```
fprintf(stdout,"Hello, World!\n");   // 与 printf("Hello, World!\n"); 完全相同
FILE* ff = fopen("My_file","w");     // 打开 My_file 以便写入
fprintf(ff,"Hello, World!\n");       // 把 "Hello, World!\n" 写到 My_file
```

文件句柄在 §27.6.3 中描述。

27.6.2 输入

最常用的 stdio 函数包括

```
int scanf(const char* format, . . . );    /* 用格式从 stdin 读取 */
int getchar(void);                        /* 从 stdin 取一个字符 */
int getc(FILE* stream);                   /* 从 stream 取一个字符 */
char* gets(char* s);                      /* 从 stdin 取字符 */
```

读取一串字符最简单的方法是用 gets()。例如：

```
char a[12];
gets(a);            /* 读入 a 所指的字符数组，直到输入 '\n' 为止 */
```

永远不要那么做！把 gets() 当作毒药。gets() 及其近亲 scanf(%s) 曾经是大约四分之一成功入侵事件的根源。它至今仍是重大的安全问题。在上面的简单例子里，你怎么知道在换行之前最多只会输入 11 个字符？你无法知道。因此，gets() 几乎必然导致内存破坏（破坏缓冲区之后的字节），而内存破坏正是破解者的主要工具。不要以为你能猜出一个“对所有用法都足够大”的最大缓冲区尺寸。也许输入流另一端的那个“人”是一个程序，它并不符合你对合理性的标准。

scanf() 用格式读取，就像 printf() 用格式写出一样。与 printf() 一样，它可能非常方便：

```
void f()
{
    int i;
    char c;
    double d;
    char* s = (char*)malloc(100);
    /* 读入作为指针传入的各个变量： */
    scanf("%i %c %g %s", &i, &c, &d, s);
    /* %s 跳过开头的空白，并以空白作为结束 */
}
```

与 printf() 一样，scanf() 不是类型安全的。格式字符与参数（全是指针）必须严格匹配，否则运行时会发生怪事。还要注意，读入 s 的 %s 可能导致溢出。永远不要用 gets() 或 scanf("%s")！

那么，怎样安全地读取字符呢？我们可以用一种 %s 形式，它对读取的字符数设置上限。例如：

```
char buf[20];
scanf("%19s",buf);
```

我们需要留出存放结尾 0 的空间（由 scanf() 补上），所以 19 是能读入 buf 的最大字符数。然而，这留下一个问题：如果有人确实输入了不止 19 个字符，该怎么办？那些“多余”的字符会留在输入流里，等着被后续的输入操作“发现”。

scanf() 的这些问题意味着，使用 getchar() 往往更谨慎、也更简单。用 getchar() 读取字符的典型写法是

```
while((x=getchar())!=EOF) {
    /* . . . */
}
```

EOF 是 stdio 的一个宏，意思是“文件结束”；另见 §27.4。

C++ 标准库中替代 scanf("%s") 与 gets() 的设施没有这些问题：

```
string s;
cin >> s;           // 读一个单词
getline(cin,s);     // 读一行
```

27.6.3 文件

在 C（或 C++）中，可以用 fopen() 打开文件，用 fclose() 关闭文件。这些函数，连同文件句柄的表示 FILE 以及 EOF（文件结束）宏，都在 <stdio.h> 中：

```
FILE *fopen(const char* filename, const char* mode);
int fclose(FILE *stream);
```

基本上，你可以这样使用文件：

```
void f(const char* fn, const char* fn2)
{
    FILE* fi = fopen(fn, "r");      /* 打开 fn 以便读取 */
    FILE* fo = fopen(fn2, "w");     /* 打开 fn2 以便写入 */

    if (fi == 0) error("failed to open input file");
    if (fo == 0) error("failed to open output file");

    /* 用 stdio 输入函数（例如 getc()）从文件读取 */
    /* 用 stdio 输出函数（例如 fprintf()）写入文件 */

    fclose(fo);
    fclose(fi);
}
```

想一想：C 中没有异常，那么无论发生哪种错误，我们怎样确保文件被关闭？

27.7 常量与宏

在 C 中，const 从来不是编译期常量：

```
const int max = 30;
const int x;            /* const 未初始化：在 C 中没问题（在 C++ 中是错误） */

void f(int v)
{
    int a1[max];   /* 错误：数组边界不是常量（在 C++ 中没问题） */
                   /* （常量表达式中不允许使用 max!） */
    int a2[x];     /* 错误：数组边界不是常量 */

    switch (v) {
    case 1:
        /* . . . */
        break;
    case max:      /* 错误：case 标号不是常量（在 C++ 中没问题） */
        /* . . . */
        break;
    }
}
```

在 C（而不是 C++）中，技术上的原因是：const 隐含地可以从其他翻译单元访问：

```
/* 文件 x.c: */
const int x;            /* 在其他地方初始化 */
```

```
/* file xx.c: */
const int x = 7;       /* 这里是真正的定义 */
```

在 C++ 里，那会是两个不同的对象，各自在自己的文件中都叫 x。C 程序员通常不用 const 表示符号常量，而倾向于使用宏。例如：

```
#define MAX 30

void f(int v)
{
    int a1[MAX];      /* 正确 */

    switch (v) {
    case 1:
        /* . . . */
        break;
    case MAX:         /* 正确 */
        /* . . . */
        break;
    }
}
```

宏名 MAX 被替换成字符 30，也就是这个宏的值；换言之，a1 的元素个数是 30，第二个 case 标号的值也是 30。按惯例，宏 MAX 全部用大写字母命名。这种命名约定有助于尽量减少宏所引发的错误。

27.8 宏

要提防宏：在 C 里没有真正有效的办法避开宏，但使用宏有严重的副作用，因为宏不遵守 C（或 C++）通常的作用域规则和类型规则。宏是一种文本替换。另见 §A.17.2。

除了（依靠 C++ 的替代设施，并）尽量少用宏之外，我们还能怎样保护自己，避开宏的潜在问题？

- 我们定义的每个宏都取全大写的名字。
- 不要把任何不是宏的东西取成全大写的名字。
- 绝不要给宏取简短或“俏皮”的名字，比如 max 或 min。
- 指望其他所有人都遵守这条简单而常见的约定。

宏的主要用途有：

- 定义“常量”
- 定义类似函数的构造
- 对语法做“改进”
- 控制条件编译

此外还有各种各样不那么常见的用法。

我们认为宏被严重滥用了，但在 C 程序里，使用宏并没有合理而完整的替代方案。即使在 C++ 程序里，要避开宏也可能很难（特别是当你写的程序必须可移植到很老的编译器，或移植到限制特殊的平台时）。

有些读者会把下面介绍的技术看作“肮脏的伎俩”，认为在体面的场合最好别提——这里先向他们致歉。不过我们相信，编程要在真实世界里做，而这些（非常温和的）宏用法与误用示例，能为新手程序员省下好几个小时的痛苦。对宏一无所知并不是福气。

27.8.1 函数式宏

下面是一个相当典型的函数式宏：

```
#define MAX(x, y) ((x)>=(y)?(x):(y))
```

我们用全大写的 MAX 来区别于（各种程序里）许多名为 max 的函数。显然，它与函数差别很大：没有参数类型，没有块，没有 return 语句，等等；还有，那么多括号都是干什么用的？考虑：

```
int aa = MAX(1,2);
double dd = MAX(aa++,2);
char cc = MAX(dd,aa)+2;
```

它展开为：

```
int aa = ((1)>=( 2)?(1):(2));
double dd = ((aa++)>=(2)?( aa++):(2));
char cc = ((dd)>=(aa)?(dd):(aa))+2;
```

要是没有“那么多括号”，最后一次展开就会变成

```
char cc = dd>=aa?dd:aa+2;
```

也就是说，cc 很容易得到一个与你按 cc 的定义作合理预期时不同的值。定义宏时，记得把参数每一次作为表达式使用的地方都加上括号。

另一方面，再多的括号也救不了第二次展开。宏参数 x 得到的值是 aa++，而 x 在 MAX 中用了两次，于是 aa 可能被自增两次。不要向宏传递带副作用的参数。

无巧不成书，还真有“天才”定义了这样一个宏，并把它塞进一个流行的头文件里。不幸的是，他还把它叫作 max 而不是 MAX，于是当 C++ 标准头文件定义

```
template<class T> inline T max(T a,T b) { return a<b?b:a; }
```

时，max 就被展开，参数是 T a 和 T b，编译器看到的是

```
template<class T> inline T ((T a)>=( T b)?( T a):( T b)) { return a<b?b:a; }
```

编译器的错误消息很“有趣”，而且帮助不大。紧急情况下，你可以“取消定义”一个宏：

```
#undef max
```

幸好那个宏并不那么重要。不过，流行的头文件里有成千上万个宏；把它们全部取消定义，不可能不造成大乱。

并非所有宏参数都当作表达式使用。考虑：

```
#define ALLOC(T,n) ((T*)malloc(sizeof(T)*n))
```

这是一个真实例子，对于避免“分配的预期类型”与“在 sizeof 中的使用”不匹配所造成的错误，它非常有用：

```
double* p = malloc(sizeof(int)*10);      /* 很可能是错误 */
```

不幸的是，要写出同时还能捕获内存耗尽的宏并不简单。下面这个也许能行，前提是你在某处恰当地定义了 error_var 和 error()：

```
#define ALLOC(T,n) (error_var = (T*)malloc(sizeof(T)*n), \
                                    (error_var==0)\
                                    ?(error("memory allocation failure"),0)\
                                    :error_var)
```

以 \ 结尾的那些行不是排版问题；这是把一个宏定义分成多行的写法。写 C++ 时，我们更愿意用 new。

27.8.2 语法宏

你可以定义一些宏，让源代码看起来更合自己的口味。例如：

```
#define forever for(;;)
#define CASE break; case
#define begin {
#define end }
```

我们强烈建议不要这样做。很多人都试过这个主意。他们（或者维护他们代码的人）发现

- 许多人对“更好的语法”的看法与你不同。
- 这种“改进过的”语法不合标准，出人意料；别人会被搞糊涂。
- “改进过的”语法有些用法会引起难以理解的编译期错误。
- 你看到的不是编译器看到的；编译器用它认识（并在源代码中看到）的词汇来报告错误，而不是用你的词汇。

不要为了“改进”代码外观而写语法宏。你和你的好友也许觉得这很不错，但经验表明，在更大的社区里你们只是极少数，所以总得有人重写你的代码（前提是它还能存活）。

27.8.3 条件编译

设想你有一个头文件的两个版本，比如说，一个用于 Linux，一个用于 Windows。在代码里怎么选择？一种常见做法是：

```
#ifdef WINDOWS
    #include "my_windows_header.h"
#else
    #include "my_linux_header.h"
#endif
```

现在，如果有人在这段代码被编译器看到之前定义了 WINDOWS，那么效果是

```
#include "my_windows_header.h"
```

否则效果是

```
#include "my_linux_header.h"
```

`#ifdef WINDOWS` 这个测试并不关心 `WINDOWS` 被定义成什么，它只测试它是否已被定义。

大多数主流系统（包括所有操作系统的各种变体）都定义了可供检查的宏。检查当前是按 C++ 还是按 C 编译的写法是

```
#ifdef __cplusplus
    // 按 C++ 编译
#else
    /* 按 C 编译 */
#endif
```

一种类似的构造通常称为包含卫士（include guard），常用来防止一个头文件被 `#include` 两次：

```
/* my_windows_header.h: */
#ifndef MY_WINDOWS_HEADER
#define MY_WINDOWS_HEADER
    /* 这里是头文件内容 */
#endif
```

`#ifndef` 测试检查某个东西没有被定义；也就是说，`#ifndef` 是 `#ifdef` 的反面。从逻辑上说，这些用来控制源文件的宏，与用来修改源代码的宏非常不同。它们只是碰巧用同样的底层机制来完成各自的工作。

27.9 例子：侵入式容器

C++ 标准库容器（如 `vector` 和 `map`）是非侵入式的；也就是说，它们不要求在用作元素的类型里存放任何数据。正因如此，只要那些类型可以拷贝，它们就能很好地泛化到几乎所有类型（内置的或用户定义的）。还有另一类容器叫作侵入式容器（intrusive container），在 C 和 C++ 中都很流行。我们将用一个非侵入式的链表来说明 `struct`、指针和自由存储的 C 风格用法。

我们来定义一个双向链表，它带九个操作：

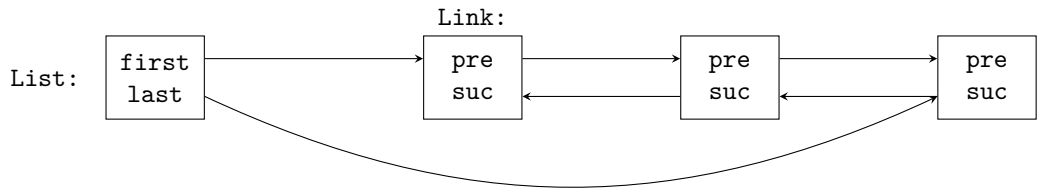
```
void init(struct List* lst);                    /* 把 lst 初始化为空表 */
struct List* create();                          /* 在自由存储上新建一个空表 */
void clear(struct List* lst);                   /* 释放 lst 的所有元素 */
void destroy(struct List* lst);                 /* 释放 lst 的所有元素，再释放 lst */
void push_back(struct List* lst, struct Link* p);    /* 把 p 加到 lst 末尾 */
void push_front(struct List*, struct Link* p);       /* 把 p 加到 lst 开头 */
```

```
/* 把 q 插到 lst 中 p 之前: */
void insert(struct List* lst, struct Link* p, struct Link* q);
struct Link* erase(struct List* lst, struct Link* p);  /* 从 lst 中移除 p */

/* 返回 p 之前或之后 n “跳”处的链接: */
struct Link* advance(struct Link* p, int n);
```

这样设计的意图是：用户只需接触 List* 和 Link* 就能使用这些操作。这意味着这些函数的实现可以彻底改动，而不影响这些用户。显然，命名受了 STL 的影响。List 和 Link 可以用最明显、最平凡的方式定义：

```
struct List {
    struct Link* first;
    struct Link* last;
};

struct Link {    /* 双向链表的链接 */
    struct Link* pre;
    struct Link* suc;
};
```

下面是 List 的图形表示：



我们的目的不是展示什么巧妙的表示技术或巧妙算法，所以这里两者都没有。不过请注意，这里完全没有提到 Link 到底保存着什么数据（也就是 List 的元素）。回头看我们提供的这些函数，会发现我们做的事很像定义一对抽象类 Link 和 List。Link 的数据以后才会补上。Link* 和 List* 有时被称为指向不透明类型的句柄；也就是说，把 Link* 和 List* 交给我们的函数，我们就能操作 List 的元素，而对 Link 或 List 的内部结构一无所知。

要实现我们的 List 函数，首先 #include 一些标准库头文件：

```
#include<stdio.h>
#include<stdlib.h>
#include<assert.h>
```

C 没有命名空间，所以我们不必操心 using 声明或 using 指令。另一方面，我们大概该担心自己占用了几个非常常见的短名字（Link、insert、init 等），因此这组函数不能在玩具程序之外“原样”使用。

初始化很简单，但要注意 assert() 的用法：

```
void init(struct List* lst)        /* 把 *lst 初始化为空表 */
{
    assert(lst);
    lst->first = lst->last = 0;
}
```

我们决定不在运行期处理指向链表的坏指针。用 assert()，只是让链表指针为空时给出一个（运行期）系统错误。这个“系统错误”会给出失败的那个 assert() 所在的文件名和行号；assert() 是定义在 <assert.h> 中的宏，而且检查只在调试期间启用。在没有异常的情况下，很难知道该拿坏指针怎么办。

create() 函数只是在自由存储上建立一个 List。它可以算是一种组合：既有构造函数的作用（init() 负责初始化），又有 new 的作用（malloc() 负责分配）：

```
struct List* create()              /* 在自由存储上新建一个空表 */
{
    struct List* lst = (struct List*)malloc(sizeof(struct List));
    init(lst);
    return lst;
}
```

clear() 函数假定所有 Link 都建在自由存储上，并把它们 free() 掉：

```
void clear(struct List* lst)           /* 释放 lst 的所有元素 */
{
    assert(lst);
    {
        struct Link* curr = lst->first;
        while(curr) {
            struct Link* next = curr->suc;
            free(curr);
            curr = next;
        }
        lst->first = lst->last = 0;
    }
}
```

注意我们用 Link 的 suc 成员进行遍历的方式。一个 struct 对象被 free() 之后，就不能再安全地访问它的成员了；所以在 free() 一个 Link 时，我们引入变量 next 来保存自己在 List 中的位置。

如果我们的 Link 并非全都分配在自由存储上，那最好别调用 clear()，否则 clear() 会制造大乱。

destroy() 函数本质上是 create() 的反面，也就是说，它兼有析构函数和 delete 的意味：

```
void destroy(struct List* lst)      /* 释放 lst 的所有元素，然后释放 lst */
{
    assert(lst);
    clear(lst);
    free(lst);
}
```

注意，我们完全没有为 Link 所表示的那些元素安排调用清理函数（析构函数）。这个设计并不是对 C++ 技术或通用性的完全忠实模仿——它做不到，大概也不该做到。

push_back() 函数——把一个 Link 作为新的最后一个 Link 接上去——相当直接：

```
void push_back(struct List* lst, struct Link* p)    /* 把 p 加到 lst 末尾 */
{
    assert(lst);
    {
        struct Link* last = lst->last;
        if (last) {
            last->suc = p;              /* 把 p 接在 last 之后 */
            p->pre = last;
        }
        else {
            lst->first = p;             /* p 是第一个元素 */
            p->pre = 0;
        }
        lst->last = p;                  /* p 是新的最后一个元素 */
        p->suc = 0;
    }
}
```

不过，要不是在草稿本上画几个方框和箭头，我们根本写不对。注意我们“忘了”考虑参数 p 为空的情况。传 0 而不是指向 Link 的指针，这段代码就会惨败。这并不算本质上糟糕的代码，但它不是工业强度的。它的目的是说明常见而有用的技术（在这个例子里，也说明一个常见的弱点 / 缺陷）。

erase() 函数可以这样写：

```
struct Link* erase(struct List* lst, struct Link* p)
/*
    从 lst 中移除 p;
    返回 p 之后那个链接的指针
*/
{
    assert(lst);
    if (p==0) return 0;             /* 对 erase(0) 也安全 */
    if (p == lst->first) {
        if (p->suc) {
            lst->first = p->suc;    /* 后继成为第一个元素 */
            p->suc->pre = 0;
```

```
            return p->suc;
        }
        else {
            lst->first = lst->last = 0;   /* 链表变为空 */
            return 0;
        }
    }
    else if (p == lst->last) {
        if (p->pre) {
            lst->last = p->pre;       /* 前驱成为最后一个元素 */
            p->pre->suc = 0;
        }
        else {
            lst->first = lst->last = 0;   /* 链表变为空 */
            return 0;
        }


    }
    else {
        p->suc->pre = p->pre;
        p->pre->suc = p->suc;
        return p->suc;
    }
}
```

其余的成员函数我们留作习题，因为在我们这个（过于简单的）测试中并不需要它们。不过，现在我们得面对这个设计最核心的谜团：list 各元素里的数据究竟存放在哪里？怎样实现一个简单的、用 C 风格字符串表示姓名的 list？请看：

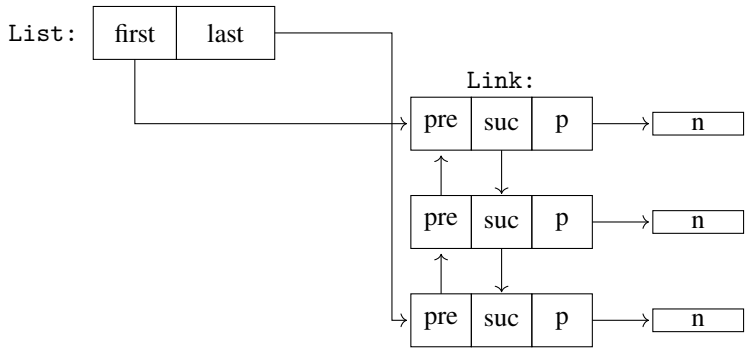
```
struct Name {
    struct Link lnk;    /* List 操作所需的 Link */
    char* p;            /* 名字字符串 */
};
```

到目前为止一切顺利，只是我们究竟怎样用上那个 Link 成员，仍然是个谜。不过，既然我们知道 List 喜欢把自己的 Link 放在自由存储区上，那就写一个在自由存储区上创建 Name 的函数：

```
struct Name* make_name(char* n)
{
    struct Name* p = (struct Name*)malloc(sizeof(struct Name));
    p->p = n;
    return p;
}
```

或者用图形表示：

现在我们来用它：




```
int main()
{
    int count = 0;
    struct List names;              /* 创建一个 list */
    struct List* curr;
    init(&names);
    /* 创建几个 Name 并把它们加入 list:  */
    push_back(&names,(struct Link*)make_name("Norah"));
    push_back(&names,(struct Link*)make_name("Annemarie"));
    push_back(&names,(struct Link*)make_name("Kris"));
    /* 删除第二个名字 (下标为 1):  */
    erase(&names,advance(names.first,1));
    curr = names.first;             /* 输出所有名字 */
    for (; curr!=0; curr=curr->suc) {
        count++;
        printf("element %d: %s\n", count, ((struct Name*)curr)->p);
    }
}
```

于是我们“作弊”了：我们做了一次强制转换，把 Name* 当作 Link* 来用。这样一来，用户知道有“库类型”Link 这回事；而“库”却不知道有“应用类型”Name 这回事。这样做允许吗？允许：在 C（以及 C++）中，可以把指向某个 struct 的指针当作指向其第一个成员的指针，反之亦然。

显然，这个 List 例子照原样写出来也是合法的 C++。

试一试

C++ 程序员和 C 程序员聊天时，常挂在嘴边的一句话是：“你能做的事，我都能做得更好！”那么，请用 C++ 重写这个侵入式 List 例子，说明怎样才能让代码更短、更易用，而既不让代码变慢，也不让对象变大。

小试身手

1. 用 C 写一个“Hello, World!”程序，编译它，然后运行它。
2. 定义两个变量，分别存放“Hello”和“World!”；用中间一个空格把它们拼接起来，再按 Hello World! 的形式输出。
3. 定义一个 C 函数，它接受一个 `char*` 参数 p 和一个 `int` 参数 x，并按这样的格式打印它们的值：p is “foo” and x is 7。用几组实参调用它。

要点回顾

以下各题中，我们所说的 C 都指 ISO 标准 C89。

1. C++ 是 C 的子集吗？
2. C 是谁发明的？
3. 举出一本评价很高的 C 教材。
4. C 和 C++ 是在哪个组织里发明的？
5. C++ 为什么（几乎）与 C 兼容？
6. C++ 为什么只是“几乎”与 C 兼容？
7. 列出十几个 C 中没有的 C++ 特性。
8. 哪个组织“拥有”C 和 C++？
9. 列出六个无法在 C 中使用的 C++ 标准库组件。
10. 哪些 C 标准库组件可以在 C++ 中使用？
11. 在 C 中怎样做到对函数实参做类型检查？
12. C 缺少哪些与函数有关的 C++ 特性？至少列出三个，并给出例子。
13. 怎样从 C++ 调用 C 函数？
14. 怎样从 C 调用 C++ 函数？
15. C 与 C++ 之间哪些类型是布局兼容的？（只需）举出例子。
16. 什么是结构标签（structure tag）？
17. 列出 20 个在 C 中不是关键字的 C++ 关键字。
18. 在 C++ 中，`int x;` 是定义吗？在 C 中呢？
19. 什么是 C 风格强制转换？它为什么危险？
20. 什么是 `void*`？它在 C 和 C++ 中有什么不同？
21. 枚举在 C 和 C++ 中有什么不同？
22. 在 C 中怎样避免那些常用名字引起的链接问题？
23. 与自由存储区的使用有关的三个最常见的 C 函数是什么？
24. C 风格字符串的定义是什么？
25. 对 C 风格字符串来说，`==` 和 `strcmp()` 有什么区别？
26. 怎样复制 C 风格字符串？
27. 怎样求 C 风格字符串的长度？
28. 你会怎样复制一个大的 `int` 数组？
29. `printf()` 好在哪里？它有哪些问题/局限？
30. 为什么绝不应该使用 `gets()`？可以改用哪些函数？
31. 在 C 中怎样打开一个文件用于读取？
32. C 中的 `const` 与 C++ 中的 `const` 有什么区别？
33. 我们为什么不喜欢宏？
34. 宏的常见用途有哪些？
35. 什么是包含守卫（include guard）？

术语

- `#define`
- `#ifdef`
- `#ifndef`

- Bell Labs（贝尔实验室）
- Brian Kernighan
- C/C++
- compatibility（兼容性）
- conditional compilation（条件编译）
- C-style cast（C 风格强制转换）
- C-style string（C 风格字符串）
- Dennis Ritchie
- `FILE`
- `fopen()`
- format string（格式字符串）
- intrusive（侵入式）
- K&R
- lexicographical（字典序）
- linkage（链接）
- macro（宏）
- `malloc()`
- non-intrusive（非侵入式）
- opaque type（不透明类型）
- overloading（重载）
- `printf()`
- `strcpy()`
- structure tag（结构标签）
- three-way comparison（三路比较）
- `void`
- `void*`

习题

做这些习题时，最好把每个程序都用 C 编译器和 C++ 编译器各编译一遍。如果你只用 C++ 编译器，就可能无意中用上非 C 的特性；如果你只用 C 编译器，类型错误就可能查不出来。

1. 实现 `strlen()`、`strcmp()` 和 `strcpy()` 的各个版本。
2. 补全 §27.9 中的侵入式 List 例子，并用每一个函数测试它。
3. 尽可能把 §27.9 中的侵入式 List 例子“修饰”得便于使用。要尽量捕获并处理各种错误。改动 struct 定义的细节、使用宏等等，都是允许的。
4. 如果还没有做，请写出 §27.9 中侵入式 List 例子的 C++ 版本，并用每一个函数测试它。
5. 比较习题 3 和习题 4 的结果。
6. 在不改变各函数所提供的用户接口的前提下，改变 §27.9 中 Link 和 List 的表示方式。把 Link 分配在一个 link 数组中，并让成员 first、last、pre 和 suc 都变成 `int`（数组下标）。
7. 与 C++ 标准的（非侵入式）容器相比，侵入式容器有哪些优点和缺点？请分别列出优点清单和缺点清单。

8. 你的机器上的字典序是什么样的？把键盘上每个字符连同它的整数值一起写出来；然后按整数值决定的顺序写出这些字符。
9. 只使用 C 的设施（包括 C 标准库），从 stdin 读入一个单词序列，并按字典序把它们写到 stdout。提示：C 的排序函数叫 `qsort()`；请到某处查一查它。另一种做法是读入时就把单词插入一个有序 list。C 标准库里并没有 list。
10. 列出 C 语言从 C++ 或“带类的 C”（§27.1）中采纳的语言特性。
11. 列出 C 语言中未被 C++ 采纳的语言特性。
12. 实现一个（C 风格字符串，int）查找表，带有 `find(struct table*, const char*)`、`insert(struct table*, const char*, int)` 和 `remove(struct table*, const char*)` 之类的操作。表的表示可以采用一个 struct pair 数组，也可以采用一对数组（`const char*[]` 和 `int*`）；由你选择。各函数的返回类型也由你选择。请把你的设计决策记录下来。
13. 写一个在 C 中完成与 `string s; cin>>s;` 等价工作的程序；也就是说，定义一个输入操作，把任意长的、以空白结尾的字符序列读入一个以零结尾的 char 数组。
14. 写一个函数，输入是一个 `int` 数组，找出其中最小和最大的元素。它还应当计算中位数和平均值。用一个保存结果的 struct 作为返回值。
15. 在 C 中模拟单继承。让每个“基类”含有一个指针，指向一个由函数指针组成的数组（以此把虚函数模拟成独立函数，它们的第一个实参是指向“基类”对象的指针）；参见 §27.2.3。实现“派生”的办法是：让“基类”成为派生类第一个成员的类型。为每个类适当地初始化那个“虚函数”数组。为了检验这些想法，请实现一版“那个老 Shape 例子”，其中基类和派生类的 `draw()` 只打印出自己类的名字。只使用标准 C 中可用的语言特性和库设施。
16. 用宏来遮掩（简化记法）上一题实现中的写法。

后记

我们确实说过，兼容性问题并不那么令人兴奋。然而“外面”有大量的 C 代码（几十亿行），如果你必须读它或写它，本章就能让你做好准备。就个人而言，我们更喜欢 C++，而本章的内容部分地说明了原因。另外，请不要低估那个“侵入式 List”例子——“侵入式 list”和不透明类型都是重要而强大的技术（在 C 和 C++ 中都是如此）。

附录 B　标准库概要

"All complexities should, if possible, be buried out of sight."
("一切复杂性，只要可能，都应埋于视线之外。")
——David J. Wheeler

本附录扼要介绍 C++ 标准库中的关键设施。这份概要有意作了取舍，面向的是那些想对标准库设施有个总体了解、并希望在本书所讲主题之外再稍作探索的初学者。

B.1　概述
- **B.1.1**　头文件
- **B.1.2**　命名空间 std
- **B.1.3**　描述风格

B.2　错误处理
- **B.2.1**　异常

B.3　迭代器
- **B.3.1**　迭代器模型
- **B.3.2**　迭代器类别

B.4　容器
- **B.4.1**　概述
- **B.4.2**　成员类型
- **B.4.3**　构造函数、析构函数与赋值
- **B.4.4**　迭代器
- **B.4.5**　元素访问
- **B.4.6**　栈与队列操作
- **B.4.7**　链表操作
- **B.4.8**　大小与容量
- **B.4.9**　其他操作
- **B.4.10**　关联容器操作

B.5　算法
- **B.5.1**　非修改序列算法
- **B.5.2**　修改序列算法
- **B.5.3**　实用算法
- **B.5.4**　排序与查找
- **B.5.5**　集合算法
- **B.5.6**　堆
- **B.5.7**　排列
- **B.5.8**　min 与 max

B.6　STL 工具
- **B.6.1**　插入器
- **B.6.2**　函数对象
- **B.6.3**　`pair` 与 `tuple`
- **B.6.4**　`initializer_list`
- **B.6.5**　资源管理指针

B.7　输入/输出流
- **B.7.1**　输入/输出流层次
- **B.7.2**　错误处理
- **B.7.3**　输入操作
- **B.7.4**　输出操作
- **B.7.5**　格式化
- **B.7.6**　标准操纵符

B.8　字符串操作
- **B.8.1**　字符分类
- **B.8.2**　字符串
- **B.8.3**　正则表达式匹配

B.9　数值
- **B.9.1**　数值限制
- **B.9.2**　标准数学函数
- **B.9.3**　复数
- **B.9.4**　valarray
- **B.9.5**　推广的数值算法
- **B.9.6**　随机数

B.10　时间

B.11　C 标准库函数
- **B.11.1**　文件
- **B.11.2**　`printf()` 家族
- **B.11.3**　C 风格字符串
- **B.11.4**　内存
- **B.11.5**　日期与时间
- **B.11.6**　其他

B.12　其他库

B.1　概述

本附录是一份参考材料。它不像普通章节那样可以从头到尾连续阅读，而是（或多或少系统地）描述 C++ 标准库的关键要素。不过它并不是一份完整的参考，只是一份带有少量关键示例的概要。若想得到更完整的说明，你往往需要翻阅相关章节。还要注意，这份概要并不试图达到标准的精确程度和术语规范。更多信息请参见 Stroustrup 的《The C++ Programming Language》。完整的定义是 ISO C++ 标准，但那份文件并不是为初学者准备、也不适合初学者。别忘了使用你的在线文档。

一份有取舍（因而并不完整）的概要有什么用？你可以快速查找某个已知的操作，也可以快速浏览某一节，看看都有哪些常用操作可用。你很可能还得去别处寻找详细解释，不过这没关系：现在你已经有了线索，知道该找什么。此外，这份概要还包含指向各章教学内容的交叉引用。本附录提供了一份标准库设施的紧凑全貌。请不要试图背下这里的信息，那并不是它的用途。恰恰相反，本附录是一件工具，能让你免于无谓的死记硬背。

这里是你查找有用设施的地方——免得你自己去重新发明它们。标准库中的每一样东西（尤其是本附录重点介绍的每一样东西）都已对大批用户证明过自己的用处。标准库设施几乎肯定比你在匆忙之中设计并实现出来的任何东西都设计得更好、实现得更好、文档更完备、可移植性更强。所以，只要能用标准库设施，就不要用“自家土法”。这样做还会让你的代码更容易被别人理解。

如果你是通情达理的人，光是设施数量之巨就会让你望而生畏。别担心，用不着的就忽略。如果你是那种“细节控”，又会觉得缺得太多。不过，完整性正是专家级指南和你的在线文档所能提供的东西。无论属于哪种情况，你都会发现许多看似神秘、或许还有趣的内容。去探索一些吧！

B.1.1　头文件

标准库设施的接口定义在头文件中。用这一节可以大致了解都有哪些设施可用，并帮助你猜测某个设施可能定义并描述在哪里：

标准模板库（容器、迭代器与算法）

`<algorithm>`	算法；`sort()`、`find()` 等（§B.5、§21.1）
`<array>`	固定大小的数组（§20.9）
`<bitset>`	`bool` 的数组（§25.5.2）
`<deque>`	双端队列
`<functional>`	函数对象（§B.6.2）
`<iterator>`	迭代器（§B.4.4）
`<list>`	双向链表（§B.4、§20.4）
`<forward_list>`	单向链表
`<map>`	（键，值）映射 `map` 与 `multimap`（§B.4、§21.6.1–3）

标准模板库（容器、迭代器与算法）（续）	
`<memory>`	容器的分配器
`<queue>`	`queue` 与 `priority_queue`
`<set>`	`set` 与 `multiset`（§B.4、§21.6.5）
`<stack>`	`stack`
`<unordered_map>`	哈希映射（§21.6.4）
`<unordered_set>`	哈希集合
`<utility>`	运算符与 `pair`（§B.6.3）
`<vector>`	`vector`（可动态扩展）（§B.4、§20.8）
输入/输出流	
`<iostream>`	输入/输出流对象（§B.7）
`<fstream>`	文件流（§B.7.1）
`<sstream>`	`string` 流（§B.7.1）
`<iosfwd>`	声明（但不定义）输入/输出流设施
`<ios>`	输入/输出流基类
`<streambuf>`	流缓冲区
`<istream>`	输入流（§B.7）
`<ostream>`	输出流（§B.7）
`<iomanip>`	格式化与操纵符（§B.7.6）
字符串处理	
`<string>`	`string`（§B.8.2）
`<regex>`	正则表达式（第 23 章）
数值	
`<complex>`	复数与复数运算（§B.9.3）
`<random>`	随机数生成（§B.9.6）
数值（续）	
`<valarray>`	数值数组
`<numeric>`	通用数值算法，例如 `accumulate()`（§B.9.5）
`<limits>`	数值限制（§B.9.1）

工具与语言支持

`<exception>`	异常类型（§B.2.1）
`<stdexcept>`	异常类层次（§B.2.1）
`<locale>`	与区域文化相关的格式化
`<typeinfo>`	标准类型信息（来自 `typeid`）
`<new>`	分配与释放函数
`<memory>`	资源管理指针，例如 `unique_ptr`（§B.6.5）

并发支持

`<thread>`	线程（超出本书范围）
`<future>`	线程间通信（超出本书范围）
`<mutex>`	互斥设施（超出本书范围）

C 标准库

`<cstring>`	C 风格字符串操作（§B.11.3）
`<cstdio>`	C 风格输入/输出（§B.11.2）
`<ctime>`	`clock()`、`time()` 等（§B.11.5）
`<cmath>`	标准浮点数学函数（§B.9.2）
`<cstdlib>`	杂项函数：`abort()`、`abs()`、`malloc()`、`qsort()` 等（第 27 章）
`<cerrno>`	C 风格错误处理（§24.8）
`<cassert>`	`assert` 宏（§27.9）
`<clocale>`	与区域文化相关的格式化
`<climits>`	C 风格数值限制（§B.9.1）
`<cfloat>`	C 风格浮点极限（§B.9.1）

C 标准库（续）

`<cstddef>`	C 语言支持；`size_t` 等
`<cstdarg>`	处理可变参数的宏
`<csetjmp>`	`setjmp()` 与 `longjmp()`（永远不要用它们）
`<csignal>`	信号处理
`<cwchar>`	宽字符
`<cctype>`	字符类型分类（§B.8.1）
`<cwctype>`	宽字符类型分类

对每个 C 标准库头文件，还存在一个去掉开头 c、并加上后缀.h 的版本，例如 `<ctime>` 对应 `<time.h>`。这些.h 版本定义的是全局名字，而不是命名空间 `std` 中的名字。

这些头文件中所定义的设施，有一些（但并非全部）会在下面各节以及各章中描述。如果你需要更多信息，请查阅你的在线文档或一本专家级的 C++ 书。

B.1.2　命名空间 std

标准库设施定义在命名空间 std 中，因此要使用它们，需要显式限定、using 声明或 using 指令：

```
std::string s;                  // 显式限定
using std::vector;              // using 声明
vector<int>v(7);

using namespace std;            // using 指令
map<string,double> m;
```

在本书中，我们对 std 使用了 using 指令。使用 using 指令一定要非常节制；参见 §A.15。

B.1.3　描述风格

哪怕是最简单的标准库操作（例如一个构造函数或一个算法），完整的描述也可能占用好几页。因此，我们采用一种极其简略的表述风格。例如：

记号的示例

p=op(b,e,x)	op 对区间 [b:e) 和 x 做些事情，然后返回 p。
foo(x)	foo 对 x 做些事情，但不返回结果。
bar(b,e,x)	x 与 [b:e) 有关系吗？

我们尽量让标识符的选取便于记忆：b、e 是标定一个区间的迭代器，p 是一个指针或一个迭代器，x 是某个值——具体所指都取决于上下文。在这种记法下，只有说明文字才能把“没有结果”和“结果是布尔值”区分开，所以只要你有意为之，完全可以把两者搞混。对于返回 bool 的操作，说明通常以问号结尾。

如果某个算法遵循通常的做法，用返回输入序列末尾的方式来表示“失败”“未找到”等情况（§B.3.1），我们就不再明确说明这一点。

B.2　错误处理

标准库由四十多年间陆续开发出来的组件组成。因此，它们处理错误的风格和做法并不一致。

- C 风格的库由函数组成，其中许多函数会设置 errno 来表示发生了错误；参见 §24.8。
- 许多对元素序列进行操作的算法会返回指向末尾之后那个元素的迭代器，以表示“未找到”或“失败”。
- 输入/输出流库依靠每个流中的状态来反映错误，并且可以（如果用户提出要求）抛出异常来表示错误；参见 §10.6、§B.7.2。
- 有些标准库组件（如 vector、string 和 bitset）会抛出异常来表示错误。

标准库的设计保证所有设施都遵守“基本保证”（参见 §19.5.3）；也就是说，即使抛出异常，也不会有任何资源（如内存）泄漏，也不会破坏任何标准库类的不变式。

B.2.1　异常

有些标准库设施通过抛出异常来报告错误：

标准库异常

bitset	抛出 invalid_argument、out_of_range、overflow_error
dynamic_cast	如果无法完成转换，抛出 bad_cast
iostream	如果启用了异常，抛出 ios_base::failure
new	如果无法分配内存，抛出 bad_alloc
regex	抛出 regex_error
string	抛出 length_error、out_of_range
typeid	如果无法给出 type_info，抛出 bad_typeid
vector	抛出 out_of_range

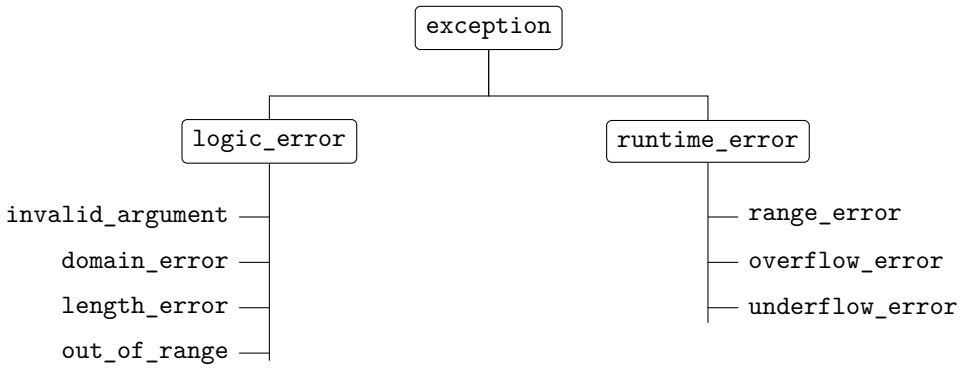
凡是直接或间接用到这些设施的代码，都可能遇到这些异常。除非你确信没有任何设施会以可能抛出异常的方式被使用，否则最好总是在某处（例如在 main() 中）捕获标准库异常层次中的某个根类（例如 exception）。

我们强烈建议你不要抛出内置类型，例如 int 和 C 风格字符串。应该改抛专门定义为异常的类型对象。从标准库类 exception 派生出的类就可以用于此目的：

```
class exception {
public:
    exception();
    exception(const exception&);
    exception& operator=(const exception&);
    virtual ~exception();
    virtual const char* what() const;
};
```

what() 函数可用来获得一个字符串，这个字符串应当说明导致该异常的错误的一些情况。

下面这个标准异常类层次可能有助于对异常进行分类：

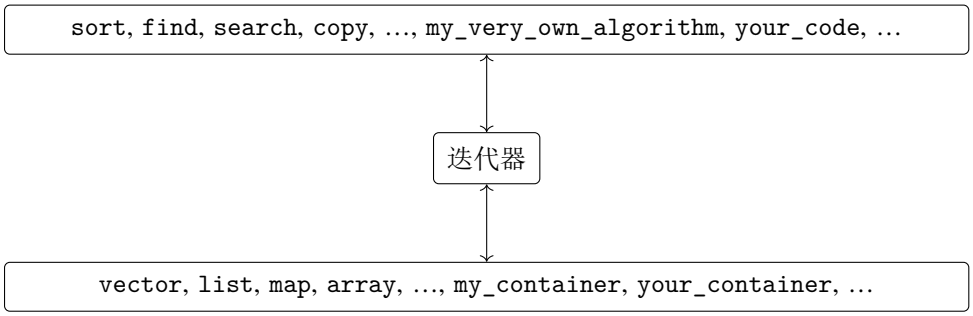


你可以通过从某个标准库异常类派生来定义自己的异常，例如：

```
struct My_error : runtime_error {
    My_error(int x) : interesting_value{x} { }
    int interesting_value;
    const char* what() const override { return "My_error"; }
}
```

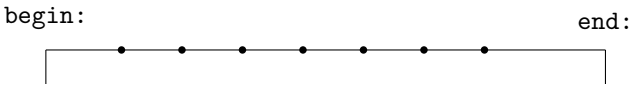
B.3　迭代器

迭代器是把标准库算法与其数据粘合起来的“胶水”。反过来也可以说，迭代器是这样一种机制：它把算法对其所操作的数据结构的依赖降到最低（§20.3）：



B.3.1　迭代器模型

迭代器有点像指针：它既提供间接访问元素的操作（例如用 * 解引用），又提供移动到新元素的操作（例如用 ++ 移到下一个元素）。一个元素序列由一对迭代器定义，这对迭代器给出一个半开区间 [begin:end)：



也就是说，begin 指向序列的第一个元素，end 指向序列末元素之后的位置。绝不要读写 *end。注意，空序列满足 begin==end；也就是说，对任意迭代器 p，[p:p) 都是空序列。

要读取一个序列，算法通常接受一对迭代器 (b,e)，并用 ++ 迭代，直到抵达末尾：

```
while (b!=e) {                  // 用 != 而不是 <
    // 做点什么
    ++b;                        // 移到下一个元素
}
```

在序列中查找某物的算法，通常返回序列末尾来表示“未找到”，例如：

```
p = find(v.begin(),v.end(),x);        // 在 v 中查找 x
if (p!=v.end()) {
    // 在 p 处找到了 x
}
else {
    // 在 [v.begin():v.end()) 中没有找到 x
}
```

另见 §20.3。

向序列写入的算法往往只被给予一个指向其首元素的迭代器。在这种情况下，程序员有责任不写到该序列末尾之后。例如：

```
template<class Iter> void f(Iter p, int n)
{
    while (n>0) *p++ = --n;
}

vector<int> v(10);
f(v.begin(),v.size());              // 正确
f(v.begin(),1000);                  // 大麻烦
```

对最后一次 f() 调用，有些标准库实现会做范围检查——也就是抛出异常——但可移植代码不能依赖这一点；很多实现并不检查。

迭代器上的操作有：

迭代器操作

++p	前置自增：让 p 指向序列中的下一个元素，或指向末元素之后的位置（“前进一个元素”）；结果值为 p+1。
p++	后置自增：让 p 指向序列中的下一个元素，或指向末元素之后的位置（“前进一个元素”）；结果值为自增之前的 p。
--p	前置自减：让 p 指向前一个元素（“回退一个元素”）；结果值为 p-1。
p--	后置自减：让 p 指向前一个元素（“回退一个元素”）；结果值为自减之前的 p。
*p	访问（解引用）：*p 指 p 所指向的那个元素。
p[n]	访问（下标）：p[n] 指 p+n 所指向的元素，等价于 *(p+n)。
p->m	访问（成员访问）：等价于 (*p).m。
p==q	相等：若 p 与 q 指向同一个元素，或都指向末元素之后，则为 true。
p!=q	不等：!(p==q)。
p<q	p 是否指向 q 所指元素之前的位置？
p<=q	p<q || p==q。
p>q	p 是否指向 q 所指元素之后的位置？
p>=q	p>q || p==q。
p+=n	前进 n：让 p 指向它当前所指元素之后的第 n 个元素。
p-=n	前进 -n：让 p 指向它当前所指元素之前的第 n 个元素。
q=p+n	q 指向 p 所指元素之后的第 n 个元素。

迭代器操作（续）

q=p-n	q 指向 p 所指元素之前的第 n 个元素；此后有 q+n==p。
advance(p,n)	与 p+=n 类似，但即使 p 不是随机访问迭代器，也可以用 advance()；它可能要一步一步走完 n 步（穿过链表）。
x=distance(p,q)	与 q-p 类似，但即使 p 不是随机访问迭代器，也可以用 distance()；它可能要一步一步走完 n 步（穿过链表）。

注意，并非每一种迭代器（§B.3.2）都支持全部迭代器操作。

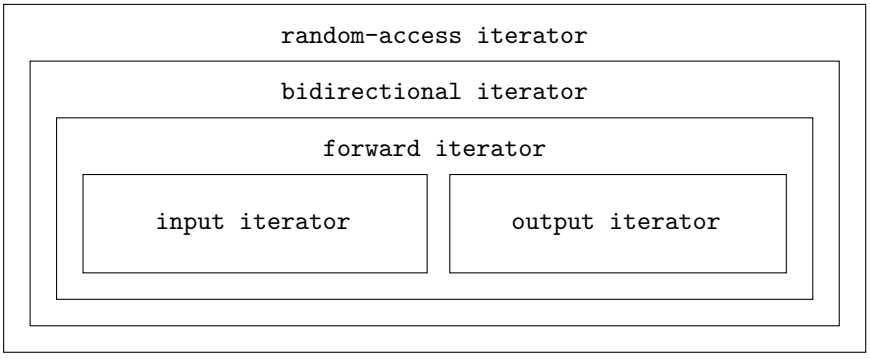
B.3.2　迭代器类别

标准库提供五种迭代器（五个“迭代器类别”）：

迭代器类别

输入迭代器（input iterator）	可以用 ++ 向前迭代，并只用 * 读取每个元素一次。可以用 == 和 != 比较迭代器。istream 提供的就是这种迭代器；见 §21.7.2。
输出迭代器（output iterator）	可以用 ++ 向前迭代，并只用 * 写入每个元素一次。ostream 提供的就是这种迭代器；见 §21.7.2。
前向迭代器（forward iterator）	可以用 ++ 反复向前迭代，并用 * 读写元素（除非元素是 const）。若它指向类对象，还可以用 -> 访问成员。
双向迭代器（bidirectional iterator）	可以向前（用 ++）和向后（用 --）迭代，并用 * 读写元素（除非元素是 const）。list、map 和 set 提供的就是这种迭代器。
随机访问迭代器（random-access iterator）	可以向前（用 ++ 或 +=）和向后（用 -- 或 -=）迭代，并用 * 或 [ ] 读写元素（除非元素是 const）。可以下标，也可以对随机访问迭代器加一个整数（用 +）、减一个整数（用 -）。把两个指向同一序列的随机访问迭代器相减，可以求出它们之间的距离。可以用 <、<=、> 和 >= 比较迭代器。vector 提供的就是这种迭代器。

从逻辑上看，这些迭代器组织成一个层次结构（§20.8）：



注意，由于迭代器类别并不是类，这个层次结构并不是用派生实现的类层次。若你要就迭代器类别做更高级的事情，请到进阶参考书里查 iterator_traits。

每个容器提供的迭代器都属于确定的类别：

- vector：随机访问

- list：双向
- forward_list：前向
- deque：随机访问
- bitset：无
- set：双向
- multiset：双向
- map：双向
- multimap：双向
- unordered_set：前向
- unordered_multiset：前向
- unordered_map：前向
- unordered_multimap：前向

B.4　容器

容器持有（hold）一个对象序列。序列中的元素属于一个名为 value_type 的成员类型。最常用的容器有：

序列容器	
array<T,N>	由 T 类型的 N 个元素构成的固定大小数组
deque<T>	双端队列
list<T>	双向链表
forward_list<T>	单向链表
vector<T>	T 类型元素的动态数组
关联容器	
map<K,V>	从 K 到 V 的映射；一个 (K,V) 对的序列
multimap<K,V>	从 K 到 V 的映射；允许重复的键
set<K>	K 的集合
multiset<K>	K 的集合（允许重复的键）
unordered_map<K,V>	使用哈希函数实现的从 K 到 V 的映射
unordered_multimap<K,V>	使用哈希函数实现的从 K 到 V 的映射；允许重复的键
unordered_set<K>	使用哈希函数实现的 K 的集合
unordered_multiset<K>	使用哈希函数实现的 K 的集合；允许重复的键

有序关联容器（map、set 等）还有一个可选的额外模板实参，用来指定比较所用的类型；例如 set<K,C> 用 C 来比较 K 值。

容器适配器

`priority_queue<T>`	优先队列
`queue<T>`	带 `push()` 和 `pop()` 的队列
`stack<T>`	带 `push()` 和 `pop()` 的栈

这些容器定义在 `<vector>`、`<list>` 等头文件中（见 §B.1.1）。序列容器是由其 `value_type` 类型的元素构成的连续分配数组或链表（即上面记号中的 `T`）。关联容器是由其 `value_type` 类型的节点构成的链式结构（树）（即上面记号中的 `pair(K,V)`）。`set`、`map` 或 `multimap` 的序列按键值（`K`）有序。`unordered_*` 的序列没有保证的顺序。`multimap` 与 `map` 的区别在于，同一个键值可以出现多次。容器适配器是用其他容器构造出来的、带有专门操作的容器。

拿不准时，就用 `vector`。除非有充分理由不用，否则就用 `vector`。容器用“分配器”（allocator）来分配和释放内存（§19.3.7）。这里不讨论分配器；如有需要，请参阅专家级参考书。默认情况下，分配器在为元素获取或释放内存时使用 `new` 和 `delete`。

在有意义的地方，访问操作都有两个版本：一个用于 `const` 对象，一个用于非 `const` 对象（§18.5）。

本节列出标准容器中常见以及几乎常见的成员。更详细的内容见第 20 章。只属于某个特定容器的成员（例如 `list` 的 `splice()`）不在列出之列；请参阅专家级参考书。

有些数据类型提供了标准容器所要求的大部分内容，但并非全部。我们有时把它们称为“准容器”（almost container）。其中最值得一提的是：

“准容器”

`T[n]`	内置数组。没有 `size()` 或其他成员函数；有选择时，优先用 `vector`、`string` 或 `array` 这类容器，而不是数组。
`string`	只存放字符，但提供对文本处理有用的操作，例如拼接（`+` 和 `+=`）；优先用标准 `string`，而不是其他字符串。
`valarray`	带有向量运算的数值向量，但有很多限制，以鼓励高性能实现；只有当你做大量向量算术时才用它。

B.4.1　概述

标准容器提供的操作可以这样概括：

- `Container`：构造函数、复制构造函数、默认构造函数，`begin()`、`end()`、`rbegin()`、`rend()`，`==`、`!=`、`<`、`<=`、`>`、`>=`，`=`、`swap()`、`size()`、`max_size()`、`empty()`、`insert()`、`erase()`、`clear()`
  - `Sequential container`（顺序容器）：`assign()`、`front()`、`back()`、`push_back()`、`pop_back()`、`resize()`
    - `list`：`push_front()`、`pop_front()`、`splice()`、`remove()`、`remove_if()`、`unique()`、`merge()`、`sort()`、`reverse()`
    - `vector`：`operator[]`、`at()`、`capacity()`、`reserve()`
    - `deque`：`operator[]`、`at()`、`push_front()`、`pop_front()`
- `Associative containers`（关联容器）：`key_comp()`、`value_comp()`、`find()`、`count()`、`lower_bound()`、`equal_range()`
  - `set`、`multiset` 与 `multimap`

- `map`：`operator[]`

我们没有列入 `array` 和 `forward_list`，因为它们并不完全符合标准库关于可互换性的理想：

- `array` 不是句柄，初始化之后元素个数不能改变，而且必须用初始化列表、而不是构造函数来初始化。
- `forward_list` 不支持尾部操作。特别是它没有 `size()`。最好把它看成一种针对空序列和接近空的序列优化过的容器。

B.4.2　成员类型

容器定义了一组成员类型：

成员类型	
`value_type`	元素的类型
`size_type`	下标、元素计数等的类型
`difference_type`	迭代器之差的类型
`iterator`	行为类似 `value_type*`
`const_iterator`	行为类似 `const value_type*`
`reverse_iterator`	行为类似 `value_type*`
`const_reverse_iterator`	行为类似 `const value_type*`
`reference`	`value_type&`
`const_reference`	`const value_type&`
`pointer`	行为类似 `value_type*`
`const_pointer`	行为类似 `const value_type*`
`key_type`	键的类型（仅关联容器）
`mapped_type`	映射值的类型（仅关联容器）
`key_compare`	比较判据的类型（仅关联容器）
`allocator_type`	内存管理器的类型

B.4.3　构造函数、析构函数与赋值

容器提供了多种构造函数和赋值操作。对于一个名为 C 的容器（例如 `vector<double>` 或 `map<string,int>`），我们有：

构造函数、析构函数与赋值	
`C c;`	c 是一个空容器。
`C{}`	创建一个空容器。
`C c(n);`	c 用 n 个具有默认元素值的元素初始化（不适用于关联容器）。
`C c(n,x);`	c 用 n 个 x 的副本初始化（不适用于关联容器）。
`C c {b,e};`	c 用 $[b:e)$ 中的元素初始化。
`C c {elems};`	c 用持有 elems 的 `initializer_list` 中的元素初始化。
`C c {c2};`	c 是 c2 的副本。
`~C()`	销毁 C 及其所有元素（通常是隐式调用）。
`c1=c2`	复制赋值：把 c2 的所有元素复制到 c1；赋值之后 c1==c2。
`c.assign(n,x)`	赋 n 个 x 的副本（不适用于关联容器）。
`c.assign(b,e)`	用 $[b:e)$ 赋值。

注意，对某些容器和某些元素类型，构造函数或元素复制可能抛出异常。

B.4.4　迭代器

一个容器既可以按容器迭代器定义的顺序看成序列，也可以按相反的顺序看成序列。对于关联容器，这个顺序取决于容器的比较判据（默认是 <）：

迭代器	
`p=c.begin()`	p 指向 c 的第一个元素。
`p=c.end()`	p 指向 c 的最后一个元素之后的位置。
`p=c.rbegin()`	p 指向 c 的反向序列的第一个元素。
`p=c.rend()`	p 指向 c 的反向序列的最后一个元素之后的位置。

B.4.5　元素访问

有些元素可以直接访问：

元素访问	
`c.front()`	c 第一个元素的引用
`c.back()`	c 最后一个元素的引用
`c[i]`	c 中第 i 个元素的引用；访问不做检查（不适用于 `list`）
`c.at(i)`	c 中第 i 个元素的引用；访问做检查（仅 `vector` 与 `deque`）

有些实现——尤其是调试版本——总是做范围检查；但你不能可移植地依赖这一点来保证正确性，也不能指望为了性能而不做检查。这类问题重要时，请仔细查看你所用实现的说明。

B.4.6　栈与队列操作

标准库的 vector 和 deque 在其元素序列的尾部（back）提供高效操作。此外，list 和 deque 还在其序列的头部（front）提供对应的操作：

栈与队列操作

c.push_back(x)	把 x 加到 c 末尾。
c.pop_back()	删除 c 的最后一个元素。
c.emplace_back(args)	把 T{args} 加到 c 末尾；T 是 c 的值类型。
c.push_front(x)	把 x 加到 c 的第一个元素之前（仅 list 与 deque）。
c.pop_front()	删除 c 的第一个元素（仅 list 与 deque）。
c.emplace_front(args)	把 T{args} 加到 c 的第一个元素之前；T 是 c 的值类型。

注意，push_front() 和 push_back() 会把一个元素复制进容器，这意味着容器的大小会增加（增加一个元素）。如果元素类型的复制构造函数可能抛出异常，那么一次 push 就可能失败。

push_front() 和 push_back() 把实参对象复制进容器。例如：

```
vector<pair<string,int>> v;
v.push_back(make_pair("Cambridge",1209));
```

如果先创建对象、再复制它显得别扭或者可能低效，我们可以在序列新分配的元素槽位中直接构造这个对象：

```
v.emplace_back("Cambridge",1209);
```

emplace 的意思是“放到位置上”（put in place）或“放到位置中”（put in position）。

注意，pop 操作不返回值。如果它们返回值，那么当复制构造函数抛出异常时，实现就可能被严重复杂化。请用 front() 和 back()（§B.4.5）访问栈和队列的元素。我们没有在这里记录完整的需求集合；不妨大胆猜测（猜错时编译器通常会告诉你），并查阅更详细的文档。

B.4.7　链表操作

容器提供链表操作：

链表操作

q=c.insert(p,x)	在 p 之前加入 x。
q=c.insert(p,n,x)	在 p 之前加入 n 个 x 的副本。
q=c.insert(p,first,last)	在 p 之前加入 $[first:last)$ 中的元素。
q=c.emplace(p,args)	在 p 之前构造 T{args}；T 是 c 的值类型。
q=c.erase(p)	从 c 中删除 p 处的元素。
q=c.erase(first,last)	删除 c 的 $[first:last)$ 区间。
c.clear()	删除 c 的所有元素。

对于 insert() 函数，结果 q 指向最后插入的那个元素。对于 erase() 函数，q 指向被删除的最后一个元素之后的那个元素。

B.4.8　大小与容量

大小（size）是容器中元素的个数；容量（capacity）是容器在再分配更多内存之前能够容纳的元素个数：

大小与容量	
`x=c.size()`	x 是 c 的元素个数。
`c.empty()`	c 是否为空？
`x=c.max_size()`	x 是 c 可能拥有的最大元素个数。
`x=c.capacity()`	x 是为 c 分配的空间（仅 `vector` 与 `string`）。
`c.reserve(n)`	为 c 预留 n 个元素的空间（仅 `vector` 与 `string`）。
`c.resize(n)`	把 c 的大小改为 n（仅 `vector`、`string`、`list` 与 `deque`）。

改变大小或容量时，元素可能被搬到新的存储位置。这意味着指向元素的迭代器（以及指针和引用）可能失效（例如仍指向旧的元素位置）。

B.4.9　其他操作

容器可以被复制（见 §B.4.3）、比较和交换：

比较与交换	
`c1==c2`	c1 与 c2 的所有对应元素都相等吗？
`c1!=c2`	c1 与 c2 是否存在对应元素不相等？
`c1<c2`	c1 在字典序上位于 c2 之前吗？
`c1<=c2`	c1 在字典序上位于 c2 之前或与 c2 相等吗？
`c1>c2`	c1 在字典序上位于 c2 之后吗？
`c1>=c2`	c1 在字典序上位于 c2 之后或与 c2 相等吗？
`swap(c1,c2)`	交换 c1 与 c2 的元素。
`c1.swap(c2)`	交换 c1 与 c2 的元素。

用运算符（例如 `<`）比较容器时，其元素是用对应的元素运算符（即 `<`）来比较的。

B.4.10　关联容器操作

关联容器提供基于键的查找：

关联容器操作	
`c[k]`	指键为 k 的元素（键唯一的容器）。
`p=c.find(k)`	p 指向键为 k 的第一个元素。
`p=c.lower_bound(k)`	p 指向键为 k 的第一个元素。

关联容器操作（续）	
`p=c.upper_bound(k)`	p 指向第一个键大于 k 的元素。
`pair(p1,p2)=c.equal_range(k)`	[p1:p2) 是键为 k 的元素。
`r=c.key_comp()`	r 是键比较对象的副本。
`r=c.value_comp()`	r 是 `mapped_value` 比较对象的副本。若未找到某个键，则返回 c.end()。

`equal_range` 返回的 pair 的第一个迭代器就是 lower_bound，第二个就是 upper_bound。你可以像下面这样打印 `multimap<string,int>` 中所有键为“Marian”的元素的值：

```
string k = "Marian";
auto pp = m.equal_range(k);
if (pp.first!=pp.second)
    cout << "elements with value '" << k << "':\n";
else
    cout << "no element with value '" << k << "'\n";
for (auto p = pp.first; p!=pp.second; ++p)
    cout << p->second << '\n';
```

我们也可以等价地写成

```
auto pp = make_pair(m.lower_bound(k),m.upper_bound(k));
```

不过，这样执行起来大约要慢一倍。`equal_range`、`lower_bound` 和 `upper_bound` 这几个算法也为有序序列提供（§B.5.4）。pair 的定义见 §B.6.3。

B.5　算法

`<algorithm>` 中定义了大约 60 个标准算法。它们都作用于由一对迭代器（用于输入）或单个迭代器（用于输出）所定义的序列。

在复制、比较等操作涉及两个序列时，第一个序列用一对迭代器 [b:e) 表示，而第二个序列只用一个迭代器 b2 表示；b2 被视为某个序列的起点，该序列含有算法所需的足够元素——例如与第一个序列同样多的元素：[b2:b2+(e-b))。

有些算法（如 `sort`）要求随机访问迭代器；而许多算法（如 `find`）只是按顺序读取元素，因此用前向迭代器就够了。

许多算法遵循一个惯例：用返回序列末尾来表示“没有找到”。对每个算法我们就不再一一说明这一点了。

B.5.1　非修改序列算法

非修改算法只读取序列的元素；它既不重新排列序列，也不改变元素的值：

非修改序列算法

`f=for_each(b,e,f)`	对 [b:e) 中的每个元素执行 f；返回 f。
`p=find(b,e,v)`	p 指向 v 在 [b:e) 中第一次出现的位置。
`p=find_if(b,e,f)`	p 指向 [b:e) 中第一个满足 f(*p) 的元素。
`p=find_first_of(b,e,b2,e2)`	p 指向 [b:e) 中第一个这样的元素：它与 [b2:e2) 中的某个 q 满足 *p==*q。
`p=find_first_of(b,e,b2,e2,f)`	p 指向 [b:e) 中第一个这样的元素：对 [b2:e2) 中的某个 q 满足 f(*p,*q)。
`p=adjacent_find(b,e)`	p 指向 [b:e) 中第一个满足 *p==*(p+1) 的 p。
`p=adjacent_find(b,e,f)`	p 指向 [b:e) 中第一个满足 f(*p,*(p+1)) 的 p。
`equal(b,e,b2)`	[b:e) 与 [b2:b2+(e-b)) 的所有元素都相等吗？
`equal(b,e,b2,f)`	以 f(*p,*q) 为判据，[b:e) 与 [b2:b2+(e-b)) 的所有元素都相等吗？
`pair(p1,p2)=mismatch(b,e,b2)`	(p1,p2) 指向 [b:e) 与 [b2:b2+(e-b)) 中第一对满足!(*p1==*p2) 的元素。
`pair(p1,p2)=mismatch(b,e,b2,f)`	(p1,p2) 指向 [b:e) 与 [b2:b2+(e-b)) 中第一对满足!f(*p1,*p2) 的元素。
`p=search(b,e,b2,e2)`	p 指向 [b:e) 中第一个等于 [b2:e2) 中某个元素的 *p。
`p=search(b,e,b2,e2,f)`	p 指向 [b:e) 中第一个这样的 *p：对 [b2:e2) 中的某个元素 *q 有 f(*p,*q)。
`p=find_end(b,e,b2,e2)`	p 指向 [b:e) 中最后一个等于 [b2:e2) 中某个元素的 *p。
`p=find_end(b,e,b2,e2,f)`	p 指向 [b:e) 中最后一个这样的 *p：对 [b2:e2) 中的某个元素 *q 有 f(*p,*q)。

非修改序列算法（续）

`p=search_n(b,e,n,v)`	p 指向 [b:e) 的第一个元素，使得 [p:p+n) 中每个元素的值都是 v。
`p=search_n(b,e,n,v,f)`	p 指向 [b:e) 的第一个元素，使得对 [p:p+n) 中的每个 *q 都有 f(*q,v)。
`x=count(b,e,v)`	x 是 v 在 [b:e) 中出现的次数。
`x=count_if(b,e,v,f)`	x 是 [b:e) 中满足 f(*p,v) 的元素个数。

请注意，并没有什么能阻止传给 `for_each` 的操作去修改元素；这被认为是可接受的。而把会修改其所检查元素的操作传给其他算法（例如 `count` 或 `==`），则是不可接受的。

下面是一个（正确使用的）例子：

```
bool odd(int x) { return x&1; }

int n_even(const vector<int>& v)            // 统计 v 中偶数值的个数
{
    return v.size()-count_if(v.begin(),v.end(),odd);
}
```

B.5.2　修改序列算法

修改算法（也称变异序列算法）可以（而且常常确实会）修改其参数序列的元素。

修改序列算法

`p=transform(b,e,out,f)`	对 [b:e) 中的每个 *p1 执行 *p2=f(*p1)，写入 [out:out+(e-b)) 中对应的 *p2；p=out+(e-b)。
`p=transform(b,e,b2,out,f)`	对 [b:e) 中的每个 *p1 以及 [b2:b2+(e-b)) 中对应的 *p2 执行 *p3=f(*p1,*p2)，写入 [out:out+(e-b)) 中的 *p3；p=out+(e-b)。
`p=copy(b,e,out)`	把 [b:e) 复制到 [out:p)。
`p=copy_backward(b,e,out)`	把 [b:e) 复制到 [out:p)，从其最后一个元素开始。

修改序列算法（续）

`p=unique(b,e)`	移动 [b:e) 中的元素，使 [b:p) 中相邻的重复元素被去掉（用 == 定义“重复”）。
`p=unique(b,e,f)`	移动 [b:e) 中的元素，使 [b:p) 中相邻的重复元素被去掉（用 f 定义“重复”）。
`p=unique_copy(b,e,out)`	把 [b:e) 复制到 [out:p)；不复制相邻的重复元素。
`p=unique_copy(b,e,out,f)`	把 [b:e) 复制到 [out:p)；不复制相邻的重复元素（用 f 定义“重复”）。
`replace(b,e,v,v2)`	把 [b:e) 中满足 *q==v 的元素 *q 替换为 v2。
`replace(b,e,f,v2)`	把 [b:e) 中满足 f(*q) 的元素 *q 替换为 v2。
`p=replace_copy(b,e,out,v,v2)`	把 [b:e) 复制到 [out:p)，并把其中满足 *q==v 的元素 *q 替换为 v2。
`p=replace_copy(b,e,out,f,v2)`	把 [b:e) 复制到 [out:p)，并把其中满足 f(*q) 的元素 *q 替换为 v2。
`p=remove(b,e,v)`	移动 [b:e) 中的元素 *q，使 [b:p) 成为满足!(*q==v) 的那些元素。
`p=remove(b,e,v,f)`	移动 [b:e) 中的元素 *q，使 [b:p) 成为满足!f(*q) 的那些元素。
`p=remove_copy(b,e,out,v)`	把 [b:e) 中满足!(*q==v) 的元素复制到 [out:p)。
`p=remove_copy_if(b,e,out,f)`	把 [b:e) 中满足!f(*q,v) 的元素复制到 [out:p)。
`reverse(b,e)`	反转 [b:e) 中元素的顺序。
`p=reverse_copy(b,e,out)`	按逆序把 [b:e) 复制到 [out:p)。
`rotate(b,m,e)`	旋转元素：把 [b:e) 看成一个圆环，最后一个元素之后紧接着第一个元素。把 *b 移到 *m；一般地，把 *(b+i) 移到 *((b+(i+(e-m))%(e-b))。
`p=rotate_copy(b,m,e,out)`	把 [b:e) 按旋转后的顺序复制到 [out:p)。
`random_shuffle(b,e)`	用默认的均匀随机数生成器，把 [b:e) 的元素打乱成一种随机分布。
`random_shuffle(b,e,f)`	用 f 作为随机数生成器，把 [b:e) 的元素打乱成一种随机分布。

打乱算法打乱其序列的方式，很像我们洗一副牌：洗过之后，元素处于随机的顺序，而这里的“随机”由随机数生成器所产生的分布来定义。

请注意，这些算法并不知道自己的参数序列是不是容器，因此它们没有能力增加或删除元素。所以像 `remove` 这样的算法不能通过删除（擦除）元素来缩短输入序列；相反，它把要保留的元素（重新）移到序列的前部：

```
template<typename Iter>
void print_digits(const string& s, Iter b, Iter e)
{
    cout << s;
    while (b!=e) { cout << *b; ++b; }
    cout << '\n';
}

void ff()
{
    vector<int> v {1,1,1, 2,2, 3, 4,4,4, 3,3,3, 5,5,5,5, 1,1,1};
    print_digits("all: ",v.begin(), v.end());

    auto pp = unique(v.begin(),v.end());
    print_digits("head: ",v.begin(),pp);
    print_digits("tail: ",pp,v.end());

    pp=remove(v.begin(),pp,4);
    print_digits("head: ",v.begin(),pp);
    print_digits("tail: ",pp,v.end());
}
```

运行得到的输出是

```
all: 1112234443335555111
head: 1234351
tail: 443335555111
head: 123351
tail: 1443335555111
```

B.5.3　实用算法

严格说来，这些实用算法也是修改序列算法，但我们认为把它们单列出来更好，以免它们被忽略。

实用算法

`swap(x,y)`	交换 x 与 y。
`iter_swap(p,q)`	交换 *p 与 *q。
`swap_ranges(b,e,b2)`	交换 [b:e) 与 [b2:b2+(e-b)) 的元素。
`fill(b,e,v)`	把 v 赋给 [b:e) 的每个元素。
`fill_n(b,n,v)`	把 v 赋给 [b:b+n) 的每个元素。
`generate(b,e,f)`	把 f() 赋给 [b:e) 的每个元素。
`generate_n(b,n,f)`	把 f() 赋给 [b:b+n) 的每个元素。
`uninitialized_fill(b,e,v)`	用 v 初始化 [b:e) 中的所有元素。
`uninitialized_copy(b,e,out)`	用 [b:e) 中对应的元素初始化 [out:out+(e-b)) 中的所有元素。

请注意，未初始化的序列只应出现在编程的最低层，通常位于容器的实现内部。`uninitialized_fill` 或 `uninitialized_copy` 的目标元素必须是内置类型，或者必须是未初始化的。

B.5.4　排序与查找

排序与查找是基本操作，而程序员的需求又相当多样。比较默认使用 < 运算符完成，两个值 a 与 b 的等价性由 `!(a<b)&&!(b<a)` 判定，而不要求定义 `operator==`。

排序与查找

`sort(b,e)`	对 [b:e) 排序。
`sort(b,e,f)`	用 f(*p,*q) 作为排序准则对 [b:e) 排序。
`stable_sort(b,e)`	对 [b:e) 排序，并保持等价元素之间的顺序。
`stable_sort(b,e,f)`	用 f(*p,*q) 作为排序准则对 [b:e) 排序，并保持等价元素之间的顺序。
`partial_sort(b,m,e)`	对 [b:e) 排序，使 [b:m) 有序；[m:e) 不必有序。
`partial_sort(b,m,e,f)`	用 f(*p,*q) 作为排序准则对 [b:e) 排序，使 [b:m) 有序；[m:e) 不必有序。

排序与查找（续）

`partial_sort_copy(b,e,b2,e2)`	对 [b:e) 排到足够的程度，把前 e2-b2 个元素复制到 [b2:e2)。
`partial_sort_copy(b,e,b2,e2,f)`	对 [b:e) 排到足够的程度，把前 e2-b2 个元素复制到 [b2:e2)；用 f 做比较。
`nth_element(b,e)`	把 [b:e) 中的第 n 个元素放到它应在的位置上。
`nth_element(b,e,f)`	把 [b:e) 中的第 n 个元素放到它应在的位置上，用 f 做比较。
`p=lower_bound(b,e,v)`	p 指向 v 在 [b:e) 中第一次出现的位置。
`p=lower_bound(b,e,v,f)`	p 指向 v 在 [b:e) 中第一次出现的位置，用 f 做比较。
`p=upper_bound(b,e,v)`	p 指向 [b:e) 中第一个大于 v 的值。
`p=upper_bound(b,e,v,f)`	p 指向 [b:e) 中第一个大于 v 的值，用 f 做比较。
`binary_search(b,e,v)`	v 在有序序列 [b:e) 中吗？
`binary_search(b,e,v,f)`	v 在有序序列 [b:e) 中吗（用 f 做比较）？
`pair(p1,p2)=equal_range(b,e,v)`	[p1,p2) 是 [b:e) 中值为 v 的子序列；本质上就是对 v 做二分查找。
`pair(p1,p2)=equal_range(b,e,v,f)`	[p1,p2) 是 [b:e) 中值为 v 的子序列（用 f 做比较）；本质上就是对 v 做二分查找。
`p=merge(b,e,b2,e2,out)`	把两个有序序列 [b2:e2) 与 [b:e) 合并到 [out:p)。
`p=merge(b,e,b2,e2,out,f)`	把两个有序序列 [b2:e2) 与 [b:e) 合并到 [out,out+p)，用 f 做比较。
`inplace_merge(b,m,e)`	把两个有序子序列 [b:m) 与 [m:e) 合并成一个有序序列 [b:e)。
`inplace_merge(b,m,e,f)`	把两个有序子序列 [b:m) 与 [m:e) 合并成一个有序序列 [b:e)，用 f 做比较。
`p=partition(b,e,f)`	把满足 f(*p1) 的元素放到 [b:p)，其余元素放到 [p:e)。
`p=stable_partition(b,e,f)`	把满足 f(*p1) 的元素放到 [b:p)，其余元素放到 [p:e)，并保持相对顺序。

例如：

```
vector<int> v {3,1,4,2};
list<double> lst {0.5,1.5,3,2.5};          // lst 已按序排列
sort(v.begin(),v.end());                   // 将 v 排好序
vector<double> v2;
merge(v.begin(),v.end(),lst.begin(),lst.end(),back_inserter(v2));
for (auto x : v2) cout << x << ", ";
```

插入器见 §B.6.1。输出为

```
0.5, 1, 1.5, 2, 2, 2.5, 3, 4,
```

equal_range、lower_bound 和 upper_bound 这几个算法的用法，与关联容器中对应的操作相同；见 §B.4.10。

B.5.5　集合算法

这些算法把序列当作元素集合，提供基本的集合运算。输入序列应当是已排序的，输出的序列也是有序的：

集合算法

`includes(b,e,b2,e2)`	[b2:e2) 的所有元素也都在 [b:e) 中吗？
`includes(b,e,b2,e2,f)`	[b2:e2) 的所有元素也都在 [b:e) 中吗（用 f 做比较）？
`p=set_union(b,e,b2,e2,out)`	构造一个有序序列 [out:p)，其中的元素在 [b:e) 或 [b2:e2) 中。
`p=set_union(b,e,b2,e2,out,f)`	构造一个有序序列 [out:p)，其中的元素在 [b:e) 或 [b2:e2) 中，用 f 做比较。
`p=set_intersection(b,e,b2,e2,out)`	构造一个有序序列 [out:p)，其中的元素既在 [b:e) 中又在 [b2:e2) 中。
`p=set_intersection(b,e,b2,e2,out,f)`	构造一个有序序列 [out:p)，其中的元素既在 [b:e) 中又在 [b2:e2) 中，用 f 做比较。
`p=set_difference(b,e,b2,e2,out)`	构造一个有序序列 [out:p)，其中的元素在 [b:e) 中而不在 [b2:e2) 中。

集合算法（续）

`p=set_difference(b,e,b2,e2,out,f)`	构造一个有序序列 [out:p)，其中的元素在 [b:e) 中而不在 [b2:e2) 中，用 f 做比较。
`p=set_symmetric_difference(b,e,b2,e2,out)`	构造一个有序序列 [out:p)，其中的元素在 [b:e) 或 [b2:e2) 中，但不同时在两者中。
`p=set_symmetric_difference(b,e,b2,e2,out,f)`	构造一个有序序列 [out:p)，其中的元素在 [b:e) 或 [b2:e2) 中，但不同时在两者中，用 f 做比较。

B.5.6　堆

堆（heap）是一种数据结构，它把值最大的元素放在最前面。堆算法让程序员可以把一个随机访问序列当作堆来使用：

堆操作

`make_heap(b,e)`	使该序列可以当作堆使用。
`make_heap(b,e,f)`	使该序列可以当作堆使用，用 f 做比较。
`push_heap(b,e)`	向堆中加入一个元素（放到它应在的位置）。
`push_heap(b,e,f)`	向堆中加入一个元素，用 f 做比较。
`pop_heap(b,e)`	从堆中删除最大（第一个）元素。
`pop_heap(b,e,f)`	从堆中删除一个元素，用 f 做比较。
`sort_heap(b,e)`	对堆排序。
`sort_heap(b,e,f)`	对堆排序，用 f 做比较。

堆的意义在于：既能快速加入元素，又能快速访问值最大的元素。堆的主要用途是实现优先队列。

B.5.7　排列

排列用来生成序列元素的组合。例如 abc 的排列是 abc、acb、bac、bca、cab 和 cba。

排列

`x=next_permutation(b,e)`	使 [b:e) 变成字典序中的下一个排列。
`x=next_permutation(b,e,f)`	使 [b:e) 变成字典序中的下一个排列，用 f 做比较。
`x=prev_permutation(b,e)`	使 [b:e) 变成字典序中的上一个排列。
`x=prev_permutation(b,e,f)`	使 [b:e) 变成字典序中的上一个排列，用 f 做比较。

`next_permutation` 的返回值 x 在 [b:e) 已经包含最后一个排列（上例中的 cba）时为 false；这种情况下，它返回第一个排列（上例中的 abc）。`prev_permutation` 的返回值在 [b:e) 已经包含第一个排列（上例中的 abc）时为 false；这种情况下，它返回最后一个排列（上例中的 cba）。

B.5.8　min 与 max

值的比较在许多场合都很有用：

min 与 max

`x=max(a,b)`	x 是 a 与 b 中较大的那个。
`x=max(a,b,f)`	x 是 a 与 b 中较大的那个，用 f 做比较。
`x=max({elems})`	x 是 {elems} 中最大的元素。
`x=max({elems},f)`	x 是 {elems} 中最大的元素，用 f 做比较。
`x=min(a,b)`	x 是 a 与 b 中较小的那个。
`x=min(a,b,f)`	x 是 a 与 b 中较小的那个，用 f 做比较。
`x=min({elems})`	x 是 {elems} 中最小的元素。
`x=min({elems},f)`	x 是 {elems} 中最小的元素，用 f 做比较。
`pair(x,y)=minmax(a,b)`	x 是 max(a,b)，y 是 min(a,b)。
`pair(x,y)=minmax(a,b,f)`	x 是 max(a,b,f)，y 是 min(a,b,f)。
`pair(x,y)=minmax({elems})`	x 是 max({elems})，y 是 min({elems})。
`pair(x,y)=minmax({elems},f)`	x 是 max({elems},f)，y 是 min({elems},f)。

min 与 max（续）

`p=max_element(b,e)`	p 指向 `[b:e)` 中最大的元素。
`p=max_element(b,e,f)`	p 指向 `[b:e)` 中最大的元素，元素比较使用 `f`。
`p=min_element(b,e)`	p 指向 `[b:e)` 中最小的元素。
`p=min_element(b,e,f)`	p 指向 `[b:e)` 中最小的元素，元素比较使用 `f`。
`lexicographical_compare(b,e,b2,e2)`	`[b:e)` 是否小于 `[b2:e2)`？
`lexicographical_compare(b,e,b2,e2,f)`	`[b:e)` 是否小于 `[b2:e2)`（元素比较使用 `f`）？

B.6　STL 工具

标准库还提供了一些设施，让标准库算法更便于使用。

B.6.1　插入器

通过容器的迭代器产生输出，意味着该迭代器所指的元素以及它后面的元素都可能被改写。这同时也意味着可能发生溢出，并由此造成内存破坏。例如：

```
void f(vector<int>& vi)
{
    fill_n(vi.begin(), 200, 7);    // 把 7 赋给 vi[0]..[199]
}
```

如果 `vi` 的元素少于 200 个，那就有麻烦了。

在 <iterator> 中，标准库提供了三种迭代器来解决这个问题：它们向容器增加（插入）元素，而不是改写旧元素。标准库提供了三个函数来生成这些插入迭代器：

插入器

r=back_inserter(c)	*r=x 会调用 c.push_back(x)。
r=front_inserter(c)	*r=x 会调用 c.push_front(x)。
r=inserter(c,p)	*r=x 会调用 c.insert(p,x)。

对于 inserter(c,p)，p 必须是容器 c 的合法迭代器。自然，每当通过插入迭代器写入一个值时，容器就增加一个元素。写入时，插入器用 push_back(x)、c.push_front() 或 insert() 把新元素插入序列，而不是改写已有元素。例如：

```
void g(vector<int>& vi)
{
    fill_n(back_inserter(vi), 200, 7);    // 向 vi 末尾添加 200 个 7
}
```

B.6.2　函数对象

许多标准算法接受函数对象（或函数）作为参数，用它们控制算法的工作方式。常见的用法是比较准则、谓词（返回 bool 的函数）和算术运算。在 <functional> 中，标准库提供了一些常用的函数对象。

谓词

p=equal_to<T>{}	若 x 和 y 的类型是 T，p(x,y) 意为 x==y。
p=not_equal_to<T>{}	若 x 和 y 的类型是 T，p(x,y) 意为 x!=y。
p=greater<T>{}	若 x 和 y 的类型是 T，p(x,y) 意为 x>y。
p=less<T>{}	若 x 和 y 的类型是 T，p(x,y) 意为 x<y。
p=greater_equal<T>{}	若 x 和 y 的类型是 T，p(x,y) 意为 x>=y。
p=less_equal<T>{}	若 x 和 y 的类型是 T，p(x,y) 意为 x<=y。
p=logical_and<T>{}	若 x 和 y 的类型是 T，p(x,y) 意为 x&&y。
p=logical_or<T>{}	若 x 和 y 的类型是 T，p(x,y) 意为 x||y。
p=logical_not<T>{}	若 x 的类型是 T，p(x) 意为 !x。

例如：

```
vector<int> v;
// . . .
sort(v.begin(), v.end(), greater<int>{});    // 按降序对 v 排序
```

注意，logical_and 和 logical_or 总是对两个实参都求值（而 && 和 || 不会）。

此外，lambda 表达式（§15.3.3）常常可以替代简单的函数对象：

```
sort(v.begin(), v.end(), [](int x, int y) { return x>y; });    // 按降序对 v 排序
```

算术运算

f=plus<T>{}	若 x 和 y 的类型是 T，f(x,y) 意为 x+y。
f=minus<T>{}	若 x 和 y 的类型是 T，f(x,y) 意为 x-y。
f=multiplies<T>{}	若 x 和 y 的类型是 T，f(x,y) 意为 x*y。
f=divides<T>{}	若 x 和 y 的类型是 T，f(x,y) 意为 x/y。
f=modulus<T>{}	若 x 和 y 的类型是 T，f(x,y) 意为 x%y。
f=negate<T>{}	若 x 的类型是 T，f(x) 意为 -x。

适配器

f=bind(g,args)	f(x) 意为 g(x,args)，其中 args 可以是一个或多个实参。
f=mem_fn(mf)	f(p,args) 意为 p->mf(args)，其中 args 可以是一个或多个实参。
Function<F> f {g}	f(args) 意为 g(args)，其中 args 可以是一个或多个实参，F 是 g 的类型。
f=not1(g)	f(x) 意为 !g(x)。
f=not2(g)	f(x,y) 意为 !g(x,y)。

注意 function 是一个模板，因此你可以定义类型为 function<T> 的变量，并把可调用对象赋给这样的变量。例如：

```
int f1(double);
function<int(double)> fct {f1};    // 初始化为 f1
int x = fct(2.3);                  // 调用 f1(2.3)
function<int(double)> fun;         // fun 可保存任何 int(double)
fun = f1;
```

B.6.3　pair 与 tuple

在 <utility> 中，标准库提供了一些“工具组件”，其中包括 pair：

```
template <class T1, class T2>
struct pair {
    typedef T1 first_type;
    typedef T2 second_type;
    T1 first;
    T2 second;
    // . . . 复制与移动操作 . . .
};
```

```
template <class T1, class T2>
constexpr pair<T1,T2> make_pair(T1 x, T2 y) { return pair<T1,T2>{x,y}; }
```

make_pair() 函数让 pair 的使用变得简单。例如，下面是一个返回“值 + 错误指示器”的函数的框架：

```
pair<double,error_indicator> my_fct(double d)
{
    errno = 0;    // 清除 C 风格的全局错误指示器
    // . . . 进行大量涉及 d 的计算，算出 x . . .
    error_indicator ee = errno;
    errno = 0;    // 清除 C 风格的全局错误指示器
    return make_pair(x,ee);
}
```

这个有用的惯用法可以这样使用：

```
pair<int,error_indicator> res = my_fct(123.456);
if (res.second==0) {
    // . . . 使用 res.first . . .
}
else {
    // 糟糕：出错了
}
```

当我们需要恰好两个元素、又不在意定义一个专门的类型时，就用 pair。如果需要零个或多个元素，可以使用 <tuple> 中的 tuple：

```
template <typename... Types>
struct tuple {
    explicit constexpr tuple(const Types& ...);        // 由 N 个值构造
    template <typename... Atypes>
    explicit constexpr tuple(const Atypes&& ...);      // 由 N 个值构造
    // . . . 复制与移动操作 . . .
};


template <class... Types>
constexpr tuple<Types...> make_tuple(Types&&...);    // 由 N 个值构造 tuple
```

tuple 的实现用到了本书范围之外的一项特性：可变参数模板（variadic templates）。前面那些省略号（...）指的就是它。不过，我们可以像使用 pair 那样使用 tuple。例如：

```
auto t0 = make_tuple();                          // 没有元素
auto t1 = make_tuple(123.456);                   // 一个 double 类型的元素
auto t2 = make_tuple(123.456, 'a');              // 两个元素：double 和 char
auto t3 = make_tuple(12, 'a', string{"How?"});   // 三个元素：int、char 和 string
```

tuple 可以有很多元素，所以不能只用 first 和 second 来访问。这里改用函数 get：

```
auto d = get<0>(t1);          // double 元素
auto n = get<0>(t3);          // int 元素
auto c = get<1>(t3);          // char 元素
auto s = get<2>(t3);          // string 元素
```

get 的下标作为模板实参提供。从上面的例子可以看出，tuple 的下标从 0 开始。

tuple 主要用于泛型编程。

B.6.4　initializer_list

在 <initializer_list> 中，我们可以找到 initializer_list 的定义：

```
template<typename T>
class initializer_list {
public:
    initializer_list() noexcept;
    size_t size() const noexcept;              // 元素个数
    const T* begin() const noexcept;           // 第一个元素
    const T* end() const noexcept;             // 最后一个元素之后的位置
    // . . .
};
```

当编译器看到一个由类型 X 的元素构成的花括号初始化列表时，就会用该列表构造一个 initializer_list<X>（§14.2.1、§18.2）。可惜的是，initializer_list 不支持下标运算符（[ ]）。

B.6.5　资源管理指针

内置指针并不表明它是否表示对其所指对象的所有权。这会严重复杂化编程（§19.5）。资源管理指针 unique_ptr 和 shared_ptr 定义在 <memory> 中，用来处理这个问题：

- unique_ptr（§19.5.4）表示独占所有权；一个对象只能有一个 unique_ptr，当它的 unique_ptr 销毁时，该对象即被删除。
- shared_ptr 表示共享所有权；一个对象可以有多个 shared_ptr，当最后一个 shared_ptr 销毁时，该对象即被删除。

unique_ptr<p>（简化版）

unique_ptr up {};	默认构造函数：up 持有 nullptr。
unique_ptr up {p};	up 持有 p。
unique_ptr up {up2};	移动构造函数：up 持有 up2 的 p；up2 持有 nullptr。
up.~unique_ptr()	删除 up 持有的指针。
p=up.get()	p 是 up 持有的指针。
p=up.release()	p 是 up 持有的指针；此后 up 持有 nullptr。
up.reset(p)	删除 up 持有的指针；up 改为持有 p。
up=make_unique<X>(args)	up 持有 new X(args)（C++14）。

unique_ptr 支持通常的指针操作，例如 *、->、== 和 <。此外，还可以让 unique_ptr 使用与普通 delete 不同的删除动作。

shared_ptr<p>（简化）

`shared_ptr sp {};`	默认构造函数：sp 持有空指针（nullptr）。
`shared_ptr sp {p};`	sp 持有 p。
`shared_ptr sp {sp2};`	复制构造函数：sp 与 sp2 都持有 sp2 的 p。
`shared_ptr sp {move(sp2)};`	移动构造函数：sp 持有 sp2 的 p；sp2 持有空指针。
`sp.~shared_ptr()`	如果 sp 是持有该指针的最后一个 shared_ptr，就删除 sp 持有的指针。
`sp = sp2`	复制赋值：如果 sp 是引用其指针的最后一个 shared_ptr，就删除该指针；sp 与 sp2 都持有 sp2 的 p。
`sp = move(sp2)`	移动赋值：如果 sp 是引用其指针的最后一个 shared_ptr，就删除该指针；sp 持有 sp2 的 p；sp2 持有空指针。
`p=sp.get()`	p 是 sp 持有的指针。
`n=sp.use_count()`	有多少个 shared_ptr 引用 sp 持有的指针？
`sp.reset(p)`	如果 sp 是引用其指针的最后一个 shared_ptr，就删除该指针；sp 持有 p。
`sp=make_shared<X>(args)`	sp 持有 new<X>(args)。

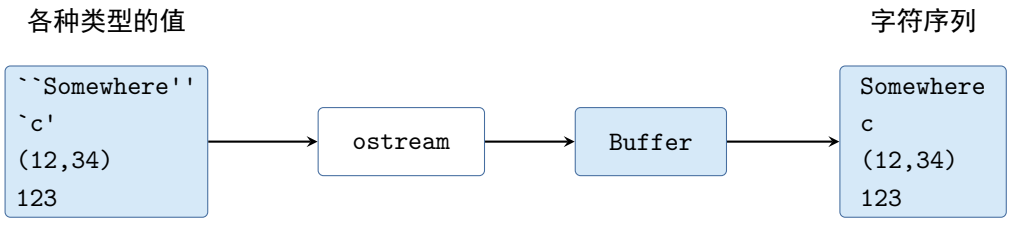
shared_ptr 支持通常的指针操作，例如 *、->、== 和 <。此外，还可以让 shared_ptr 使用与普通 delete 不同的删除动作。

此外还有 weak_ptr，用来打破 shared_ptr 之间的循环。

B.7　I/O 流

I/O 流库提供带缓冲的格式化与非格式化 I/O，可处理文本和数值。I/O 流设施的定义位于 <istream>、<ostream> 等头文件中；参见 §B.1.1。

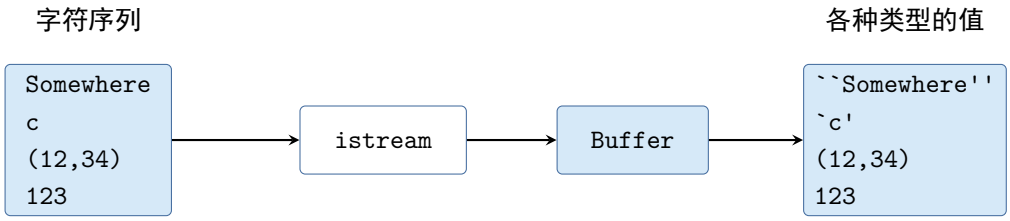
ostream 把各种有类型的对象转换成字符（字节）流：



istream 把字符（字节）流转换成各种有类型的对象：

iostream 是这样一种流：它既能充当 istream，又能充当 ostream。上面两幅图中的缓冲区就是“流缓冲区”（stream buffer，即 streambuf）。如果你需要定义从 iostream 到某种新设备、文件或内存的映射，请到专家级教材中查阅它们。

有三个标准流：

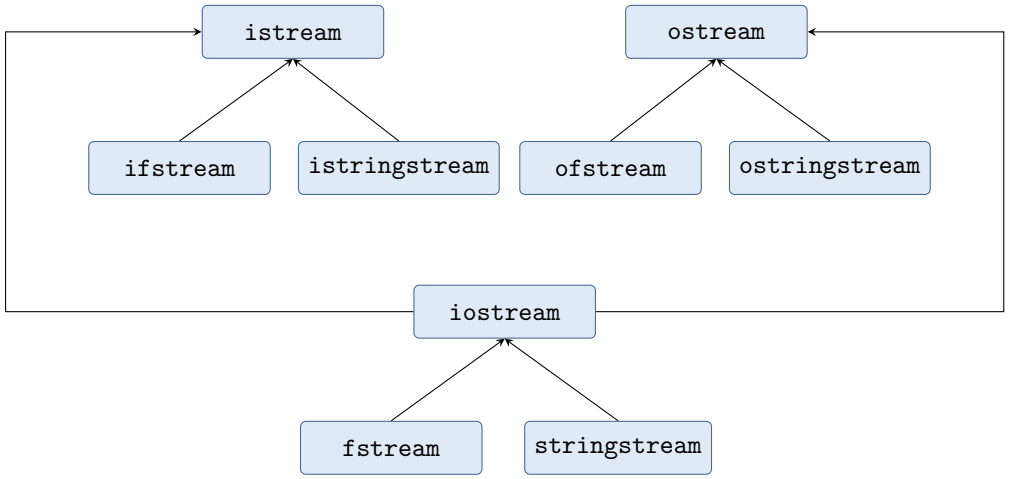


标准 I/O 流

`cout`	标准字符输出（默认通常是屏幕）
`cin`	标准字符输入（默认通常是键盘）
`cerr`	标准字符错误输出（无缓冲）

B.7.1　I/O 流类层次结构

`istream` 可以连接到输入设备（例如键盘）、文件或字符串。类似地，`ostream` 可以连接到输出设备（例如文本窗口）、文件或字符串。I/O 流设施被组织成一个类层次结构：



流既可以通过构造函数打开，也可以调用 `open()` 打开：

流类型

`stringstream(m)`	创建一个空的字符串流，模式为 m。
`stringstream(s,m)`	创建一个含字符串 s 的字符串流，模式为 m。
`fstream()`	创建一个文件流，留待以后再打开。
`fstream(s,m)`	以模式 m 打开名为 s 的文件，并创建一个引用它的文件流。
`fs.open(s,m)`	以模式 m 打开名为 s 的文件，并让 fs 引用它。
`fs.is_open()`	fs 是否已打开？

对文件流来说，文件名是 C 风格字符串。

可以用下面的几种模式之一来打开文件：

流模式

`ios_base::app`	追加（即添加到文件末尾）
`ios_base::ate`	“位于末尾”（打开并定位到末尾）
`ios_base::binary`	二进制模式——小心与具体系统相关的行为
`ios_base::in`	用于读取
`ios_base::out`	用于写入
`ios_base::trunc`	把文件截断为 0 长度

在每种情况下，打开文件的确切效果都可能取决于操作系统；如果操作系统无法满足以某种方式打开文件的请求，得到的流就不会处于 `good()` 状态。

一个例子：

```
void my_code(ostream& os);                  // 我的代码可以使用任何 ostream
ostringstream os;                           // o 代表“输出”
ofstream of("my_file");
if (!of) error("couldn't open 'my_file' for writing");
my_code(os);                                // 使用字符串流
my_code(of);                                // 使用文件流
```

参见 §11.3。

B.7.2　错误处理

`iostream` 可以处于四种状态之一：

流状态

`good()`	操作成功。
`eof()`	遇到了输入末尾（“文件末尾”）。
`fail()`	发生了预料之外的事（例如我们要找数字，却遇到了 `'x'`）。
`bad()`	发生了预料之外的严重问题（例如磁盘读取错误）。

通过 `s.exceptions()`，程序员可以要求：当 `iostream` 从 `good()` 转入其他状态时抛出异常（参见 §10.6）。

对不处于 `good()` 状态的流尝试任何操作都不会有任何效果，这就是所谓的“空操作”（no op）。

`iostream` 可以用作条件。这时，如果 `iostream` 的状态是 `good()`，条件就为真（即成功）。下面这个读取一串值的常用惯用法就以此为基础：

```
for (X buf; cin>>buf; ) {  // buf 是“输入缓冲区”，用来存放一个类型为 X 的值
    // . . . 对 buf 做点什么 . . .
}
// 当 >> 无法再从 cin 读入一个 X 时，我们就执行到这里
```

B.7.3　输入操作

输入操作位于 <istream> 中；只有读入字符串的那些操作例外，它们在 <string> 中：

格式化输入

`in >> x`	按 x 的类型从 in 读到 x 中。
`getline(in,s)`	从 in 读一行到字符串 s 中。

除非另有说明，istream 操作都返回其 istream 的引用，这样我们就能把操作“串接”起来，例如 `cin>>x>>y;`。

非格式化输入

`x=in.get()`	从 in 读一个字符，并返回它的整数值。
`in.get(c)`	从 in 读一个字符到 c 中。
`in.get(p,n)`	从 in 至多读 n 个字符，放到从 p 开始的数组中。
`in.get(p,n,t)`	从 in 至多读 n 个字符，放到从 p 开始的数组中；把 t 看作终止符。
`in.getline(p,n)`	从 in 至多读 n 个字符，放到从 p 开始的数组中；并把终止符从 in 中删除。
`in.getline(p,n,t)`	从 in 至多读 n 个字符，放到从 p 开始的数组中；把 t 看作终止符，并把终止符从 in 中删除。
`in.read(p,n)`	从 in 至多读 n 个字符，放到从 p 开始的数组中。
`x=in.gcount()`	x 是在 in 上最近一次非格式化输入操作所读的字符数。
`in.unget()`	回退流，使下一次读到的字符与上一次读到的相同。
`in.putback(x)`	把 x “放回”流中，使它成为下一次读到的字符。

`get()` 和 `getline()` 会在写入 `p[0]` 及后续位置的字符（如果有）末尾放一个 0；`getline()` 会把终止符（t）从输入中删除（如果找到了的话），而 `get()` 不会。`read(p,n)` 不会在读到的字符之后向数组写入 0。显然，格式化输入运算符比非格式化的那些更好用，也更不容易出错。

B.7.4　输出操作

输出操作位于 <ostream> 中；只有写出字符串的那些操作例外，它们在 <string> 中：

输出操作

`out << x`	按 x 的类型把 x 写到 out。
`out.put(c)`	把字符 c 写到 out。
`out.write(p,n)`	把字符 p[0]..p[n-1] 写到 out。

除非另有说明，ostream 操作都返回其 ostream 的引用，这样我们就能把操作“串接”起来，例如 `cout << x<<y;`。

B.7.5　格式化

流 I/O 的格式由对象类型、流状态、区域（locale）信息（参见 <locale>）以及显式操作共同决定。第 10 章和第 11 章解释了其中的大部分内容。这里我们只列出标准操纵符（修改流状态的操作），因为它们提供了修改格式的最直接途径。

区域（locale）的内容超出了本书的范围。

B.7.6　标准操纵符

标准库提供了与各种格式状态和状态变化相对应的操纵符。标准操纵符定义在 <ios>、<istream>、<ostream>、<iostream> 和 <iomanip>（用于带参数的操纵符）中：

I/O 操纵符

s<<boolalpha	用符号形式表示 true 和 false（输入输出均适用）。
s<<noboolalpha	s.unsetf(ios_base::boolalpha)。
s<<showbase	输出时给八进制加前缀 0，给十六进制加前缀 0x。
s<<noshowbase	s.unsetf(ios_base::showbase)。
s<<showpoint	总是显示小数点。
s<<noshowpoint	s.unsetf(ios_base::showpoint)。
s<<showpos	对正数显示 +。

I/O 操纵符（续）

s<<noshowpos	s.unsetf(ios_base::showpos)。
s>>skipws	跳过空白。
s>>noskipws	s.unsetf(ios_base::skipws)。
s<<uppercase	数值输出使用大写字母，例如 1.2E10 和 0X1A2，而不是 1.2e10 和 0x1a2。
s<<nouppercase	用 x 和 e，而不是 X 和 E。
s<<internal	在格式化模式中标记的位置填充。
s<<left	在值的后面填充。
s<<right	在值的前面填充。
s<<dec	整数基数为 10。
s<<hex	整数基数为 16。
s<<oct	整数基数为 8。
s<<fixed	浮点格式 dddd.dd。
s<<scientific	科学格式 d.ddddEdd。
s<<defaultfloat	采用能给出最精确浮点输出的那种格式。
s<<endl	放入 '\n' 并刷新。
s<<ends	放入 '\0'。
s<<flush	刷新流。
s>>ws	吃掉空白。
s<<resetiosflags(f)	清除标志 f。
s<<setiosflags(f)	设置标志 f。
s<<setbase(b)	以基数 b 输出整数。
s<<setfill(c)	让 c 成为填充字符。
s<<setprecision(n)	精度为 n 位数字。
s<<setw(n)	下一个字段宽度为 n 个字符。

上述每个操作都返回其第一个（流）操作数 s 的引用。例如：

```
cout << 1234 << ',' << hex << 1234 << ',' << oct << 1234 << endl;
```

输出为：

```
1234,4d2,2322
```

以及：

```
cout << '(' << setw(4) << setfill('#') << 12 << ") (" << 12 << ")\n";
```

输出为：

```
(##12) (12)
```

若要显式设置浮点数的一般输出格式，可使用

```
b.setf(ios_base::fmtflags(0), ios_base::floatfield)
```

参见第 11 章。

B.8　字符串操作

标准库在 <cctype> 中提供字符分类操作，在 <string> 中提供字符串及与之相关的操作，在 <regex> 中提供正则表达式匹配，并在 <cstring> 中提供对 C 风格字符串的支持。

B.8.1　字符分类

基本执行字符集中的字符可以这样分类：

字符分类

isspace(c)	是否为空白（空格、制表符、换行等）?
isalpha(c)	是否为字母（'a'..'z'、'A'..'Z'）?（注意：不含下划线。）
isdigit(c)	是否为十进制数字（'0'..'9'）?
isxdigit(c)	是否为十六进制数字（十进制数字，或 'a'..'f'，或 'A'..'F'）?
isupper(c)	是否为大写字母?
islower(c)	是否为小写字母?
isalnum(c)	是否为字母或十进制数字?
iscntrl(c)	是否为控制字符（ASCII 0..31 与 127）?
ispunct(c)	是否既不是字母、数字、空白，也不是不可见控制字符?
isprint(c)	是否可打印（ASCII 空格到 ~）?
isgraph(c)	是否为 isalpha(c) 或 isdigit(c) 或 ispunct(c)?（注意：不含空格。）

此外，标准库还提供两个有用的函数来消除大小写差异：

大写与小写

toupper(c)	c 或其大写等价字符
tolower(c)	c 或其小写等价字符

扩展字符集（例如 Unicode）也受支持，但超出了本书的范围。

B.8.2　字符串

标准库的字符串类 string 是通用字符串模板 basic_string 针对字符类型 char 的特化；也就是说，string 是一个 char 序列：

字符串操作

s=s2	把 s2 赋给 s；s2 可以是 string 或 C 风格字符串。
s+=x	在 s 末尾追加 x；x 可以是字符、string 或 C 风格字符串。
s[i]	下标。
s+s2	连接；结果是一个新字符串，先为 s 中的字符，后为 s2 中的字符。
s==s2	比较字符串值；s 或 s2（不能同时）可以是 C 风格字符串。
s!=s2	比较字符串值；s 或 s2（不能同时）可以是 C 风格字符串。
s<s2	按字典序比较字符串值；s 或 s2（不能同时）可以是 C 风格字符串。
s<=s2	按字典序比较字符串值；s 或 s2（不能同时）可以是 C 风格字符串。
s>s2	按字典序比较字符串值；s 或 s2（不能同时）可以是 C 风格字符串。
s>=s2	按字典序比较字符串值；s 或 s2（不能同时）可以是 C 风格字符串。
s.size()	s 中的字符数。
s.length()	s 中的字符数。
s.c_str()	s 中字符的 C 风格字符串版本（以零结尾）。
s.begin()	指向首字符的迭代器。
s.end()	指向 s 末尾之后一个位置的迭代器。
s.insert(pos,x)	在 s[pos] 之前插入 x；x 可以是 string 或 C 风格字符串。
s.append(x)	在 s 最后一个字符之后插入 x；x 可以是 string 或 C 风格字符串。
s.erase(pos)	从 s[pos] 开始删除 s 的尾部字符。s 的大小变为 pos。
s.erase(pos,n)	从 s[pos] 开始删除 n 个字符。s 的大小变为 $\max(pos, size - n)$。
s.push_back(c)	追加字符 c。
pos=s.find(x)	在 s 中查找 x；x 可以是字符、string 或 C 风格字符串；pos 是找到的第一个字符的下标，或 string::npos（s 末尾之外的位置）。
in>>s	从 in 读入一个单词到 s。

B.8.3　正则表达式匹配

正则表达式设施位于 <regex> 中。主要的函数有：

- 在（任意长的）数据流中搜索与某个正则表达式匹配的字符串——由 regex_search() 支持；

- 让某个正则表达式与一个（已知大小的）字符串匹配——由 regex_match() 支持；
- 替换匹配项——由 regex_replace() 支持；本书不介绍它，参见专家级教材或手册。

regex_search() 或 regex_match() 的结果是一组匹配项，通常用 smatch 表示：

```
regex row("^[\\w ]+(          \\d+)(  \\d+)( \\d+)$");   // 数据行
while (getline(in,line)) {
    // 检查数据行
    smatch matches;
    if (!regex_match(line, matches, row))
        error("bad line", lineno);
    // 检查该行:
    int field1 = from_string<int>(matches[1]);
    int field2 = from_string<int>(matches[2]);
    int field3 = from_string<int>(matches[3]);
    // . . .
}
```

正则表达式的语法建立在一批具有特殊含义的字符之上（第 23 章）：

正则表达式特殊字符

.	任意单个字符（“通配符”）
[	字符类
{	计数
(	开始分组
)	结束分组
\	下一个字符具有特殊含义
*	零次或多次
+	一次或多次
?	可选（零次或一次）
|	择一（或）
^	行首；取反
$	行尾

重复

{ n }	恰好 n 次
{ n, }	n 次或更多
{n,m}	至少 n 次、至多 m 次
*	零次或多次，即 {0,}
+	一次或多次，即 {1,}
?	可选（零次或一次），即 {0,1}

字符类

alnum	任意字母数字字符或下划线
alpha	任意字母字符
blank	任意不是行分隔符的空白字符
cntrl	任意控制字符
d	任意十进制数字
digit	任意十进制数字
graph	任意图形字符
lower	任意小写字符
print	任意可打印字符
punct	任意标点字符
s	任意空白字符
space	任意空白字符
upper	任意大写字符
w	任意单词字符（字母数字字符）
xdigit	任意十六进制数字字符

若干字符类还支持简写记法：

字符类缩写

\d	一个十进制数字	[[:digit:]]
\l	一个小写字符	[[:lower:]]
\s	一个空格（空格、制表符等）	[[:space:]]
\u	一个大写字符	[[:upper:]]
\w	一个字母、十进制数字或下划线	[[:alnum:]]
\D	非 \d	[^[:digit:]]
\L	非 \l	[^[:lower:]]
\S	非 \s	[^[:space:]]
\U	非 \u	[^[:upper:]]
\W	非 \w	[^[:alnum:]]

B.9　数值

C++ 标准库为数学（科学、工程等）计算提供了最基本的构件。

B.9.1　数值限制

每个 C++ 实现都规定了内置类型的各种性质，于是程序员可以用这些性质检查限制值、设置哨兵值等。

从 `<limits>` 中，我们可以为每个内置类型或库类型 T 得到 `numeric_limits<T>`。此外，程序员可以为用户定义的数值类型 X 定义 `numeric_limits<X>`。例如：

```
class numeric_limits<float> {
public:
    static const bool is_specialized = true;
    static constexpr int radix = 2;          // 指数的底（本例中为二进制）
    static constexpr int digits = 24;        // 尾数中的 radix 位数
    static constexpr int digits10 = 6;       // 尾数中的十进制位数
    static constexpr bool is_signed = true;
    static constexpr bool is_integer = false;
    static constexpr bool is_exact = false;
    static constexpr float min() { return 1.17549435E-38F; }      // 示例值
    static constexpr float max() { return 3.40282347E+38F; }      // 示例值
    static constexpr float lowest() { return -3.40282347E+38F; }  // 示例值
    static constexpr float epsilon() { return 1.19209290E-07F; }  // 示例值
    static constexpr float round_error() { return 0.5F; }         // 示例值
    static constexpr float infinity() { return /* some value */; }
    static constexpr float quiet_NaN() { return /* some value */; }
    static constexpr float signaling_NaN() { return /* some value */; }
    static constexpr float denorm_min() { return min(); }
    static constexpr int min_exponent = -125;               // 示例值
    static constexpr int min_exponent10 = -37;              // 示例值
    static constexpr int max_exponent = +128;               // 示例值
    static constexpr int max_exponent10 = +38;              // 示例值
    static constexpr bool has_infinity = true;
    static constexpr bool has_quiet_NaN = true;
    static constexpr bool has_signaling_NaN = true;
    static constexpr float_denorm_style has_denorm = denorm_absent;
    static constexpr bool has_denorm_loss = false;
    static constexpr bool is_iec559 = true;                 // 符合 IEC-559
    static constexpr bool is_bounded = true;
    static constexpr bool is_modulo = false;
    static constexpr bool traps = true;
    static constexpr bool tinyness_before = true;
    static constexpr float_round_style round_style = round_to_nearest;
};
```

从 <limits.h> 和 <float.h> 中，我们可以得到一批指定整数与浮点数关键性质的宏，其中包括：

限制宏

CHAR_BIT	一个 char 的位数（通常为 8）
CHAR_MIN	char 的最小值
CHAR_MAX	char 的最大值（通常，char 有符号时为 127，无符号时为 255）

B.9.2　标准数学函数

标准库提供了最常见的数学函数（定义在 <cmath> 和 <complex> 中）：

标准数学函数

abs(x)	绝对值
ceil(x)	不小于 x 的最小整数
floor(x)	不大于 x 的最大整数
round(x)	舍入到最接近的整数（.5 远离零舍入）
sqrt(x)	平方根；x 必须非负
cos(x)	余弦
sin(x)	正弦
tan(x)	正切
acos(x)	反余弦；结果非负
asin(x)	反正弦；返回最接近 0 的结果
atan(x)	反正切
sinh(x)	双曲正弦
cosh(x)	双曲余弦
tanh(x)	双曲正切
exp(x)	以 e 为底的指数
log(x)	自然对数，以 e 为底；x 必须为正
log10(x)	以 10 为底的对数

这些函数都有接受 float、double、long double 和 complex 实参的版本。对每个函数，返回类型与实参类型相同。

如果某个标准数学函数无法产生数学上有效的结果，它就设置变量 errno。

B.9.3　复数

标准库提供了复数类型 complex<float>、complex<double> 和 complex<long double>。若 complex<Scalar> 中的 Scalar 是其他支持常规算术运算的类型，通常也能工作，但不保证可移植。

```
template<class Scalar> class complex {
    // 复数是一对标量值，本质上是一对坐标
    Scalar re, im;
public:
    constexpr complex(const Scalar & r, const Scalar & i) :re{r}, im{i} { }
    constexpr complex(const Scalar & r) :re{r}, im(Scalar{}} { }
    constexpr complex() :re{Scalar{}}, im{Scalar{}} { }
    Scalar real() { return re; }       // 实部
    Scalar imag() { return im; }       // 虚部
    // 运算符: =  +=  -=  *=  /=
};
```

除 complex 的成员之外，<complex> 还提供大量有用的操作：

复数运算符

z1+z2	加法
z1-z2	减法
z1*z2	乘法
z1/z2	除法
z1==z2	相等
z1!=z2	不等
norm(z)	abs(z) 的平方
conj(z)	共轭：若 z 为 {re,im}，则 conj(z) 为 {re,-im}
polar(x,y)	由极坐标 (rho,theta) 构造一个复数
real(z)	实部
imag(z)	虚部
abs(z)	也称 rho
arg(z)	也称 theta
out << z	复数输出
in >> z	复数输入

标准数学函数（见 §B.9.2）同样适用于复数。注意：complex 不提供 < 或 %；另见 §24.9。

B.9.4　valarray

标准库的 valarray 是一维数值数组；也就是说，它为一个数组类型提供算术运算（很像第 24 章的 Matrix），并支持切片与步长。

B.9.5　推广的数值算法

来自 <numeric> 的这些算法为数值序列上的常见操作提供了通用版本：

数值算法

x = accumulate(b,e,i)	x 为 i 与 [b:e) 中各元素之和。
x = accumulate(b,e,i,f)	累加，但用 f 代替 +。
x = inner_product(b,e,b2,i)	x 为 [b:e) 与 [b2:b2+(e-b)) 的内积，即 i 与所有 (*p1)*(*p2) 之和，其中 p1 取遍 [b:e)，p2 是 [b2:b2+(e-b)) 中的对应元素。
x = inner_product(b,e,b2,i,f,f2)	内积，但分别用 f 和 f2 代替 + 与 *。
p=partial_sum(b,e,out)	[out:p) 的第 i 个元素是 [b:e) 中第 0..i 个元素之和。
p=partial_sum(b,e,out,f)	部分和，用 f 代替 +。
p=adjacent_difference(b,e,out)	对 i>0，[out:p) 的第 i 个元素为 *(b+i)-*(b+i-1)；若 e-b>0，则 *out 为 *b。
p=adjacent_difference(b,e,out,f)	相邻差，用 f 代替 -。
iota(b,e,v)	对 [b:e) 的每个元素赋 ++v。

例如：

```
vector<int> v(100);
iota(v.begin(),v.end(),0);          // v=={1, 2,3,4,5 . . . 100}
```

B.9.6　随机数

在 <random> 中，标准库提供随机数引擎和分布（§24.7）。默认情况下应使用 default_random_engine，它因适用范围广、开销低而被选用。

分布包括：

分布	
`uniform_int_distribution<int>`	
allowbreak	
texttt{low, high}	值落在 $[low{:}high]$ 内
`uniform_real_distribution<int>`	
allowbreak	
texttt{low,high}	值落在 $[low{:}high)$ 内
`exponential_distribution<double>`	
allowbreak	
texttt{lambda}	值落在 $[0{:}\infty)$ 内
`bernoulli_distribution{p}`	值为 `true` 或 `false`
`normal_distribution<double>`	
allowbreak	
texttt{median,spread}	值落在 $(-\infty{:}\infty)$ 内

分布可以把一个引擎作为参数来调用。例如：

```
uniform_real_distribution<> dist;
default_random_engine engn;
for (int i = 0; i<10; ++i)
    cout << dist(engn) << ' ';
```

B.10　时间

在 `<chrono>` 中，标准库提供了计时设施。时钟以时钟周期（clock tick）数计时，并通过调用 `now()` 的结果报告当前时间点。定义了三种时钟：

- `system_clock`：默认的系统时钟。
- `steady_clock`：一种时钟 c，对 now() 的连续调用满足 `c.now()` ≤ `c.now()`，且相邻时钟周期之间的时间间隔恒定。
- `high_resolution_clock`：系统上可用的分辨率最高的时钟。

给定时钟的时钟周期数由函数 `duration_cast<>()` 换算成常规的时间单位，例如秒、毫秒和纳秒。例如：

```
auto t = steady_clock::now();
// ……做点什么……
auto d = steady_clock::now()-t;          // 某件事花了 d 个时间单位
cout << "something took "
    << duration_cast<milliseconds>(d).count() << "ms";
```

这会以毫秒为单位打印出那件“某件事”所花的时间。另见 §26.6.1。

B.11　C 标准库函数

C 语言的标准库只做了很小的修改就并入了 C++ 标准库。C 标准库提供了相当多的函数，这些函数多年来在各种不同场合中都证明是有用的——尤其是对于相对底层的编程。这里我们把它们归入几个常规类别：

- C 风格 I/O
- C 风格字符串
- 内存
- 日期与时间
- 其他

C 标准库函数比我们这里介绍的要多；如果需要了解更多，请参阅一本好的 C 语言教材，例如 Kernighan 和 Ritchie 的《The C Programming Language》（K&R）。

B.11.1　文件

`<stdio>` 的 I/O 系统建立在“文件”之上。一个文件（`FILE*`）既可以指一个文件，也可以指标准输入输出流之一：`stdin`、`stdout` 和 `stderr`。标准流默认可用；其他文件则需要打开：

文件打开与关闭

`f=fopen(s,m)`	为名为 s 的文件以模式 m 打开一个文件流。
`x=fclose(f)`	关闭文件流 f；成功时返回 0。

所谓“模式”是一个字符串，其中含有一条或多条指令，说明文件应当如何打开：

文件模式

`"r"`	读取
`"w"`	写入（丢弃原有内容）
`"a"`	追加（加到末尾）
`"r+"`	读取和写入
`"w+"`	读取和写入（丢弃原有内容）
`"b"`	二进制；须与其他一个或多个模式合用

在特定系统上可能（而且通常确实）还有更多选项。有些选项可以组合；例如 `fopen("foo","rb")` 试图以二进制读取方式打开名为 foo 的文件。对 stdio 和 iostream（§B.7.1）而言，I/O 模式应当是一致的。

B.11.2　printf() 家族

最受欢迎的 C 标准库函数是 I/O 函数。不过，我们推荐 iostream，因为那个库是类型安全的，而且可扩展。格式化输出函数 `printf()` 得到了广泛使用（在 C++ 程序中也是如此），并被其他编程语言广泛模仿：

printf

`n=printf(fmt,args)`	把“格式串”fmt 打印到 stdout，并按需要插入参数 args。
`n=fprintf(f,fmt,args)`	把“格式串”fmt 打印到文件 f，并按需要插入参数 args。
`n=sprintf(s,fmt,args)`	把“格式串”fmt 打印到 C 风格字符串 s，并按需要插入参数 args。

对每个版本，n 是写出的字符数；如果输出失败，则是一个负数。`printf()` 的返回值几乎总是被忽略。

`printf()` 的声明是

```
int printf(const char* format . . .);
```

换句话说，它取一个 C 风格字符串（通常是字符串字面量），后面跟任意数量、任意类型的参数。这些“额外参数”的含义由格式串中的转换说明控制，例如%c（按字符打印）和%d（按十进制整数打印）。例如：

```
int x = 5;
const char* p = "asdf";
printf("the value of x is '%d' and the value of p is '%s'\n",x,p);
```

跟在% 后面的字符控制着一个参数的处理方式。第一个% 作用于第一个“额外参数”（这里%d 作用于 x），第二个% 作用于第二个“额外参数”（这里%s 作用于 p），依此类推。具体来说，那次 `printf()` 调用的输出是

```
the value of x is '5' and the value of p is 'asdf'
```

后面跟一个换行符。

一般来说，% 转换指令与它所应用的类型之间的对应关系是无法检查的；即使能够检查，通常也没有检查。例如：

```
printf("the value of x is '%s' and the value of p is '%d'\n",x,p);  // 糟糕
```

转换说明的集合相当大，提供了很大程度的灵活性（也带来了混淆的可能）。在% 之后，可以有：

`-` 可选的负号，指定转换后的值在字段内左对齐。

`+` 可选的正号，指定有符号类型的值总是以 + 或 - 号开头。

`0` 可选的零，指定用前导零填充数值。如果指定了 - 或精度，则忽略这个 0。

`#` 可选的 #，指定浮点值即使后面没有非零数字也要打印小数点、要打印末尾的零、八进制值以开头的 0 打印、十六进制值以开头的 0x 或 0X 打印。

字段宽度 可选的数字串，指定字段宽度；如果转换后的值字符数少于字段宽度，则在左边用空白填充（若给了左对齐标志，则在右边填充）以凑足字段宽度；如果字段宽度以零开头，则改用零填充，而不是空白填充。

. 可选的小数点，用来把字段宽度与后面的数字串分开。

精度 可选的数字串，指定精度：对 e 和 f 转换，它指定小数点后出现的数字位数；对字符串，它指定最多打印的字符数。

* 字段宽度或精度可以用 * 代替数字串。此时由一个整数参数提供字段宽度或精度。

h 可选字符 h，指定后面的 d、o、x 或 u 对应 short 整数参数。

l 可选字符 l（字母 l），指定后面的 d、o、x 或 u 对应 long 整数参数。

L 可选字符 L，指定后面的 e、E、g、G 或 f 对应 long double 参数。

% 表示要打印字符% 本身；不使用参数。

c 一个字符，指定要应用的转换类型。这些转换字符及其含义是：

d 整数参数转换为十进制记法。

i 整数参数转换为十进制记法。

o 整数参数转换为八进制记法。

x 整数参数转换为十六进制记法。

X 整数参数转换为十六进制记法。

f 浮点或双精度参数按 [-]ddd.ddd 的形式转换为十进制记法。小数点后 d 的个数等于该参数的精度。必要时对数字进行舍入。若省略精度，则给出六位数字；若精度显式为 0 且未指定 #，则不打印小数点。

e 浮点或双精度参数按科学记法 [-]d.ddde+dd 或 [-]d.ddde-dd 转换，其中小数点前有一位数字，小数点后的位数等于该参数的精度说明。必要时对数字进行舍入。若省略精度，则给出六位数字；若精度显式为 0 且未指定 #，则不打印数字，也不打印小数点。

E 与 e 相同，但用大写 E 标识指数。

g 浮点或双精度参数按 d 风格、f 风格或 e 风格打印，取其中能以最小空间给出最高精度的那个。

G 与 g 相同，但用大写 E 标识指数。

c 打印字符参数。空字符被忽略。

s 参数被当作字符串（字符指针），打印其中的字符，直到遇到空字符，或达到精度说明所指定的字符数为止；不过，若精度为 0 或省略，则打印到空字符为止的所有字符。

p 参数被当作指针。打印出的表示依赖于实现。

u 无符号整数参数转换为十进制记法。

n 到目前为止 `printf()`、`fprintf()` 或 `sprintf()` 调用所写出的字符数，被写入那个指向 int 的指针参数所指向的 int。

字段宽度不存在或过小，绝不会导致字段被截断；只有指定的字段宽度超过实际宽度时才进行填充。

由于 C 没有 C++ 意义上的用户定义类型，因此也就没有为用户定义类型（例如 `complex`、`vector` 或 `string`）定义输出格式的机制。

C 的标准输出 `stdout` 对应 `cout`。C 的标准输入 `stdin` 对应 `cin`。C 的标准错误输出 `stderr` 对应 `cerr`。C 标准 I/O 与 C++ I/O 流之间的这种对应关系如此紧密，以至于 C 风格 I/O 与 I/O 流可以共用一个缓冲区。例如，可以混用 `cout` 和 `stdout` 操作来产生单个输出流（在 C 与 C++ 混合的代码中这并

不罕见）。这种灵活性是有代价的。为了获得更好的性能，不要对同一个流混用 stdio 和 iostream 操作，并在第一次 I/O 操作之前调用 `ios_base::sync_with_stdio(false)`。

stdio 库提供了一个函数 `scanf()`，它是一种模仿 `printf()` 风格的输入操作。例如：

```
int x;
char s[buf_size];
int i = scanf("the value of x is '%d' and the value of s is '%s'\n",&x,s);
```

这里，`scanf()` 试图把一个整数读入 x，并把一串非空白字符读入 s。非格式字符则要求输入中应当含有该字符。例如，

```
the value of x is '123' and the value of s is 'string '
```

将把 123 读入 x，并把 string 后跟一个 0 读入 s。如果 `scanf()` 调用成功，结果值（上面调用中的 i）就是被赋值的参数指针个数（在例子中希望是 2）；否则为 EOF。这种指定输入的方式容易出错（例如，如果在那行输入里忘了 string 后面的空格，会怎样？）。`scanf()` 的所有参数都必须是指针。我们强烈建议不要使用 `scanf()`。

那么，如果不得不用 stdio 来做输入，我们该怎么办？一个流行的答案是："使用标准库函数 gets()"：

```
// 非常危险的代码:
char s[buf_size];
char* p = gets(s);              // 把一行读入 s
```

调用 `p=gets(s)` 会把字符读入 s，直到遇到换行符或文件结束，并在写入 s 的最后一个字符之后放一个 0 字符。如果遇到文件结束或发生了错误，p 被置为 NULL（即 0）；否则被置为 s。绝不要使用 `gets(s)` 或它的粗略等价物（`scanf("%s",s)`）！多年来，它们一直是病毒作者的最爱：通过提供一段使输入缓冲区（例子中的 s）溢出的输入，程序可能被破坏，计算机也可能被攻击者接管。`sprintf()` 函数也有类似的缓冲区溢出问题。

stdio 库还提供了一些简单实用的字符读写函数：

stdio 字符函数

`x=getc(st)`	从输入流 st 读一个字符；返回该字符的整数值；若遇文件结束或发生错误，则 x==EOF。
`x=putc(c,st)`	把字符 c 写到输出流 st；返回所写字符的整数值；若发生错误，则 x==EOF。
`x=getchar()`	从 stdin 读一个字符；返回该字符的整数值；若遇文件结束或发生错误，则 x==EOF。
`x=putchar(c)`	把字符 c 写到 stdout；返回所写字符的整数值；若发生错误，则 x==EOF。
`x=ungetc(c,st)`	把 c 放回输入流 st；返回被推回字符的整数值；若发生错误，则 x==EOF。

注意这些函数的结果是 int（而不是 char，否则就无法返回 EOF）。例如，下面是一个典型的 C 风格输入循环：

```
int ch; /* 不是 char ch; */
while ((ch=getchar())!=EOF) { /* 做点什么 */ }
```

不要对同一个流连续调用两次 ungetc()。那样做的结果未定义，因而（也就）不可移植。

stdio 函数还有很多；如果需要了解更多，请参阅一本好的 C 语言教材，例如 K&R。

B.11.3　C 风格字符串

C 风格字符串是以 0 结尾的 char 数组。这种字符串概念由一组函数支撑，这些函数定义在 <cstring>（或 <string.h>；注意：不是 <string>）以及 <cstdlib> 中。这些函数通过 char* 指针（对只读内存则用 const char* 指针）来操作 C 风格字符串：

C 风格字符串操作

x=strlen(s)	统计字符数（不包括结尾的 0）。
p=strcpy(s,s2)	把 s2 复制到 s；[s:s+n) 与 [s2:s2+n) 不得重叠；p=s；结尾的 0 也被复制。
p=strcat(s,s2)	把 s2 复制到 s 的末尾；p=s；结尾的 0 也被复制。
x=strcmp(s,s2)	按字典序比较：若 s < s2 则 x 为负；若 s==s2 则 x==0；若 s > s2 则 x 为正。

C 风格字符串操作（续）

p=strncpy(s,s2,n)	strcpy；最多 n 个字符；可能复制不到结尾的 0；p=s。
p=strncat(s,s2,n)	strcat；最多 n 个字符；可能复制不到结尾的 0；p=s。
x=strncmp(s,s2,n)	strcmp；最多 n 个字符。
p=strchr(s,c)	令 p 指向 s 中第一个 c。
p=strrchr(s,c)	令 p 指向 s 中最后一个 c。
p=strstr(s,s2)	令 p 指向 s 中第一个这样的字符：从它开始的子串等于 s2。
p=strpbrk(s,s2)	令 p 指向 s 中第一个也在 s2 中出现的字符。
x=atof(s)	从 s 中提取一个 double。
x=atoi(s)	从 s 中提取一个 int。
x=atol(s)	从 s 中提取一个 long int。
x=strtod(s,p)	从 s 中提取一个 double；令 p 指向该 double 之后的第一个字符。
x=strtol(s,p)	从 s 中提取一个 long int；令 p 指向该 long 之后的第一个字符。
x=strtoul(s,p)	从 s 中提取一个 unsigned long int；令 p 指向该 long 之后的第一个字符。

注意，在 C++ 中 strchr() 和 strstr() 都有重复的重载版本，以保证类型安全（它们不能像 C 中的对应函数那样把 const char* 转成 char*）；另见 §27.5。

提取函数会在其 C 风格字符串参数中寻找按常规格式表示的数字，例如 "124" 和 exttt" 1.4"。如果找不到这样的表示，提取函数返回 0。例如：

```
int x = atoi("fortytwo");            /* x 变为 0 */
```

B.11.4　内存

内存操作函数通过 void* 指针（对只读内存则用 const void* 指针）来操作“原始内存”（不涉及任何类型）：

C 风格内存操作

q=memcpy(p, p2, n)	从 p2 复制 n 个字节到 p（类似 strcpy）；[p:p+n) 与 [p2:p2+n) 不得重叠；q=p。
q=memmove(p,p2,n)	从 p2 复制 n 个字节到 p；q=p。
x=memcmp(p,p2,n)	把 p2 开始的 n 个字节与 p 开始的对应 n 个字节相比较（类似 strcmp）。

C 风格内存操作（续）

q=memchr(p,c,n)	在 p[0]..p[n-1] 中查找 c（转换为 unsigned char），并令 q 指向该元素；若找不到 c，则 q=0。
q=memset(p,c,n)	把 c（转换为 unsigned char）复制到 p[0]..[n-1] 的每个元素中；q=p。
p=calloc(n,s)	在自由存储上分配 n*s 个字节并初始化为 0；若 n*s 个字节无法分配，则 p=0。
p=malloc(s)	在自由存储上分配 s 个未初始化的字节；若 s 个字节无法分配，则 p=0。
q=realloc(p,s)	在自由存储上分配 s 个字节；p 必须是由 malloc() 或 calloc() 返回的指针；若可能，重用 p 所指的空间；若不可能，把 p 所指区域中的所有字节复制到新区域；若 s 个字节无法分配，则 q=0。
free(p)	释放 p 所指的内存；p 必须是由 malloc()、calloc() 或 realloc() 返回的指针。

注意 malloc() 等函数不调用构造函数，free() 也不调用析构函数。不要对有构造函数或析构函数的类型使用这些函数。另外，memset() 绝不应当用于任何有构造函数的类型。

mem* 系列函数在 <cstring> 中，分配函数在 <cstdlib> 中。

另见 §27.5.2。

B.11.5　日期与时间

在 <ctime> 中，你可以找到若干与日期和时间相关的类型和函数：

日期与时间类型

clock_t	一种算术类型，用于保存较短的时间间隔（也许只有几分钟的间隔）
time_t	一种算术类型，用于保存较长的时间间隔（也许长达数百年）
tm	一个结构体，用于保存日期和时间（从 1900 年起算）

struct tm 是这样定义的：

```
struct tm {
    int tm_sec;     // 一分钟内的秒 [0:61]; 60 和 61 表示闰秒
    int tm_min;     // 一小时内的分 [0,59]
    int tm_hour;    // 一天内的小时 [0,23]
    int tm_mday;    // 一月内的日 [1,31]
    int tm_mon;     // 一年内的月 [0,11]; 0 表示一月 (注意: 不是 [1:12])
    int tm_year;    // 从 1900 年起的年数; 0 表示 1900 年, 102 表示 2002 年
    int tm_wday;    // 从星期日起的天数 [0,6]; 0 表示星期日
    int tm_yday;    // 从 1 月 1 日起的天数 [0,365]; 0 表示 1 月 1 日
    int tm_isdst;   // 夏令时的小时数
};
```

日期和时间函数：

```
clock_t clock();                          // 从程序开始以来的时钟周期数
time_t time(time_t* pt);                  // 当前日历时间
double difftime(time_t t2, time_t t1);    // t2-t1，单位为秒
tm* localtime(const time_t* pt);          // *pt 的本地时间 tm
tm* gmtime(const time_t* pt);             // *pt 的格林尼治标准时间（GMT）tm，或 0
time_t mktime(tm* ptm);                   // *ptm 对应的 time_t，或 time_t(-1)
char* asctime(const tm* ptm);             // *ptm 的 C 风格字符串表示
char* ctime(const time_t* t) { return asctime(localtime(t)); }
```

asctime() 调用结果的一个例子是 "Sun Sep 16 01:03:52 1973 extbackslash n"。

一个名为 strftime() 的函数为 tm 提供了一大堆令人惊叹的格式化选项。需要的话可以自己去查。

B.10.6　其他

在 <cstdlib> 中我们可以找到：

其他 stdlib 函数

abort()	使程序“异常”终止。
exit(n)	以值 n 终止程序；n==0 表示成功终止。
system(s)	把 C 风格字符串当作命令执行（依赖于系统）。
qsort(b,n,s,cmp)	对从 b 开始、有 n 个大小为 s 的元素的数组排序，使用比较函数 cmp。
bsearch(k,b,n,s,cmp)	在从 b 开始、有 n 个大小为 s 的元素的已排序数组中查找 k，使用比较函数 cmp。

qsort() 和 bsearch() 所使用的比较函数（cmp）必须具有类型

```
int (*cmp)(const void* p, const void* q);
```

也就是说，排序函数完全不知道类型信息，它只是把数组看成一串字节。返回的整数是

- 若 *p 被认为小于 *q，则为负
- 若 *p 被认为等于 *q，则为零
- 若 *p 被认为大于 *q，则为正

注意 exit() 和 abort() 不调用析构函数。如果你希望为已构造的自动对象和静态对象（§A.4.2）调用析构函数，就抛出异常。

要了解更多的标准库函数，请参阅 K&R 或其他可靠的 C 语言参考书。

B.12　其他库

翻遍标准库的各种设施，你无疑会发现找不到某个本来能用的东西。与程序员所面对的挑战以及世界上现有库的数量相比，C++ 标准库是微不足道的。有许多库可用于

- 图形用户界面（GUI）
- 高等数学
- 数据库访问

- 网络
- XML
- 日期与时间
- 文件系统操作
- 三维图形
- 动画
- 其他

然而，这类库并不是标准的一部分。你可以通过网络搜索，或者向朋友和同事打听来找到它们。请不要以为只有属于标准库的那些库才是有用的库。

附录 D　安装 FLTK

"If the code and the comments disagree,（如果代码与注释不一致，）
then both are probably wrong.（那么两者很可能都是错的。）"
——Norm Schryer

本附录介绍如何下载并安装 FLTK，以及如何在自己的代码中链接到这套图形与 GUI 工具包。

D.1　引言
D.2　下载 FLTK
D.3　安装 FLTK
D.4　在 Visual Studio 中使用 FLTK
D.5　测试是否一切正常

D.1　引言

我们选择 FLTK，也就是 Fast Light Tool Kit（读作"full tick"），作为讲解图形与 GUI 相关问题的基础，因为它可移植、相对简单、相对传统，安装起来也相对容易。我们用 Microsoft Visual Studio 来演示如何安装 FLTK，因为我们的学生大多用它，而且它也是最难装的一种。如果你用的是别的系统（我们的一些学生也是如此），只要到下载文件的主文件夹（目录）里去找适合你所用系统的说明即可（§D.3）。

每当你使用一个不属于 ISO C++ 标准的库时，你（或者别人）都必须下载它、安装它，并在自己的代码中正确地使用它。这件事很少能完全轻松愉快，而安装 FLTK 大概算一次很好的练习——因为即使是最好用的库，如果你以前没装过，下载和安装也可能相当令人沮丧。别太不情愿向有经验的人请教，但也别让他们替你包办：要从他们那里学。

请注意，实际的文件和步骤可能与我们这里写的略有出入。例如，FLTK 可能已经有了新版本，或者你用的 Visual Studio 版本与 §D.4 中描述的不同，又或者你用的是完全不同的 C++ 实现。

D.2　下载 FLTK

动手之前，先看看你的机器上是不是已经装好了 FLTK，见 §D.5。如果还没有，第一件事就是把文件弄到你的计算机上：

1. 访问 `http://fltk.org`。（万一网站访问不了，也可以从本书的支持网站下载一份副本：`www.stroustrup.com/Programming/FLTK`。）
2. 在导航菜单中点击 Download。
3. 在下拉列表中选择 FLTK 1.1.x，然后点击 Show Download Locations。
4. 选择一个下载位置，下载那个.zip 文件。

你得到的文件是.zip 格式。这是一种压缩格式，适合在网上传输大量文件。你的机器上需要一个程序把它"解压"成普通文件；在 Windows 上，WinZip 和 7-Zip 就是这类程序的例子。

D.3　安装 FLTK

照我们的说明做，你遇到的主要问题很可能是下面两种之一：要么是我们写完并测试过这些说明之后情况有了变化（这种事时有发生），要么是那些术语对你来说太陌生（这一点我们帮不上忙，抱歉）。如果是后一种情况，找个朋友给你翻译一下。

1. 解压下载的文件，打开主文件夹 `fltk-1.1.?`。在 Visual C++ 文件夹（例如 `vc2005` 或 `vcnet`）中，打开 `fltk.dsw`。如果程序询问是否更新旧的项目文件，选择 Yes to All。
2. 从 Build 菜单中选择 Build Solution。这可能需要几分钟。源代码正在被编译成静态链接库，这样你每次新建项目时都不必重新编译 FLTK 的源代码。过程结束后，关闭 Visual Studio。
3. 从 FLTK 主目录打开 lib 文件夹。把所有的.lib 文件（应该共有七个，`README.lib` 除外）复制（不是仅仅移动或拖拽）到 `C:\Program Files\Microsoft Visual Studio\Vc\lib`。
4. 回到 FLTK 主目录，把 FL 文件夹复制到 `C:\Program Files\Microsoft Visual Studio\Vc\include`。

专家会告诉你，安装还有比复制到 `C:\Program Files\Microsoft Visual Studio\Vc\lib` 和 `C:\Program Files\Microsoft Visual Studio\Vc\include` 更好的办法。他们说得对，但我们并不想把你培养成 VS 专家。如果专家坚持，那就让他们负责向你演示更好的替代方案。

D.4　在 Visual Studio 中使用 FLTK

1. 在 Visual Studio 中新建一个项目，只需对通常步骤做一处改动：选择项目类型时，创建“Win32 项目”而不是“控制台应用程序”。一定要创建“空项目”；否则某个“软件向导”会往你的项目里塞一大堆你很可能既用不上也看不懂的东西。
2. 在 Visual Studio 中，从主（顶部）菜单选择 Project，然后从下拉菜单中选择 Properties。
3. 在 Properties 对话框中，点击左侧菜单里的 Linker 文件夹。这会展开一个子菜单。在这个子菜单中点击 Input。在右侧的 Additional Dependencies 文本框中输入以下文本：

   ```
   fltkd.lib wsock32.lib comctl32.lib fltkjpegd.lib fltkimagesd.lib
   ```

   [接下来这一步可能并不需要，因为它现在已是默认设置。] 在 Ignore Specific Library 文本框中输入以下文本：

   ```
   libcd.lib
   ```
4. [这一步可能并不需要，因为 /MDd 现在已是默认设置。] 在同一个 Properties 窗口的左侧菜单中点击 C/C++，展开另一个子菜单。点击 Code Generation 子菜单项。在右侧菜单中，把 Runtime Library 下拉框改为 Multi-threaded Debug DLL (/MDd)。点击 OK 关闭 Properties 窗口。

D.5　测试是否一切正常

在你新建的项目中创建一个新的.cpp 文件，输入下面的代码。它应该能顺利编译。

```
#include <FL/Fl.h>
#include <FL/Fl_Box.h>
#include <FL/Fl_Window.h>

int main()
{
    Fl_Window window(200, 200, "Window title");
    Fl_Box box(0,0,200,200, "Hey, I mean, Hello, World!");
    window.show();
    return Fl::run();
}
```

如果没能成功：

- “编译器报告找不到某个.lib 文件”：问题最可能出在安装环节。注意第 3 步，它讲的是把链接库（.lib）文件放到编译器容易找到的地方。
- “编译器报告无法打开某个.h 文件”：问题最可能出在安装环节。注意第 4 步，它讲的是把头文件（.h）放到编译器容易找到的地方。
- “链接器报告涉及未解析外部符号的错误”：问题最可能出在项目设置环节。

如果这些都没能帮上忙，就找个朋友问问吧。

附录 E　图形用户界面实现

"When you finally understand what you are doing, things will go right.
（当你最终明白自己在做什么时，事情就会顺利起来。）"
——Bill Fairbank

本附录给出回调（callback）、Window、Widget 与 Vector_ref 的实现细节。在第 16 章，我们还不能假定你已具备更完整的解释所需的指针与类型转换知识，于是把那段解释放到了本附录。

E.1　回调的实现
E.2　Widget 的实现
E.3　Window 的实现
E.4　Vector_ref
E.5　一个例子：操纵 Widget

E.1　回调的实现

我们是这样实现回调的：

```
void Simple_window::cb_next(Address, Address addr)
    // 为位于 addr 的窗口调用 Simple_window::next()
{
    reference_to<Simple_window>(addr).next();
}
```

读过第 17 章之后，你就能明显看出：Address 必定是 void*；而 reference_to<Simple_window>(addr) 必定以某种方式由名为 addr 的 void* 创建出对 Simple_window 的引用。可是，如果你此前没有编程经验，那么在读第 17 章之前，这两点既不"明显"也不"当然"，所以我们来仔细看看地址的用法。

正如 §A.17 所述，C++ 提供了为类型取名的办法。例如：

```
typedef void* Address;    // Address 是 void* 的同义词
```

这意味着，现在可以用名字 Address 代替 void*。在这里，我们用 Address 强调传入的是一个地址，同时也为了掩盖这样一件事：void* 这个类型名，表示的是"指向某个我们并不知道其类型的对象的指针"。

因此，cb_next() 接收一个名为 addr 的 void* 作为实参，然后——以某种方式——立刻把它转换成 Simple_window&：

```
reference_to<Simple_window>(addr)
```

reference_to 是一个模板函数（§A.13）：

```
template<class W> W& reference_to(Address pw)
    // 把地址当作 W 的引用
{
    return *static_cast<W*>(pw);
}
```

这里，我们用一个模板函数为自己写出了一个操作，它相当于把 void* 转换成 Simple_window& 的类型转换。类型转换 static_cast 见 §17.8。

编译器无法验证我们“addr 指向一个 Simple_window”这一断言，但语言规则要求编译器在这里信任程序员。幸运的是，我们是对的。我们之所以知道自己是对的，是因为 FLTK 交还给我们的，正是我们当初交给它的那个指针。既然交出指针时我们知道它的类型，就可以用 reference_to 把它“取回来”。这种做法很脏、不做检查，在系统的底层并不算罕见。

一旦拿到了 Simple_window 的引用，我们就可以用它调用 Simple_window 的成员函数。例如（§16.3）：

```
void Simple_window::cb_next(Address, Address pw)
    // 为位于 pw 的窗口调用 Simple_window::next()
{
    reference_to<Simple_window>(pw).next();
}
```

我们用这个脏兮兮的回调函数 cb_next()，不过是为了按需调整类型，好去调用一个再普通不过的成员函数 next()。

E.2　Widget 的实现

我们的 Widget 接口类是这样的：

```
class Widget {
    // Widget 是 Fl_widget 的句柄——它本身并不是 Fl_widget
    // 我们尽量让自己的接口类与 FLTK 保持距离
public:
    Widget(Point xy, int w, int h, const string& s, Callback cb)
        :loc(xy), width(w), height(h), label(s), do_it(cb)
    { }
    virtual ~Widget() { }    // 析构函数
    virtual void move(int dx,int dy)
        { hide(); pw->position(loc.x+=dx, loc.y+=dy); show(); }
    virtual void hide() { pw->hide(); }
    virtual void show() { pw->show(); }
    virtual void attach(Window&) = 0;    // 每个 Widget 至少为窗口定义一个动作
    Point loc;
    int width;
    int height;
    string label;
```

```
    Callback do_it;
protected:
    Window* own;       // 每个 Widget 都属于某个 Window
    Fl_Widget* pw;     // Widget "知道" 自己的 Fl_Widget
};
```

注意，我们的 Widget 记录着自己的 FLTK 部件，以及与自己关联的 Window。注意，为此我们需要指针，因为一个 Widget 在其生命期内可能关联到不同的 Window。引用或具名对象都做不到这一点。（为什么做不到？）

它有一个位置（loc）、一个矩形外形（width 和 height）以及一个标签。真正有意思的是，它还有一个回调函数（do_it）——它把 Widget 在屏幕上的图像与我们的一段代码连接起来。这些操作（move()、show()、hide() 和 attach()）的含义应该很明显。

Widget 看上去有几分"半成品"的样子。它是作为实现类设计的，用户通常不该经常见到它。它是重新设计的好候选。我们对它那些公有数据成员心存疑虑，而"显然"的操作通常也需要重新审视，看看有没有意料之外的微妙之处。

Widget 含有虚函数，可以作为基类使用，因此它带有一个虚析构函数（§17.5.2）。

E.3　Window 的实现

什么时候用指针，什么时候改用引用？我们在 §8.5.6 讨论了这个一般性问题。这里我们只需指出：有些程序员喜欢指针；而当我们需要在程序的不同时刻指向不同的对象时，就必须用指针。

到目前为止，我们还没有展示图形与图形用户界面库中的一个核心类：Window。最重要的原因是它使用了指针，而且它基于 FLTK 的实现需要自由存储（free store）。在 Window.h 中可以找到它，如下所示：

```
class Window : public Fl_Window {
public:
    // 让系统选择位置:
    Window(int w, int h, const string& title);
    // 左上角位于 xy:
    Window(Point xy, int w, int h, const string& title);
    virtual ~Window() { }
    int x_max() const { return w; }
    int y_max() const { return h; }
    void resize(int ww, int hh) { w=ww, h=hh; size(ww,hh); }
    void set_label(const string& s) { label(s.c_str()); }
    void attach(Shape& s) { shapes.push_back(&s); }
    void attach(Widget&);
    void detach(Shape& s);              // 从 shapes 中移除 w
    void detach(Widget& w);             // 从窗口中移除 w
                                        // (会停用回调)
    void put_on_top(Shape& p);          // 把 p 放到其他形状之上
protected:
    void draw();
```

```
private:
    vector<Shape*> shapes;              // 附加到窗口的形状
    int w,h;                            // 窗口大小
    void init();
};
```

所以，当我们 attach() 一个 Shape 时，会把它的指针存入 shapes，好让 Window 能绘制它。由于以后还可能 detach() 掉那个形状，我们需要指针。基本上，被 attach() 的形状仍然由我们的代码拥有；我们只是给 Window 一个指向它的引用。Window::attach() 把实参转换成指针，以便存储。如上所示，attach() 很简单；detach() 稍微复杂一点。在 Window.cpp 中可以看到：

```
void Window::detach(Shape& s)
    // 假定最后附加的会最先释放
{
    for (vector<Shape*>::size_type i = shapes.size(); 0<i; --i)
        if (shapes[i-1]==&s)
            shapes.erase(shapes.begin()+(i-1));
}
```

erase() 成员函数从向量中移除（“擦除”）一个值，使向量的大小减一（§20.7.1）。

Window 就是当作基类使用的，因此它带有虚析构函数（§17.5.2）。

E.4　Vector_ref

基本上，Vector_ref 模拟一个存放引用的向量。你可以用引用或指针来初始化它：

- 如果对象以引用传给 Vector_ref，则认为它由调用者拥有，调用者负责它的生命期（例如该对象是一个作用域内的变量）。
- 如果对象以指针传给 Vector_ref，则认为它是由 new 分配的，Vector_ref 有责任 delete 它。

元素以指针的形式（而不是对象的副本）存入 Vector_ref，并具有引用语义。例如，你可以把一个 Circle 放进 Vector_ref<Shape>，而不必遭受切片（slicing）。

```
template<class T> class Vector_ref {
    vector<T*> v;
    vector<T*> owned;
public:
    Vector_ref() {}
    Vector_ref(T* a, T* b = 0, T* c = 0, T* d = 0);
    ~Vector_ref() { for (int i=0; i<owned.size(); ++i) delete owned[i]; }
    void push_back(T& s) { v.push_back(&s); }
    void push_back(T* p) { v.push_back(p); owned.push_back(p); }
    T& operator[](int i) { return *v[i]; }
    const T& operator[](int i) const { return *v[i]; }
    int size() const { return v.size(); }
};
```

Vector_ref 的析构函数会 delete 每一个以指针形式传给它的对象。

E.5　一个例子：操纵 Widget

下面是一个完整的程序。它用到了 Widget/Window 的许多特性。它的注释只有少得可怜的几条。不幸的是，这种注释不足的情况并不罕见。让这个程序跑起来并解释它，是一道习题。

基本说来，运行它时，它看起来定义了四个按钮：

```
#include "../GUI.h"
using namespace Graph_lib;

class W7 : public Window {
    // 让按钮看起来在移动的四种办法:
    // 显示/隐藏、改变位置、新建一个、以及附加/分离
public:
    W7(int w, int h, const string& t);
    Button* p1;              // 显示/隐藏
    Button* p2;
    bool sh_left;

    Button* mvp;            // 移动
    bool mv_left;
    Button* cdp;             // 创建/销毁
    bool cd_left;
    Button* adp1;            // 激活/停用
    Button* adp2;
    bool ad_left;
    void sh();               // 动作
    void mv();
    void cd();
    void ad();
    static void cb_sh(Address, Address addr)    // 回调
                { reference_to<W7>(addr).sh(); }
    static void cb_mv(Address, Address addr)
                { reference_to<W7>(addr).mv(); }
    static void cb_cd(Address, Address addr)
                { reference_to<W7>(addr).cd(); }
    static void cb_ad(Address, Address addr)
                { reference_to<W7>(addr).ad(); }
};
```

不过，一个 W7（第 7 号窗口实验）其实有六个按钮；它只是把其中两个隐藏起来：

```
W7::W7(int w, int h, const string& t)
    :Window{w,h,t},
    sh_left{true}, mv_left{true}, cd_left{true}, ad_left{true}
{
    p1 = new Button{Point{100,100},50,20,"show",cb_sh};
    p2 = new Button{Point{200,100},50,20, "hide",cb_sh};
    mvp = new Button{Point{100,200},50,20,"move",cb_mv};
    cdp = new Button{Point{100,300},50,20,"create",cb_cd};
    adp1 = new Button{Point{100,400},50,20,"activate",cb_ad};
    adp2 = new Button{Point{200,400},80,20,"deactivate",cb_ad};
    attach(*p1);
    attach(*p2);
    attach(*mvp);
    attach(*cdp);
    p2->hide();
    attach(*adp1);
}
```

共有四个回调。每一个都让你觉得：按下的按钮消失了，又出现一个新按钮。不过，这是用四种不同的方式实现的：

```
void W7::sh()                    // 隐藏一个按钮，显示另一个
{
    if (sh_left) {
        p1->hide();
        p2->show();
    }
    else {
        p1->show();
        p2->hide();
    }
    sh_left = !sh_left;
}

void W7::mv()
    // 移动按钮
{
    if (mv_left) {
        mvp->move(100,0);
    }
    else {
        mvp->move(-100,0);
    }
    mv_left = !mv_left;
}
```

```
void W7::cd()
    // 删除按钮并新建一个
{
    cdp->hide();
    delete cdp;
    string lab = "create";
    int x = 100;
    if (cd_left) {
        lab = "delete";
        x = 200;
    }
    cdp = new Button{Point{x,300}, 50, 20, lab, cb_cd};
    attach(*cdp);
    cd_left = !cd_left;
}

void W7::ad()
    // 把按钮从窗口分离，并附加它的替换者
{
    if (ad_left) {
        detach(*adp1);
        attach(*adp2);
    }
    else {
        detach(*adp2);
        attach(*adp1);
    }
    ad_left = !ad_left;
}

int main()
{
    W7 w{400,500,"move"};
    return gui_main();
}
```

这个程序演示了向窗口添加部件和从窗口移除部件的几种基本方式——或者说，只是看起来如此。

术语表

“Often, a few well-chosen words are worth a thousand pictures.”
（“常常，几个精心挑选的词，胜过一千幅图。”）
——佚名

术语表是对文中用词所作的简要解释。下面这份表相当简短，只收录我们认为最要紧的那些术语，尤其是在学习编程的早期阶段。各章的索引与“术语”一节也许同样有帮助。与 C++ 专门相关、内容更为详尽的术语表可以在 www.stroustrup.com/glossary.html 找到；网上还有多得难以置信的各种专门术语表（质量参差不齐）。请注意，一个术语可能有若干彼此相关的含义（因此我们偶尔会列出几条），而我们列出的大多数术语在其他语境中也都有（往往很弱的）相关含义；例如，我们并不去定义“抽象”在现代绘画、法律实务或哲学中的含义。

抽象类（abstract class） 不能直接用来创建对象的类；通常用来为派生类定义接口。若一个类含有纯虚函数或受保护的构造函数，它就成为抽象类。

抽象（abstraction） 对某事物的描述，有选择地、刻意地忽略（隐藏）细节（例如实现细节）；即有选择的忽略。

地址（address） 一个值，借助它可以在计算机的内存中找到某个对象。

算法（algorithm） 解决问题的过程或公式；为产生结果而进行的一串有限的计算步骤。

别名（alias） 指称同一个对象的另一种方式；通常是一个名字、指针或引用。

应用程序（application） 被其用户视为一个整体的程序或一组程序。

近似（approximation） 接近完美或理想（值或设计）的东西（值或设计）。近似常常是各种理想之间权衡的结果。

实参（argument） 传给函数或模板的值，在函数或模板中通过形参来访问它。

数组（array） 元素的同质序列，通常带编号，例如 [0:max)。

断言（assertion） 插入程序中的语句，用来声明（断言）在程序的这个位置上某件事必须始终为真。

基类（base class） 用作类层次结构之基础的类。基类通常有一个或多个虚函数。

位（bit） 计算机中最基本的信息单位。一个位的值可以是 0 或 1。

缺陷（bug） 程序中的错误。

字节（byte） 大多数计算机中基本的寻址单位。一个字节通常保存 8 位。

类（class） 一种用户自定义类型，可以包含数据成员、函数成员和成员类型。

代码（code） 一个程序或程序的一部分；这个词含义不明确，源代码和目标代码都用它。

编译器（compiler） 把源代码变成目标代码的程序。

复杂度（complexity） 一个难以精确定义的概念或度量，表示构造问题解法之难，或解法本身的难。有时复杂度只是简单地指执行一个算法所需操作次数的估计。

计算（computation） 执行某些代码的过程，通常取一些输入并产生一些输出。

概念（concept） (1) 观念、想法；(2) 一组需求，通常是对模板实参提出的。

具体类（concrete class） 可以为其创建对象的类。

常量（constant） （在给定作用域内）不能被改变的值；不可变的。

构造函数（constructor） 初始化（“构造”）对象的操作。构造函数通常建立一个不变式，并且常常获取对象使用所需的资源（这些资源随后通常由析构函数释放）。

容器（container） 存放元素（其他对象）的对象。

复制（copy） 使两个对象的值比较起来相等的操作。另见 move（移动）。

正确性（correctness） 一个程序或程序的一部分若能满足其规约，就是正确的。不幸的是，规约可能不完整、不一致，也可能不符合用户合理的期望。因此，要写出可接受的代码，我们有时不得不比单纯遵循形式规约做得更多。

代价（cost） 编写或执行一个程序的开销（例如程序员时间、运行时间或空间）。理想情况下，代价应当是复杂度的函数。

数据（data） 计算中使用的值。

调试（debugging） 查找并清除程序中错误的行为；通常远不如测试那样系统。

声明（declaration） 在程序中指定一个名字及其类型。

定义（definition） 一种声明，它提供使用该实体完成程序所需的全部信息。简化的定义：分配内存的声明。

派生类（derived class） 由一个或多个基类派生出来的类。

设计（design） 关于一块软件应当如何运作以满足其规约的总体描述。

析构函数（destructor） 对象被销毁（例如在作用域结束时）时被隐式调用（invoke）的操作。它通常会释放资源。

封装（encapsulation） 保护本应私有的东西（例如实现细节）不被未授权访问。

错误（error） 对程序行为的合理期望（常表述为需求或用户指南）与程序实际行为之间的不一致。

可执行文件（executable） 已准备好可在计算机上运行（执行）的程序。

功能蔓延（feature creep） 倾向于“以防万一”地给程序添加多余的功能。

文件（file） 计算机中保存永久性信息的容器。

浮点数（floating-point number） 计算机对实数的近似表示，例如 7.93 和 10.78×10^{-3}。

函数（function） 一段有名字的代码单元，可以从程序的不同地方调用（call）；一个逻辑的计算单元。

泛型编程（generic programming） 一种编程风格，关注算法的设计与高效实现。一个泛型算法适用于所有满足其需求的实参类型。在 C++ 中，泛型编程通常使用模板。

句柄（handle） 一种类，它通过成员指针或引用访问另一个对象。另见 copy、move、resource。

头文件（header） 包含声明的文件，用来在程序的各部分之间共享接口。

隐藏（hiding） 阻止某条信息被直接看到或访问的行为。例如，来自嵌套（内层）作用域的名字，可以阻止外层（外围）作用域中的同名被直接使用。

理想（ideal） 我们所追求的事物的完美版本。通常我们不得不做出权衡，接受一个近似。

实现（implementation） (1) 编写并测试代码的行为；(2) 实现一个程序所用的代码。

无限循环（infinite loop） 终止条件永远不会变为真的循环。另见 iteration（迭代）。

无限递归（infinite recursion） 直到机器用完保存调用所需的内存才结束的递归。现实中这种递归从不是无限的，而是被某种硬件错误终止。

信息隐藏（information hiding） 把接口与实现分离开的行为，从而隐藏不打算让用户注意的实现细节，并提供一个抽象。

初始化（initialize） 给对象赋予它的第一个（初始）值。

输入（input） 计算所使用的值（例如函数实参和键盘上键入的字符）。

整数（integer） 一个完整的数，例如 42 和 −99。

接口（interface） 一个声明或一组声明，规定一段代码（例如一个函数或一个类）可以怎样被调用。

不变式（invariant） 在程序中某个（或某些）位置上必须始终为真的事物；通常用来描述对象的状态（值的集合），或循环进入重复语句之前的状态。

迭代（iteration） 反复执行一段代码的行为；另见 recursion（递归）。

迭代器（iterator） 标识序列中某个元素的对象。

库（library） 类型、函数、类等的集合，实现一组设施（抽象），目的是供一个以上的程序使用。

生存期（lifetime） 从对象初始化到它变得不可用（离开作用域、被删除，或程序终止）之间的时间。

链接器（linker） 把目标代码文件和库组合成一个可执行程序的程序。

字面量（literal） 直接指定一个值的记法，例如用 12 直接指定整数值“十二”。

循环（loop） 被反复执行的一段代码；在 C++ 中通常是 for 语句或 while 语句。

移动（move） 把一个值从一个对象转移到另一个对象的操作，原对象留下一个表示“空”的值。另见 copy（复制）。

可变的（mutable） 可改变的；与 immutable（不可变的）、constant（常量）相对。

对象（object） (1) 一块已知类型、保存该类型值的已初始化内存区域；(2) 一块内存区域。

目标代码（object code） 编译器的输出，用作链接器的输入（由链接器产生可执行代码）。

目标文件（object file） 包含目标代码的文件。

面向对象编程（object-oriented programming） 一种编程风格，关注类的设计与使用以及类层次结构。

操作（operation） 能够执行某种动作的东西，例如函数和运算符。

输出（output） 计算产生的值（例如函数的结果，或写在屏幕上的若干行字符）。

溢出（overflow） 产生了一个无法存储在预期目标中的值。

重载（overload） 定义两个名字相同但实参（操作数）类型不同的函数或运算符。

覆盖（override） 在派生类中定义一个与基类虚函数同名且实参类型相同的函数，从而使该函数可以通过基类所定义的接口来调用。

所有者（owner） 负责释放某个资源的对象。

范式（paradigm） 对设计或编程风格的一种略显做作的说法；使用时往往带着（错误的）暗示：存在某种优于其他一切范式的范式。

形参（parameter） 对函数或模板的一个显式输入的声明。函数被调用时，可以通过其形参的名字访问传入的实参。

指针（pointer） (1) 用来标识内存中某个带类型对象的值；(2) 保存这样一个值的变量。

后置条件（post-condition） 在离开一段代码（例如一个函数或一个循环）时必须成立的条件。

前置条件（pre-condition） 在进入一段代码（例如一个函数或一个循环）时必须成立的条件。

程序（program） （可能带有相关数据的）代码，其完整程度足以由计算机执行。

编程（programming） 把问题的解法表达为代码的技艺。

编程语言（programming language） 用来表达程序的语言。

伪代码（pseudo code） 用非正式记法而非编程语言写成的计算描述。

纯虚函数（pure virtual function） 必须在派生类中被覆盖的虚函数。

RAII（“Resource Acquisition Is Initialization”，资源获取即初始化） 一种基于作用域管理资源的基本技术。

范围（range） 可以用起点和终点描述的一串值。例如 [0:5) 表示值 0、1、2、3 和 4。

递归（recursion） 函数调用自身的行为；另见 iteration（迭代）。

引用（reference） (1) 描述内存中某个带类型值的位置的值；(2) 保存这样一个值的变量。

正则表达式（regular expression） 用于描述字符串中模式的记法。

需求（requirement） (1) 对程序或程序某一部分所期望行为的描述；(2) 对函数或模板就其参数所作假设的描述。

资源（resource） 被获取之后必须在稍后释放的东西，例如文件句柄、锁或内存。另见 handle、owner。

舍入（rounding） 把一个值转换为精度较低的类型中在数学上最接近的值。

作用域（scope） 程序文本（源代码）中可以引用某个名字的区域。

序列（sequence） 可以按线性顺序访问的元素。

软件（software） 代码片段及相关数据的集合；常与 program（程序）互换使用。

源代码（source code） 由程序员编写、（原则上）可被其他程序员阅读的代码。

源文件（source file） 包含源代码的文件。

规约（specification） 对一段代码应当做什么的描述。

标准（standard） 对某事物（例如一种编程语言）的官方一致定义。

状态（state） 一组值。

字符串（string） 字符的序列。

风格（style） 一套编程技术，使语言特性的使用保持一致；有时在非常狭窄的意义上，只指关于命名和代码外观的低层规则。

子类型（subtype） 派生类型；具有某个类型的全部性质、并且可能还有更多性质的类型。

超类型（supertype） 基类型；其性质是某个类型的性质的一个子集的类型。

系统（system） (1) 在计算机上执行某项任务的程序或一组程序；(2)“操作系统”的简称，即计算机的基本执行环境与工具。

模板（template） 由一个或多个类型或（编译期）值参数化的类或函数；支持泛型编程的基本 C++ 语言构造。

测试（testing） 系统地查找程序中错误的过程。

权衡（trade-off） 平衡若干设计与实现标准的结果。

截断（truncation） 把一种类型转换为另一种无法精确表示待转换值的类型时发生的信息丢失。

类型（type） 定义对象的一组可能值以及一组操作的东西。

未初始化（uninitialized） 对象在被初始化之前所处的（未定义的）状态。

单位（unit） (1) 使一个值具有意义的标准度量（例如距离用 km）；(2) 一个更大整体中具有特殊地位的（例如有名字的）部分。

用例（use case） 对程序的一种具体（通常简单）的使用，用来检验其功能并展示其用途。

值（value） 内存中的一组位，按某个类型来解释。

变量（variable） 给定类型的一个具名对象；除非未初始化，否则它含有一个值。

虚函数（virtual function） 可以在派生类中被覆盖的成员函数。

字（word） 计算机中基本的存储单位，通常用来存放一个整数。

参考文献

本参考文献列出本书正文引用与推荐的著作、标准与网络资源。为便于按作者与书名检索，条目一律保留原文著录格式，按作者姓氏的英文字母顺序排列；ISBN 与网址照原书给出，网址的时效性请以实际访问为准。

Aho, Alfred V., Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*, Second Edition (usually called "The Dragon Book"). Addison-Wesley, 2006. ISBN 0321486811.

Andrews, Mike, and James A. Whittaker. *How to Break Software: Functional and Security Testing of Web Applications and Web Services*. Addison-Wesley, 2006. ISBN 0321369440.

Bergin, Thomas J., and Richard G. Gibson, eds. *History of Programming Languages, Volume 2*. Addison-Wesley, 1996. ISBN 0201895021.

Blanchette, Jasmin, and Mark Summerfield. *C++ GUI Programming with Qt 4*. Prentice Hall, 2006. ISBN 0131872494.

Boost.org. "A Repository for Libraries Meant to Work Well with the C++ Standard Library." `www.boost.org`.

Cox, Russ. "Regular Expression Matching Can Be Simple and Fast (but Is Slow in Java, Perl, PHP, Python, Ruby, . . .)." `http://swtch.com/~rsc/regexp/regexp1.html`.

dmoz.org. `http://dmoz.org/Computers/Programming/Languages`.

Freeman, T. L., and C. Phillips. *Parallel Numerical Algorithms*. Prentice Hall, 1992. ISBN 0136515975.

Gamma, Erich, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994. ISBN 0201633612.

Goldthwaite, Lois, ed. *Technical Report on C++ Performance*. ISO/IEC PDTR 18015. `www.stroustrup.com/performanceTR.pdf`.

Gullberg, Jan. *Mathematics — From the Birth of Numbers*. W. W. Norton, 1996. ISBN 039304002X.

Hailpern, Brent, and Barbara G. Ryder, eds. *Proceedings of the Third ACM SIGPLAN Conference on the History of Programming Languages (HOPL-III)*. San Diego, CA, 2007. `http://portal.acm.org/toc.cfm?id=1238844`.

ISO/IEC 9899:2011. *Programming Languages — C*. The C standard.

ISO/IEC 14882:2011. *Programming Languages — C++*. The C++ standard.

Kernighan, Brian W., and Dennis M. Ritchie. *The C Programming Language*, Second Edition. Prentice Hall, 1988. ISBN 0131103628.

Knuth, Donald E. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*, Third Edition. Addison-Wesley, 1997. ISBN 0201896842.

Koenig, Andrew, and Barbara E. Moo. *Accelerated C++: Practical Programming by Example*. Addison-Wesley, 2000. ISBN 020170353X.

Langer, Angelika, and Klaus Kreft. *Standard C++ IOStreams and Locales: Advanced Programmer's Guide and Reference*. Addison-Wesley, 2000. ISBN 0321585585.

Lippman, Stanley B., José Lajoie, and Barbara E. Moo. *The C++ Primer*, Fifth Edition. Addison-Wesley, 2005. ISBN 0321714113. (Use only the 5th edition.)

Lockheed Martin Corporation. "Joint Strike Fighter Air Vehicle Coding Standards for the System Development and Demonstration Program." Document Number 2RDU00001 Rev C. December 2005. Colloquially known as "JSF++." `www.stroustrup.com/JSF-AV-rules.pdf`.

Lohr, Steve. *Go To: The Story of the Math Majors, Bridge Players, Engineers, Chess Wizards, Maverick Scientists and Iconoclasts — The Programmers Who Created the Software Revolution*. Basic Books, 2002. ISBN 978-0465042265.

Meyers, Scott. *Effective STL: 50 Specific Ways to Improve Your Use of the Standard Template Library*. Addison-Wesley, 2001. ISBN 0201749629.

Meyers, Scott. *Effective C++: 55 Specific Ways to Improve Your Programs and Designs*, Third Edition. Addison-Wesley, 2005. ISBN 0321334876.

Programming Research. *High-Integrity C++ Coding Standard Manual Version 2.4*. `www.programmingresearch.com`.

Richards, Martin. *BCPL — The Language and Its Compiler*. Cambridge University Press, 1980. ISBN 0521219655.

Ritchie, Dennis. "The Development of the C Programming Language." *Proceedings of the ACM History of Programming Languages Conference (HOPL-2)*. ACM SIGPLAN Notices, Vol. 28 No. 3, 1993.

Salus, Peter H. *A Quarter Century of UNIX*. Addison-Wesley, 1994. ISBN 0201547775.

Sammet, Jean E. *Programming Languages: History and Fundamentals*. Prentice Hall, 1969. ISBN 0137299885.

Schmidt, Douglas C., and Stephen D. Huston. *C++ Network Programming, Volume 1: Mastering Complexity with ACE and Patterns*. Addison-Wesley, 2002. ISBN 0201604647.

Schmidt, Douglas C., and Stephen D. Huston. *C++ Network Programming, Volume 2: Systematic Reuse with ACE and Frameworks*. Addison-Wesley, 2003. ISBN 0201795256.

Schwartz, Randal L., Tom Phoenix, and Brian D. Foy. *Learning Perl*, Fourth Edition. O'Reilly, 2005. ISBN 0596101058.

Scott, Michael L. *Programming Language Pragmatics*. Morgan Kaufmann, 2000. ISBN 1558604421.

Sebesta, Robert W. *Concepts of Programming Languages*, Sixth Edition. Addison-Wesley, 2003. ISBN 0321193628.

Shepherd, Simon. "The Tiny Encryption Algorithm (TEA)." `www.tayloredge.com/reference/Mathematics/TEA-XTEA.pdf` and `http://143.53.36.235:8080/tea.htm`.

Stepanov, Alexander. `www.stepanovpapers.com`.

Stewart, G. W. *Matrix Algorithms, Volume I: Basic Decompositions*. SIAM, 1998. ISBN 0898714141.

Stone, Debbie, Caroline Jarrett, Mark Woodroffe, and Shailey Minocha. *User Interface Design and Evaluation*. Morgan Kaufmann, 2005. ISBN 0120884364.

Stroustrup, Bjarne. "A History of C++: 1979–1991." *Proceedings of the ACM History of Programming Languages Conference (HOPL-2)*. ACM SIGPLAN Notices, Vol. 28 No. 3, 1993.

Stroustrup, Bjarne. *The Design and Evolution of C++*. Addison-Wesley, 1994. ISBN 0201543303.

Stroustrup, Bjarne. "Learning Standard C++ as a New Language." *C/C++ Users Journal*, May 1999.

Stroustrup, Bjarne. "C and C++: Siblings"; "C and C++: A Case for Compatibility"; and "C and C++: Case Studies in Compatibility." *The C/C++ Users Journal*, July, Aug., and Sept. 2002.

Stroustrup, Bjarne. “Evolving a Language in and for the Real World: C++ 1991–2006.” *Proceedings of the Third ACM SIGPLAN Conference on the History of Programming Languages (HOPL-III)*. San Diego, CA, 2007. `http://portal.acm.org/toc.cfm?id=1238844`.

Stroustrup, Bjarne. *The C++ Programming Language*, Fourth Edition. Addison-Wesley, 2013. ISBN 0321563840.

Stroustrup, Bjarne. *A Tour of C++*. Addison-Wesley, 2013. ISBN 978-0321958310.

Stroustrup, Bjarne. Author’s home page, `www.stroustrup.com`.

Sutter, Herb. *Exceptional C++: 47 Engineering Puzzles, Programming Problems, and Solutions*. Addison-Wesley, 2000. ISBN 0201615622.

Sutter, Herb, and Andrei Alexandrescu. *C++ Coding Standards: 101 Rules, Guidelines, and Best Practices*. Addison-Wesley, 2004. ISBN 0321113586.

University of St. Andrews. *The MacTutor History of Mathematics archive*. `http://www-gap.dcs.st-and.ac.uk/~history`.

Wexelblat, Richard L., ed. *History of Programming Languages*. Academic Press, 1981. ISBN 0127450408.

Whittaker, James A. *How to Break Software: A Practical Guide to Testing*. Addison-Wesley, 2002. ISBN 0201796198.

Wood, Alastair. *Introduction to Numerical Analysis*. Addison-Wesley, 2000. ISBN 020134291X.